树与二叉树

树与二叉树

树与二叉树的定义

二叉树的实现与遍历

树与森林

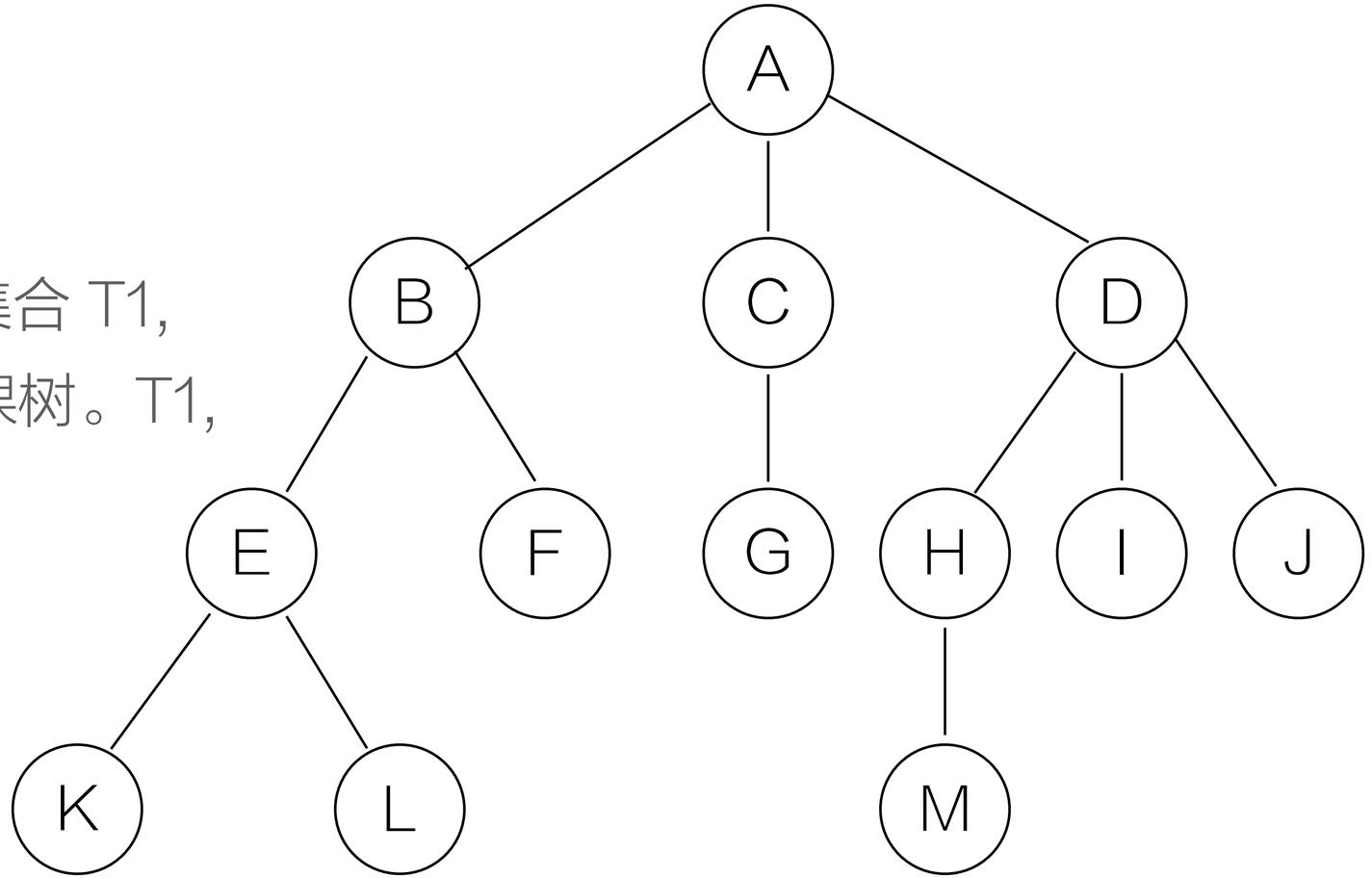
哈夫曼树

树

树是一个或多个结点的有限集合

存在一个称为根的　特定结点

其余的结点被分为 n 个互不相交的集合 T1, T2, …, Tn, 其中的每个集合都是一棵树。T1, T2, …, Tn称为根节点的子树。



树

结点：树中的一个独立单元。

结点的度：结点拥有的子树数称为结点的度。

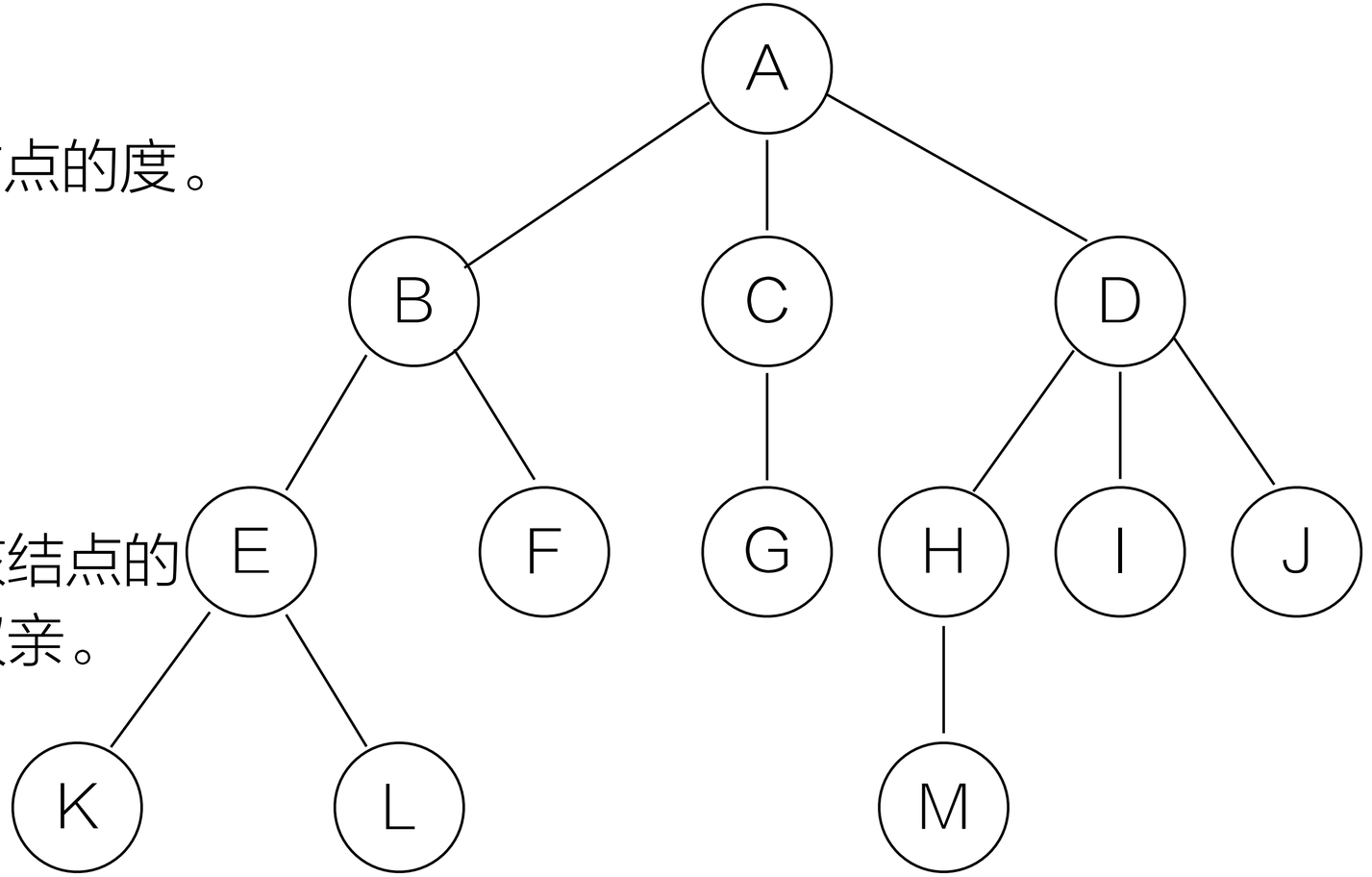
树的度：树内各结点度的最大值。

叶子：度为 0 的结点或终端结点。

非终端结点：度不为 0 的结点。

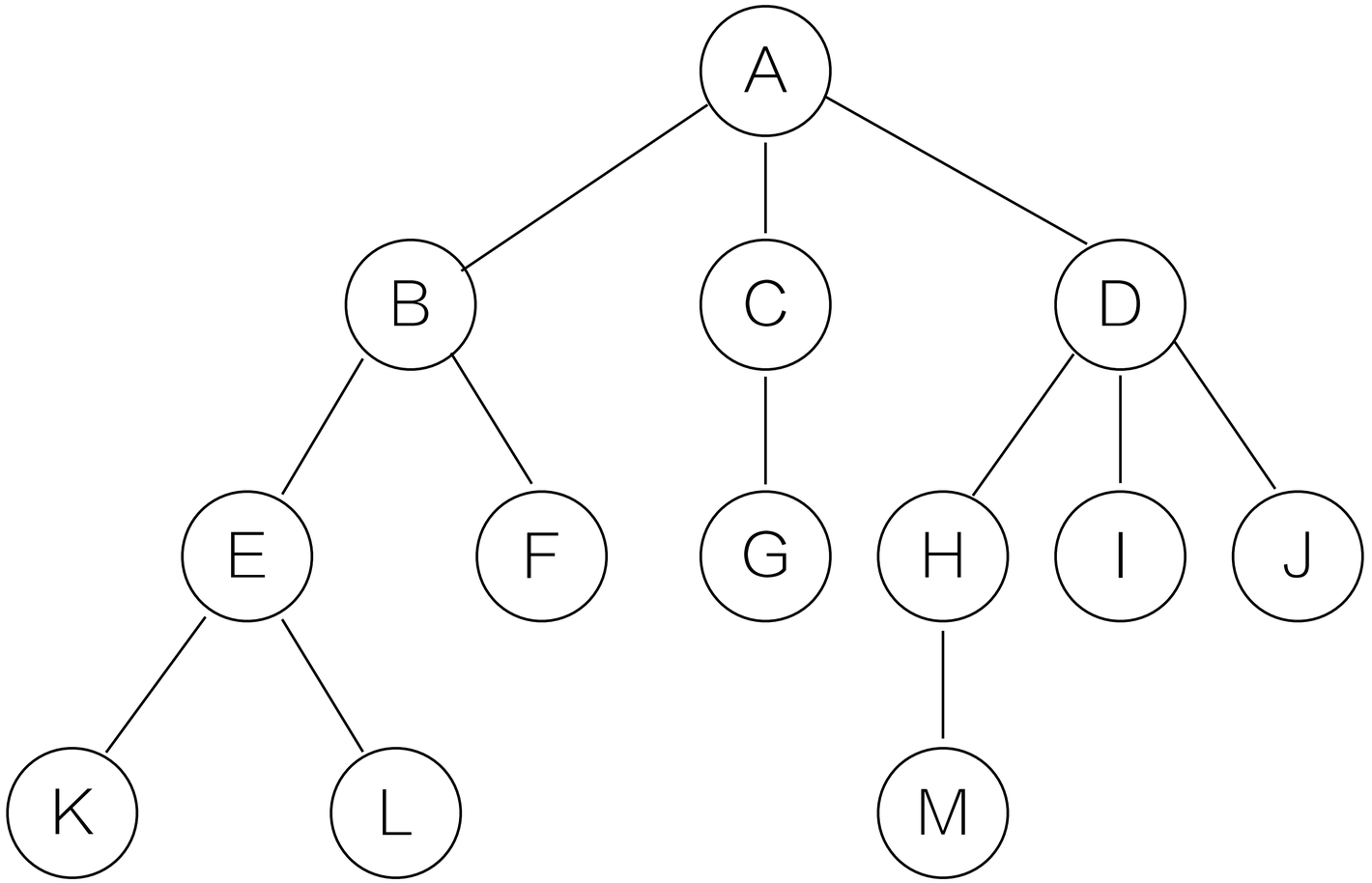
双亲和孩子：结点的子树的根称为该结点的孩子，相应地，该结点称为孩子的双亲。

层次：结点的层次从根开始定义，根为第一层，根的孩子为第二层，以此类推。



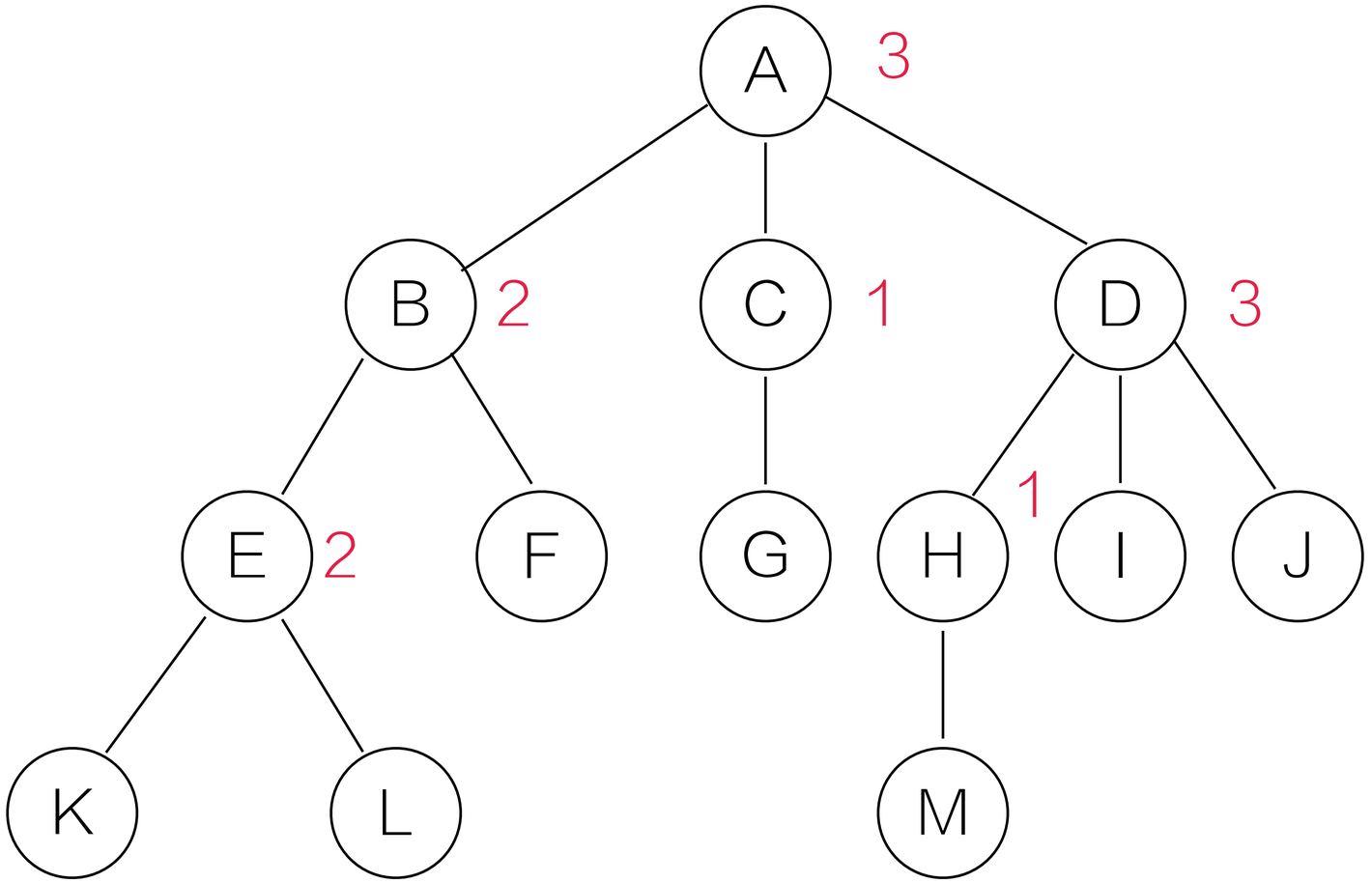
树的基本性质

性质一：树中所有结点数等于所有结点的度数之和加 1。



树的基本性质

性质一：树中所有结点数等于所有结点的度数之和加 1。



树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

当前树的所有结点数为：

树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

当前树的所有结点数为：$20\times4+10\times3+1\times2+10\times1+1=123$

树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

当前树的所有结点数为：$20\times4+10\times3+1\times2+10\times1+1=123$

假设树的总结点数为 n，度数为 0 的结点有 n_0 个，度数为 1 的结点有 n_1 个，
度数为 2 的结点有 n_2 个，度数为 3 的结点有 n_3 个，度数为 4 的结点有 n_4 个。

树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

当前树的所有结点数为：$20\times4+10\times3+1\times2+10\times1+1=123$

假设树的总结点数为 n，度数为 0 的结点有 n_0 个，度数为 1 的结点有 n_1 个，
度数为 2 的结点有 n_2 个，度数为 3 的结点有 n_3 个，度数为 4 的结点有 n_4 个。

$n = n_0 + n_1 + n_2 + n_3 + n_4$

树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

当前树的所有结点数为：$20\times4+10\times3+1\times2+10\times1+1=123$

假设树的总结点数为 n，度数为 0 的结点有 n_0 个，度数为 1 的结点有 n_1 个，
度数为 2 的结点有 n_2 个，度数为 3 的结点有 n_3 个，度数为 4 的结点有 n_4 个。

$n = n_0 + n_1 + n_2 + n_3 + n_4$

$123 = n_0 + 10 + 1 + 10 + 20$

树的基本性质 – 真题解析

[2010] 5. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是_____。

A. 41　　B. 82　　C. 113　　D. 122

当前树的所有结点数为：$20\times4+10\times3+1\times2+10\times1+1=123$

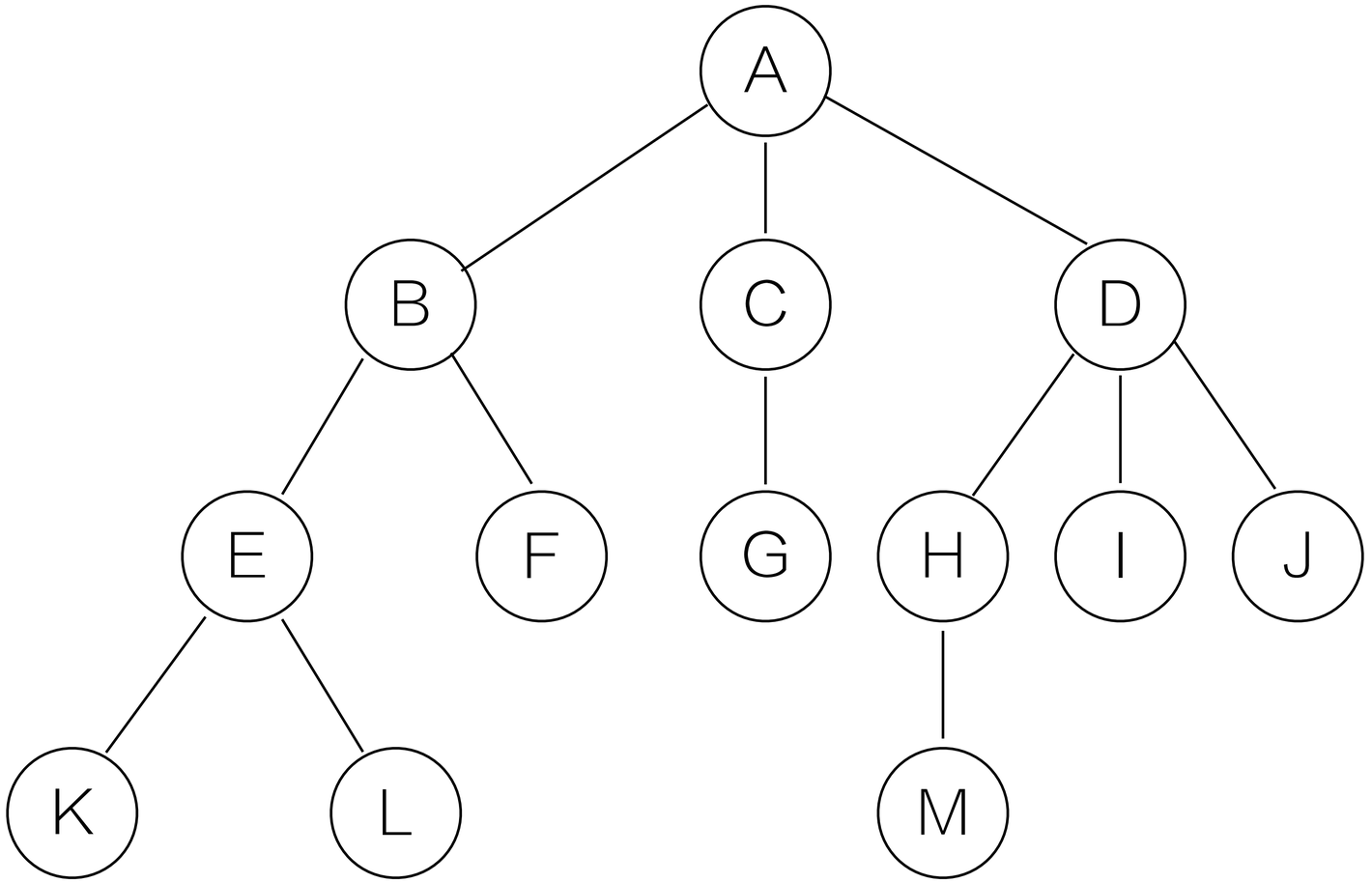
假设树的总结点数为 n，度数为 0 的结点有 n_0 个，度数为 1 的结点有 n_1 个，
度数为 2 的结点有 n_2 个，度数为 3 的结点有 n_3 个，度数为 4 的结点有 n_4 个。

$n = n_0 + n_1 + n_2 + n_3 + n_4$

$123 = n_0 + 10 + 1 + 10 + 20$

树的基本性质

性质二：对于度为 m 的树，第 i 层上最多有 m^{i-1} 个结点。



树的基本性质

性质三：对于高度为 h，度为 m 的树，最多有 $(m^h-1)/(m-1)$ 个结点。

二叉树

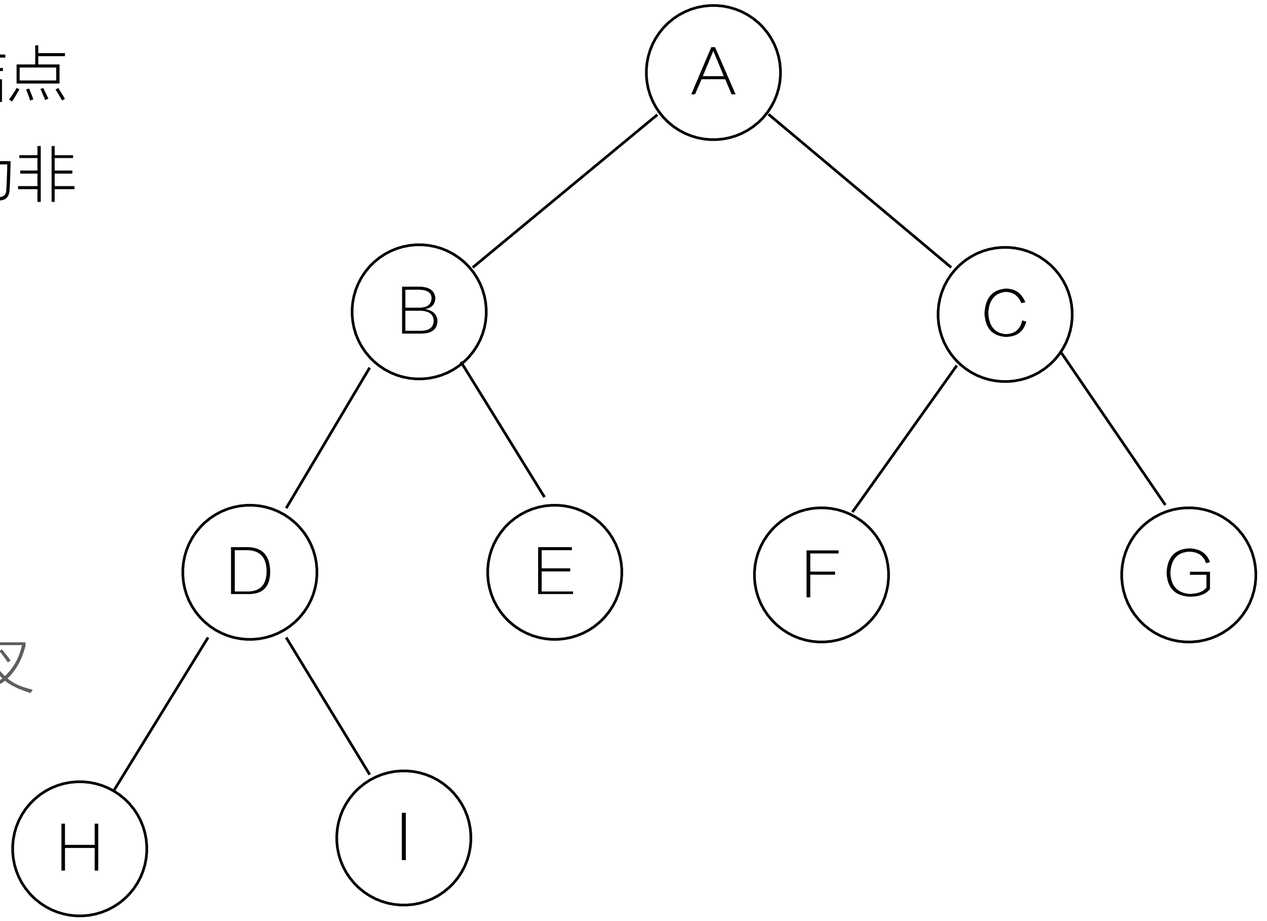
二叉树（Binary Tree）是 n（$n \geqslant 0$）个结点所构成的集合，它或为空树（n=0），或为非空树。对于非空树 T：

（1）有且仅有一个　称为根的结点

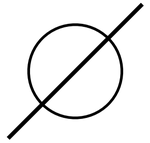
（2）除根结点以外的其余结点分为两个互不相交的子集 T_1 和 T_2，分别称为 T 的左子树和右子树，且 T_1 和 T_2 本身又都是二叉树。

（3）二叉树每个结点至多只有两棵子树。

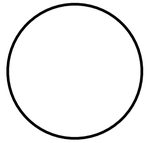
（4）二叉树的子树有左右之分，其次序不能任意颠倒。



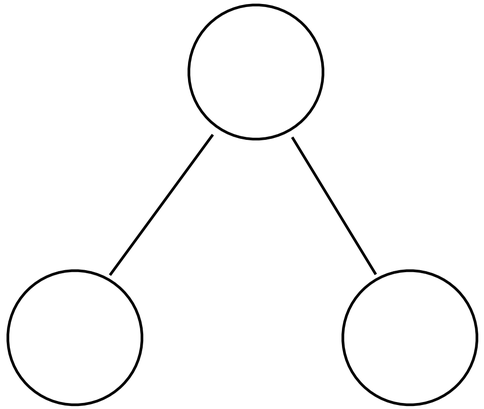
二叉树基本形态



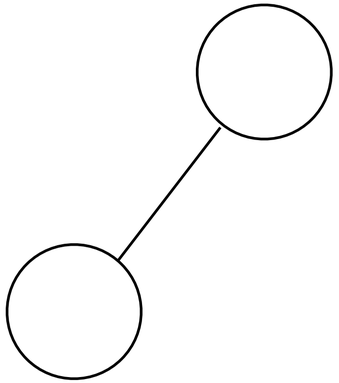
空二叉树



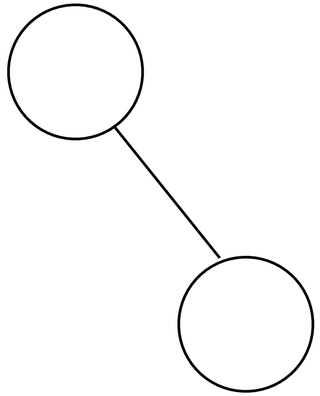
仅有根结点



左右子树均非空



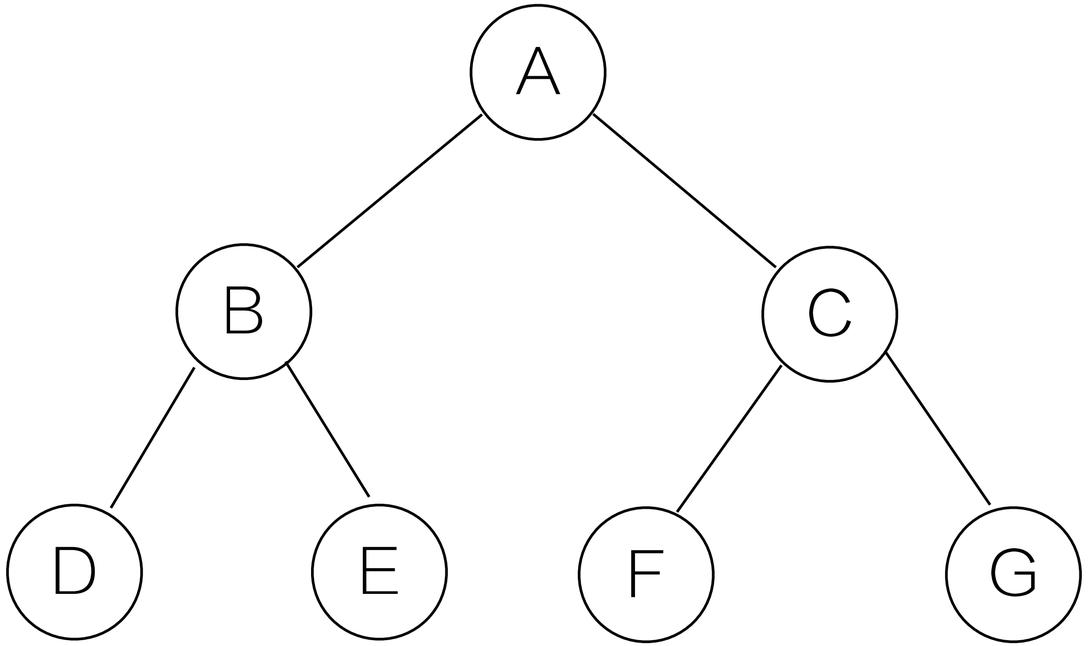
右子树为空



左子树为空

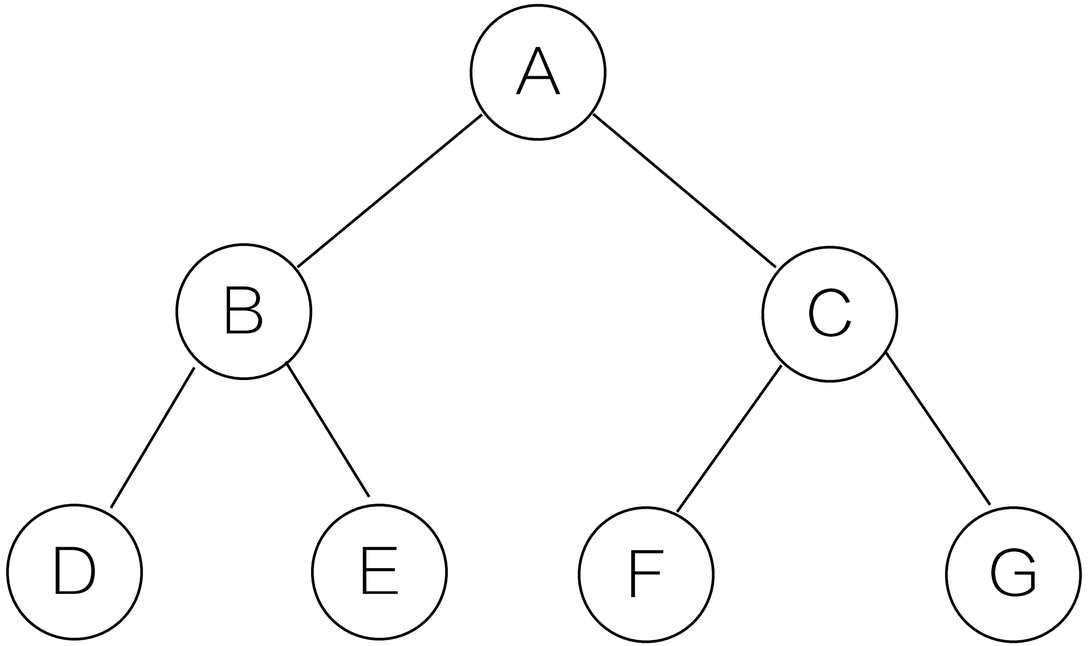
二叉树的性质

性质一：二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。



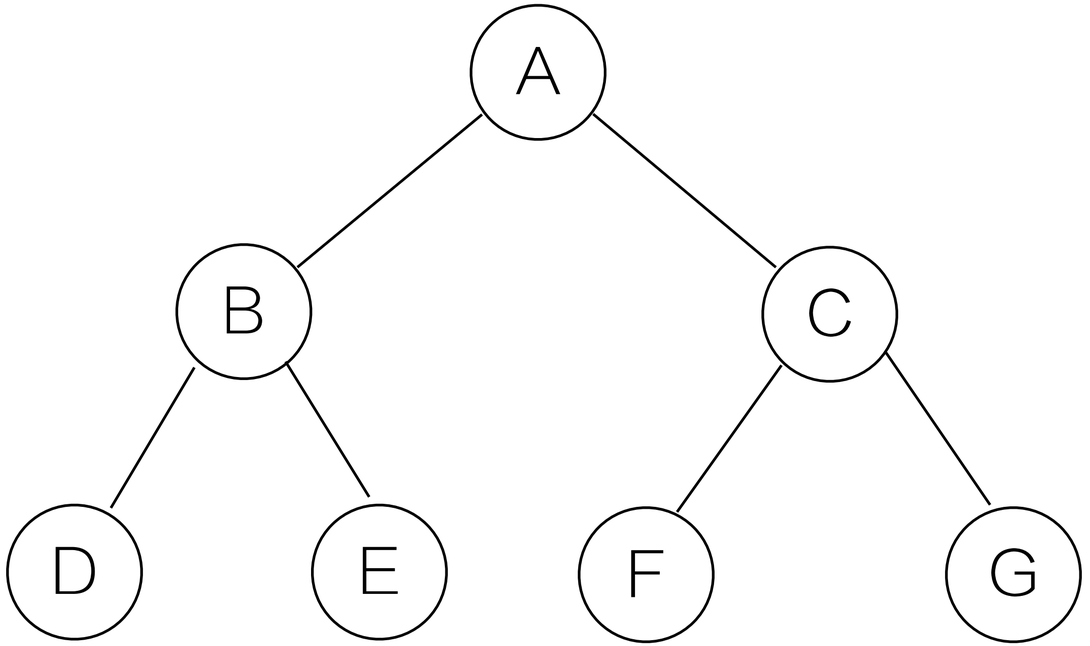
二叉树的性质

性质二：深度为 k 的二叉树最多有2^k-1（$k \geqslant 1$）个结点。



二叉树的性质

性质三：对于任何非空的二叉树T，如果叶子结点的个数为n_0，而度为2的结点数为n_2，则$n_0=n_2+1$



特殊二叉树

满二叉树

完全二叉树

斜树

二叉排序树

二叉搜索树

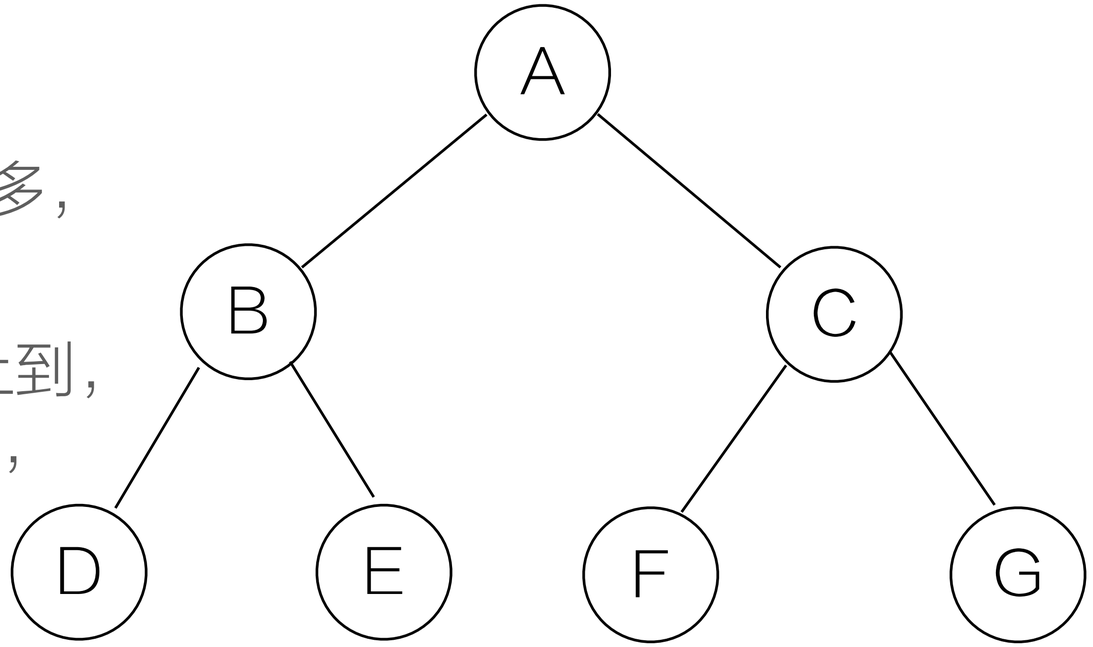
特殊二叉树 – 满二叉树

满二叉树：深度为 k 且含有 2^k-1 个结点的二叉树

所有的叶子结点只能出现在最后一层

对于同样深度的二叉树，满二叉树的结点个数最多，叶子结点的数量也是最多的。

如果对满二叉树进行编号，根结点从1开始，从上到，下从左到右，对于编号为i的结点，若存在左孩子，则左孩子的编号为2i，右孩子为2i+1。



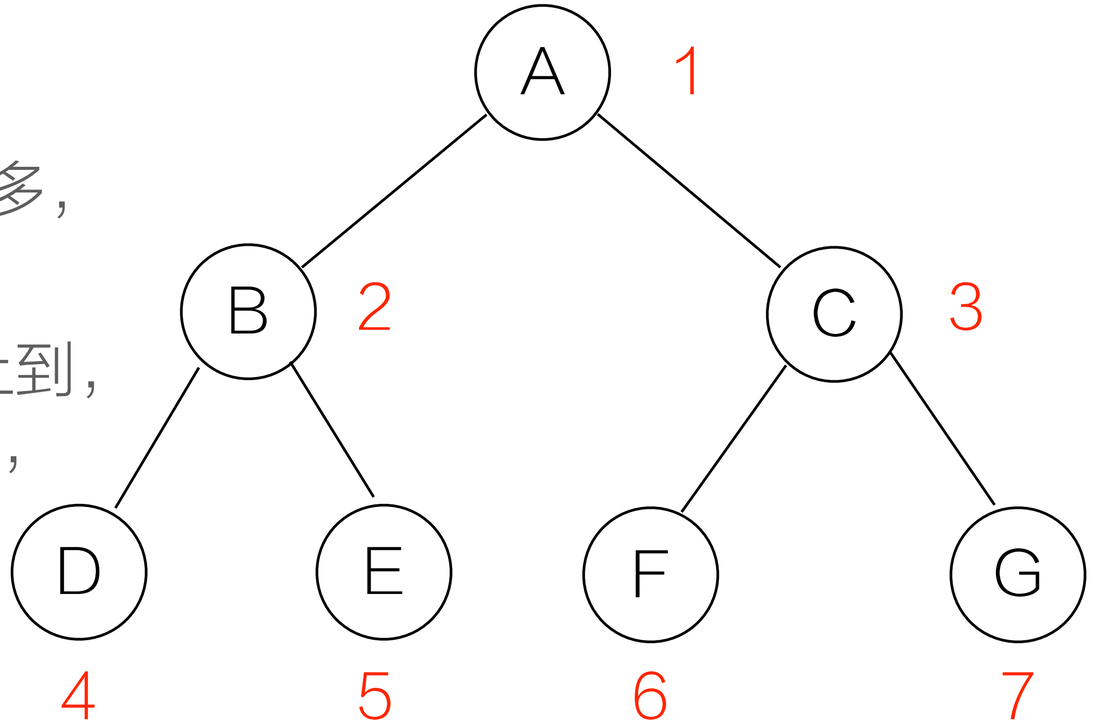
特殊二叉树 – 满二叉树

满二叉树：深度为 k 且含有 2^k-1 个结点的二叉树

所有的叶子结点只能出现在最后一层

对于同样深度的二叉树，满二叉树的结点个数最多，叶子结点的数量也是最多的。

如果对满二叉树进行编号，根结点从1开始，从上到，下从左到右，对于编号为i的结点，若存在左孩子，则左孩子的编号为2i，右孩子为2i+1。



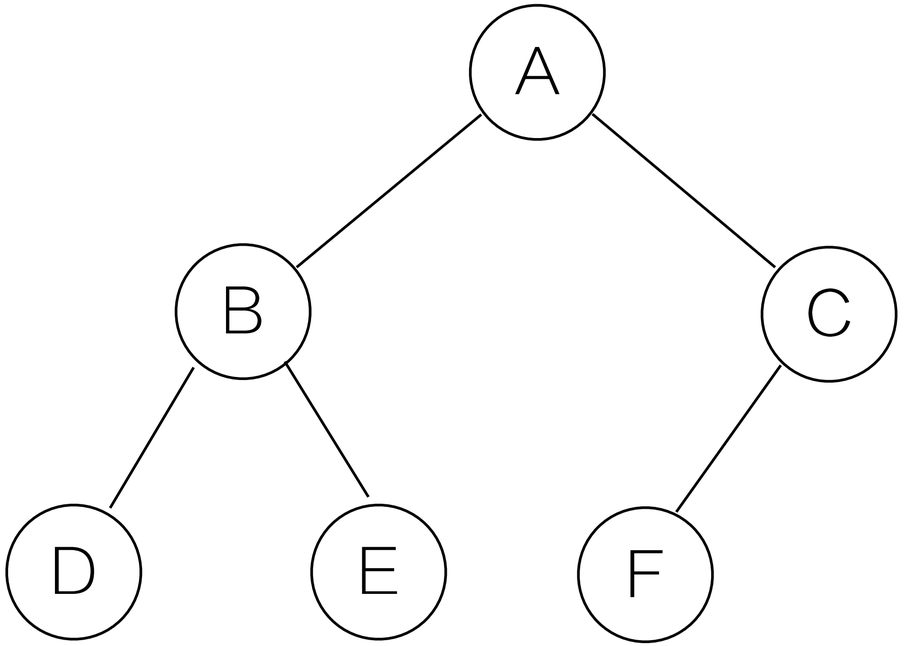
特殊二叉树 – 完全二叉树

完全二叉树：深度为 k 的、有 n 个结点的二叉树，当且仅当其每一个结点都与深度为 k 的满二叉树中编号从 1 至 n 的结点一一对应时，称之为完全二叉树。

完全二叉树的特点是：

（1）叶子结点只可能在层次最大的两层上出现；

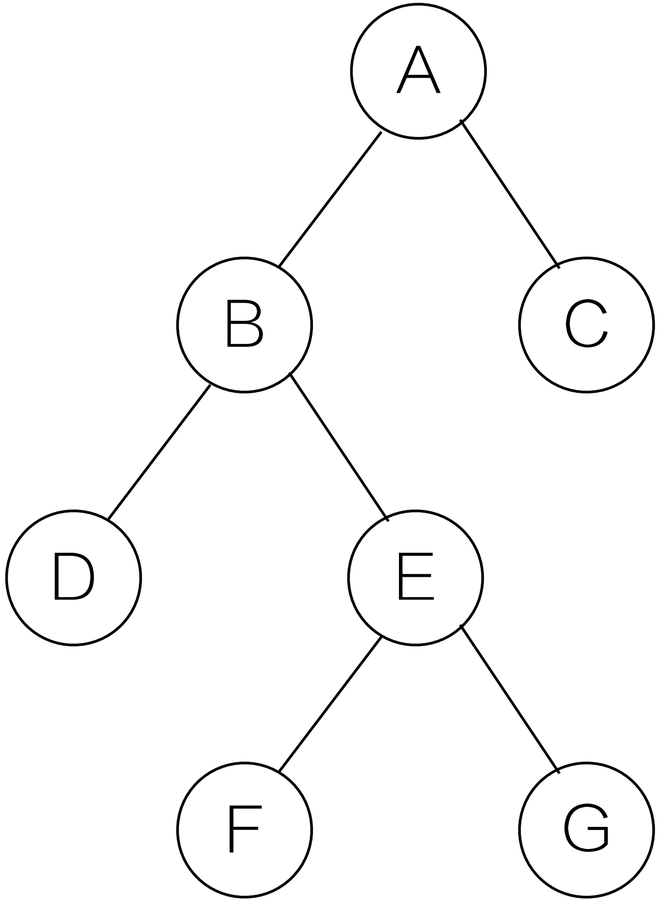
（2）对任一结点，若其右分支下的子孙的最大层次为 l，则其左分支下的子孙的最大层次必为 l 或 l+1。

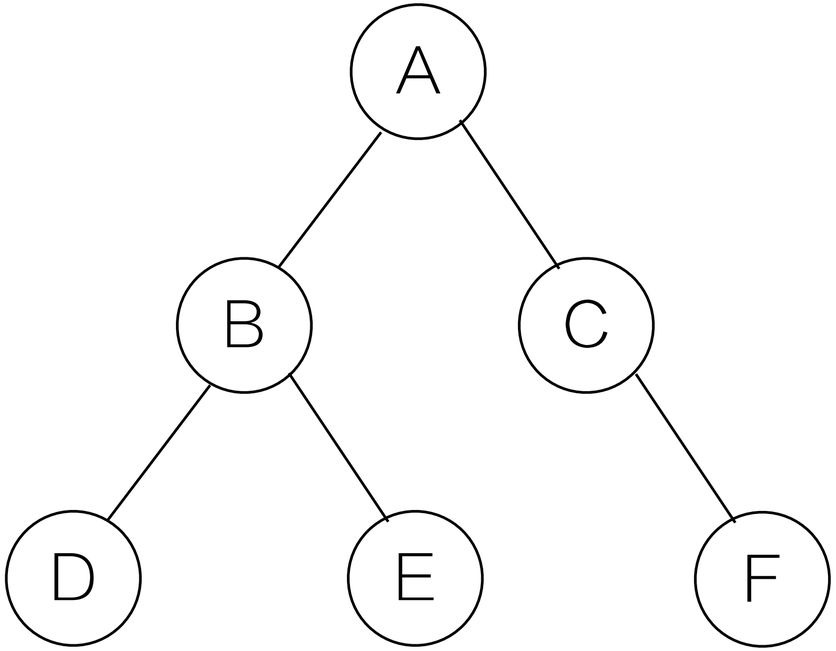


完全二叉树

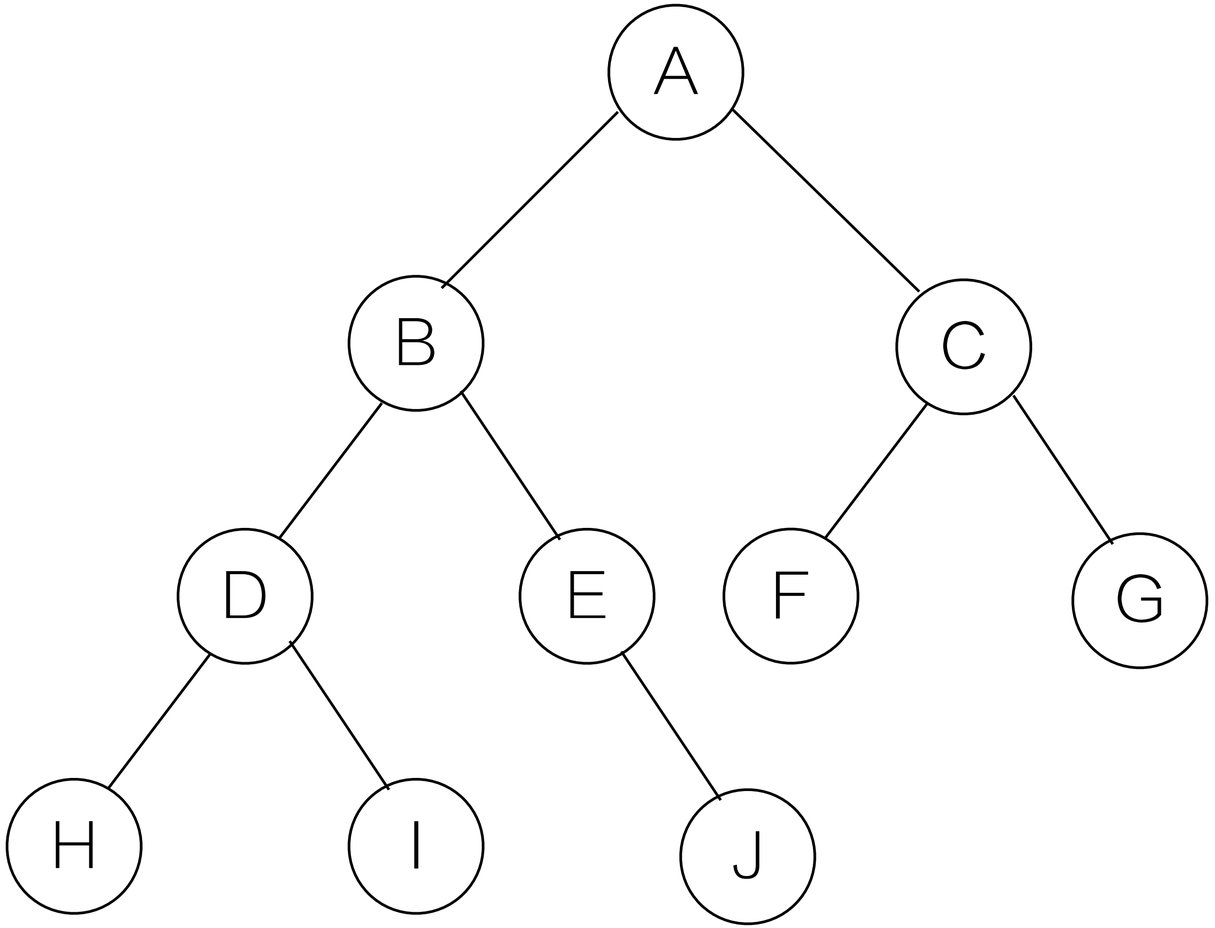
没有左子树，不能有右子树，上一层没有铺满，不能有下一层

判断完全二叉树

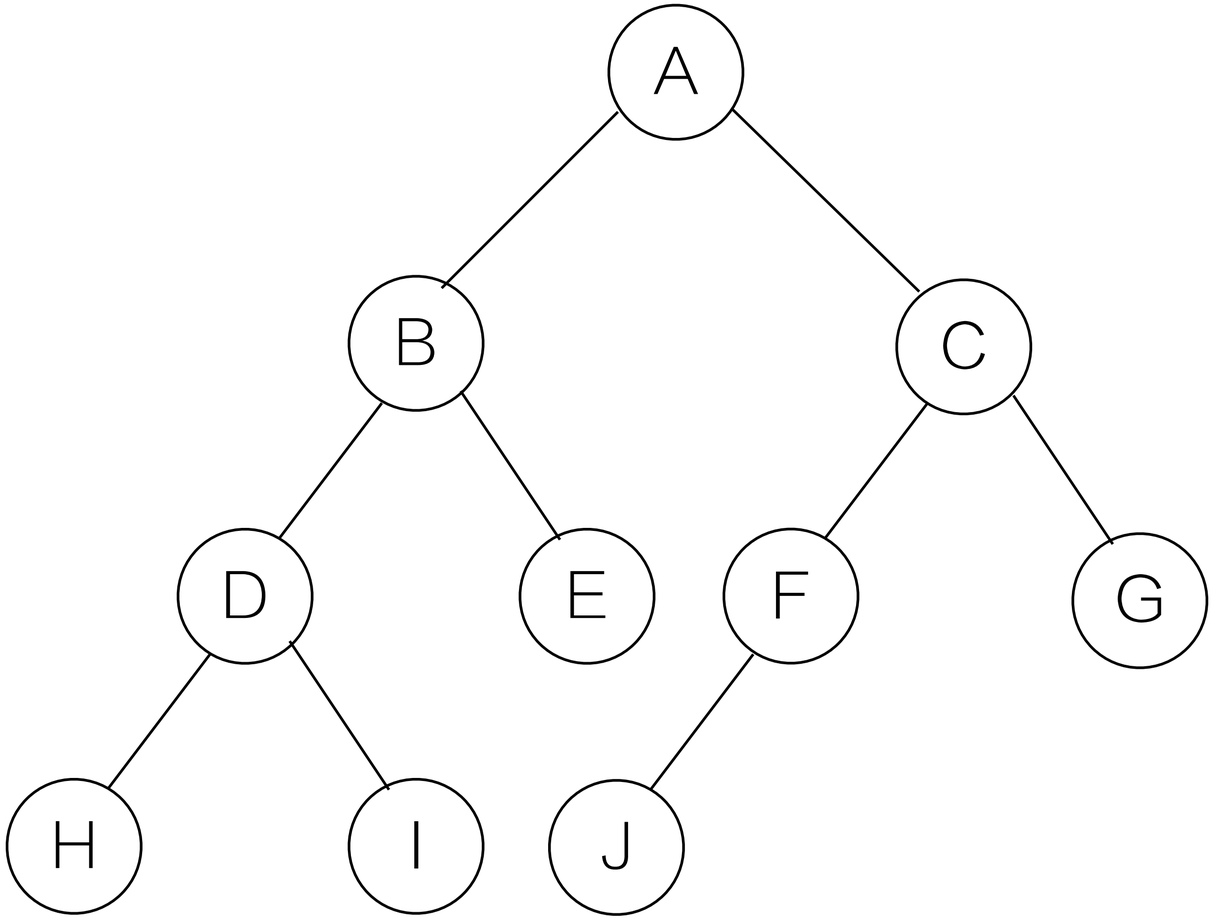




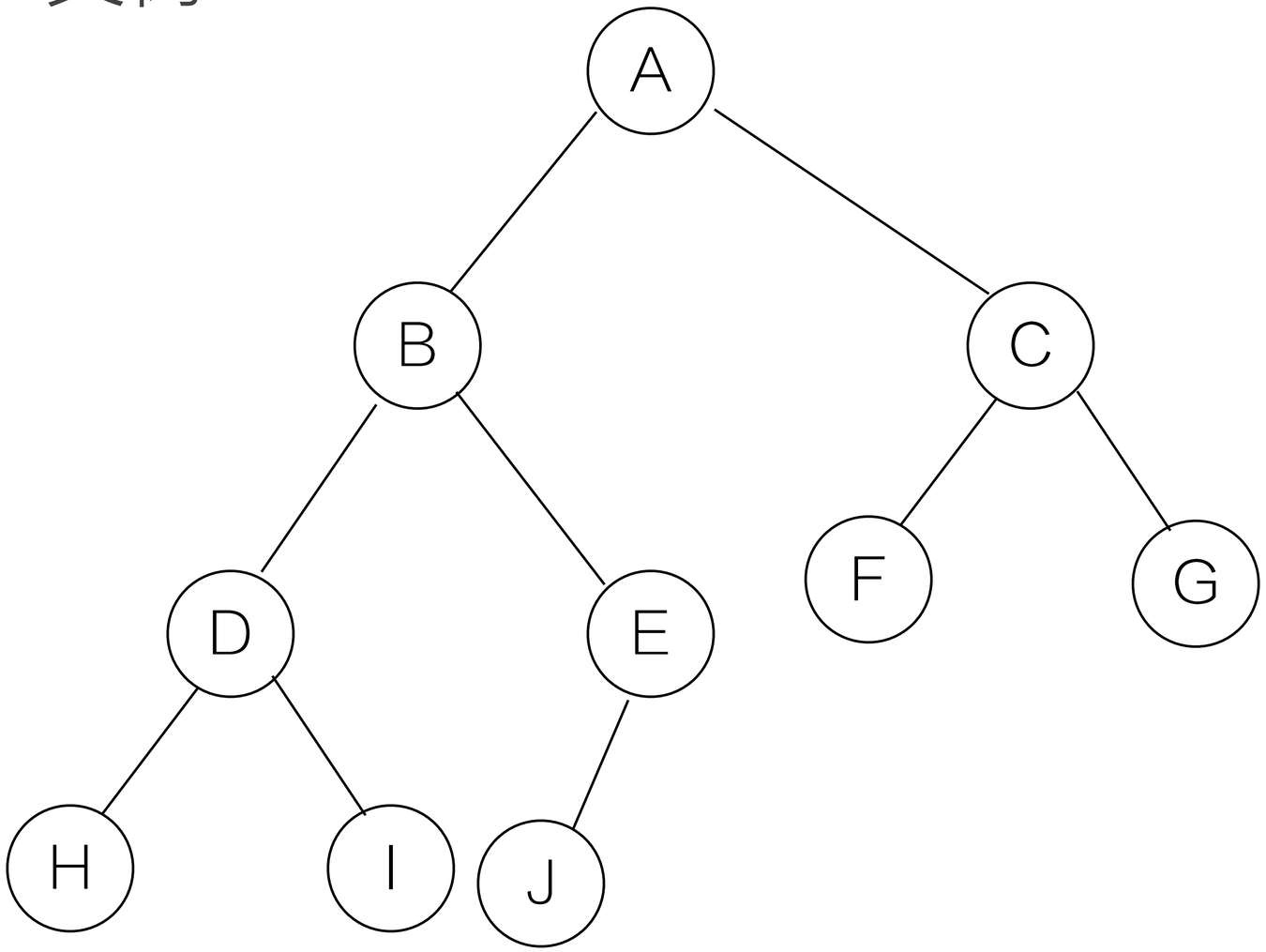
判断完全二叉树



判断完全二叉树



判断完全二叉树



完全二叉树的性质

性质四：具有 n 个结点的完全二叉树的深度为 $\log_2 n+1$（向下取整）

性质五：如果对一棵有 n 个结点的完全二叉树（其深度为 $\log_2 n+1$（向下取整））的结点按层序编号（从第 1 层到第 $\log_2 n+1$（向下取整层），每层左到右），则对任一结点 i（$1 \leqslant i \leqslant n$），以下结论成立。

（1）如果 i=1，则结点 i 是二叉树的根，无双亲；如果结点 i>1则其双亲是结点 i/2（向下取整）

（2）如果 2i>n，则结点 i 无左孩子（结点 i 为叶子结点）；否则其左孩子是结点2i

（3）如果 2i+1>n，则结点 i 无右孩子；否则其右孩子的结点是 2i+1。

二叉树 – 真题解析

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层?

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点?

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

第 6 层共有多少个非叶结点？

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

第 6 层共有多少个非叶结点？　32 - 8 = 24个

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

第 6 层共有多少个非叶结点？　32 - 8 = 24个

第 7 层最多有多少个结点？

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

第 6 层共有多少个非叶结点？　32 - 8 = 24个

第 7 层最多有多少个结点？　48个

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

第 6 层共有多少个非叶结点？　32 - 8 = 24个

第 7 层最多有多少个结点？ 48个

前 6 层最多有多少个结点？

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

可能出现叶子结点的地方在哪些层？　最后两层　当前完全二叉树最多到第 7 层

第 6 层共有多少个结点？　二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。32个

第 6 层共有多少个非叶结点？　32 - 8 = 24个

第 7 层最多有多少个结点？　48个

前 6 层最多有多少个结点？　深度为 k 的二叉树最多有$2^{k}-1$（$k \geqslant 1$）个结点。　63个

二叉树 – 真题解析

[2009] 5. 已知一棵完全二叉树的第6层（设根为第1层）有8个叶结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

二叉树 – 真题解析

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

$n = n_2 + 1 + n_1 + n_2$

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

$n = n_2 + 1 + n_1 + n_2$

当前二叉树共有偶数个结点，说明，有 1 个度为 1 的结点

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

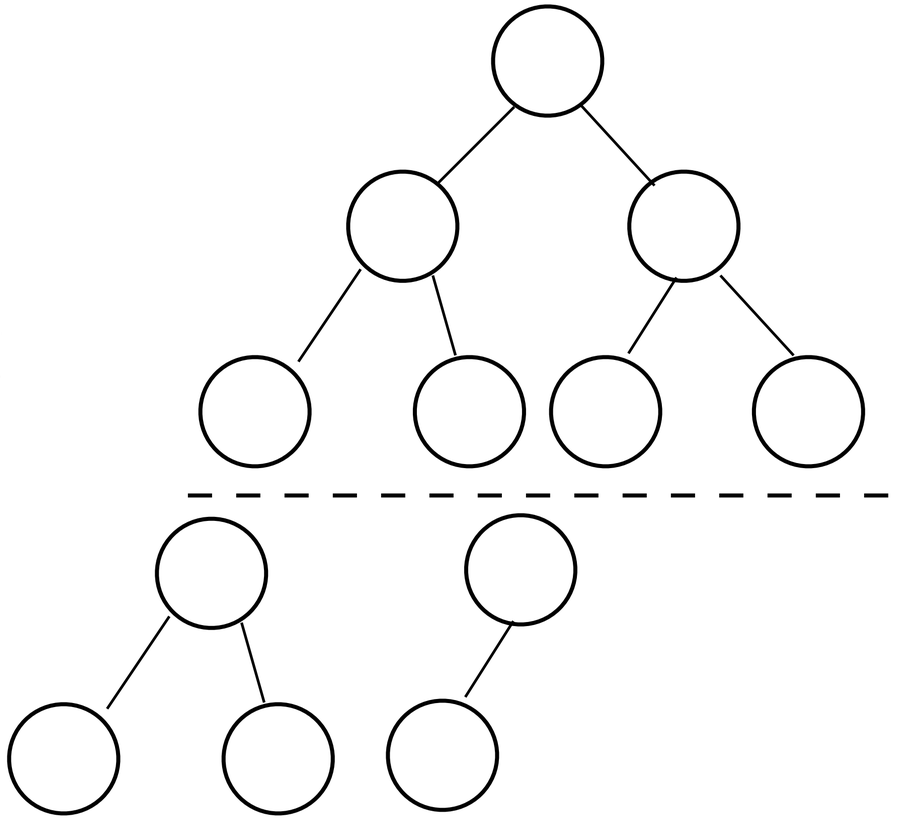
A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

$n = n_2 + 1 + n_1 + n_2$

当前二叉树共有偶数个结点，说明，有 1 个度为 1 的结点



[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

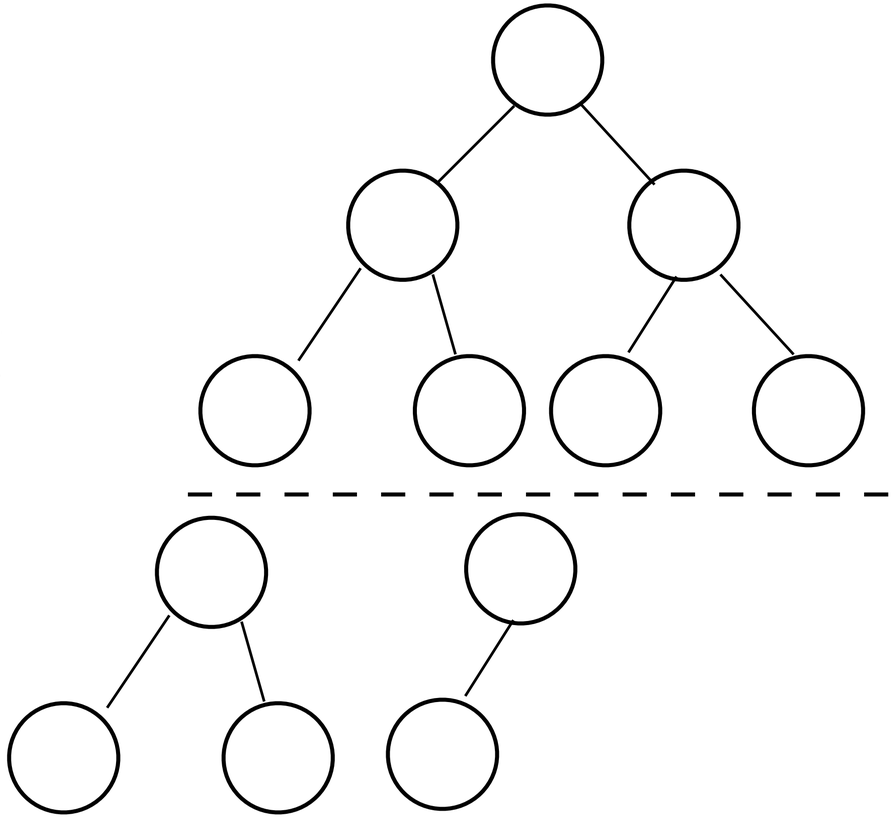
$n = n_0 + n_1 + n_2$

对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

$n = n_2 + 1 + n_1 + n_2$

当前二叉树共有偶数个结点，说明，有 1 个度为 1 的结点

$n = n_2 + 1 + 1 + n_2$



[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

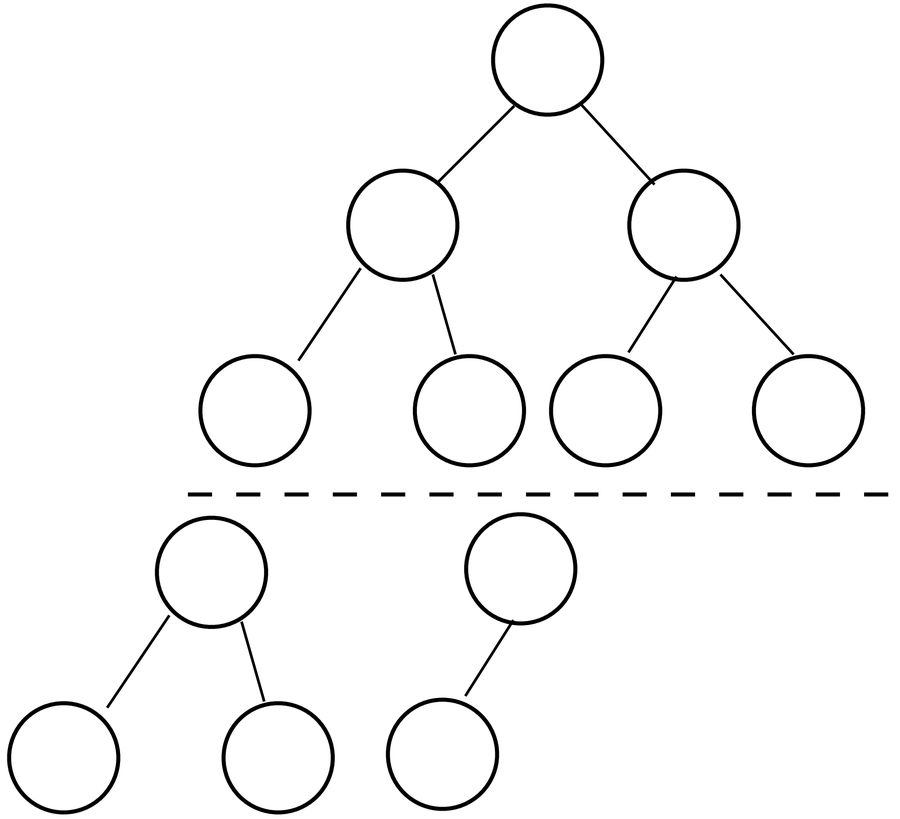
对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

$n = n_2 + 1 + n_1 + n_2$

当前二叉树共有偶数个结点，说明，有 1 个度为 1 的结点

$n = n_2 + 1 + 1 + n_2$

$768 = 2n_2 + 2$



[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

$n = n_0 + n_1 + n_2$

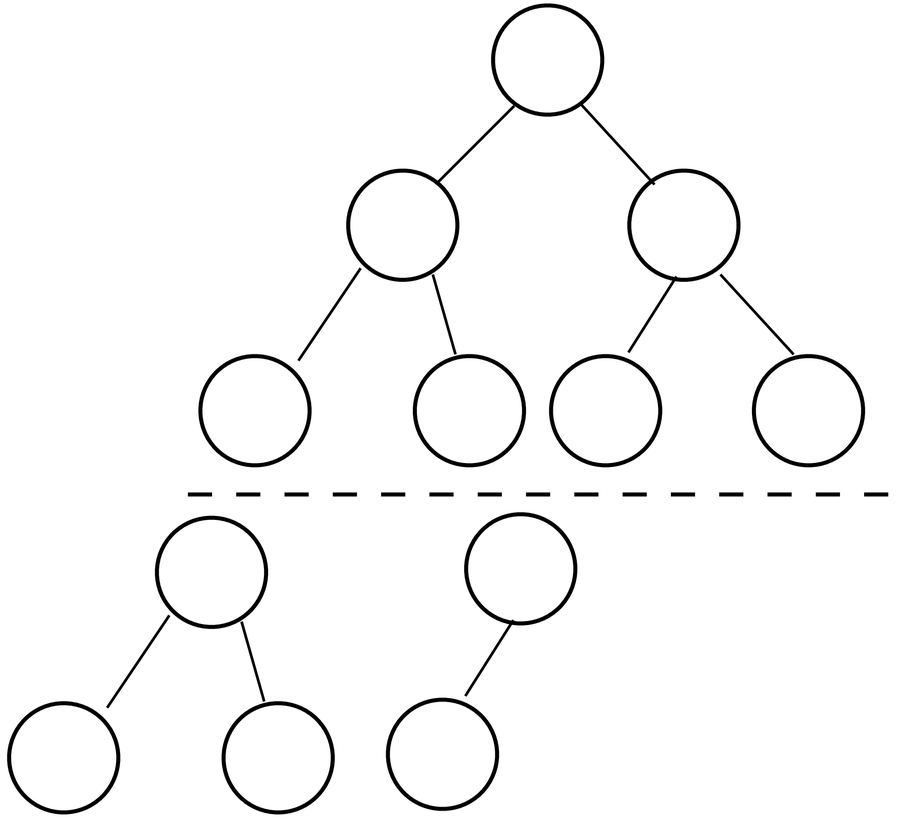
对于任何非空的二叉树T，如果叶子结点的个数为n_0，
而度为2的结点数为n_2，则 $n_0=n_2+1$

$n = n_2 + 1 + n_1 + n_2$

当前二叉树共有偶数个结点，说明，有 1 个度为 1 的结点

$n = n_2 + 1 + 1 + n_2$

$768 = 2n_2 + 2$　　$n_2 = 383$



二叉树 – 真题解析

[2011] 4. 若一棵完全二叉树有768个结点，则该二叉树中叶结点的个数是_____。

A. 257　　B. 258　　C. 384　　D. 385

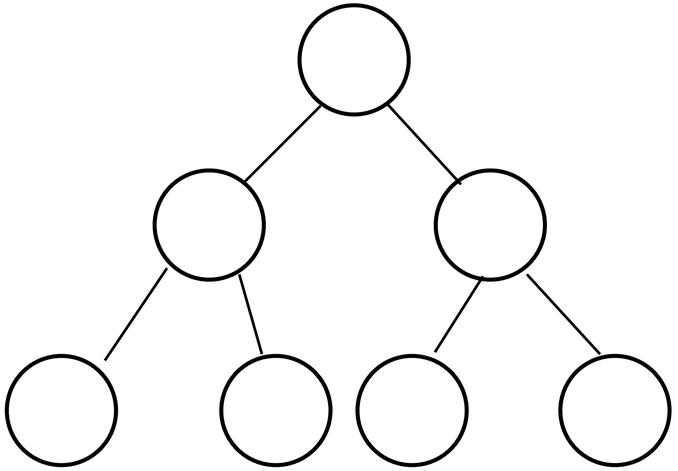
二叉树 – 真题解析

[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

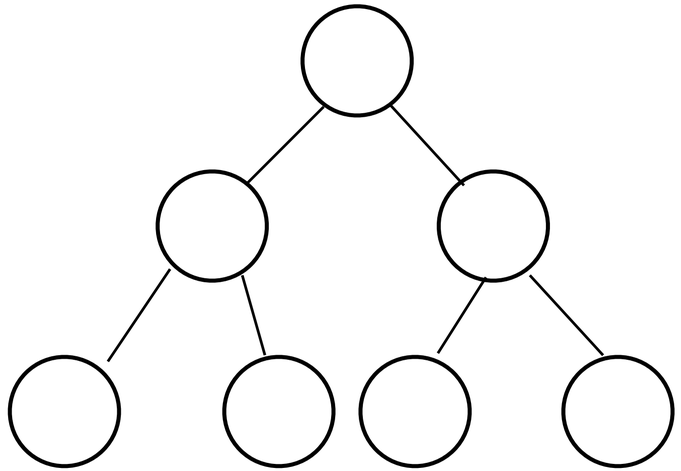
A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1



[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

A. 2k−1　　B. 2k　　C. k^2　　D. 2^k-1

假设深度为 h ，第 h 层叶结点个数 k 为?

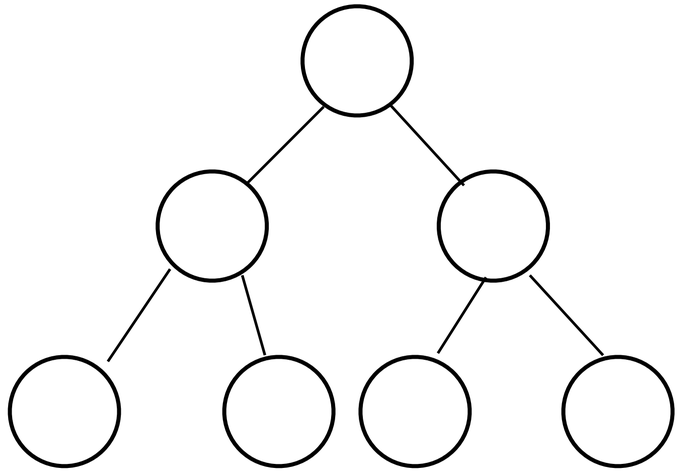


[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

A. 2k-1　　B. 2k　　C. k^2　　D. 2^k-1

假设深度为 h ，第 h 层叶结点个数 k 为?

性质一：二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。



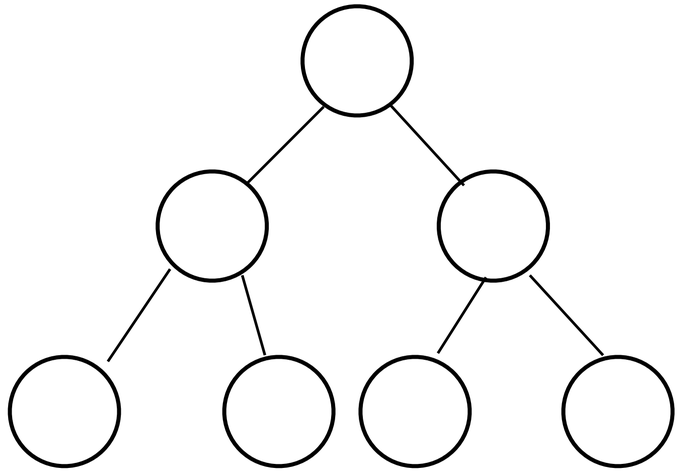
[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

假设深度为 h ，第 h 层叶结点个数 k 为?

性质一：二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。

$k = 2^{h-1}$



[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

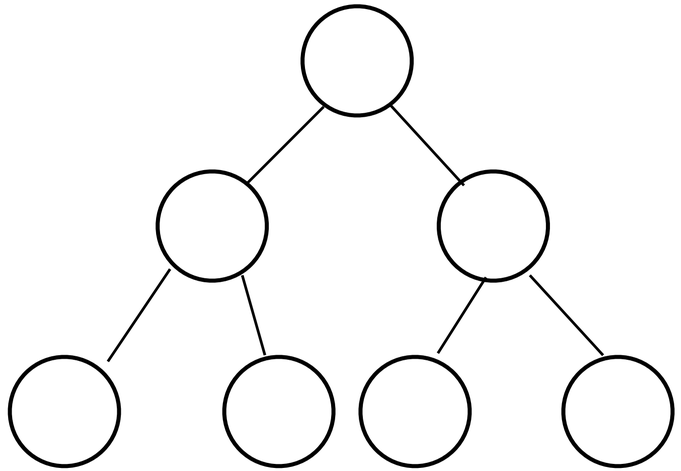
A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

假设深度为 h ，第 h 层叶结点个数 k 为?

性质一：二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。

$k = 2^{h-1}$

$2k = 2^h$



[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

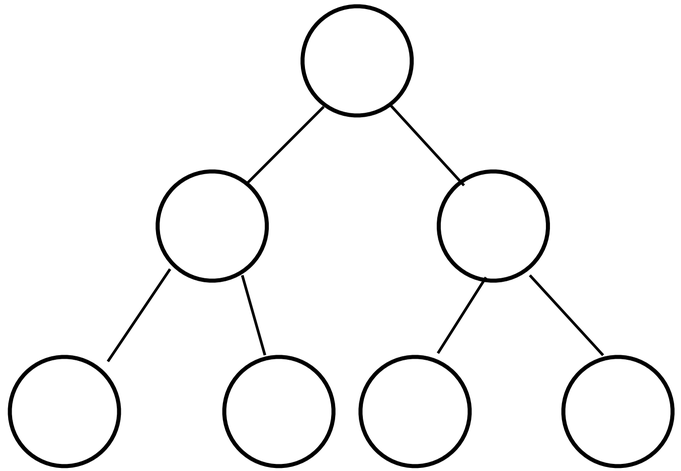
A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

假设深度为 h ，第 h 层叶结点个数 k 为?

性质一：二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。

$k = 2^{h-1}$

$2k = 2^h$



满二叉树：深度为 k 且含有 2^k-1 个结点的二叉树

[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

假设深度为 h ，第 h 层叶结点个数 k 为?

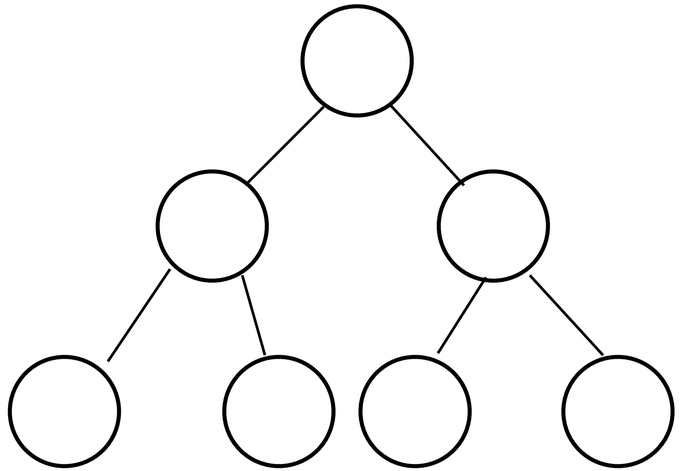
性质一：二叉树的第 i 层最多有2^{i-1}（$i \geqslant 1$）个结点。

$k = 2^{h-1}$

$2k = 2^h$

满二叉树：深度为 h 且含有 2^h-1 个结点的二叉树

共有 2^h-1个结点　$2k-1$



二叉树 – 真题解析

[2018] 4. 设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有2个子结点。若 T 有 k 个叶结点，则 T 的结点总数是_____。

A. $2k-1$　　B. $2k$　　C. k^2　　D. 2^k-1

二叉树的存储结构-顺序结构



二叉树的存储结构-顺序结构



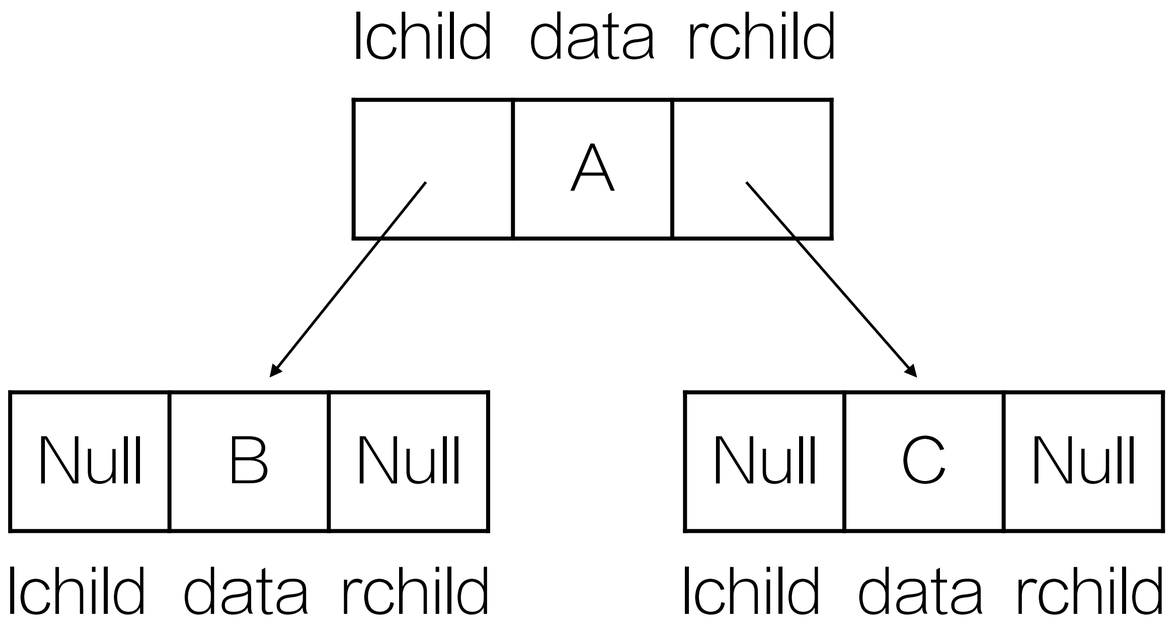
除满二叉树和完全二叉树外，其它场景比较浪费空间

二叉树的存储结构 – 链式结构

```
typedef char ElemType;

typedef struct TreeNode
{
    ElemType data;
    TreeNode *lchild;
    TreeNode *rchild;
}TreeNode;

typedef TreeNode* BiTree;
```



二叉树的遍历 - 前序遍历

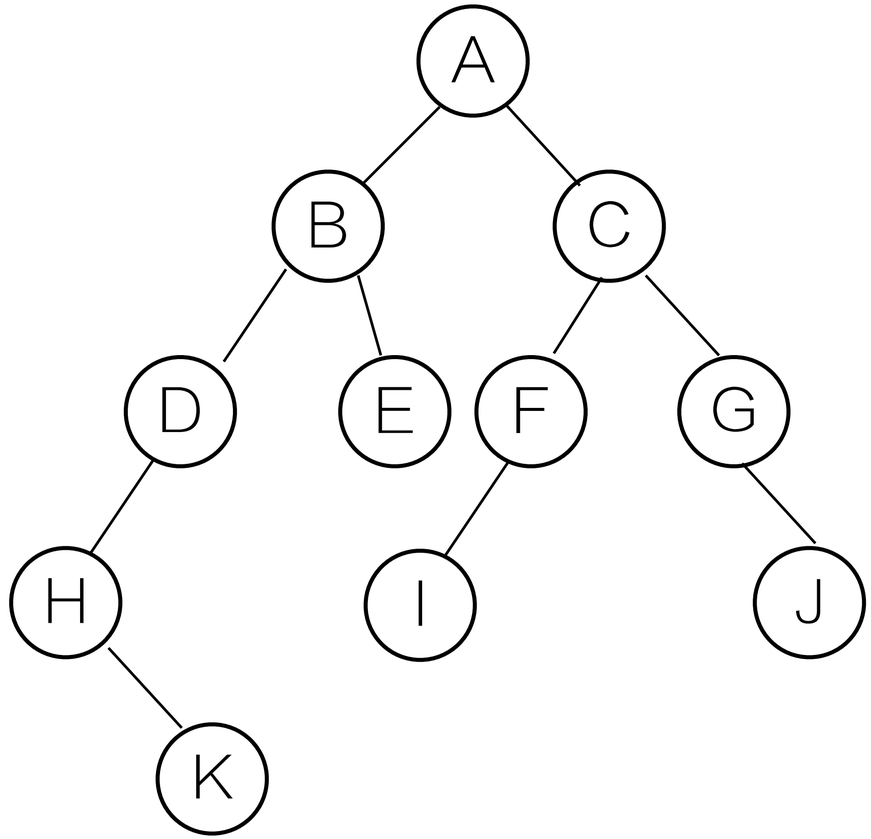
先访问根结点，然后访问左分支上遇到的每一个结点，持续这一过程，直到遇到空结点为止。这时，返回到最近的有右孩子的祖先结点，并从该结点的右孩子开始继续遍历。

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

二叉树的遍历 – 前序遍历

输出内容：

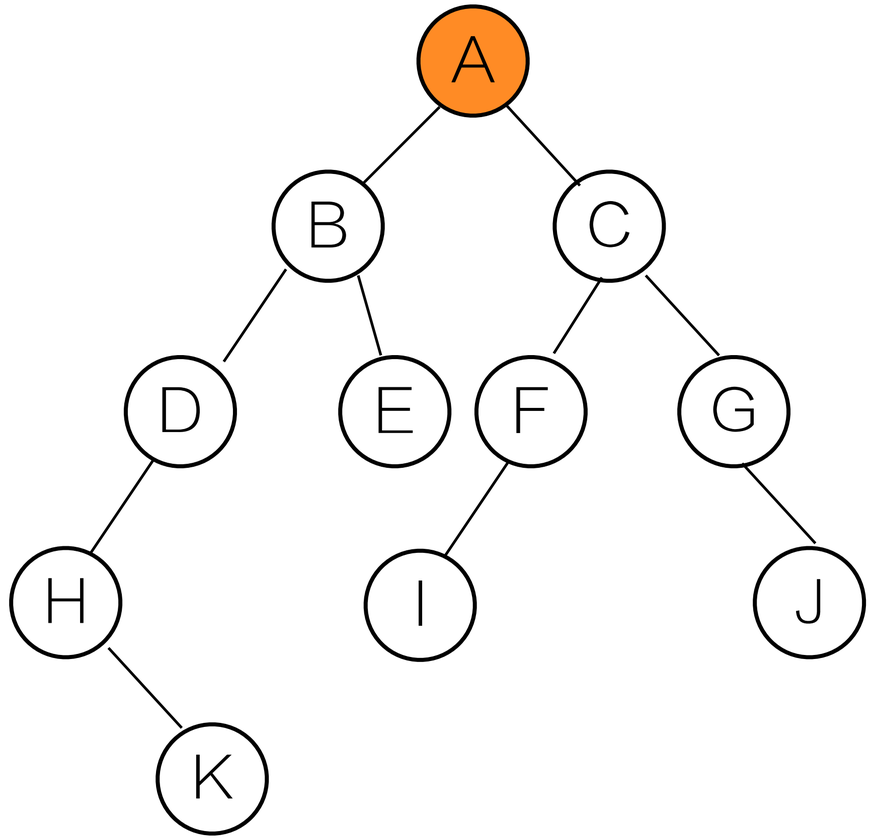
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



二叉树的遍历 – 前序遍历

输出内容：

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

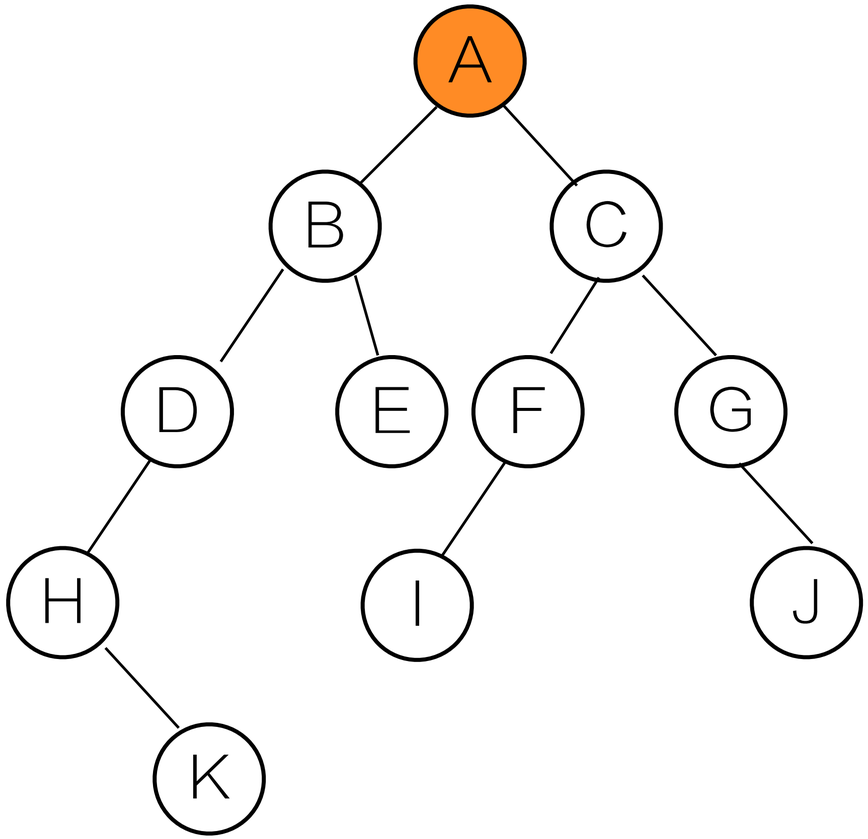


二叉树的遍历 – 前序遍历

输出内容：

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(A)

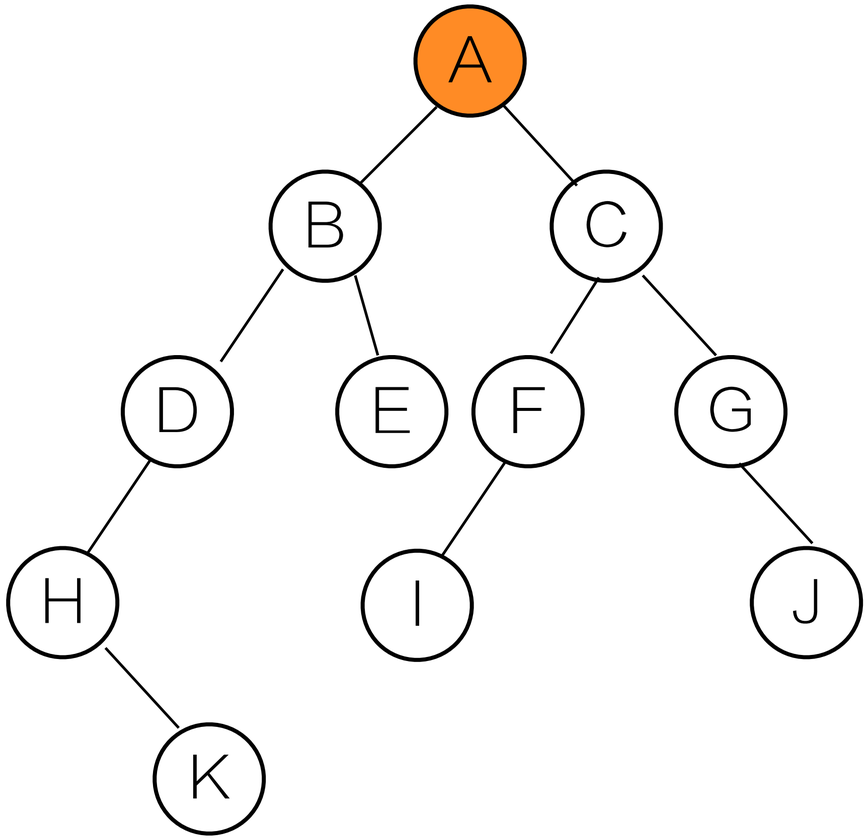


二叉树的遍历 – 前序遍历

输出内容：A

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(A)



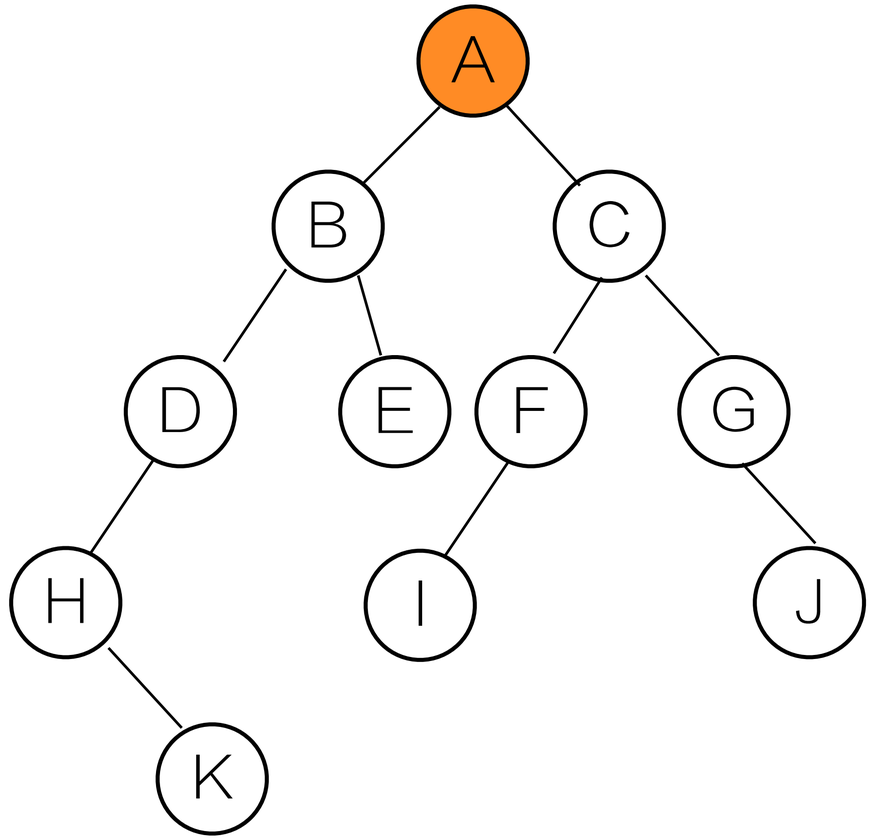
二叉树的遍历 – 前序遍历

输出内容：A

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



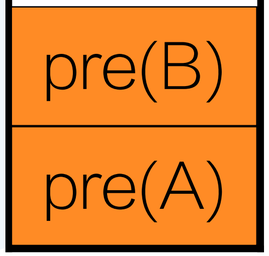
pre(B) pre(C)



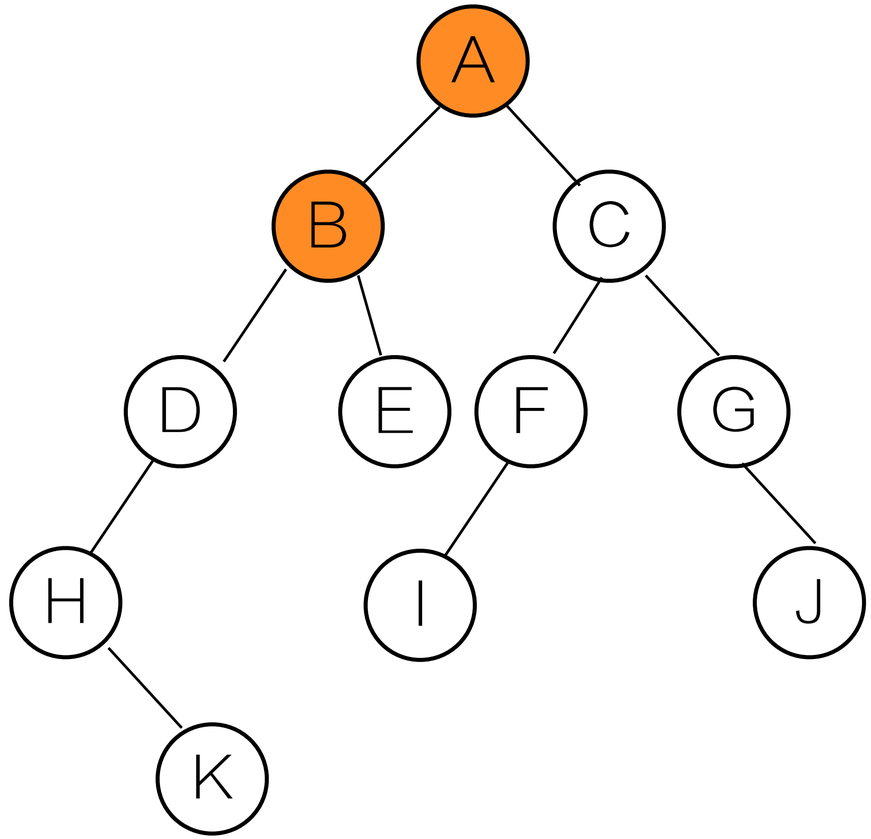
二叉树的遍历 – 前序遍历

输出内容：A

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



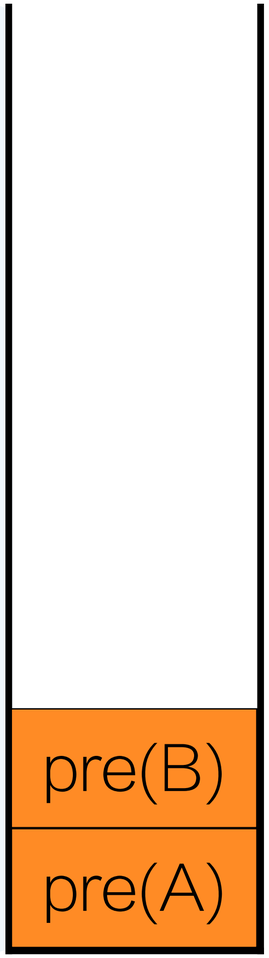
pre(B) pre(C)



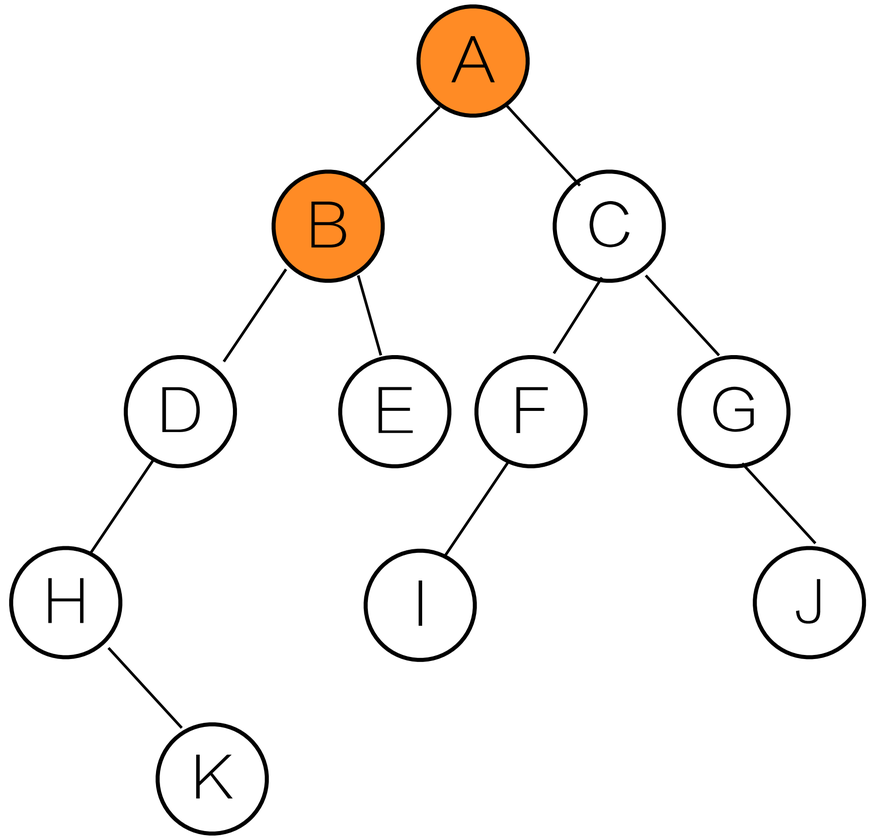
二叉树的遍历 – 前序遍历

输出内容：A B

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



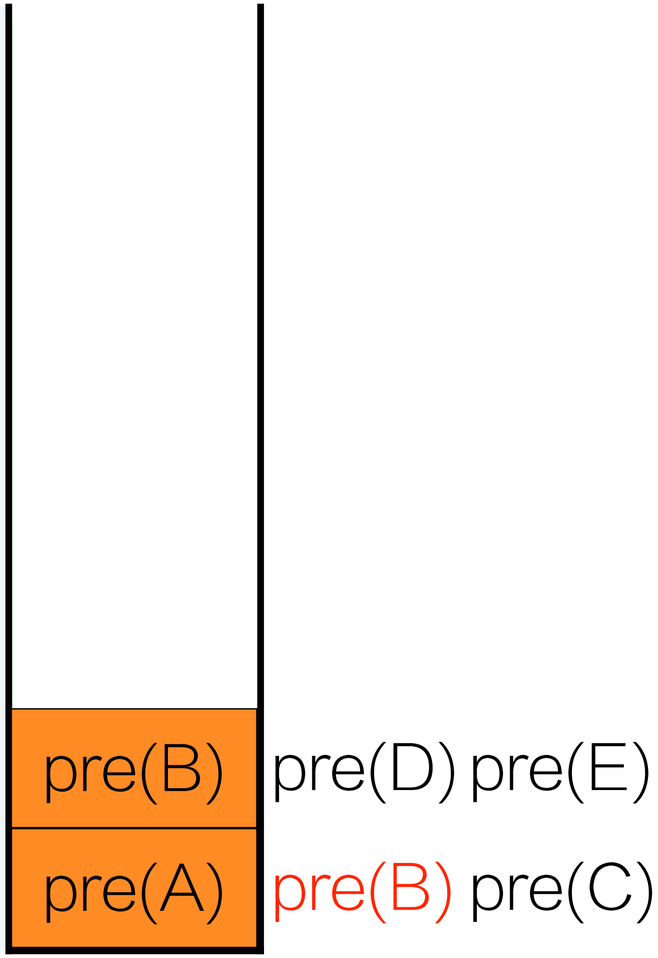
pre(B) pre(C)



二叉树的遍历 – 前序遍历

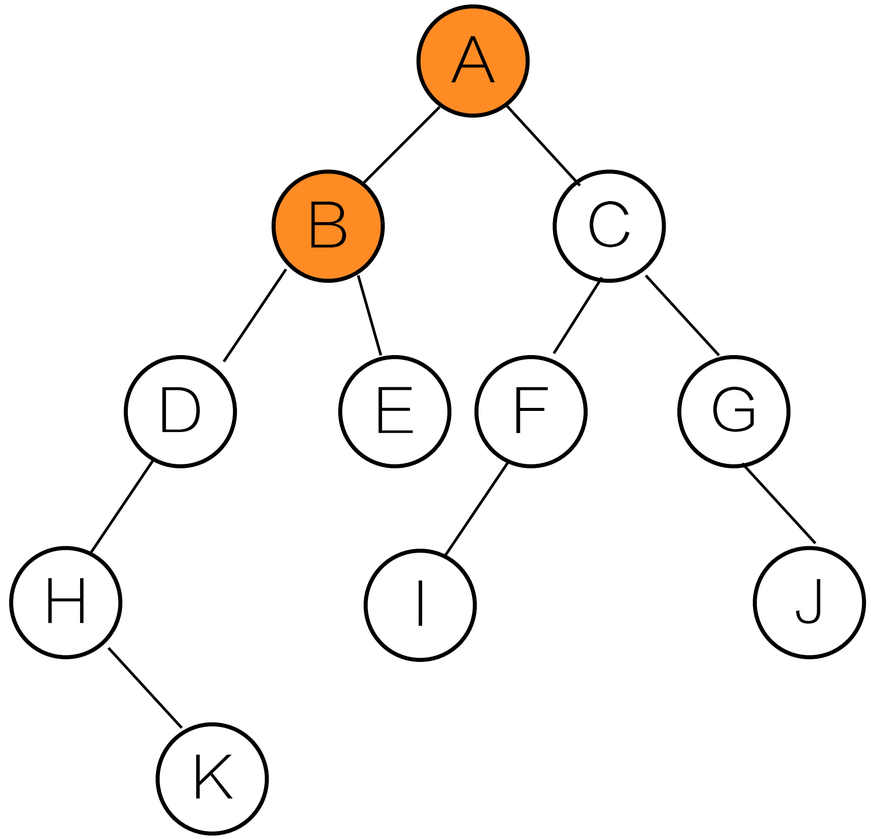
输出内容：A B

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(D) pre(E)

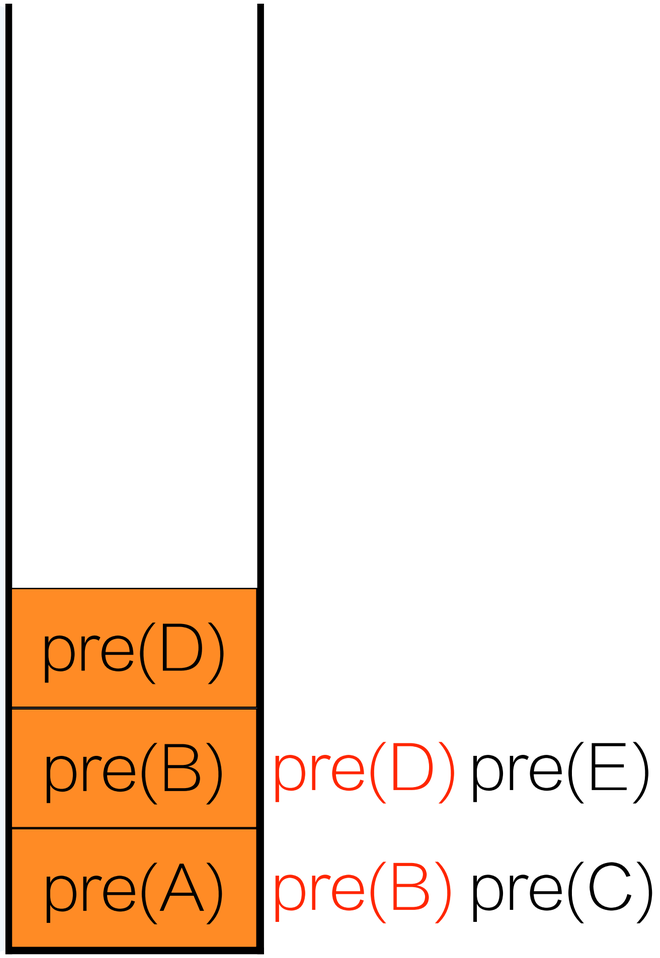
pre(B) pre(C)

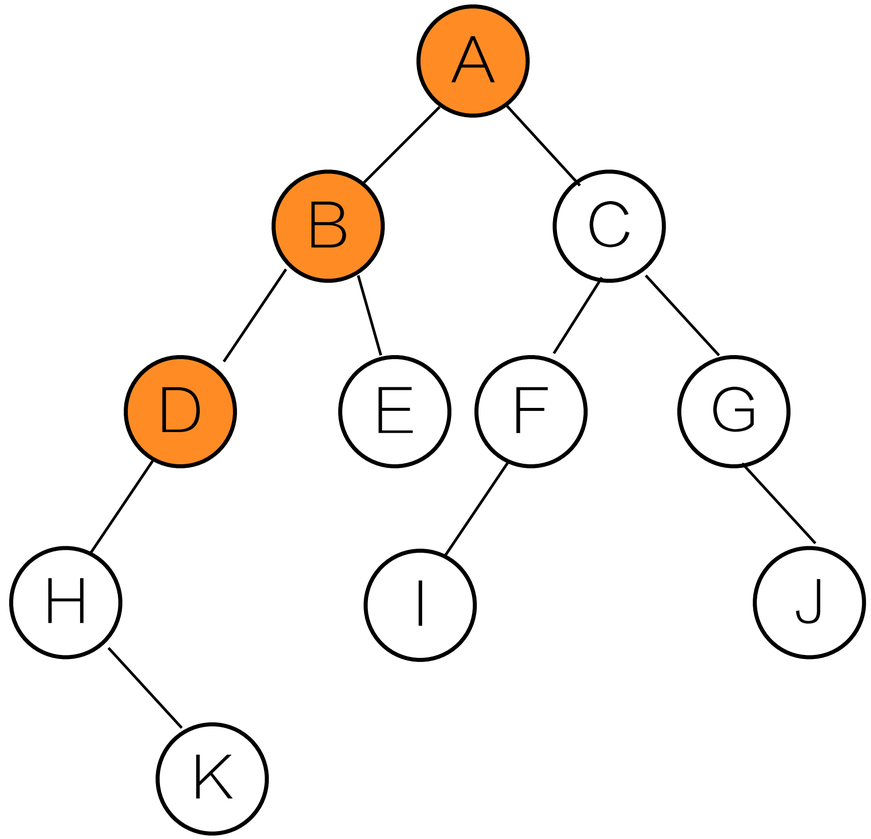


二叉树的遍历 – 前序遍历

输出内容：A B

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

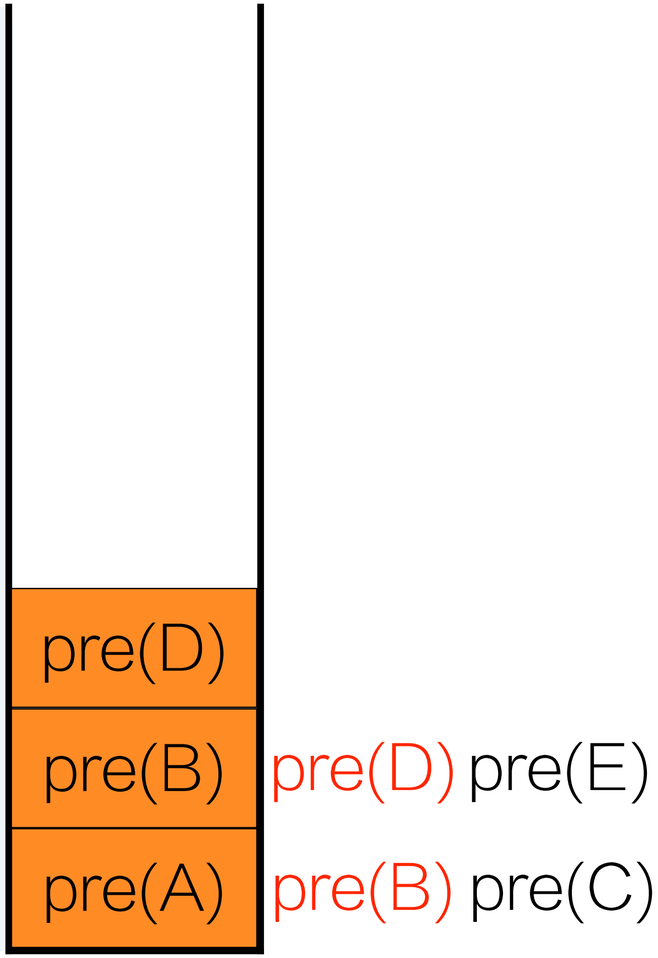


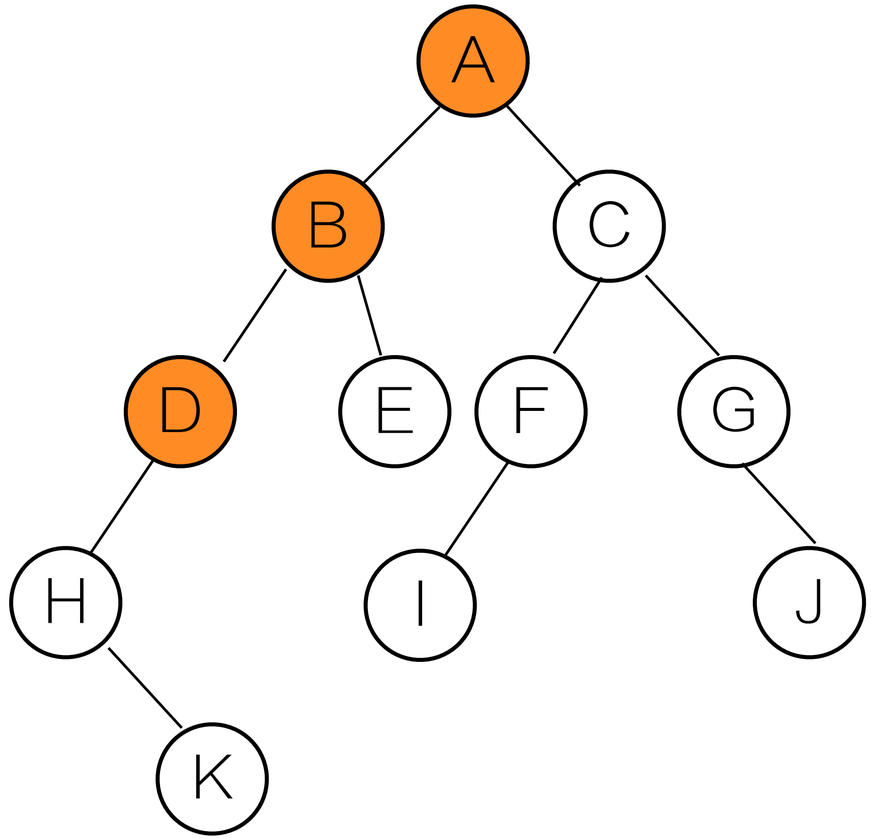


二叉树的遍历 – 前序遍历

输出内容：A B D

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

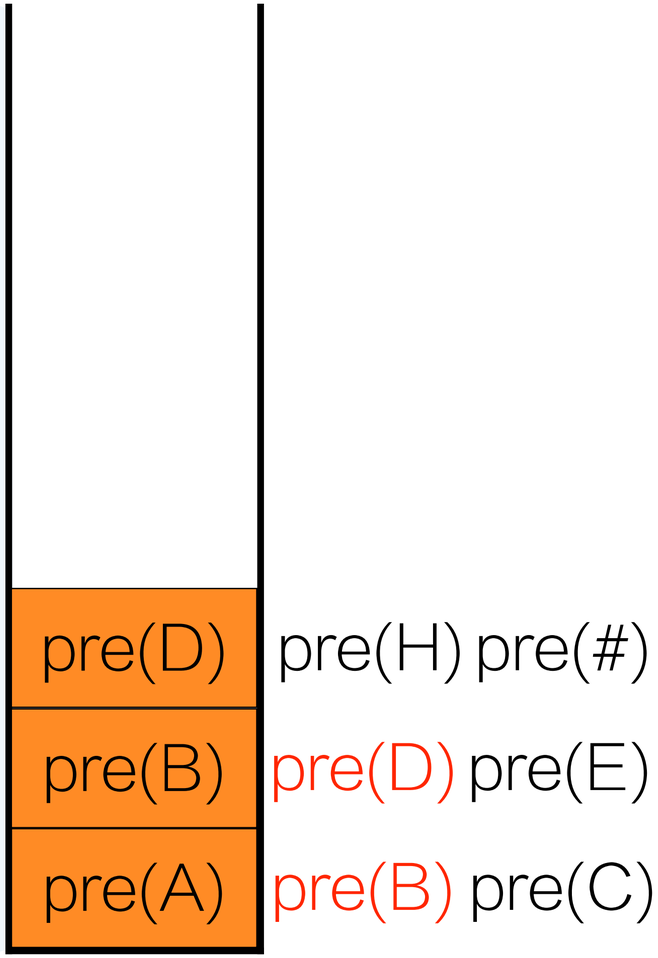




二叉树的遍历 – 前序遍历

输出内容：A B D

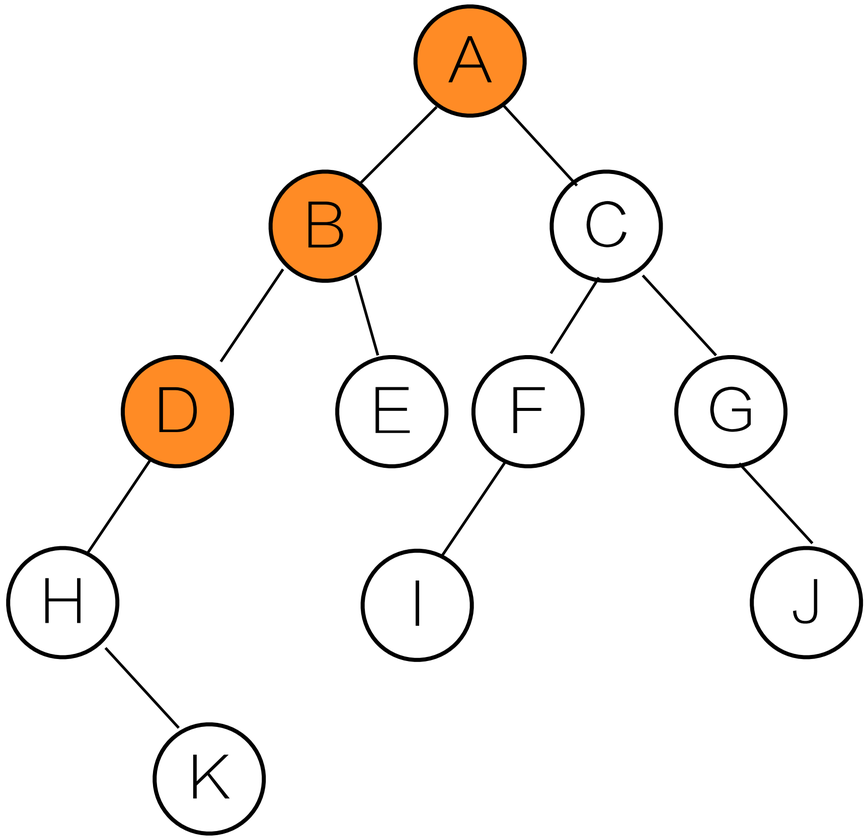
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(H) pre(#)

pre(D) pre(E)

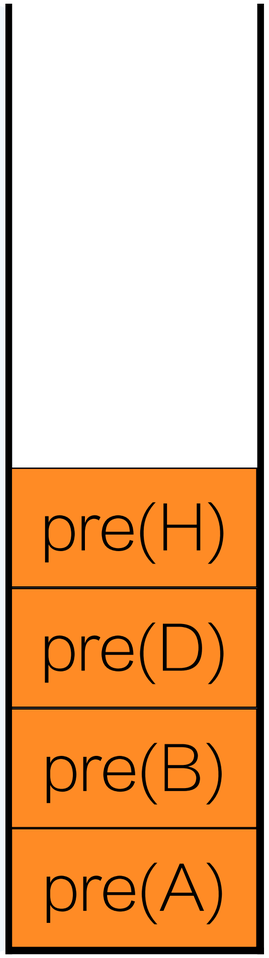
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D

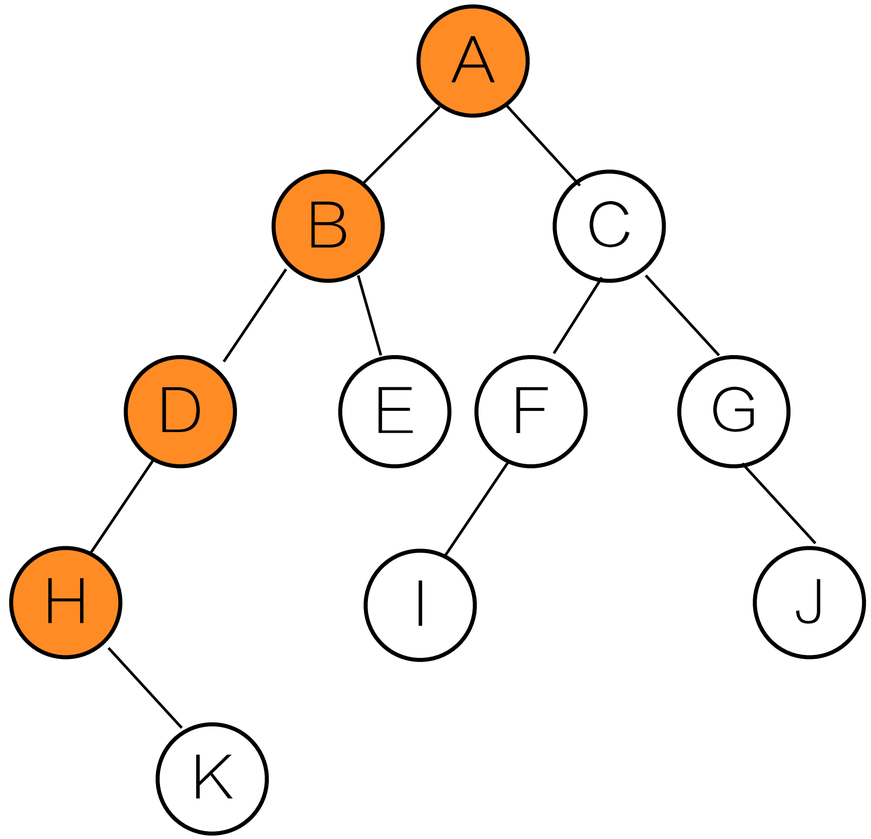
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(H) pre(#)

pre(D) pre(E)

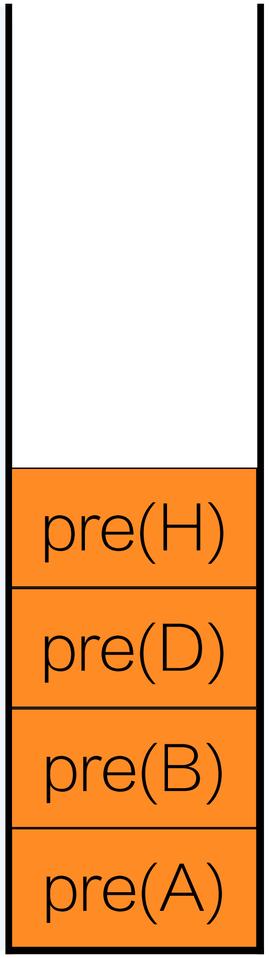
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H

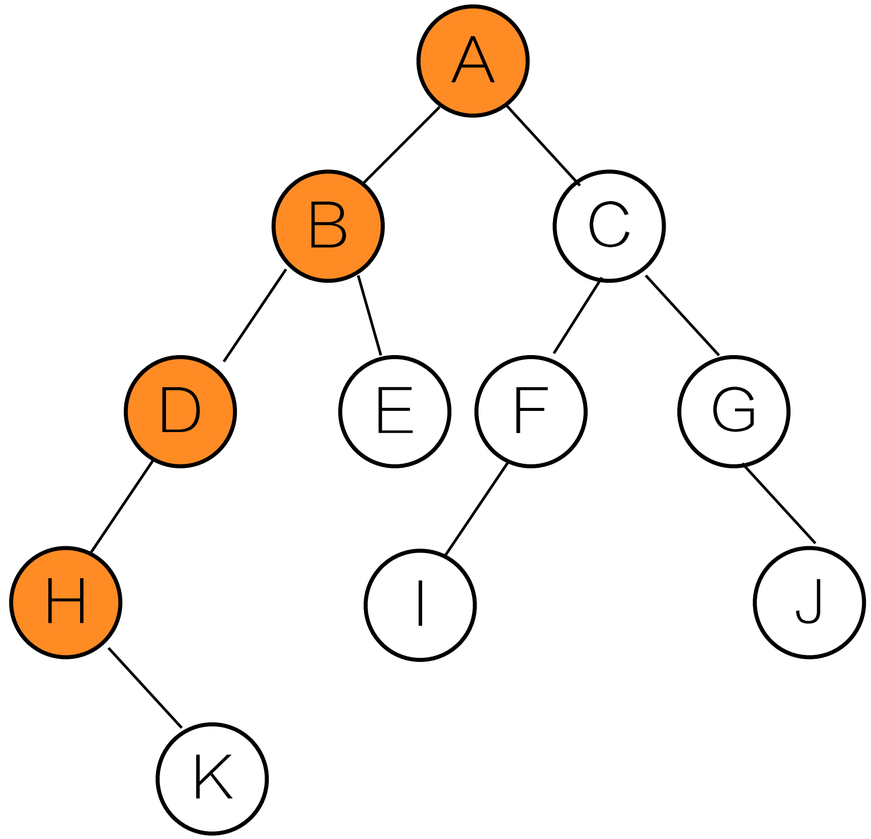
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(H) pre(#)

pre(D) pre(E)

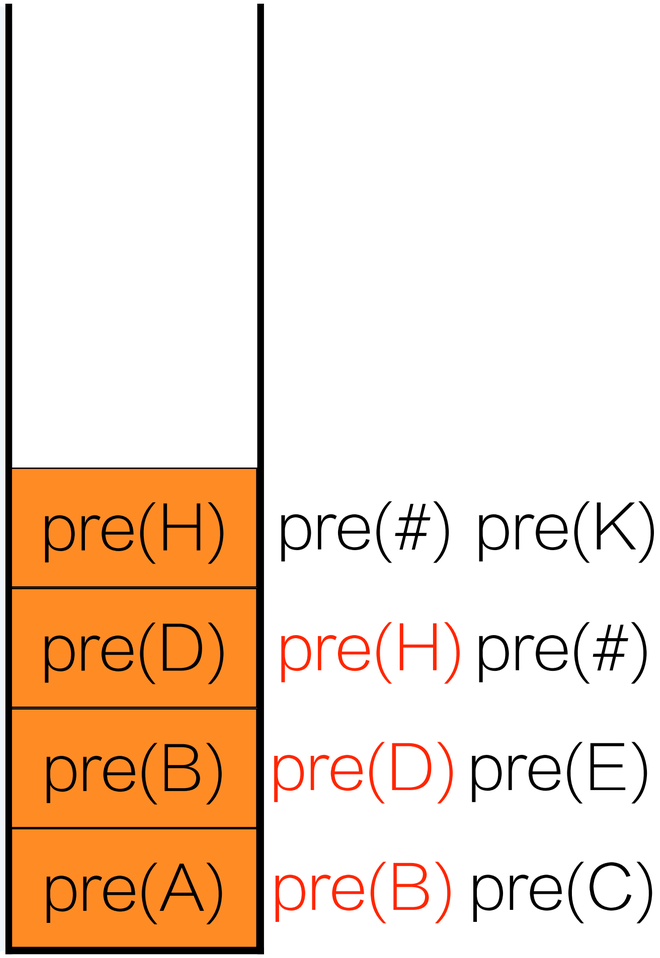
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

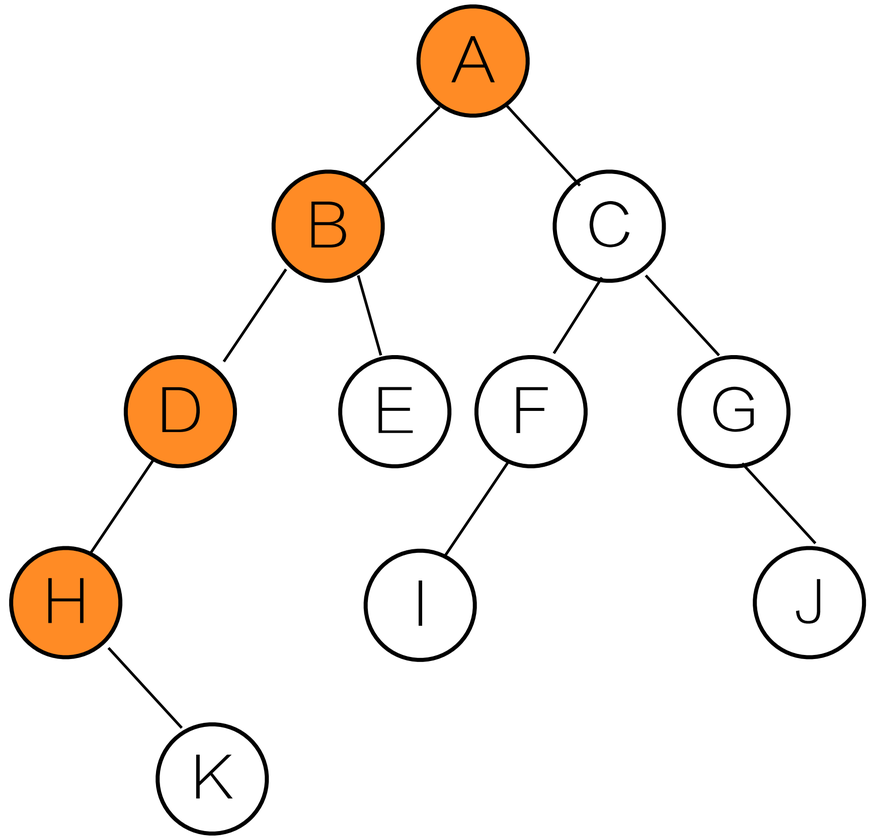


pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

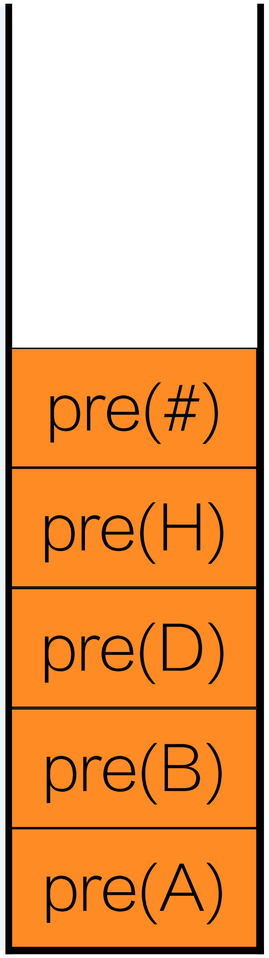
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

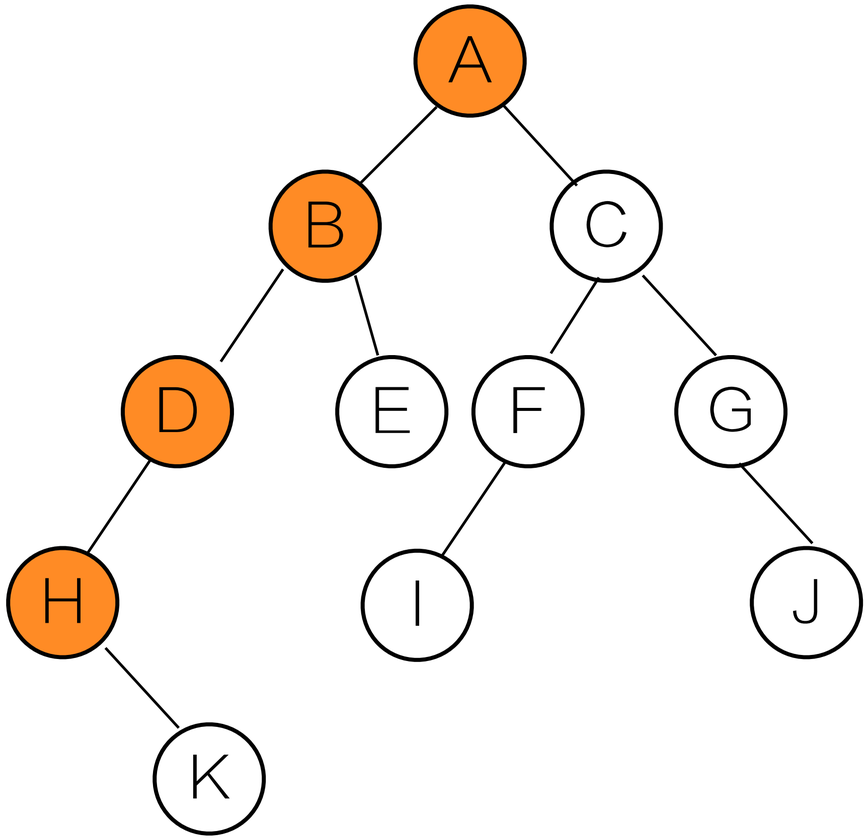


pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

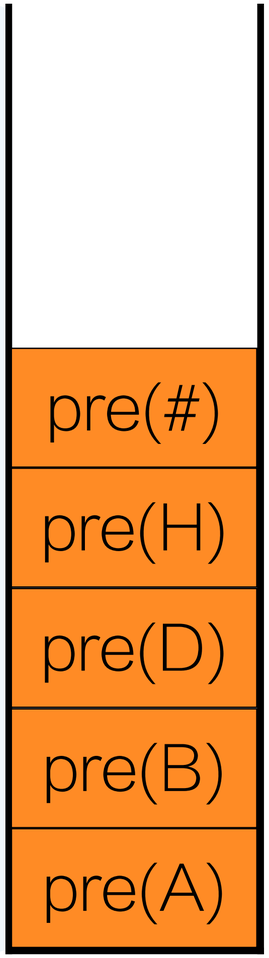
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



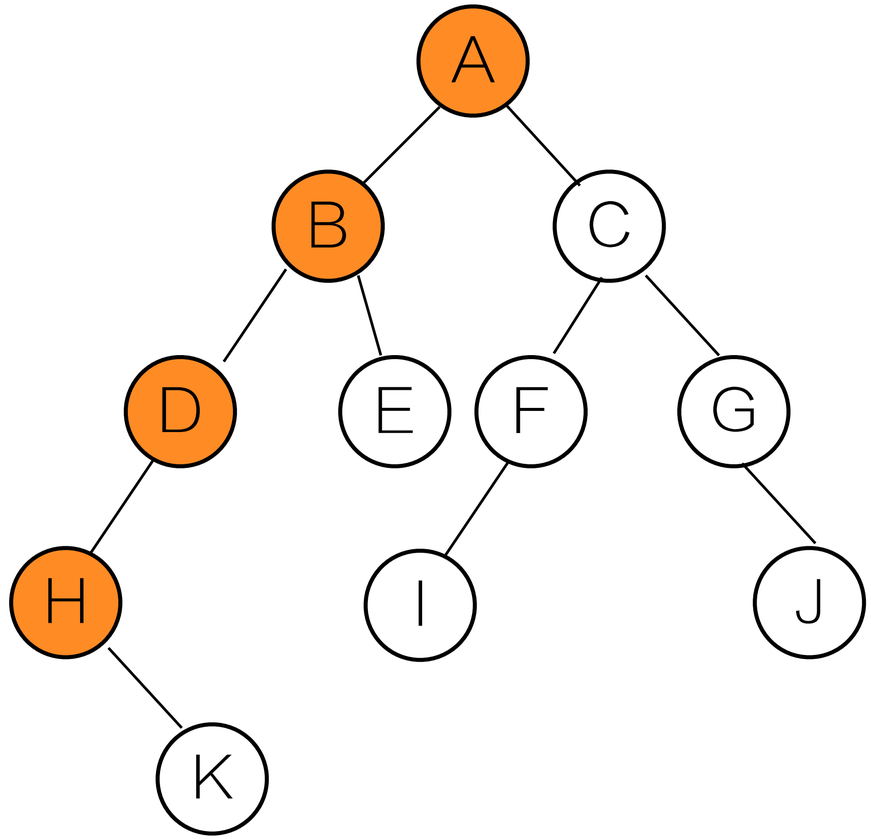
return

pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

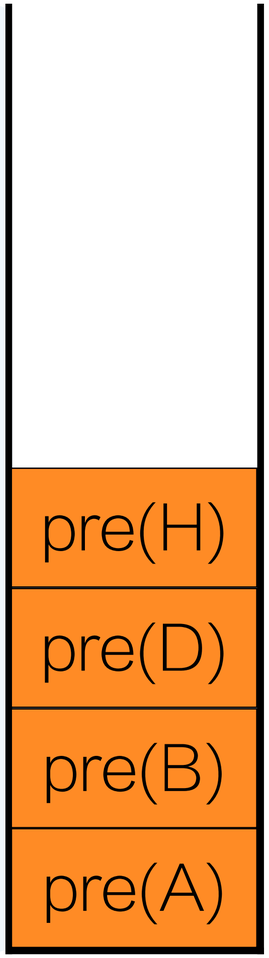
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

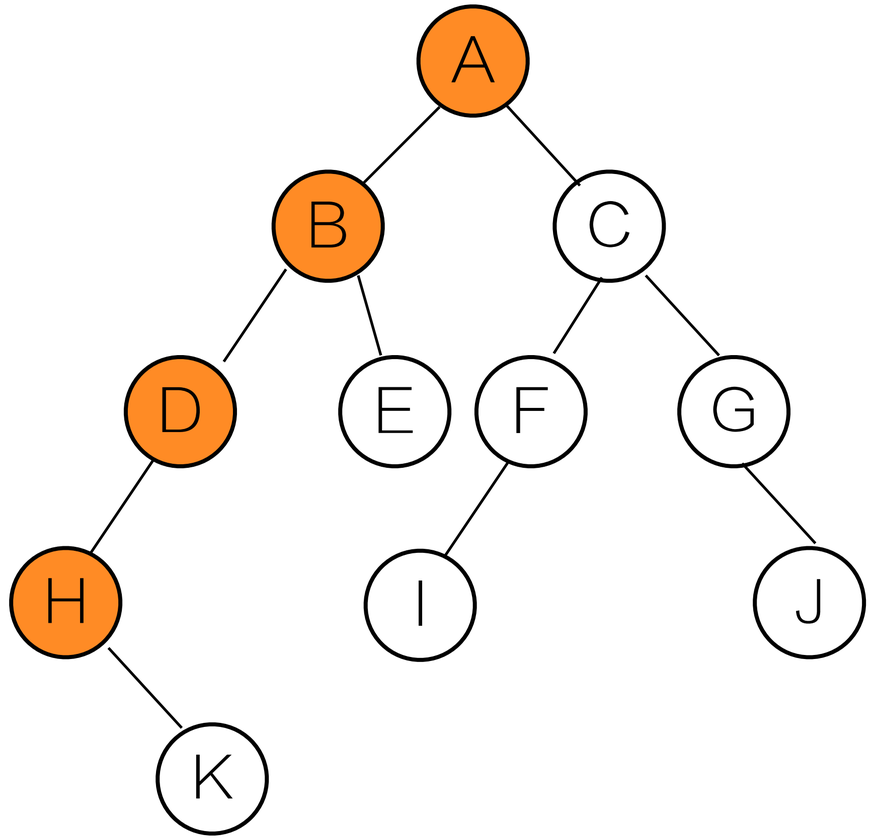


pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

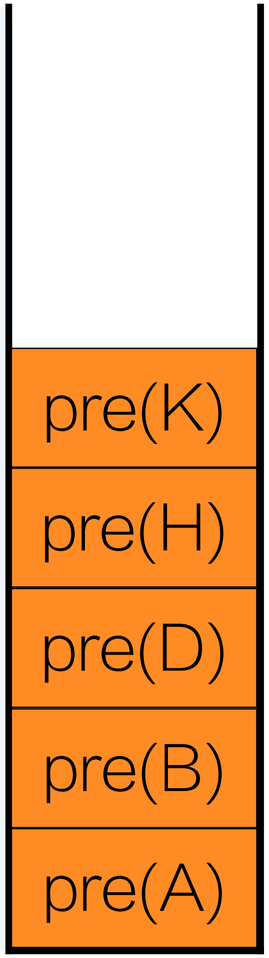
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

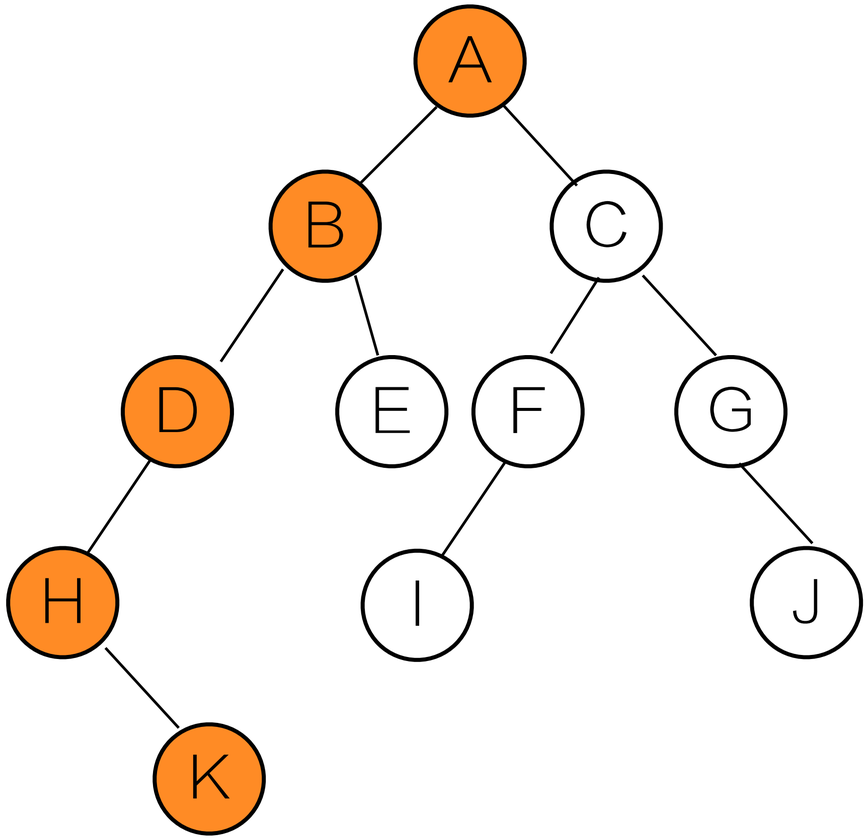


pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

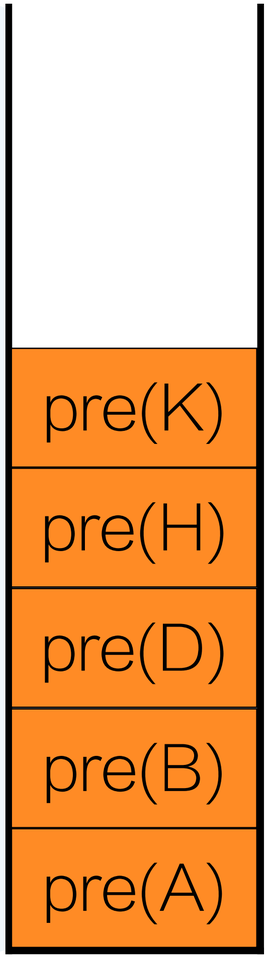
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

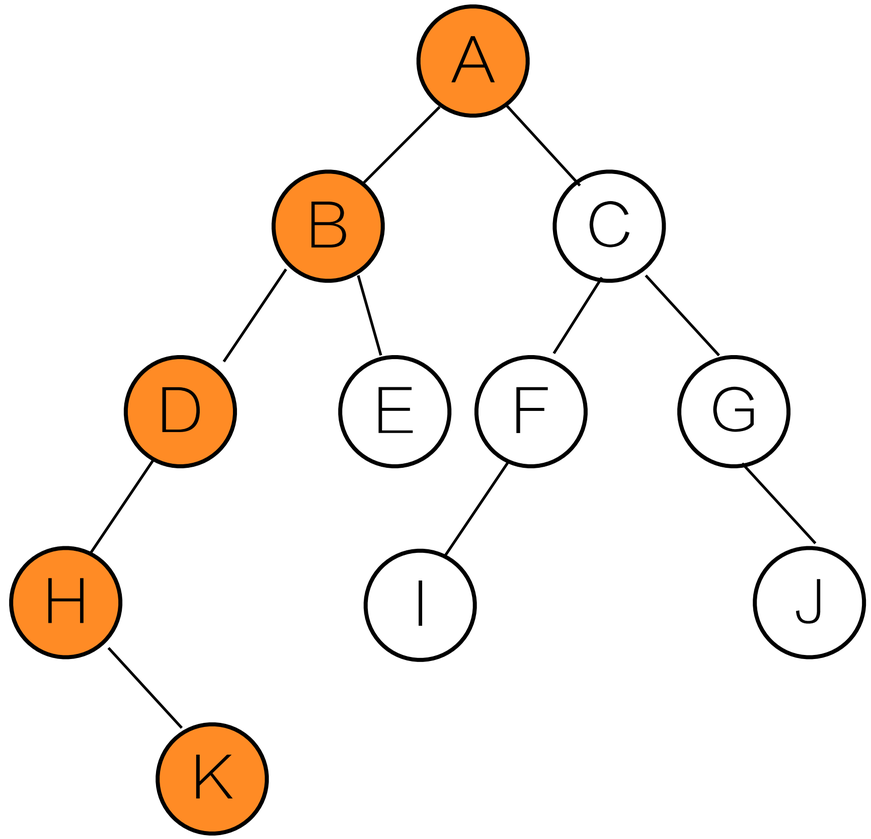


pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

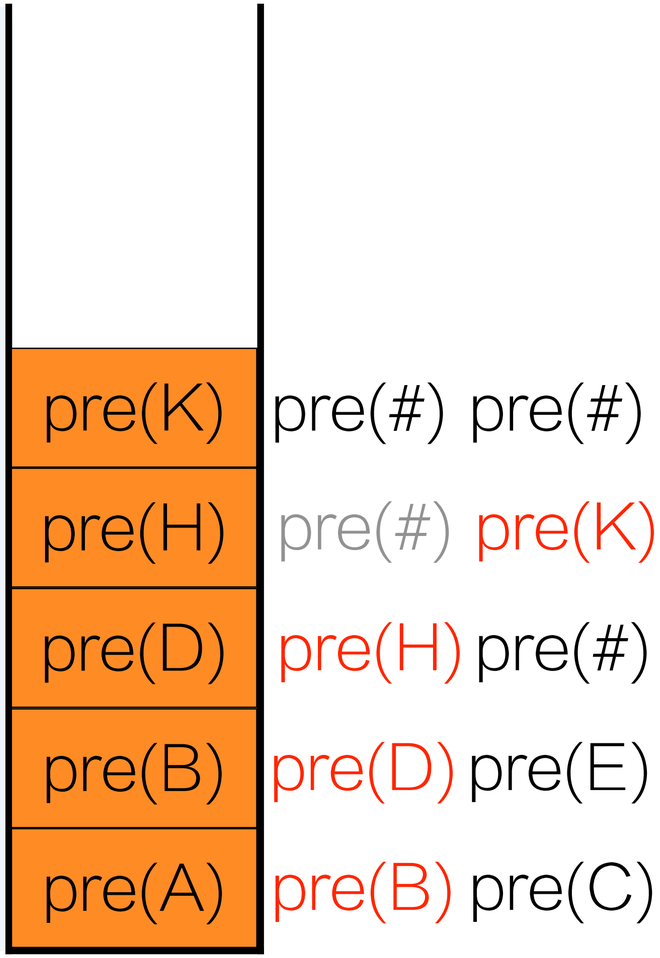
pre(B) pre(C)

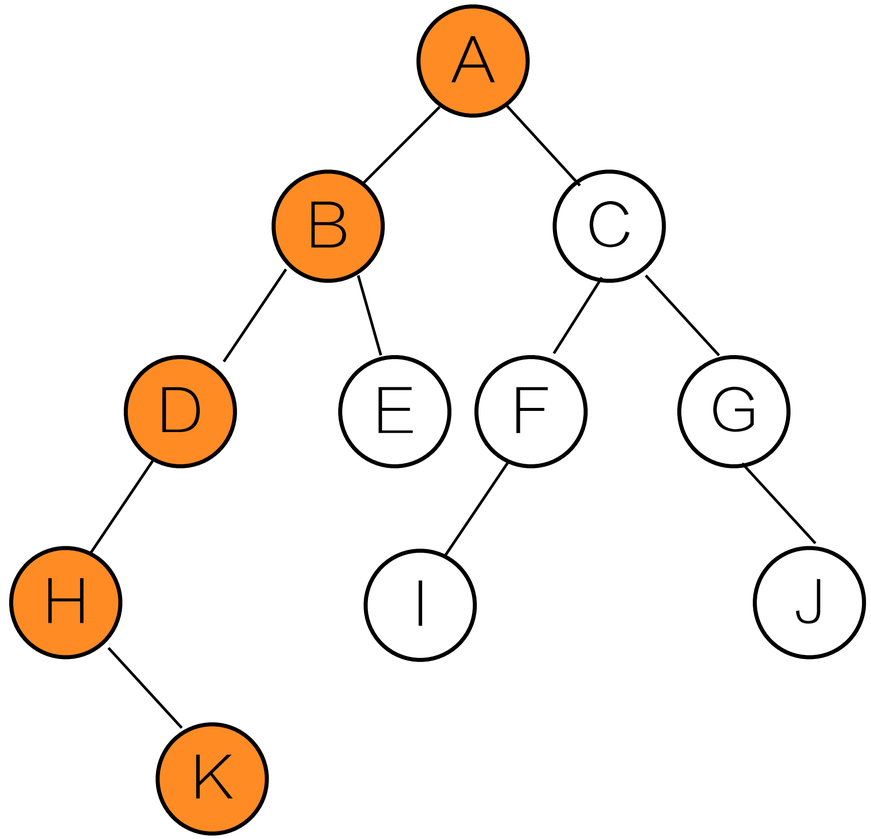


二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

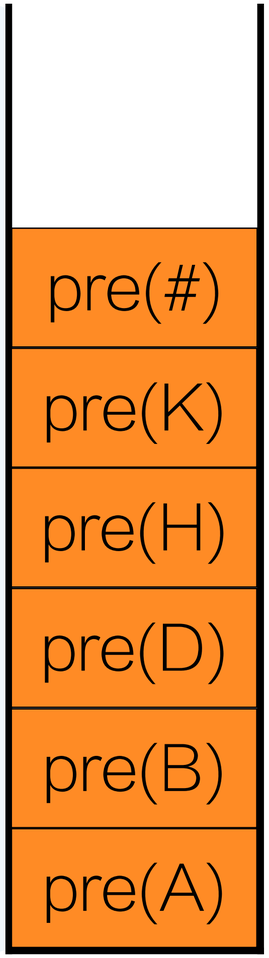




二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



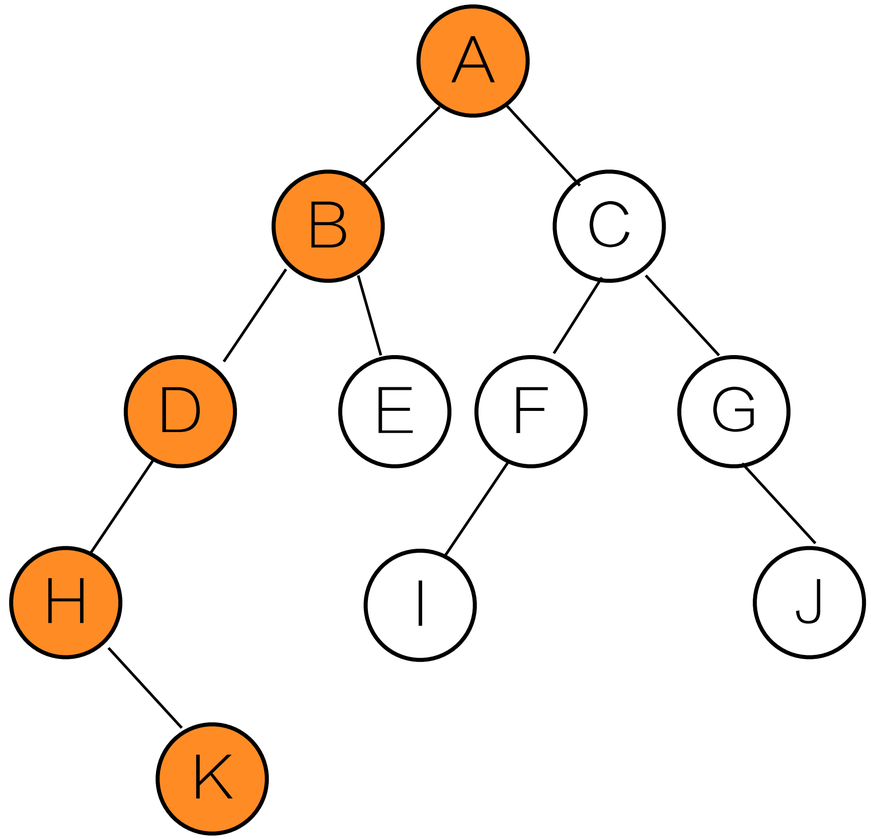
pre(#) pre(#)

pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

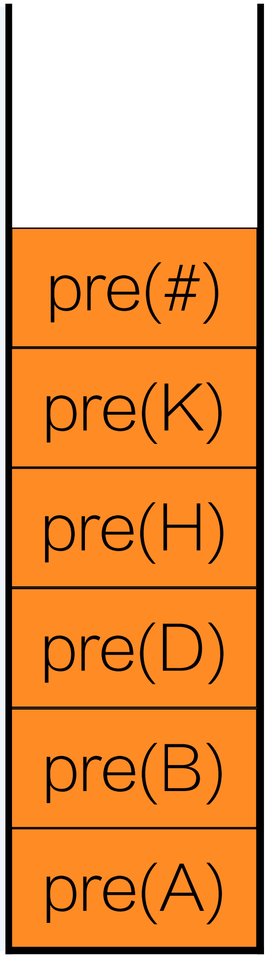
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



return

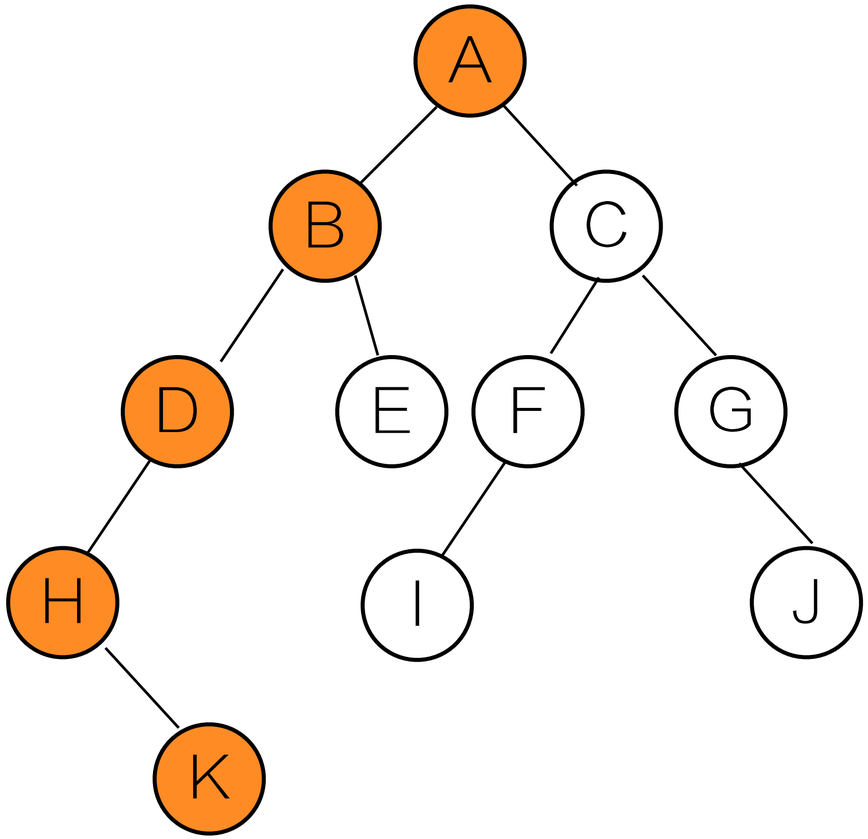
pre(#) pre(#)

pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

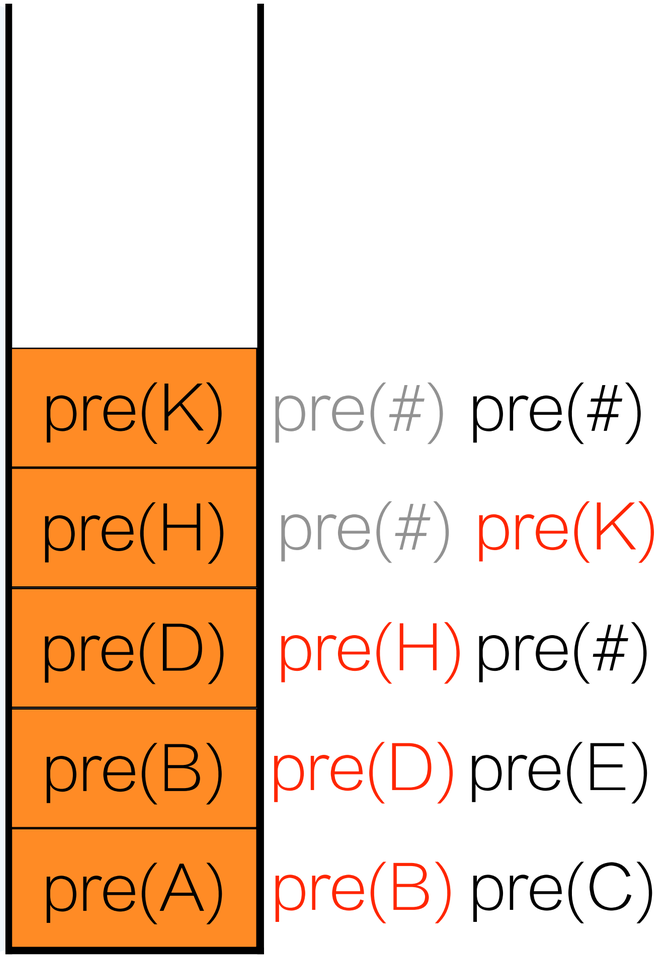
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



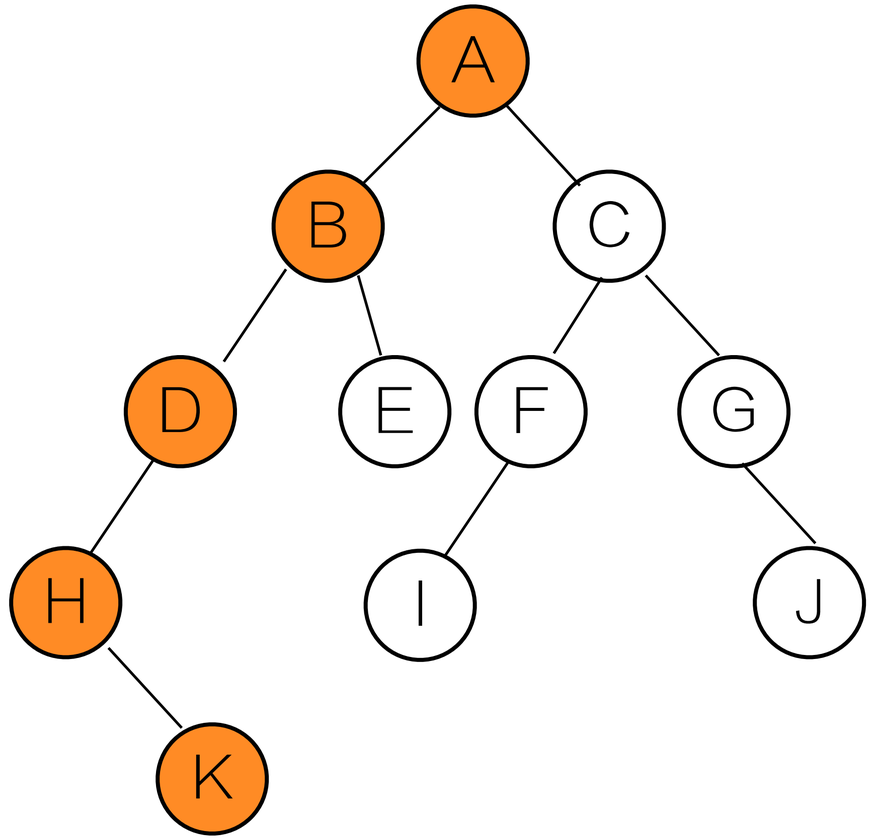
pre(#) pre(#)

pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

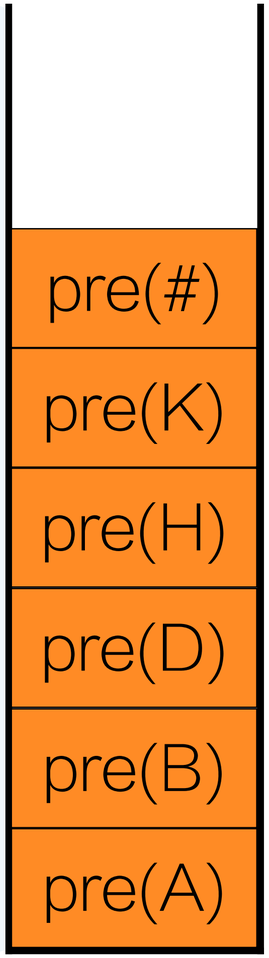
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



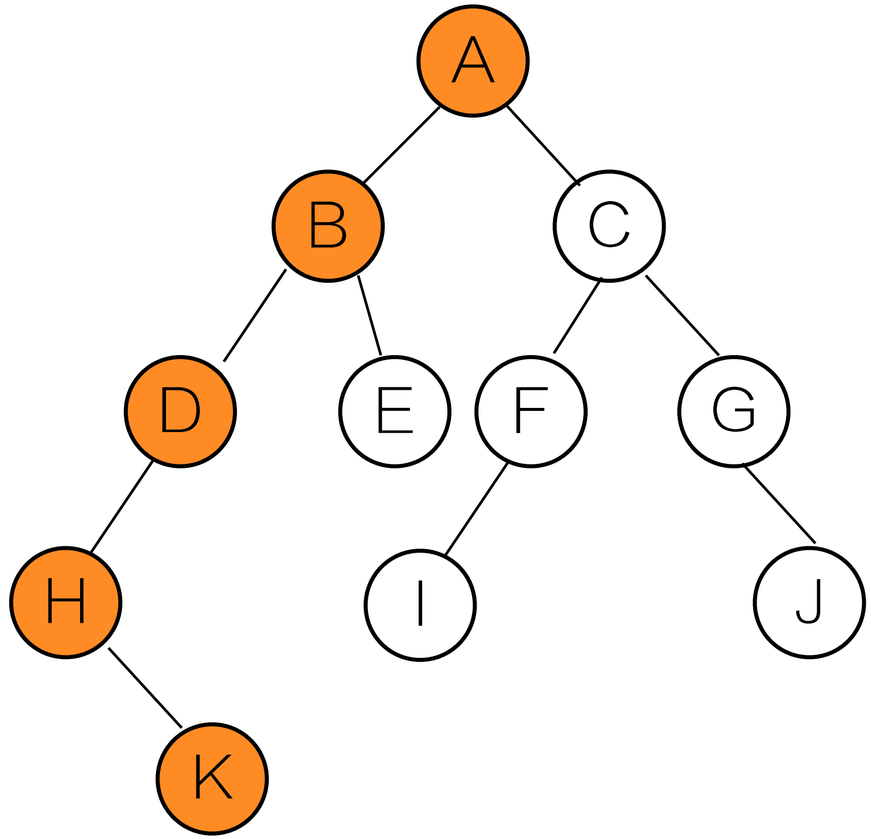
pre(#) pre(#)

pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

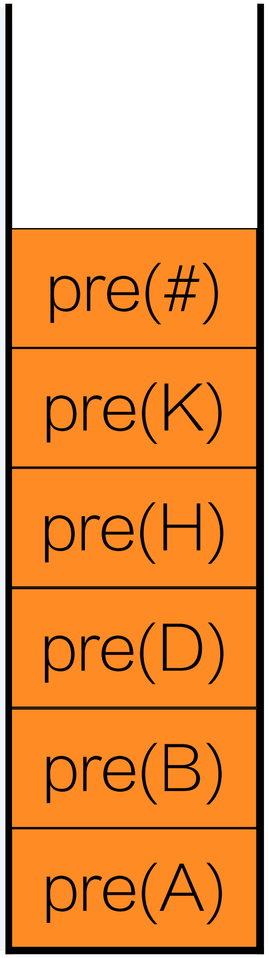
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



return

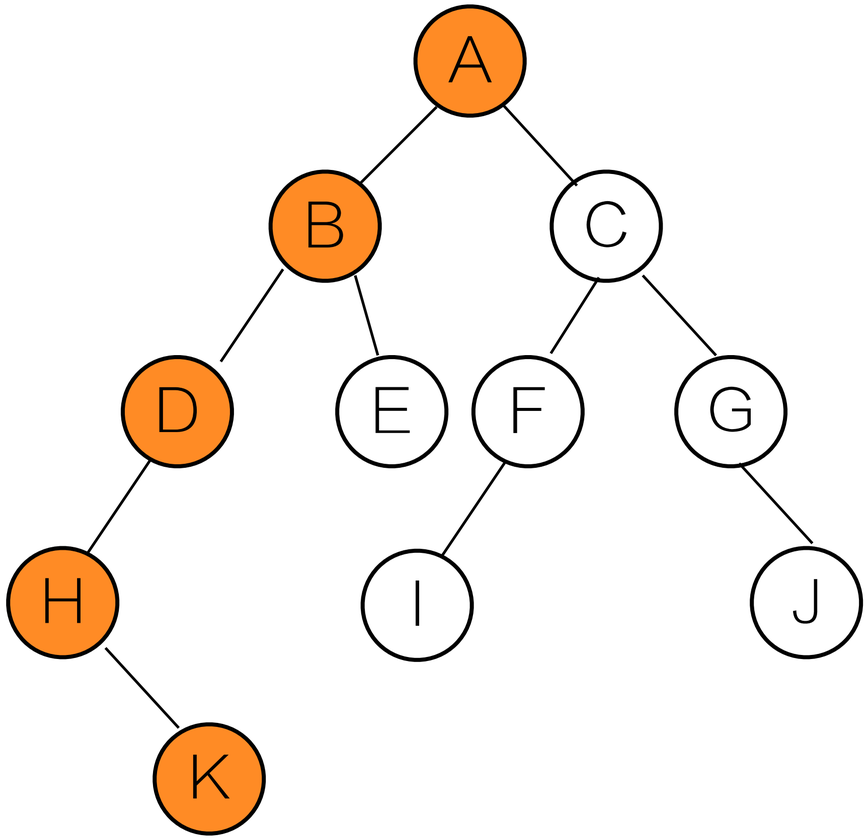
pre(#) pre(#)

pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

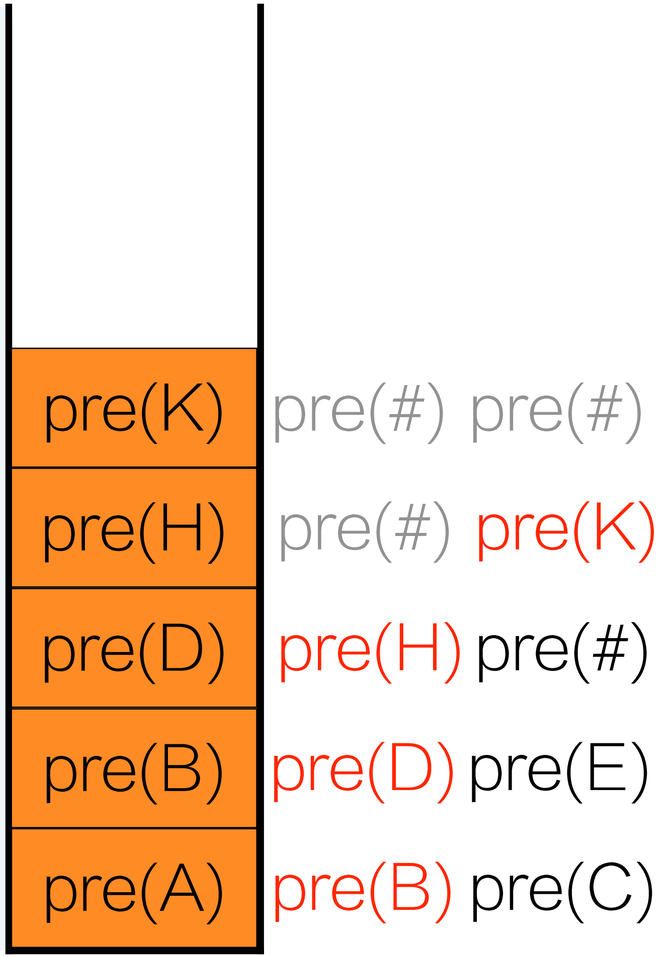
pre(B) pre(C)

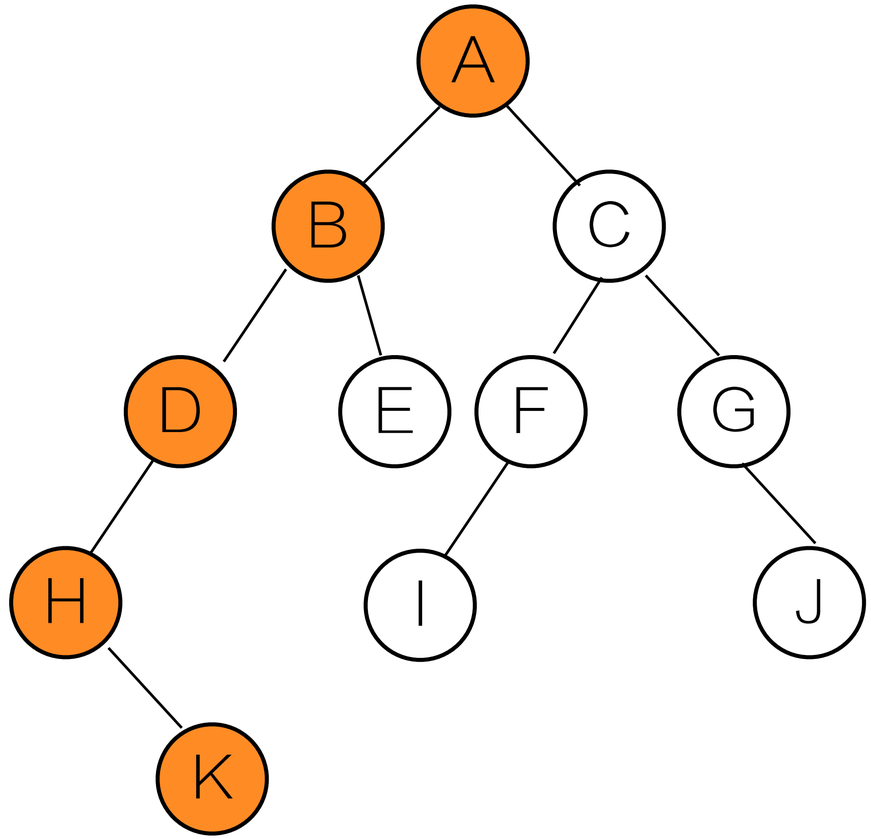


二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

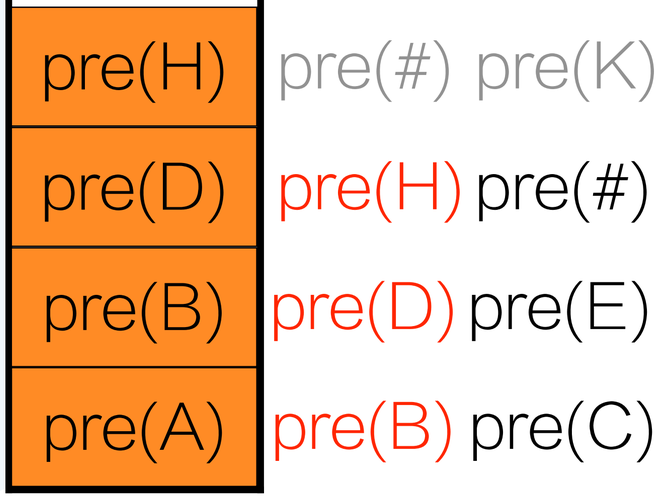




二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

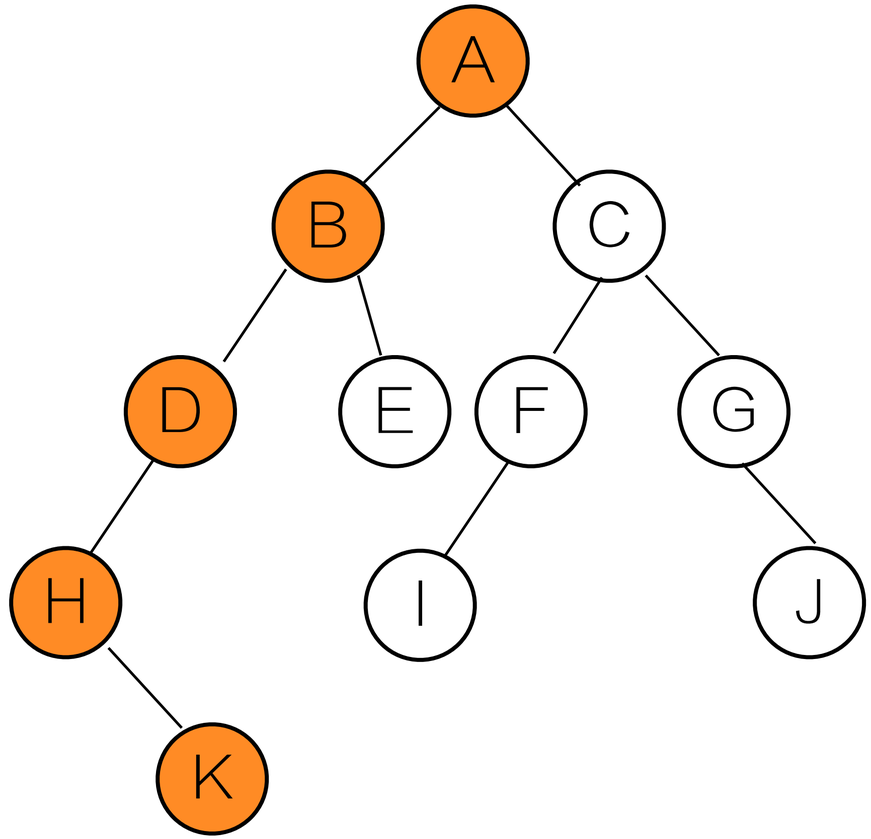


pre(#) pre(K)

pre(H) pre(#)

pre(D) pre(E)

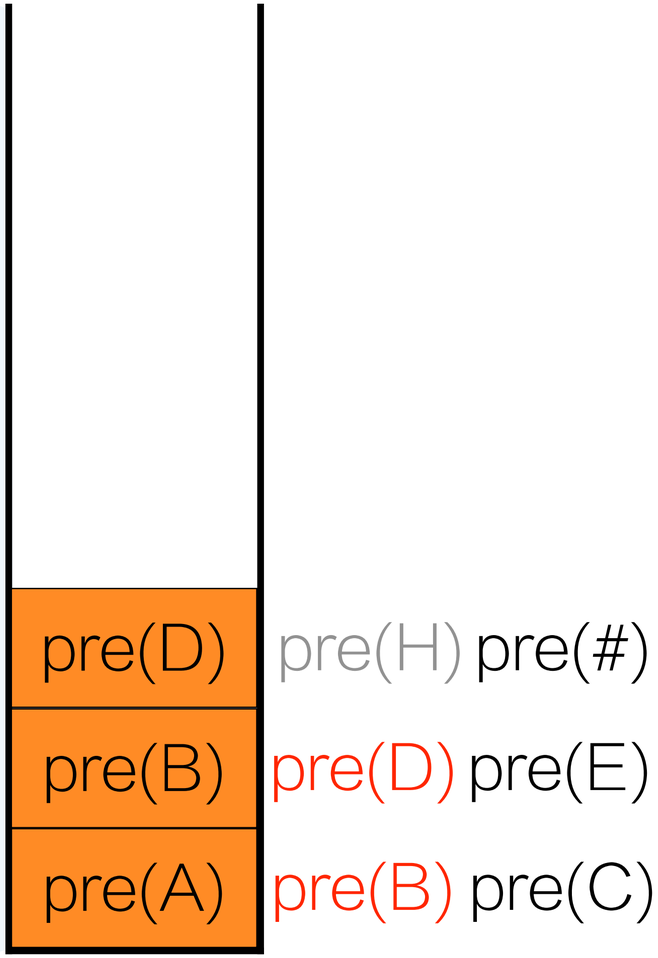
pre(B) pre(C)

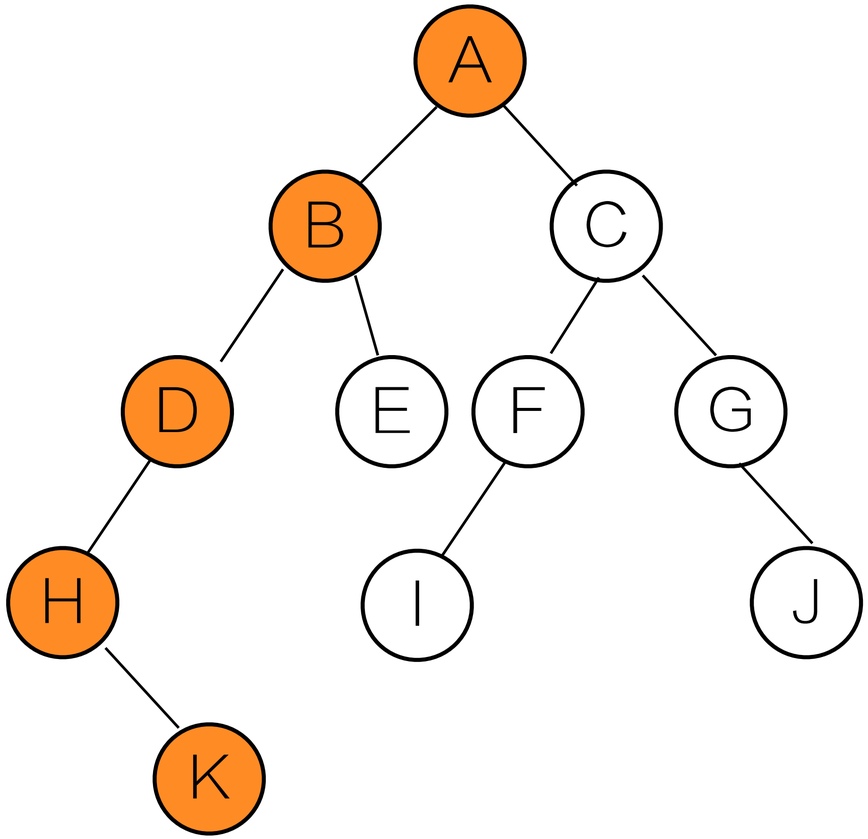


二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

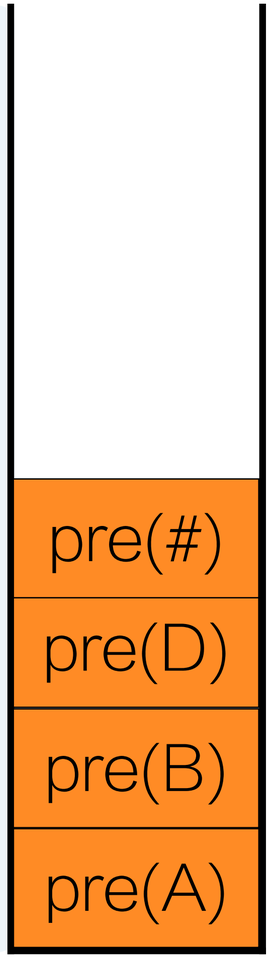




二叉树的遍历 – 前序遍历

输出内容：A B D H K

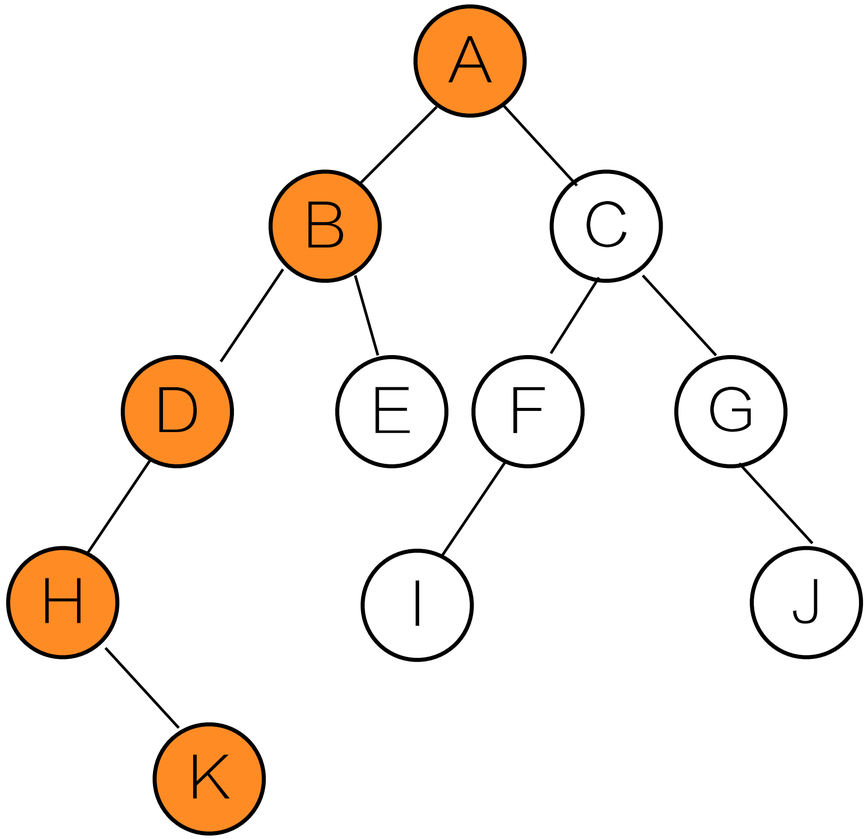
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(H) pre(#)

pre(D) pre(E)

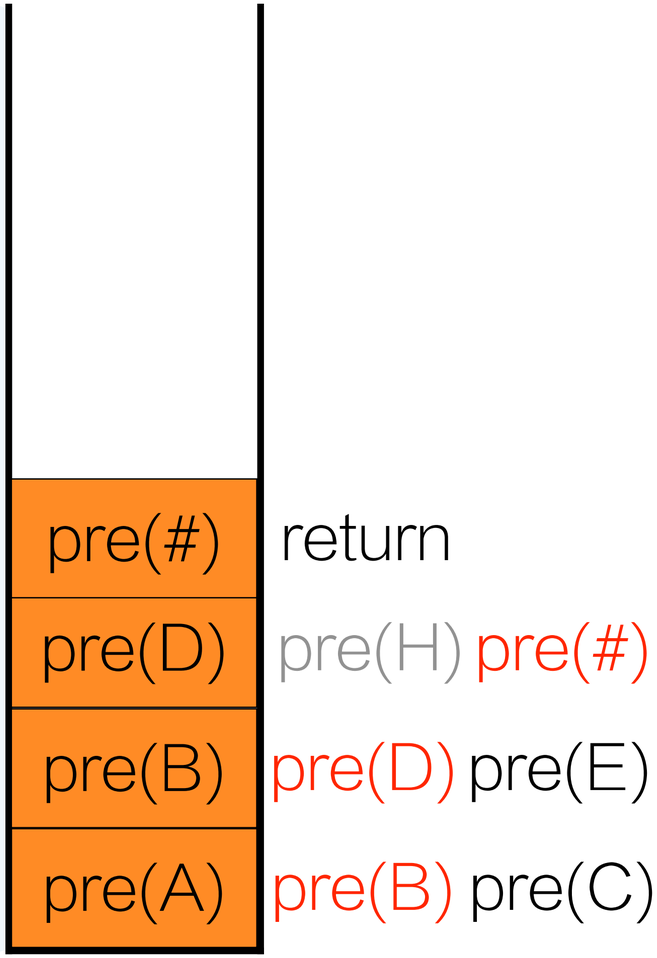
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

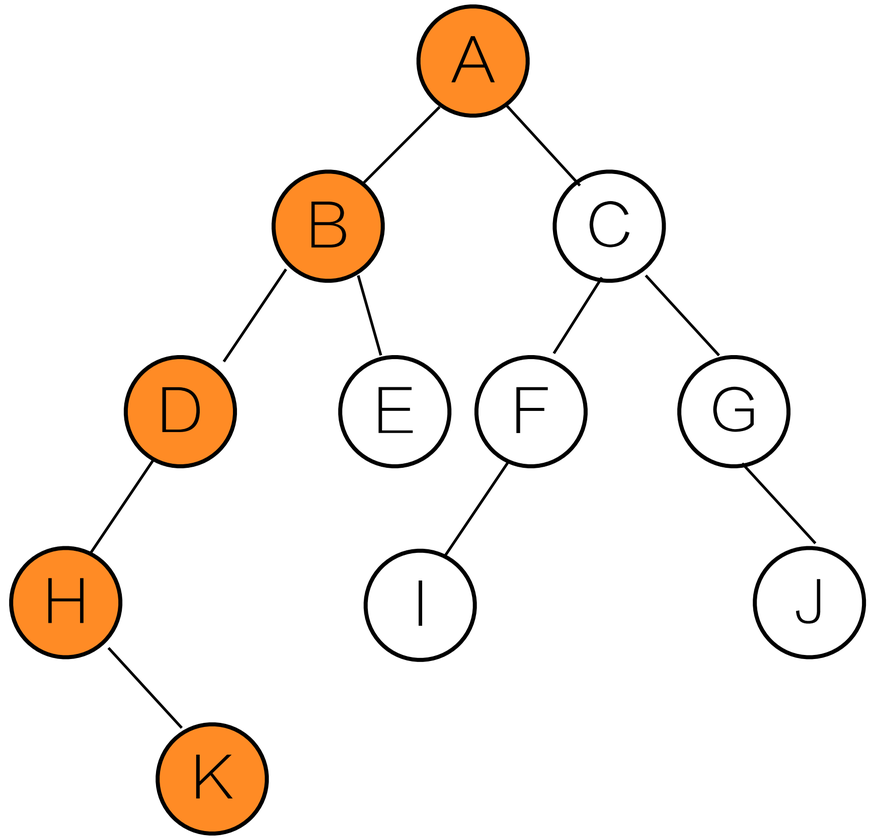


return

pre(H) pre(#)

pre(D) pre(E)

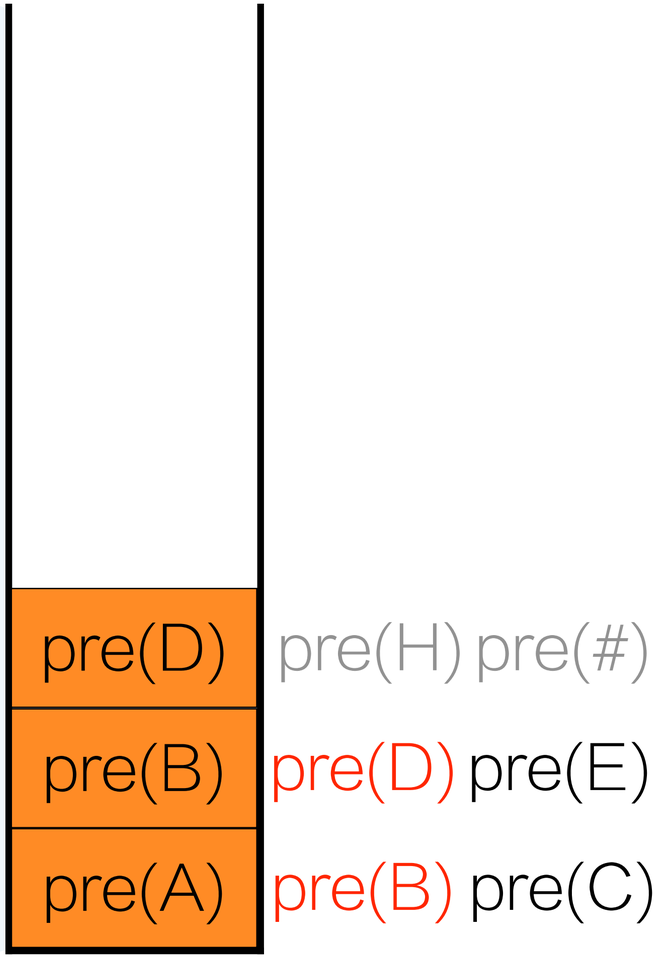
pre(B) pre(C)

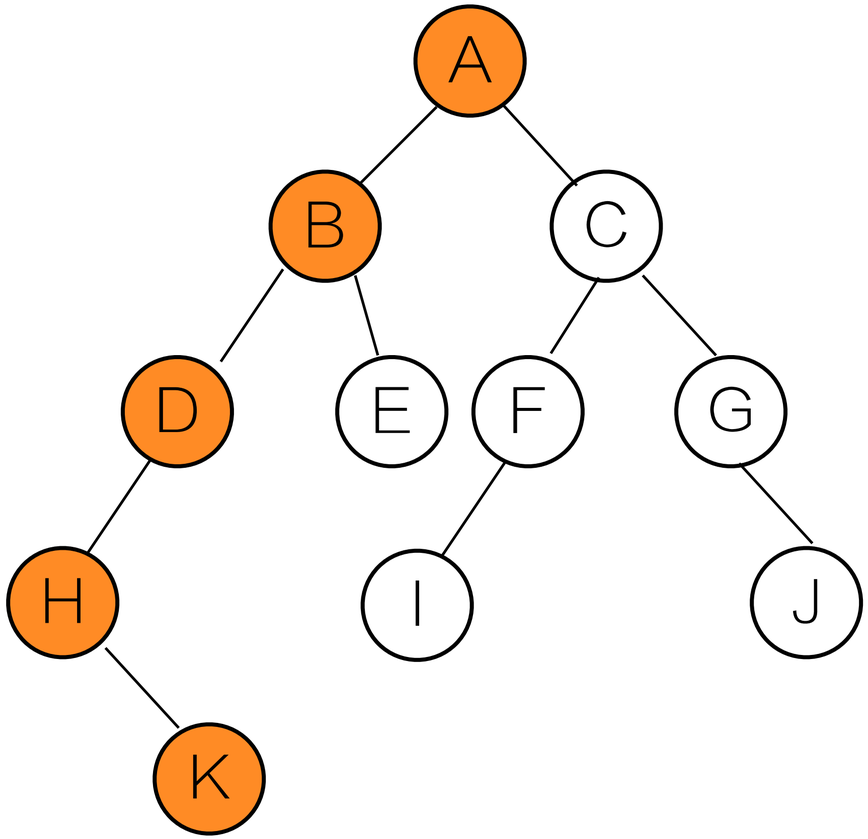


二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

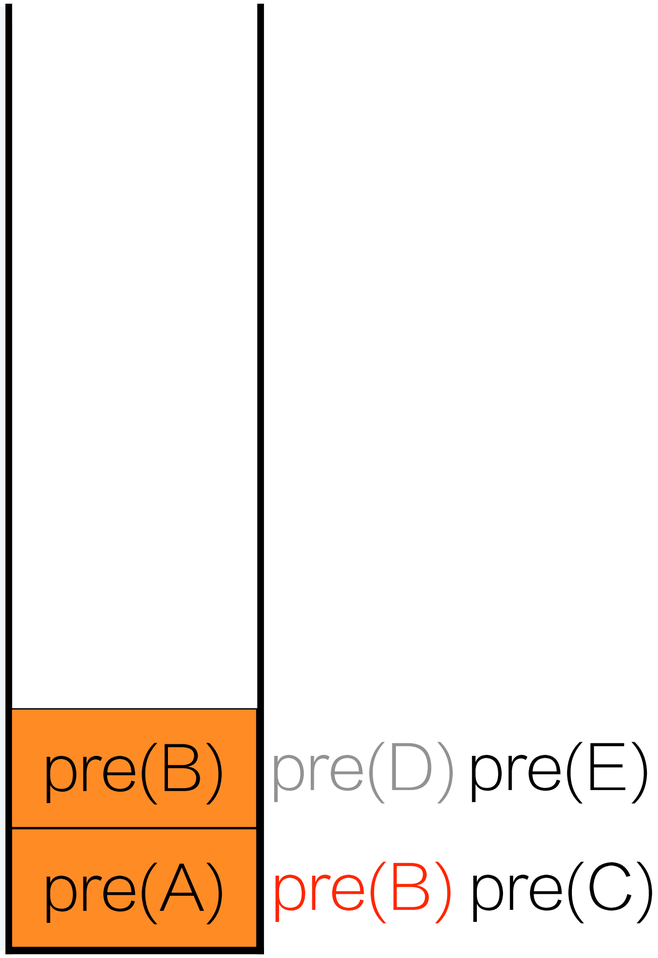


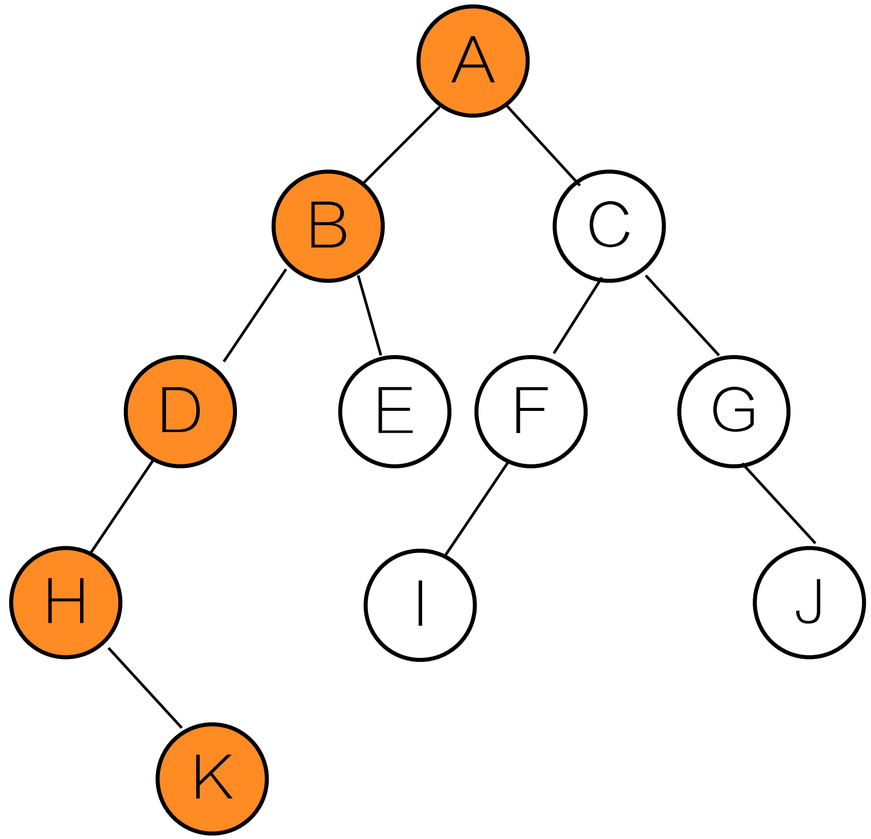


二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

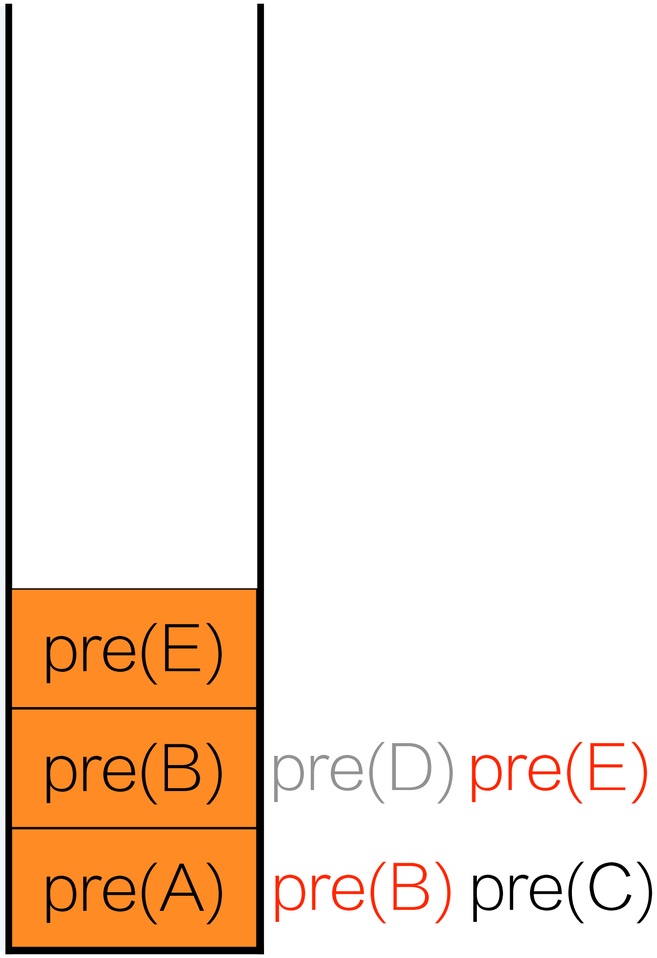


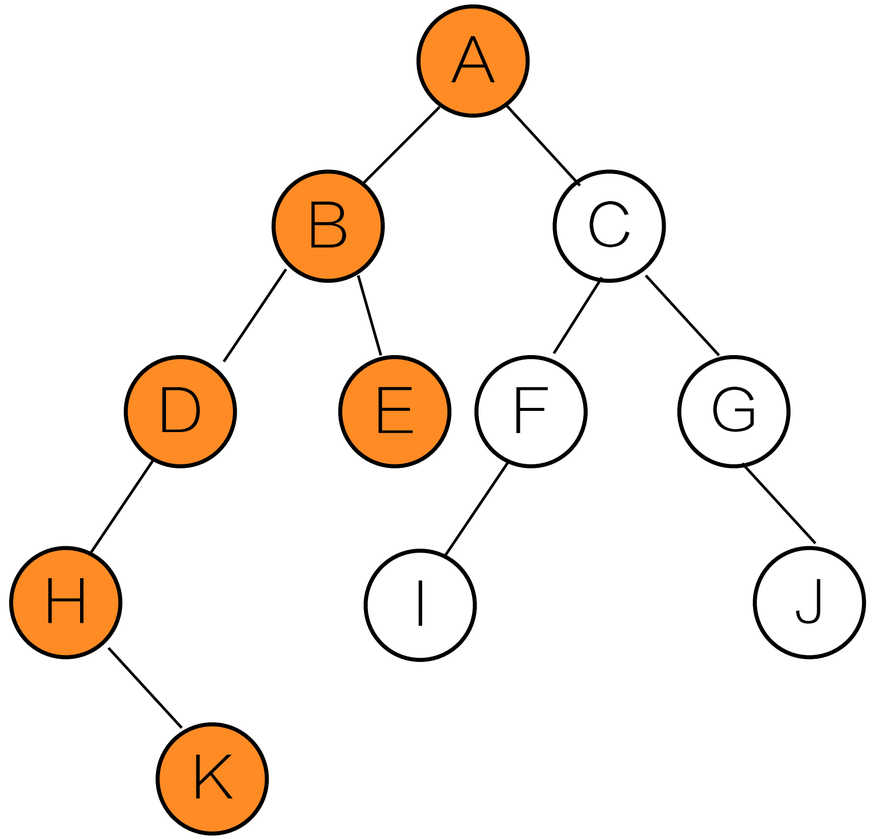


二叉树的遍历 – 前序遍历

输出内容：A B D H K

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

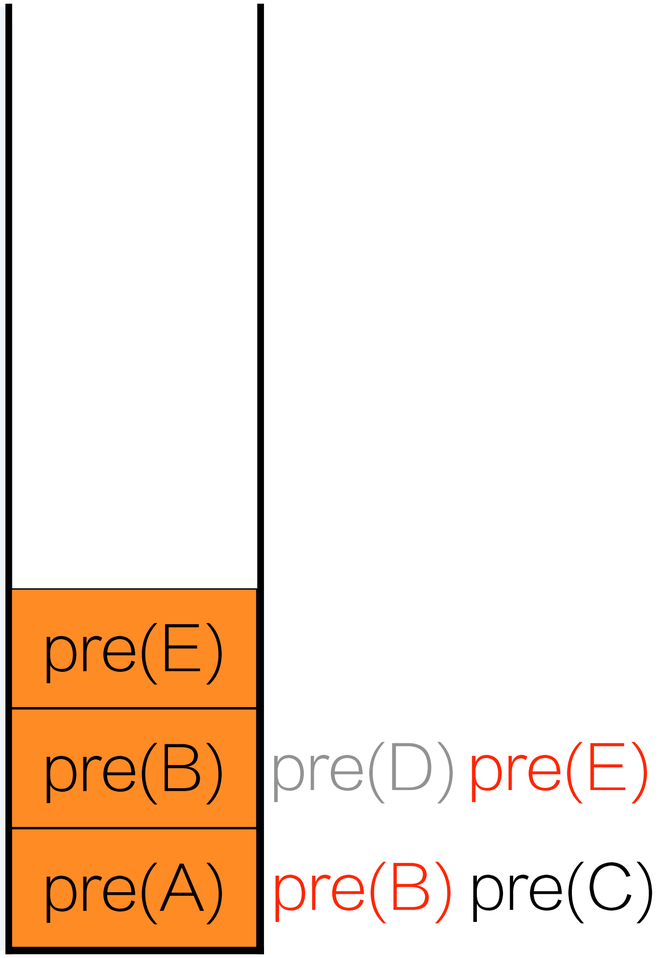


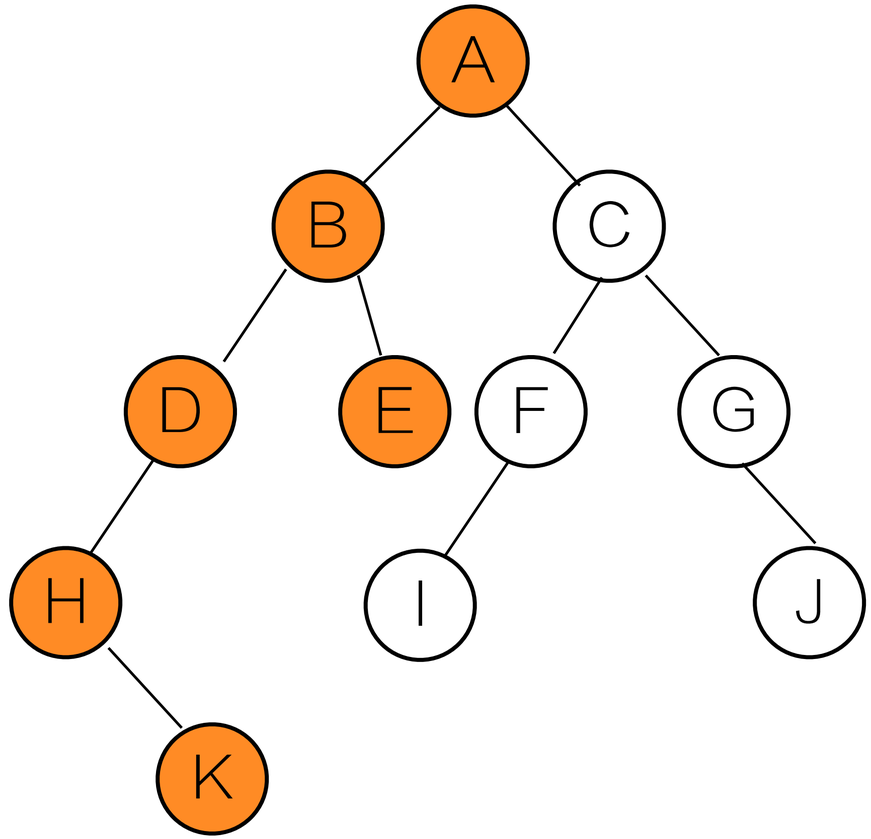


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

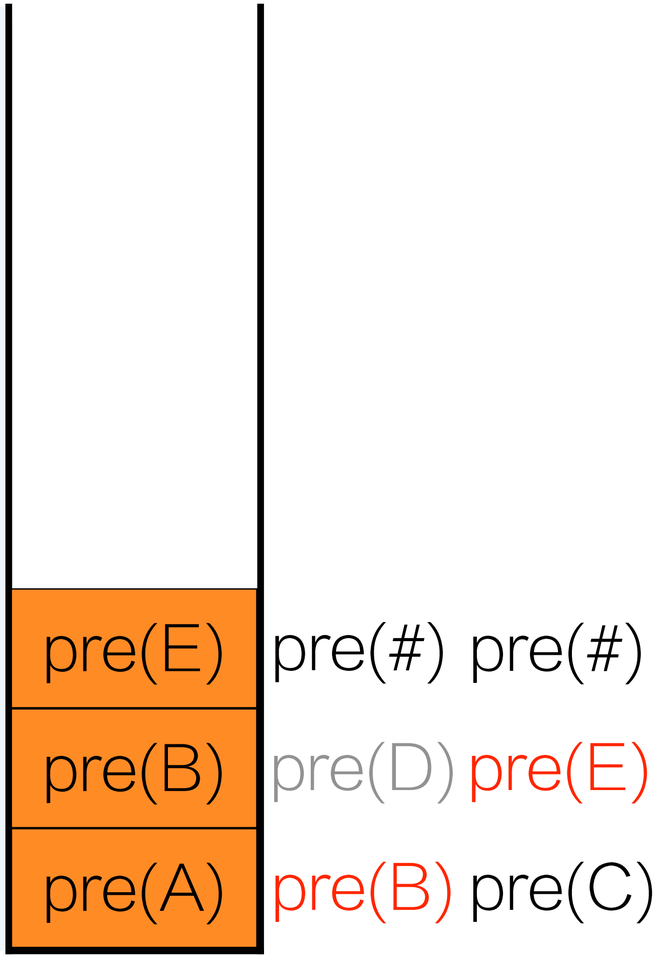


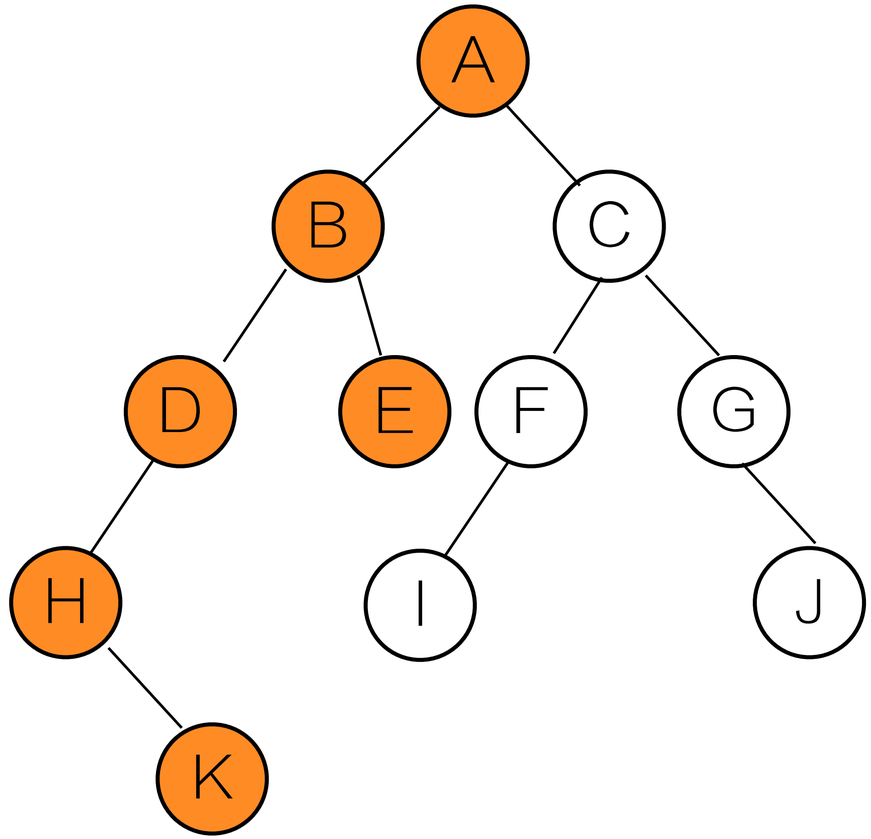


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

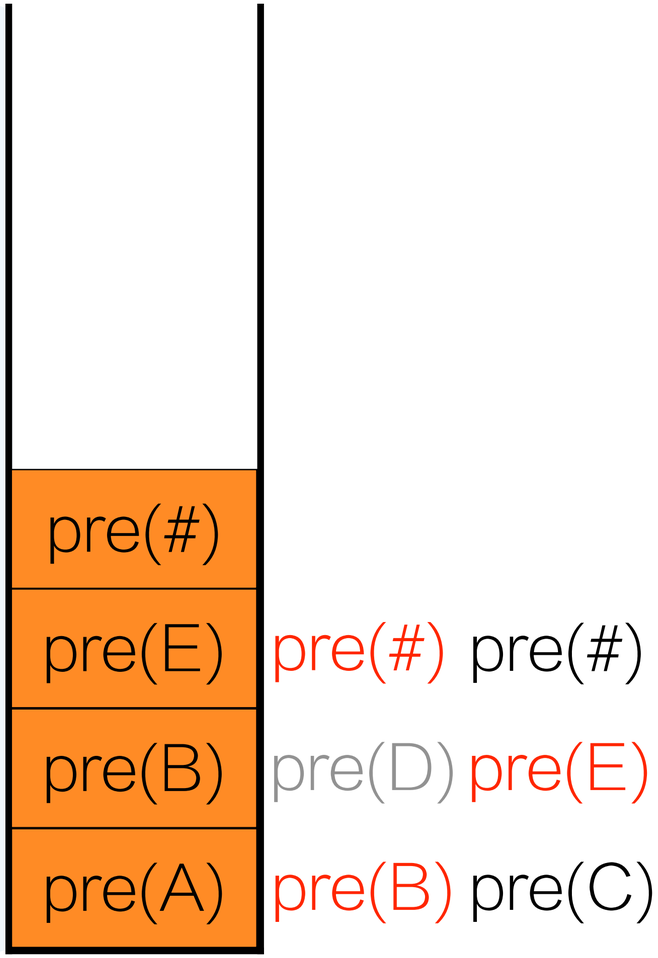


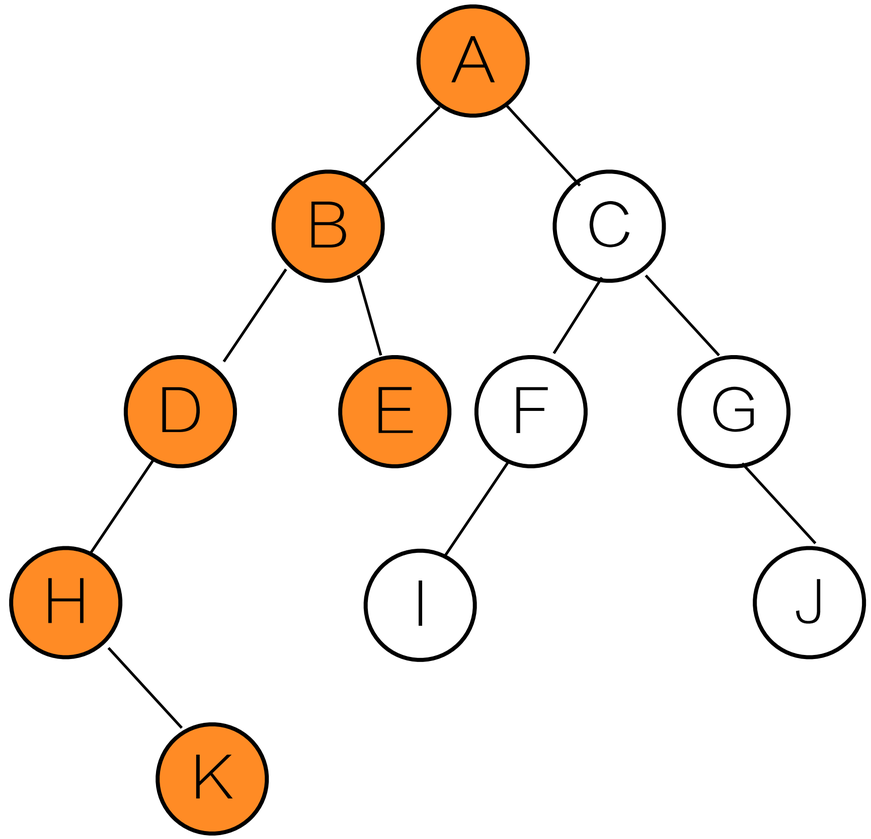


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

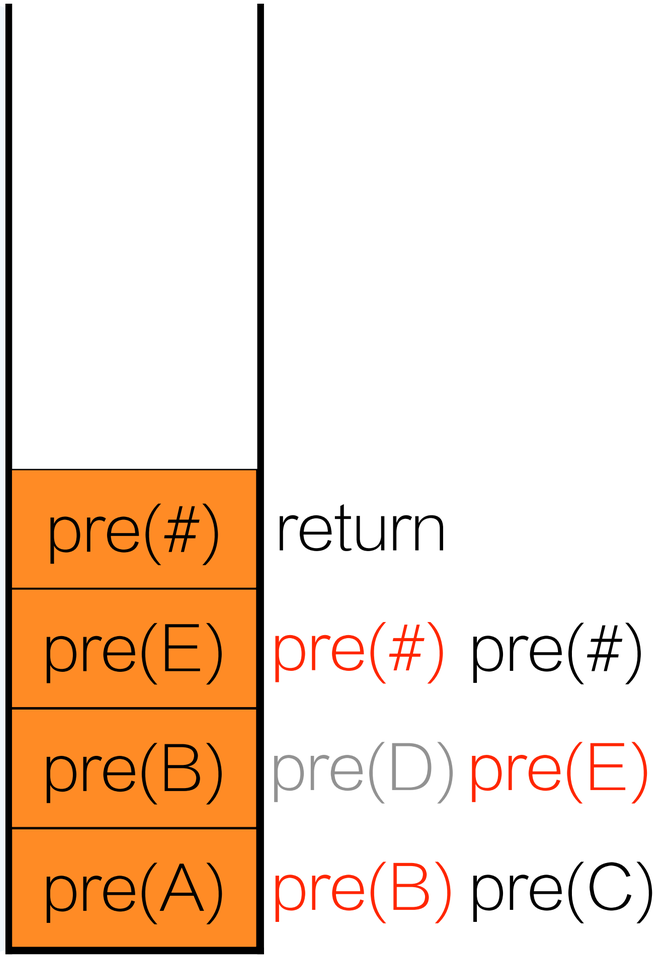




二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

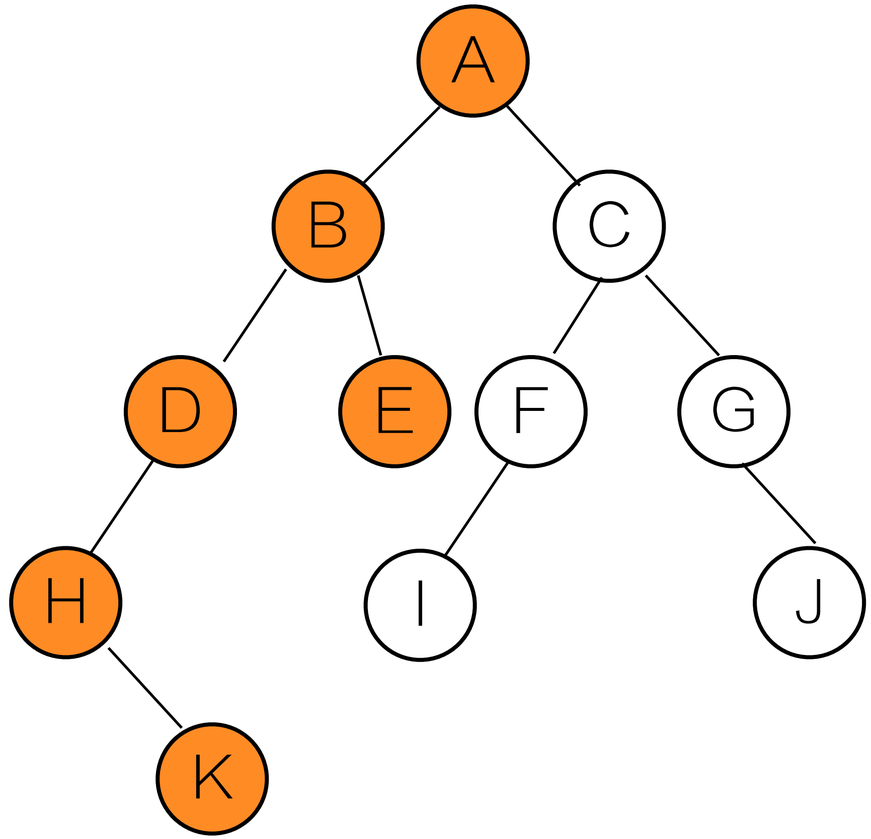


return

pre(#) pre(#)

pre(D) pre(E)

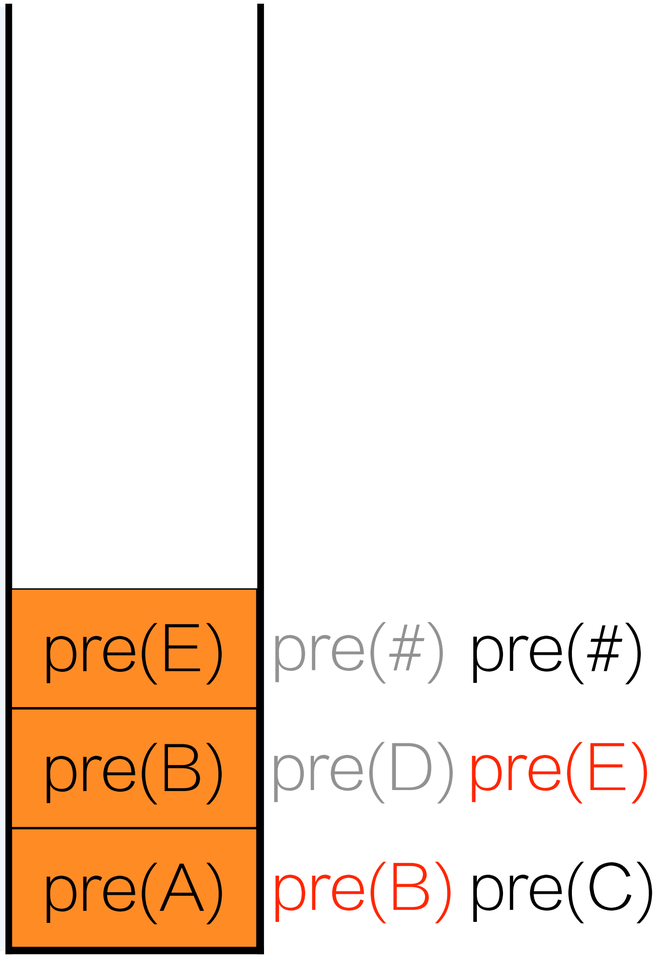
pre(B) pre(C)

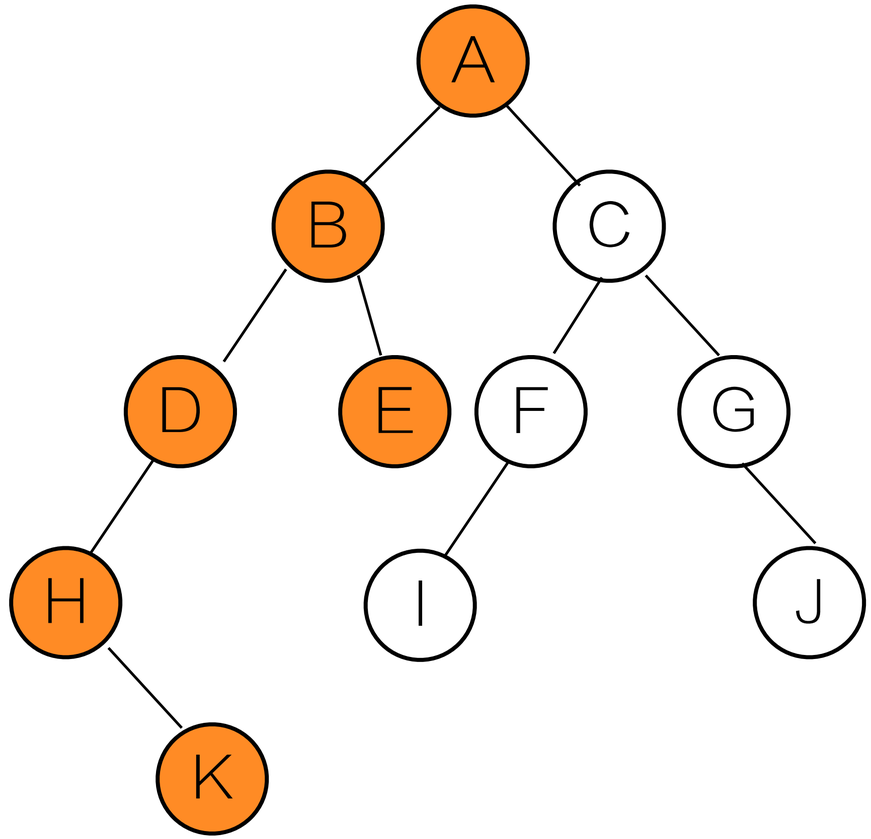


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

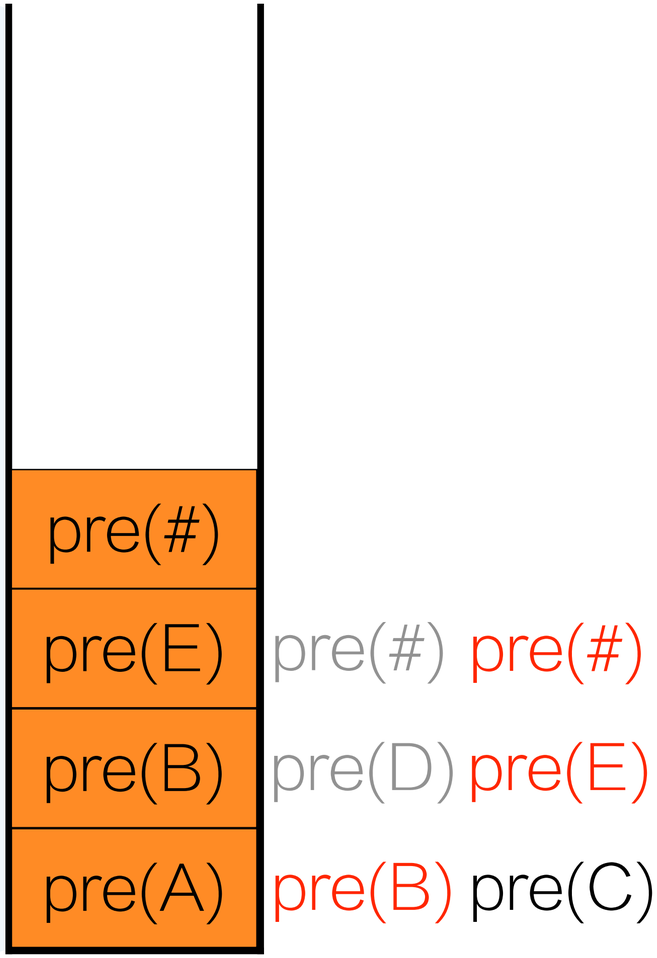


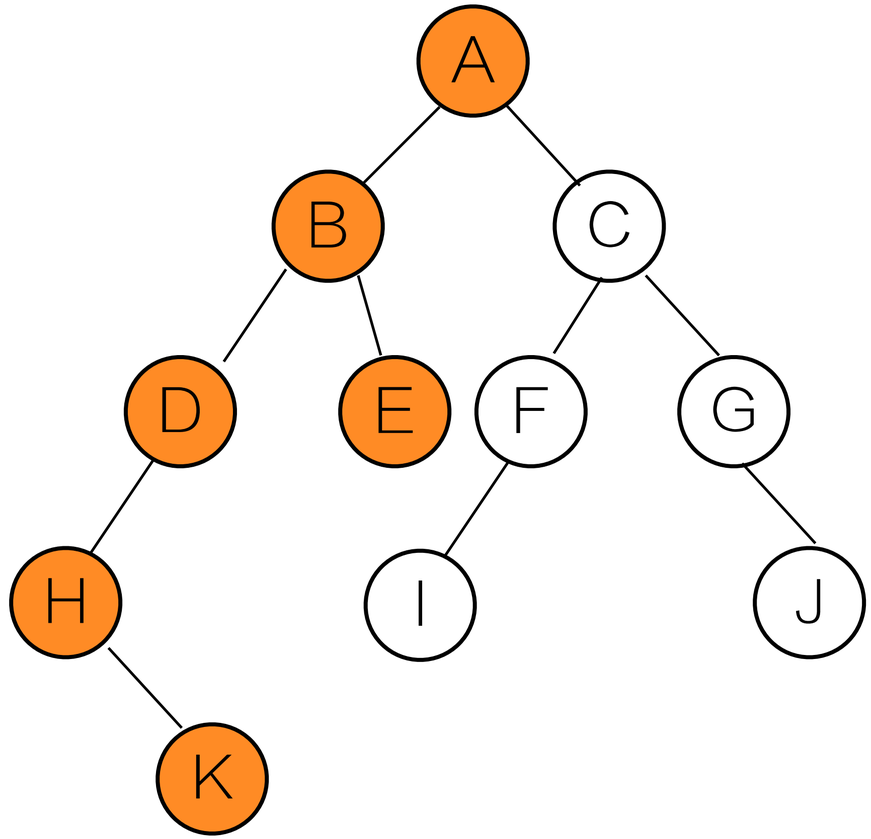


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

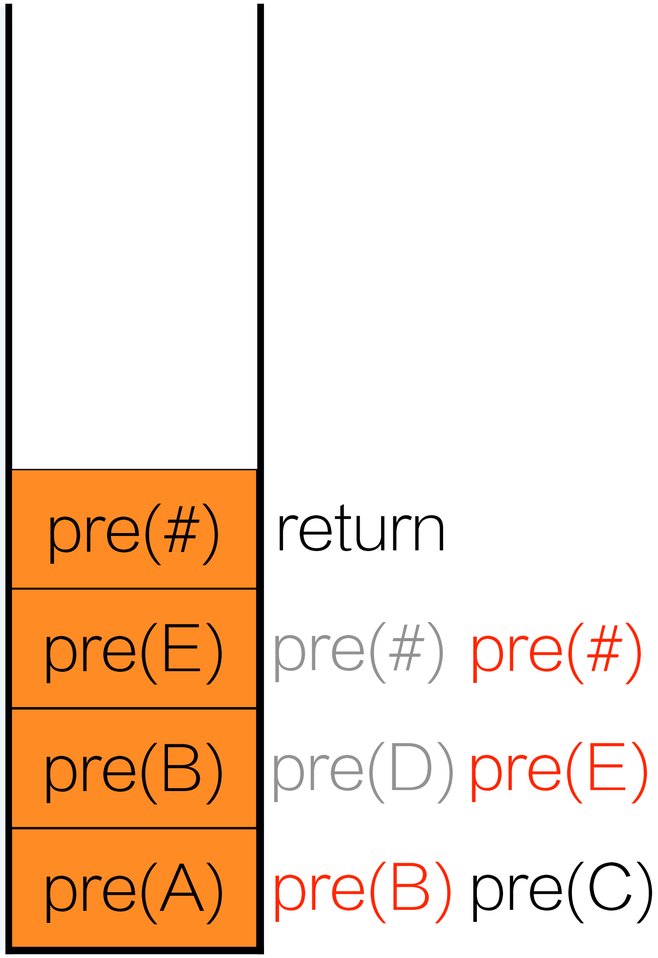




二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

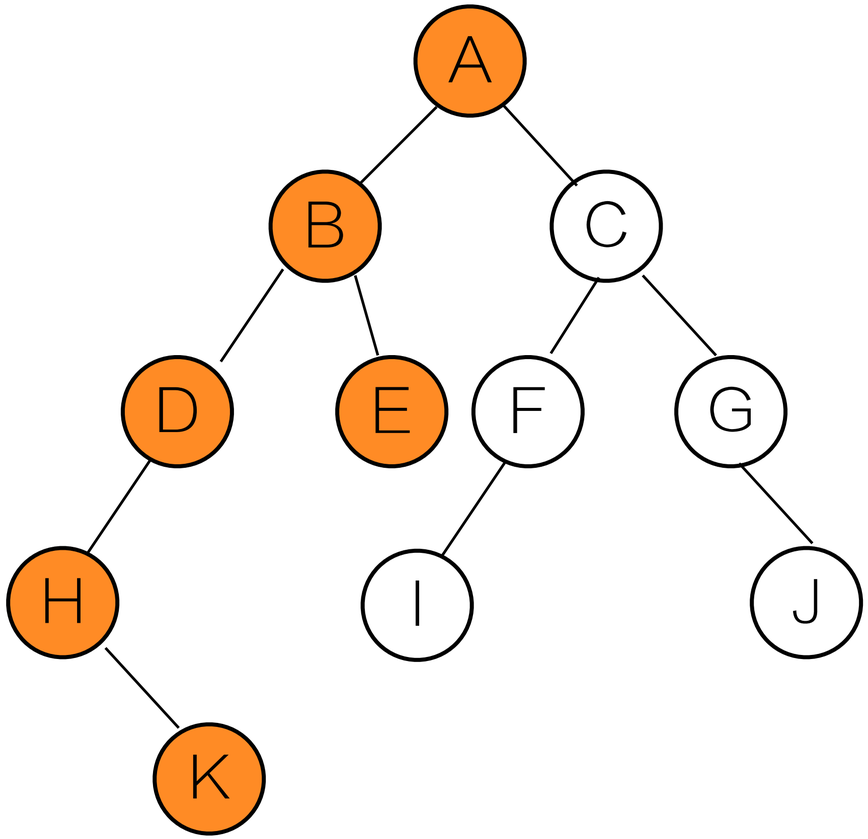


return

pre(#) pre(#)

pre(D) pre(E)

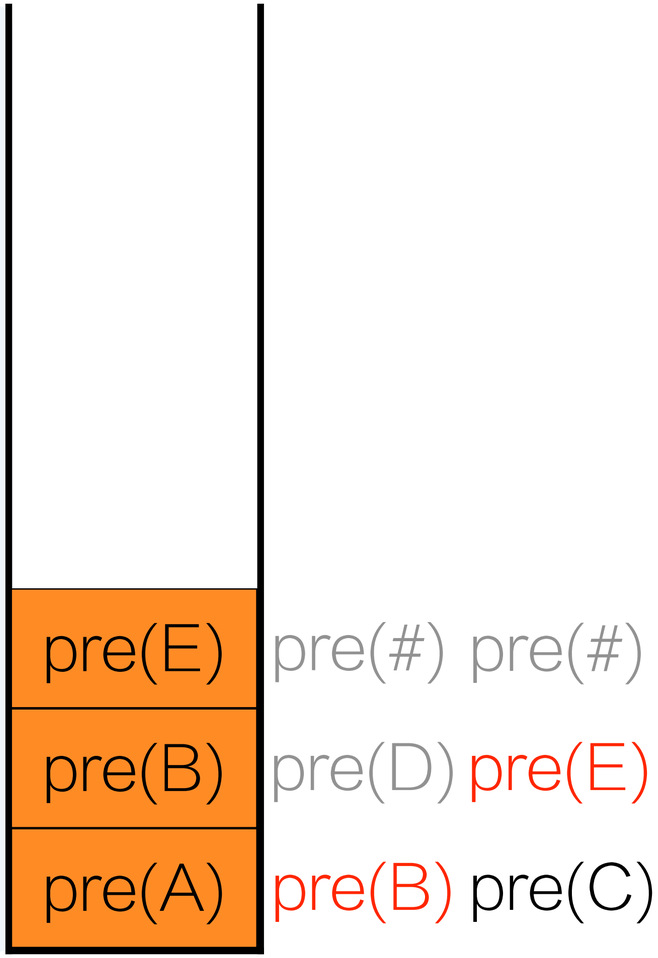
pre(B) pre(C)

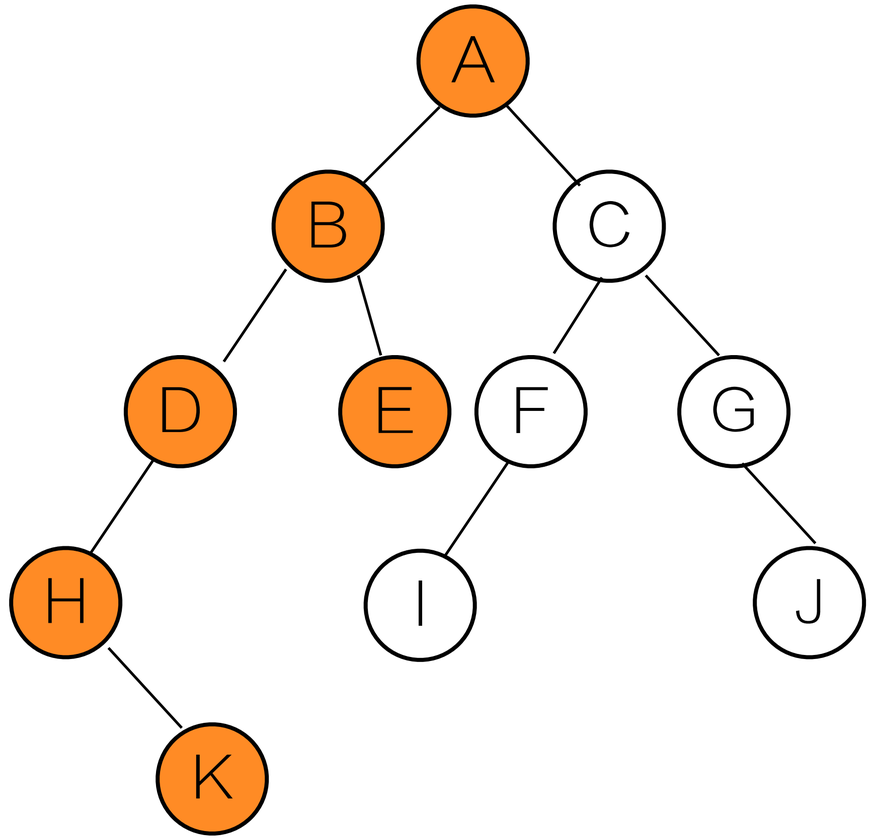


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

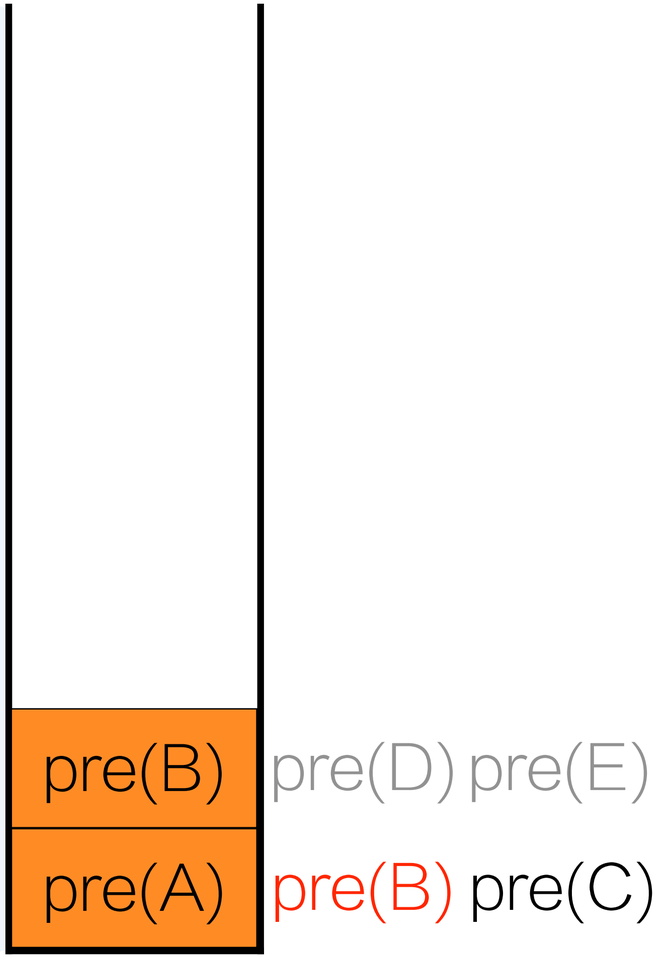


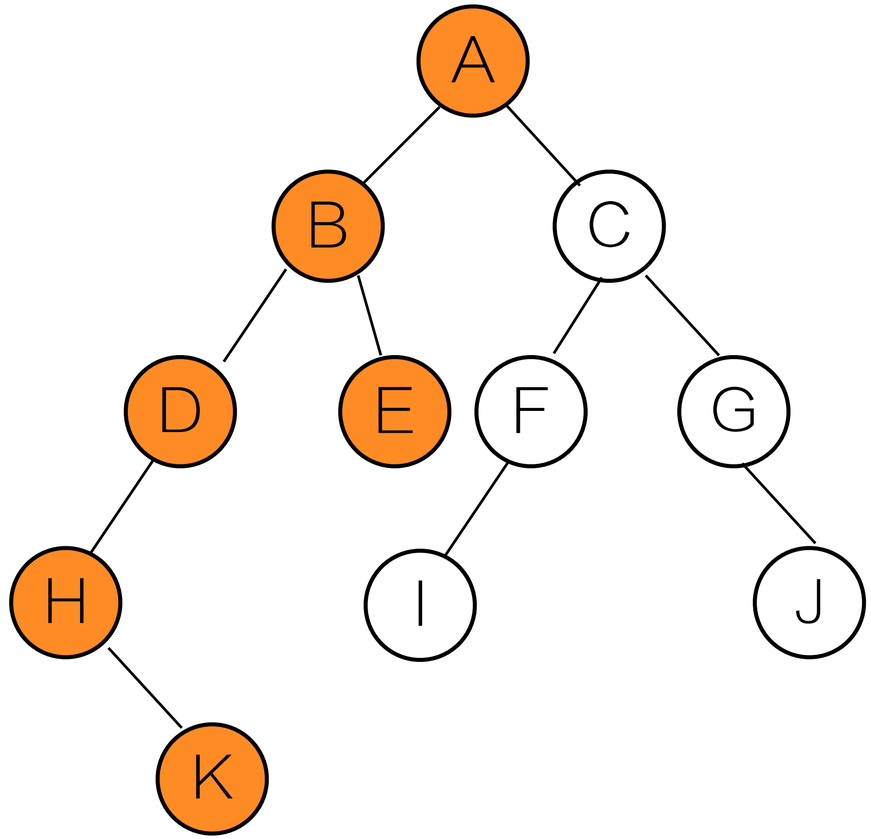


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



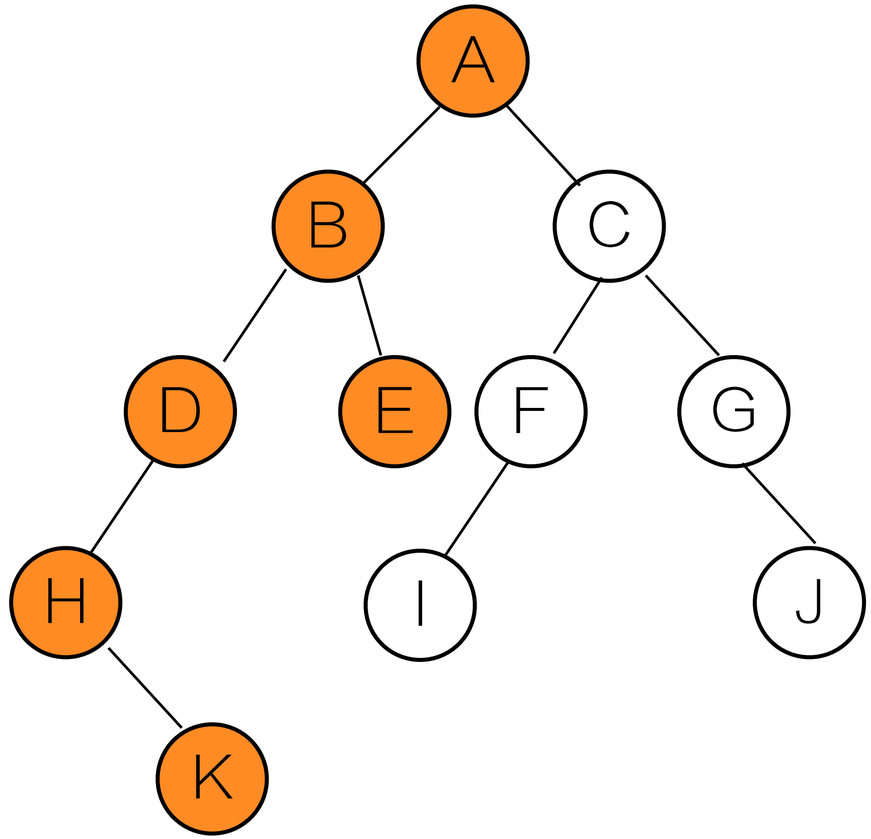


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(A) pre(B) pre(C)

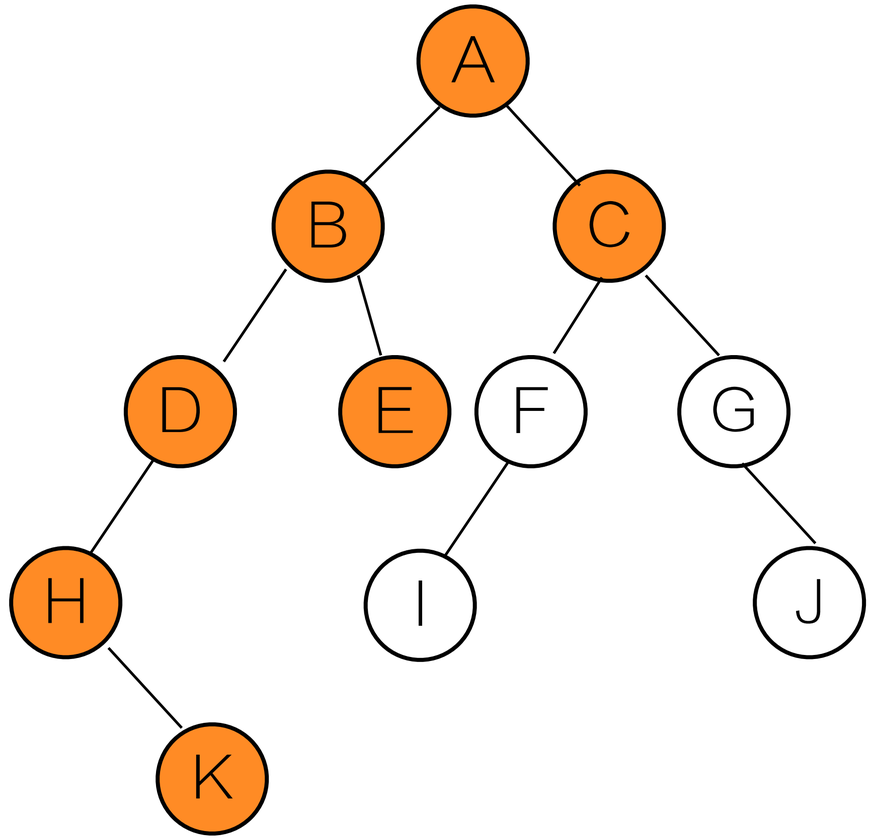


二叉树的遍历 – 前序遍历

输出内容：A B D H K E

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(C)
pre(A) pre(B) pre(C)

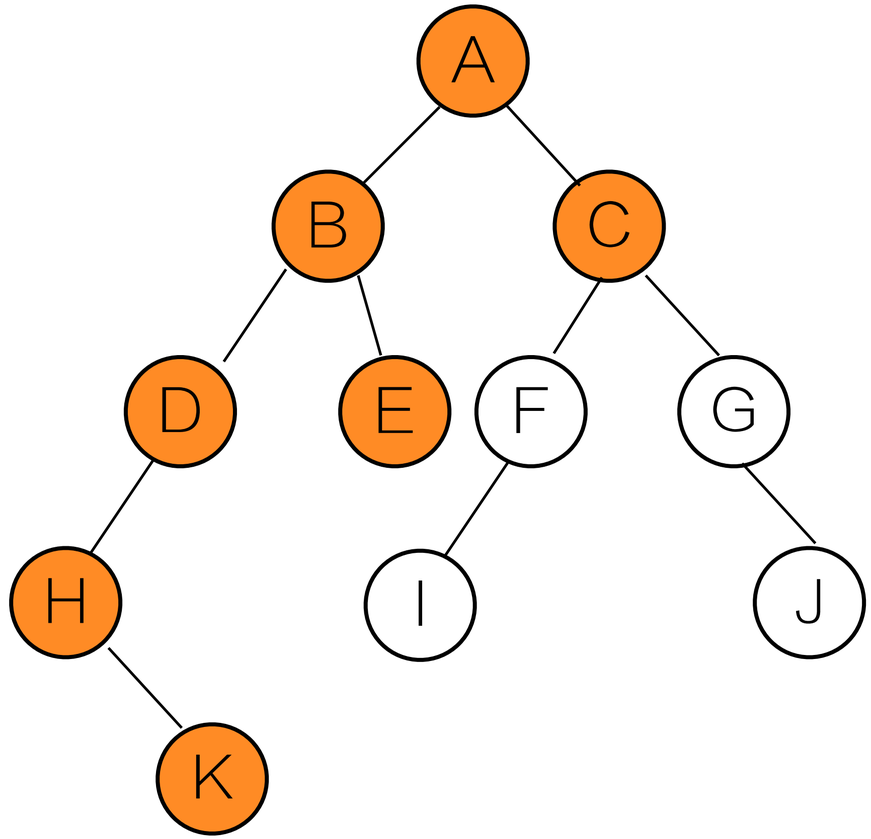


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(C)
pre(A) pre(B) pre(C)



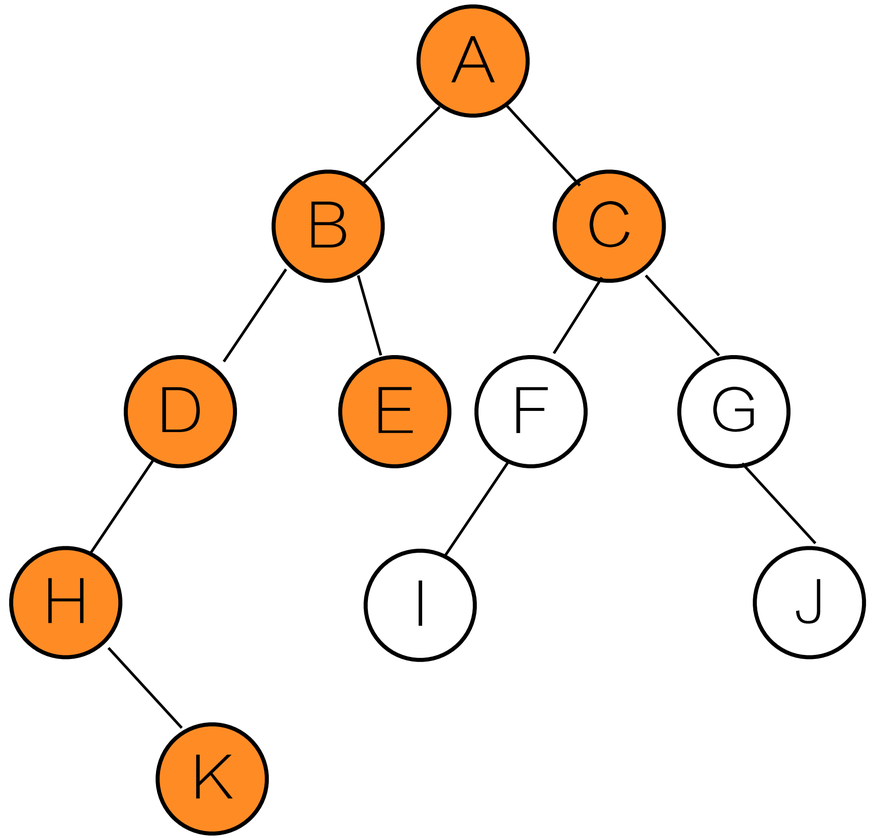
二叉树的遍历 – 前序遍历

输出内容：A B D H K E C

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(C) pre(F) pre(G)

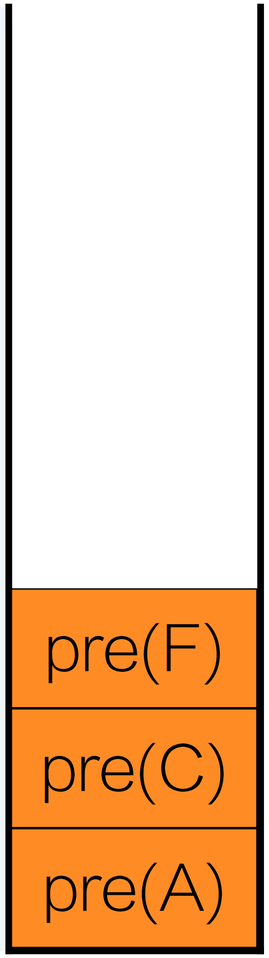
pre(A) pre(B) pre(C)



二叉树的遍历 – 前序遍历

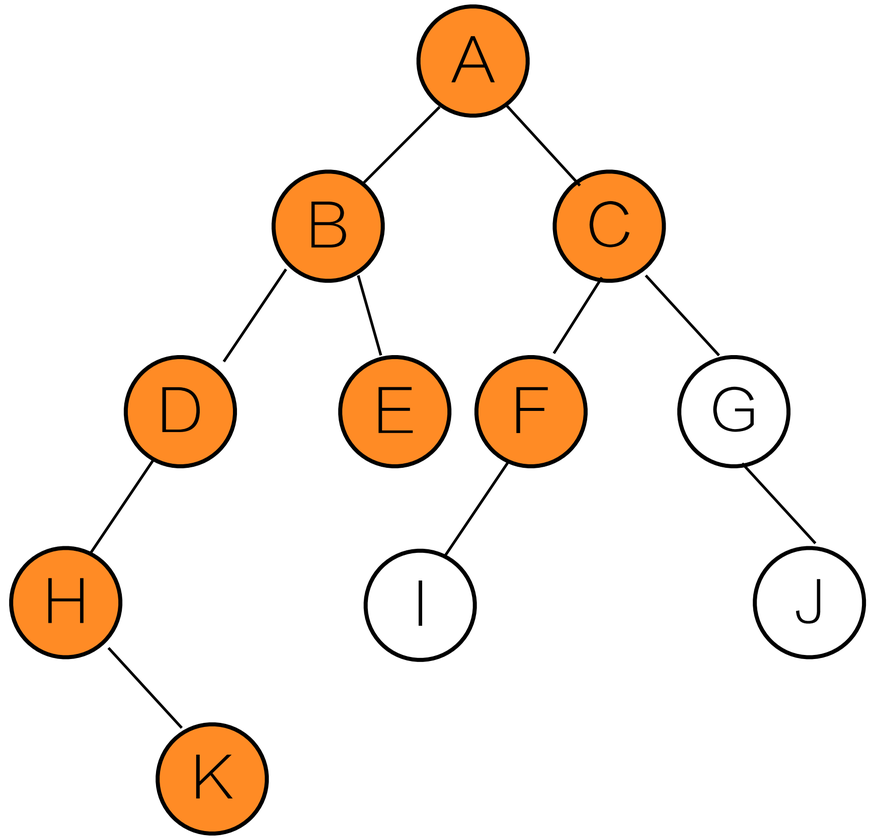
输出内容：A B D H K E C

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(F) pre(G)

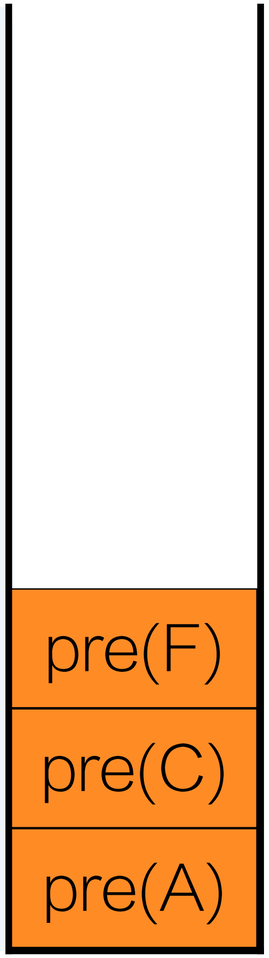
pre(B) pre(C)



二叉树的遍历 – 前序遍历

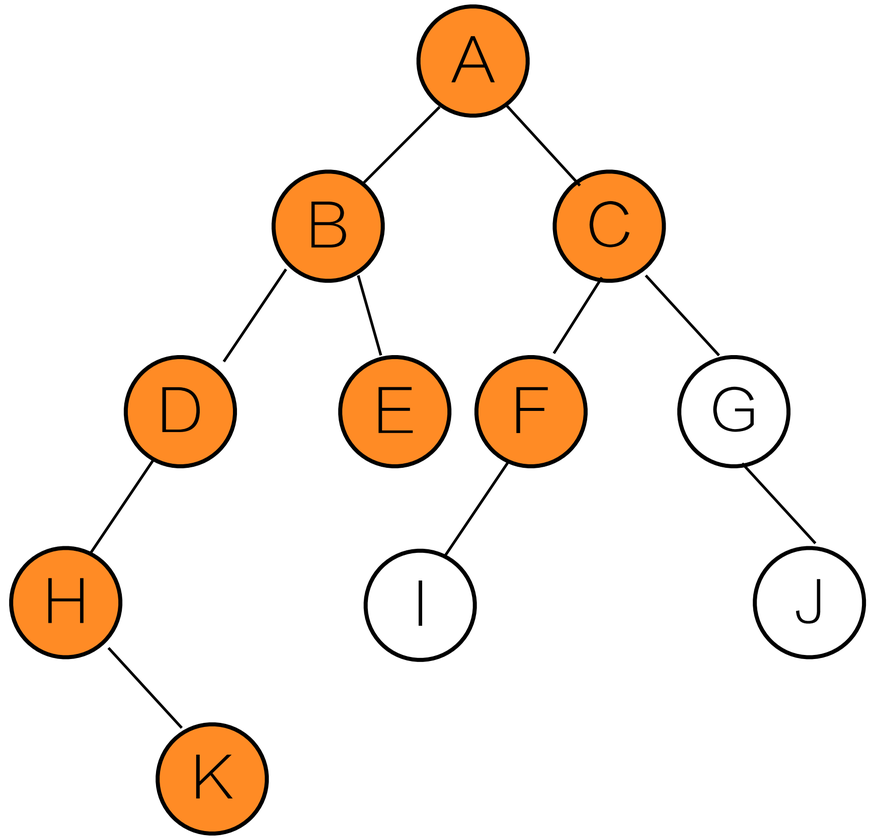
输出内容：A B D H K E C F

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(F) pre(G)

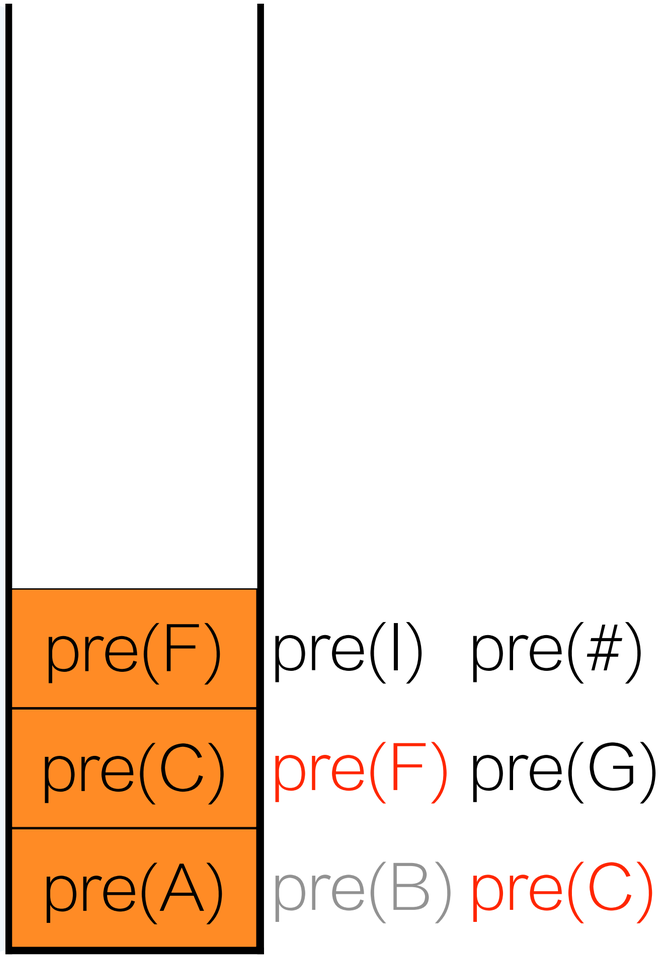
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F

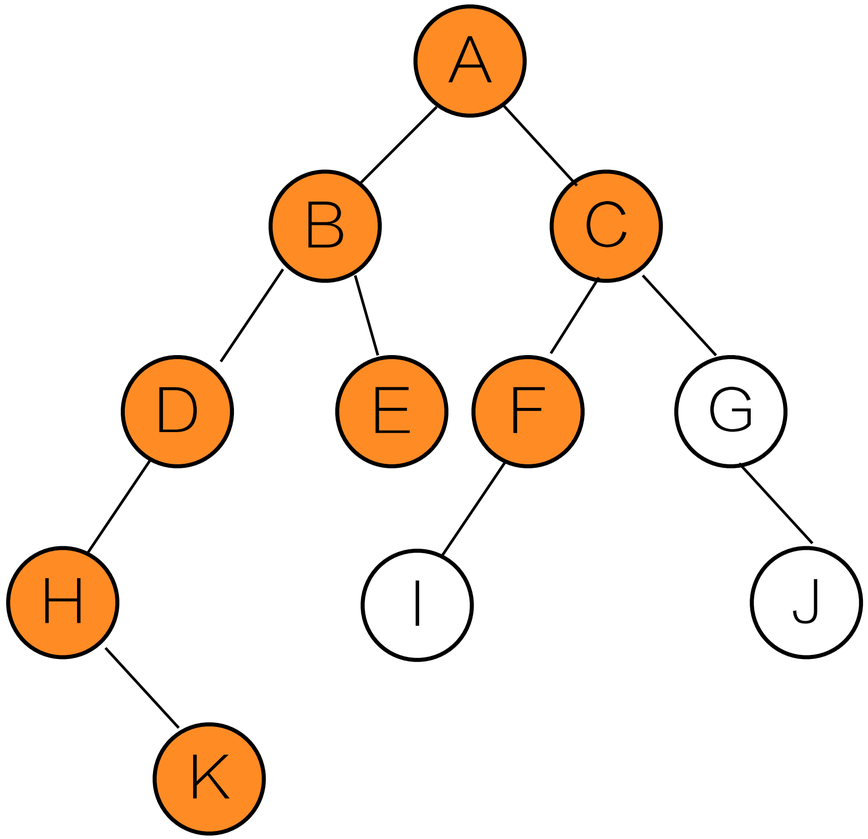
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(I) pre(#)

pre(F) pre(G)

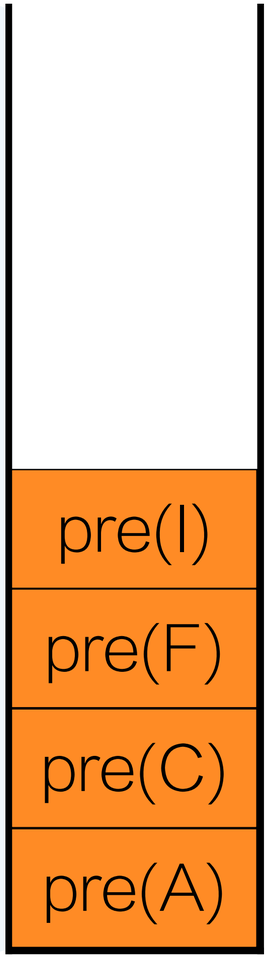
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F

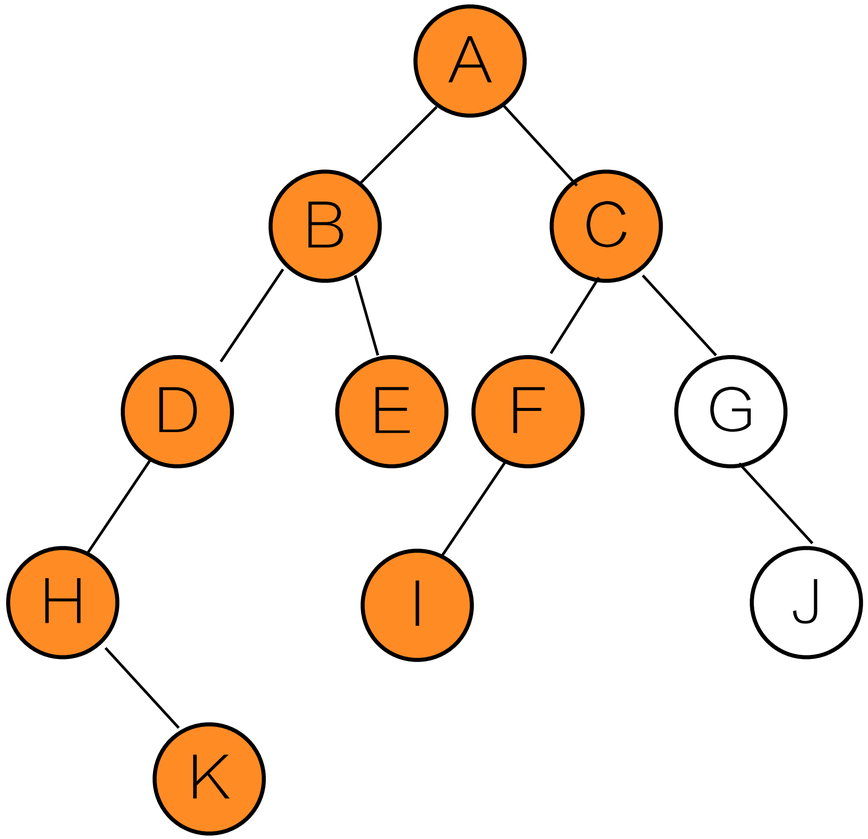
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(I) pre(#)

pre(F) pre(G)

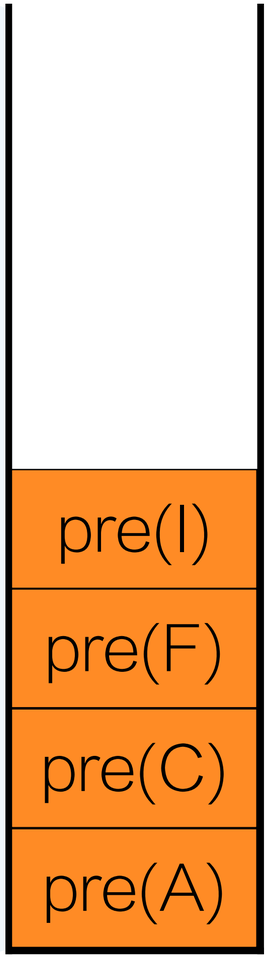
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

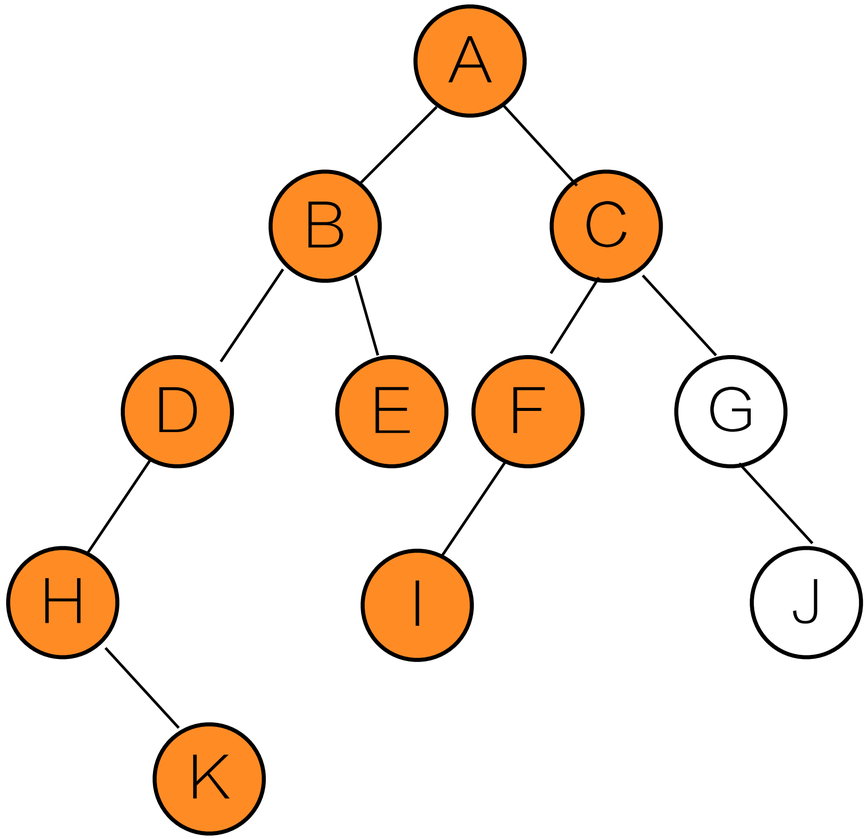
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(I) pre(#)

pre(F) pre(G)

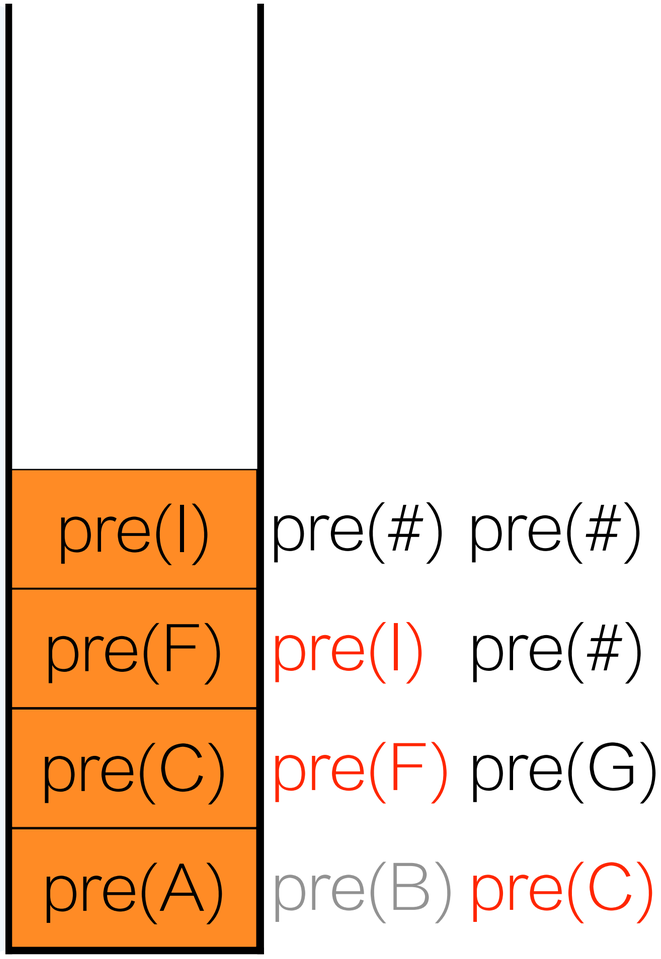
pre(B) pre(C)

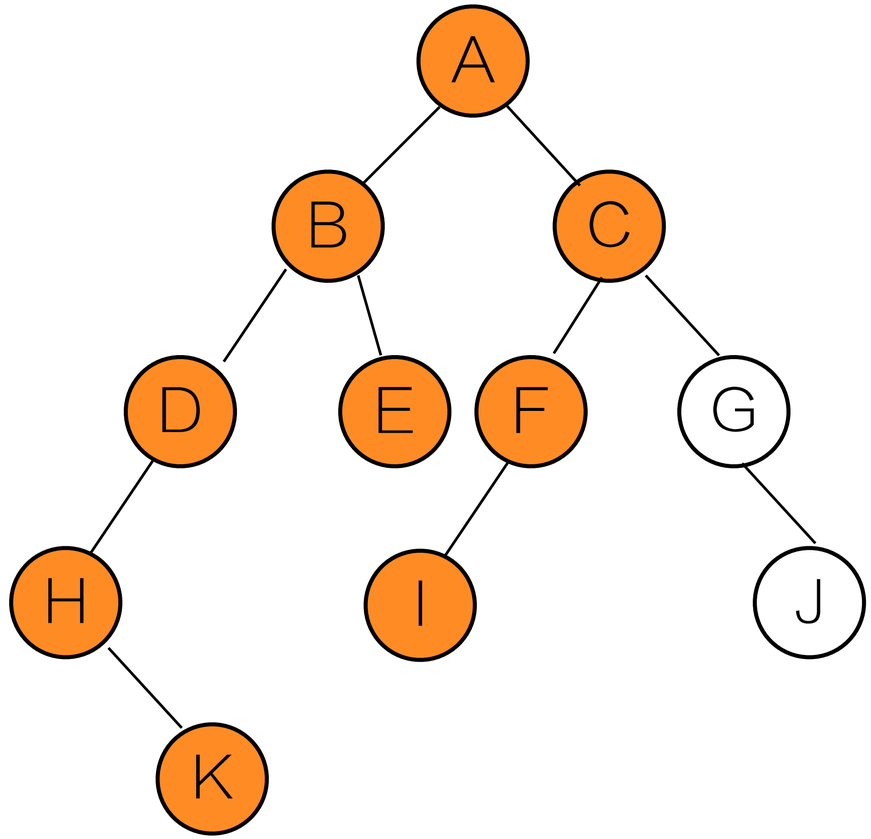


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

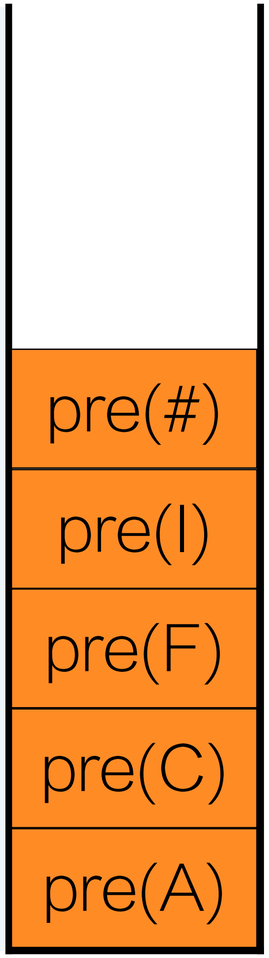




二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

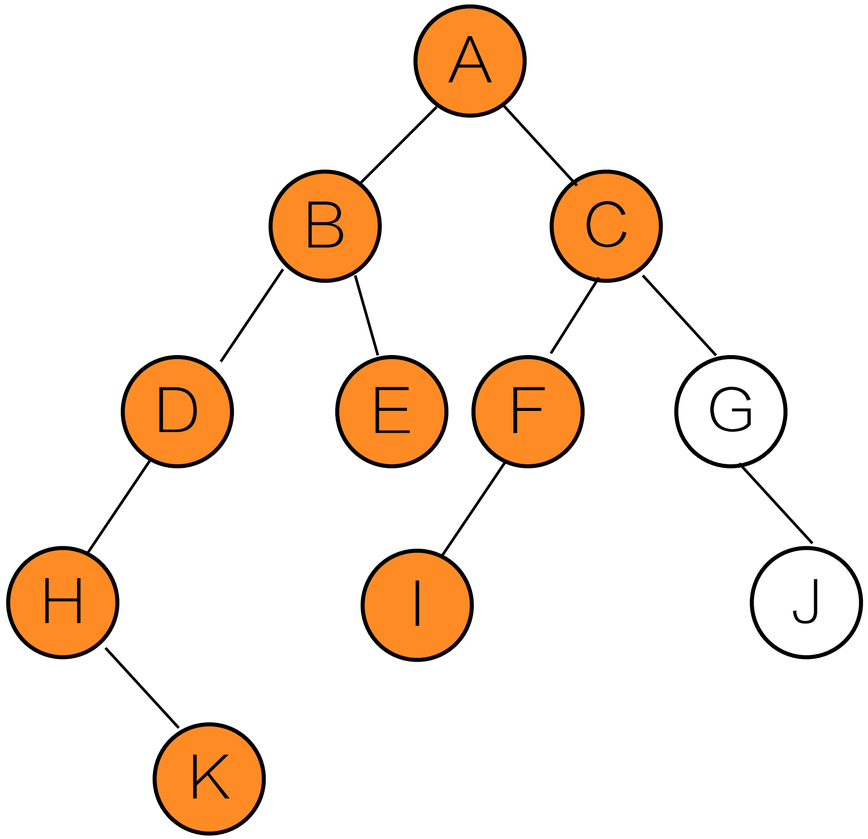


pre(#) pre(#)

pre(I) pre(#)

pre(F) pre(G)

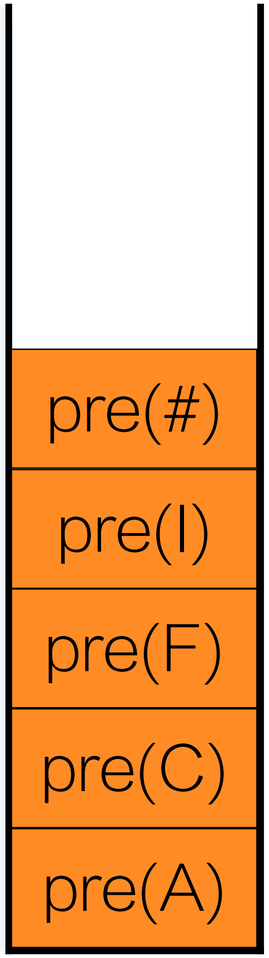
pre(B) pre(C)



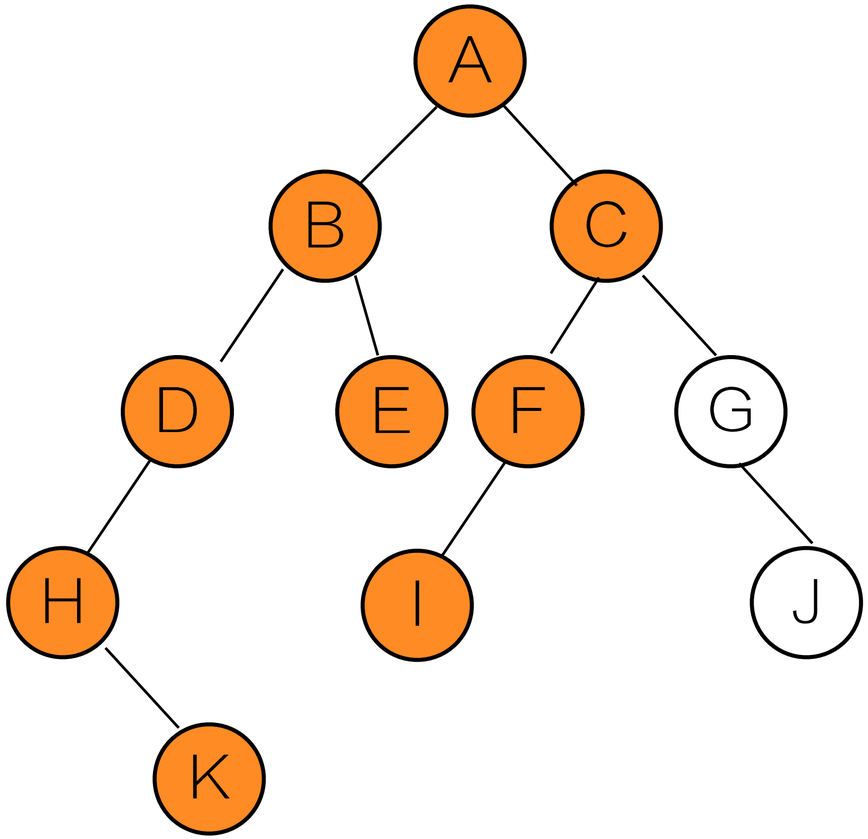
二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



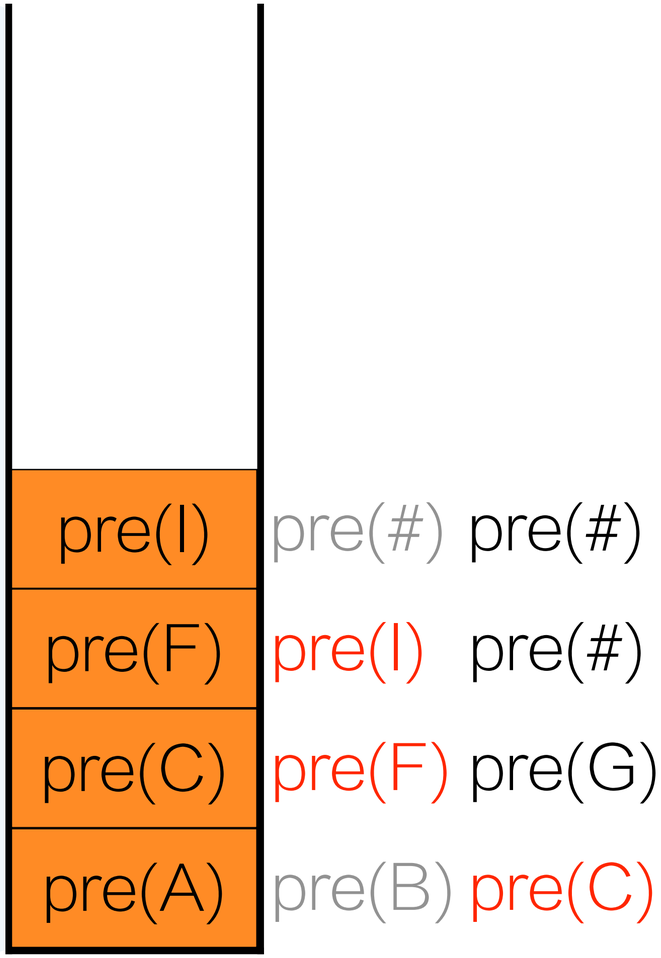
return
pre(#) pre(#)
pre(I) pre(#)
pre(F) pre(G)
pre(B) pre(C)

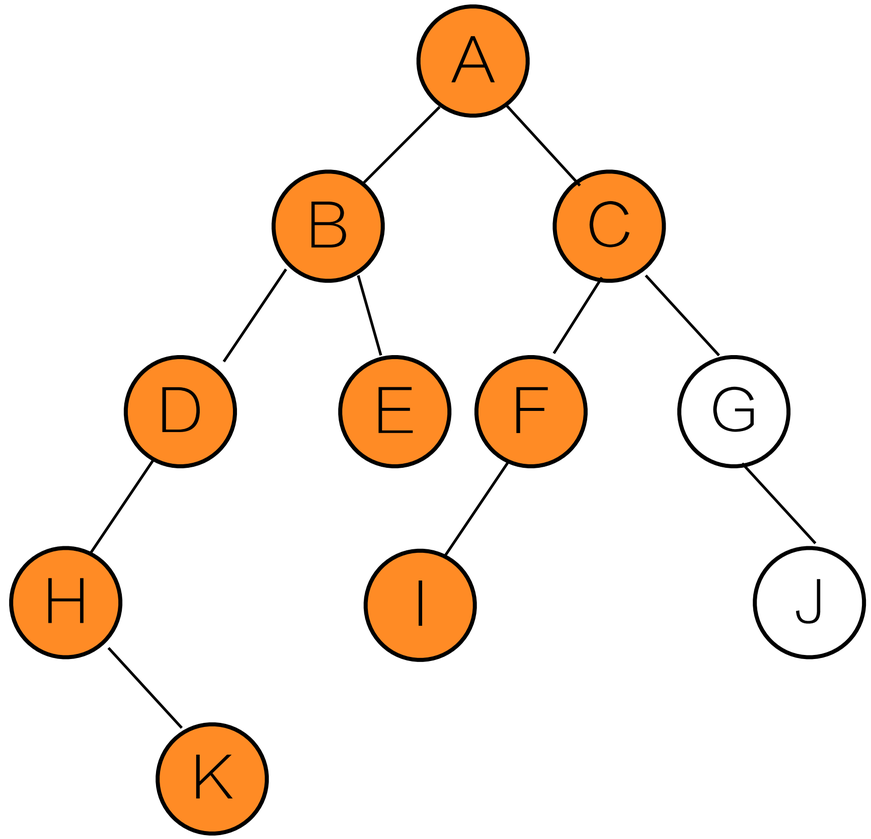


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

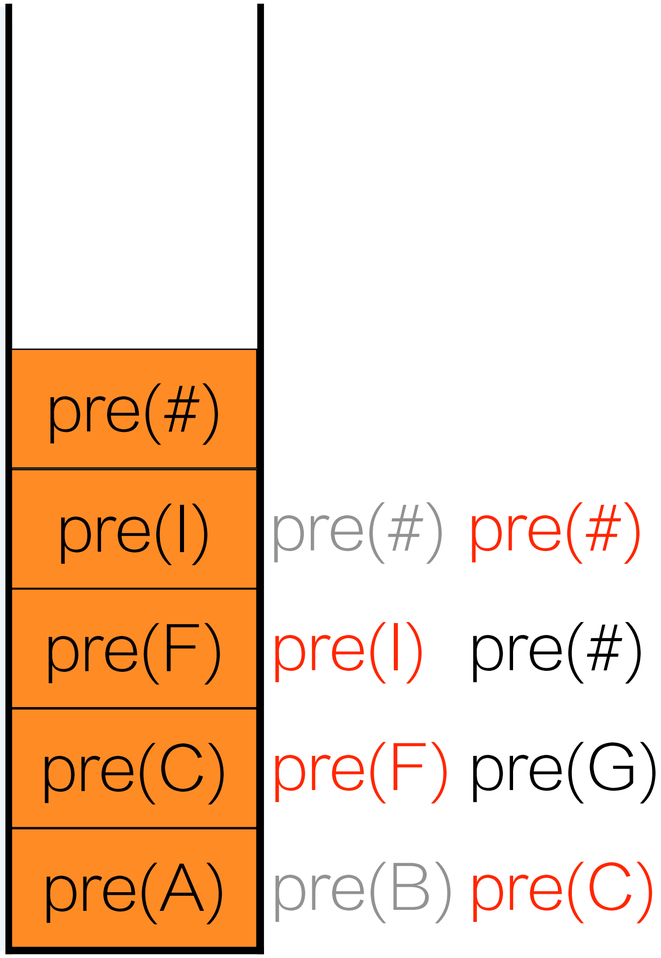


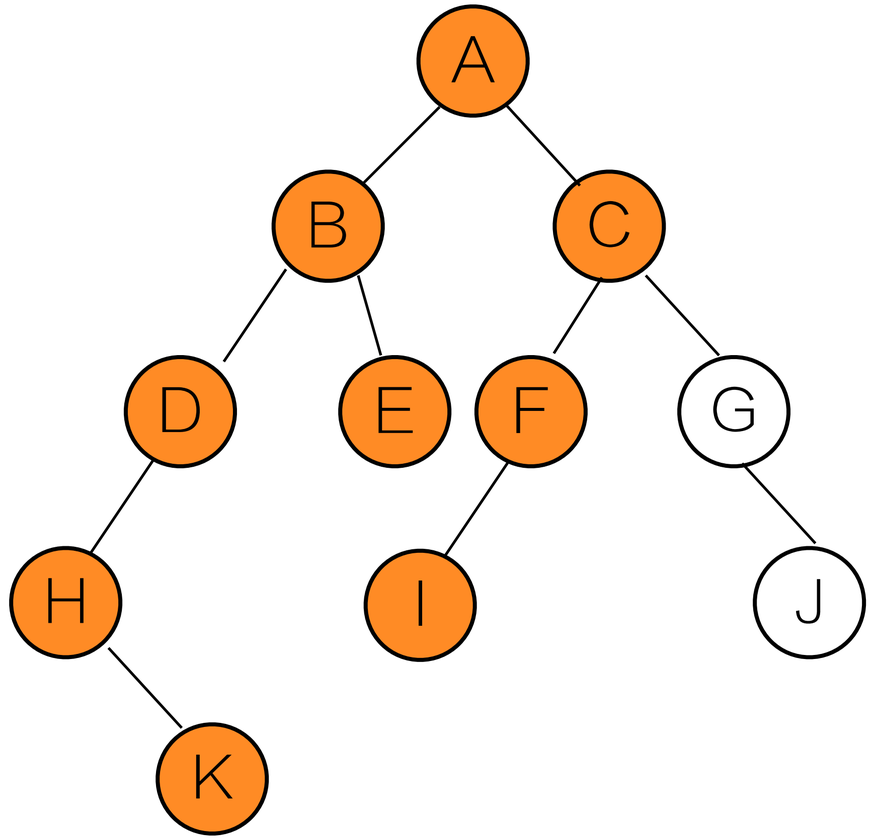


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

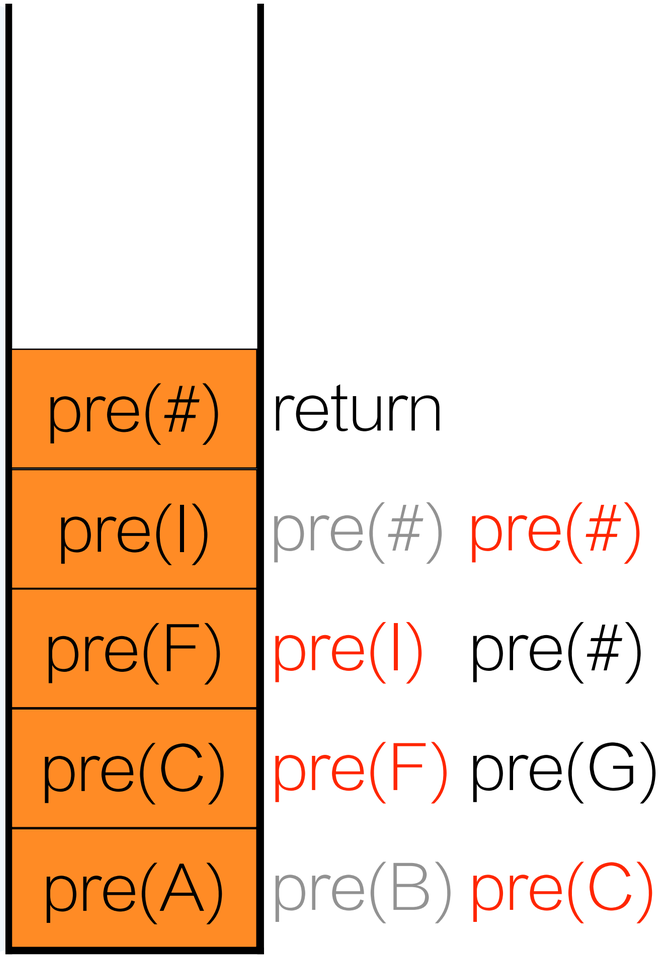


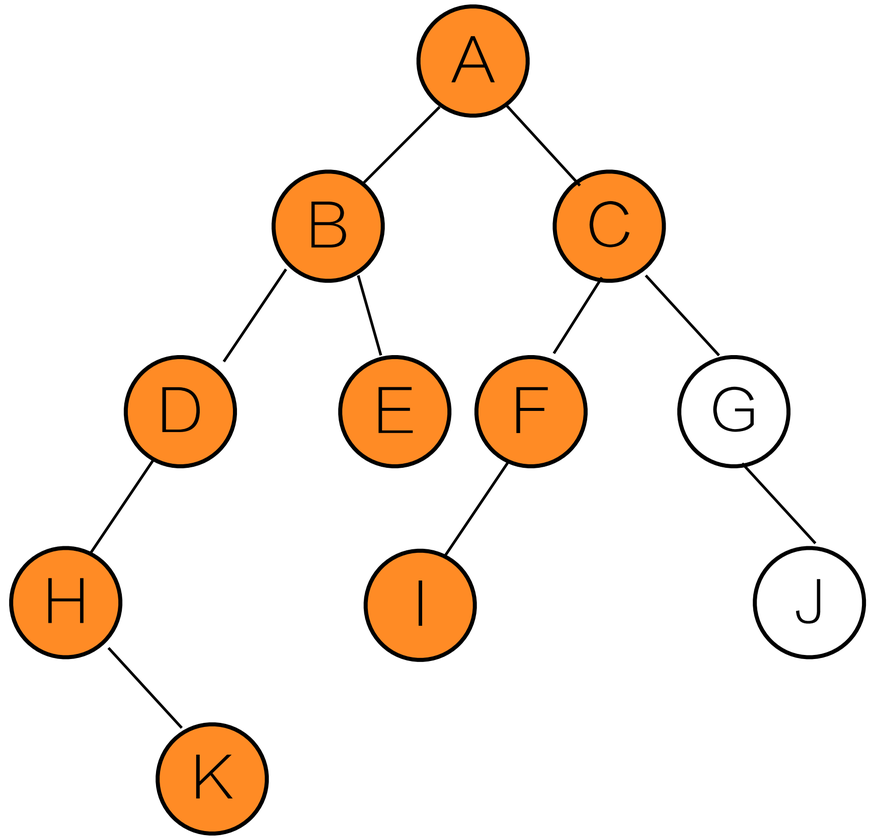


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

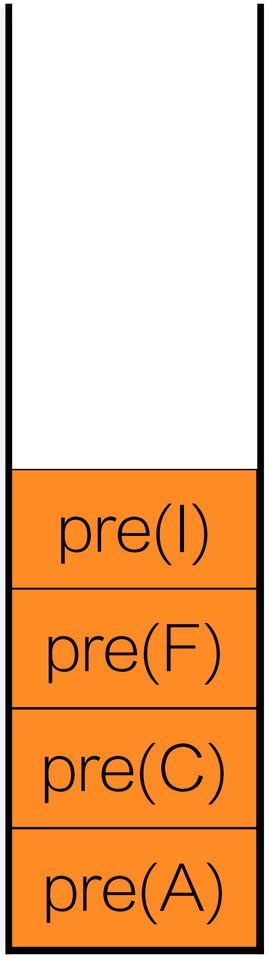




二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

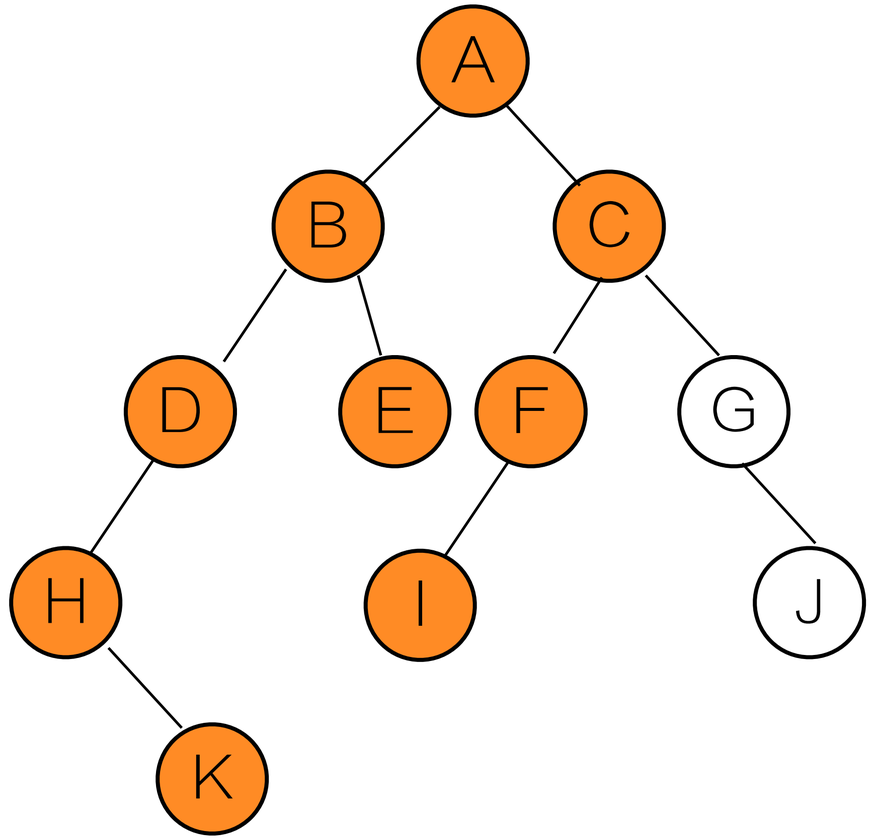


pre(#) pre(#)

pre(I) pre(#)

pre(F) pre(G)

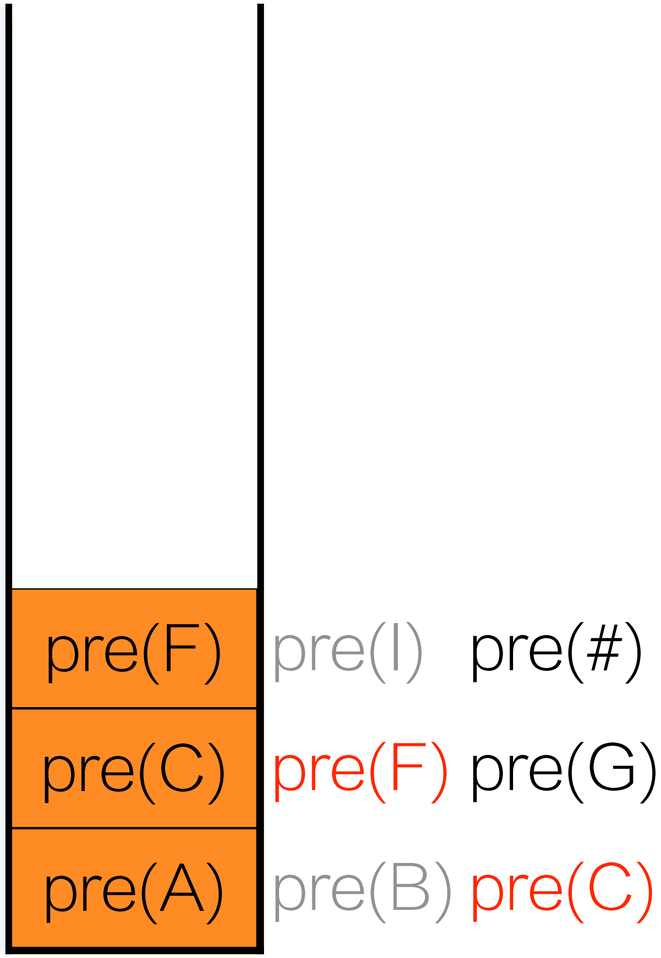
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

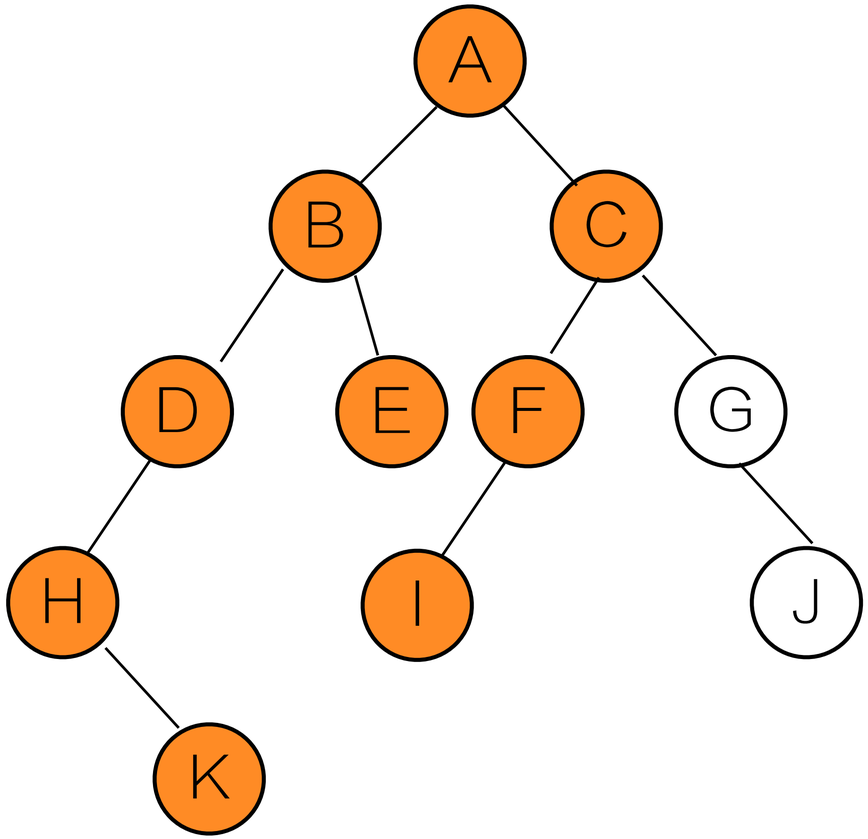
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(I) pre(#)

pre(F) pre(G)

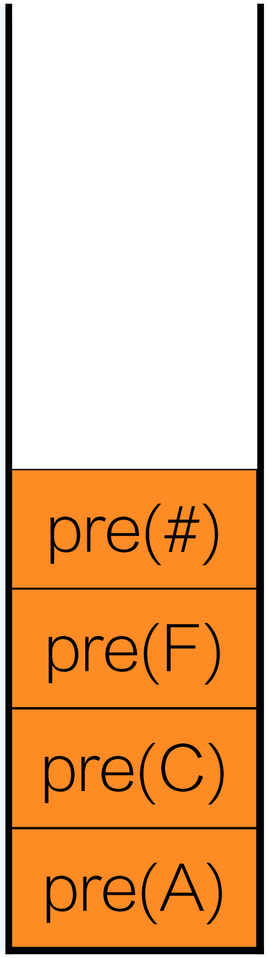
pre(B) pre(C)



二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

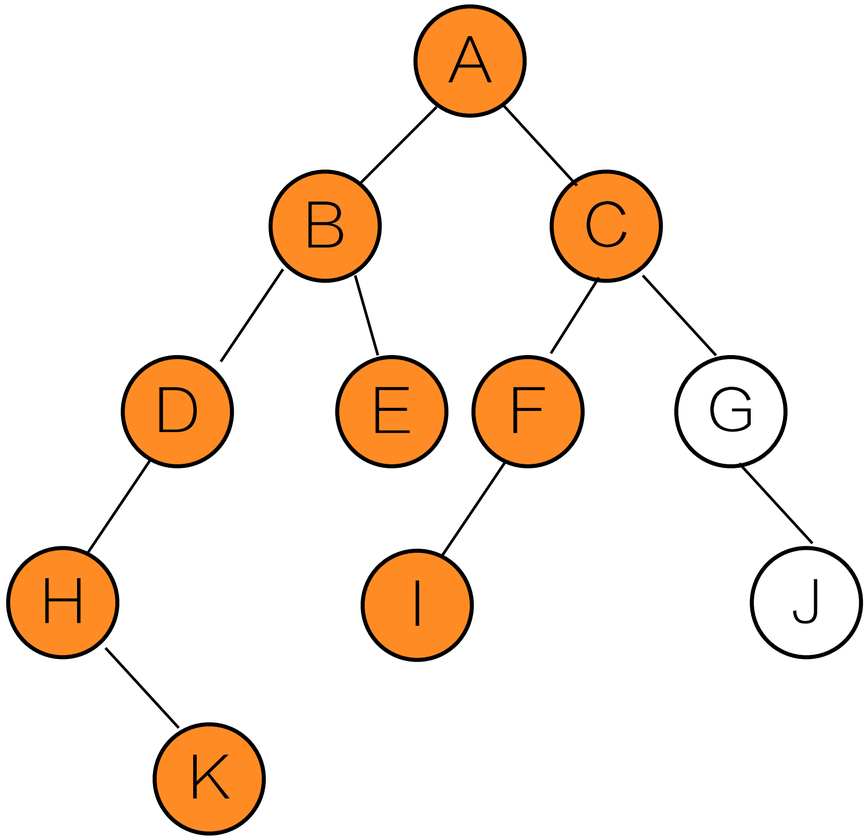
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(I) pre(#)

pre(F) pre(G)

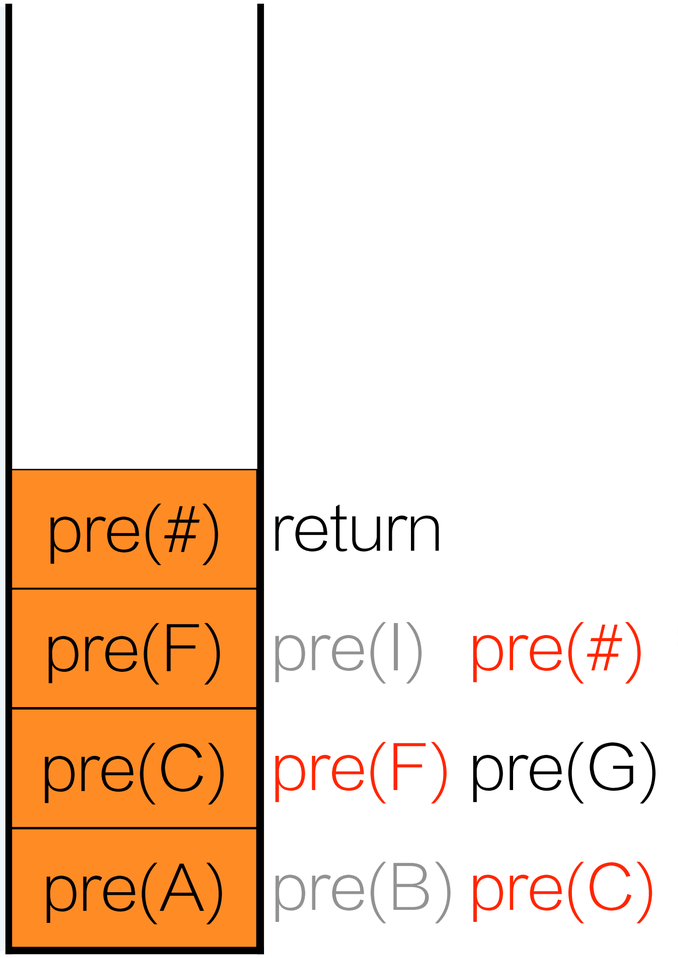
pre(B) pre(C)

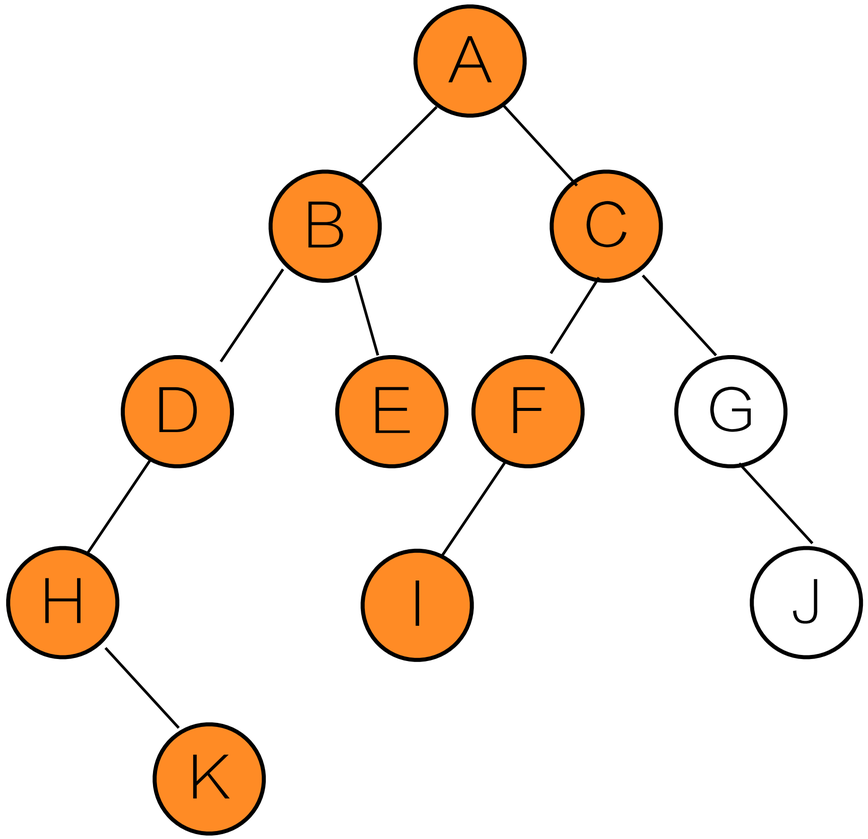


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

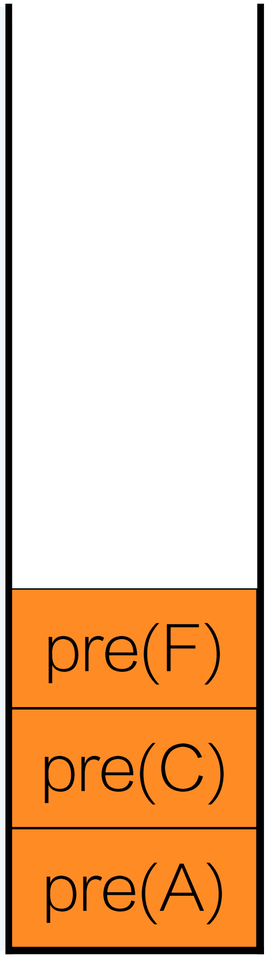




二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

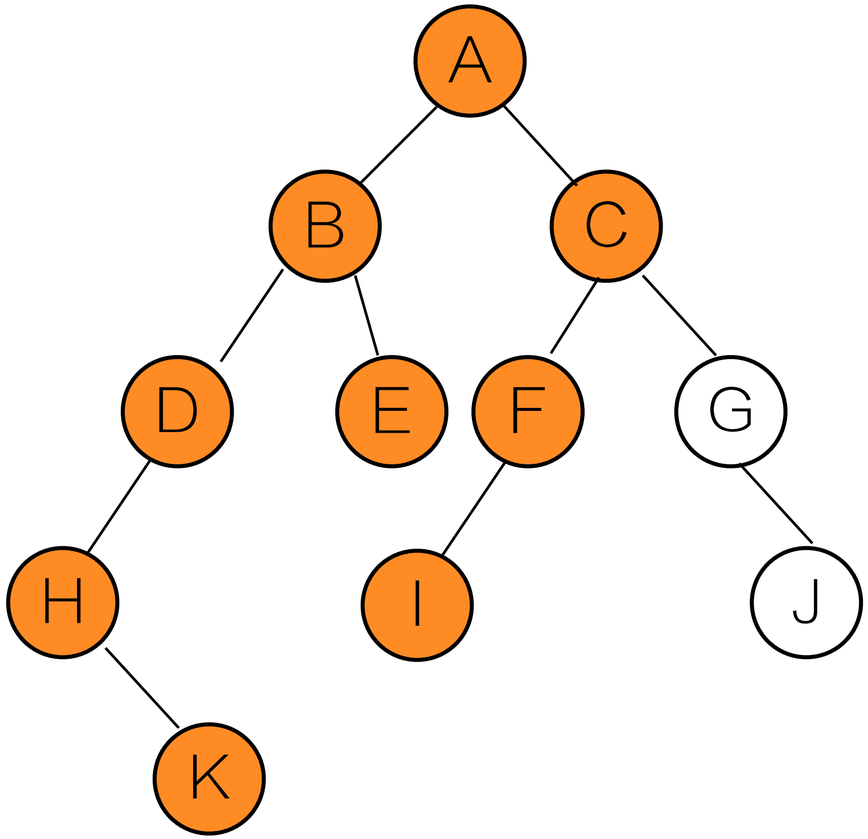
```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(I) pre(#)

pre(F) pre(G)

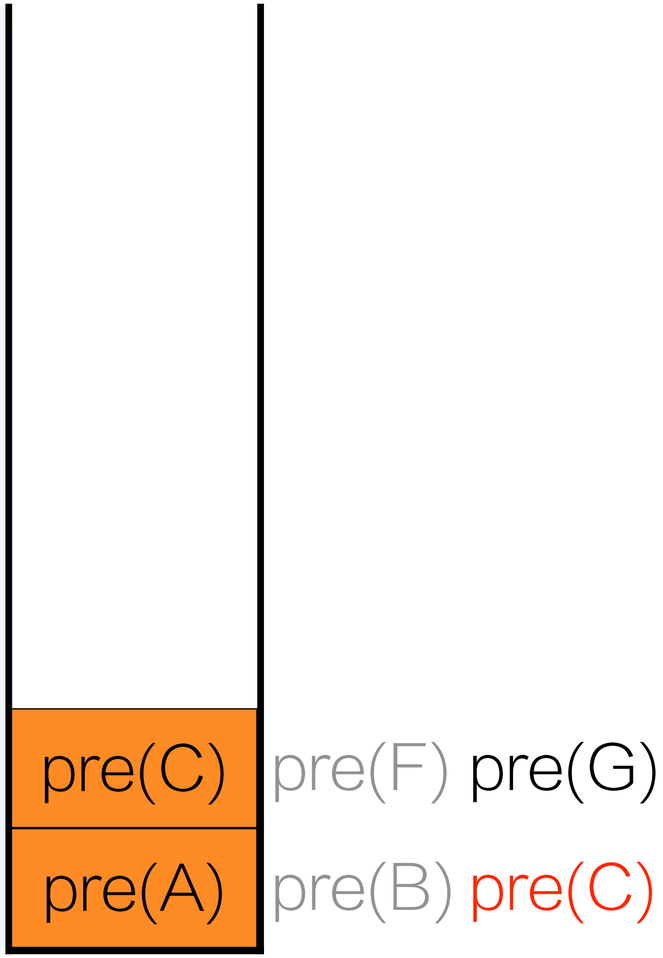
pre(B) pre(C)

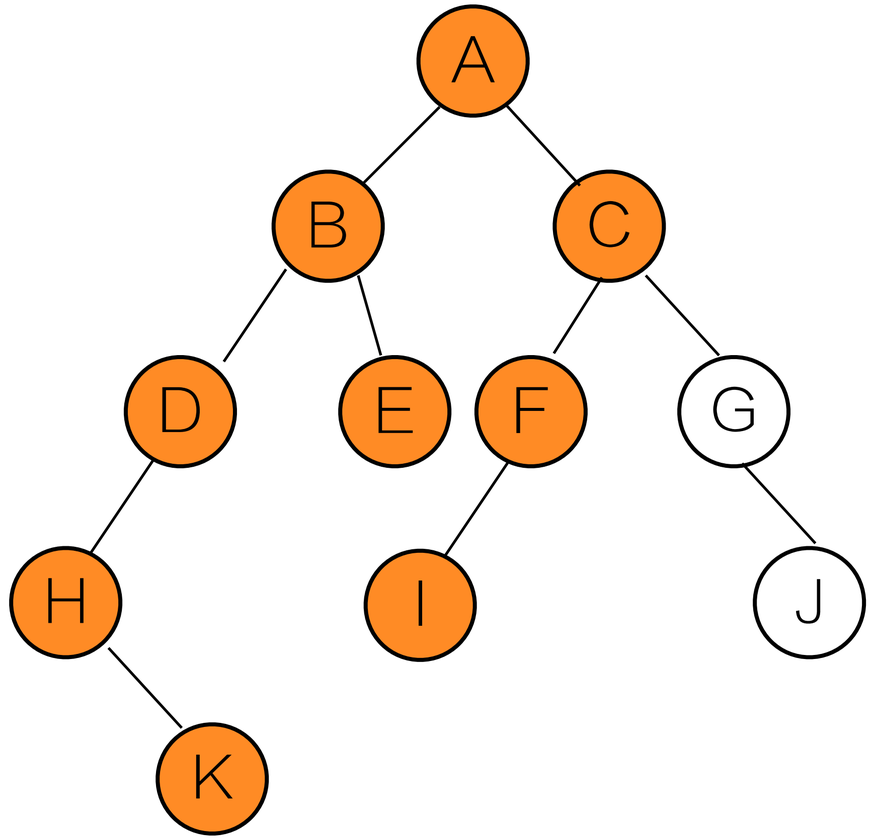


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

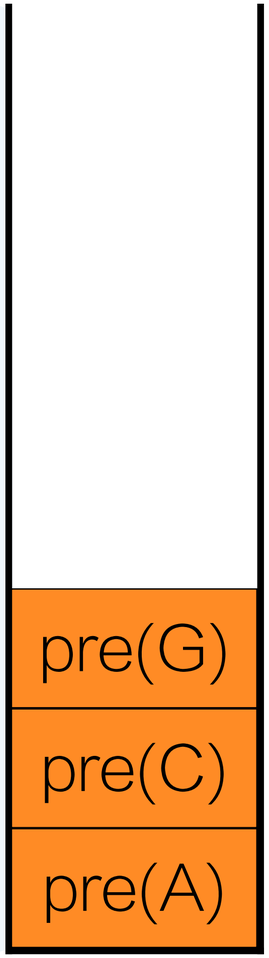




二叉树的遍历 – 前序遍历

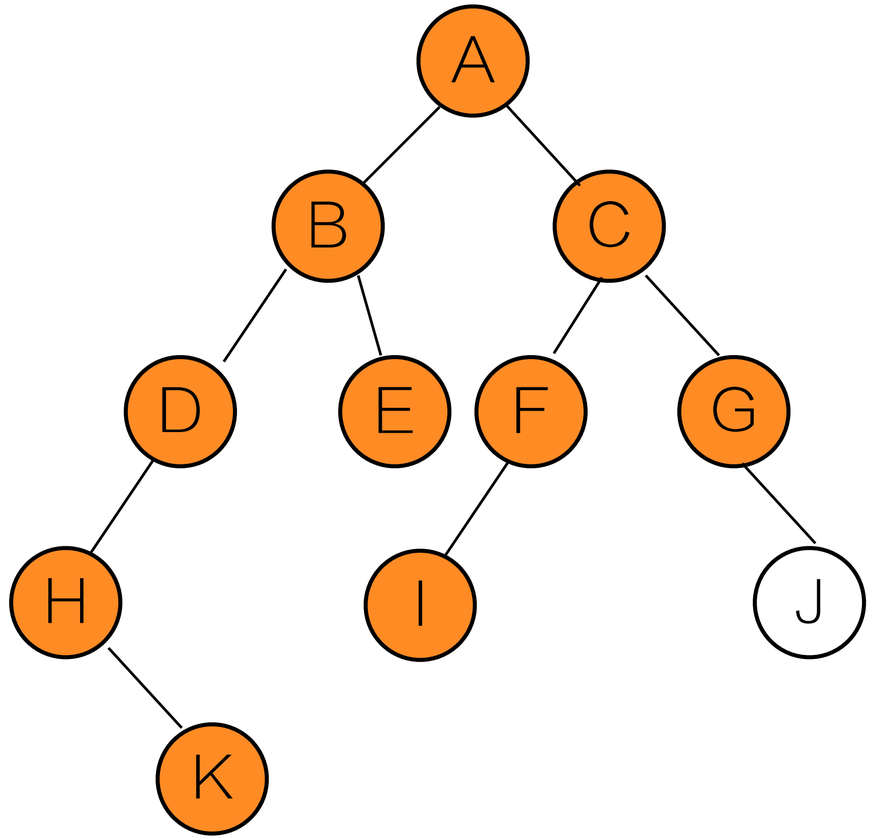
输出内容：A B D H K E C F I

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(F) pre(G)

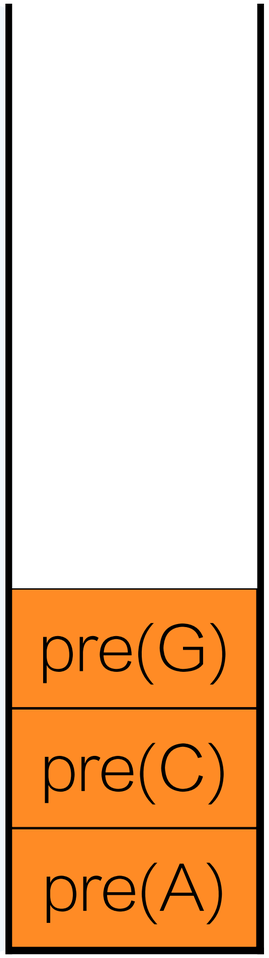
pre(B) pre(C)



二叉树的遍历 – 前序遍历

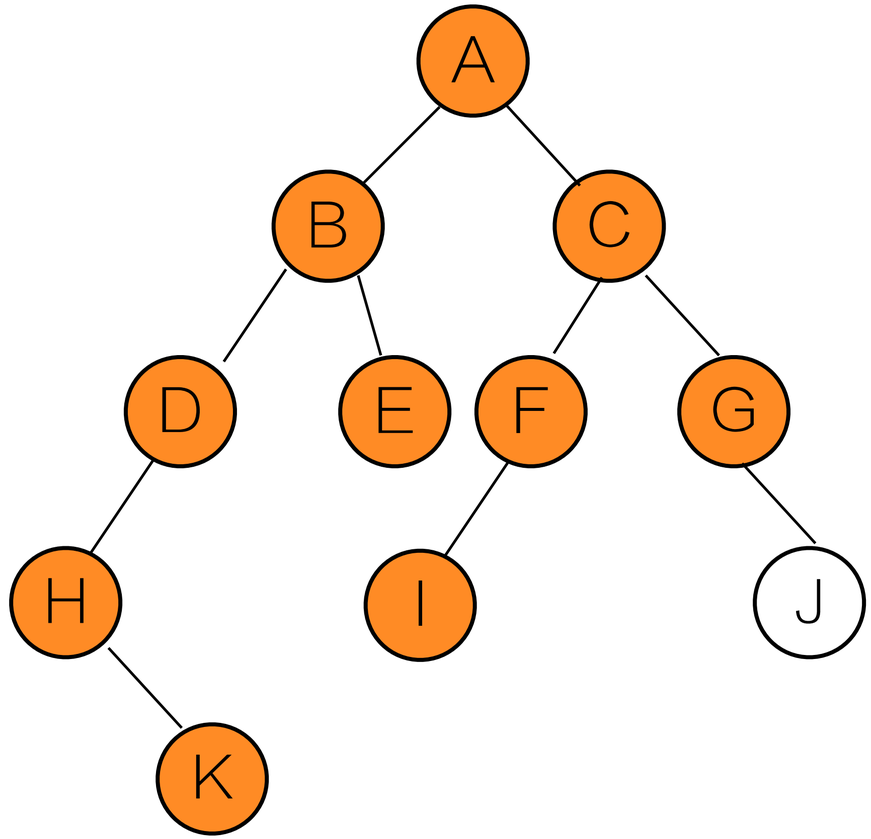
输出内容：A B D H K E C F I G

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



pre(F) pre(G)

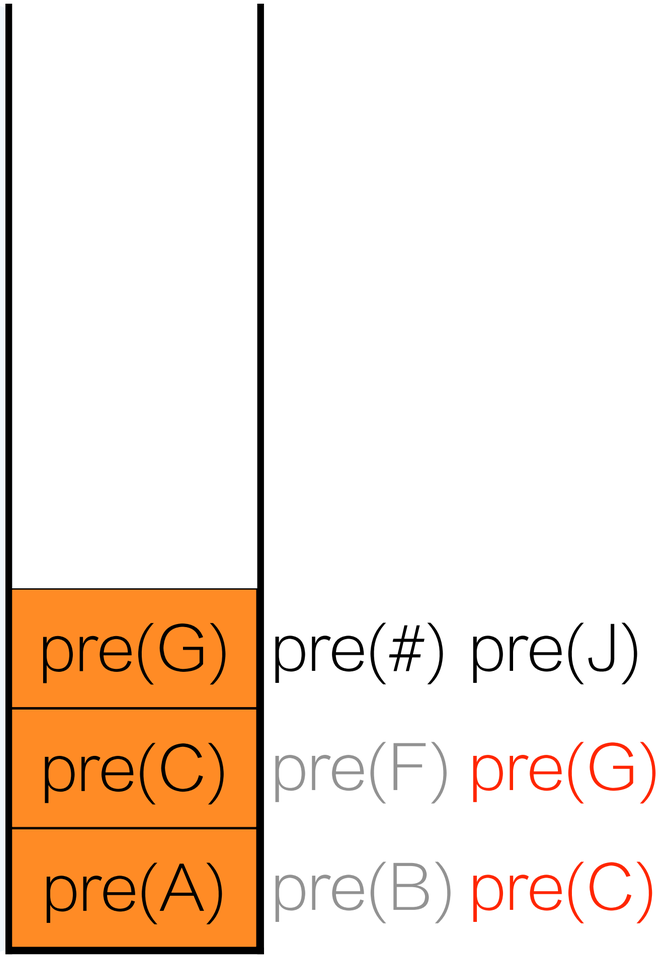
pre(B) pre(C)

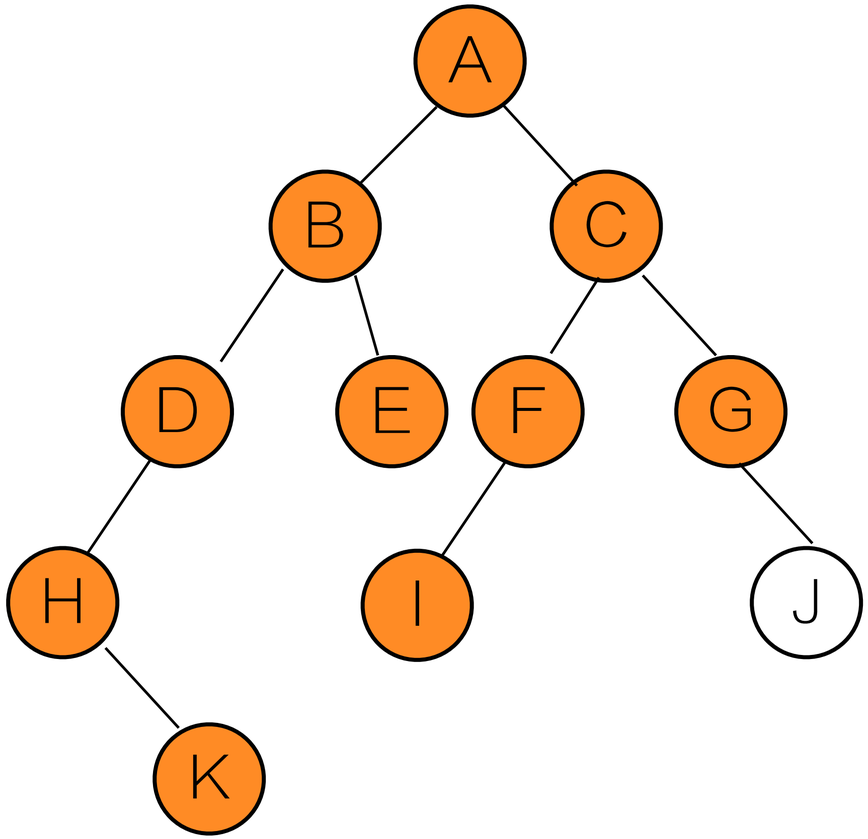


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

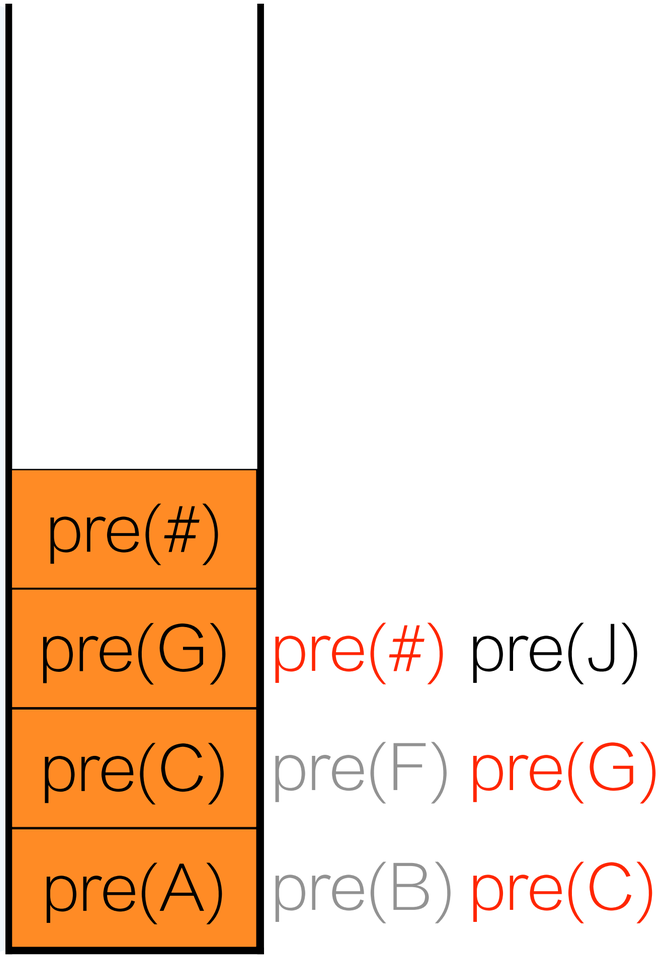


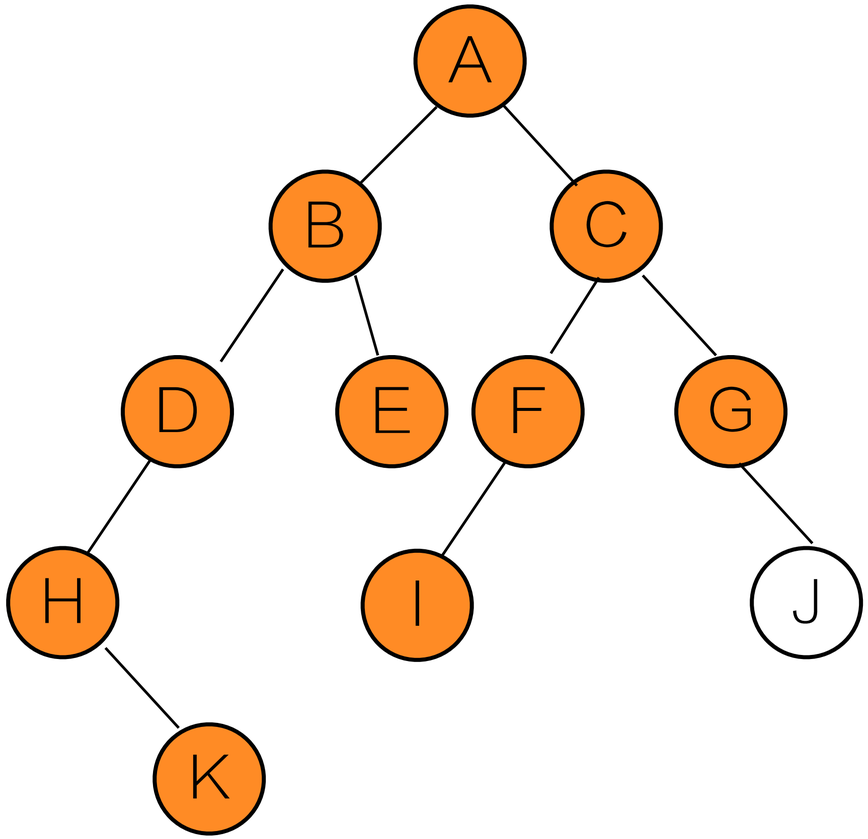


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

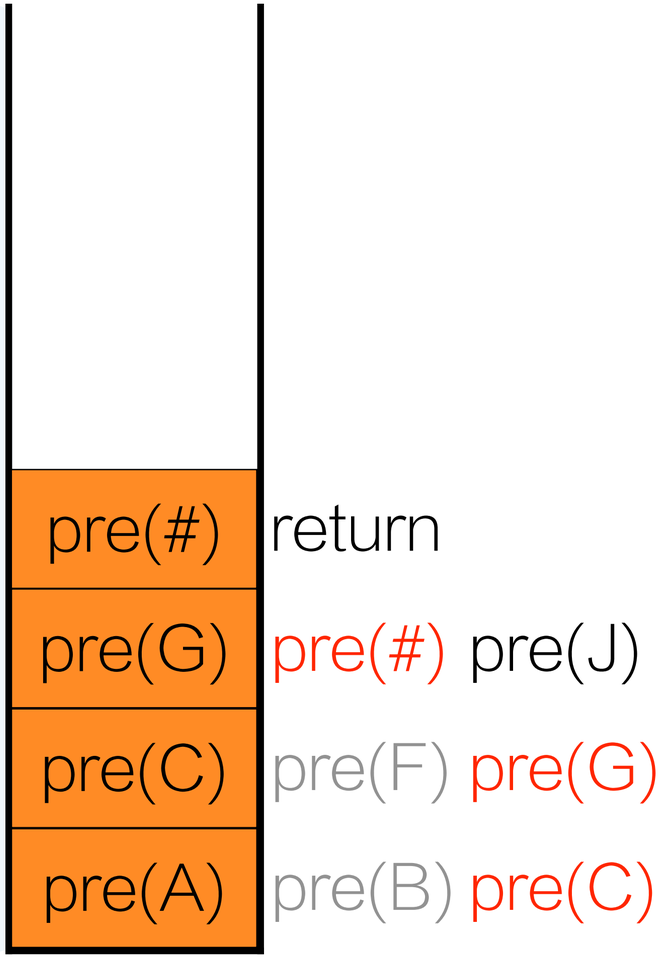


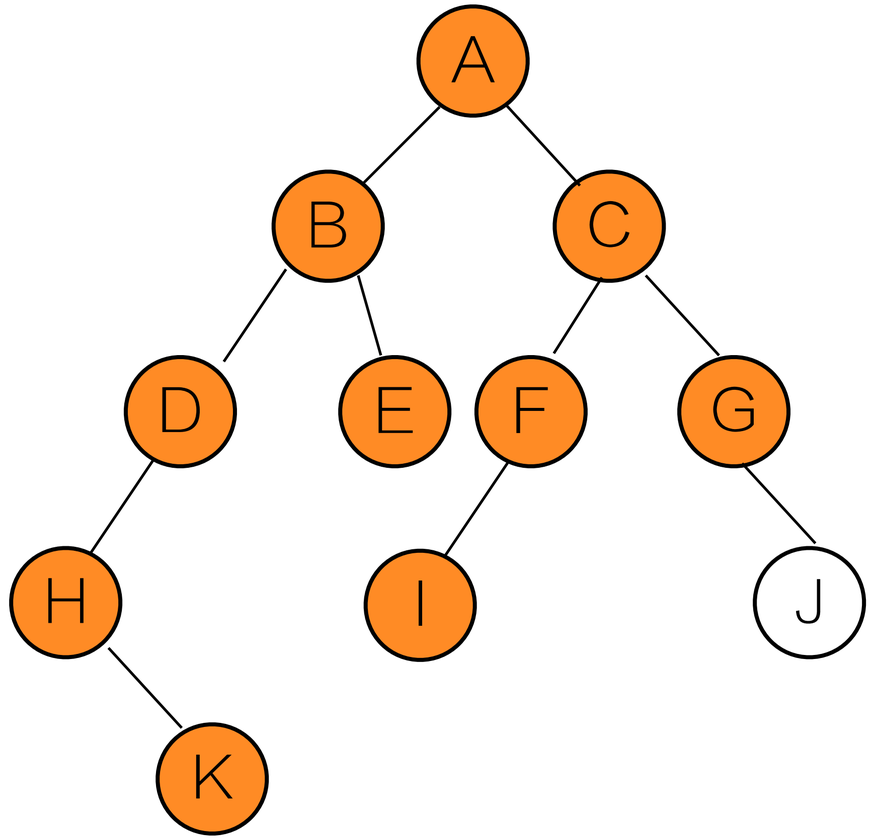


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

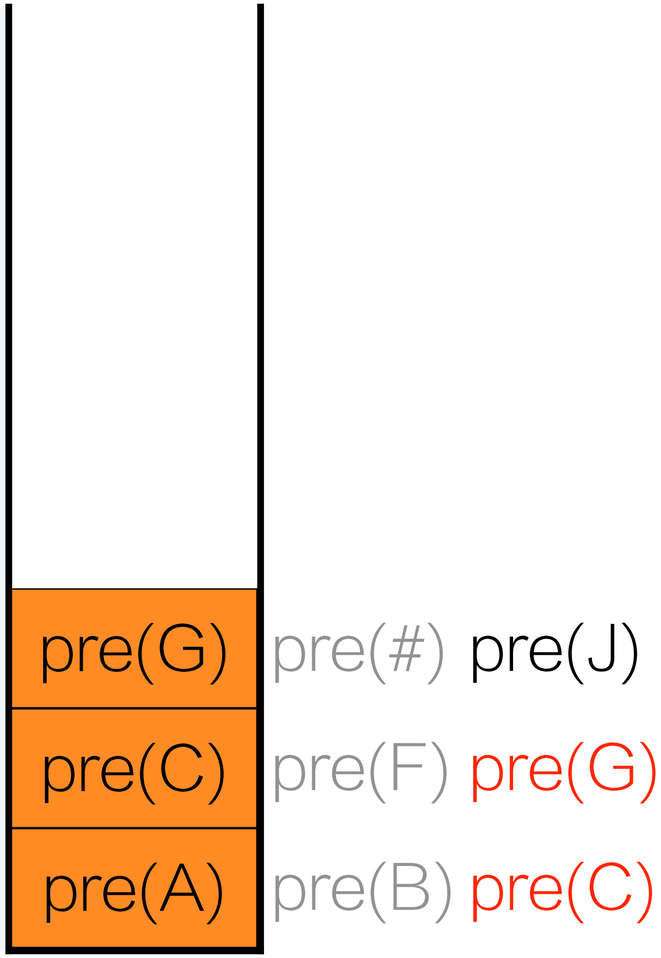


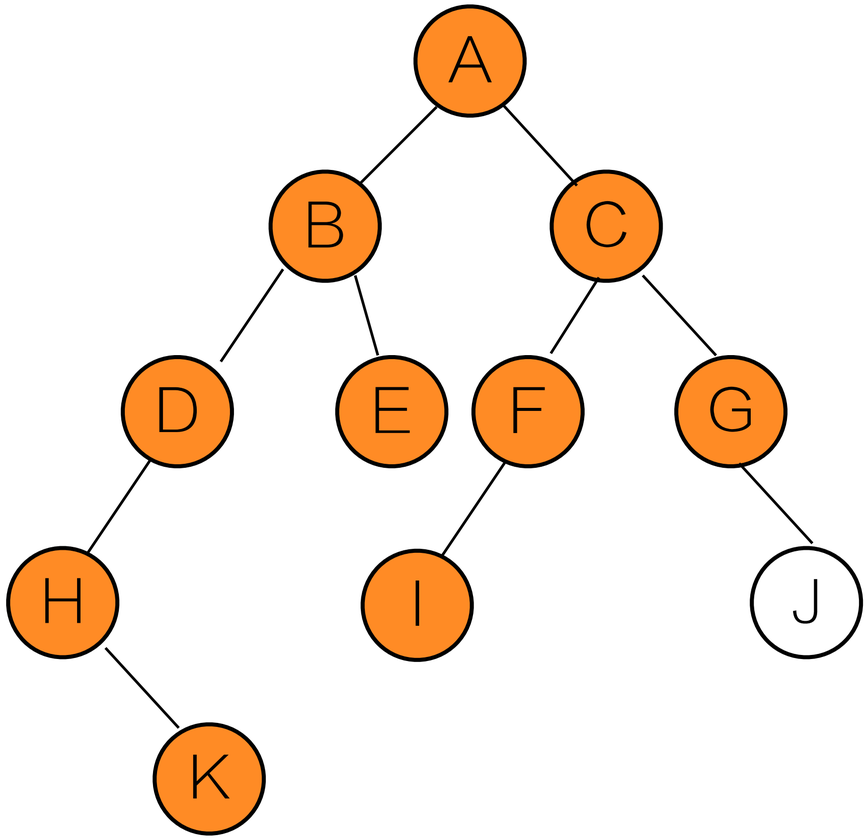


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

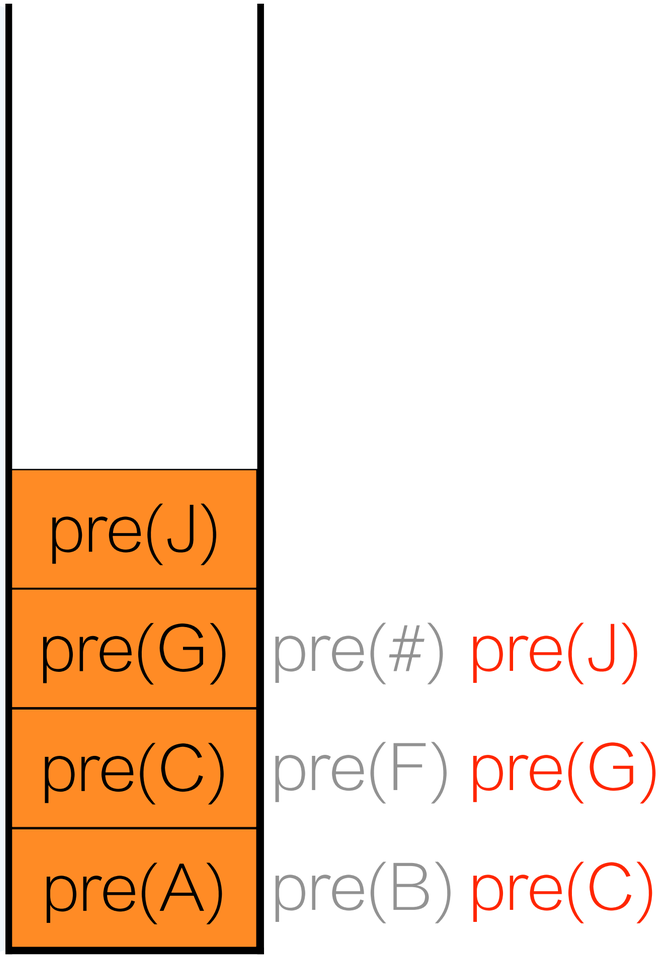


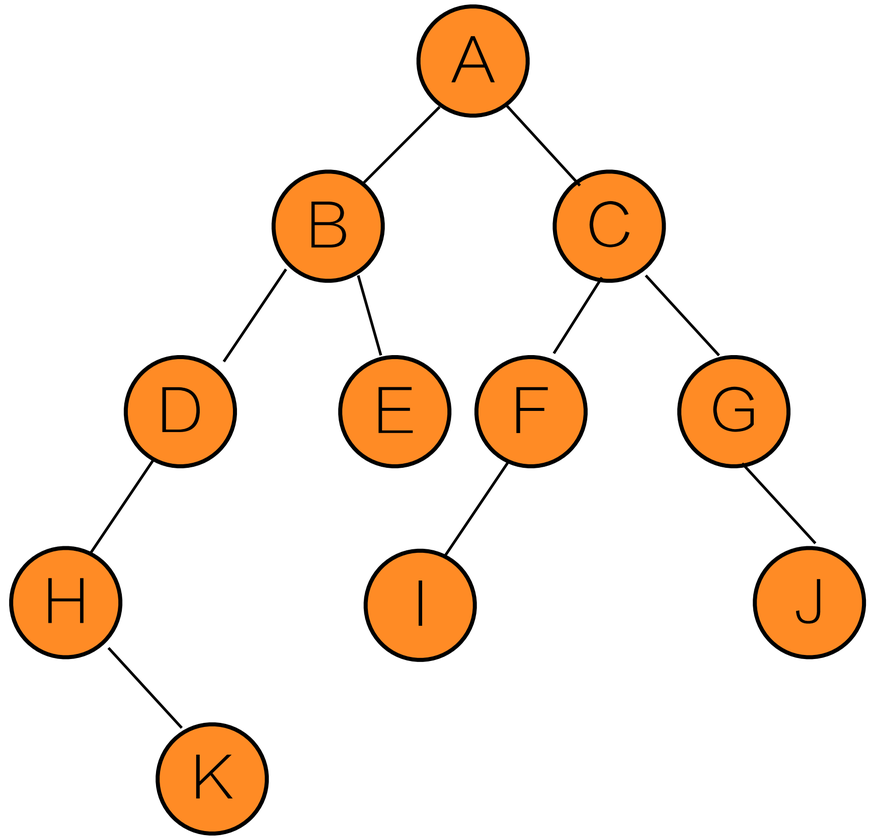


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

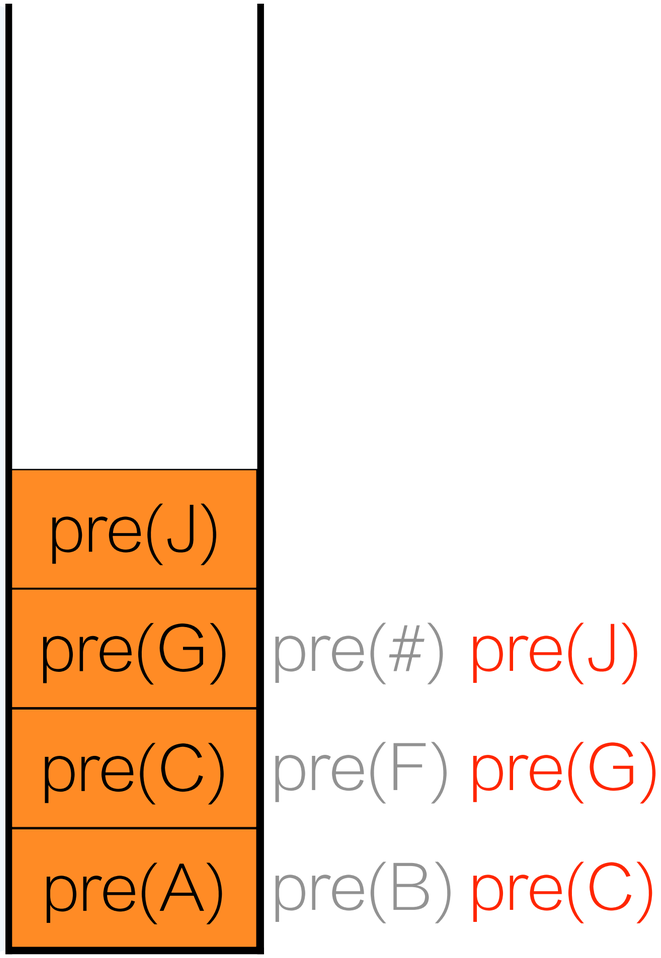


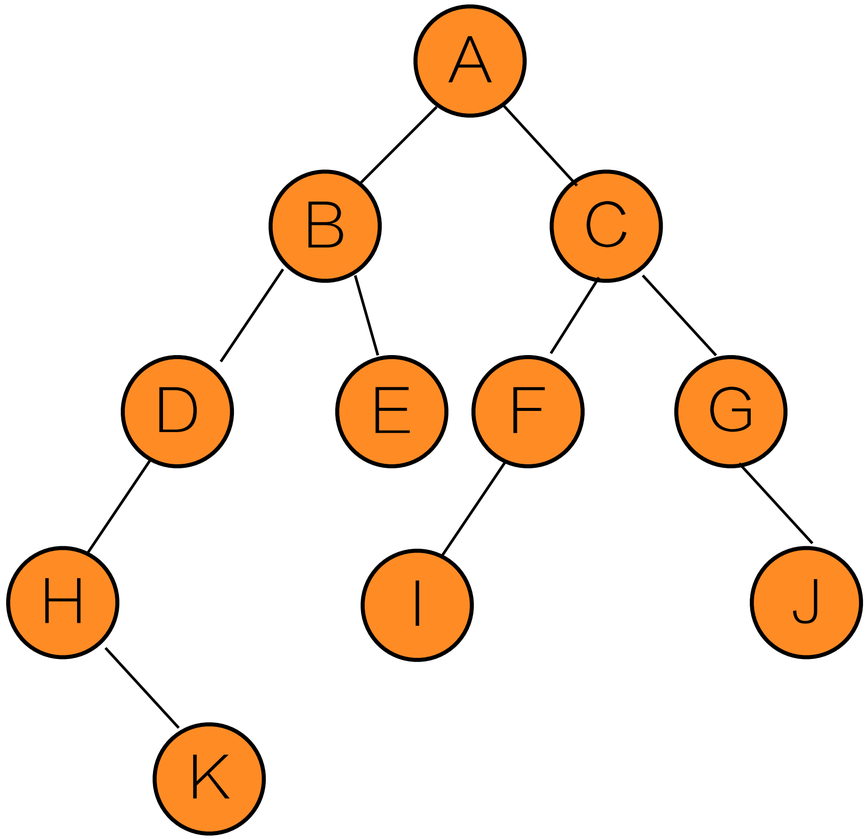


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

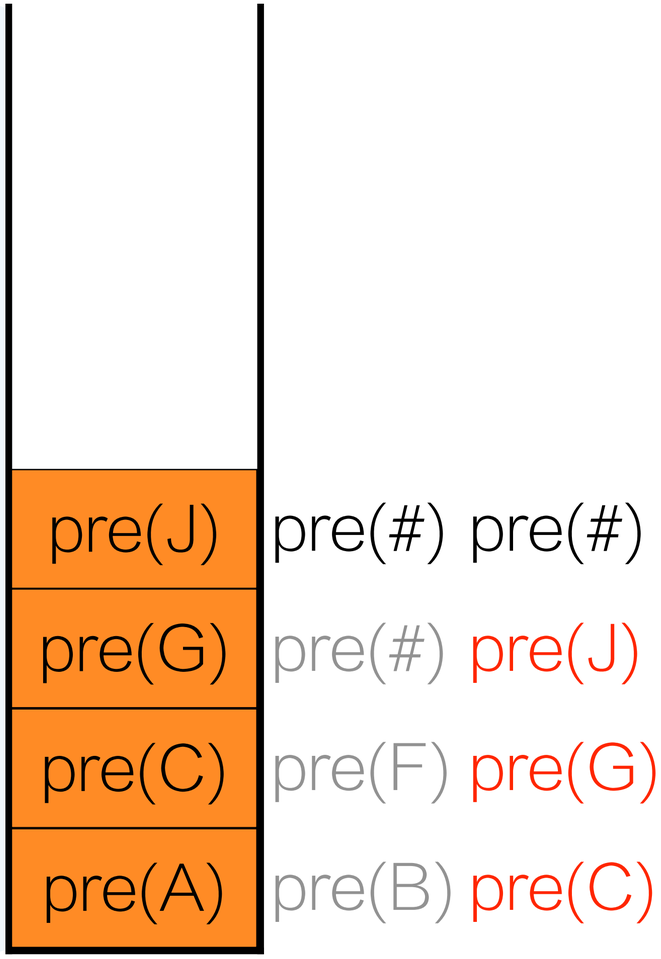


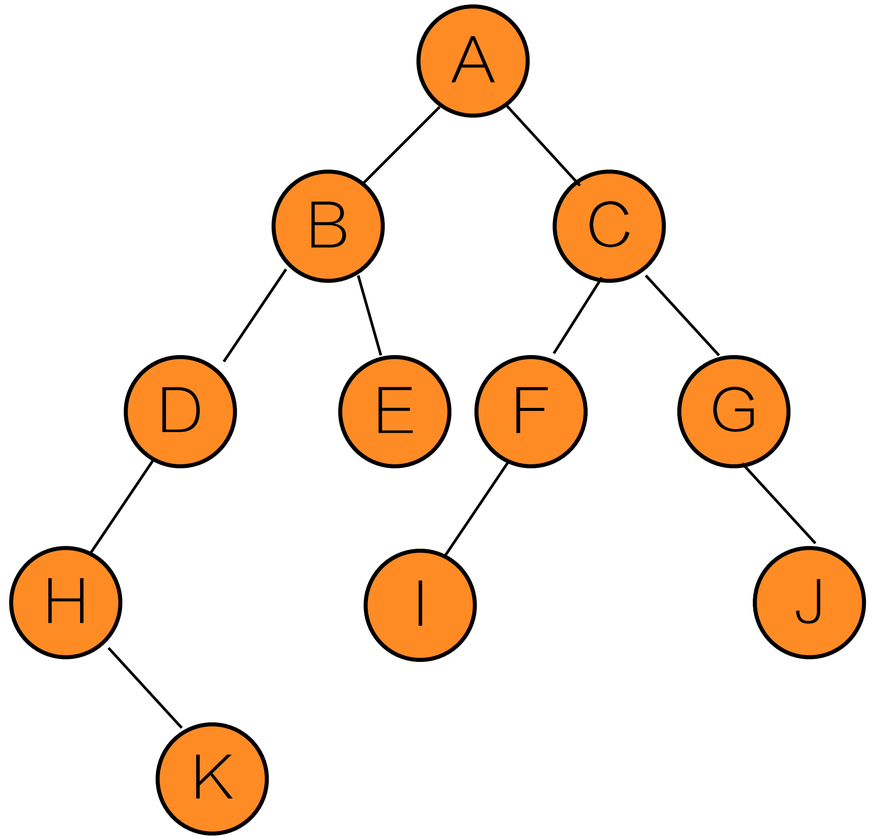


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

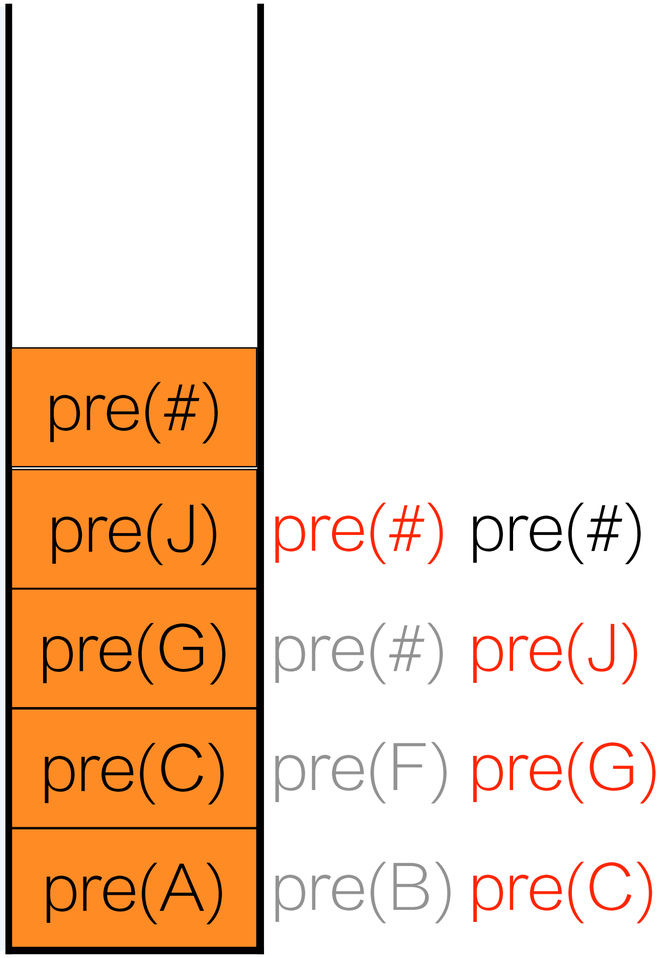


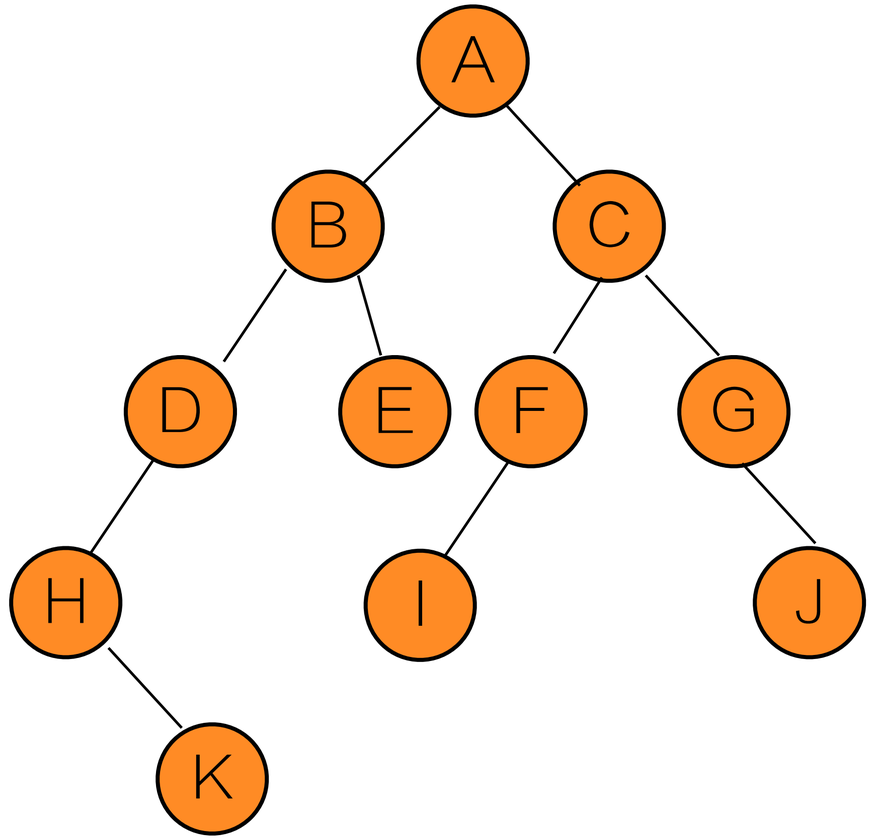


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

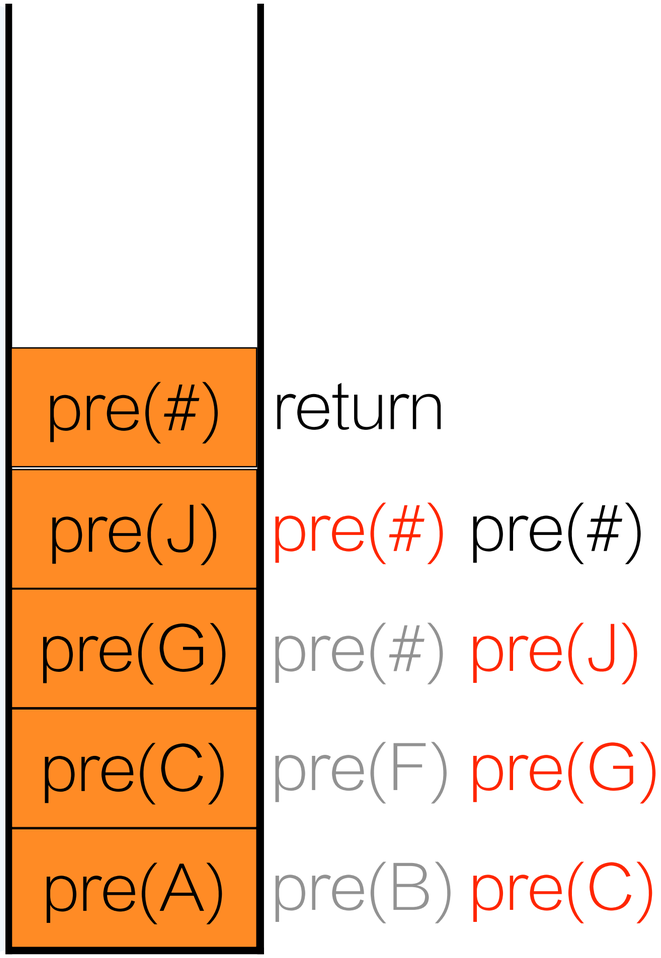


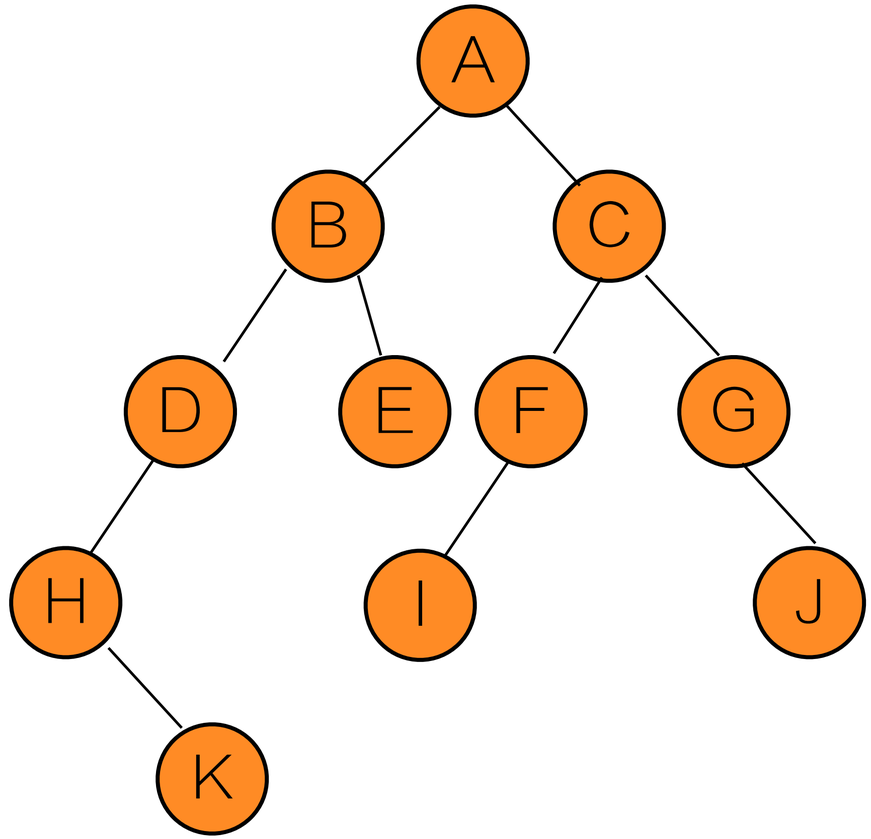


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

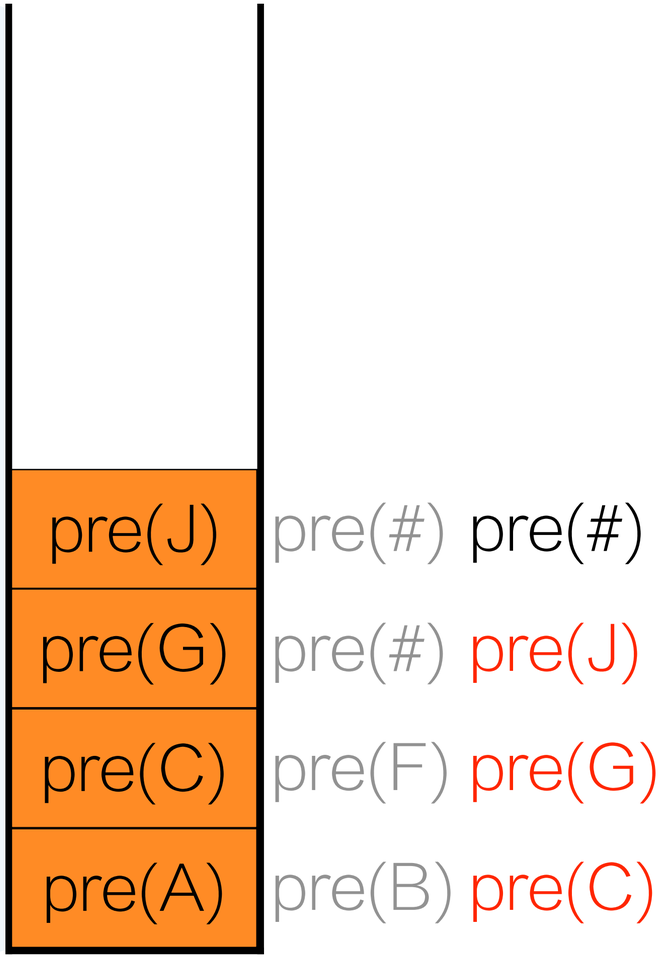


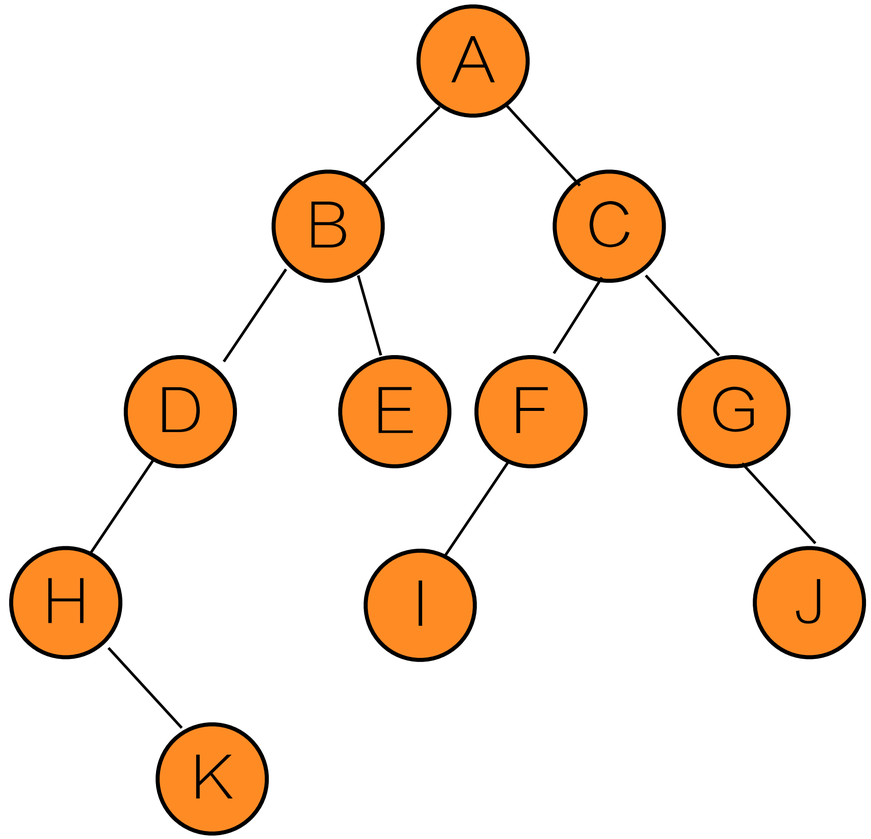


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

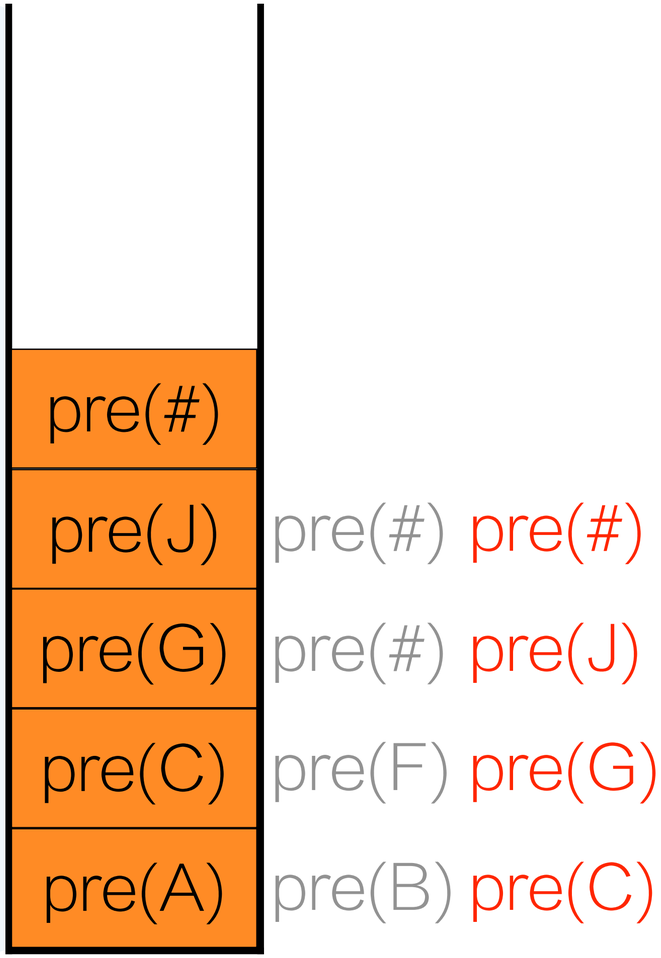


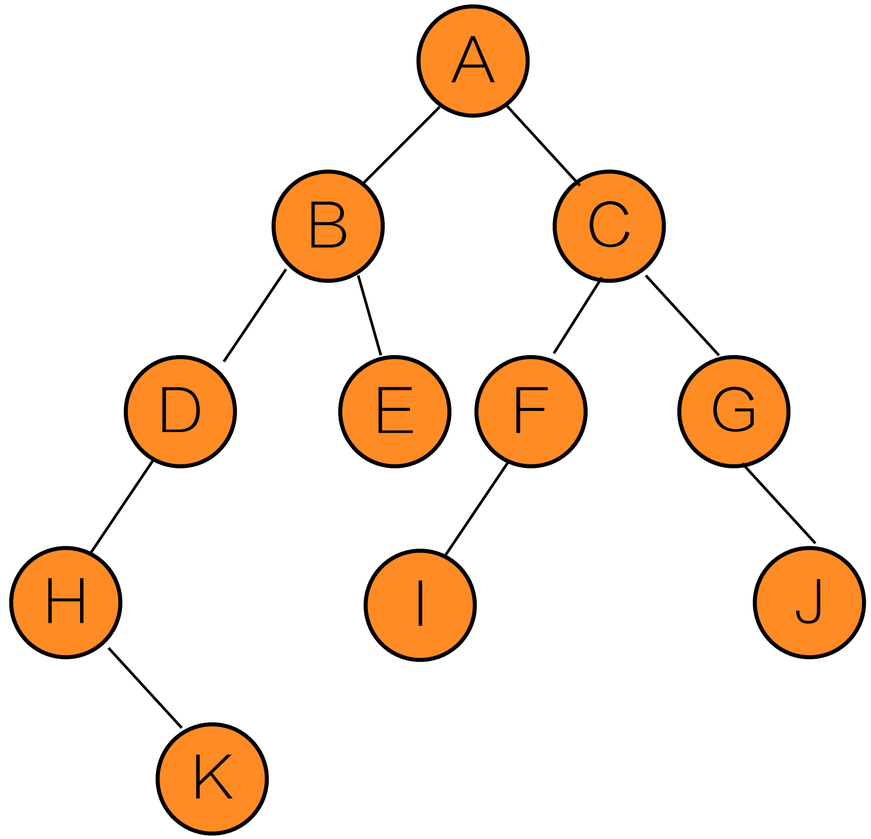


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

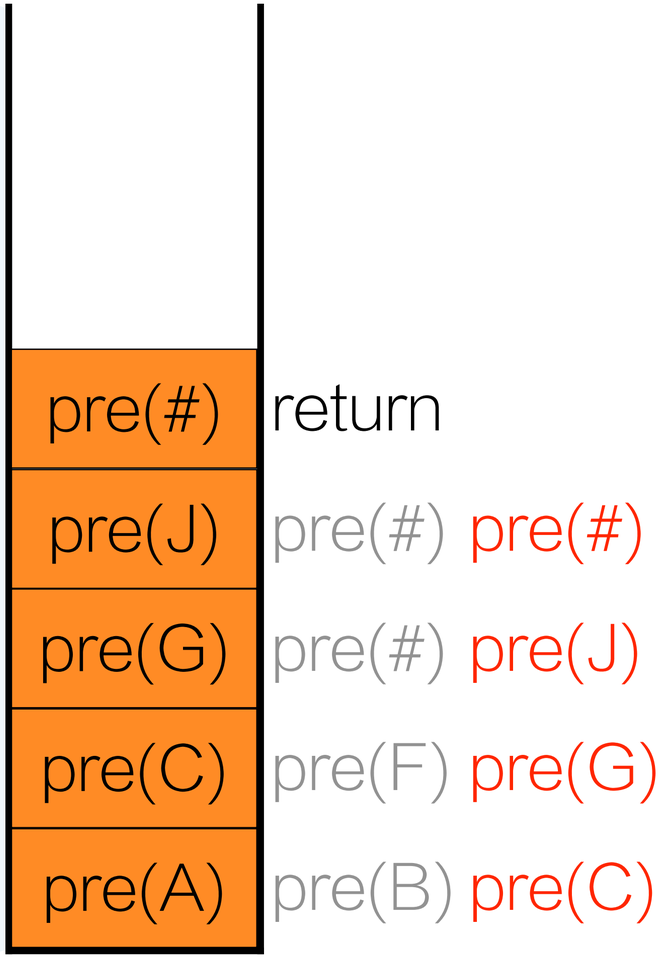


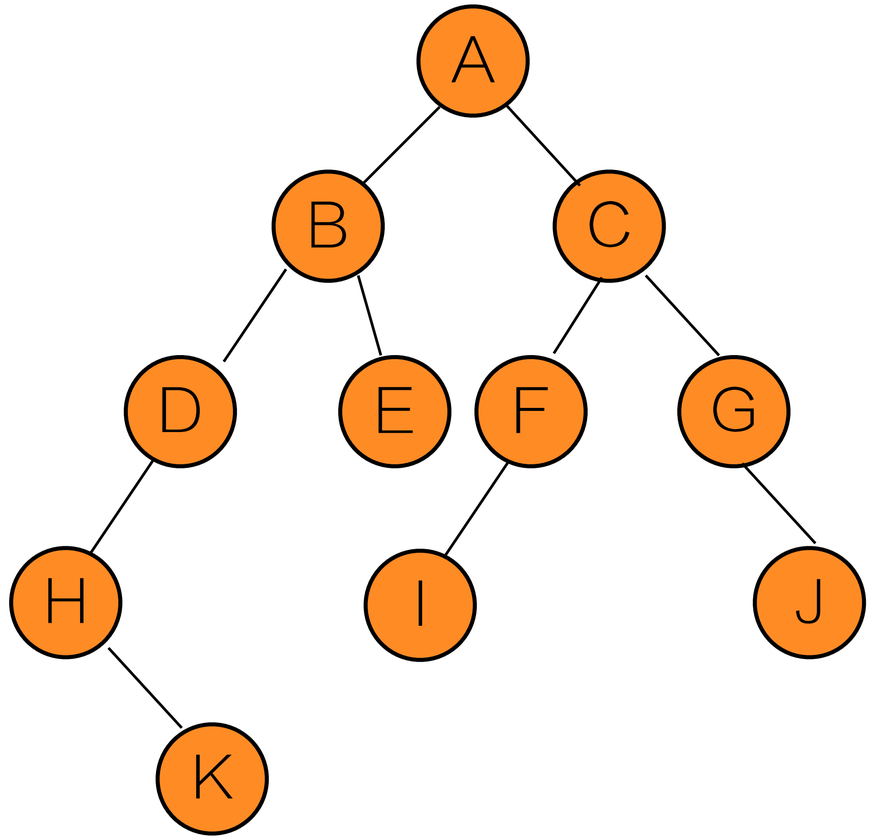


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

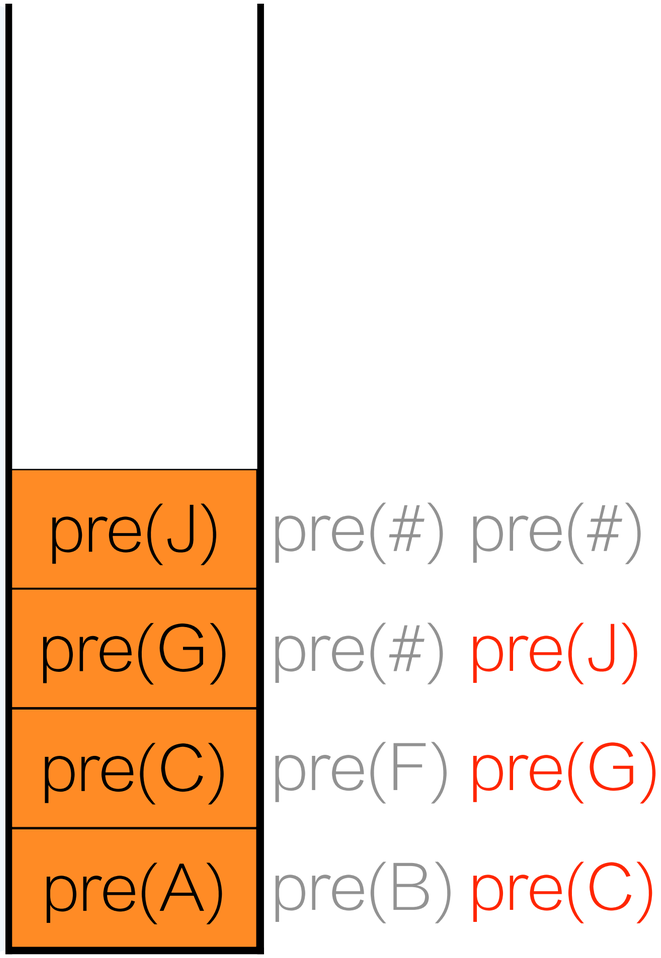


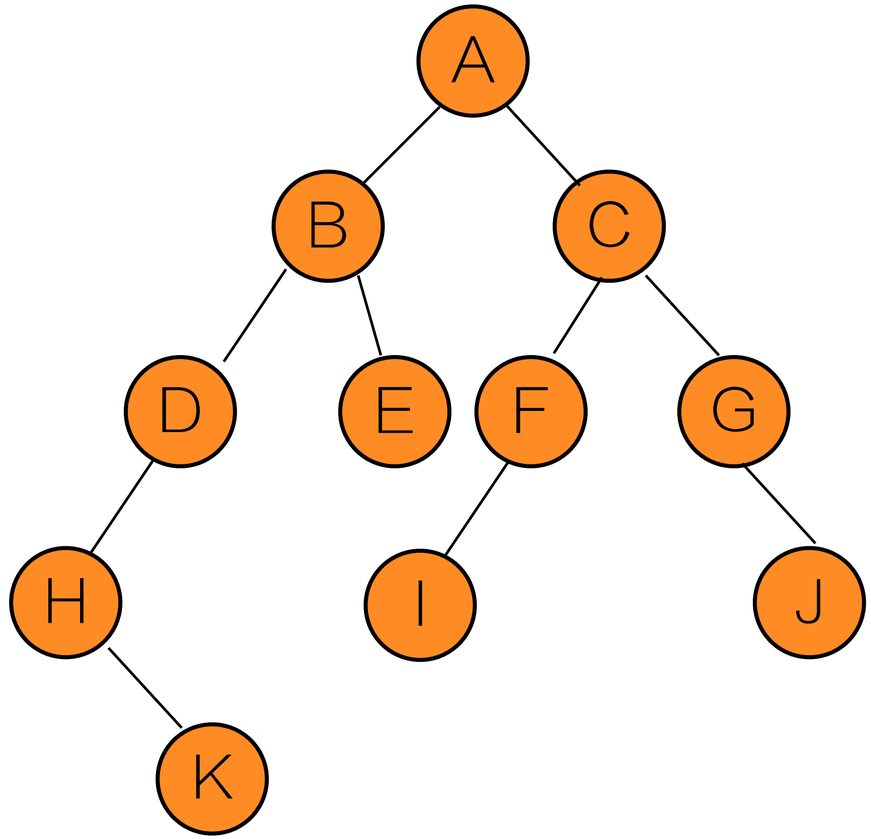


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

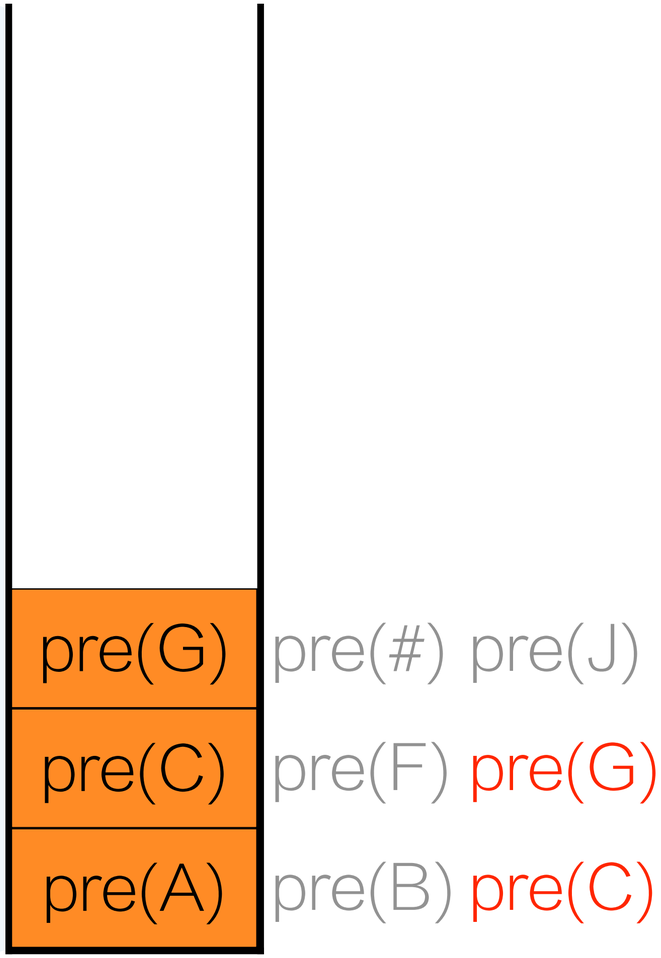


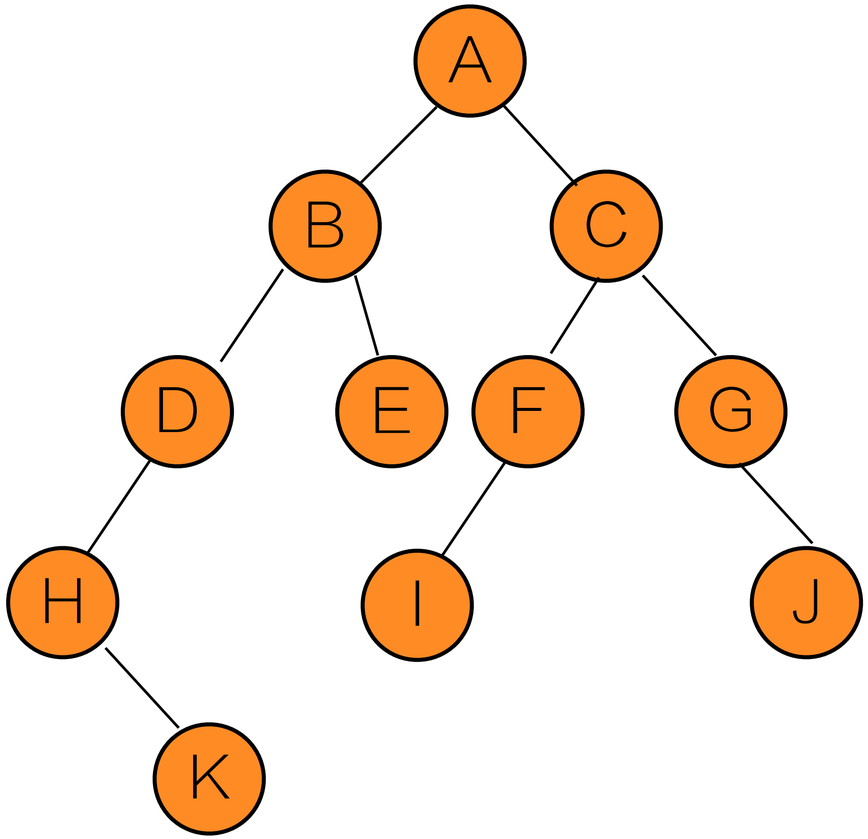


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```





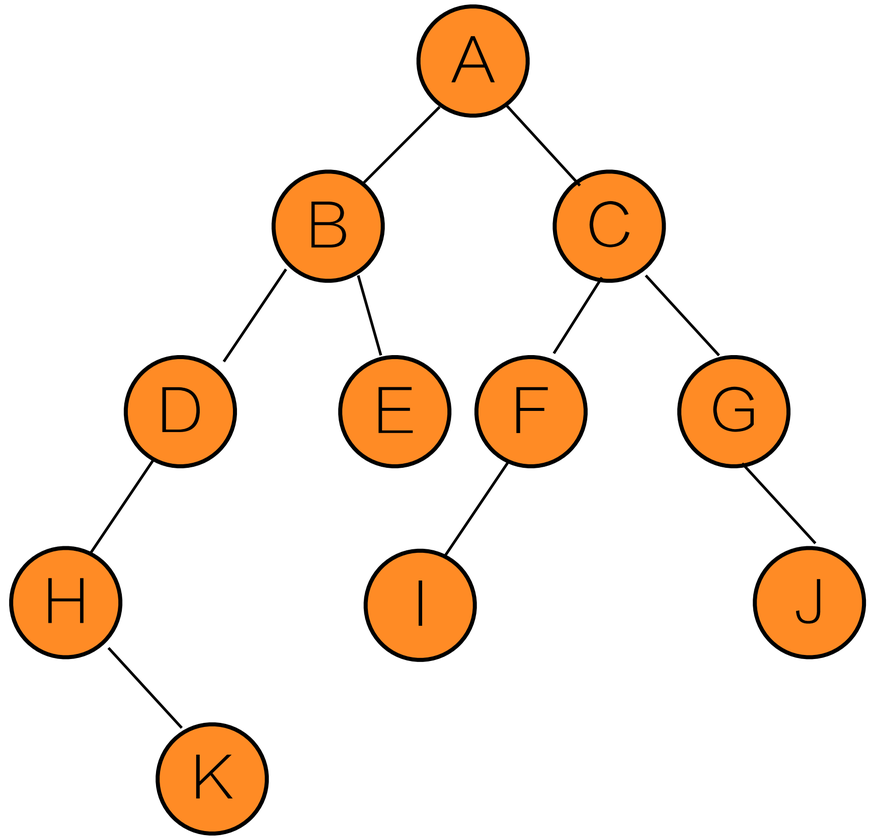
二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```

pre(C) pre(F) pre(G)

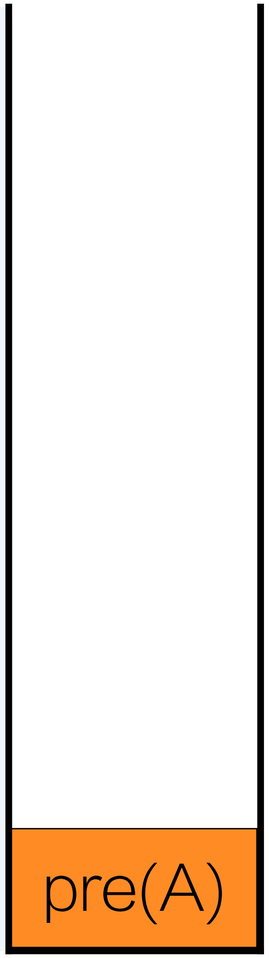
pre(A) pre(B) pre(C)

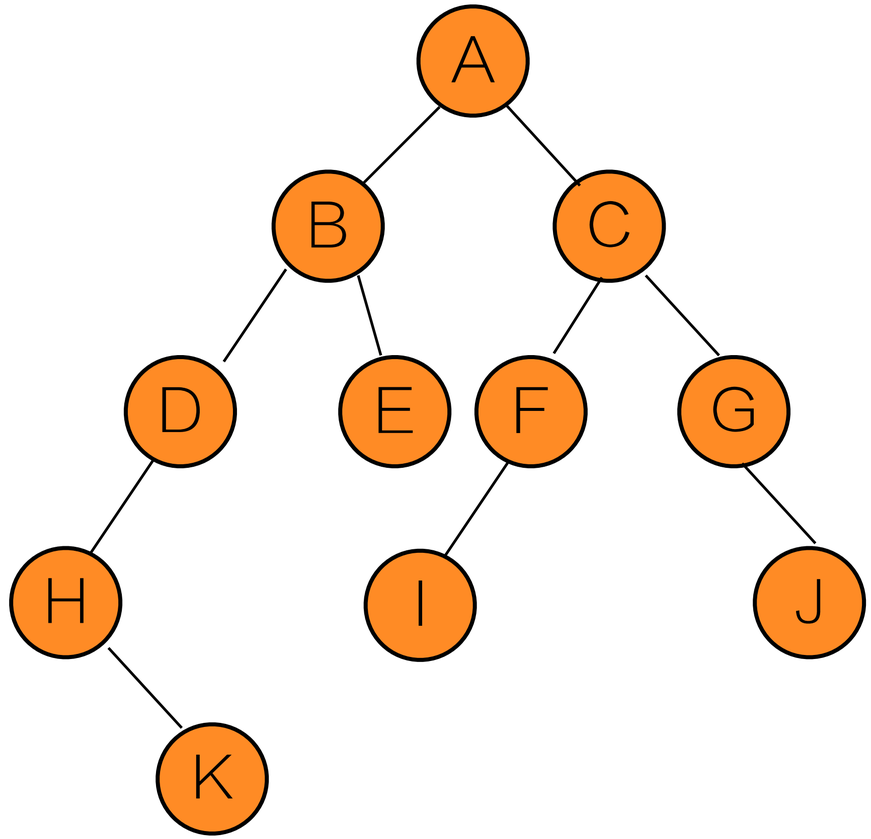


二叉树的遍历 – 前序遍历

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



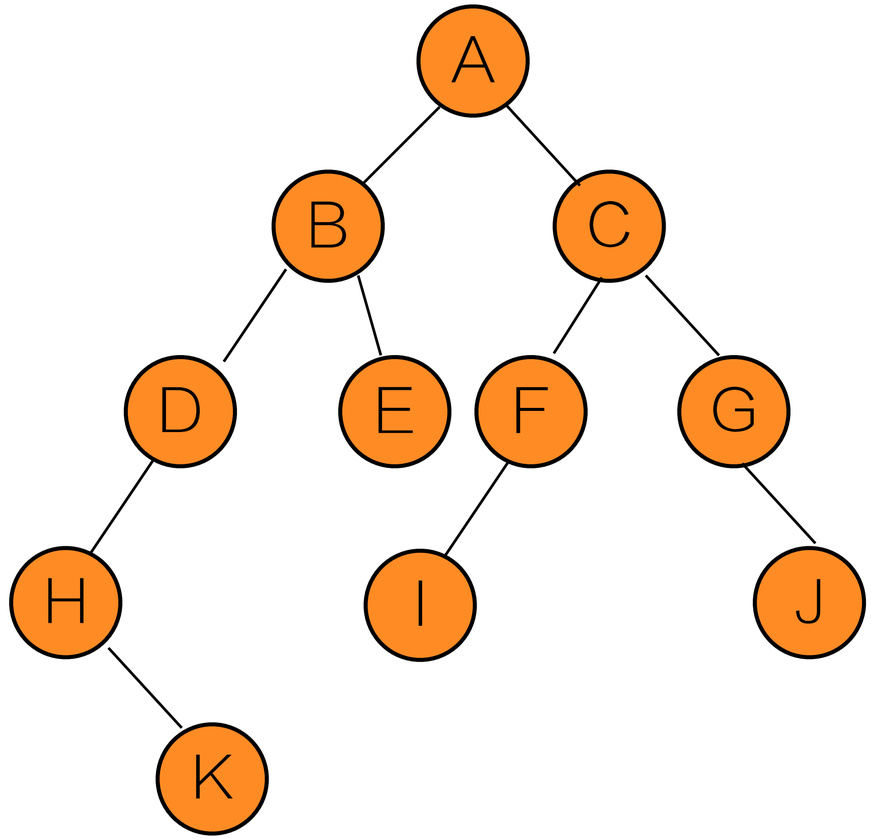


二叉树的遍历 – 前序遍历

案例：树 / test1.c

输出内容：A B D H K E C F I G J

```
void preOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    printf("%c ", T->data);
    preOrder(T->lchild);
    preOrder(T->rchild);
}
```



二叉树的遍历 - 中序遍历

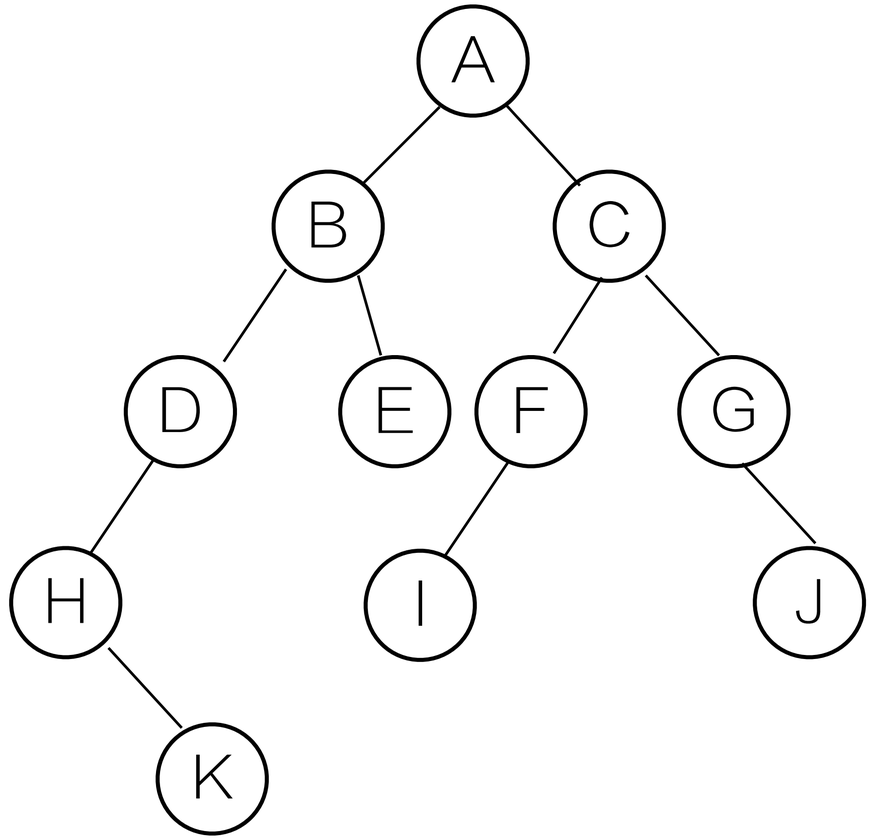
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

先访问根结点，向树的左下方移动，直到遇到空结点为止，然后访问空结点的父结点。接着继续遍历该结点的右子树，如果右子树没的子树可以遍历，那么继续遍历上一层最后一个未被访问的结点

二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

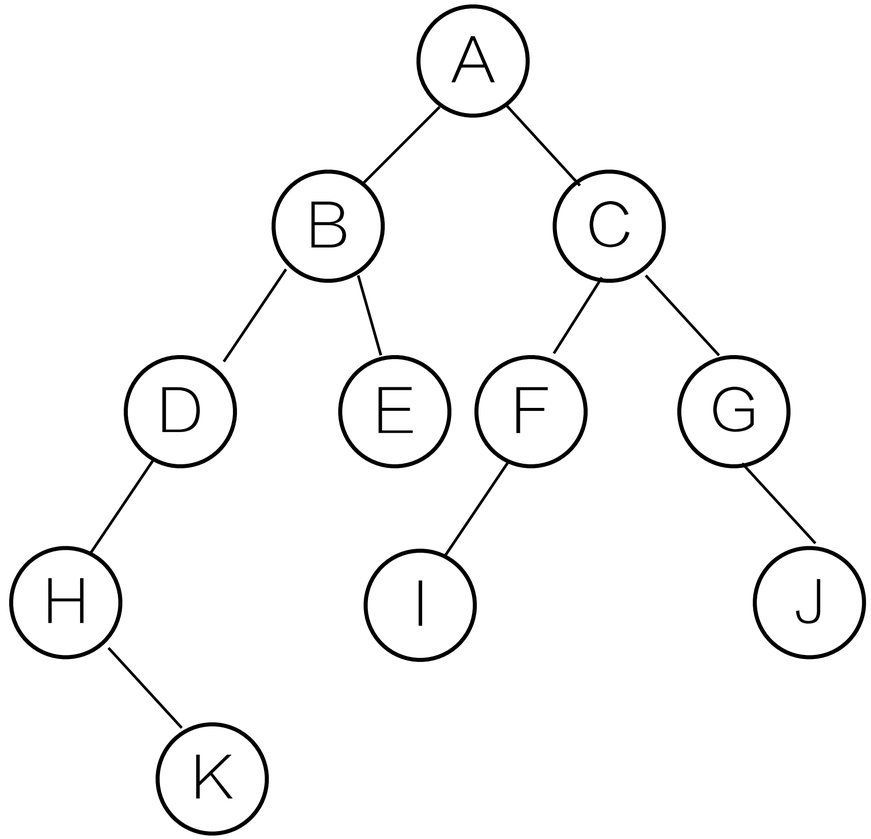


二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

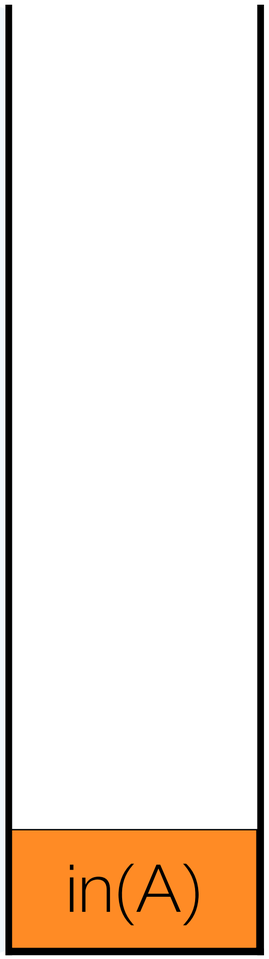
in(A)



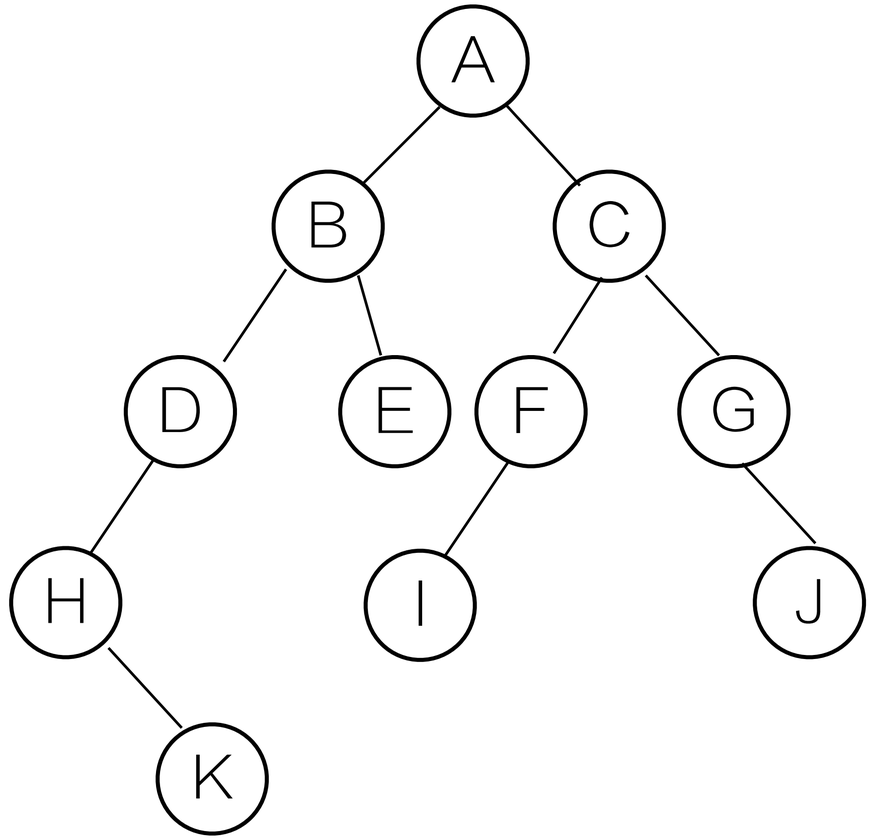
二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



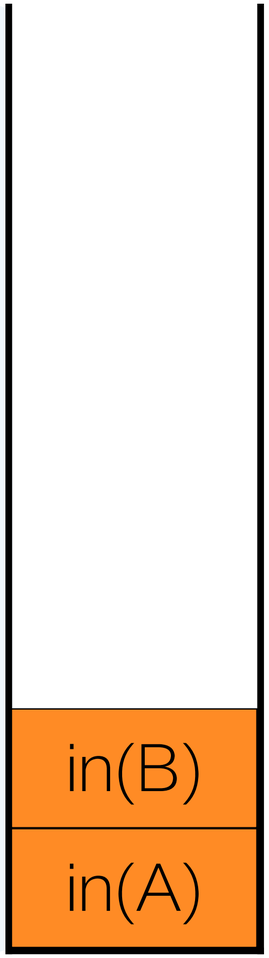
in(B) p in(C)



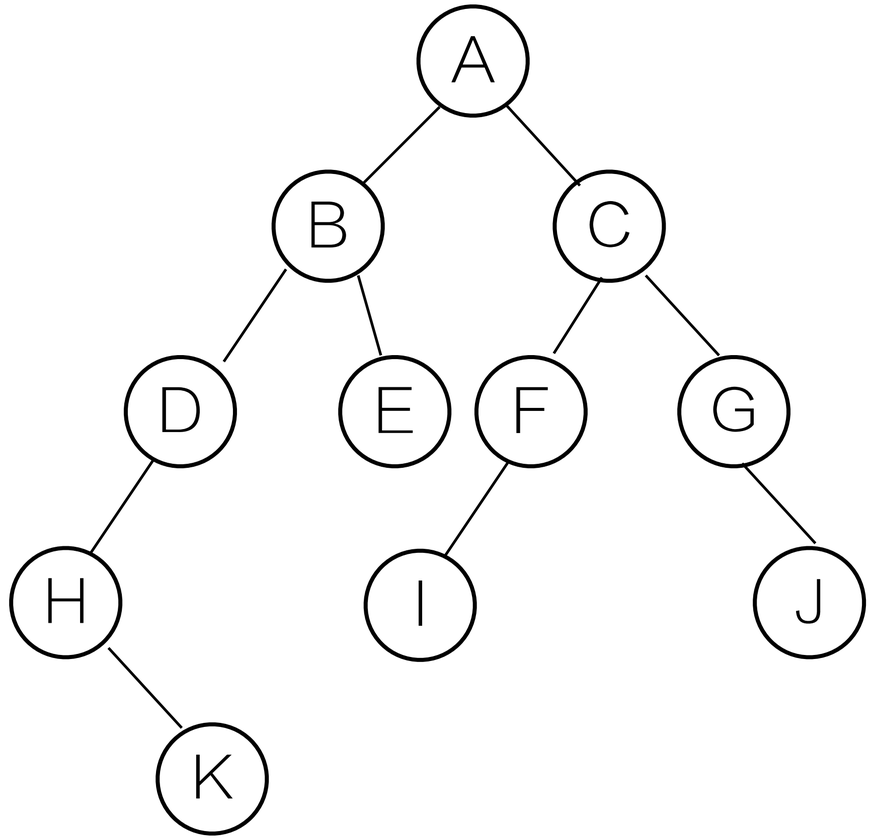
二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



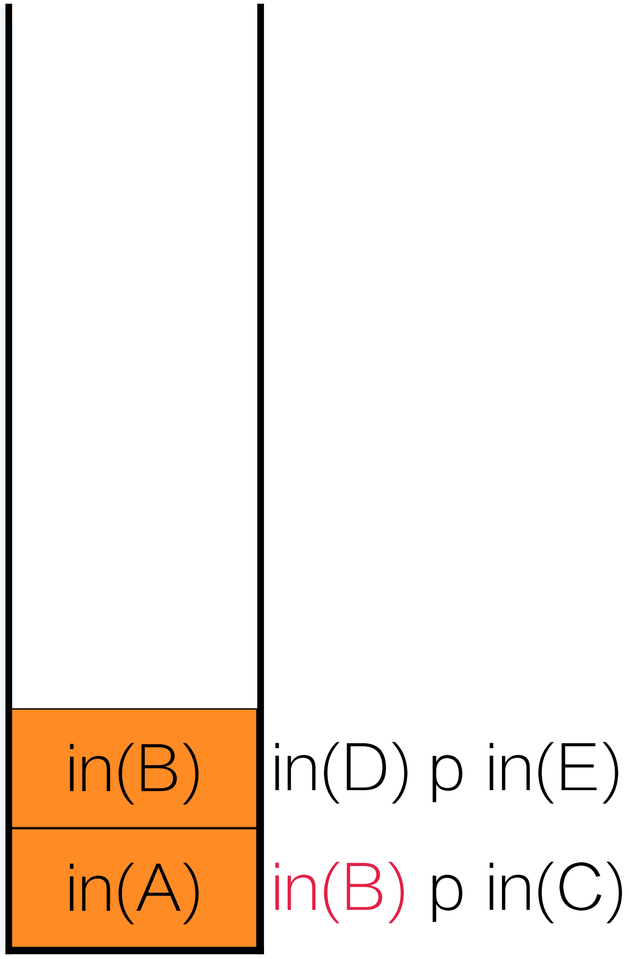
in(B) p in(C)



二叉树的遍历 – 中序遍历

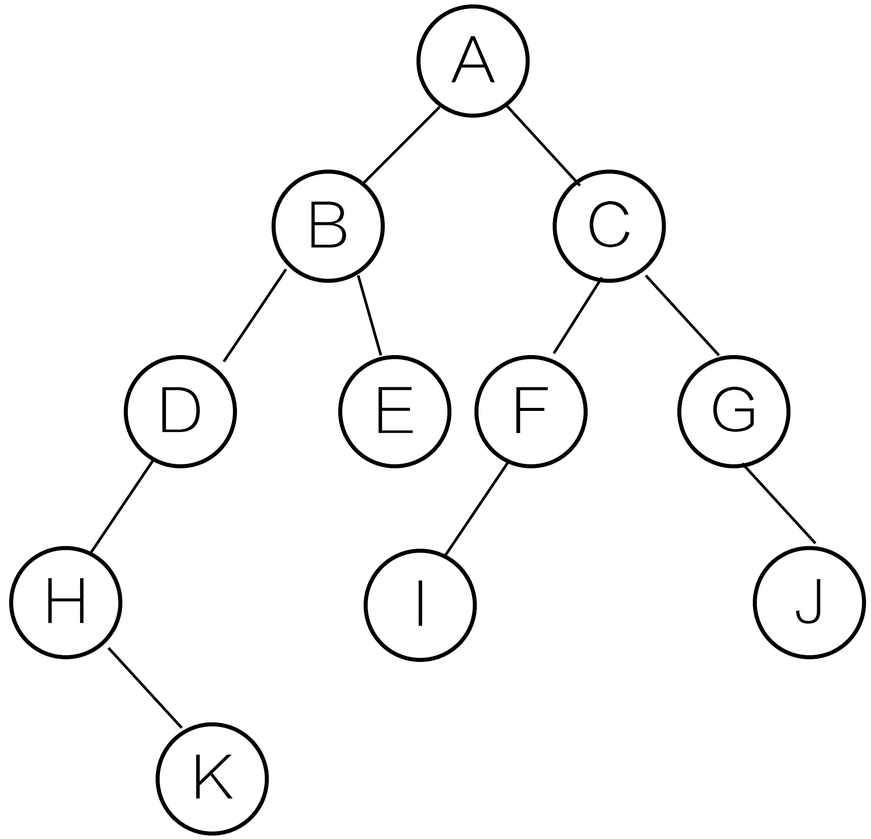
输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(D) p in(E)

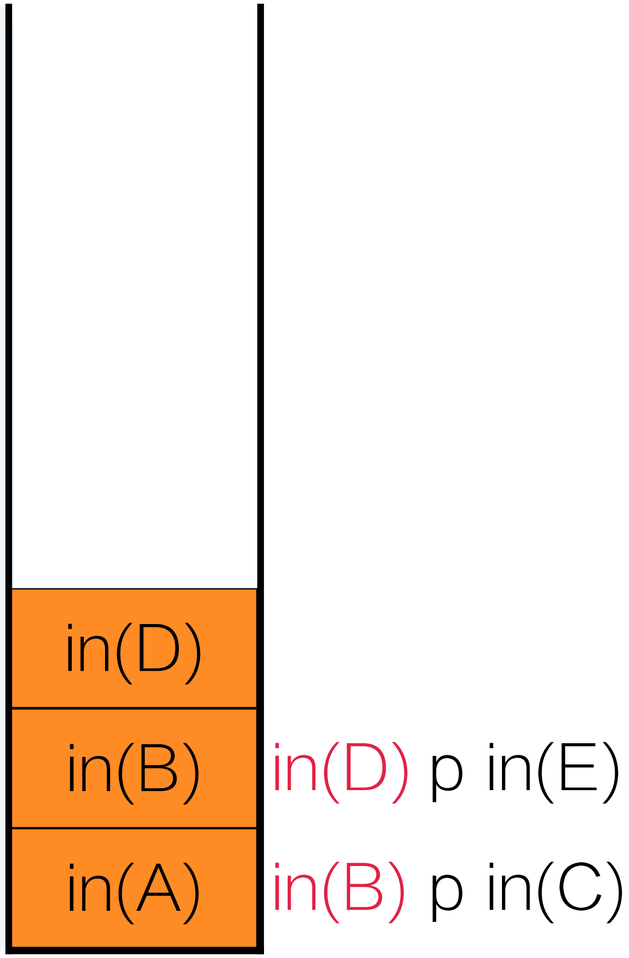
in(B) p in(C)



二叉树的遍历 – 中序遍历

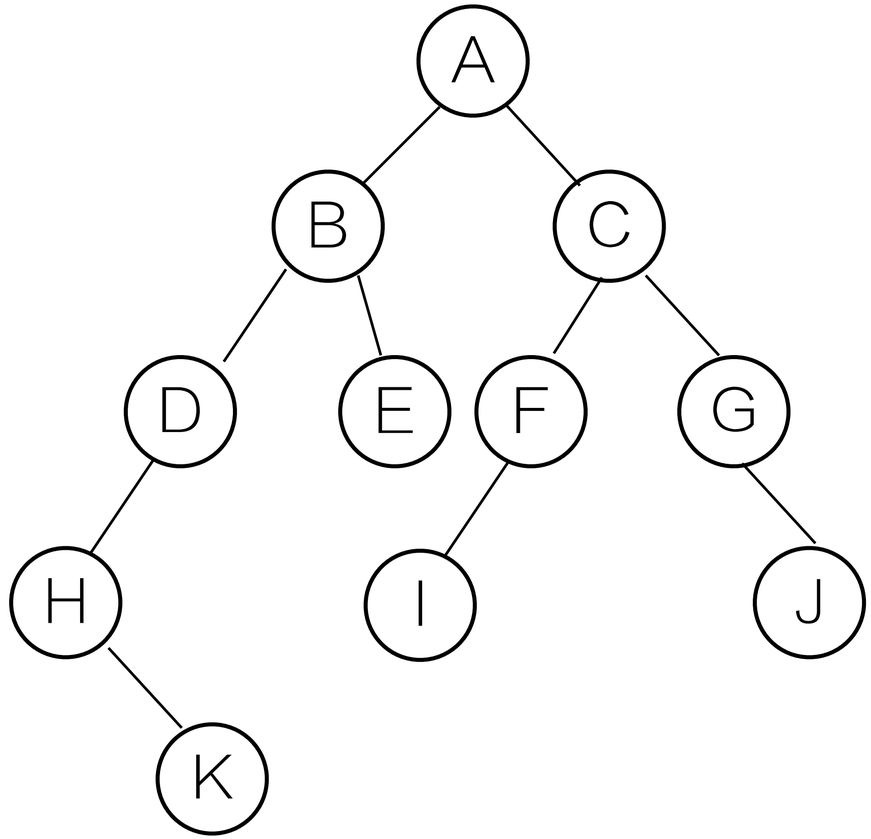
输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(D) p in(E)

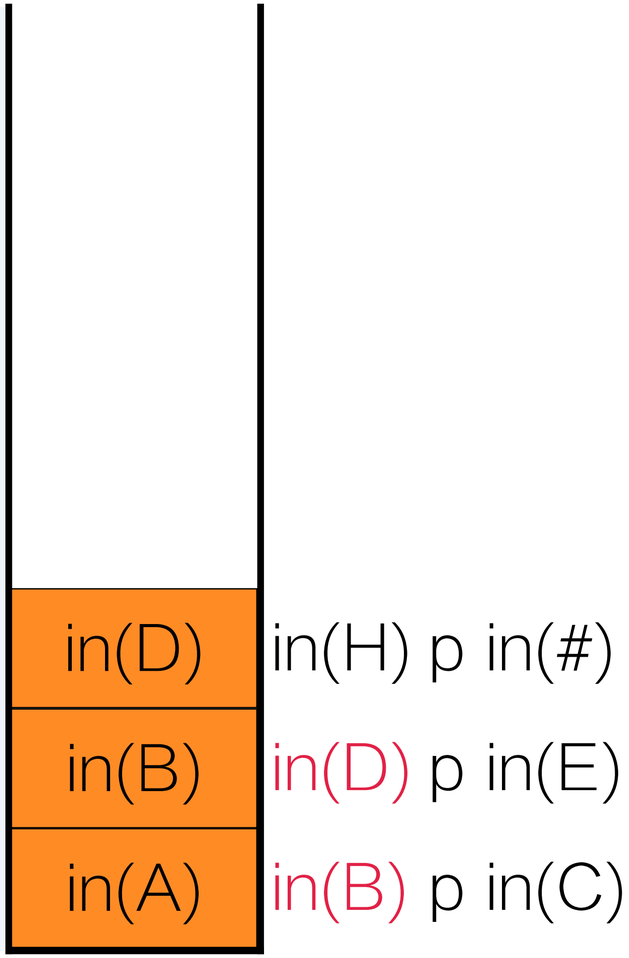
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：

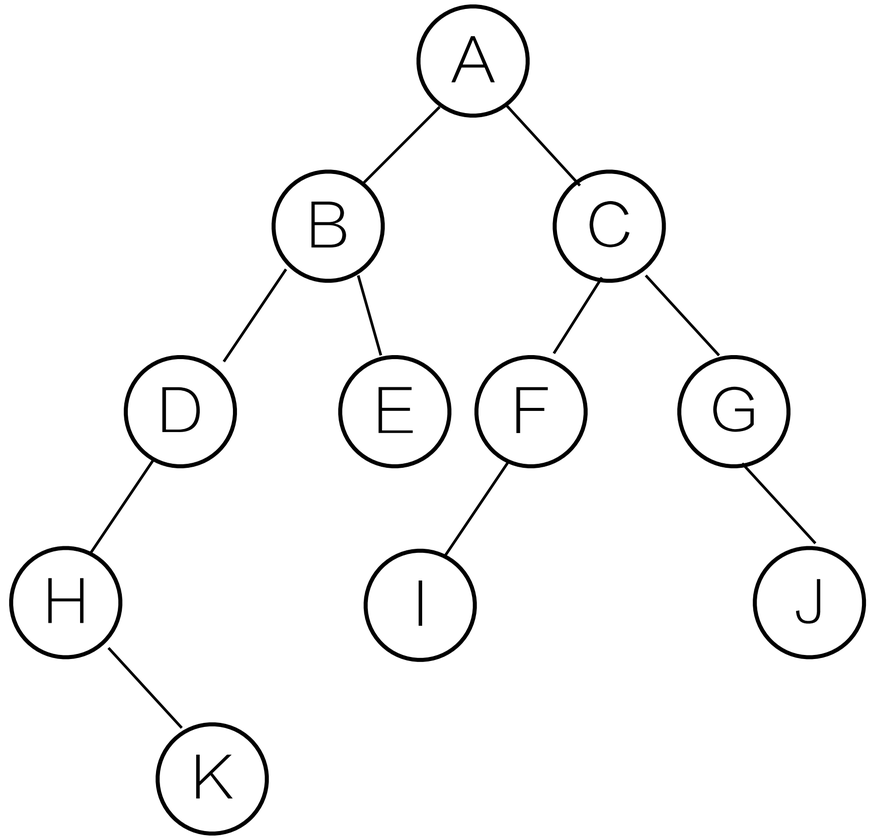
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(H) p in(#)

in(D) p in(E)

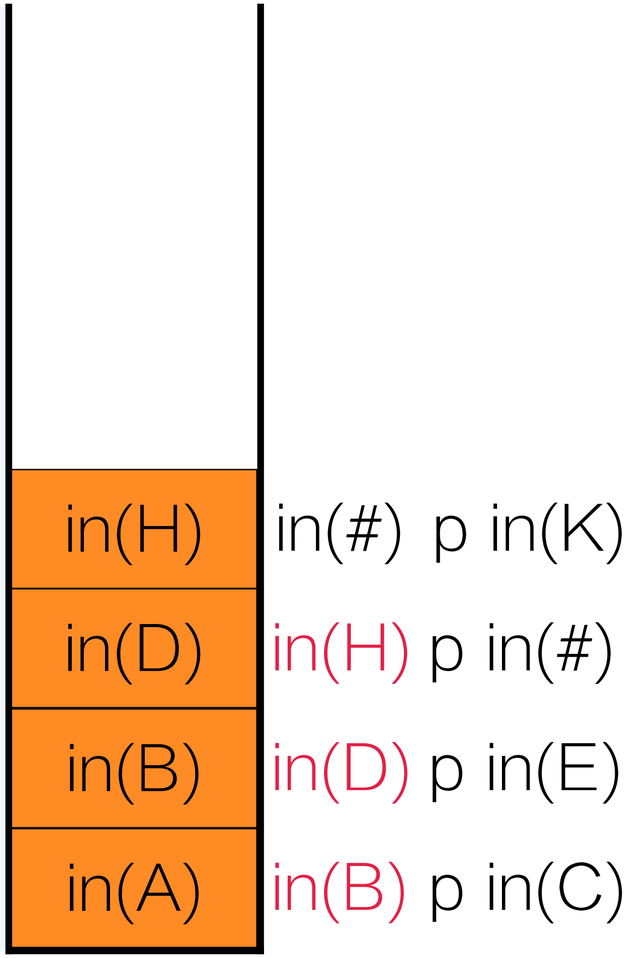
in(B) p in(C)

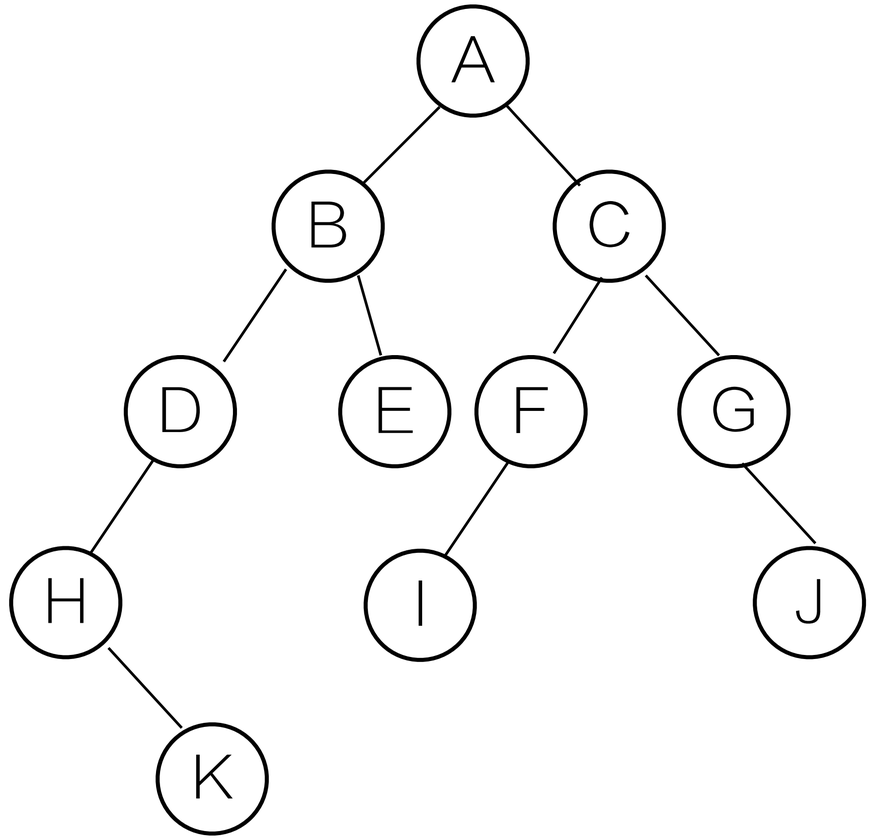


二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

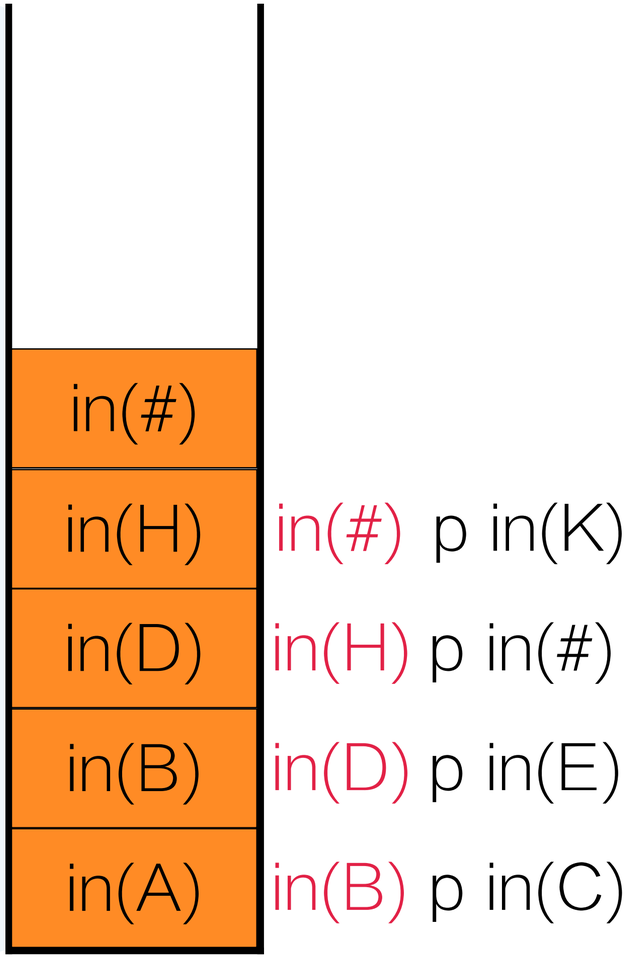


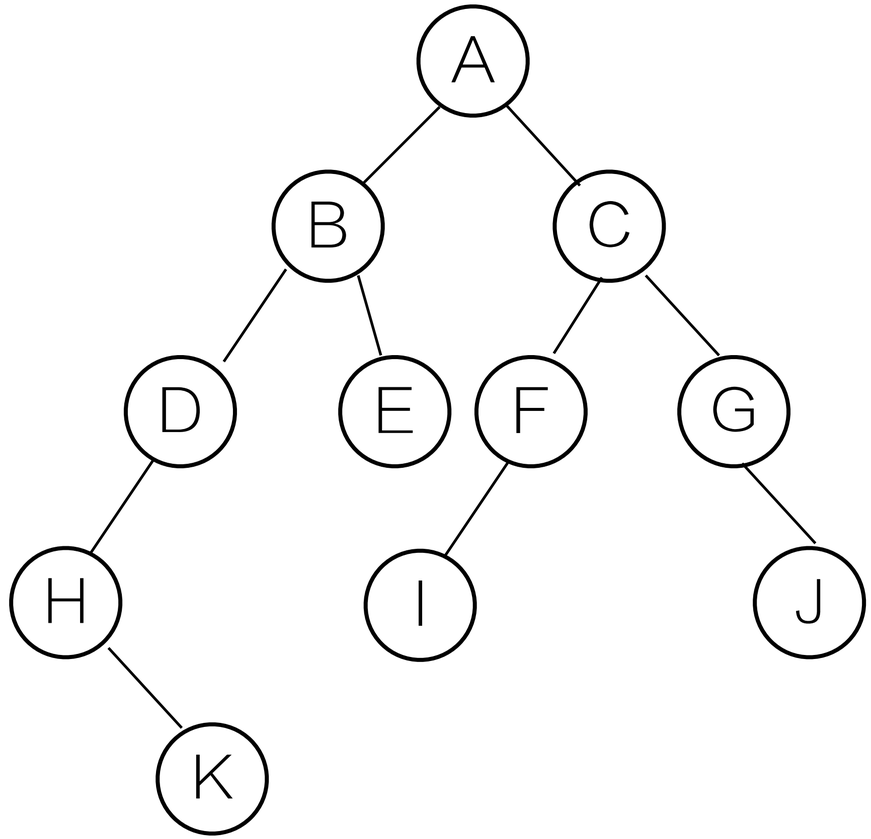


二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

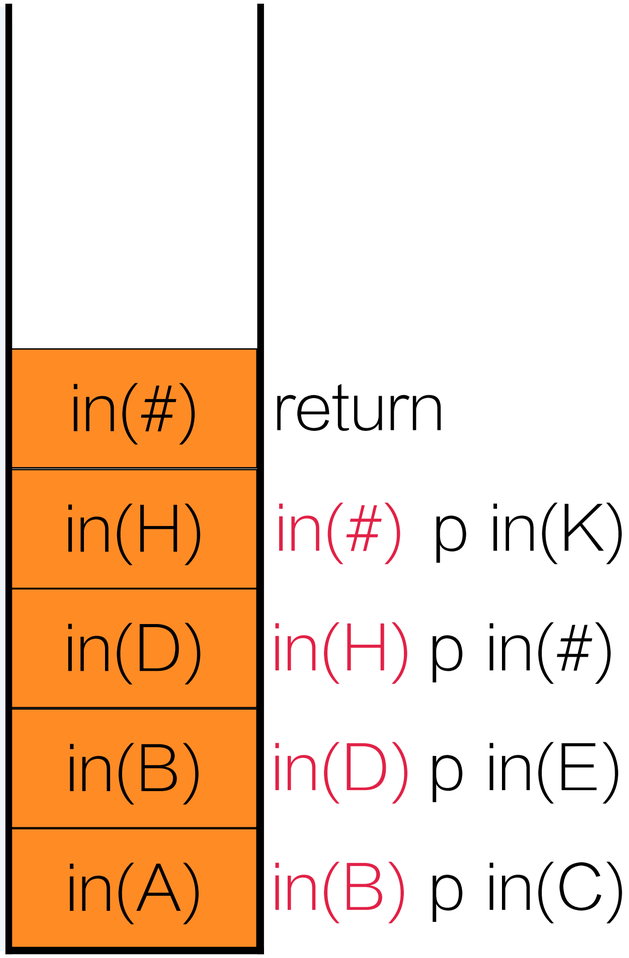


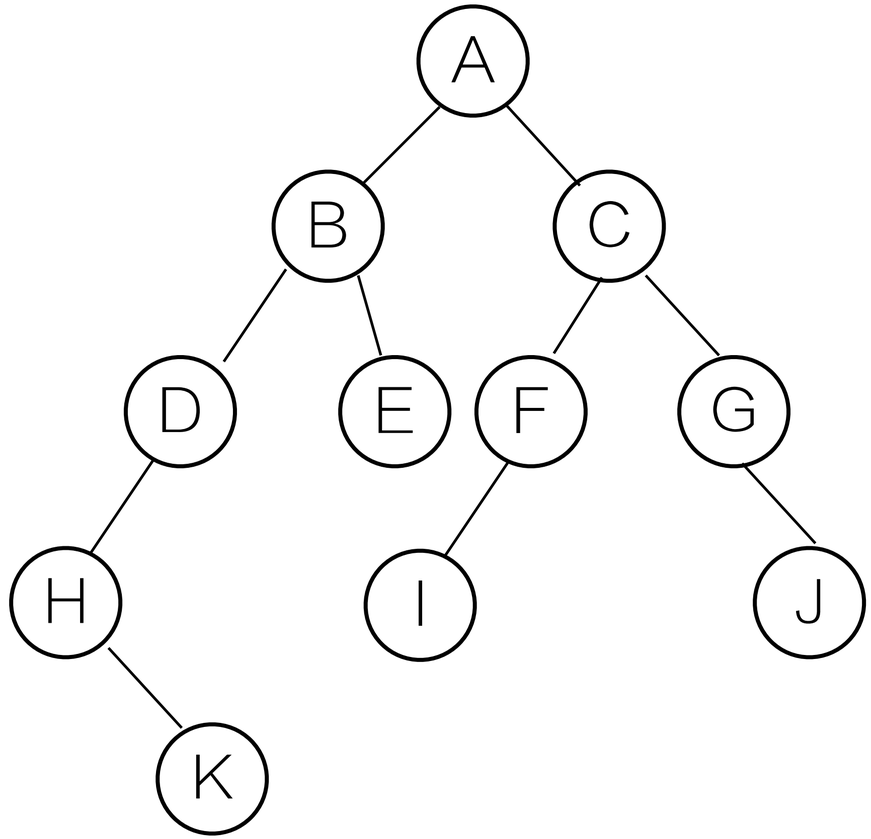


二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

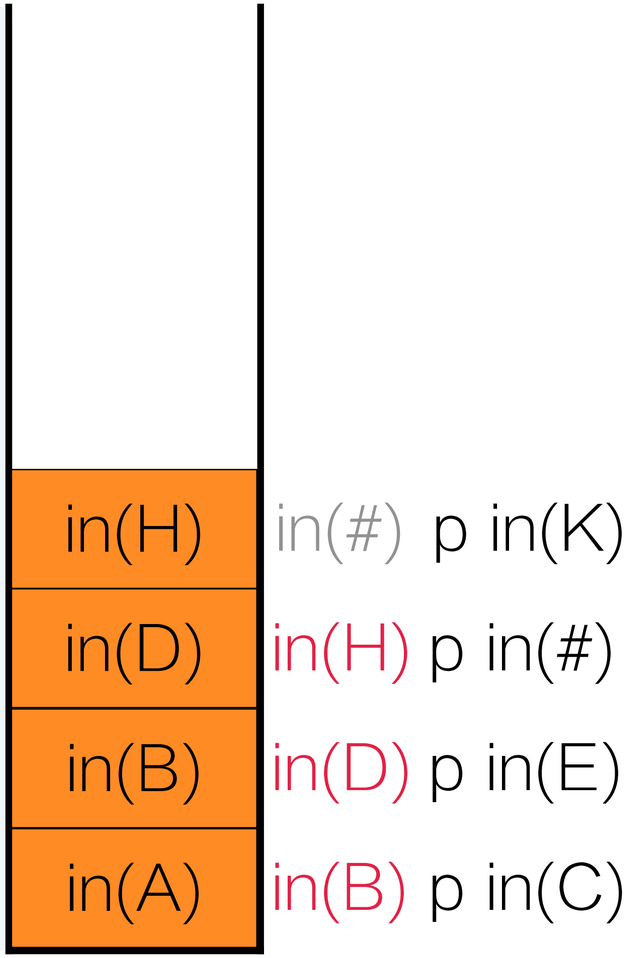




二叉树的遍历 – 中序遍历

输出内容：

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

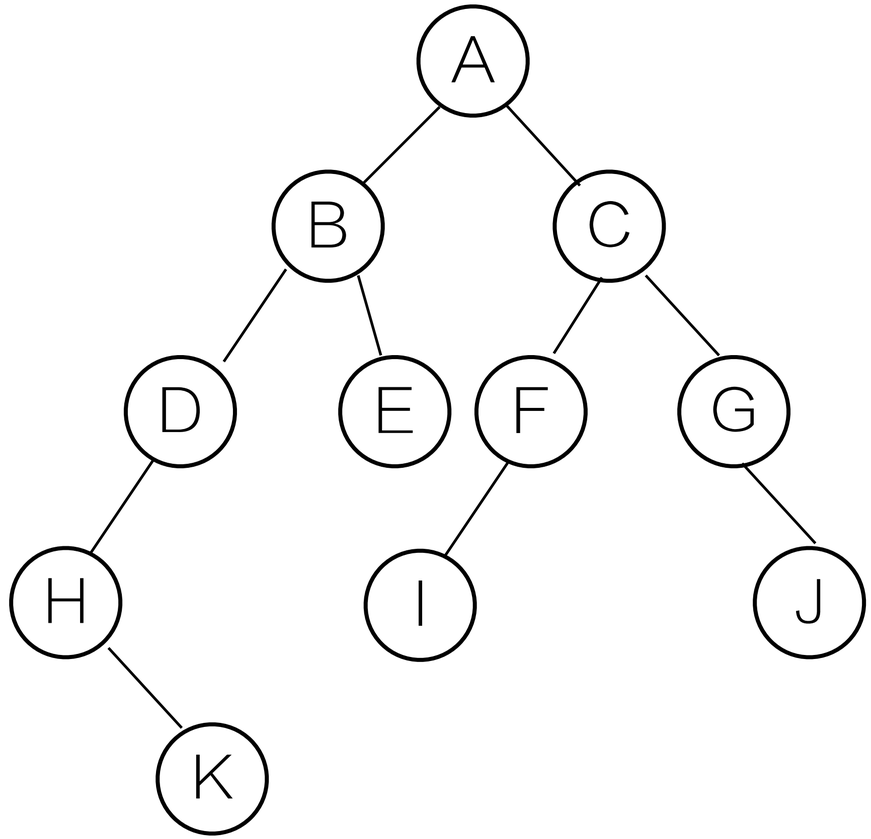


in(#) p in(K)

in(H) p in(#)

in(D) p in(E)

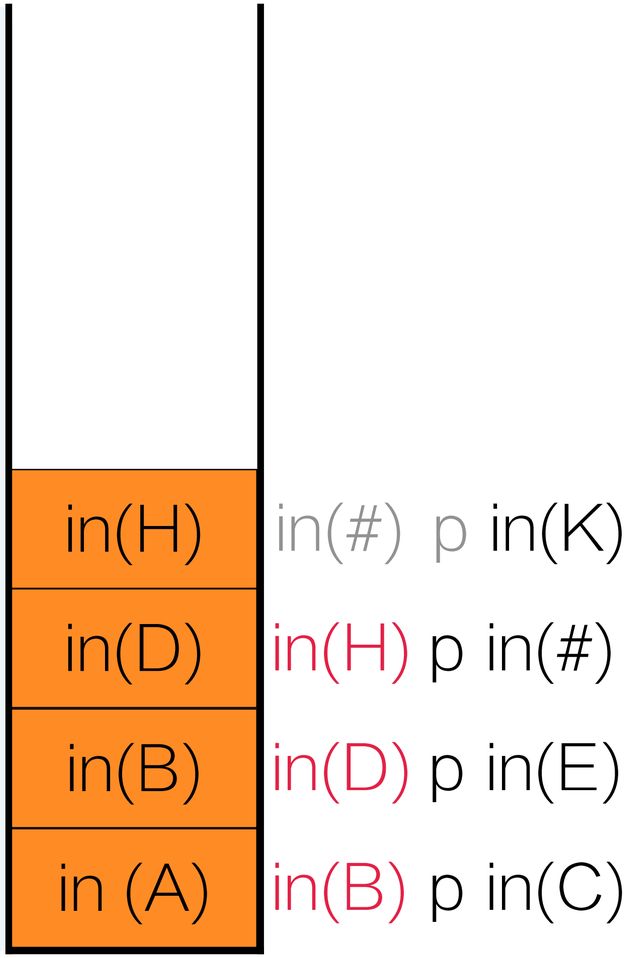
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

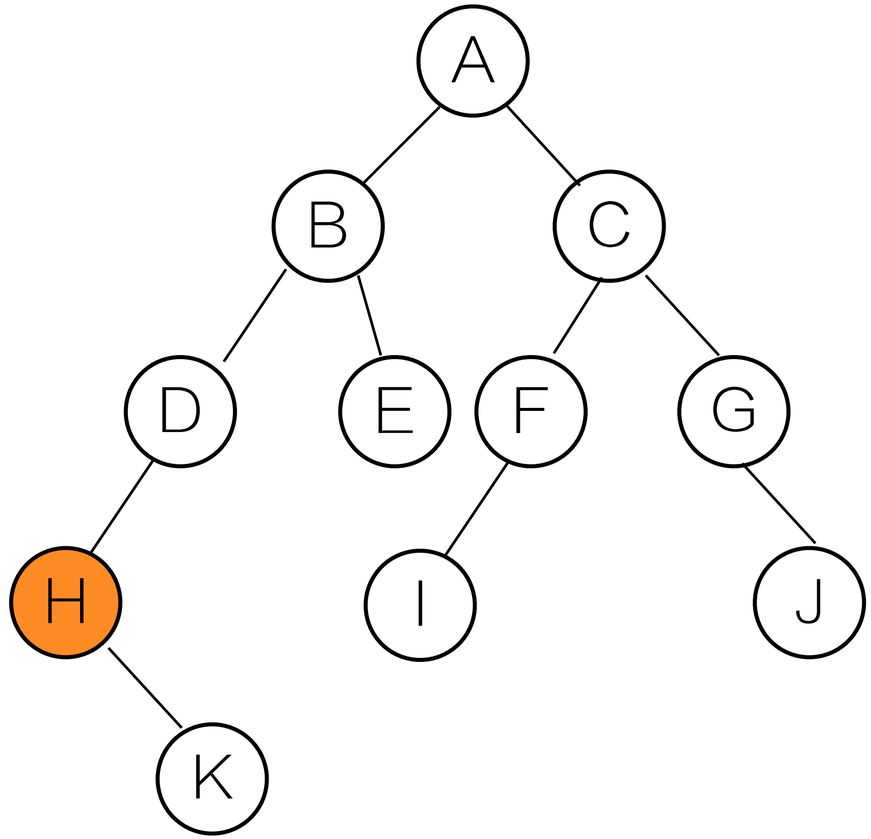


in(#) p in(K)

in(H) p in(#)

in(D) p in(E)

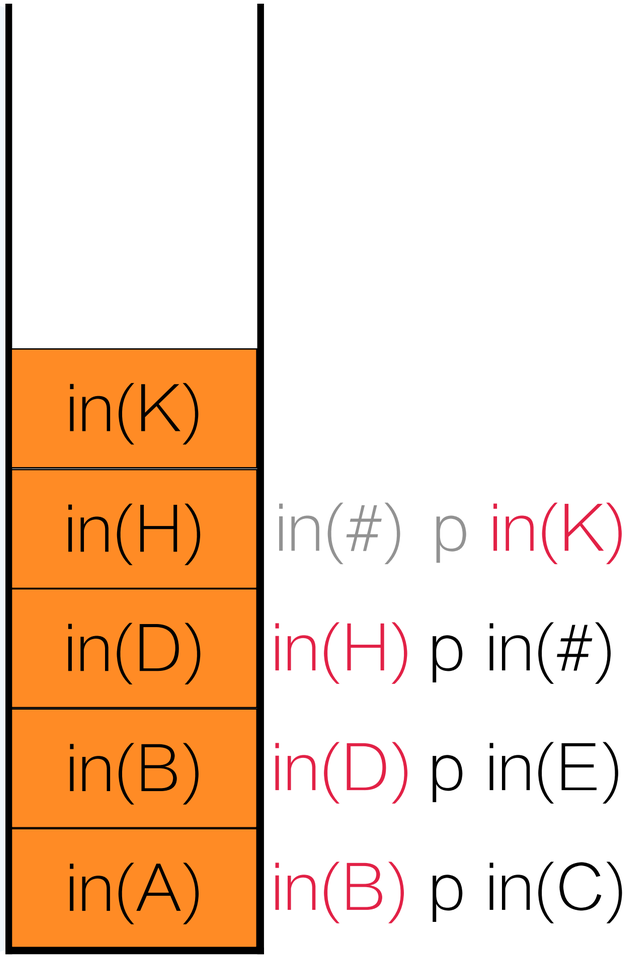
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

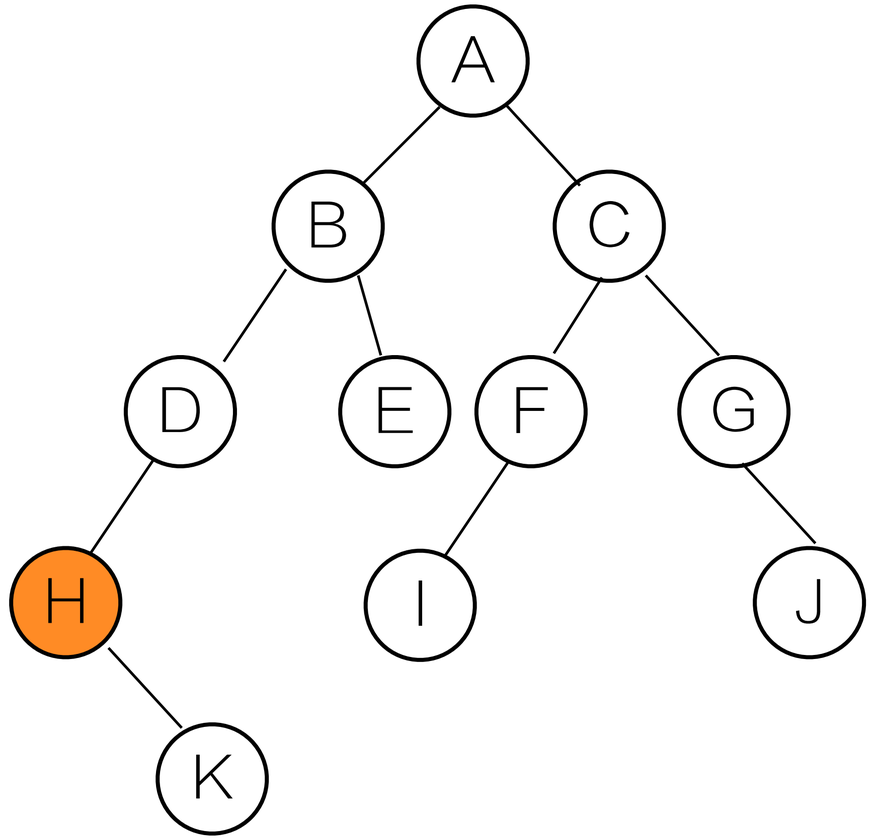


in(#) p in(K)

in(H) p in(#)

in(D) p in(E)

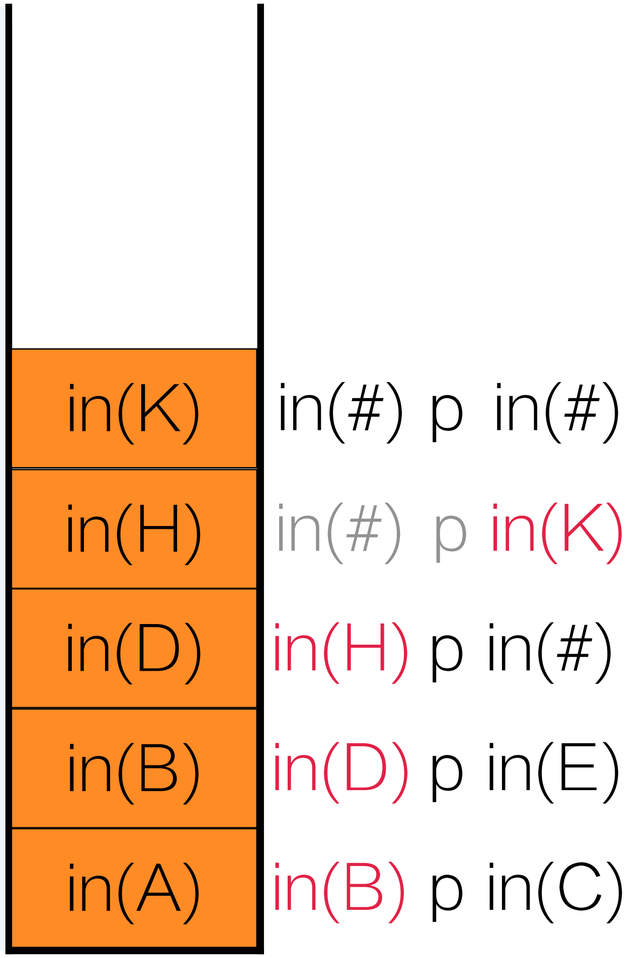
in(B) p in(C)

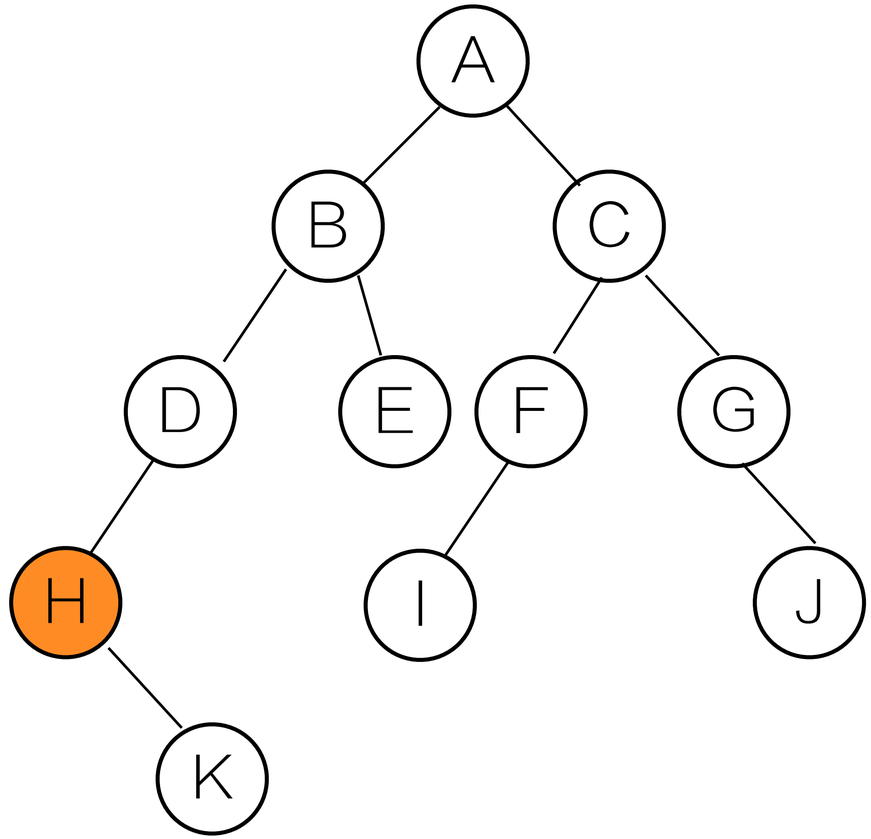


二叉树的遍历 – 中序遍历

输出内容：H

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

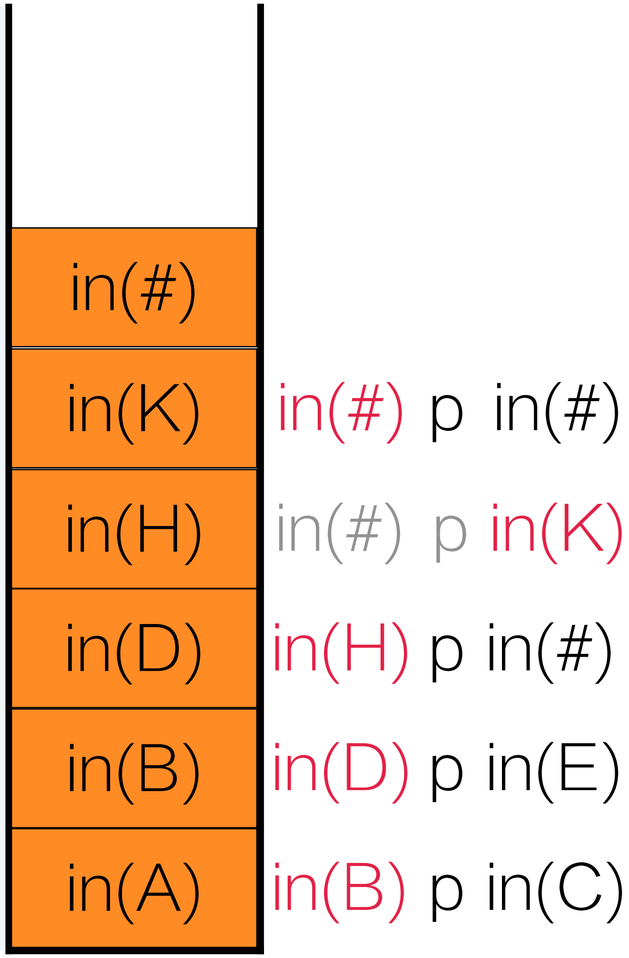


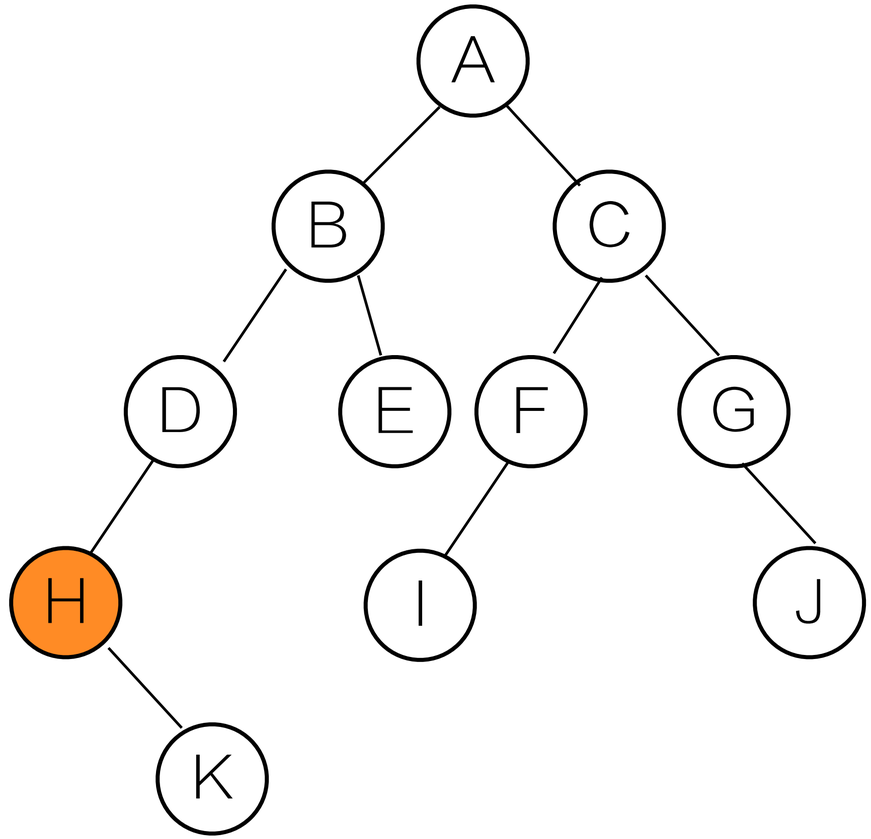


二叉树的遍历 – 中序遍历

输出内容：H

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

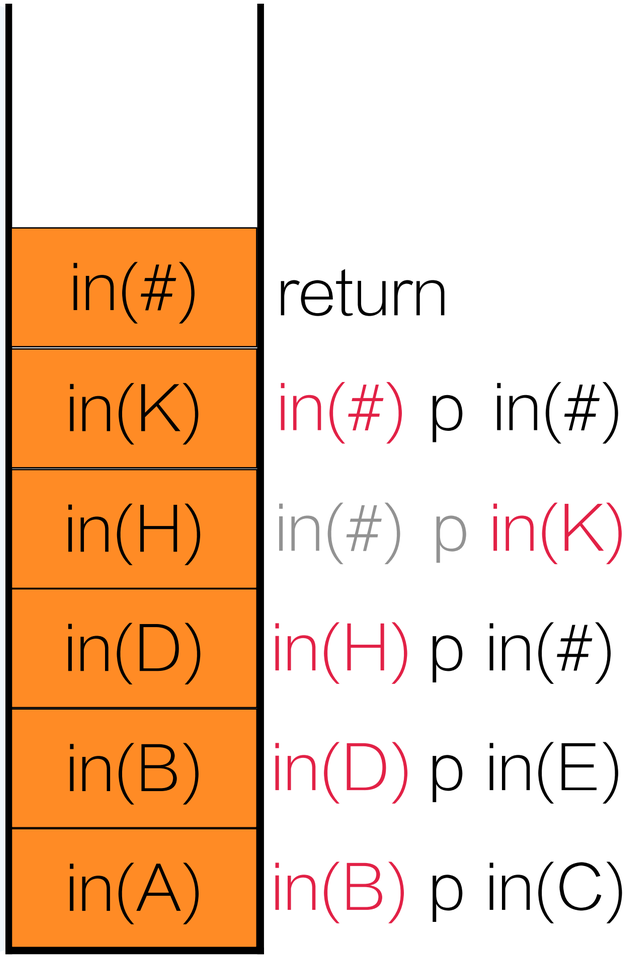




二叉树的遍历 – 中序遍历

输出内容：H

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



return

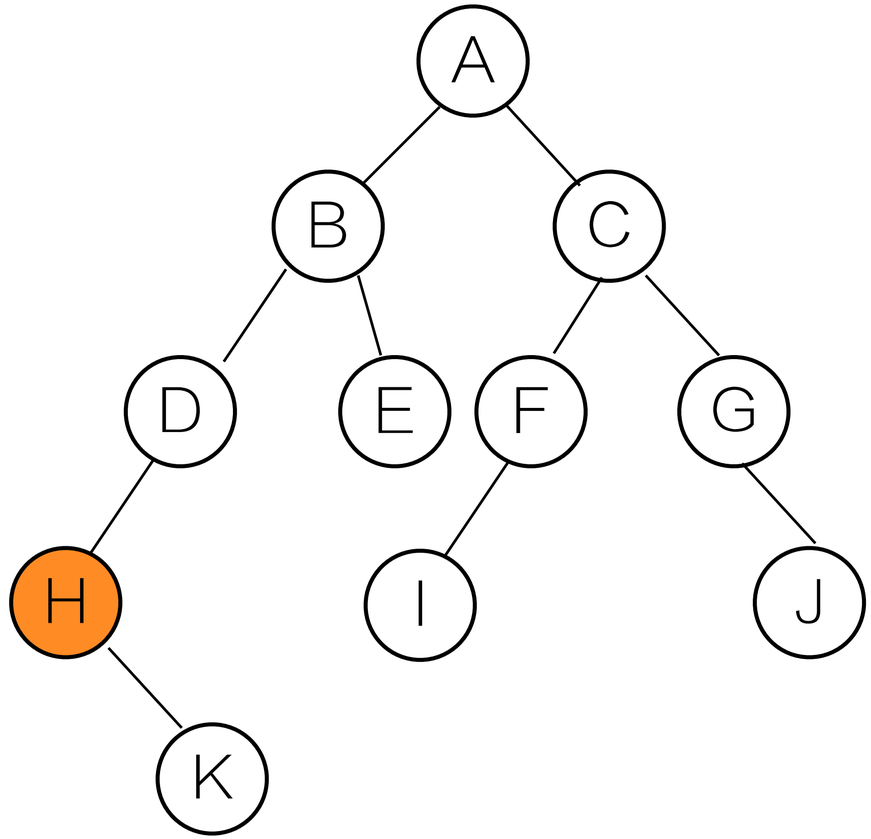
in(#) p in(#)

in(#) p in(K)

in(H) p in(#)

in(D) p in(E)

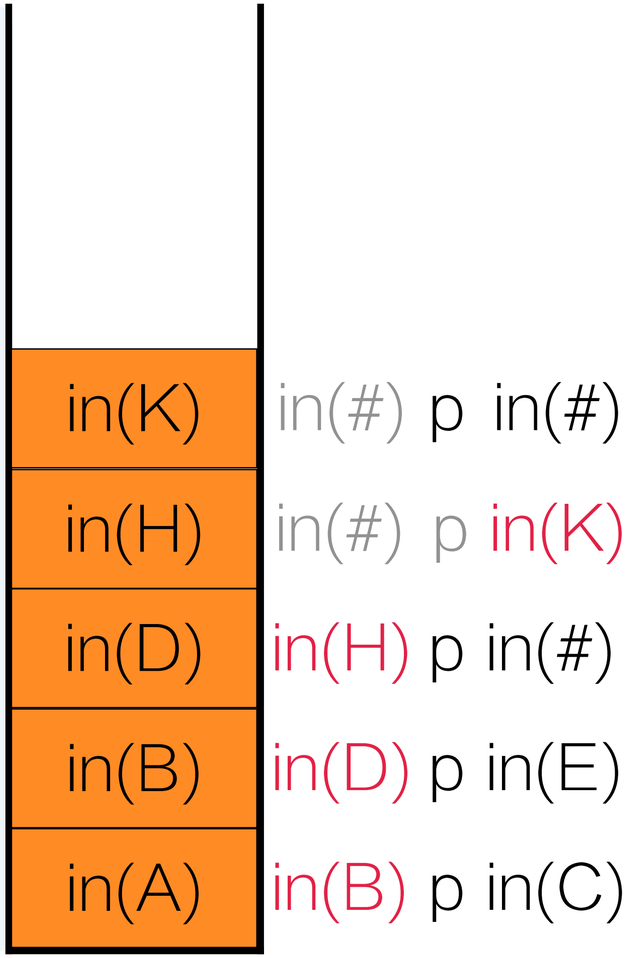
in(B) p in(C)

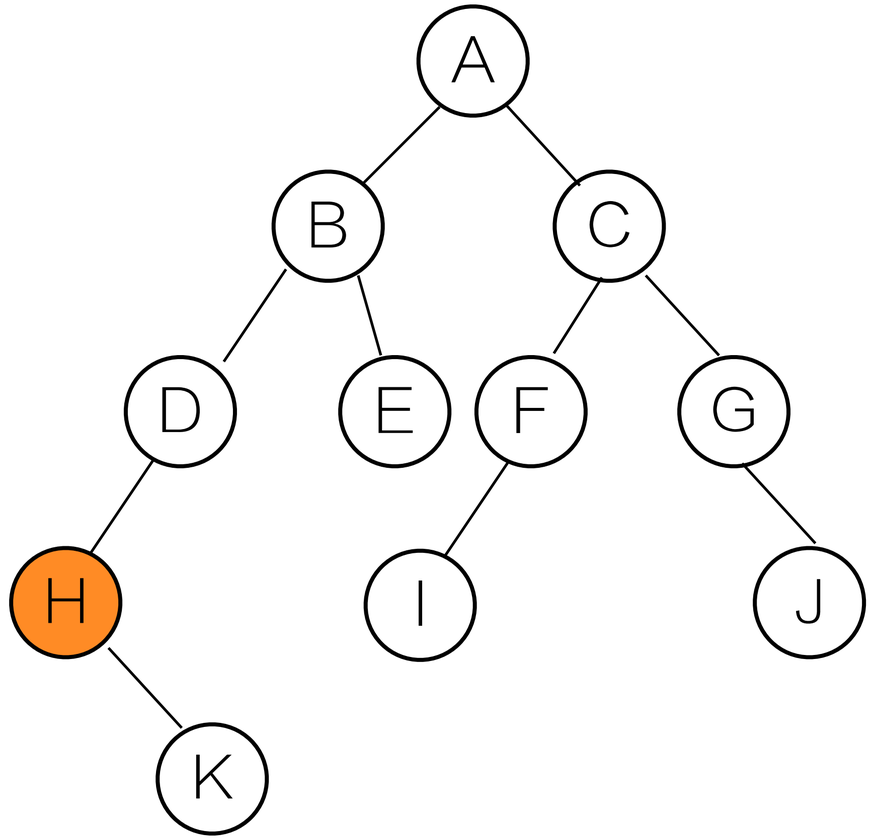


二叉树的遍历 – 中序遍历

输出内容：H

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

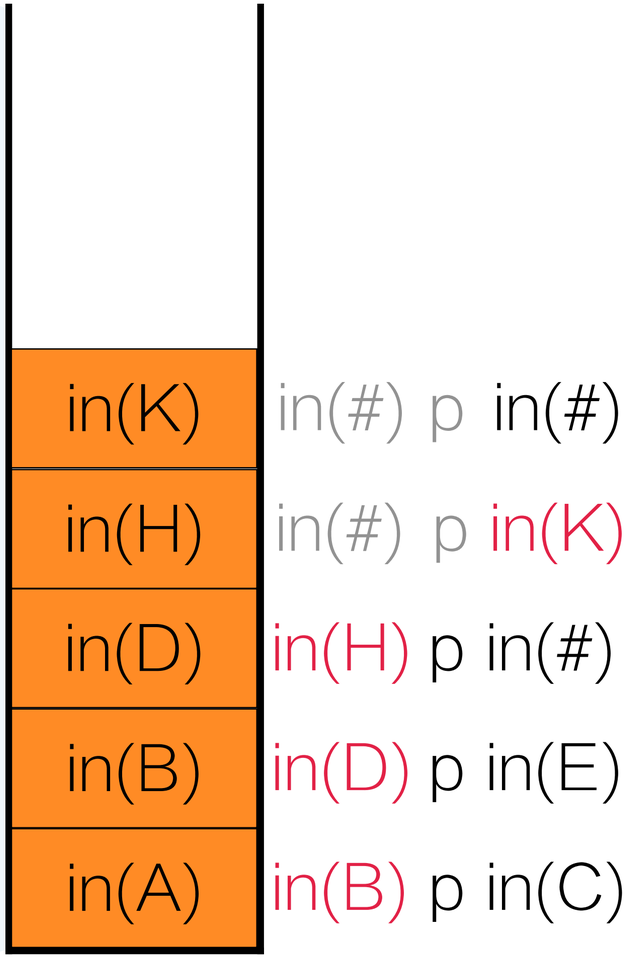


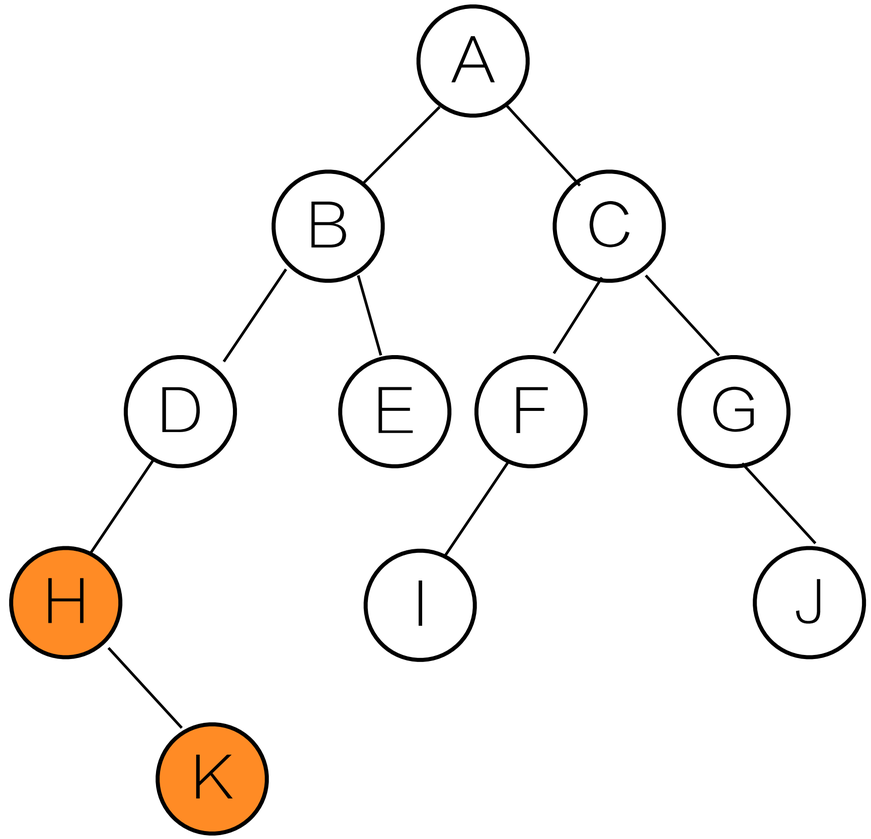


二叉树的遍历 – 中序遍历

输出内容：H K

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

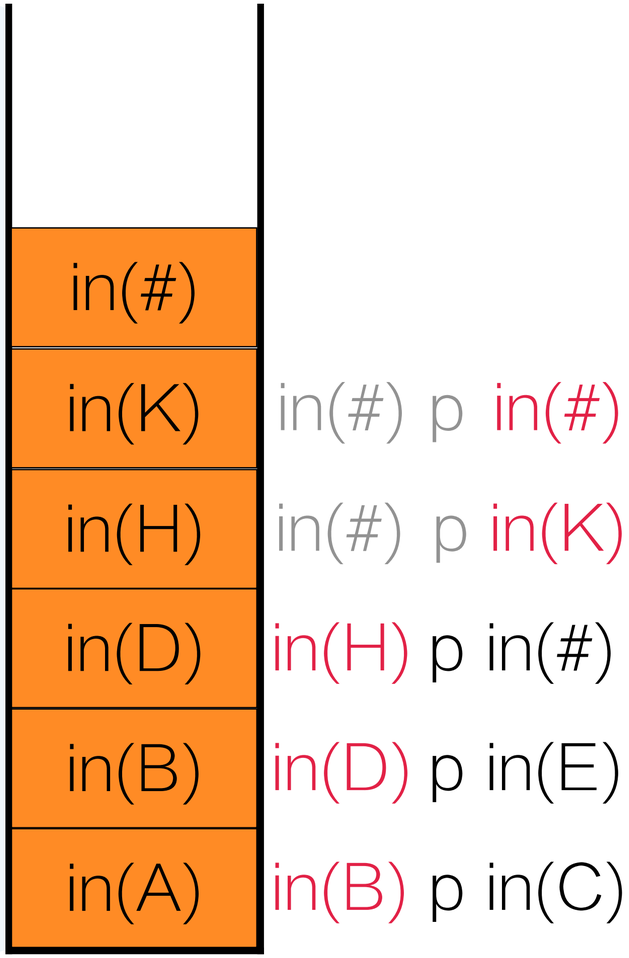




二叉树的遍历 – 中序遍历

输出内容：H K

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



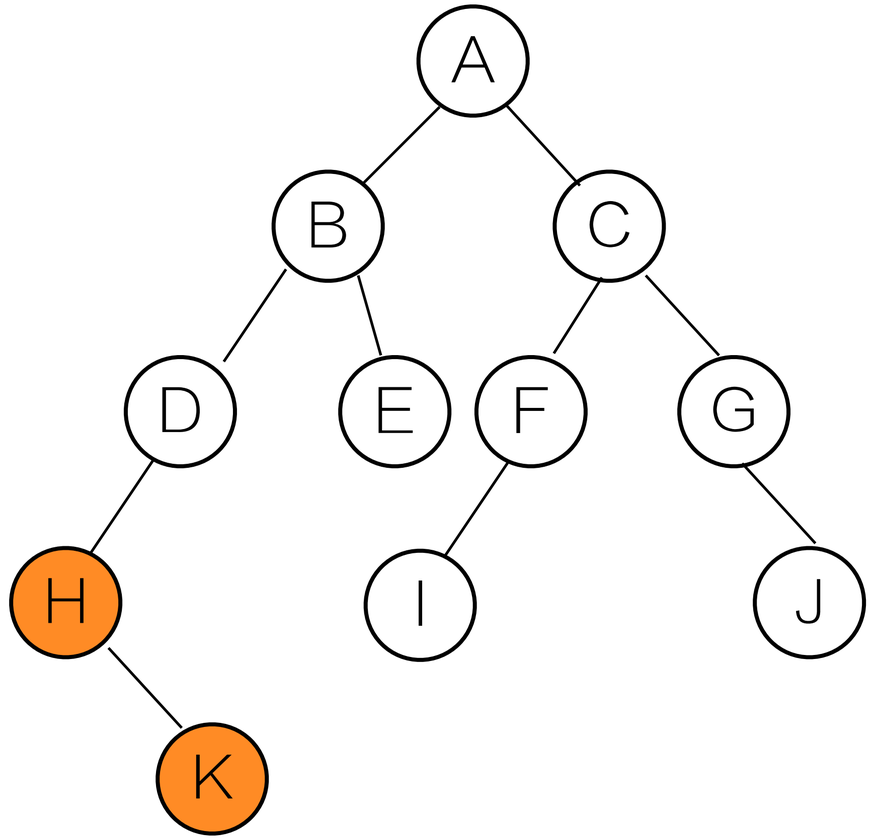
in(#) p in(#)

in(#) p in(K)

in(H) p in(#)

in(D) p in(E)

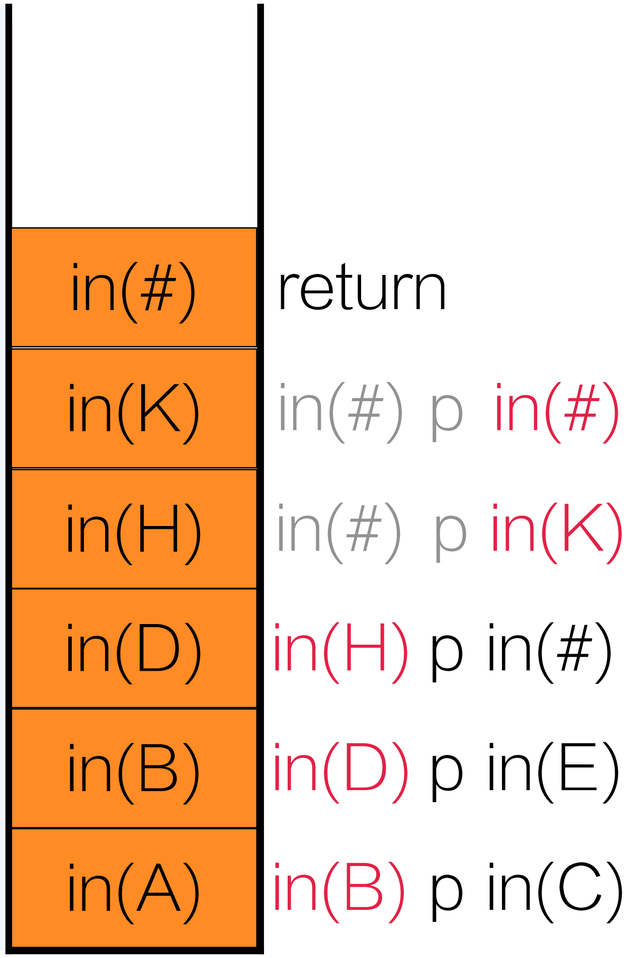
in(B) p in(C)

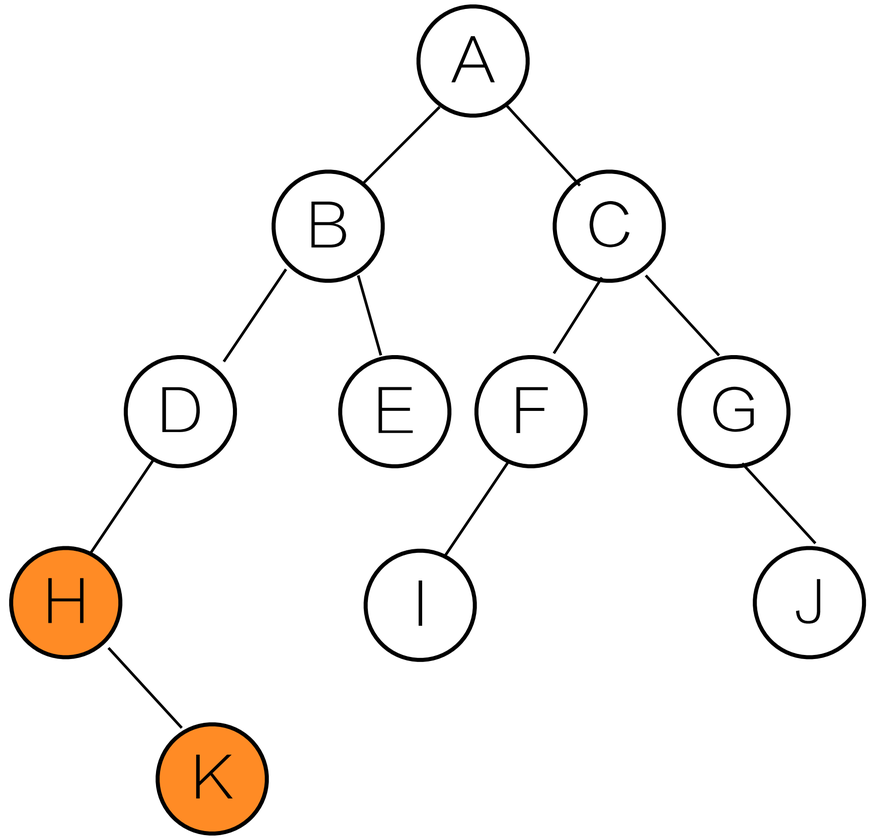


二叉树的遍历 – 中序遍历

输出内容：H K

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

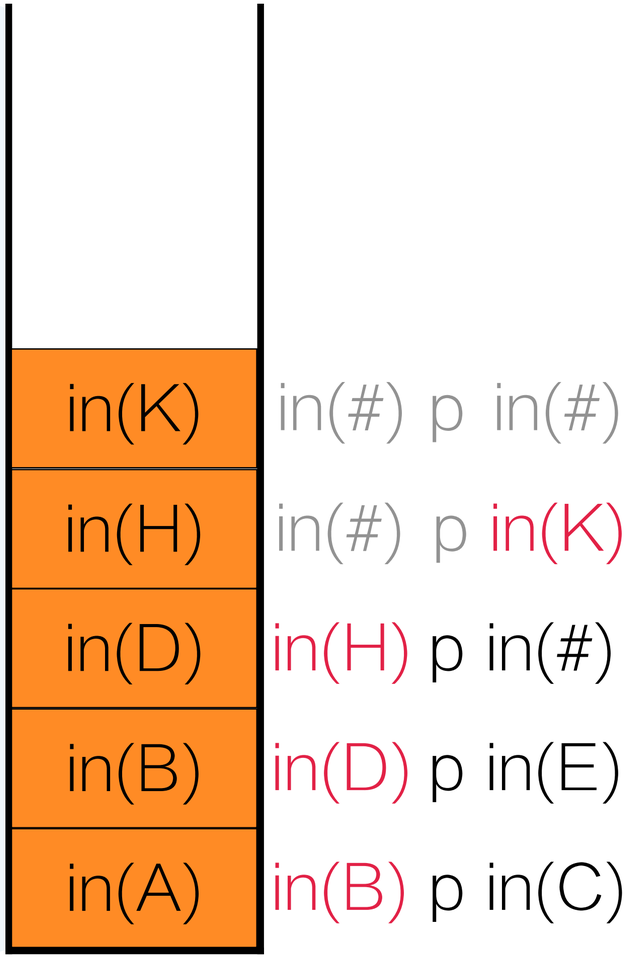


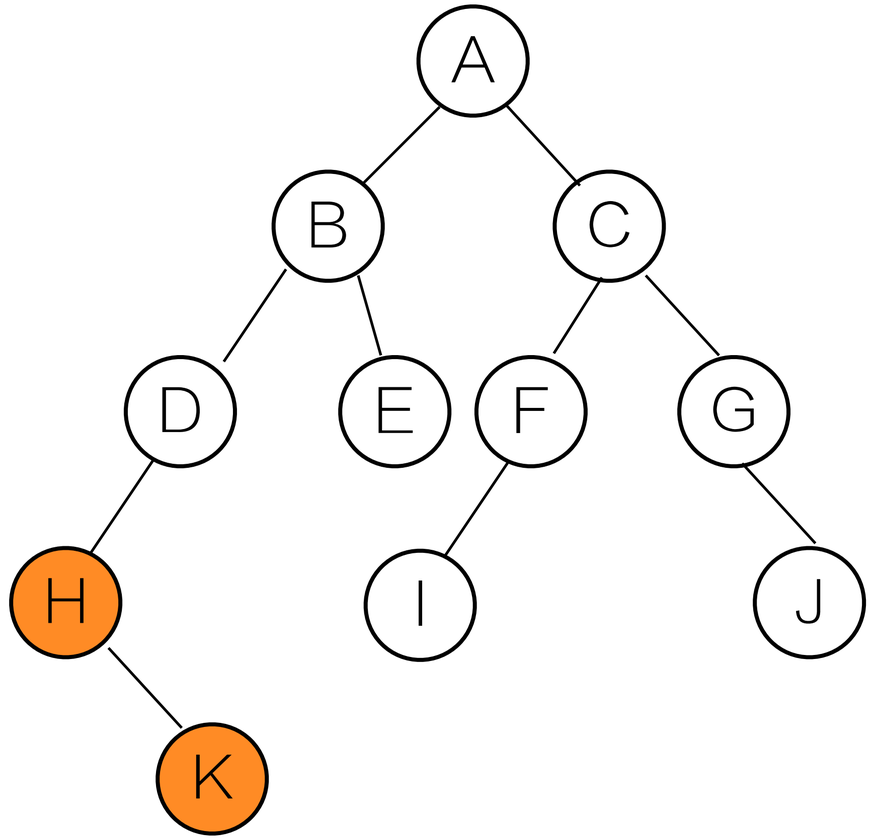


二叉树的遍历 – 中序遍历

输出内容：H K

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

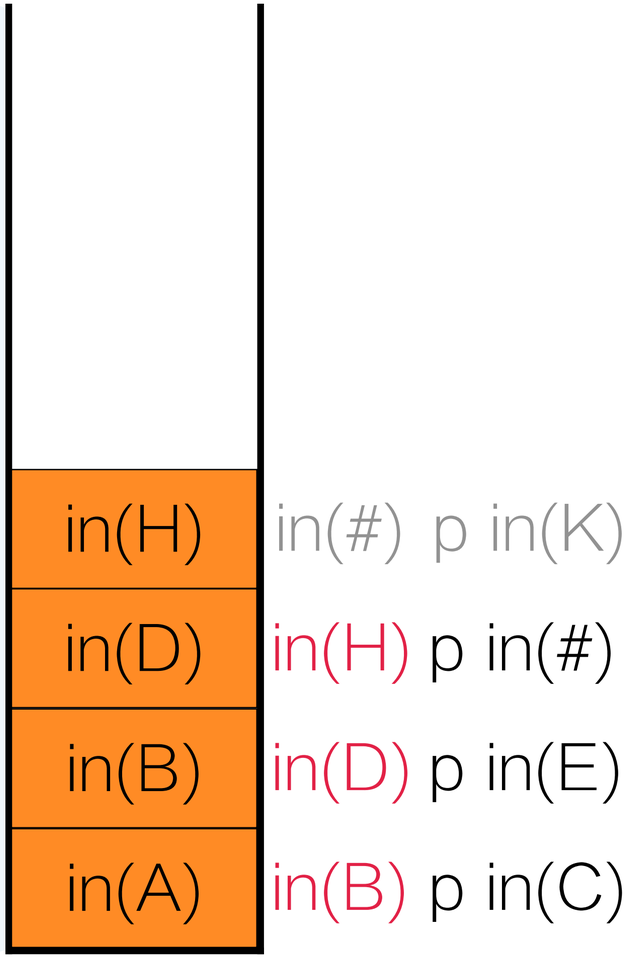




二叉树的遍历 – 中序遍历

输出内容：H K

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

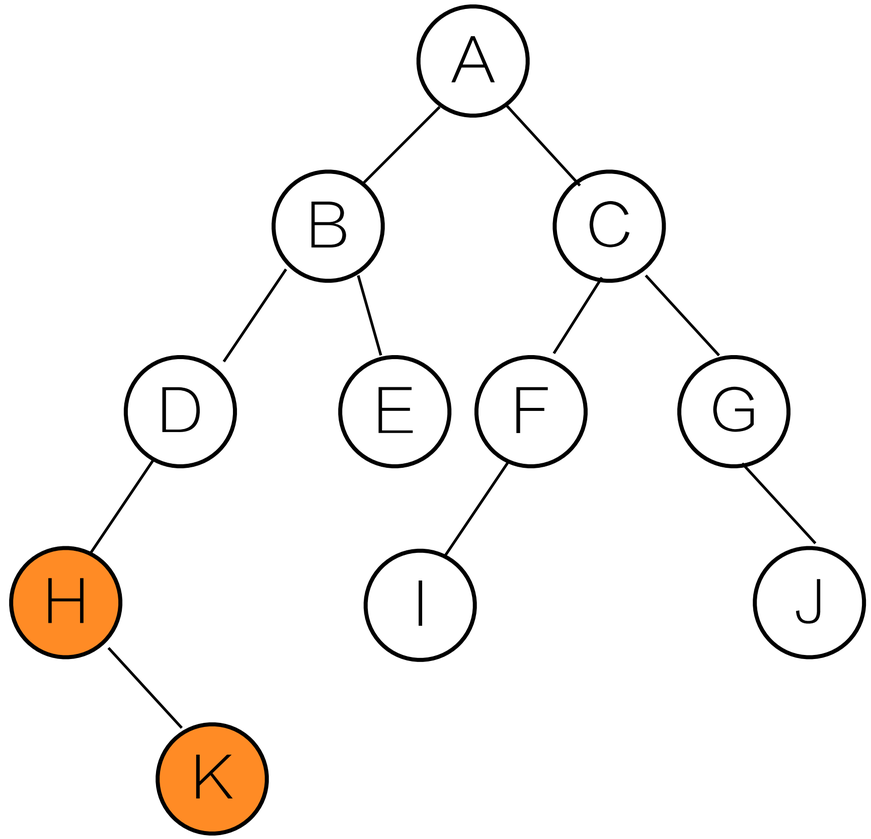


in(#) p in(K)

in(H) p in(#)

in(D) p in(E)

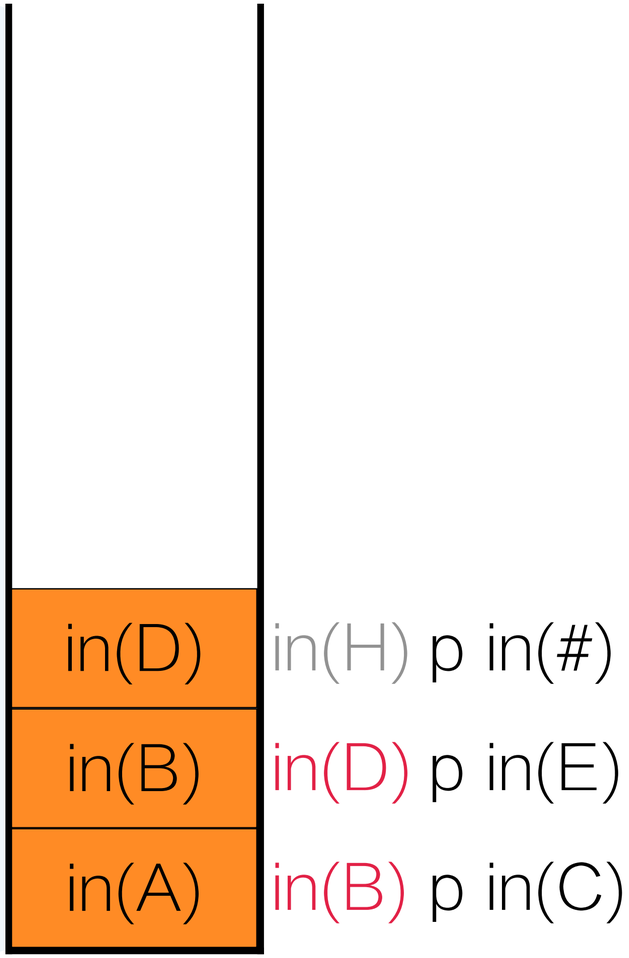
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K

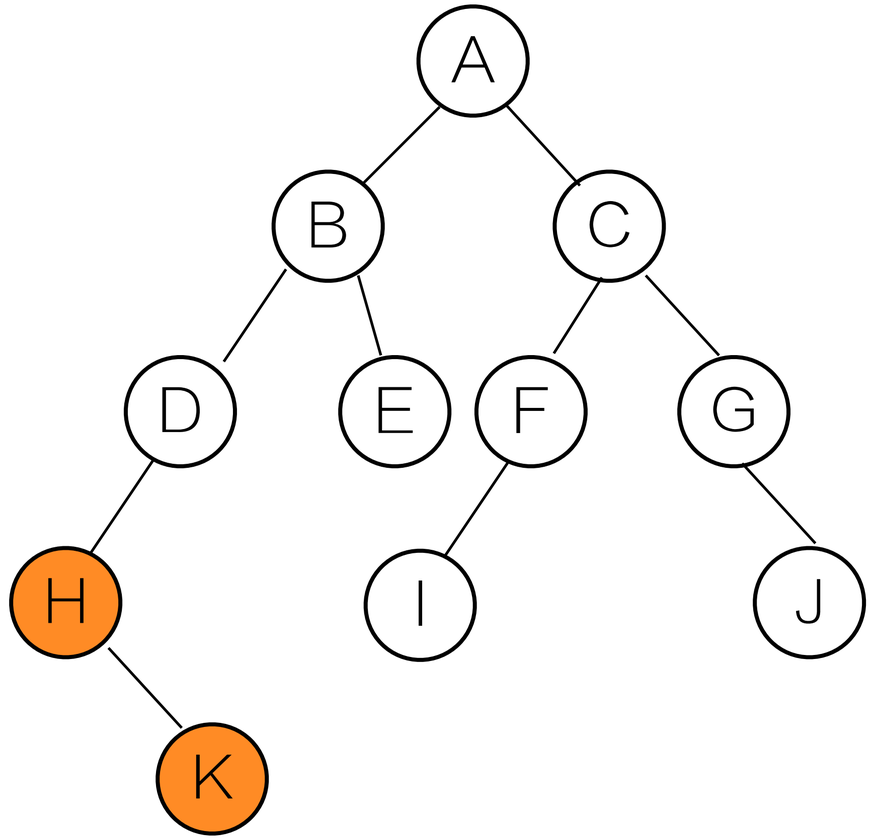
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(H) p in(#)

in(D) p in(E)

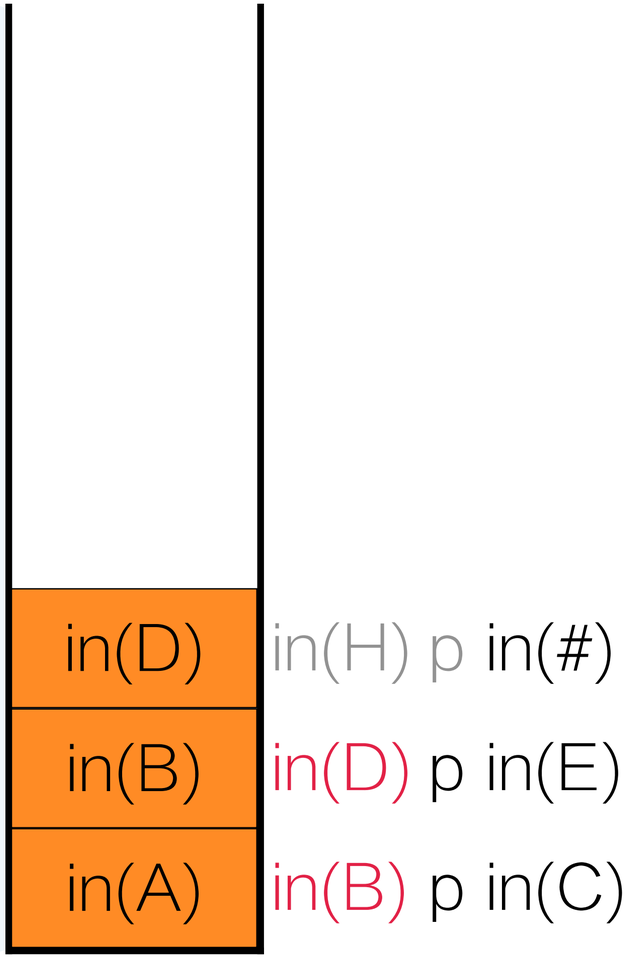
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D

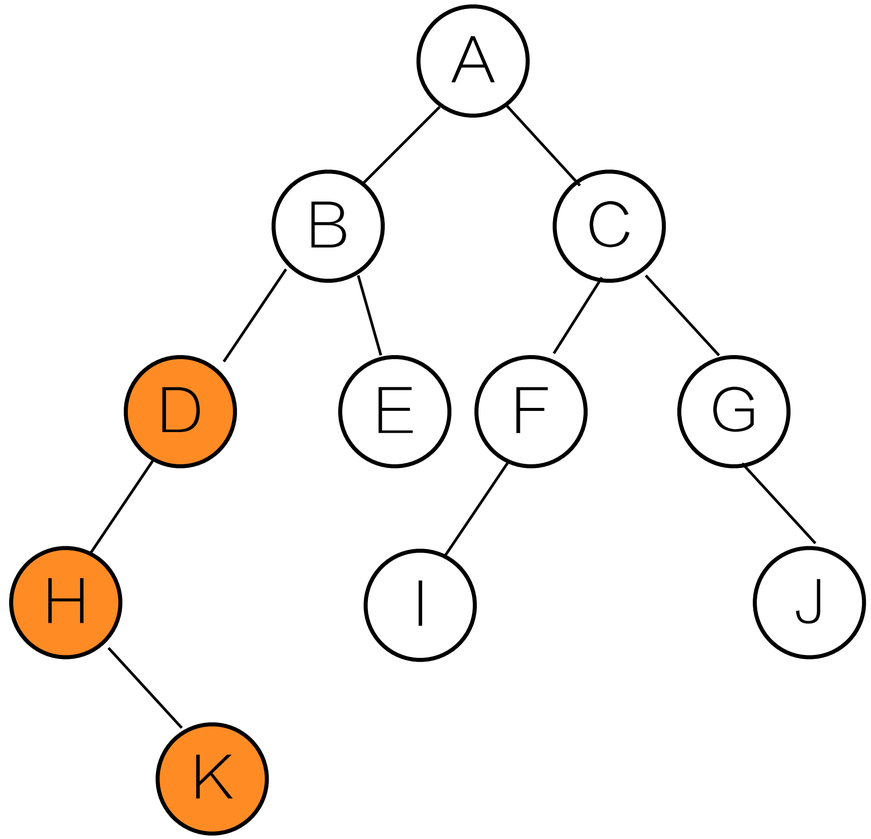
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(H) p in(#)

in(D) p in(E)

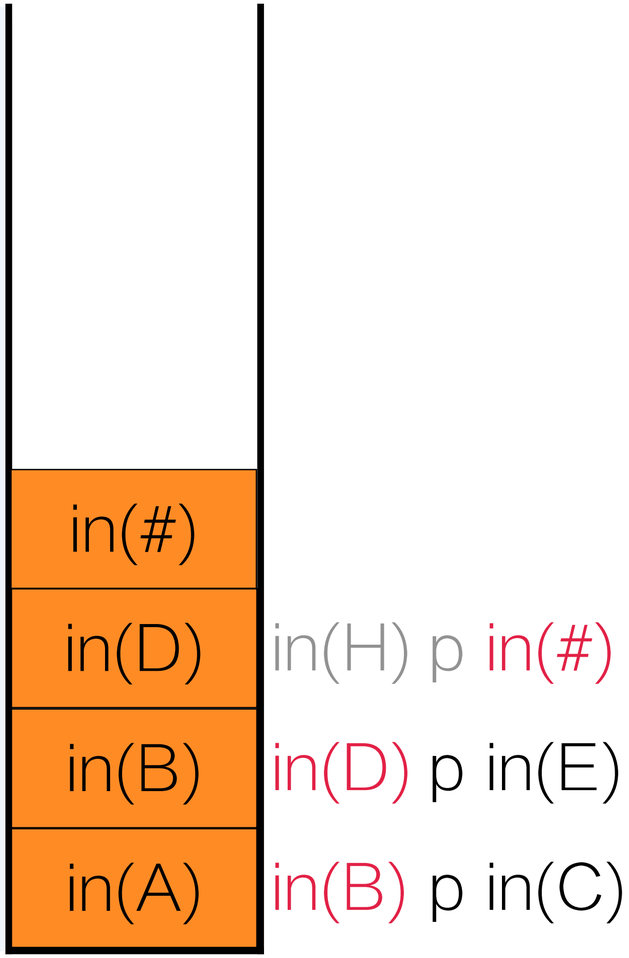
in(B) p in(C)

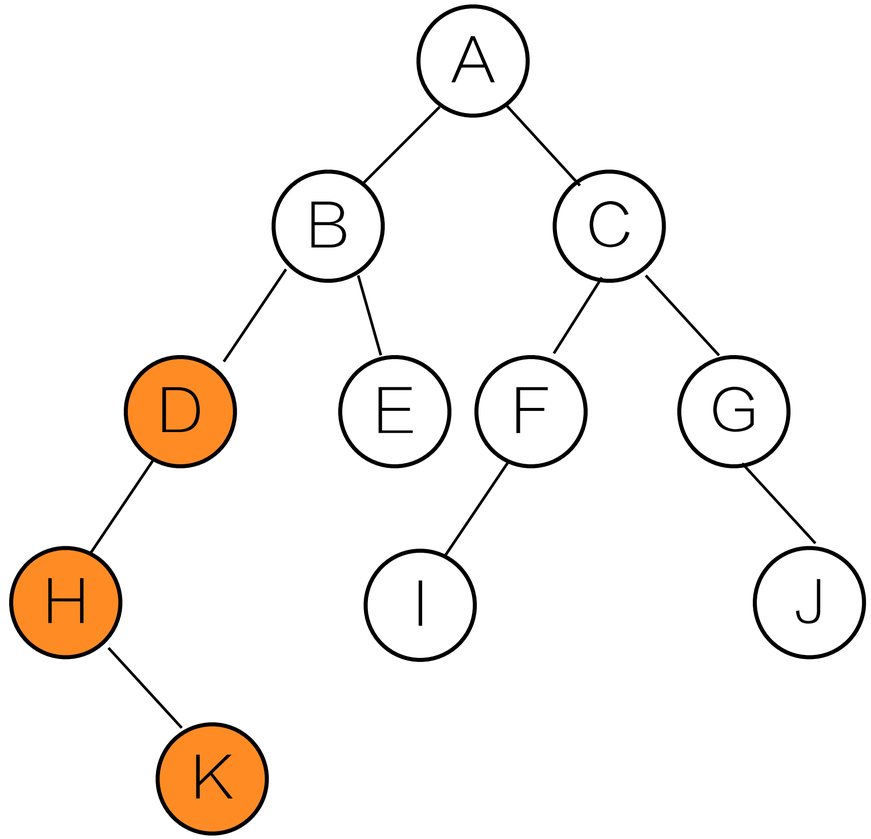


二叉树的遍历 – 中序遍历

输出内容：H K D

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

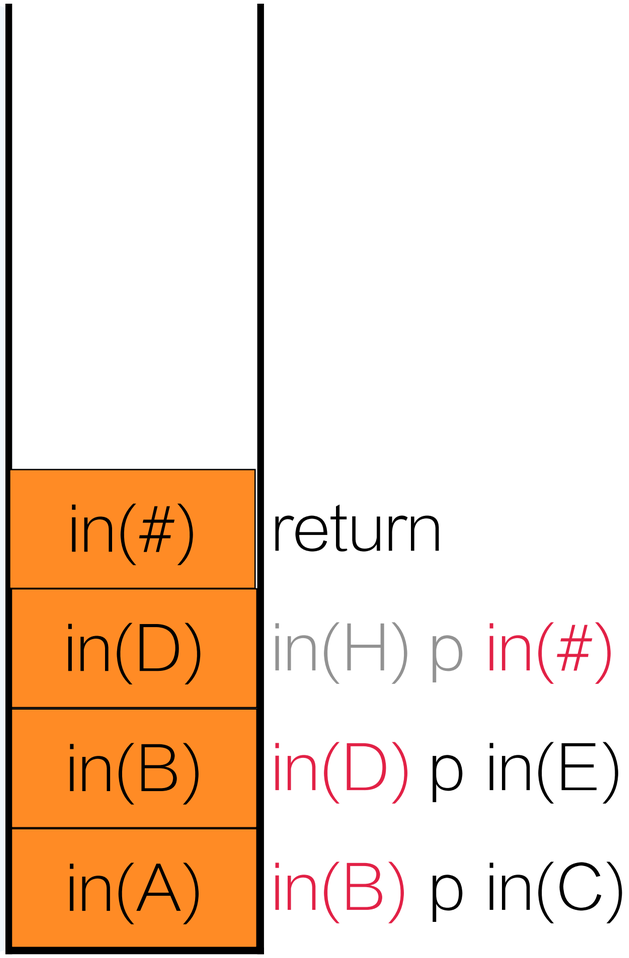


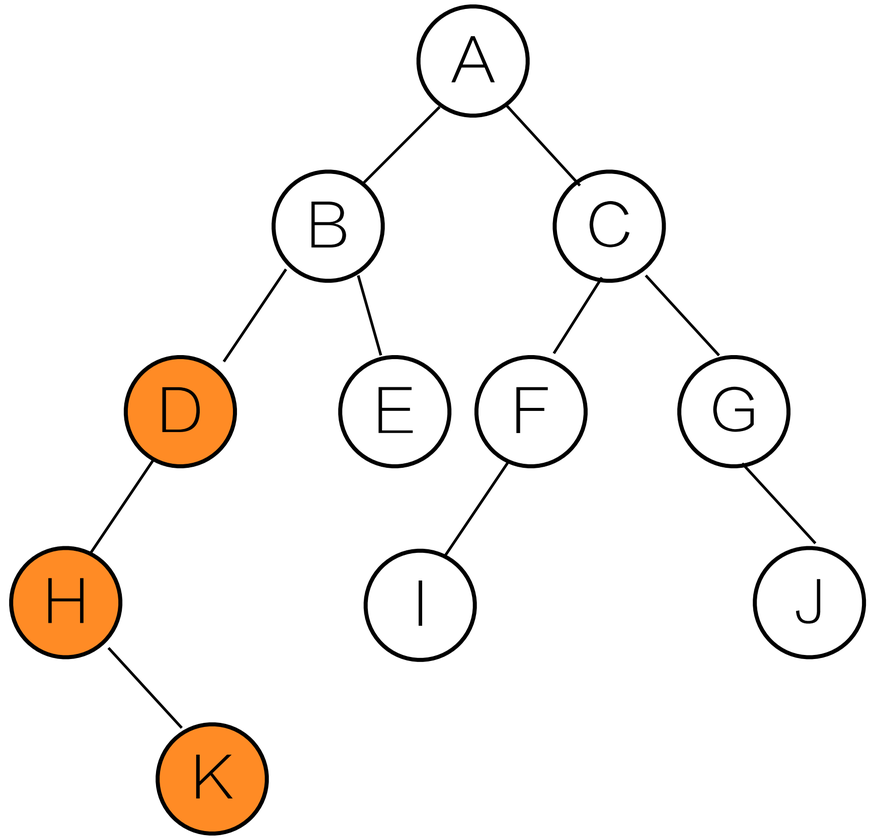


二叉树的遍历 – 中序遍历

输出内容：H K D

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

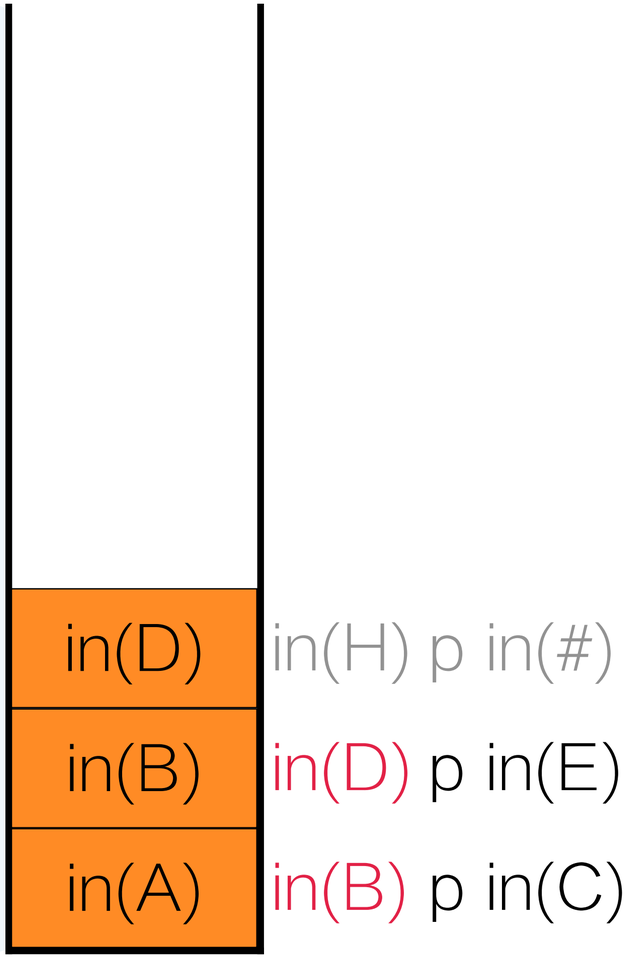




二叉树的遍历 – 中序遍历

输出内容：H K D

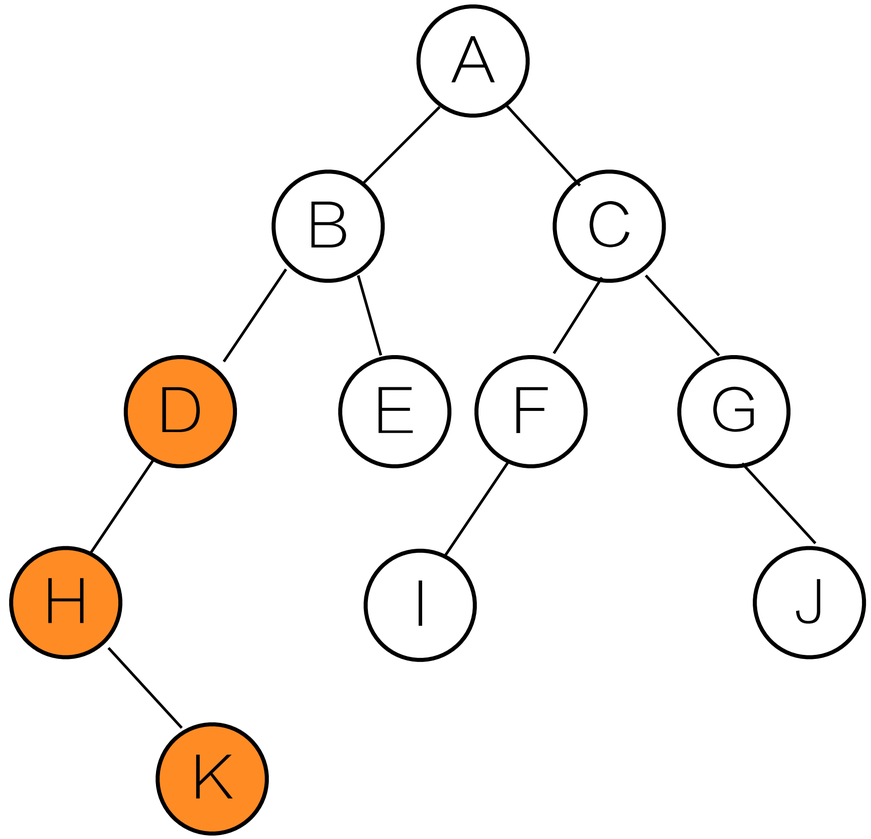
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(H) p in(#)

in(D) p in(E)

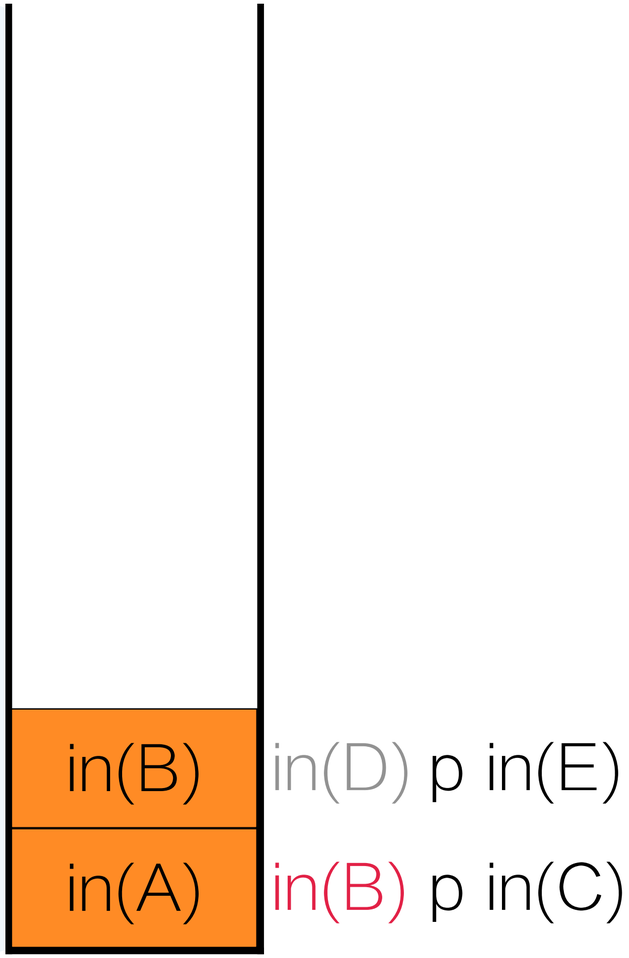
in(B) p in(C)



二叉树的遍历 – 中序遍历

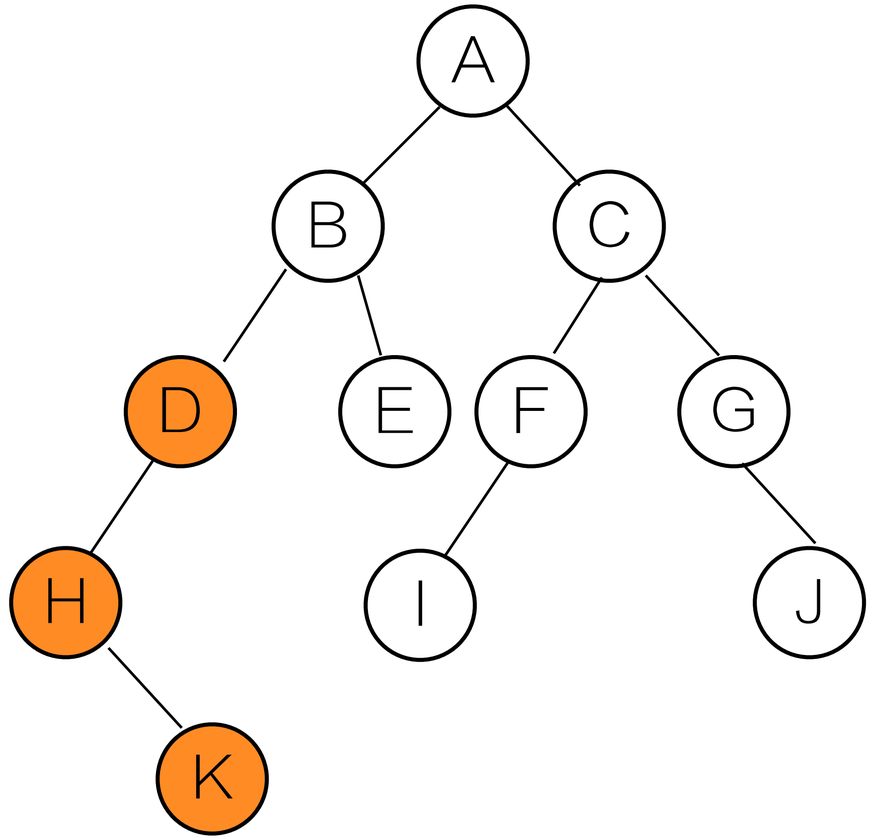
输出内容：H K D

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(D) p in(E)

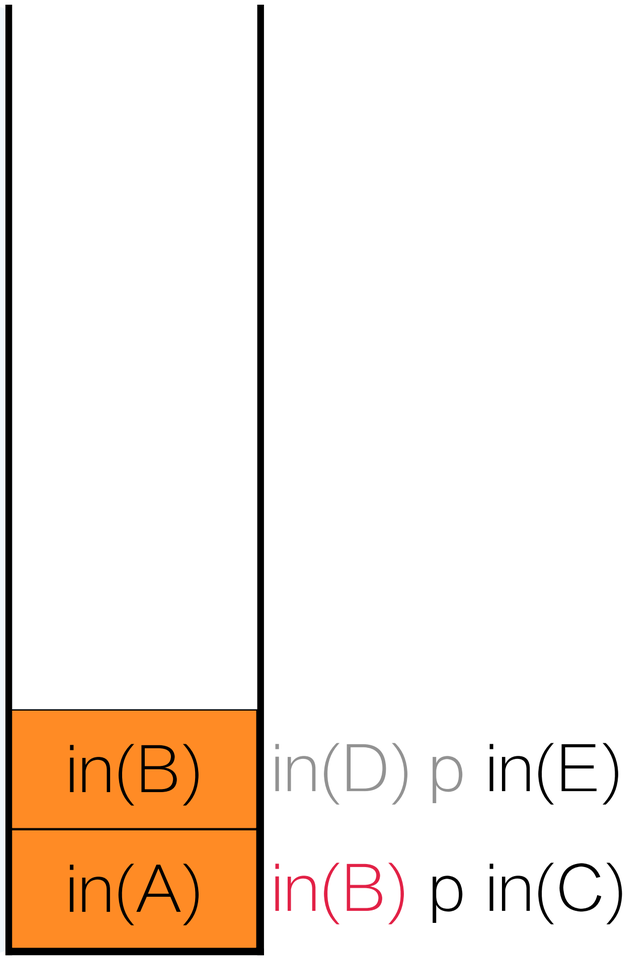
in(B) p in(C)



二叉树的遍历 – 中序遍历

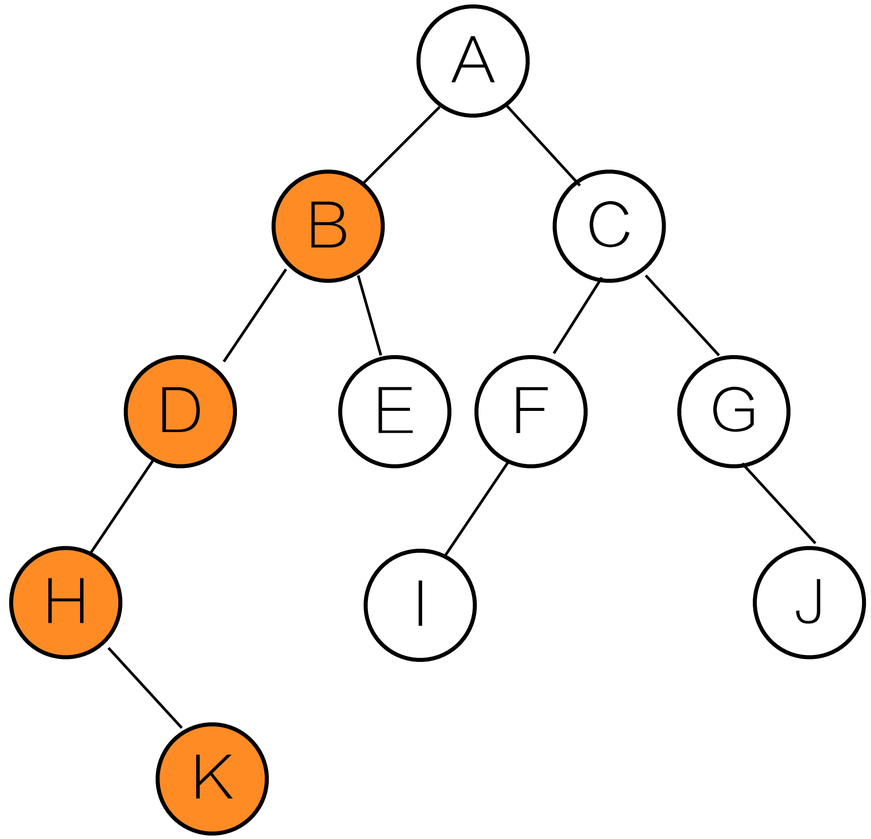
输出内容：H K D B

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(D) p in(E)

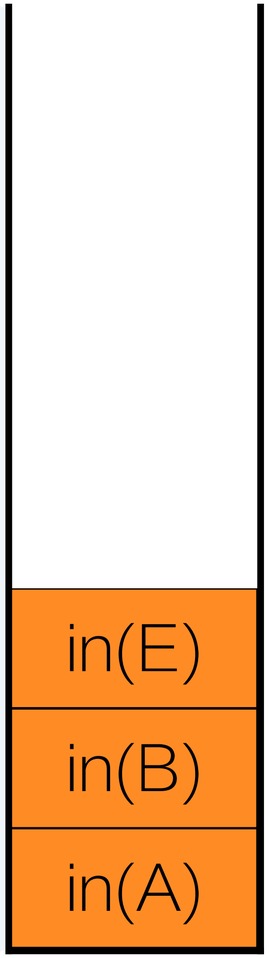
in(B) p in(C)



二叉树的遍历 – 中序遍历

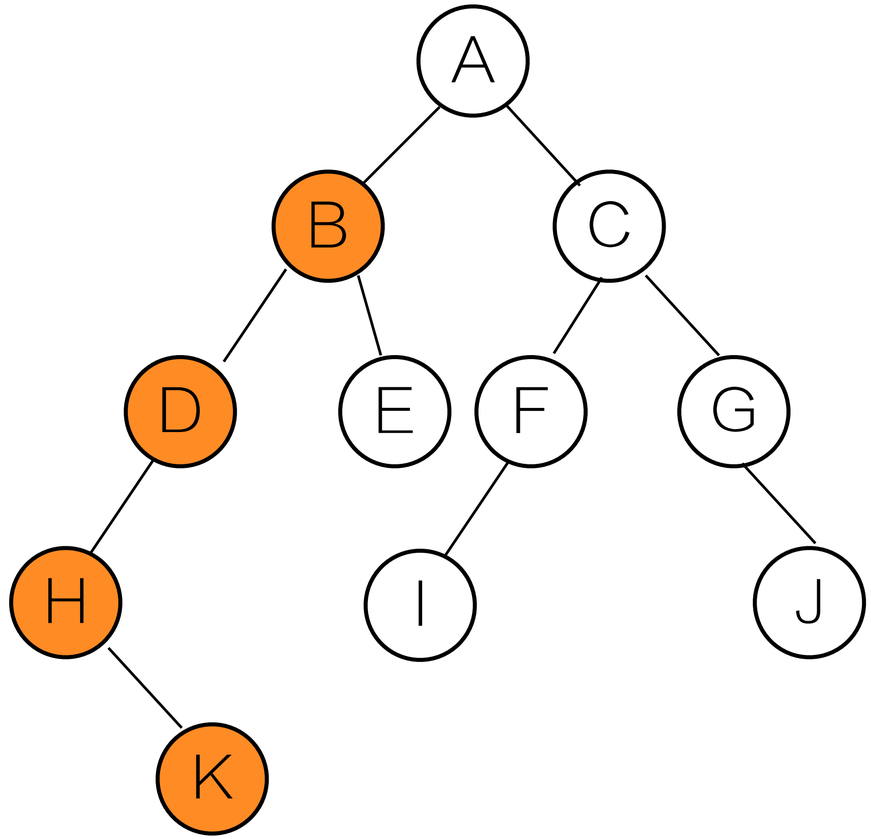
输出内容：H K D B

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(D) p in(E)

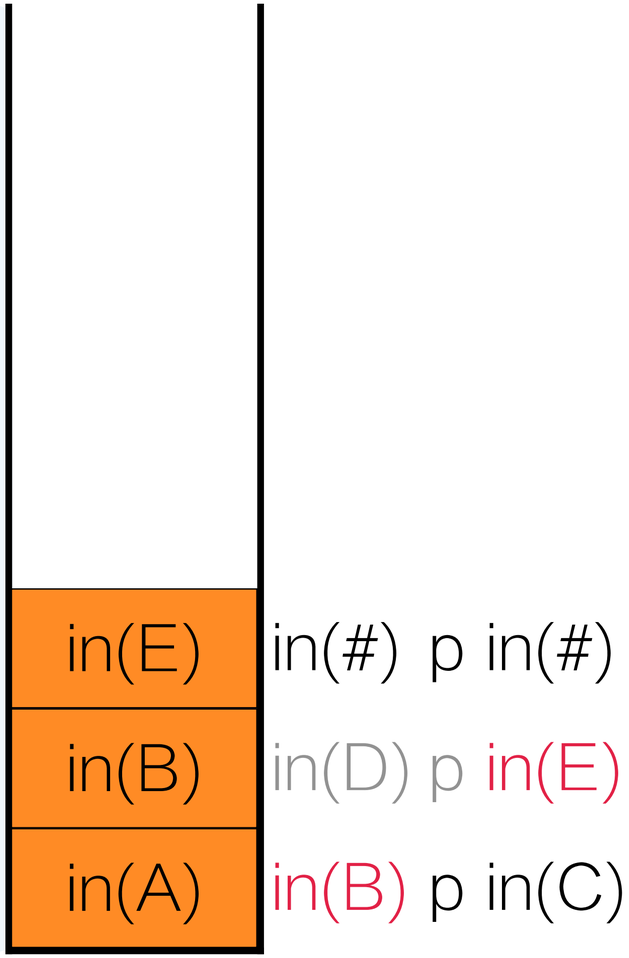
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B

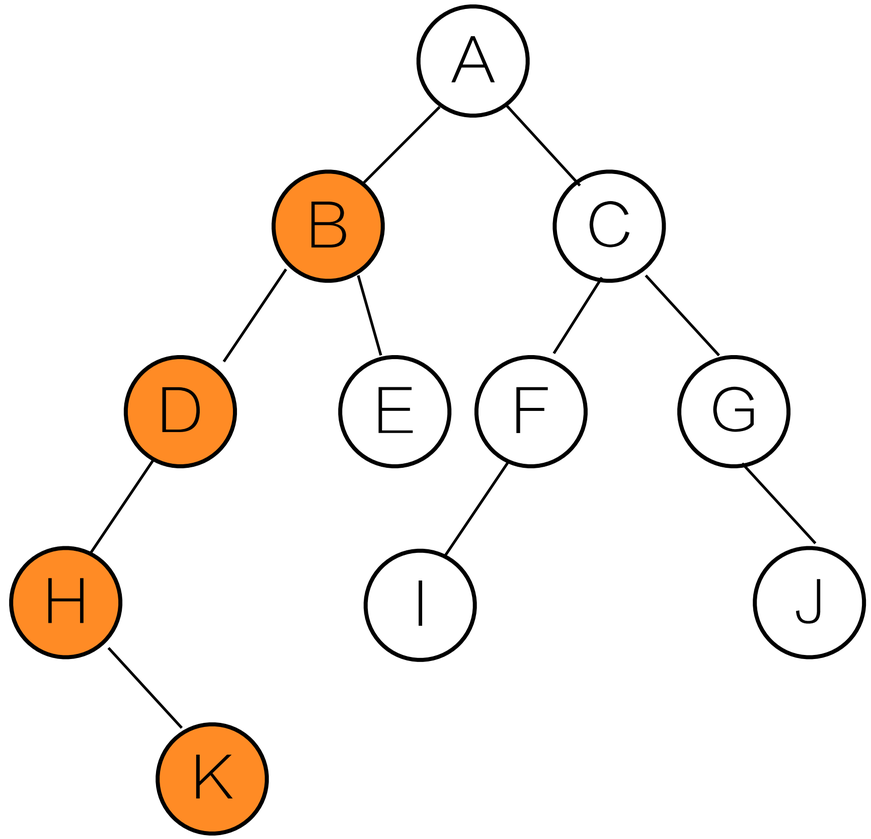
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(#)

in(D) p in(E)

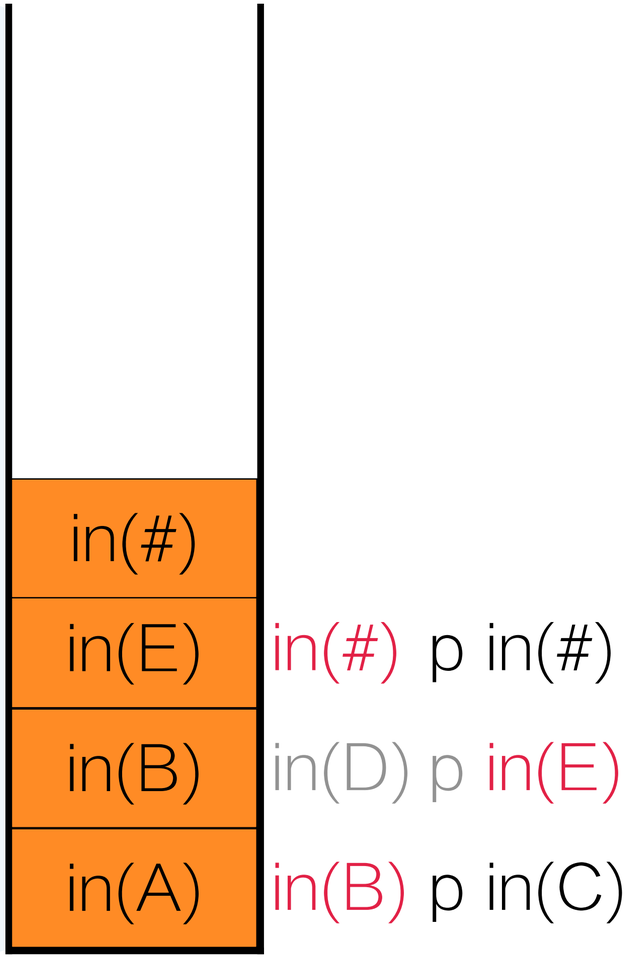
in(B) p in(C)

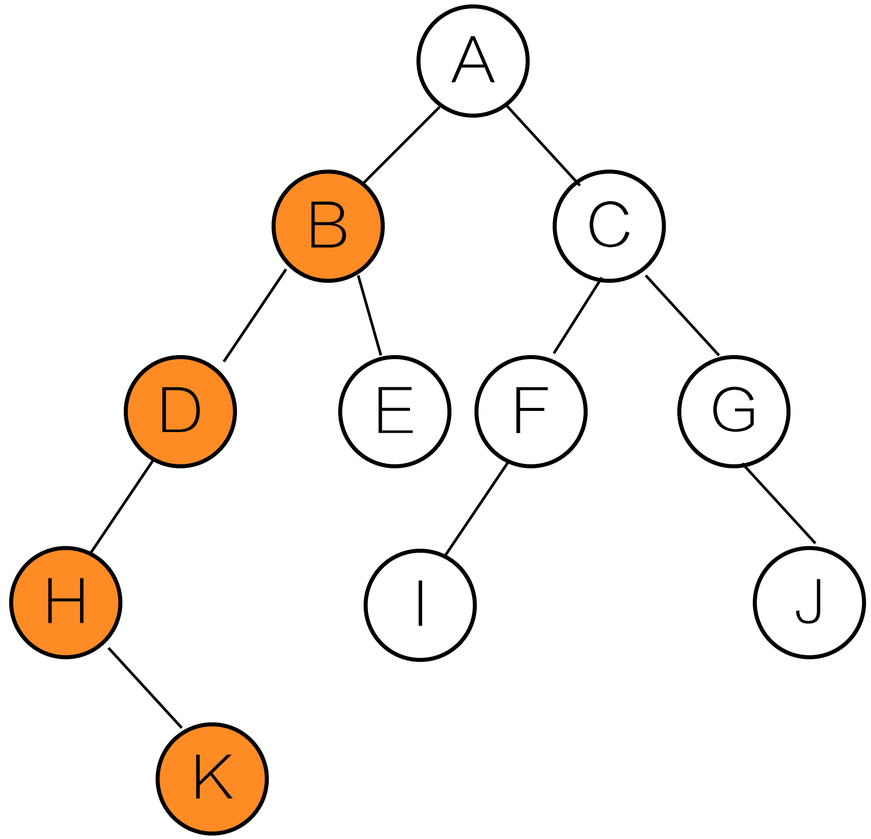


二叉树的遍历 – 中序遍历

输出内容：H K D B

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

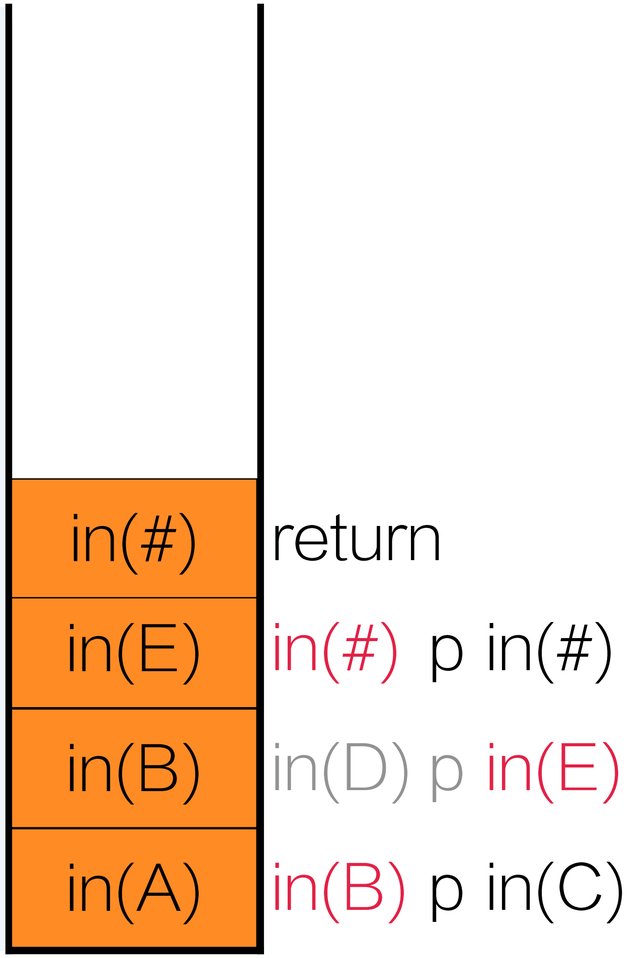


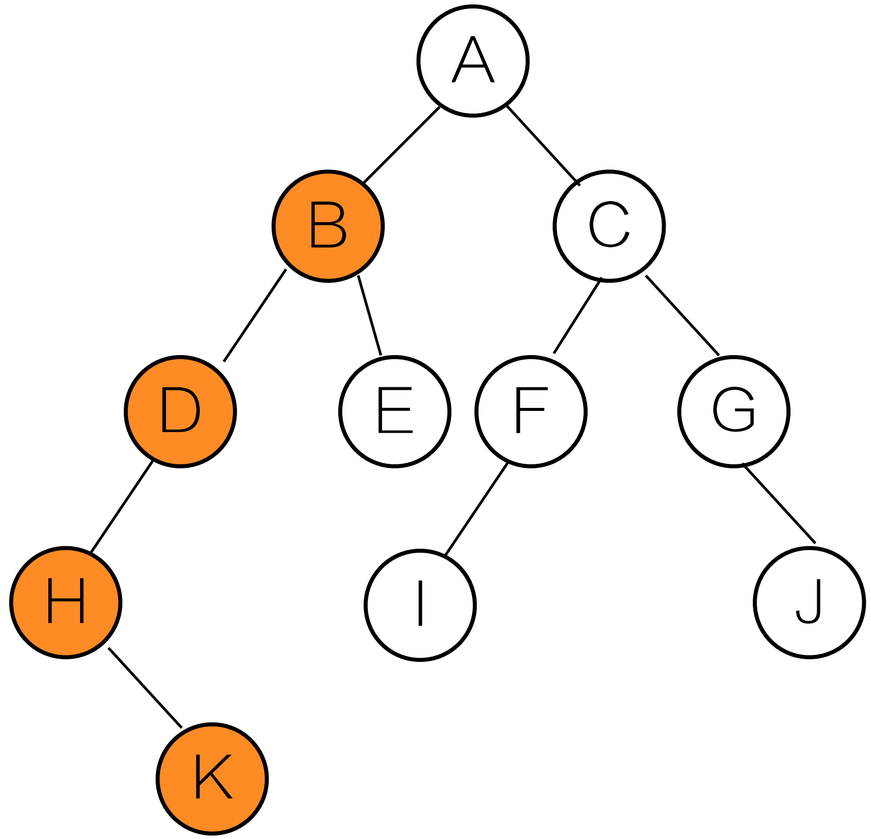


二叉树的遍历 – 中序遍历

输出内容：H K D B

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

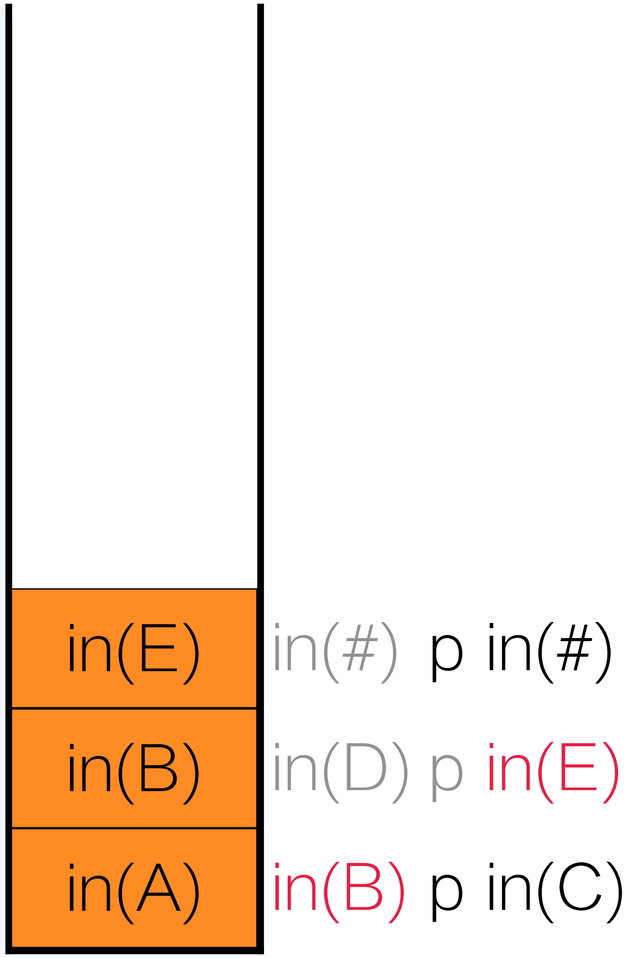




二叉树的遍历 – 中序遍历

输出内容：H K D B

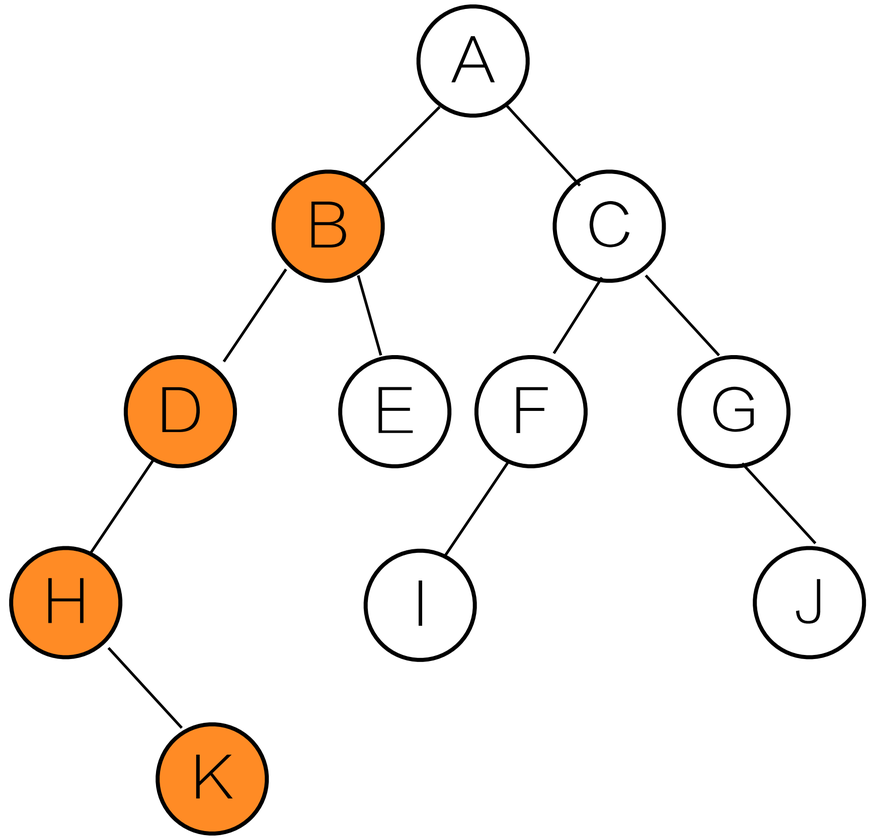
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(#)

in(D) p in(E)

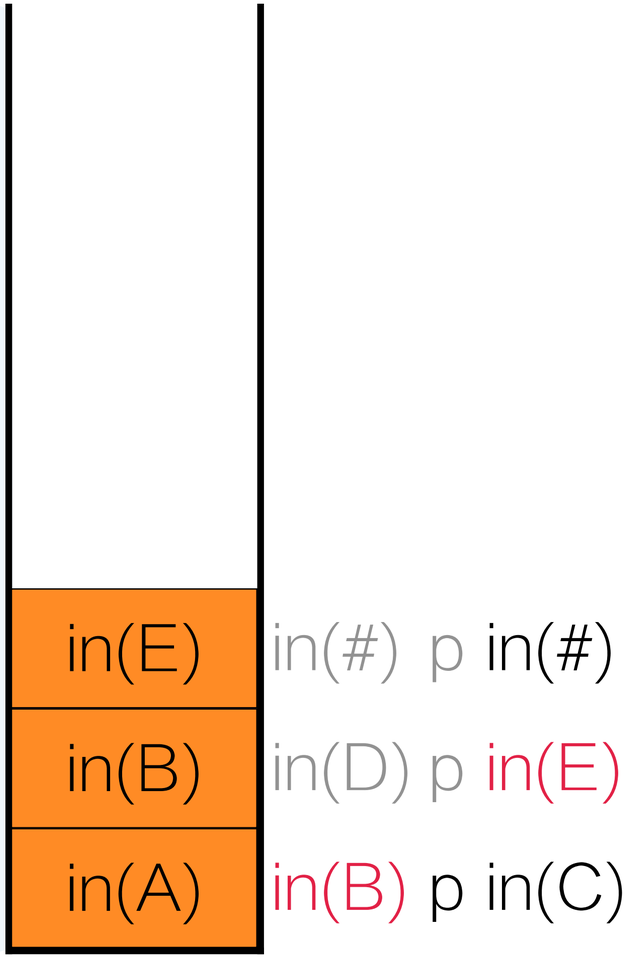
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E

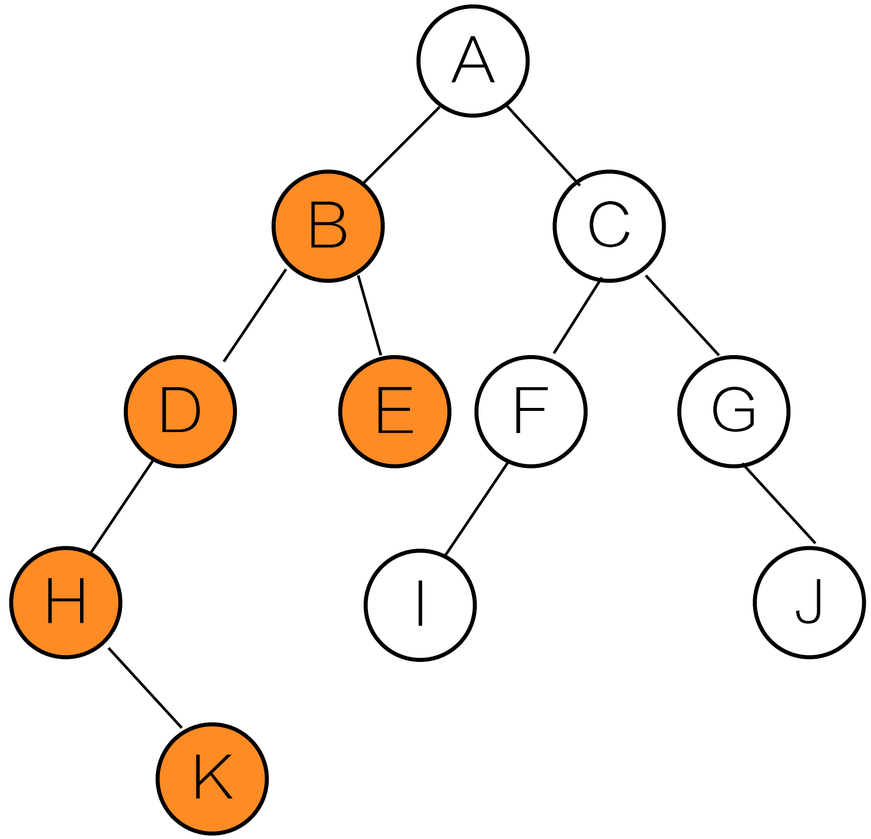
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(#)

in(D) p in(E)

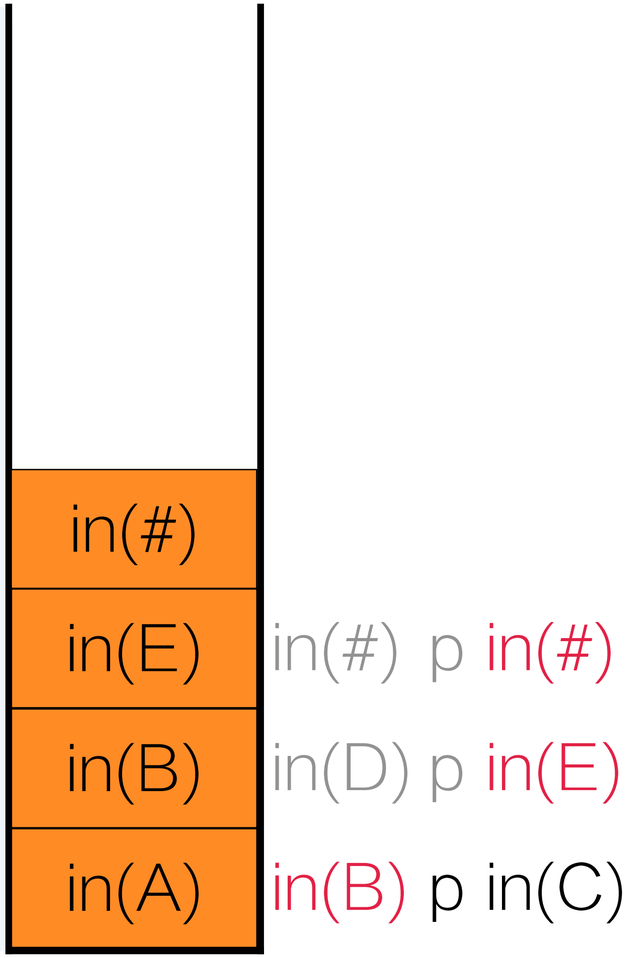
in(B) p in(C)

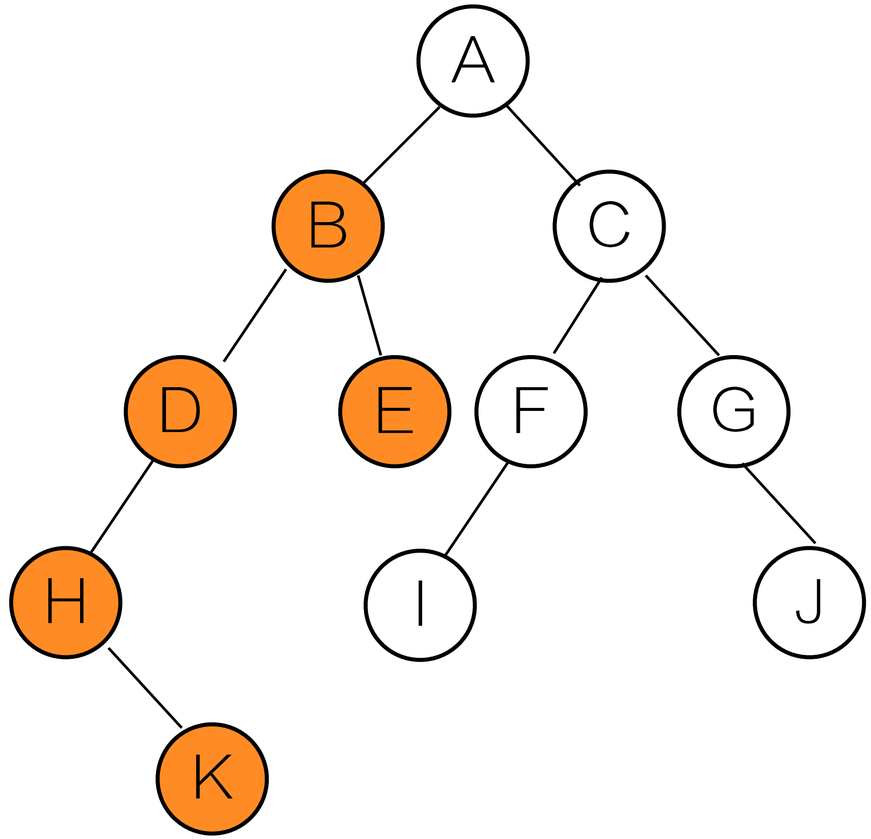


二叉树的遍历 – 中序遍历

输出内容：H K D B E

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

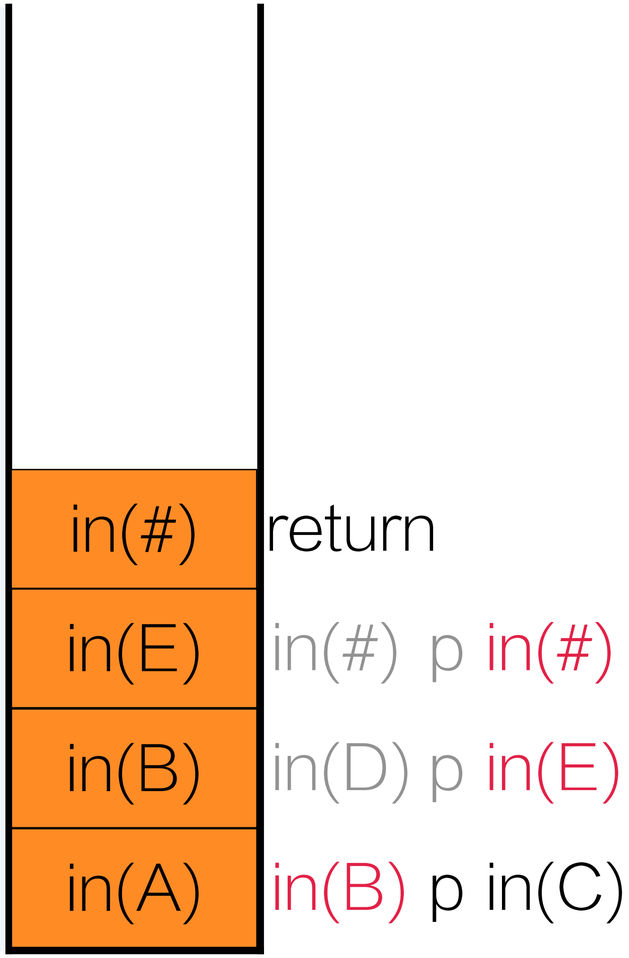


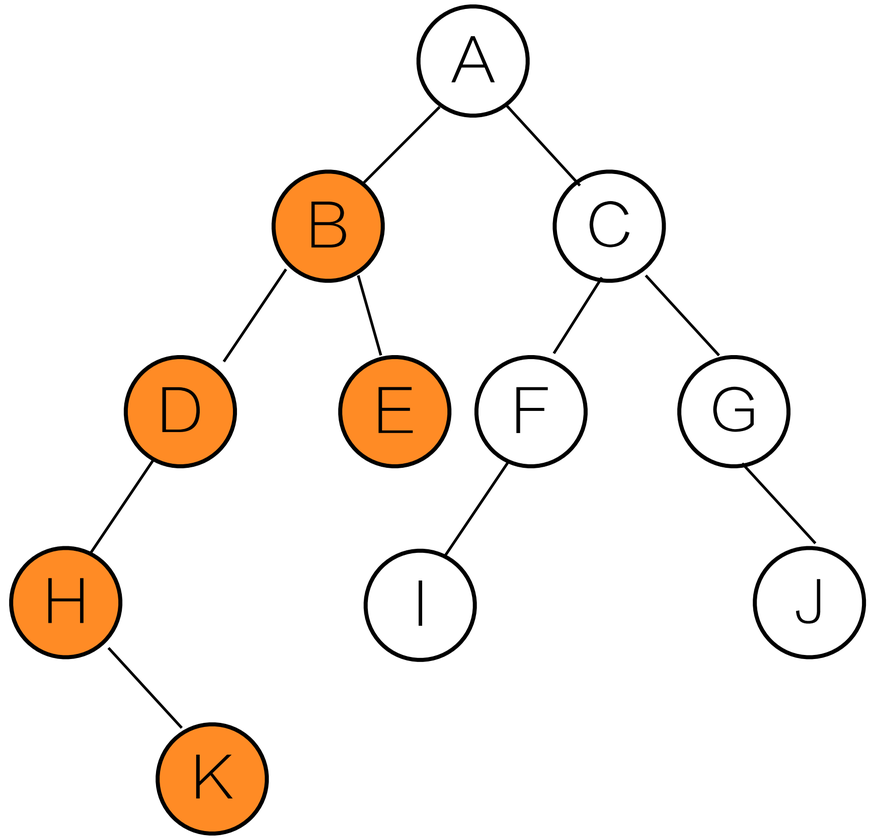


二叉树的遍历 – 中序遍历

输出内容：H K D B E

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

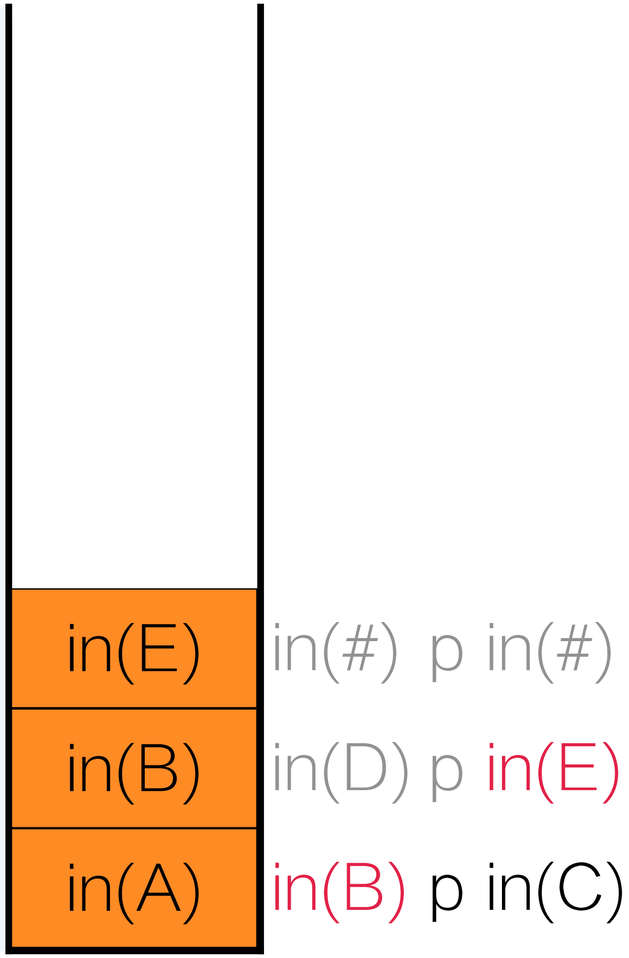




二叉树的遍历 – 中序遍历

输出内容：H K D B E

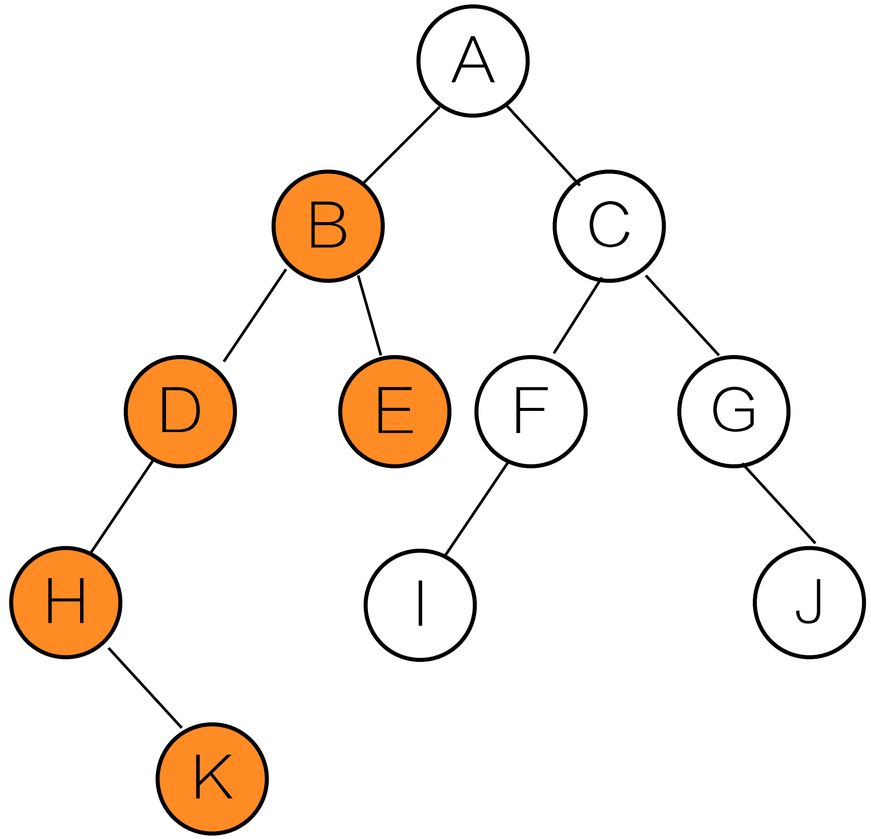
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(#)

in(D) p in(E)

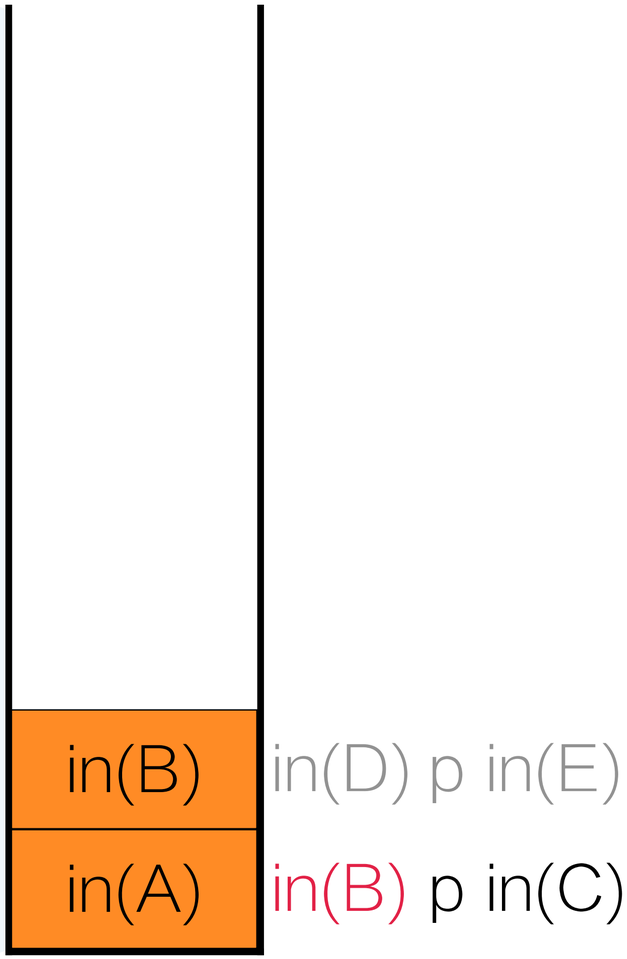
in(B) p in(C)



二叉树的遍历 – 中序遍历

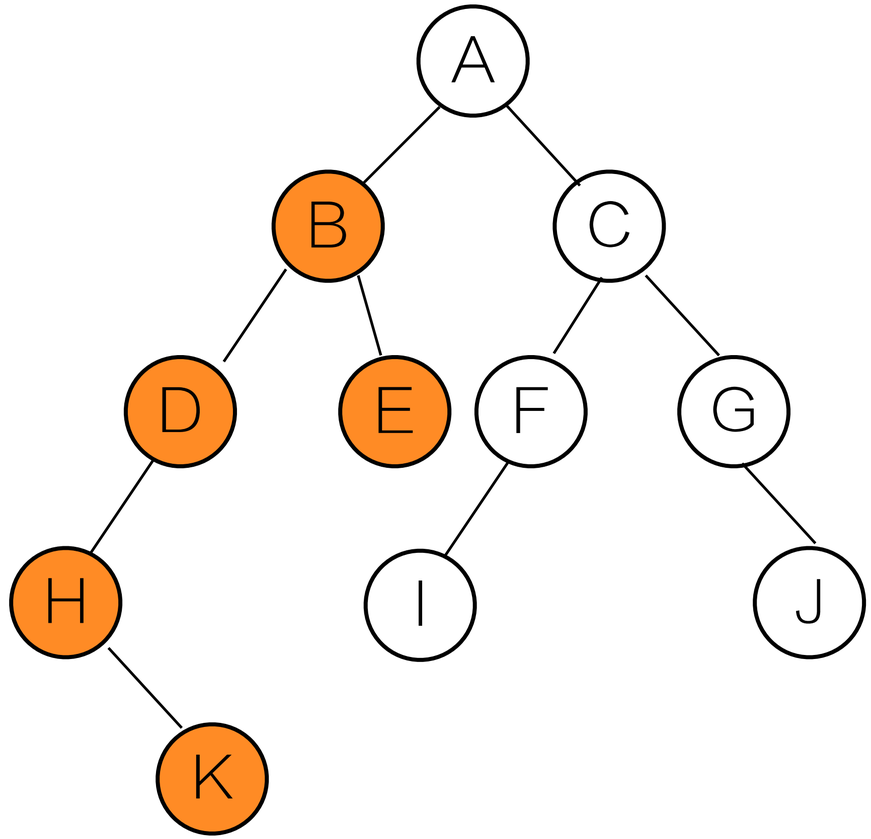
输出内容：H K D B E

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(D) p in(E)

in(B) p in(C)

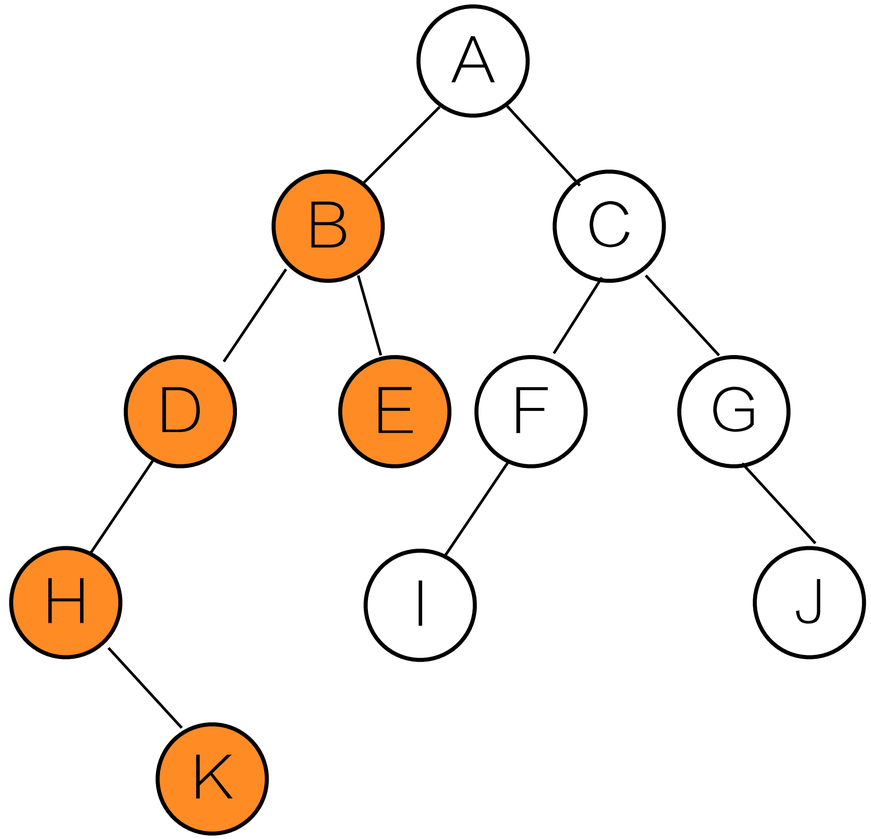


二叉树的遍历 – 中序遍历

输出内容：H K D B E

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

in(A) in(B) p in(C)

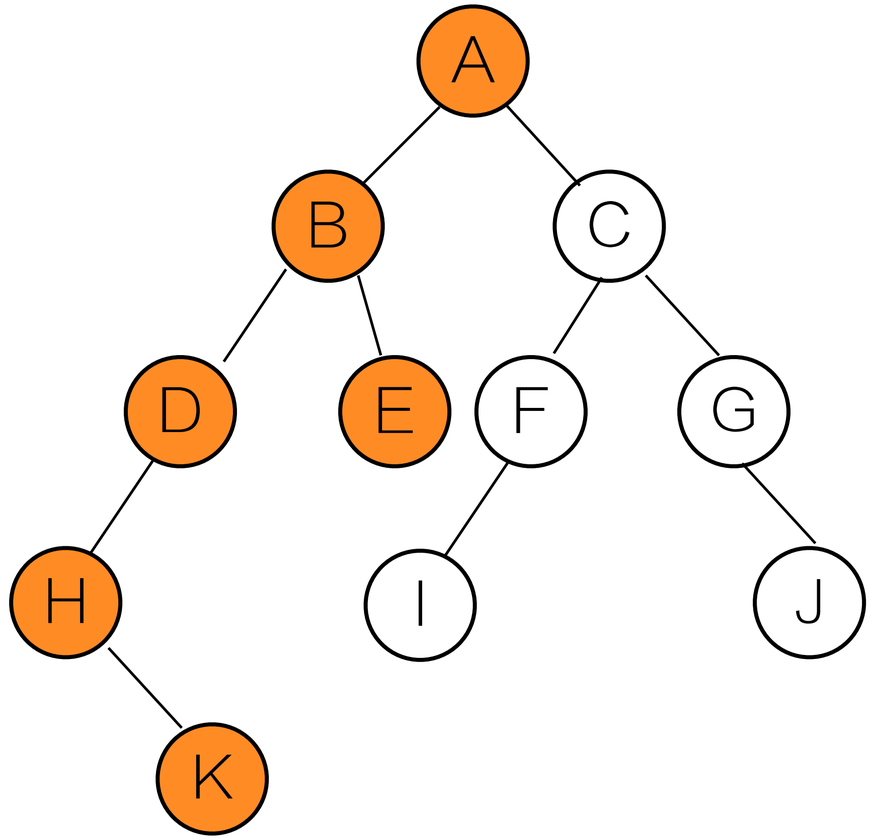


二叉树的遍历 – 中序遍历

输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

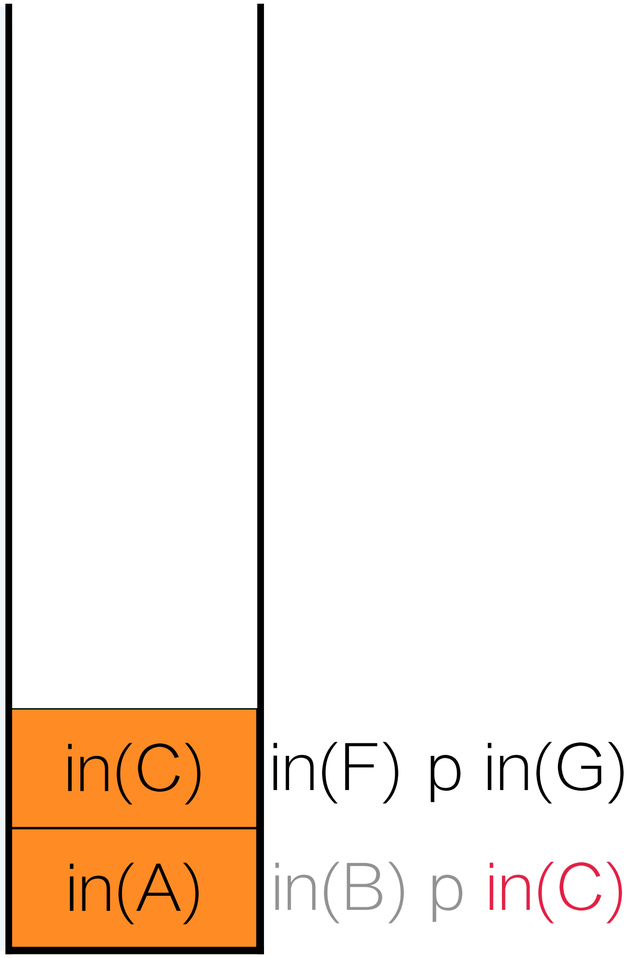
in(A) in(B) p in(C)



二叉树的遍历 – 中序遍历

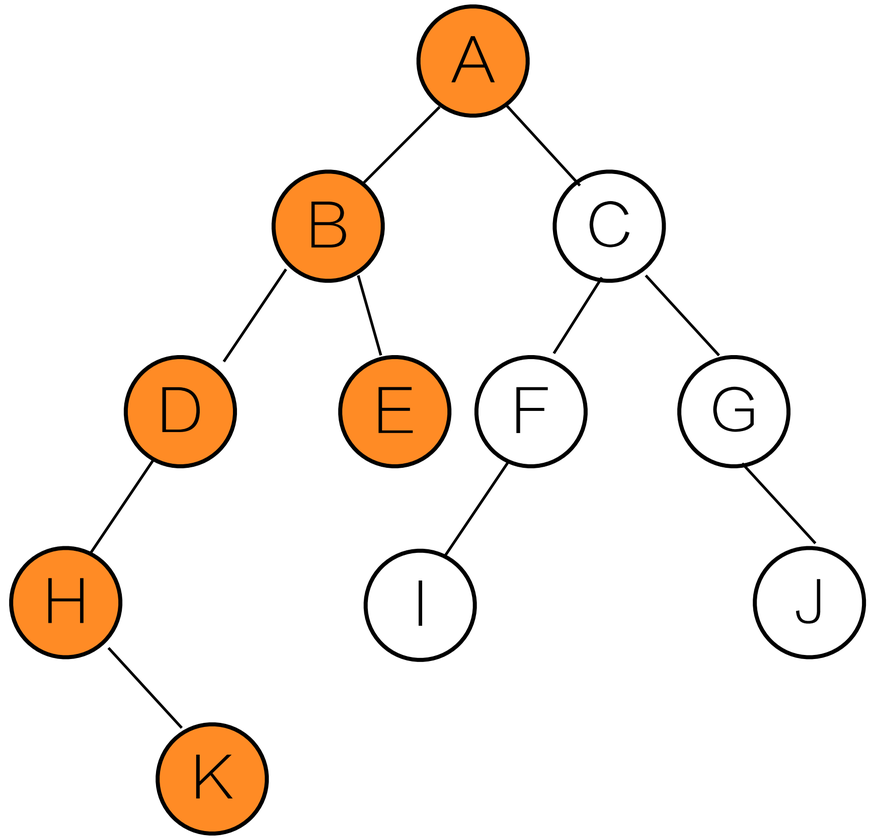
输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(F) p in(G)

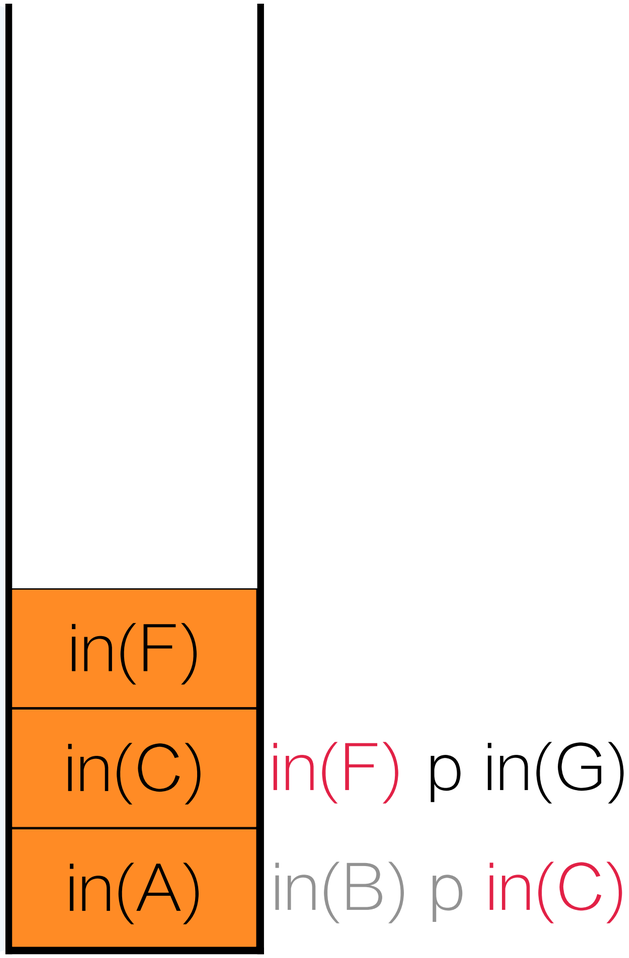
in(B) p in(C)



二叉树的遍历 – 中序遍历

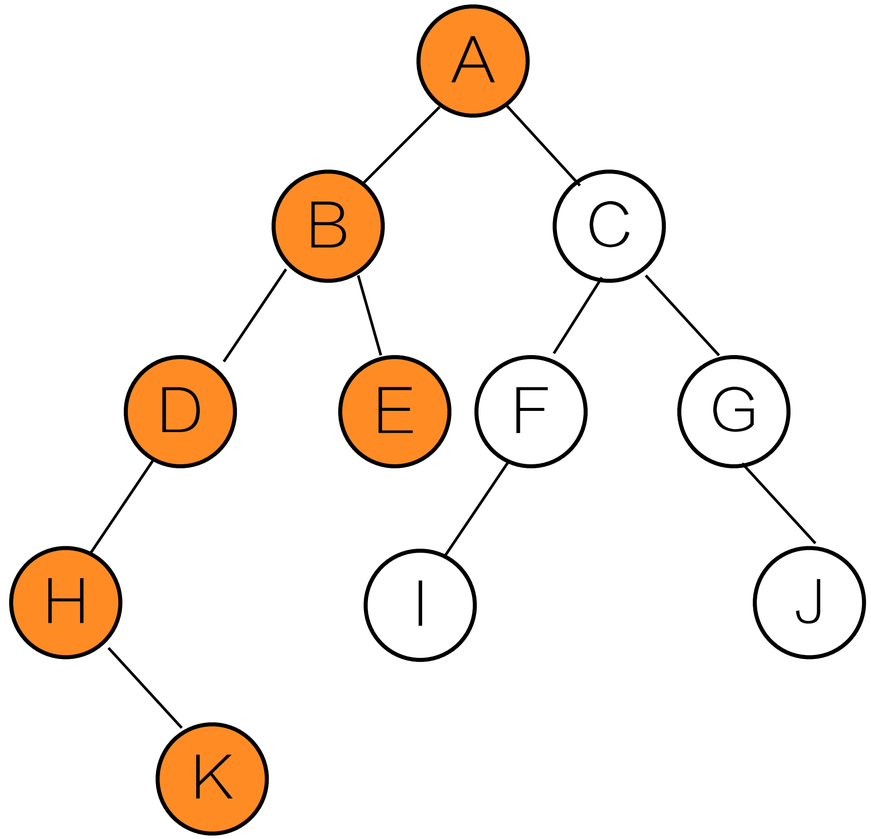
输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(F) p in(G)

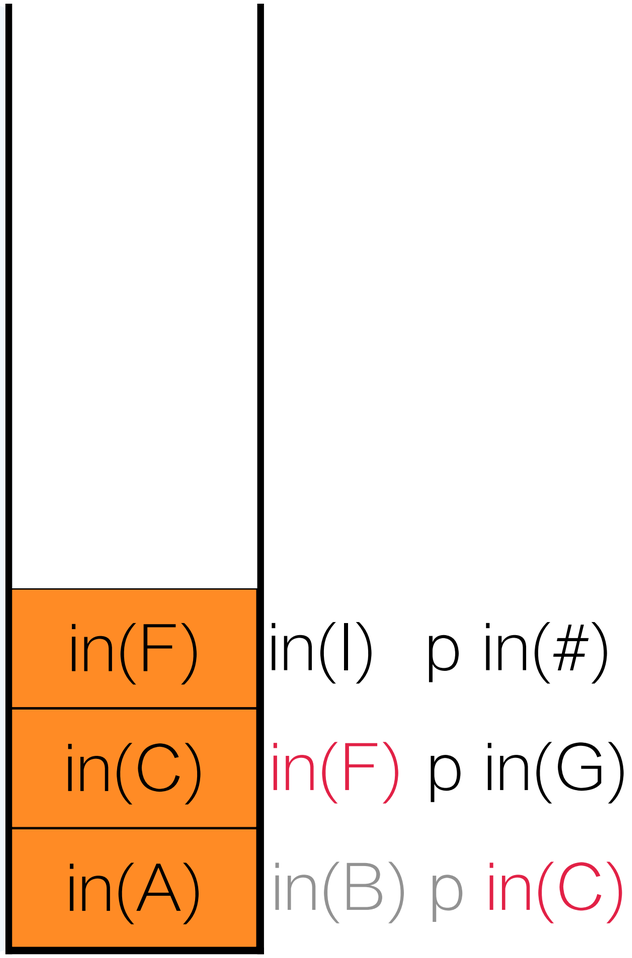
in(B) p in(C)

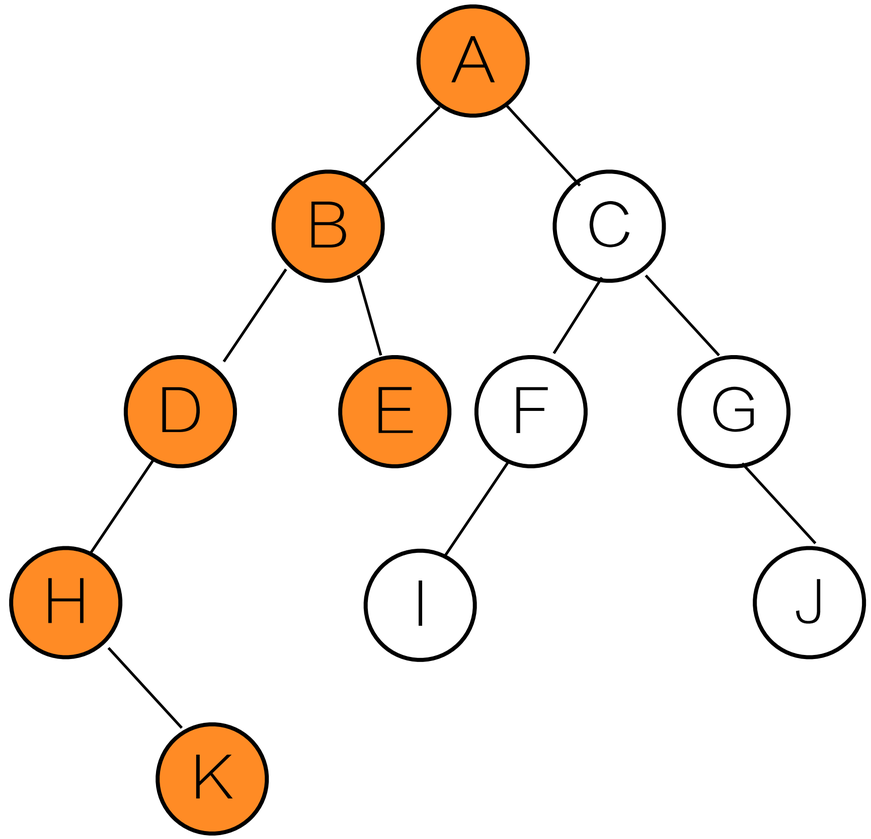


二叉树的遍历 – 中序遍历

输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

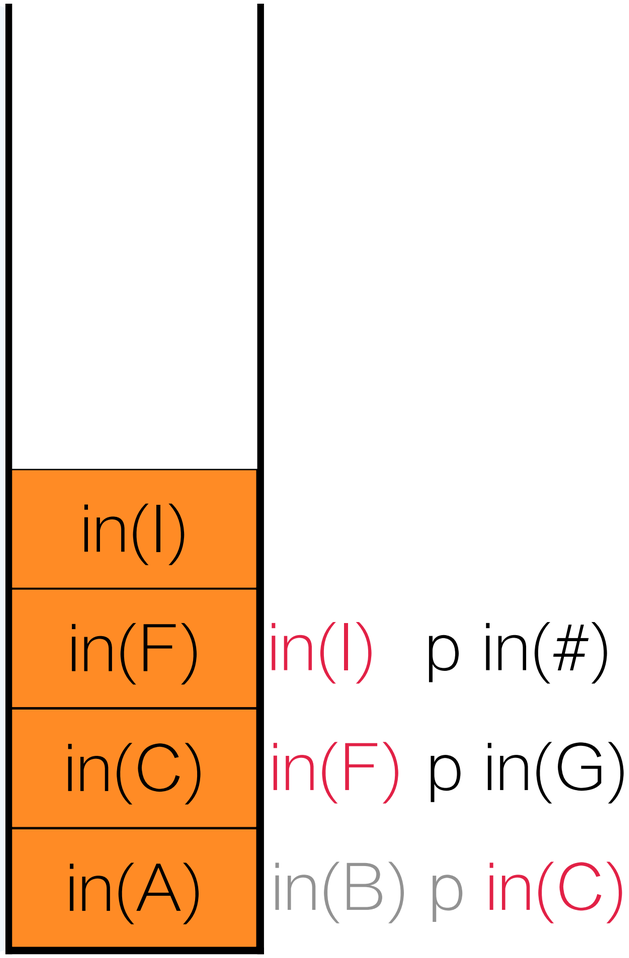




二叉树的遍历 – 中序遍历

输出内容：H K D B E A

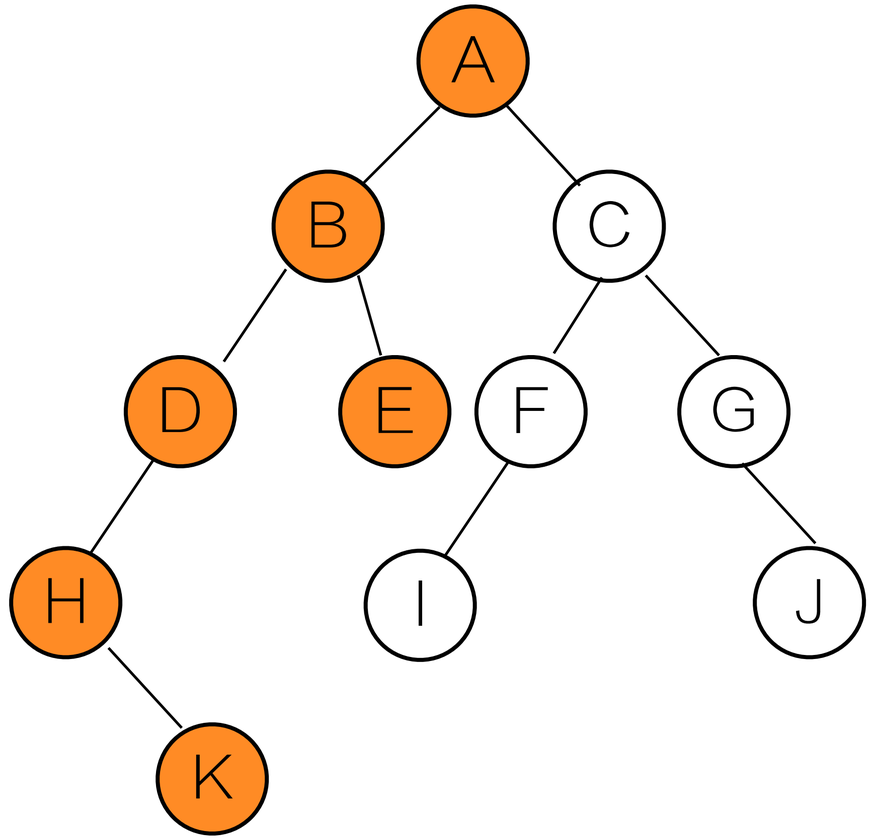
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(I) p in(#)

in(F) p in(G)

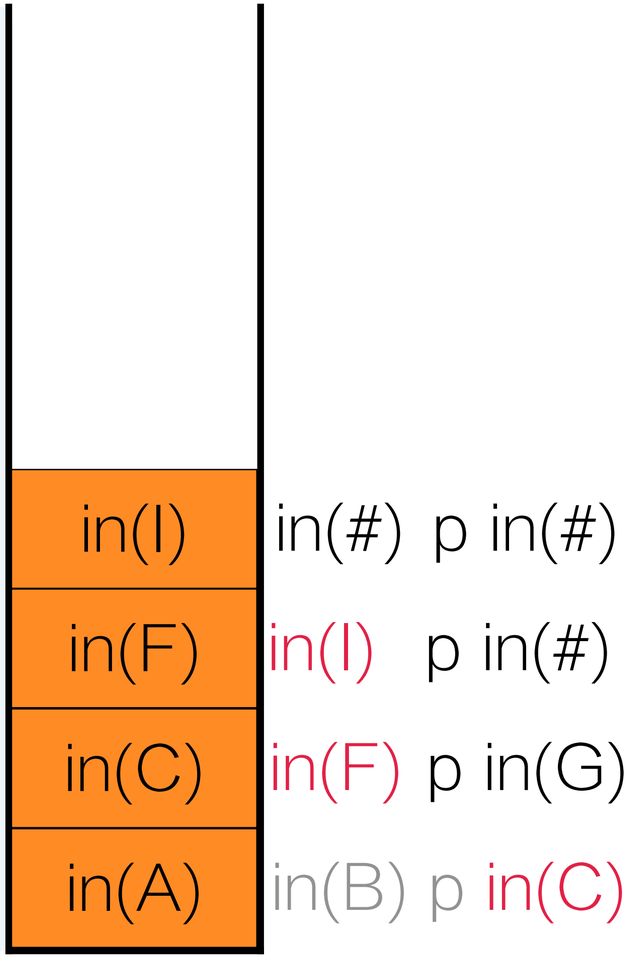
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

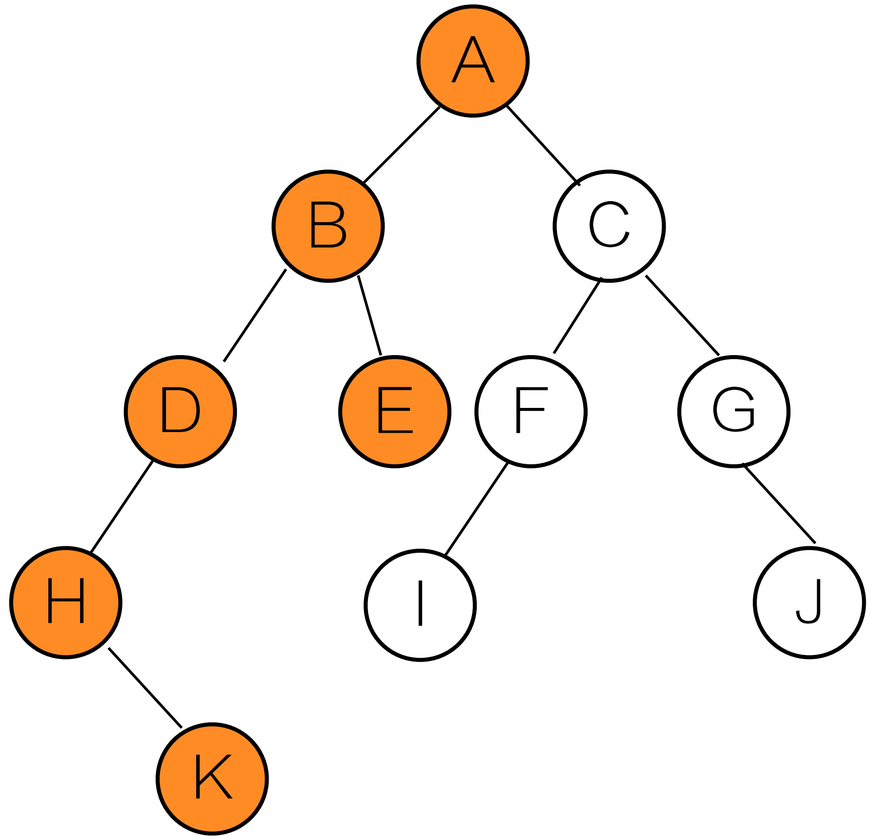


in(#) p in(#)

in(I) p in(#)

in(F) p in(G)

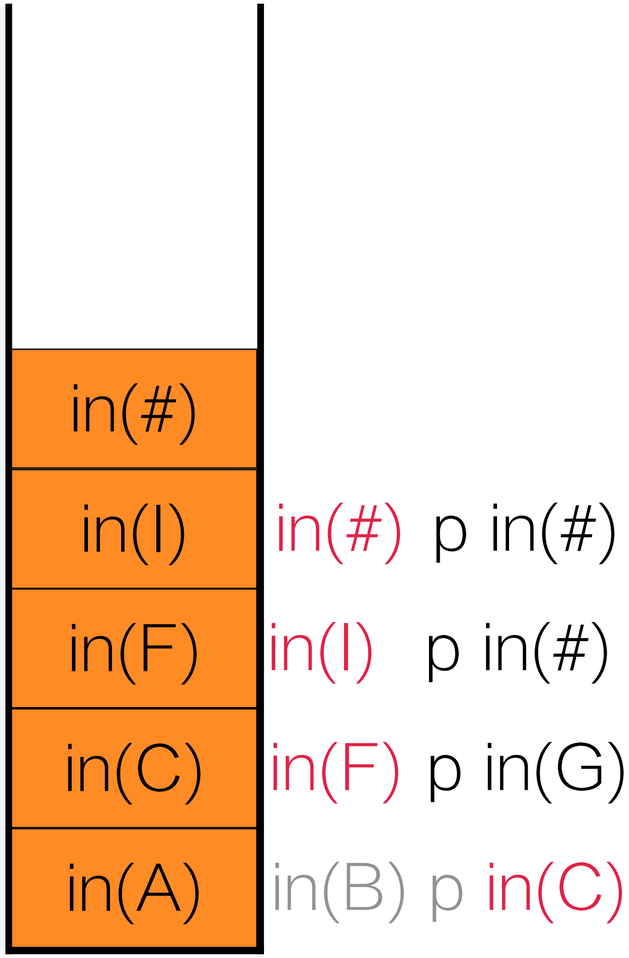
in(B) p in(C)

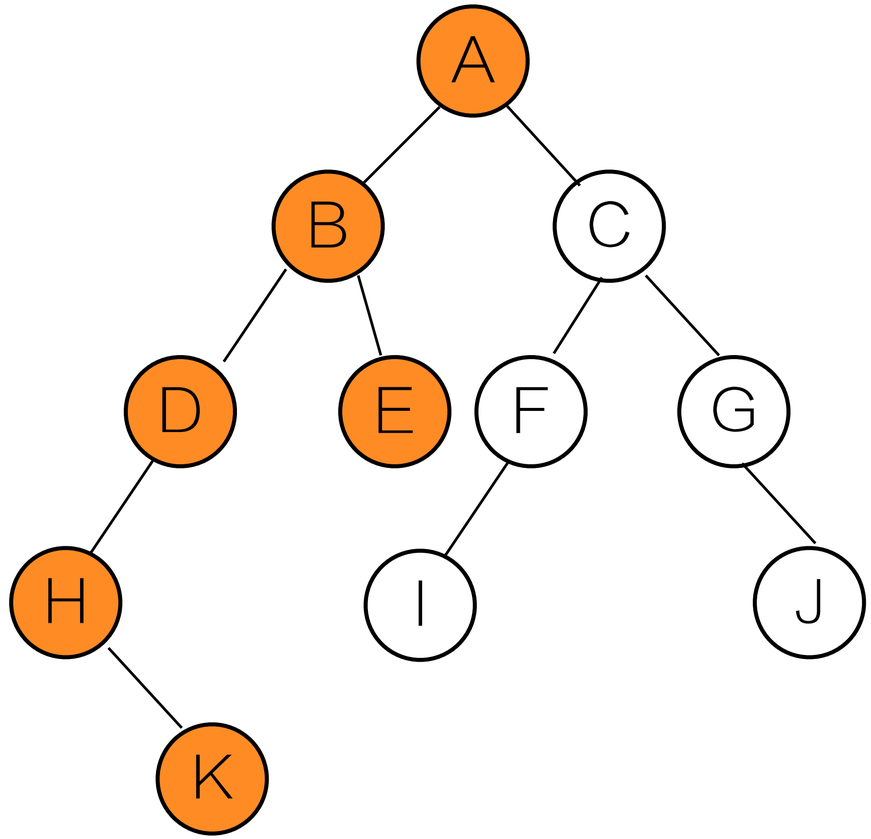


二叉树的遍历 – 中序遍历

输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

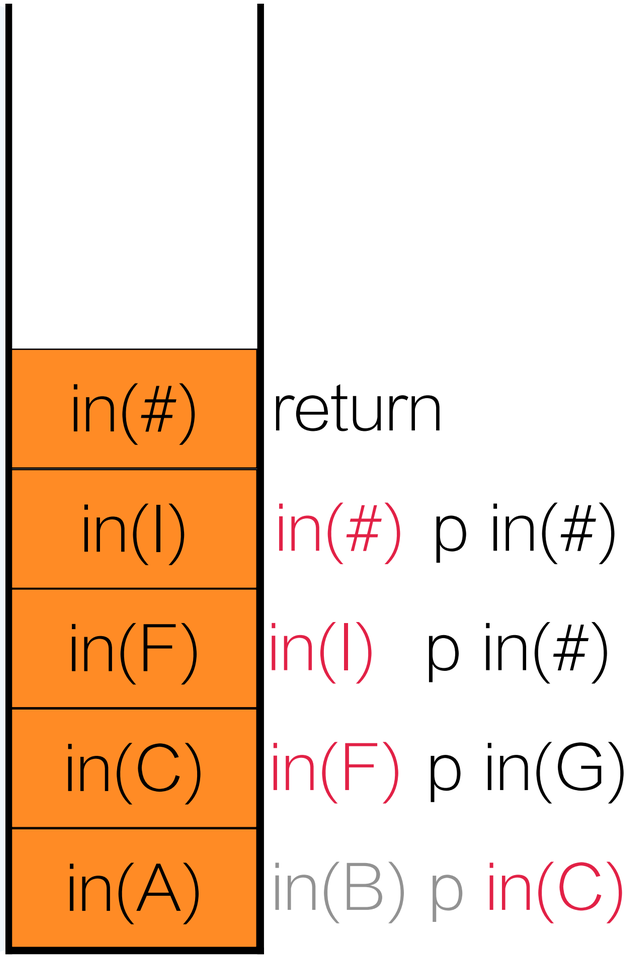


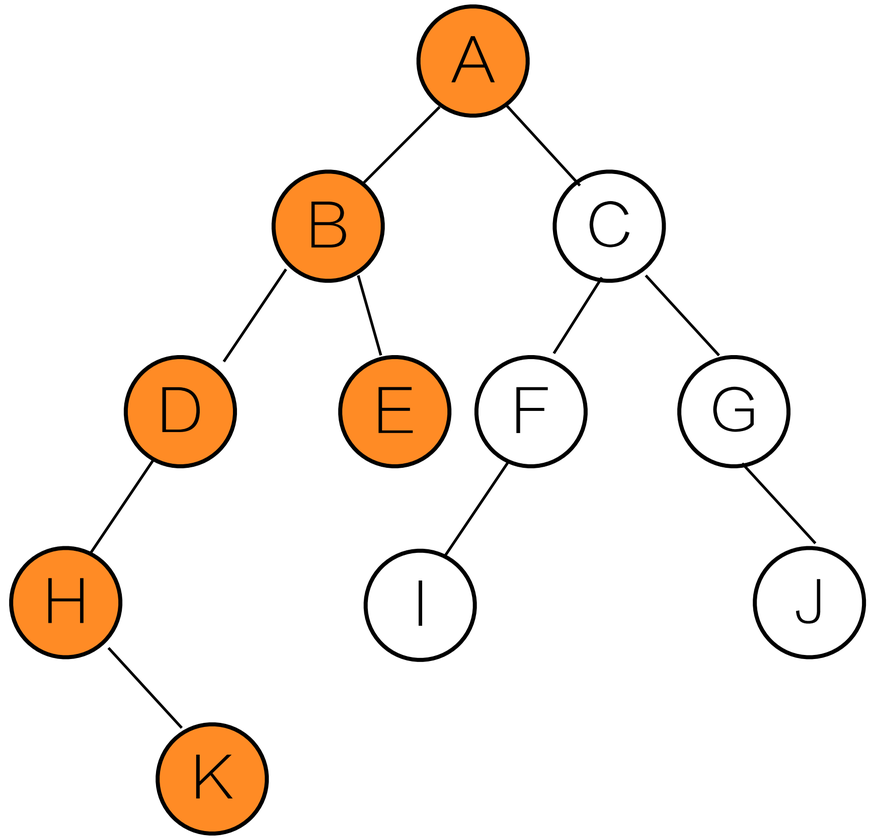


二叉树的遍历 – 中序遍历

输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

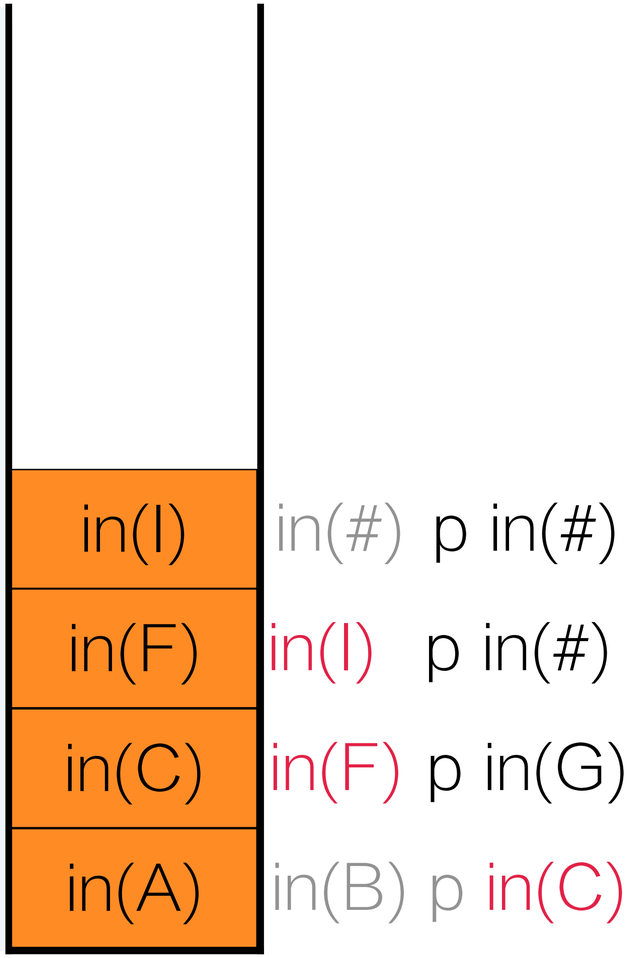




二叉树的遍历 – 中序遍历

输出内容：H K D B E A

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

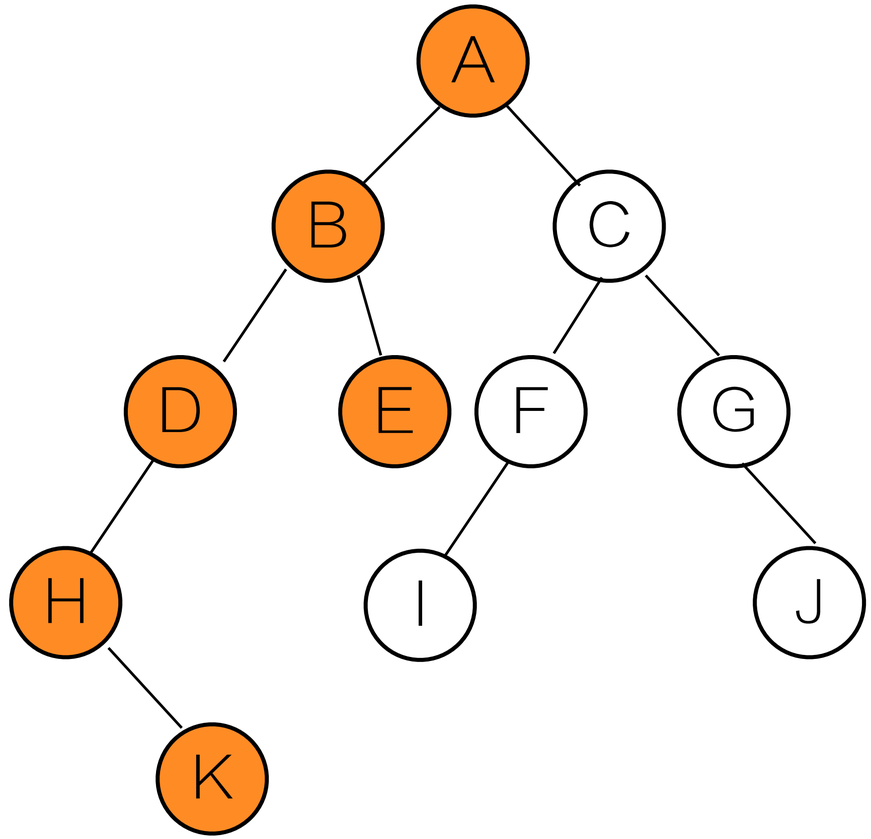


in(#) p in(#)

in(I) p in(#)

in(F) p in(G)

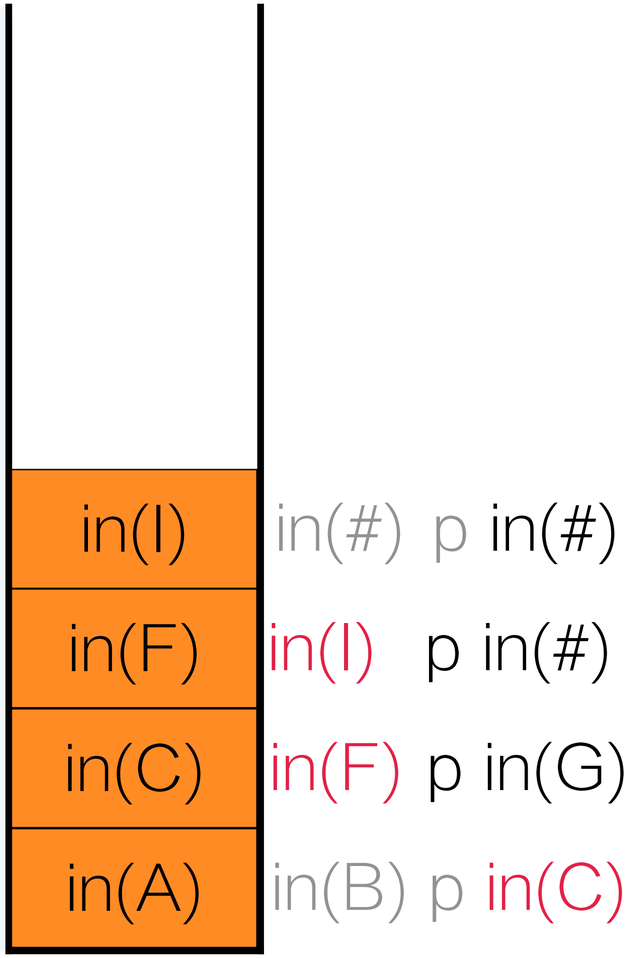
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

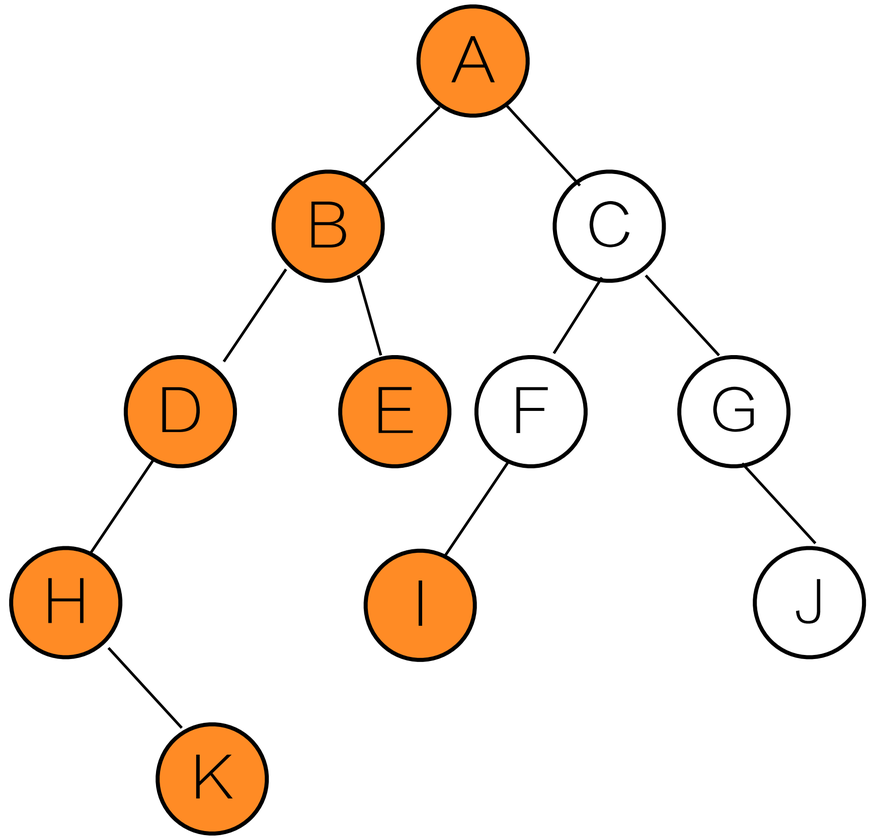


in(#) p in(#)

in(I) p in(#)

in(F) p in(G)

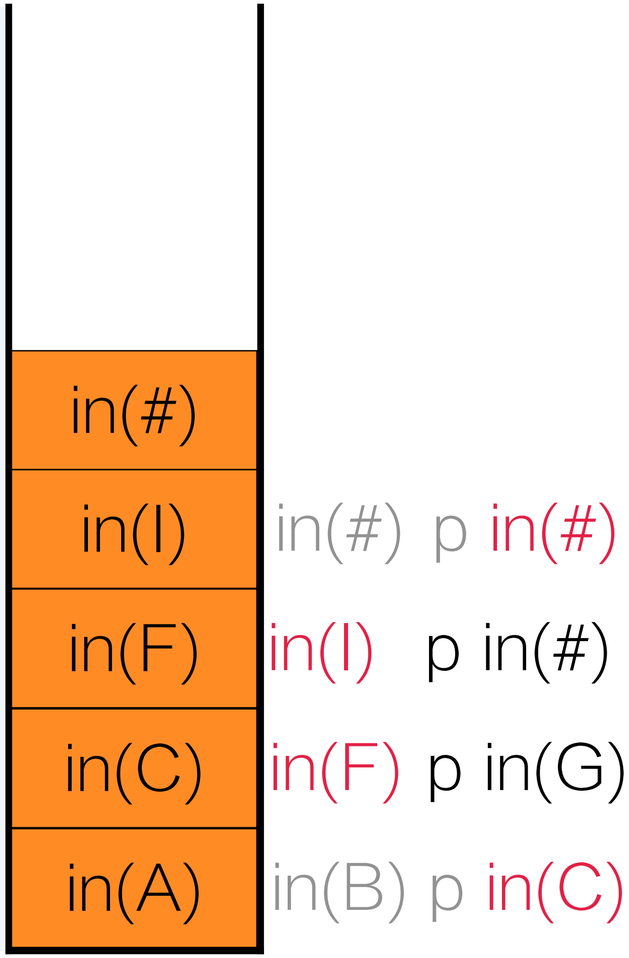
in(B) p in(C)

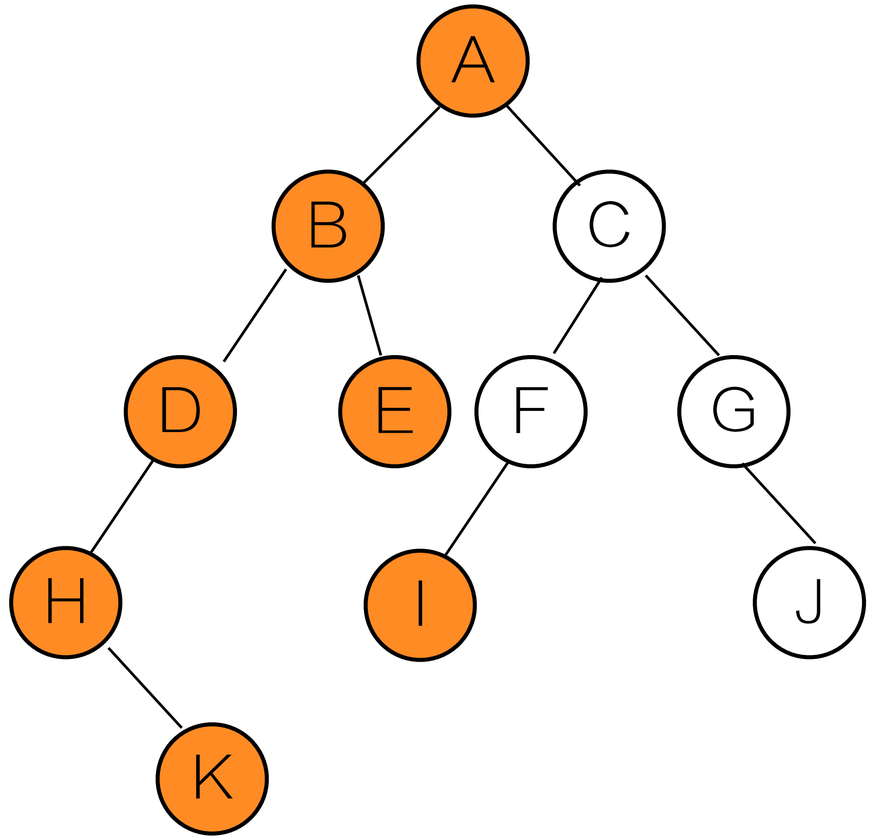


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

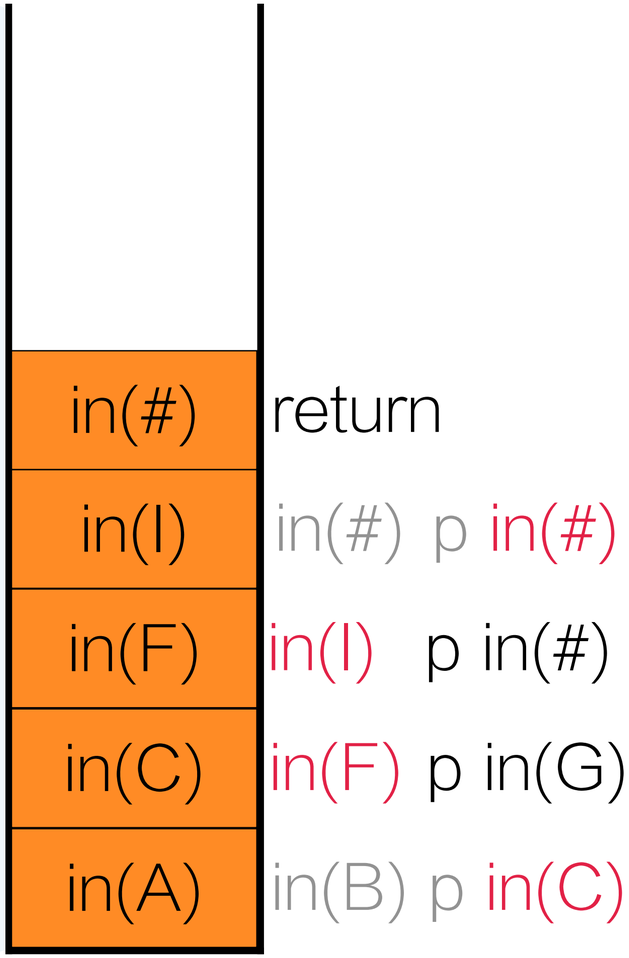


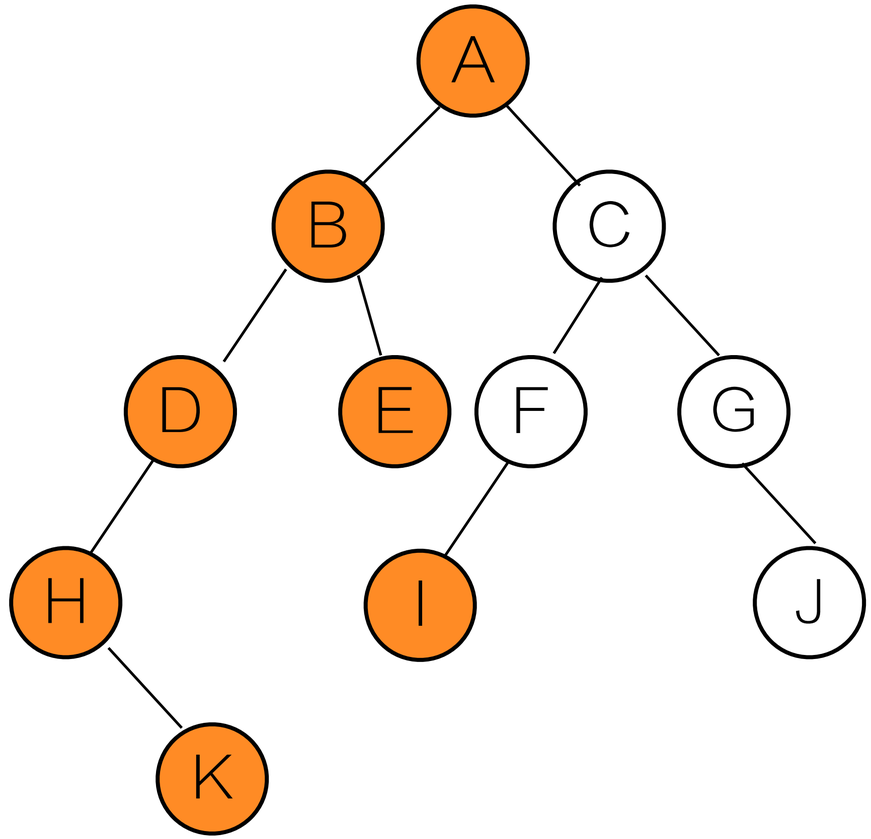


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

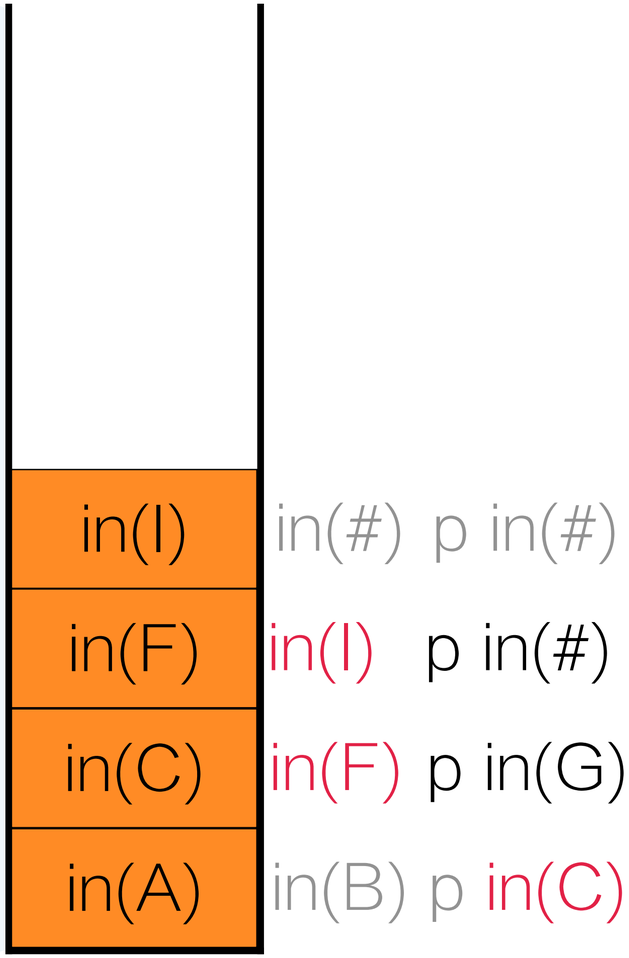




二叉树的遍历 – 中序遍历

输出内容：H K D B E A I

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

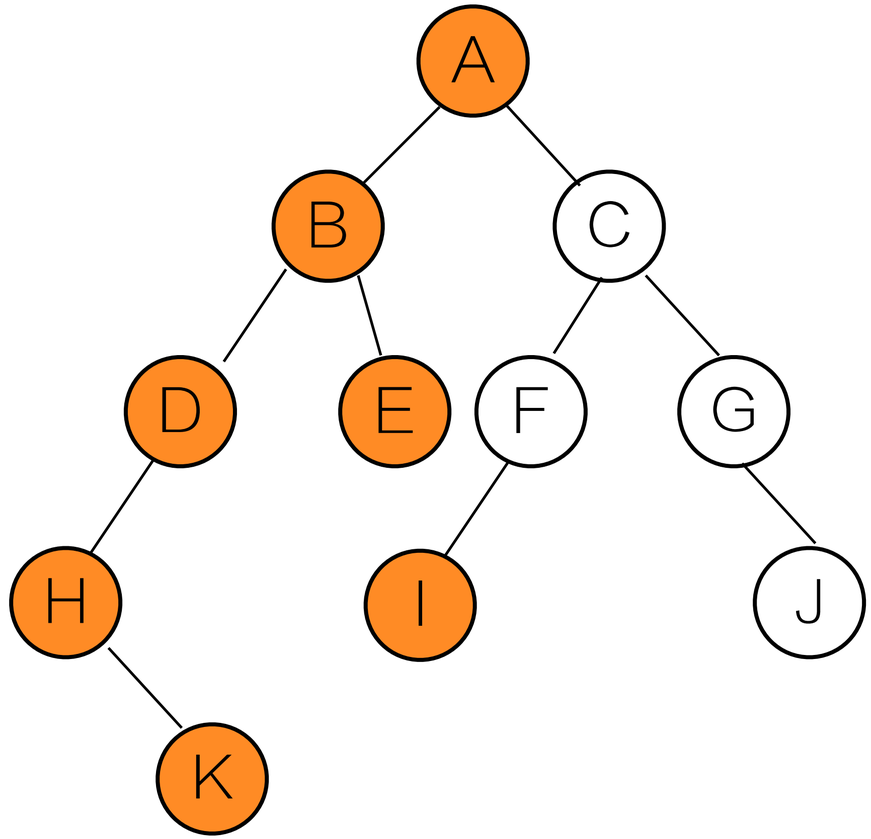


in(#) p in(#)

in(I) p in(#)

in(F) p in(G)

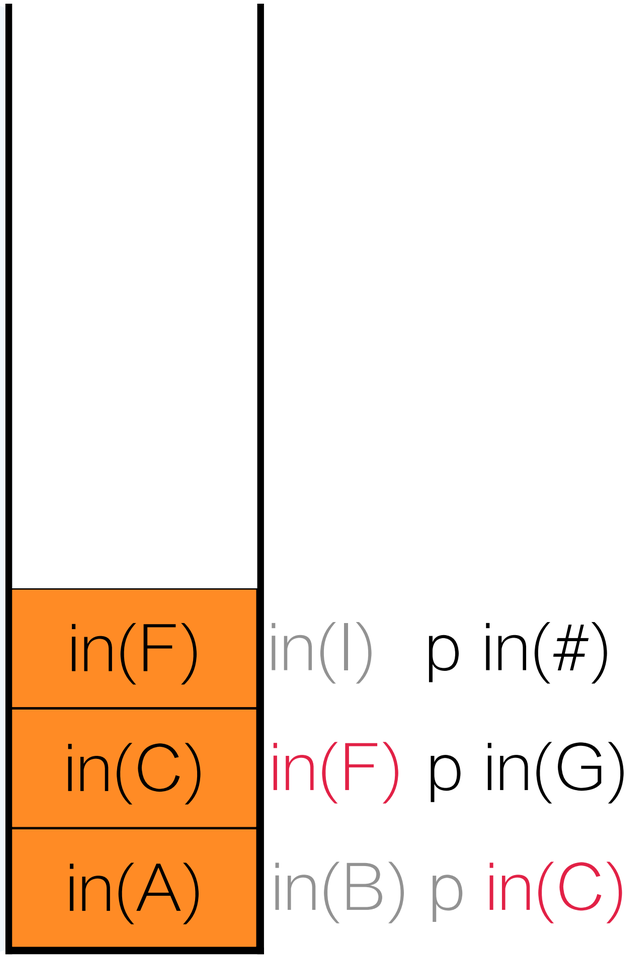
in(B) p in(C)

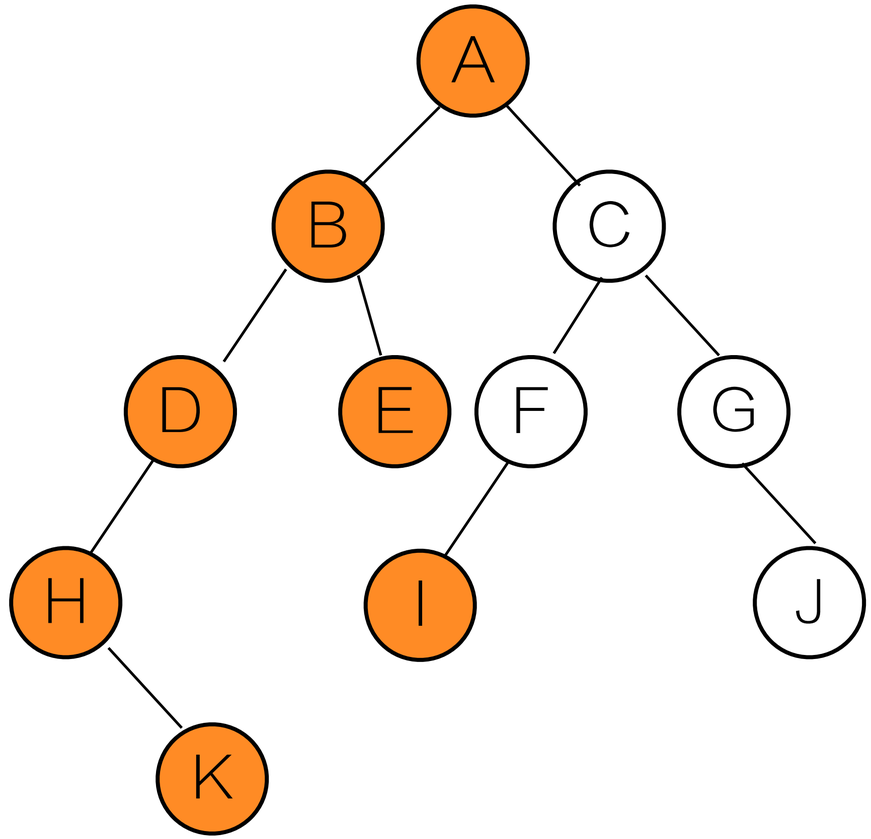


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

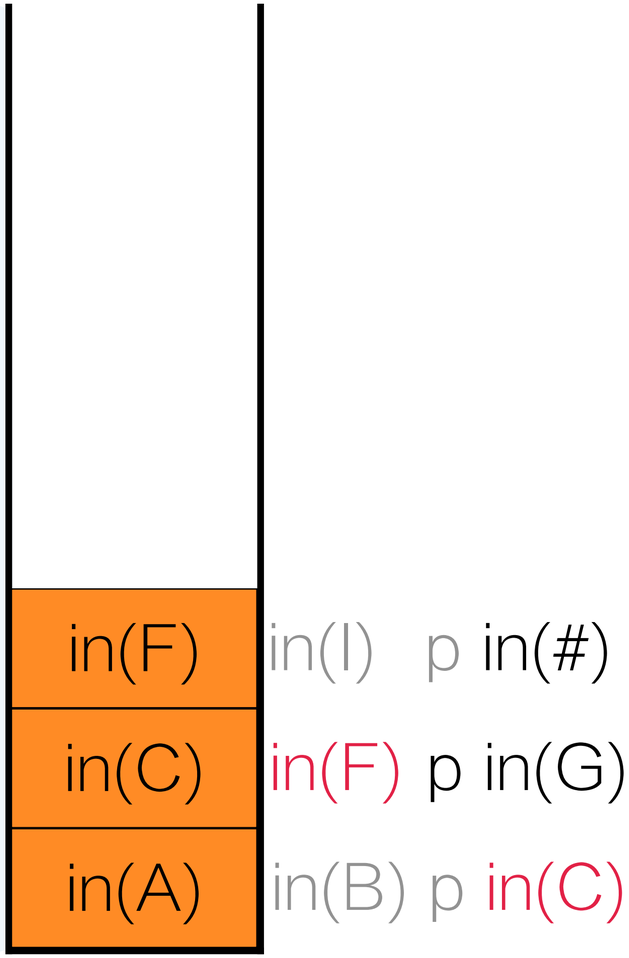




二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F

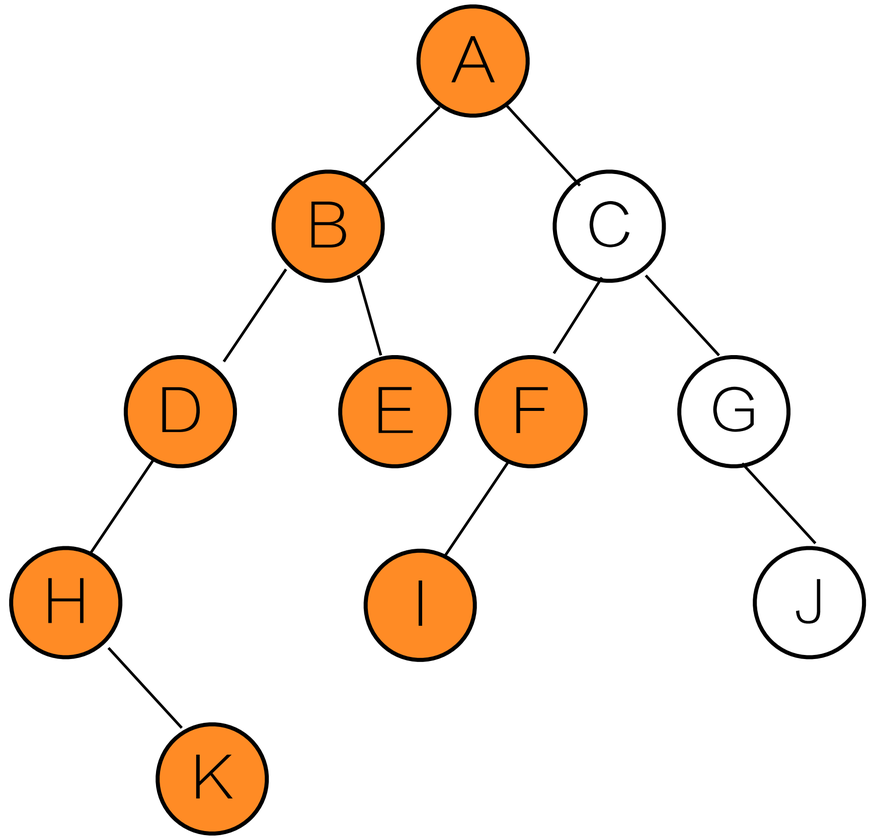
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(I) p in(#)

in(F) p in(G)

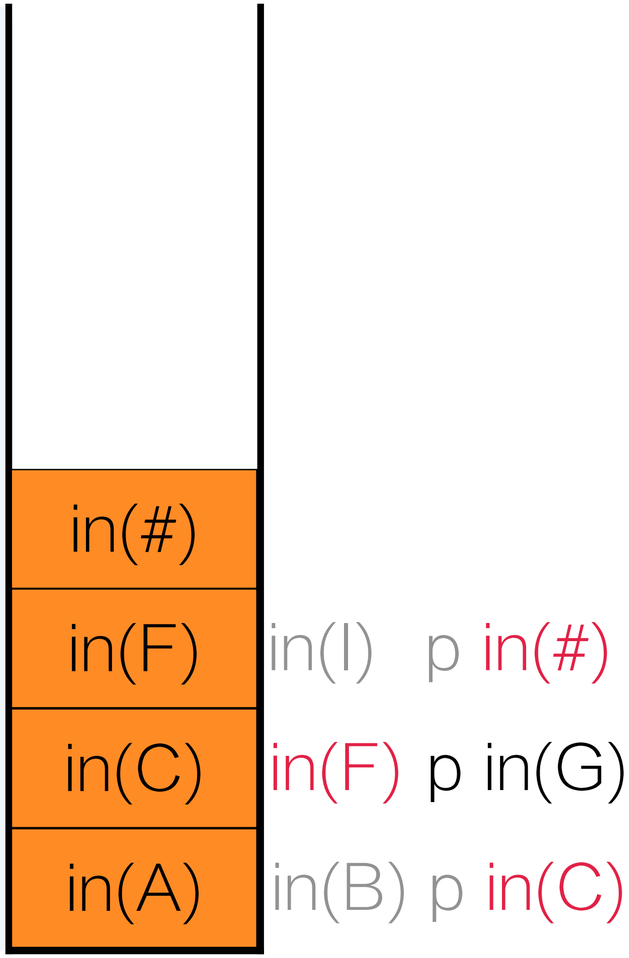
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F

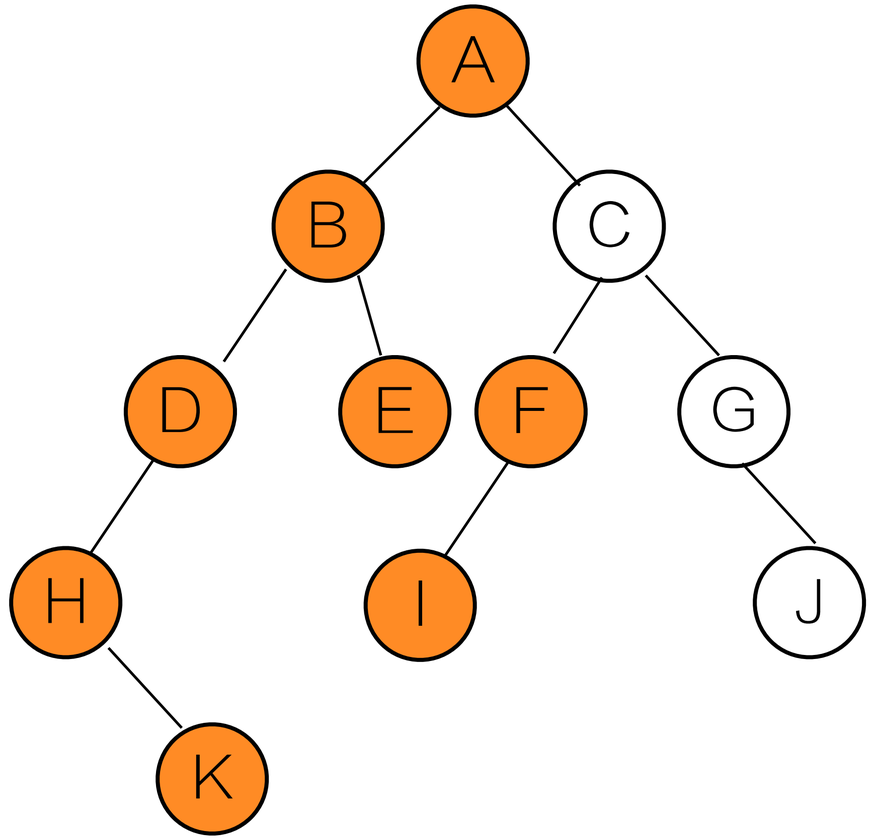
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(I) p in(#)

in(F) p in(G)

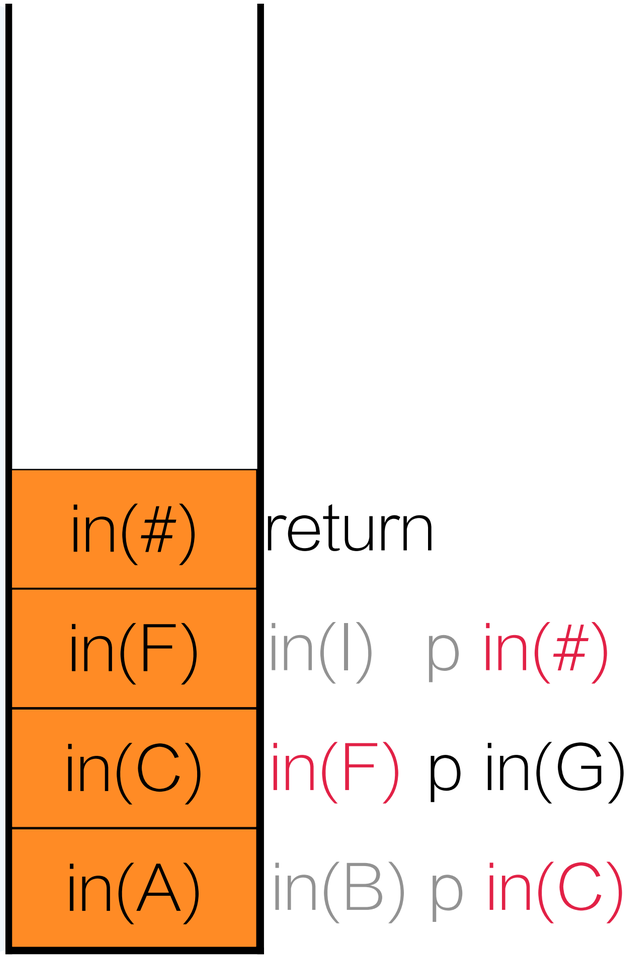
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

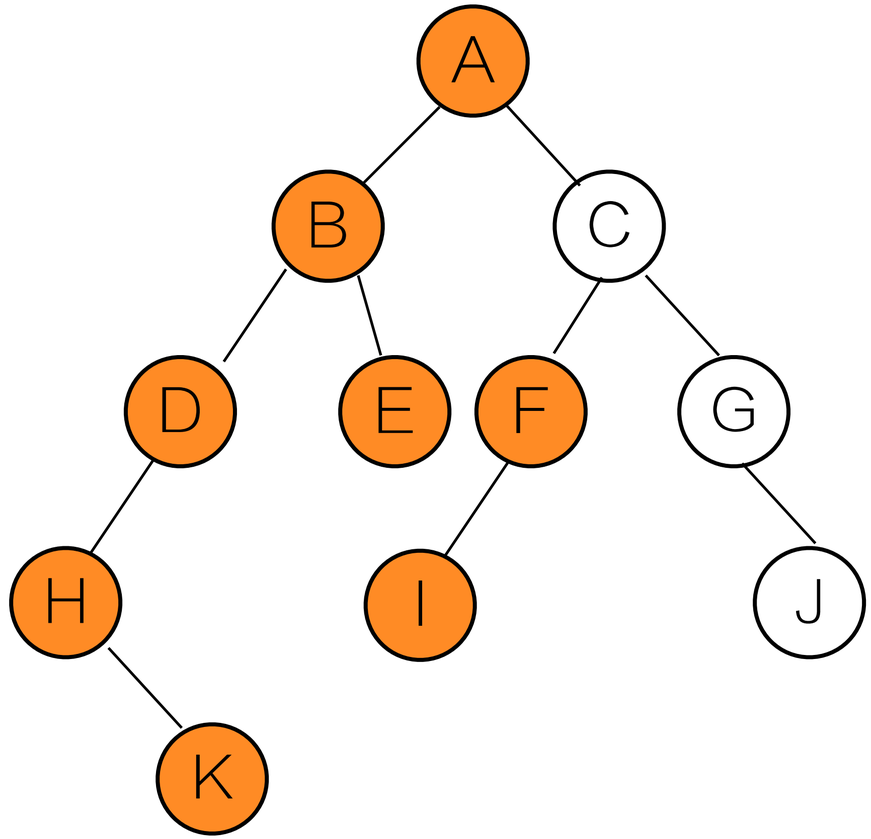


return

in(I) p in(#)

in(F) p in(G)

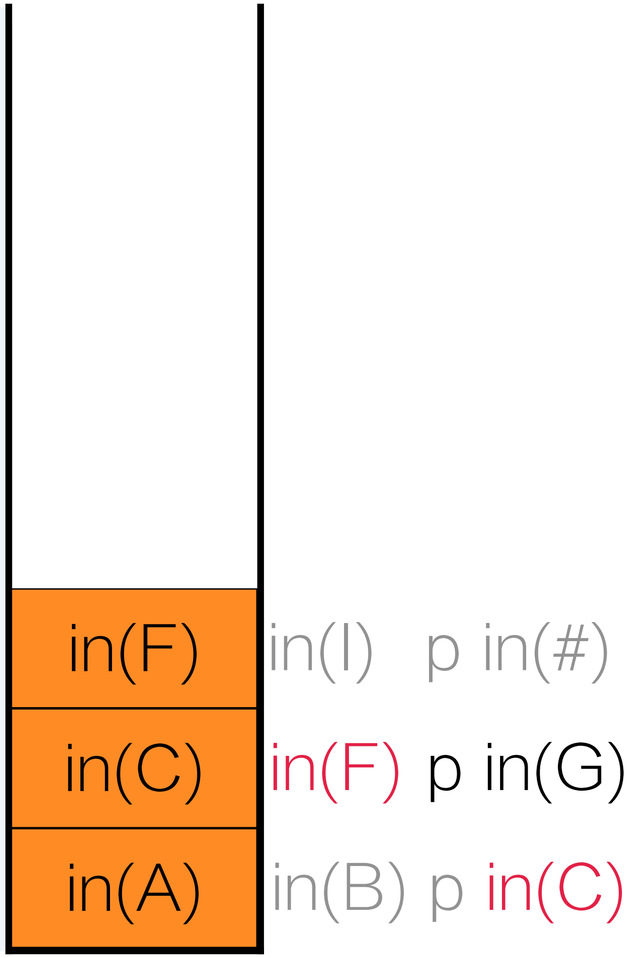
in(B) p in(C)

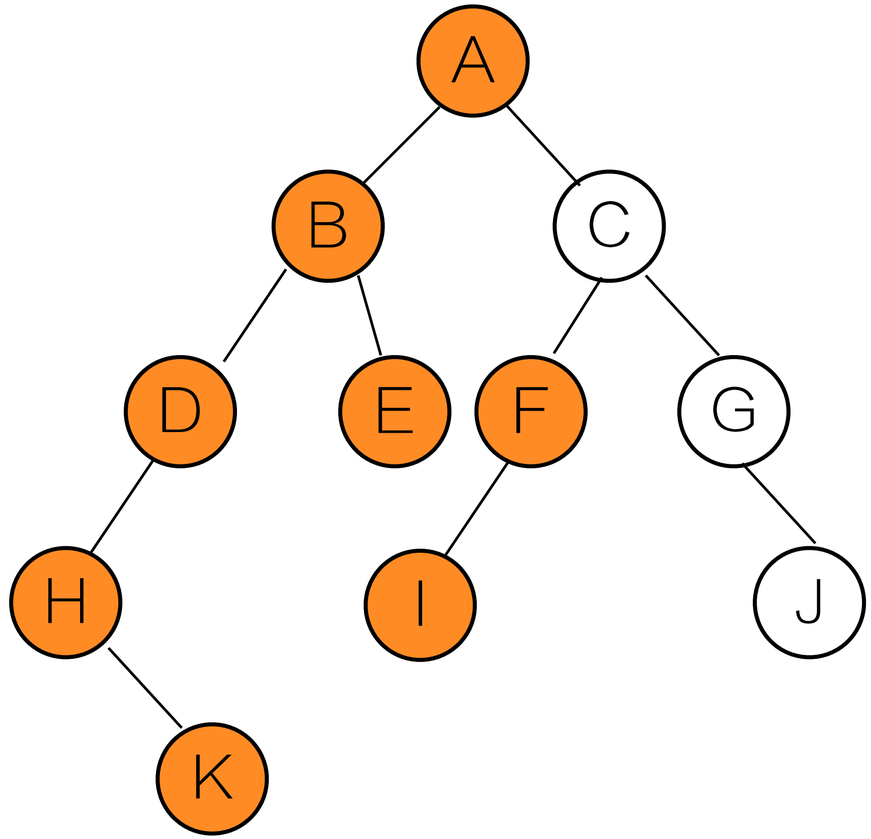


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

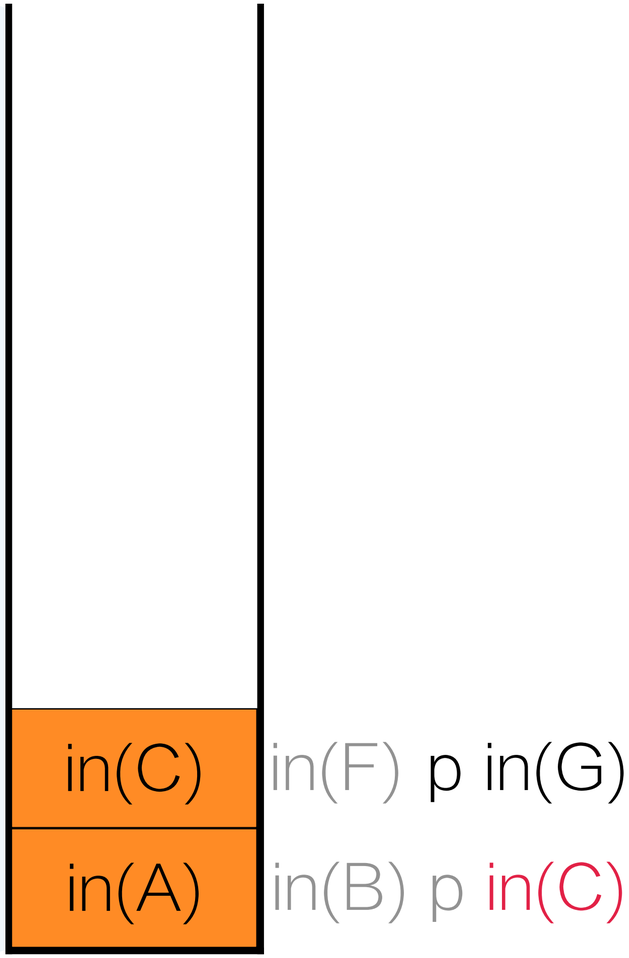




二叉树的遍历 – 中序遍历

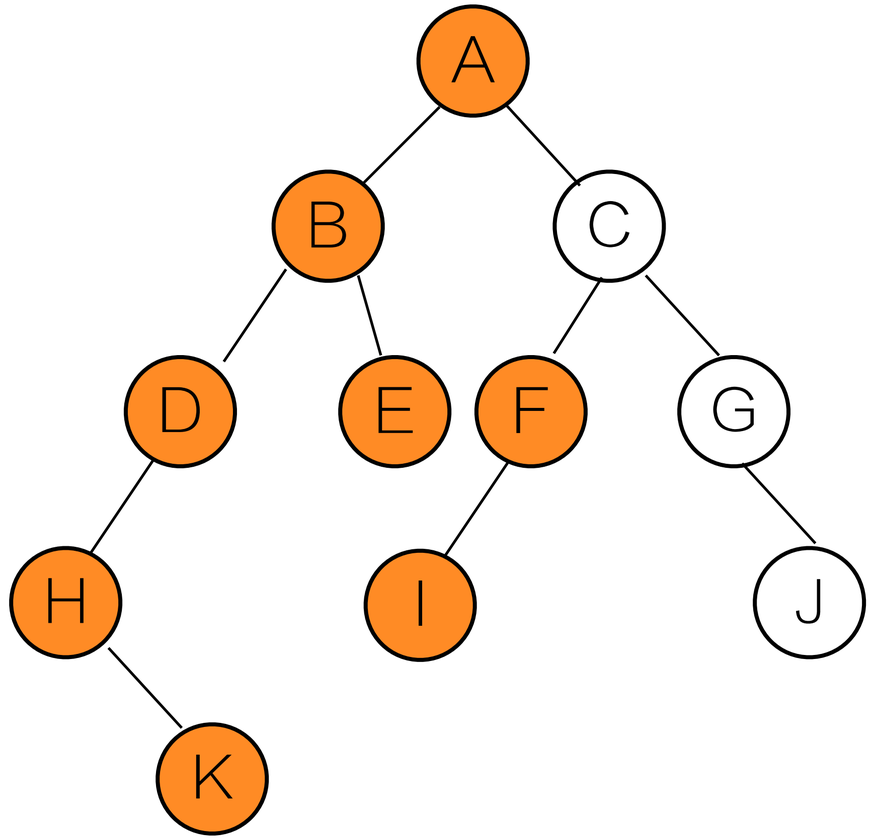
输出内容：H K D B E A I F

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(F) p in(G)

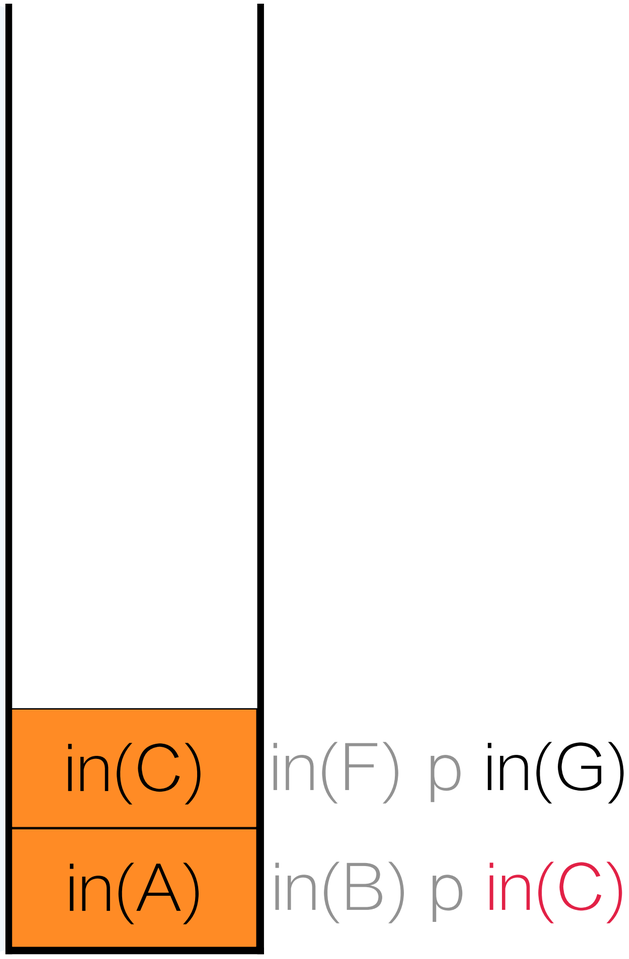
in(B) p in(C)



二叉树的遍历 – 中序遍历

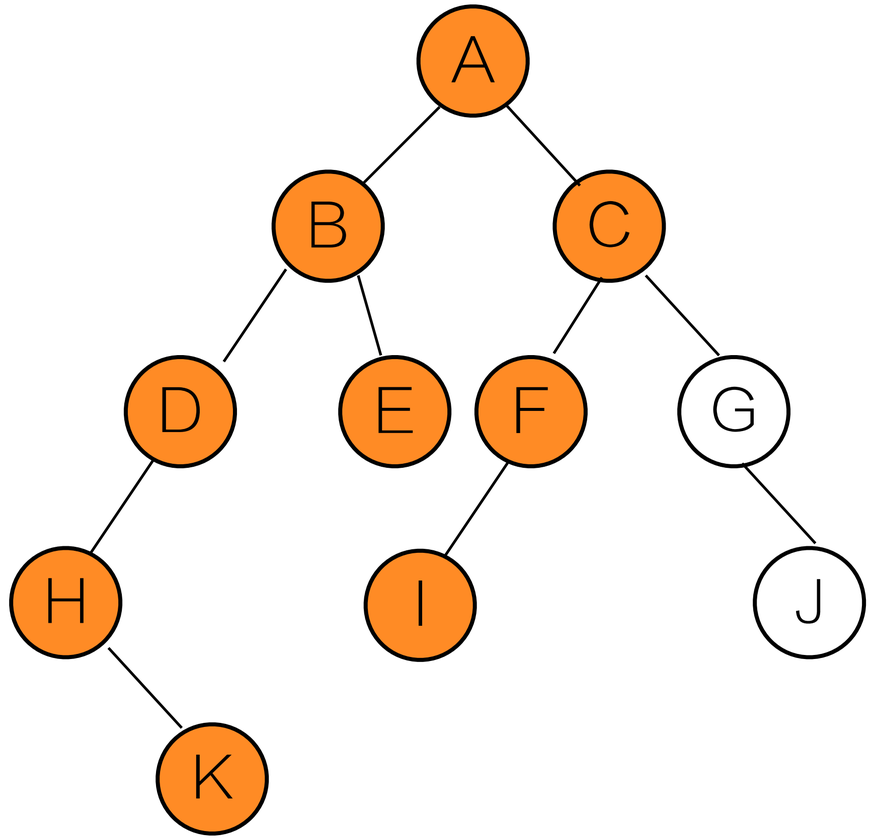
输出内容：H K D B E A I F C

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(F) p in(G)

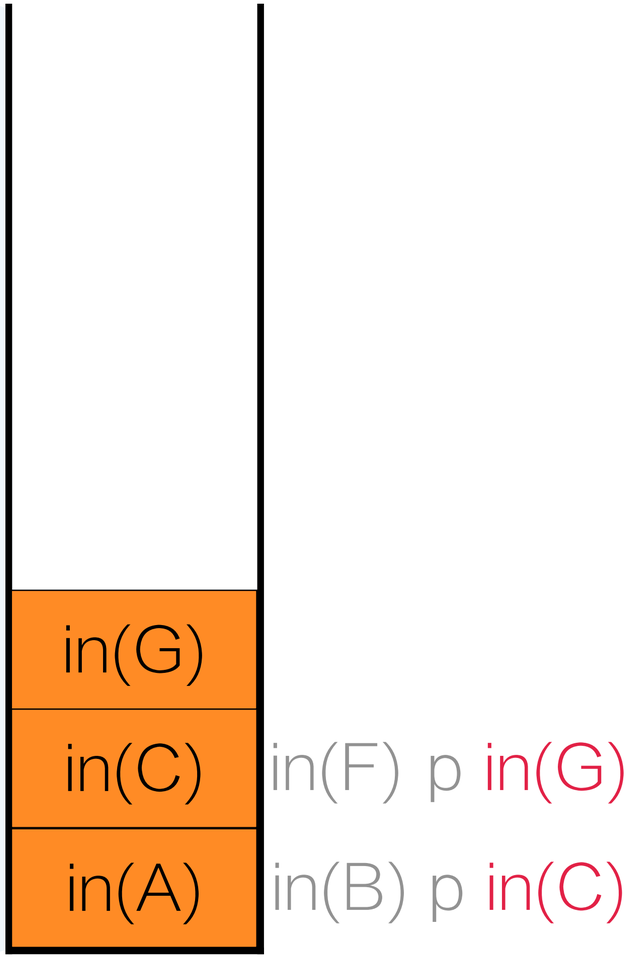
in(B) p in(C)



二叉树的遍历 – 中序遍历

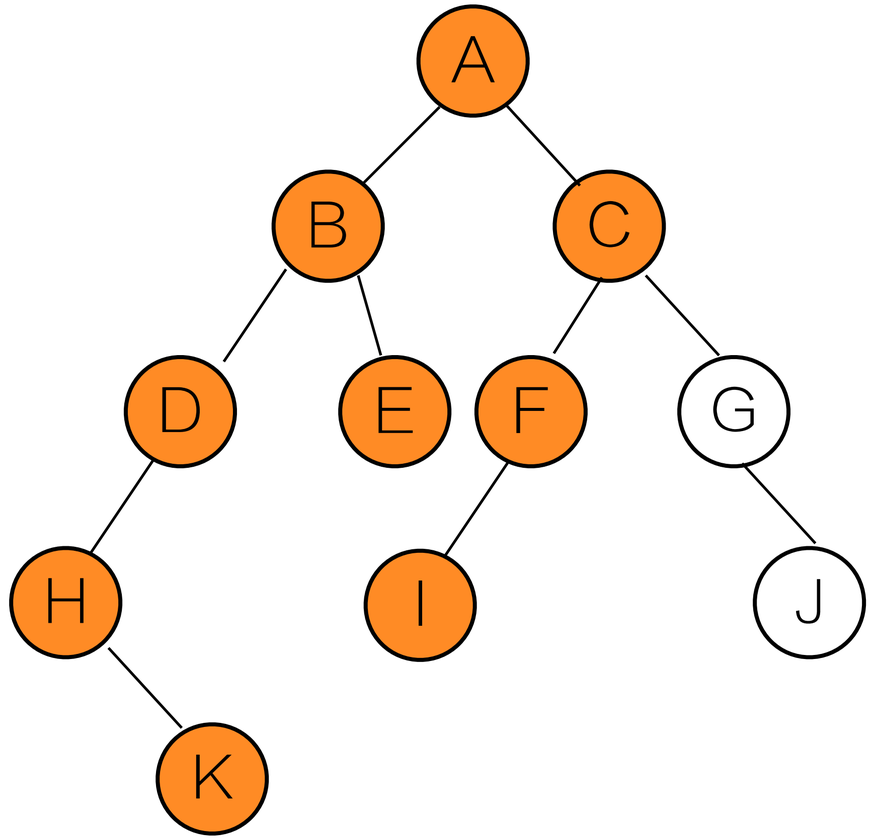
输出内容：H K D B E A I F C

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(F) p in(G)

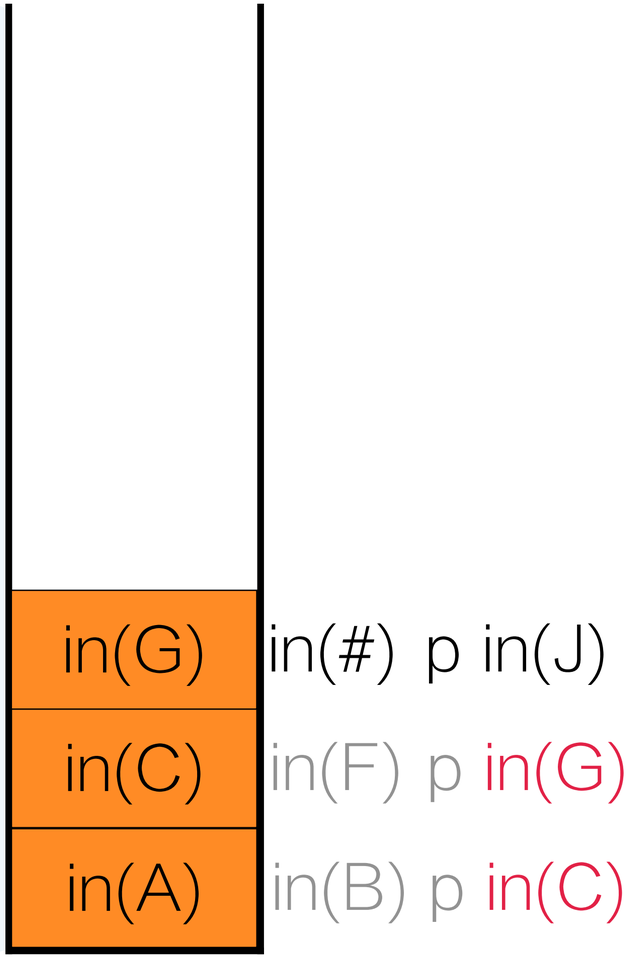
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C

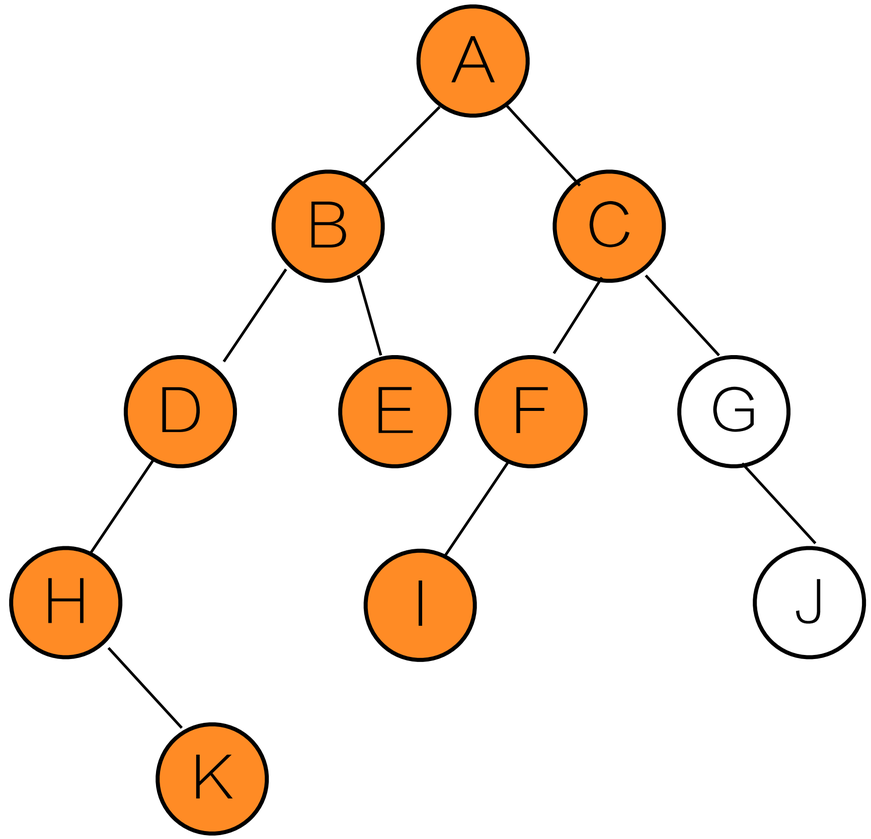
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(J)

in(F) p in(G)

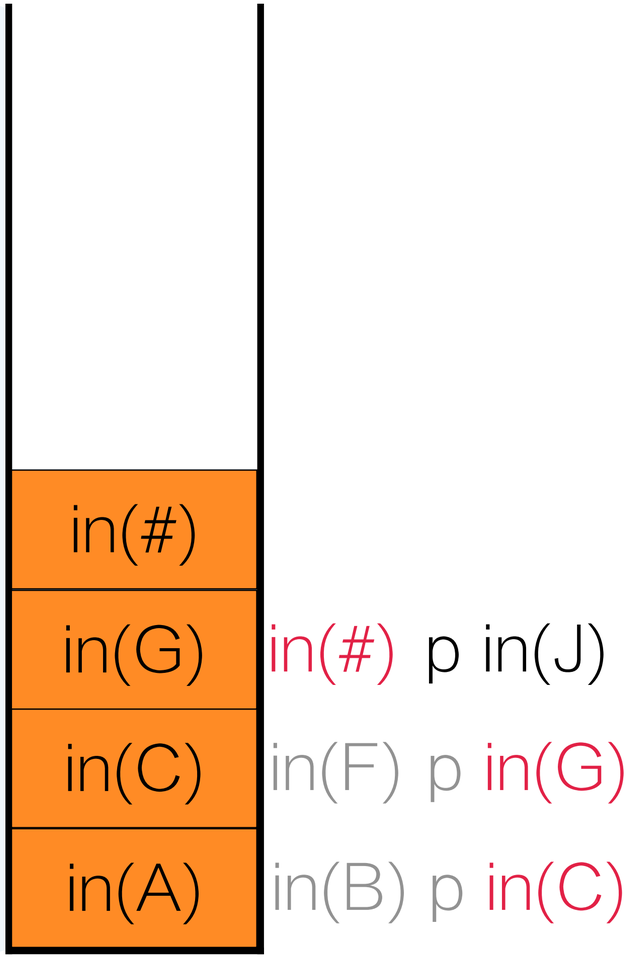
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C

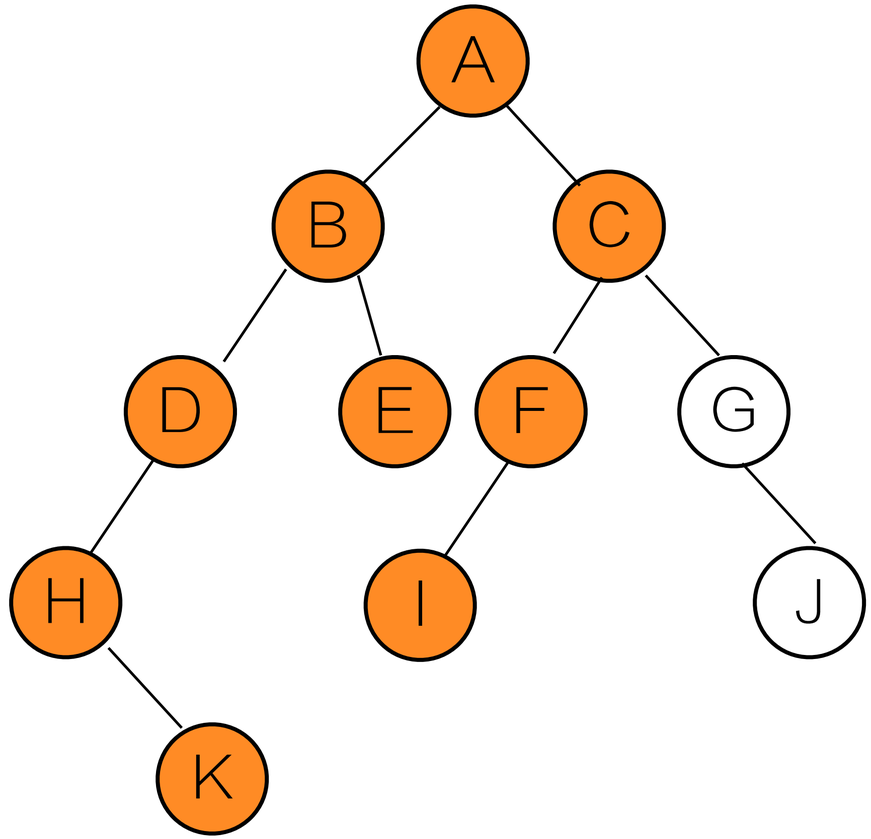
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(J)

in(F) p in(G)

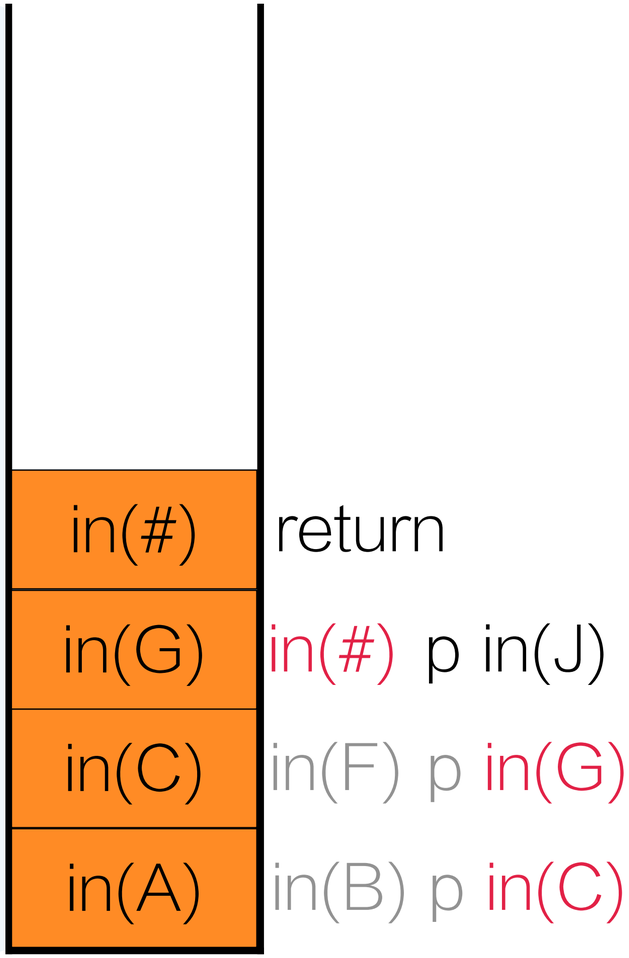
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

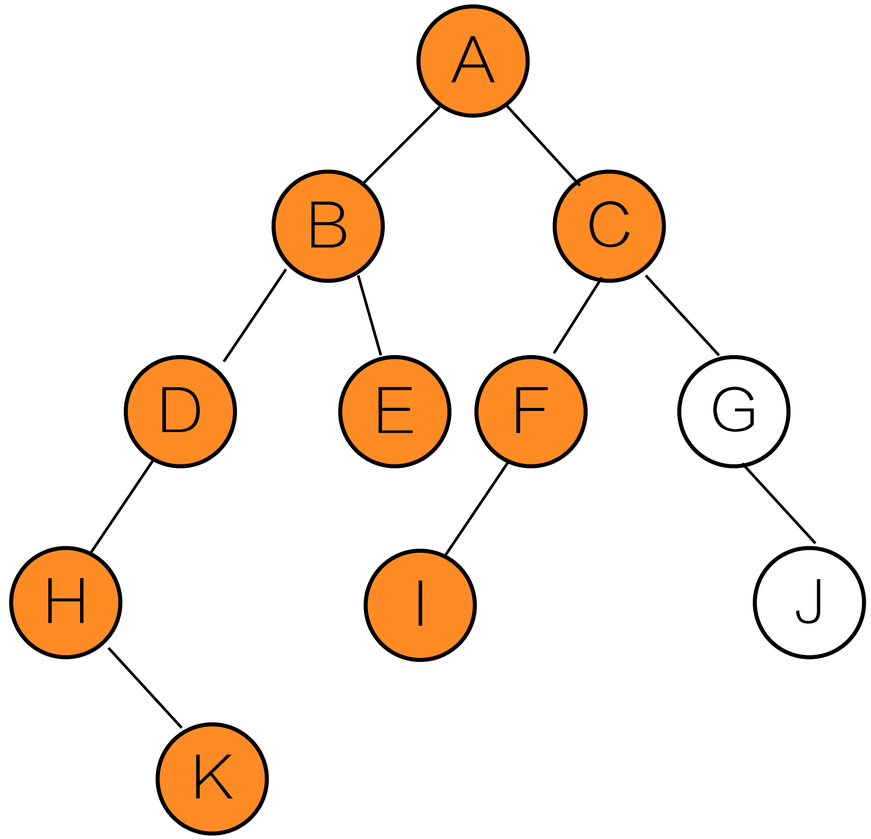


return

in(#) p in(J)

in(F) p in(G)

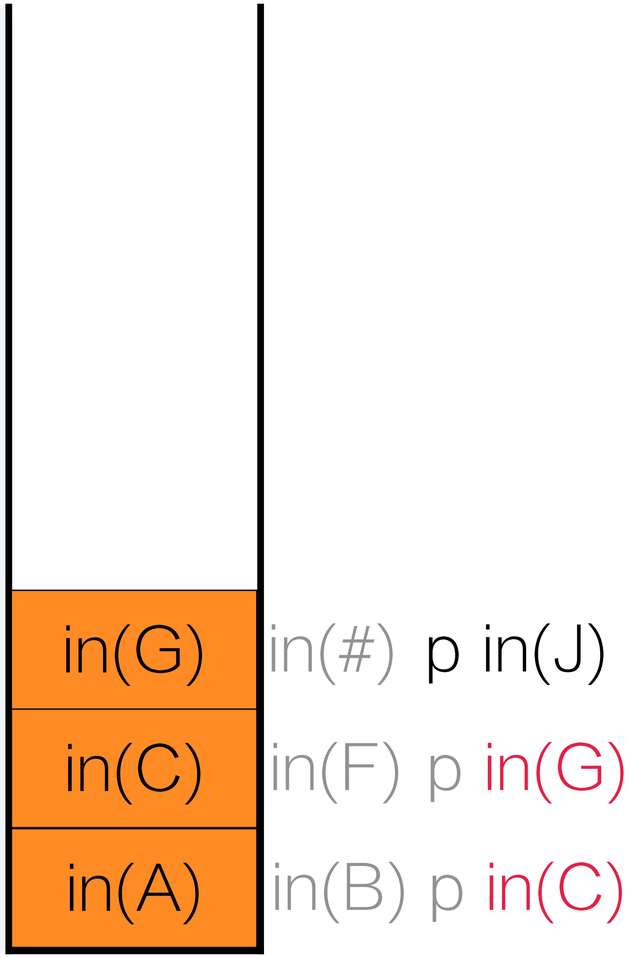
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C

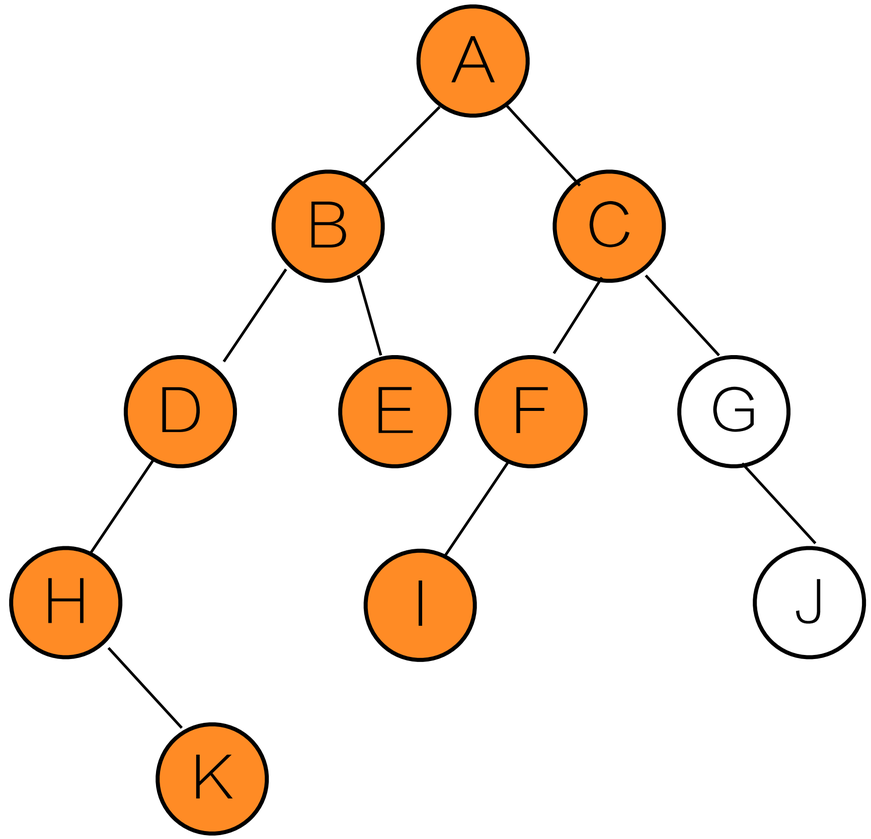
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(J)

in(F) p in(G)

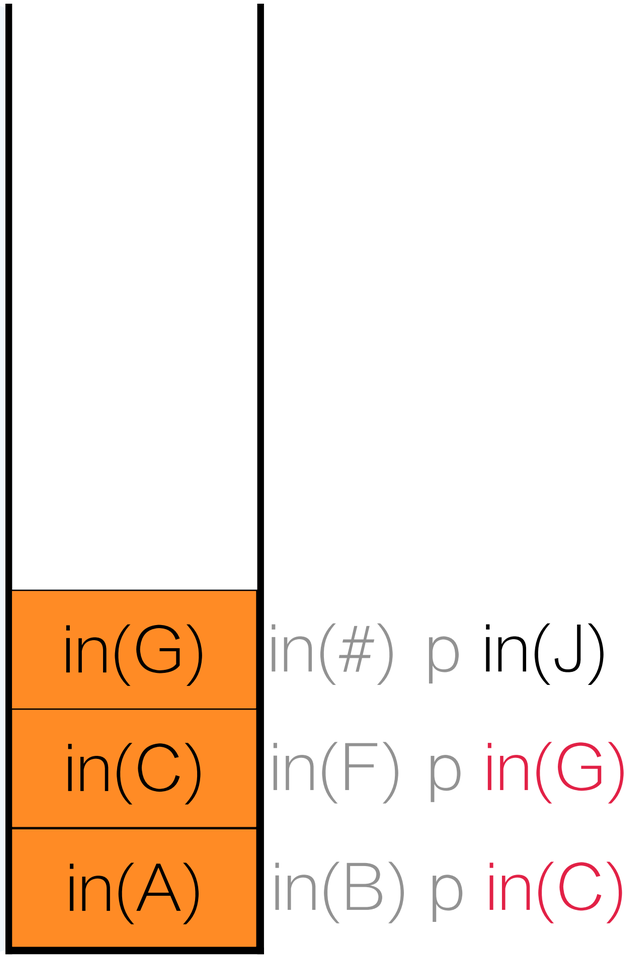
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G

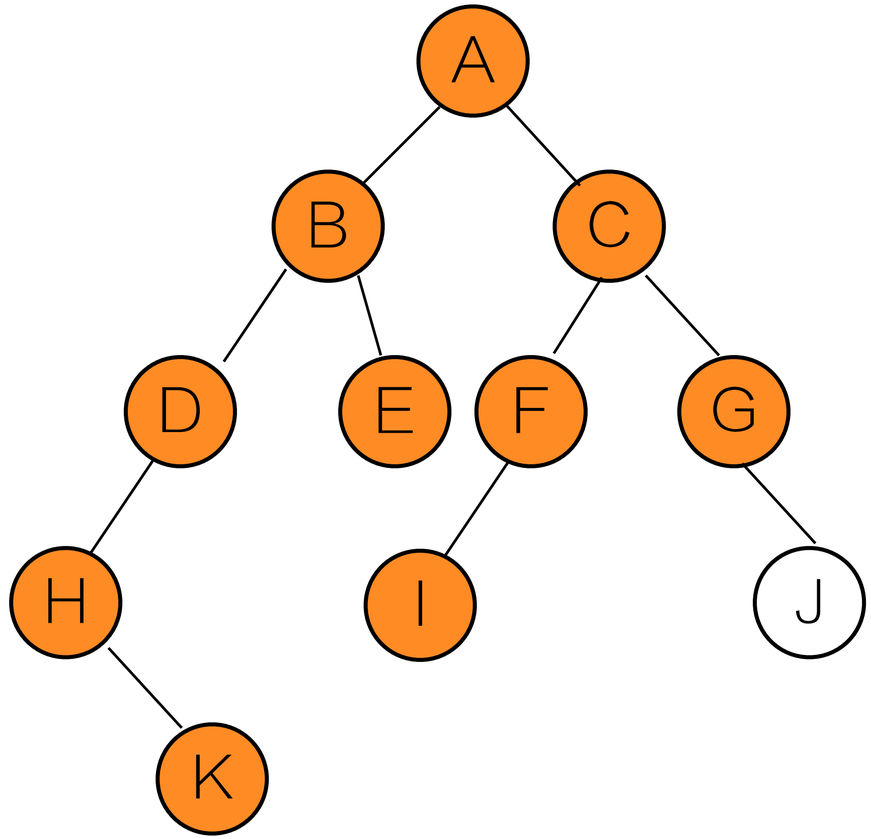
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(J)

in(F) p in(G)

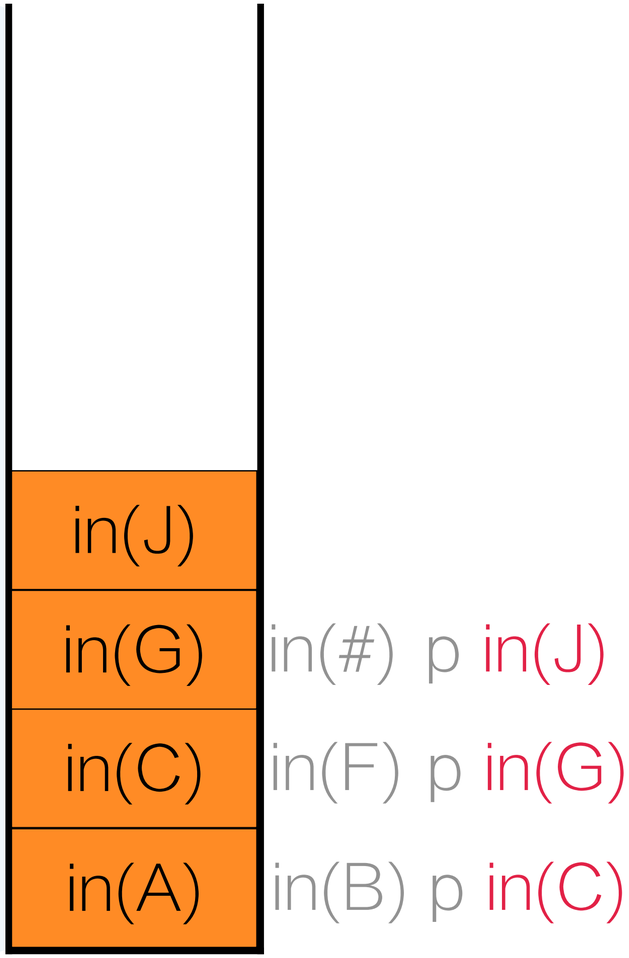
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G

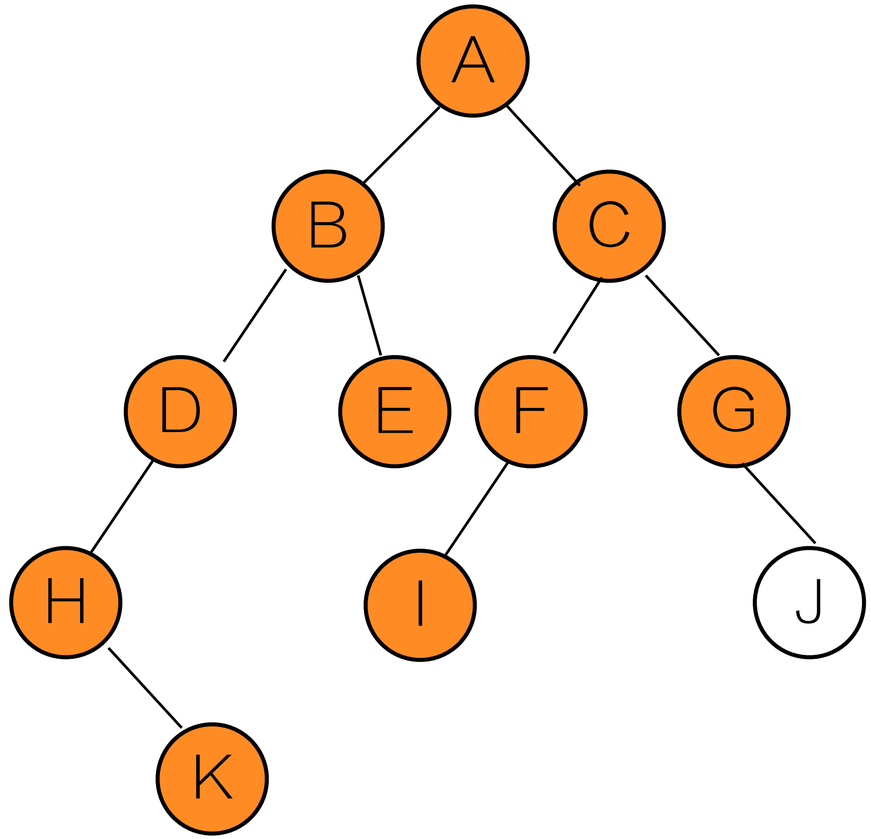
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(J)

in(F) p in(G)

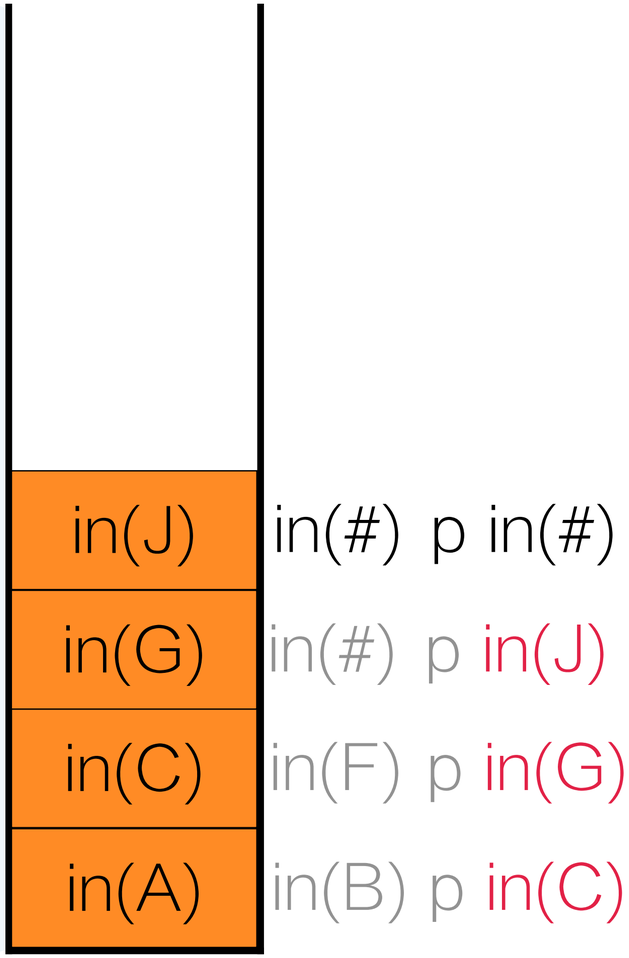
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

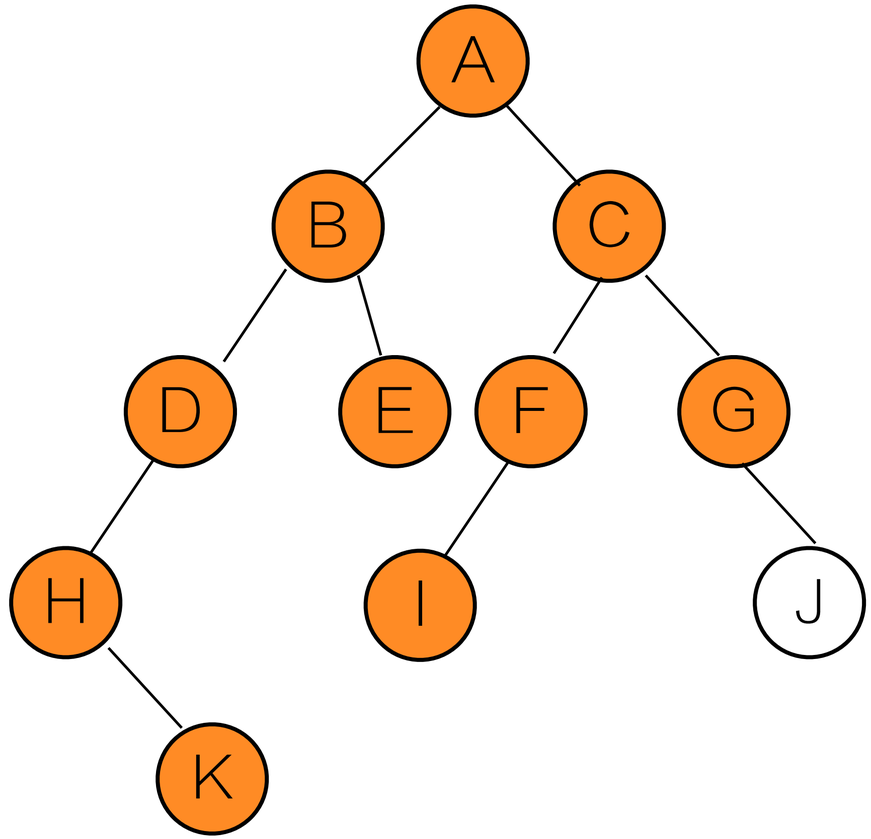


in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

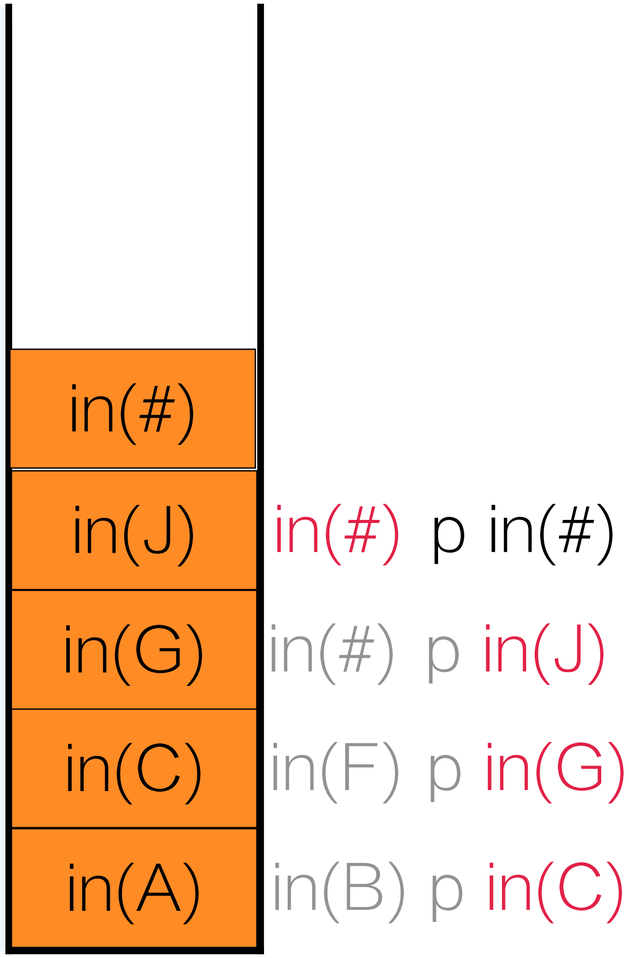
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

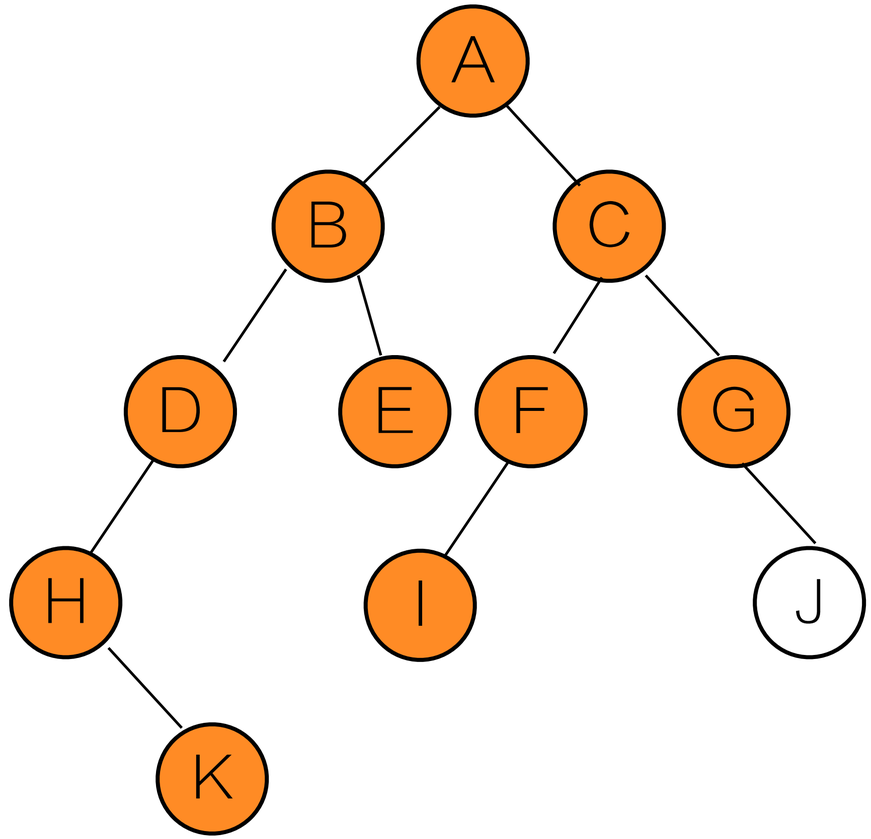


in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

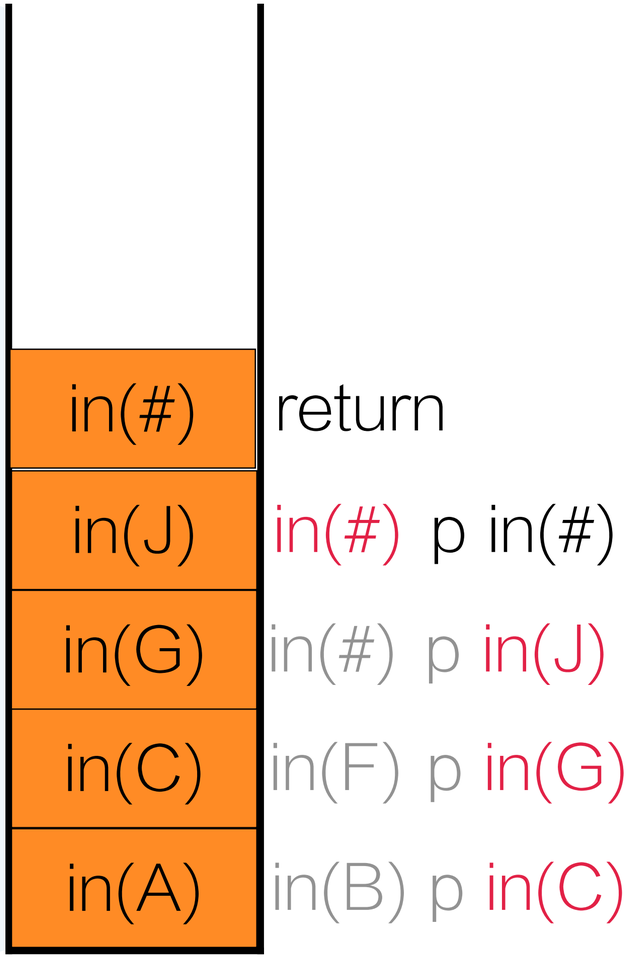
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



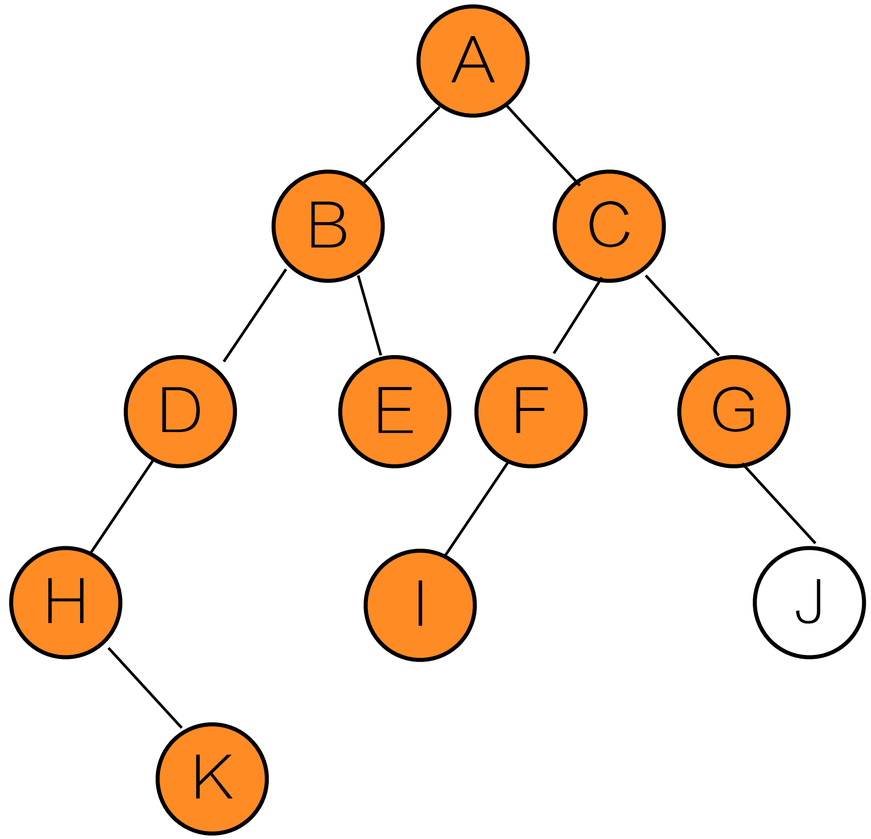
return

in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

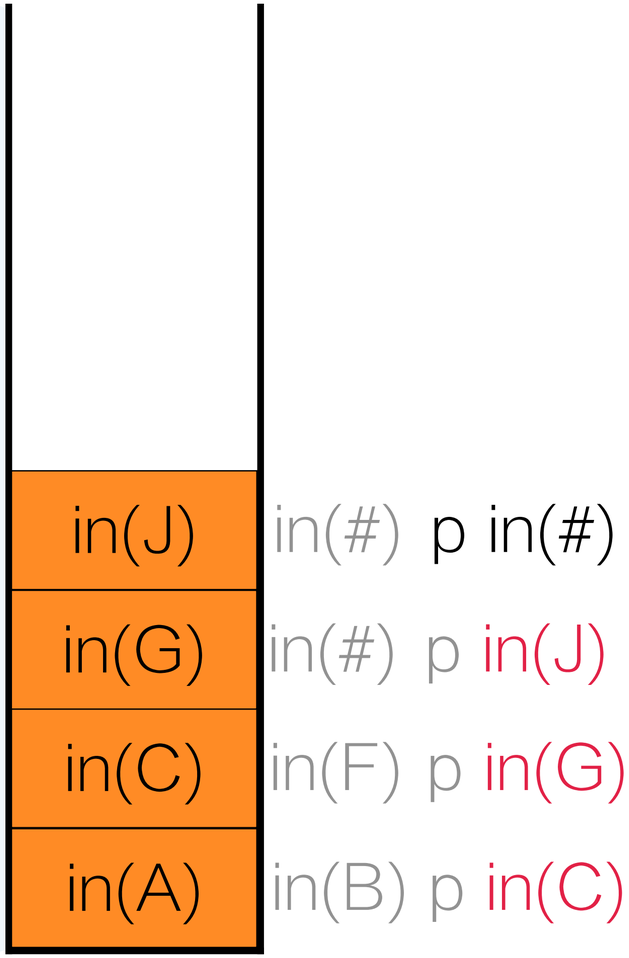
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

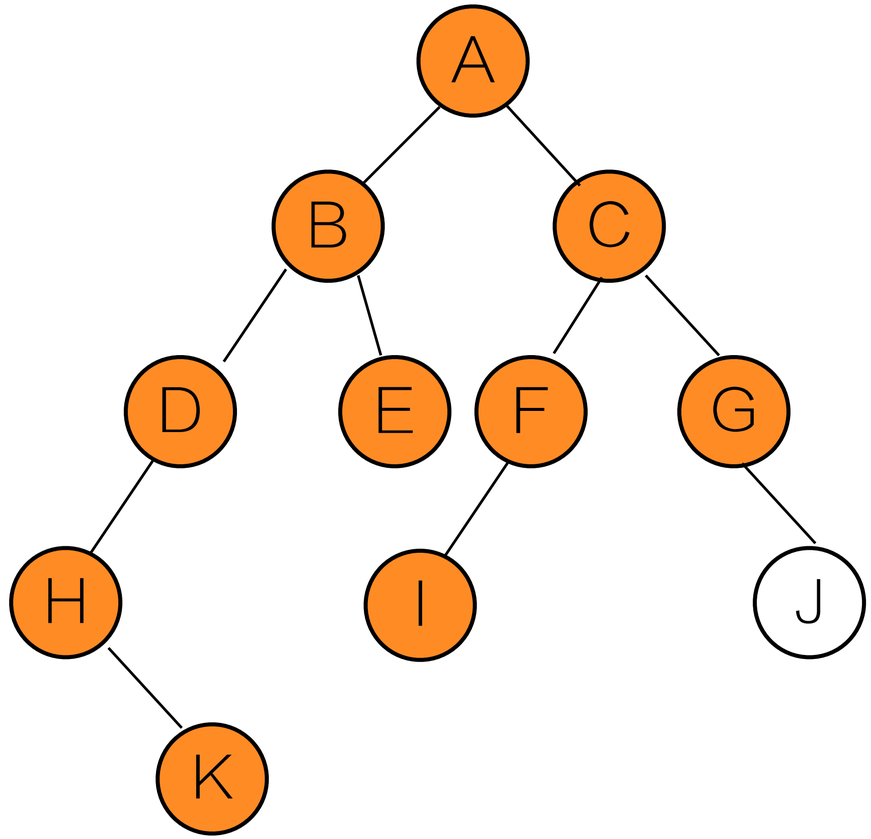


in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

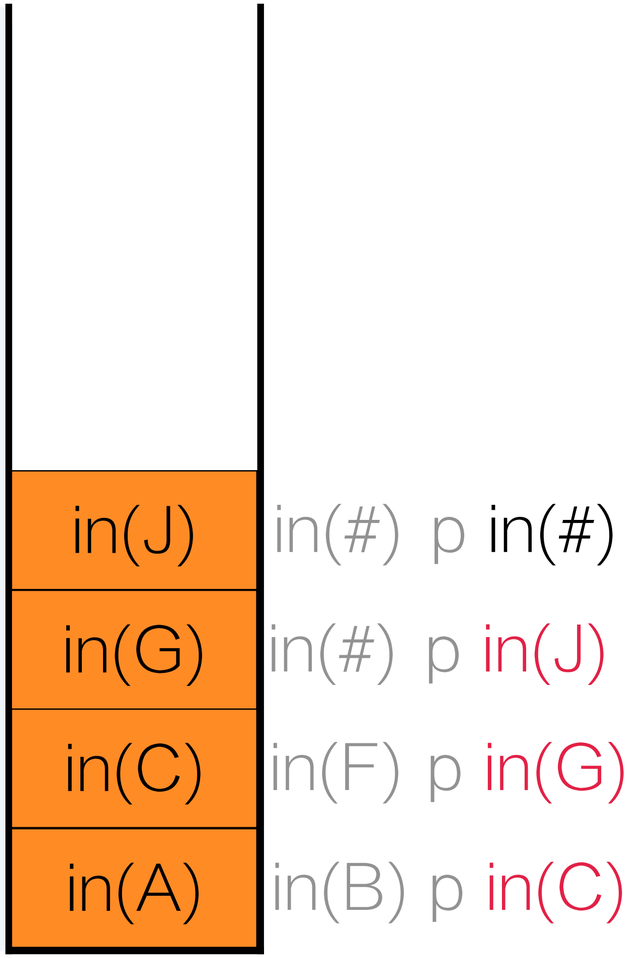
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

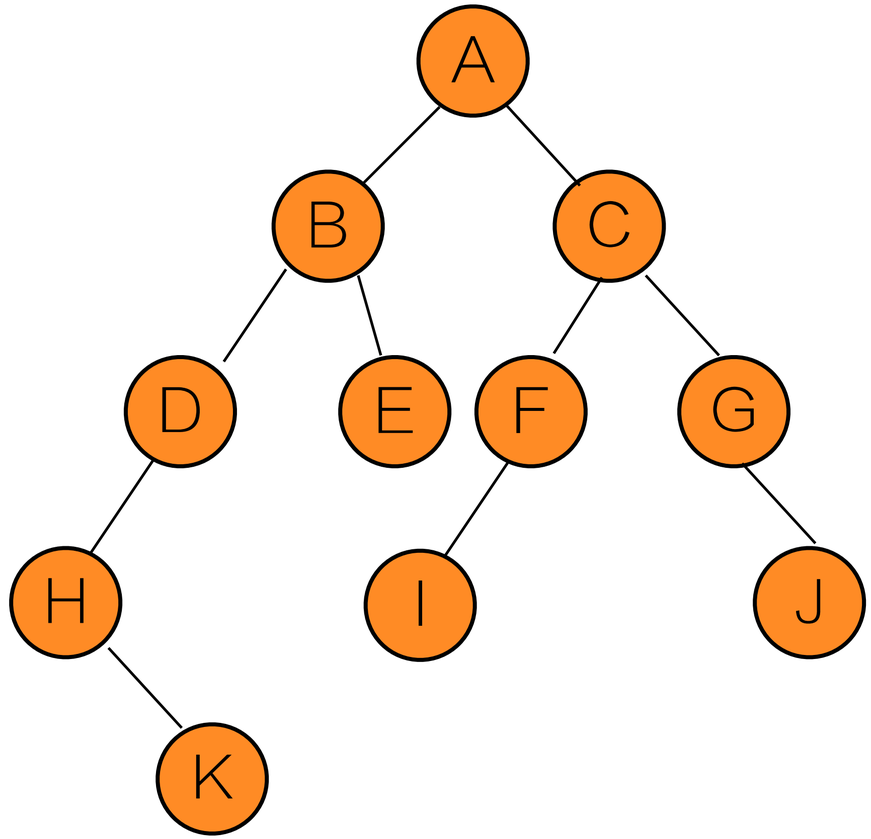


in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

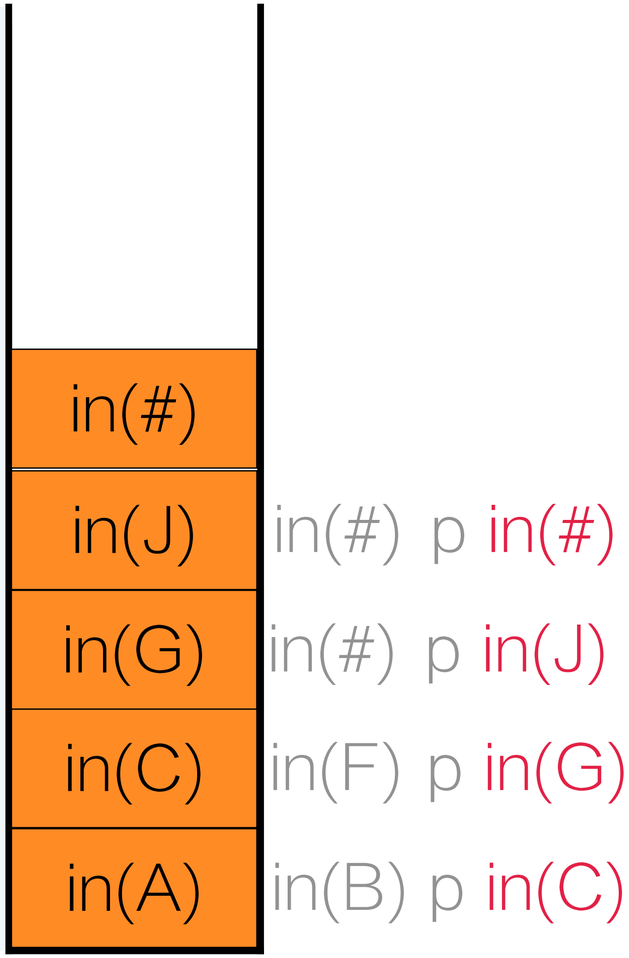
in(B) p in(C)

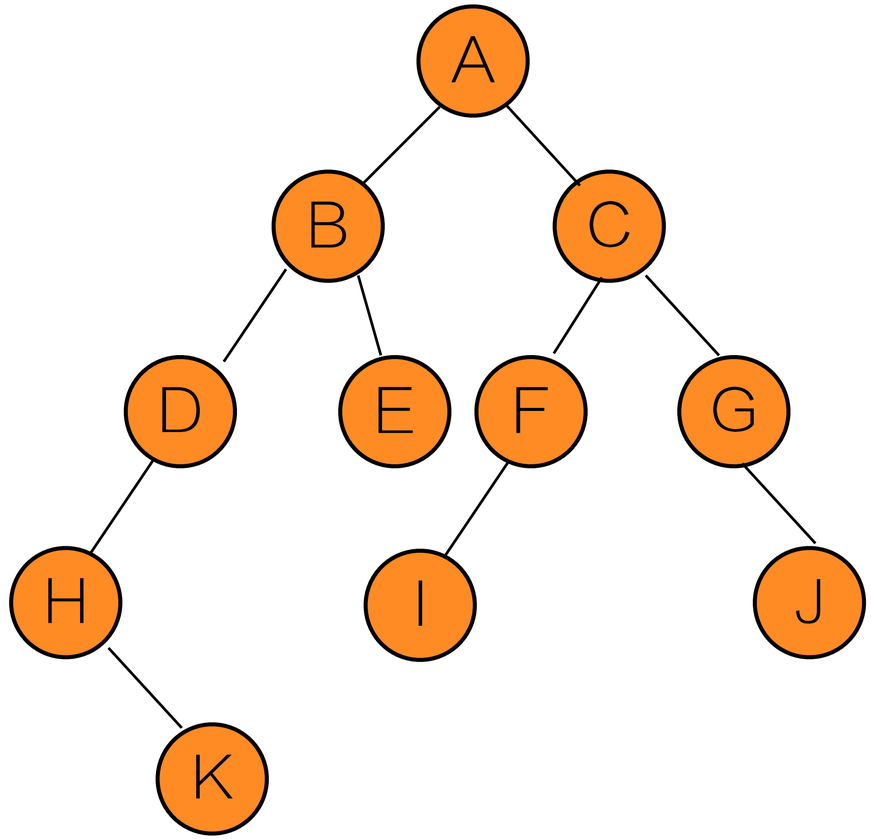


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

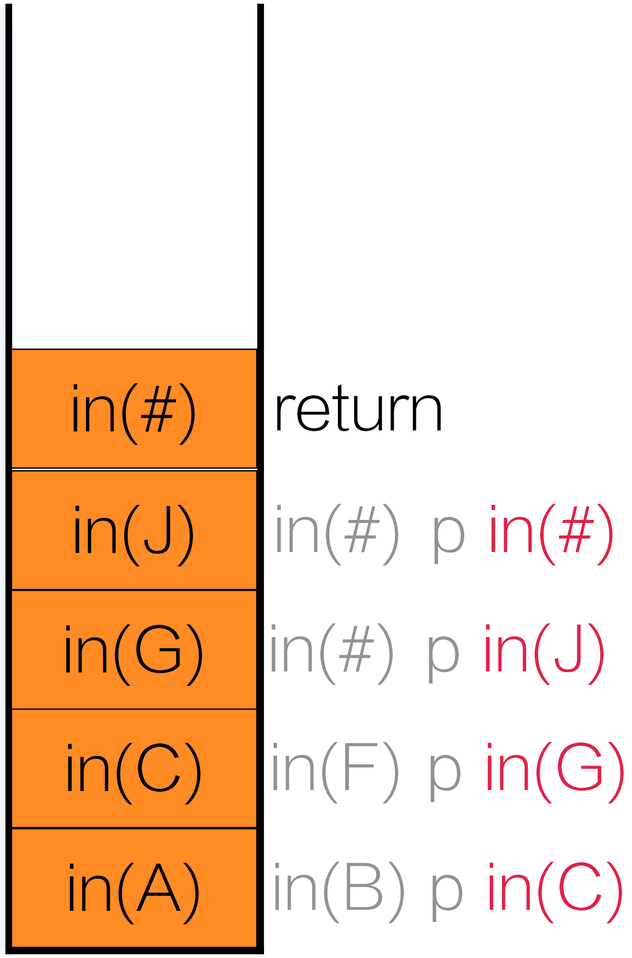




二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



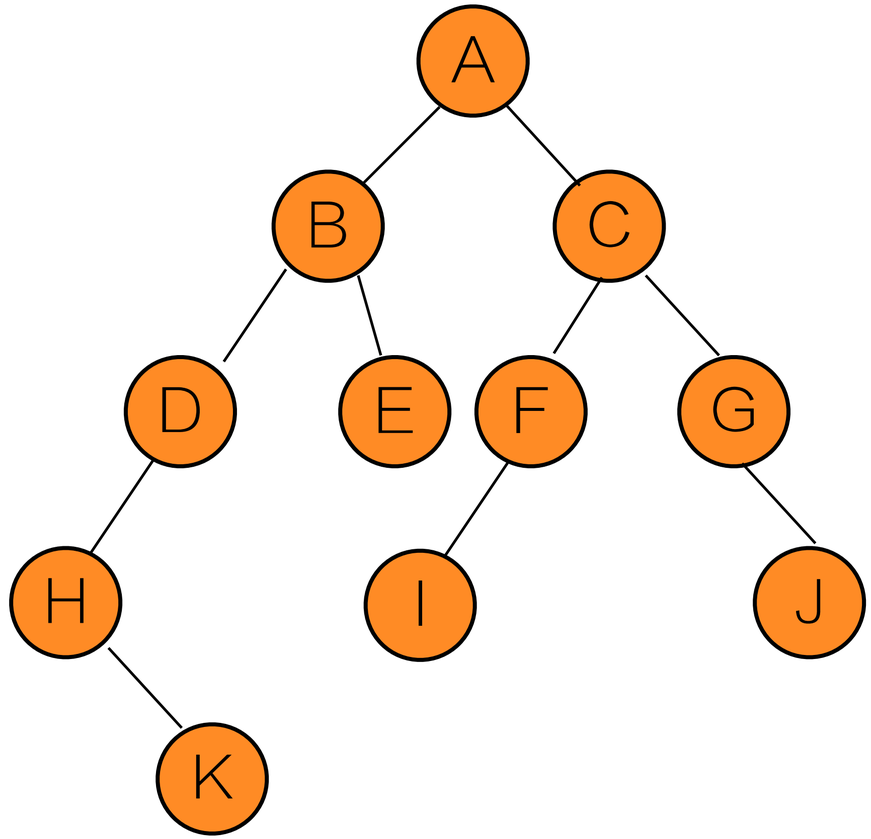
return

in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

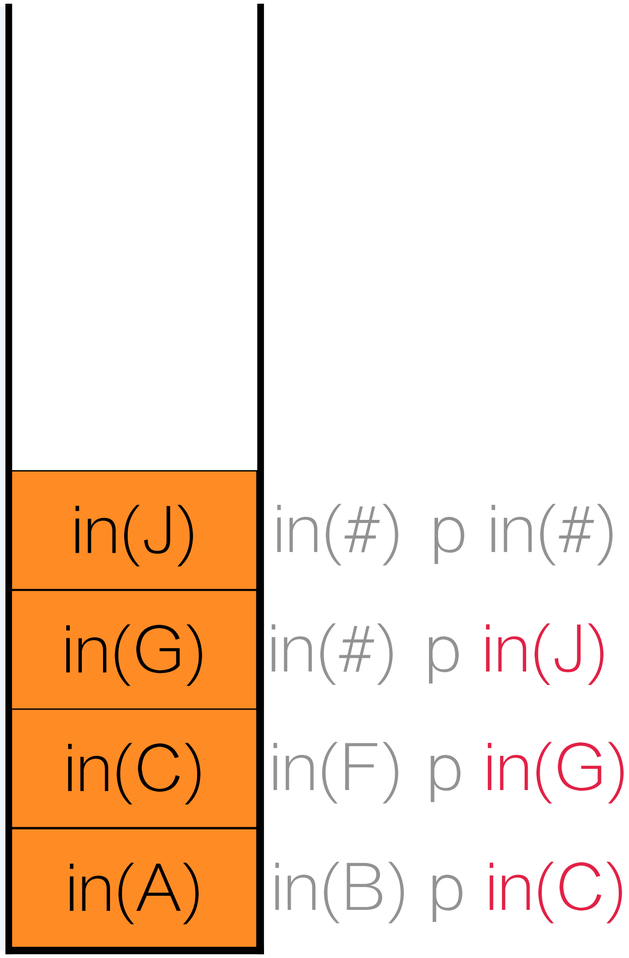
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

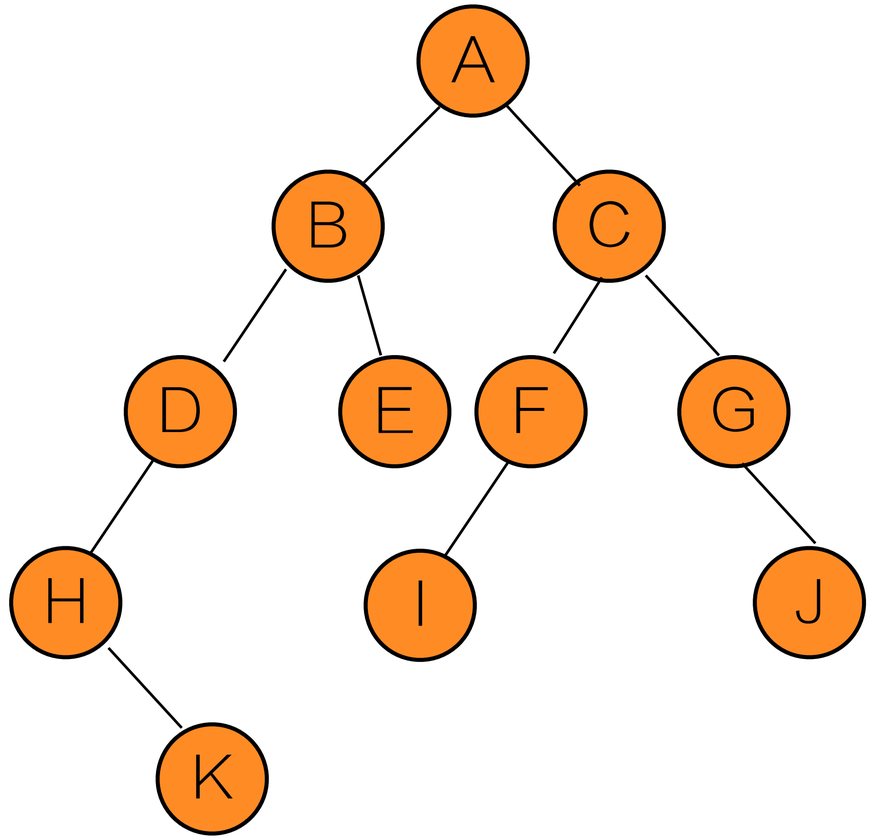


in(#) p in(#)

in(#) p in(J)

in(F) p in(G)

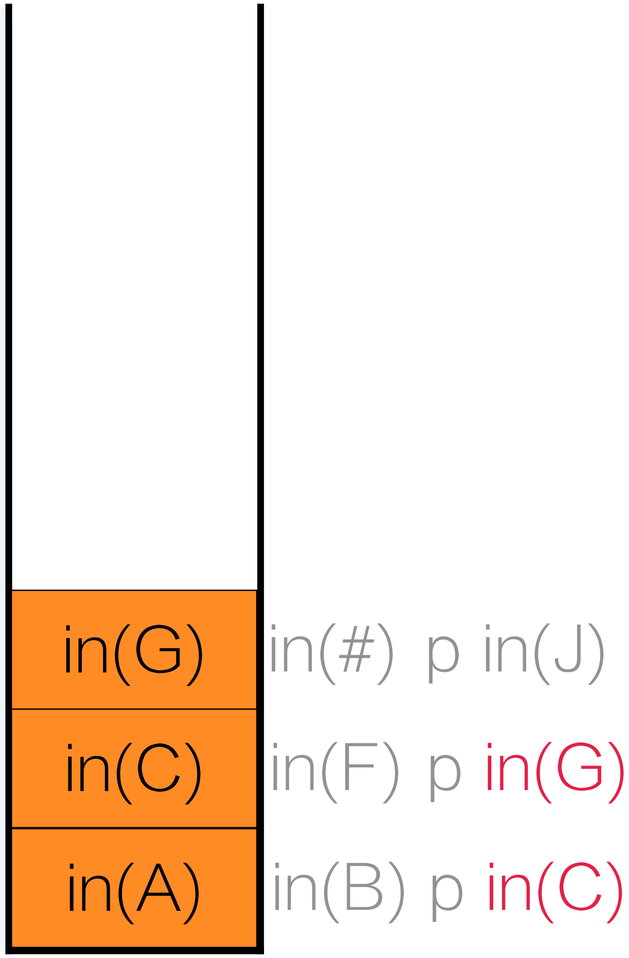
in(B) p in(C)



二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

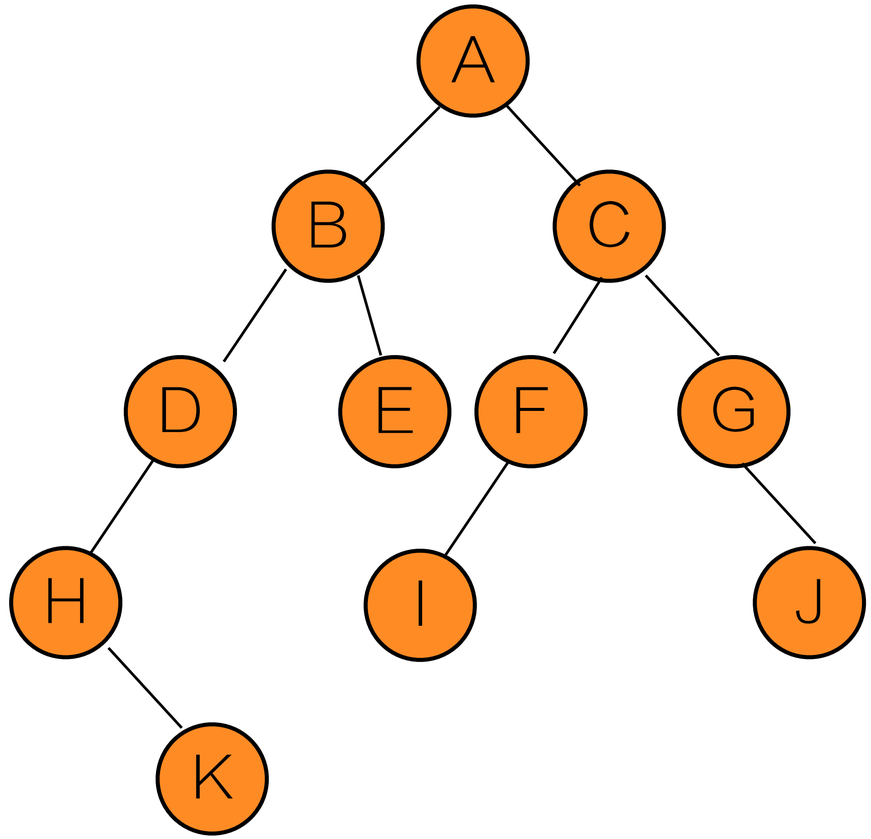
```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```



in(#) p in(J)

in(F) p in(G)

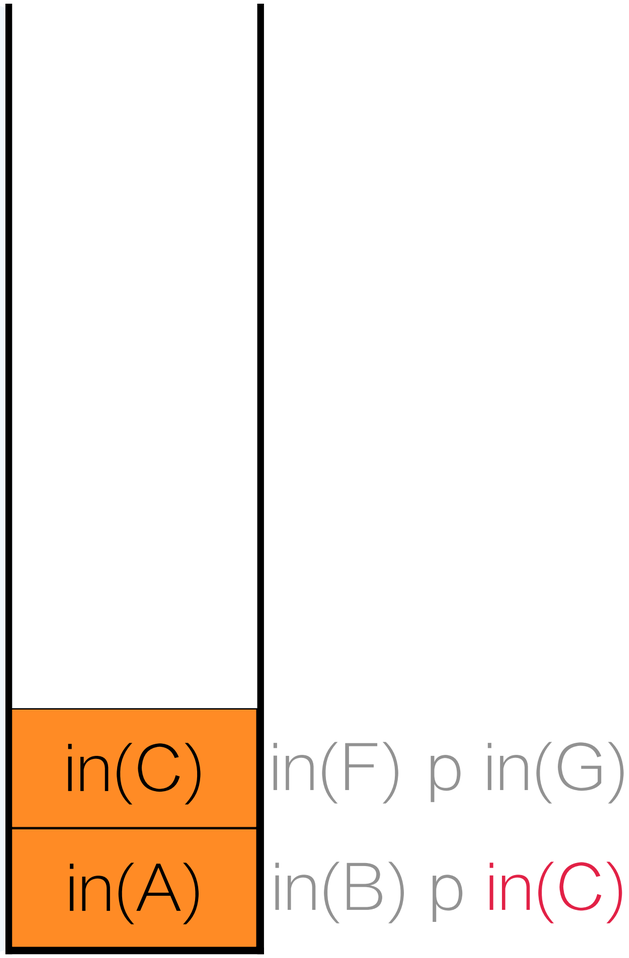
in(B) p in(C)

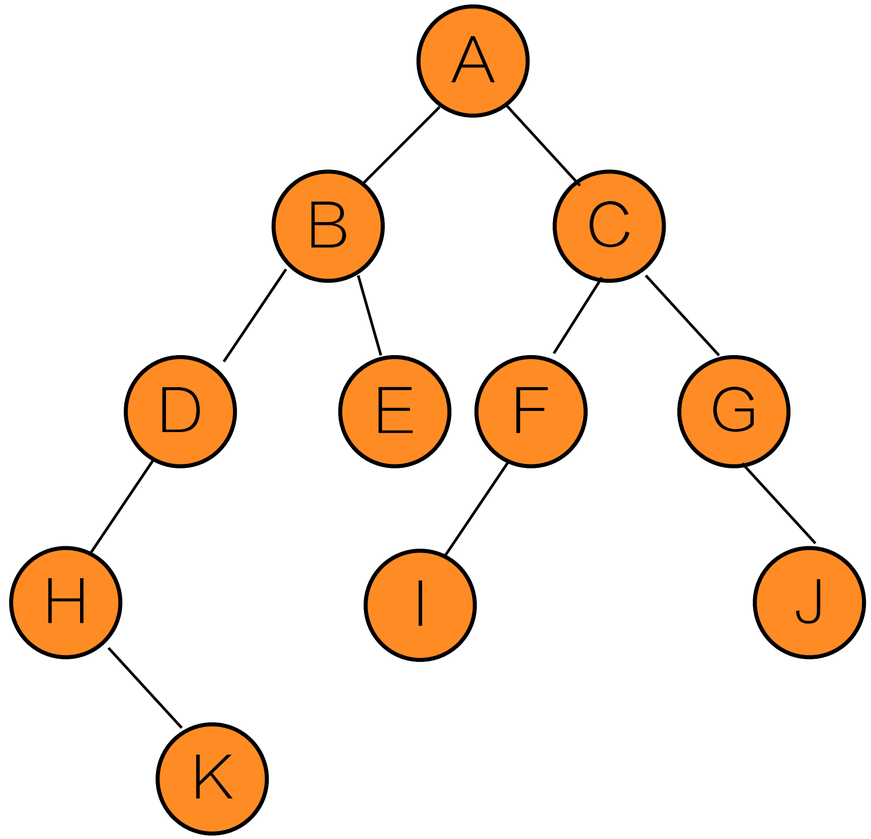


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```





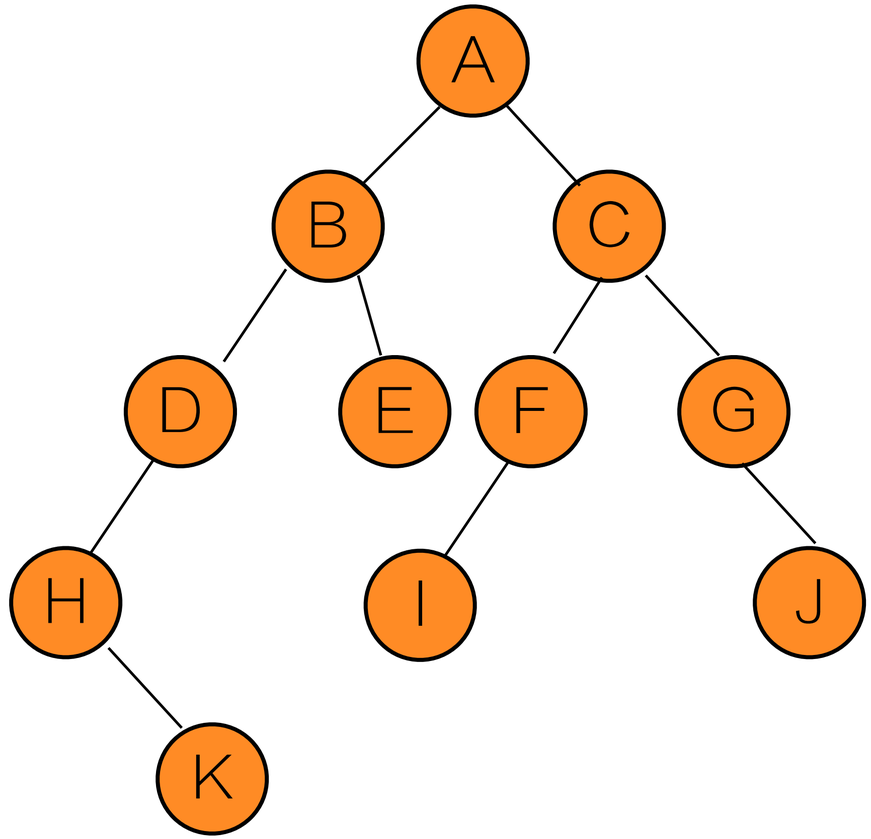
二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

in(A)

in(B) p in(C)

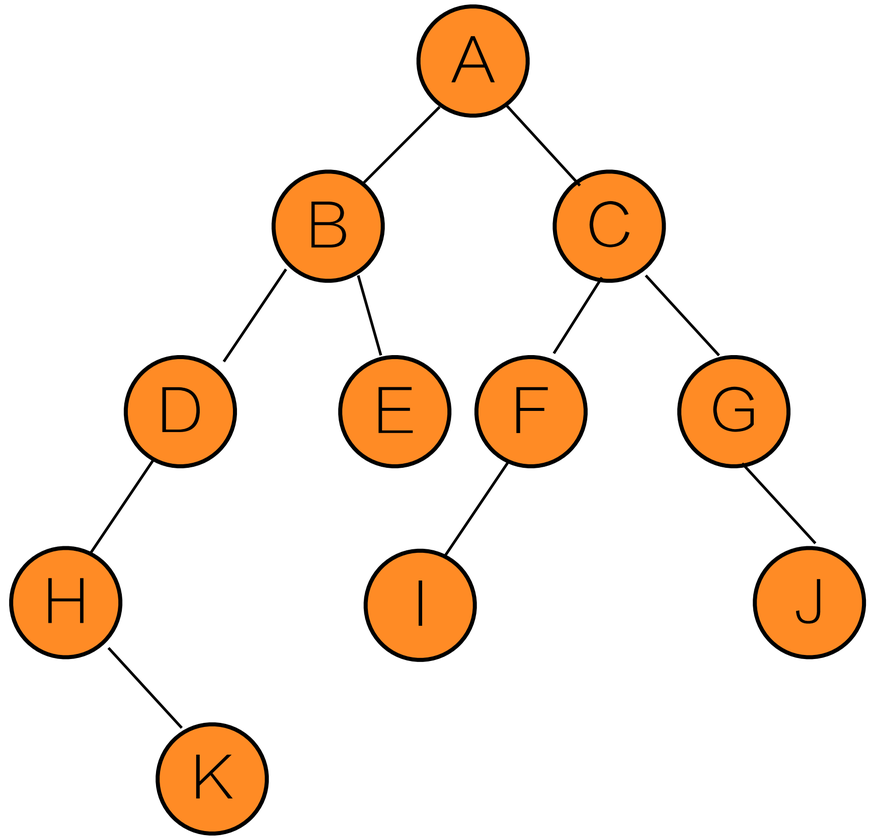


二叉树的遍历 – 中序遍历

输出内容：H K D B E A I F C G J

案例：树 / test1.c

```
void inOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    inOrder(T->lchild);
    printf("%c ", T->data);
    inOrder(T->rchild);
}
```

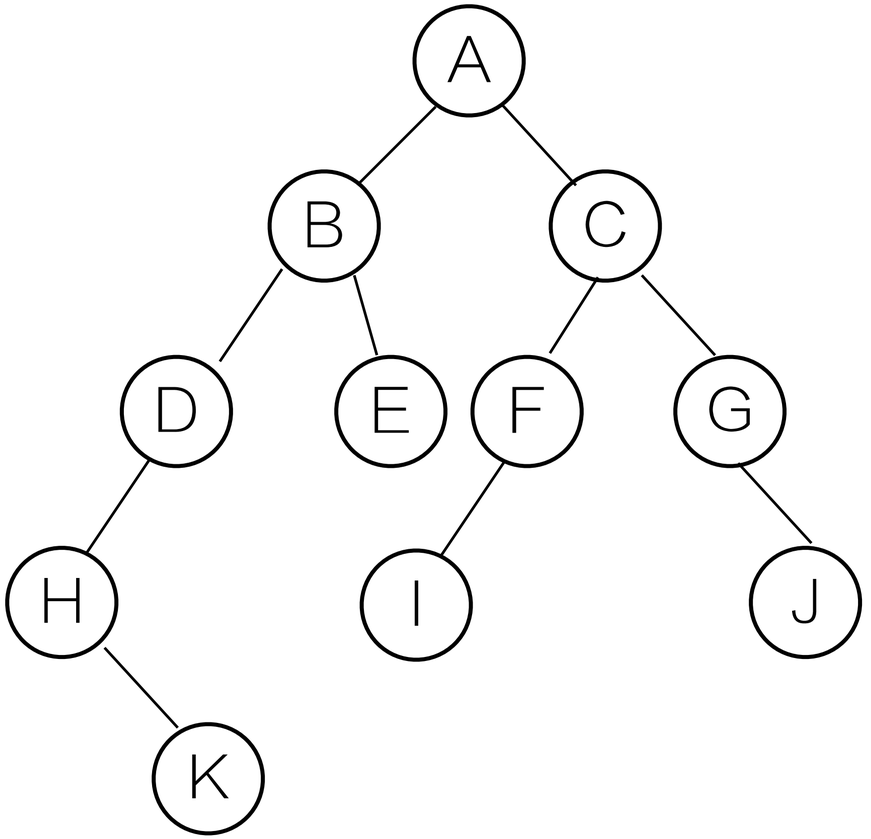


二叉树的遍历 – 后序遍历

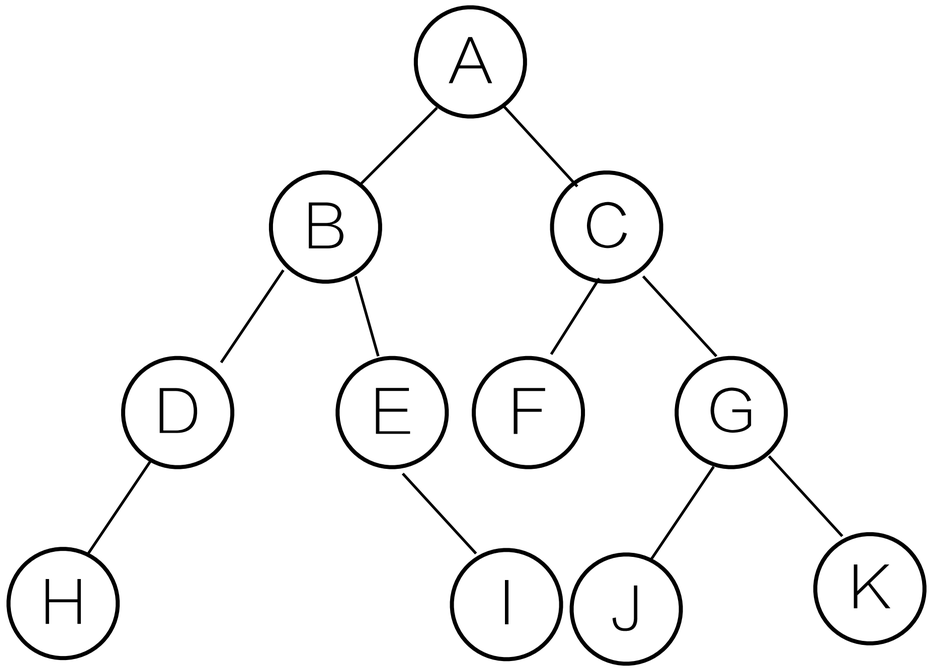
案例：树 / test1.c

从根结点开始先访问结点的左右儿子，再对该结点进行访问。这就意味着结点的儿子将在该结点之前输出。

```
void postOrder(BiTree T)
{
    if (T == NULL)
    {
        return;
    }
    postOrder(T->lchild);
    postOrder(T->rchild);
    printf("%c ", T->data);
}
```



下列二叉树的遍历结果是?

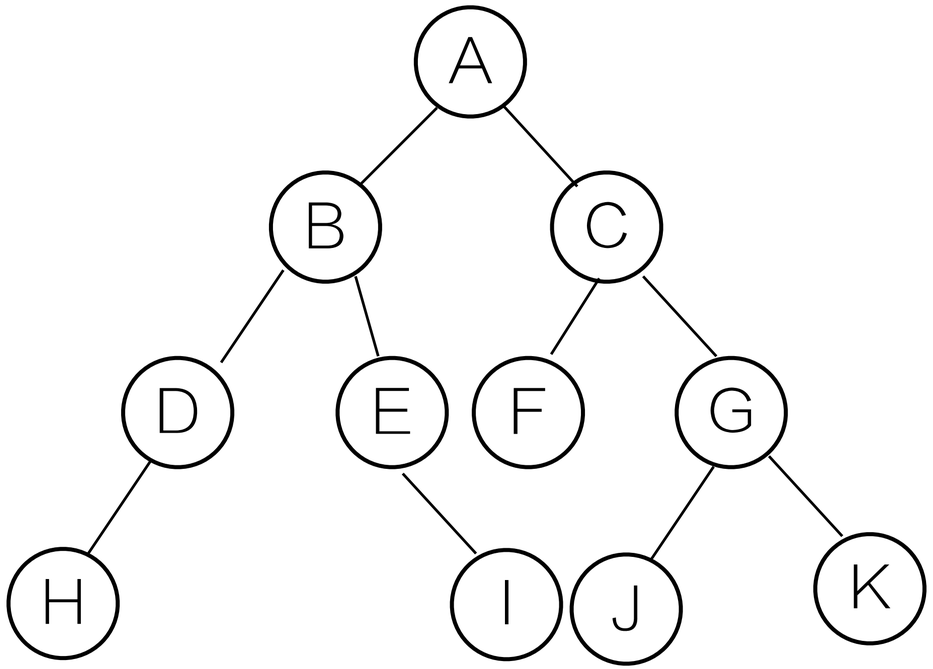


前序:

中序:

后序:

下列二叉树的遍历结果是?

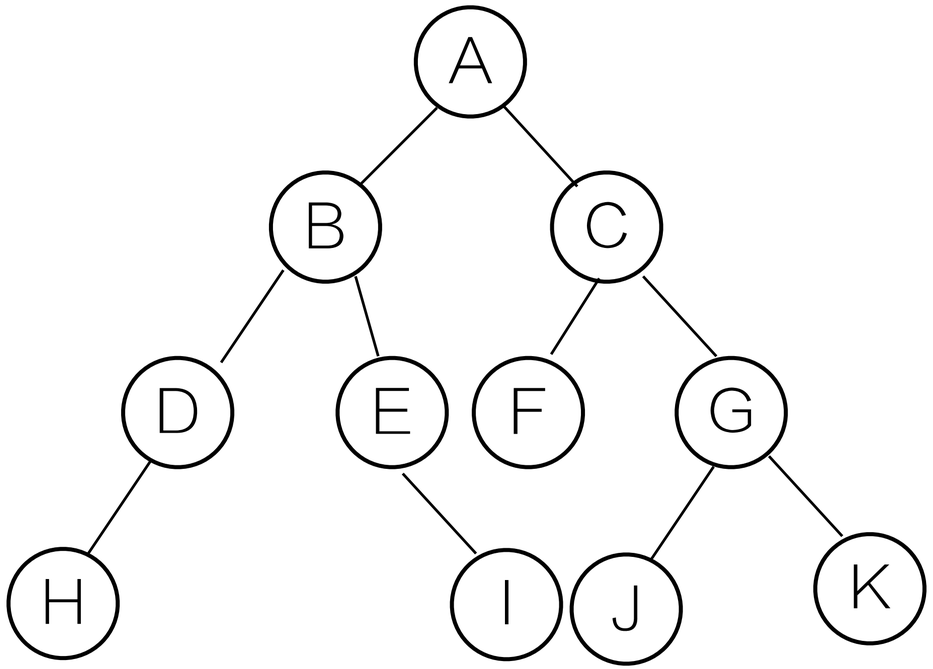


前序：A B D H E I C F G J K

中序：

后序：

下列二叉树的遍历结果是?

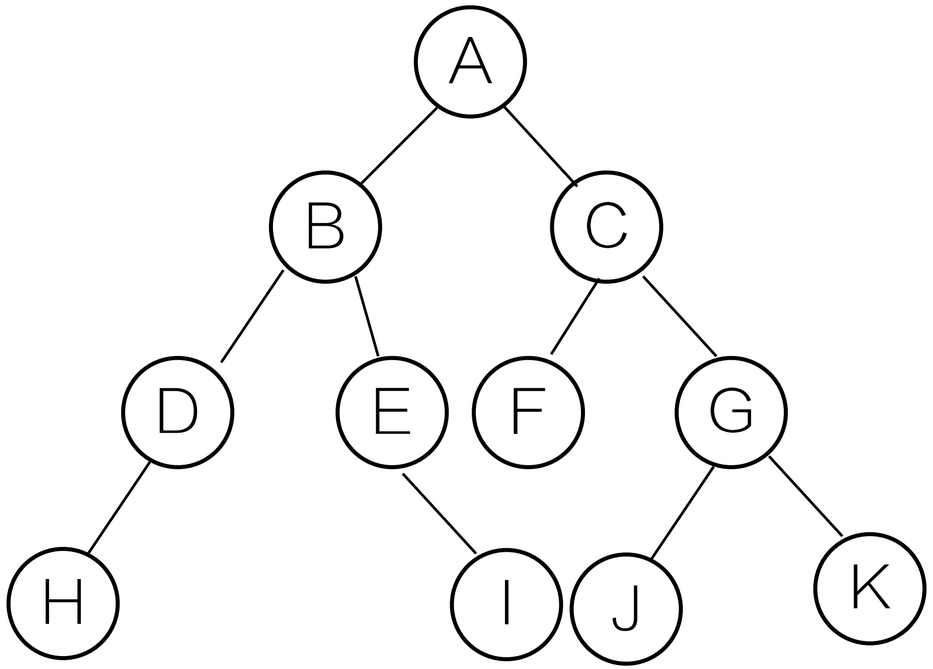


前序：A B D H E I C F G J K

中序：H D B E I A F C J G K

后序：

下列二叉树的遍历结果是?

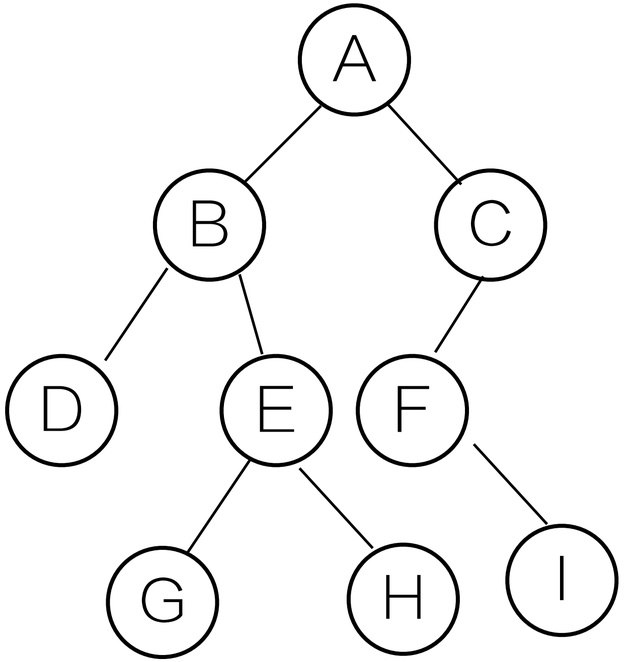


前序：A B D H E I C F G J K

中序：H D B E I A F C J G K

后序：H D I E B F J K G C A

下列二叉树的遍历结果是?

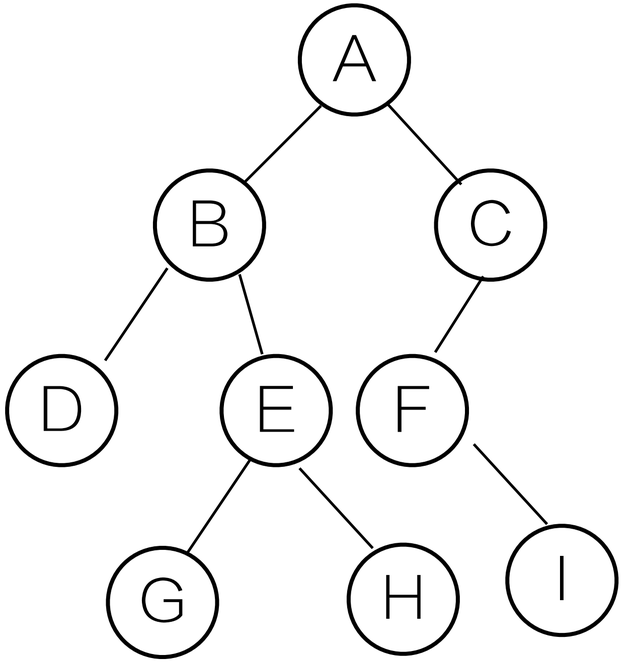


前序:

中序:

后序:

下列二叉树的遍历结果是?

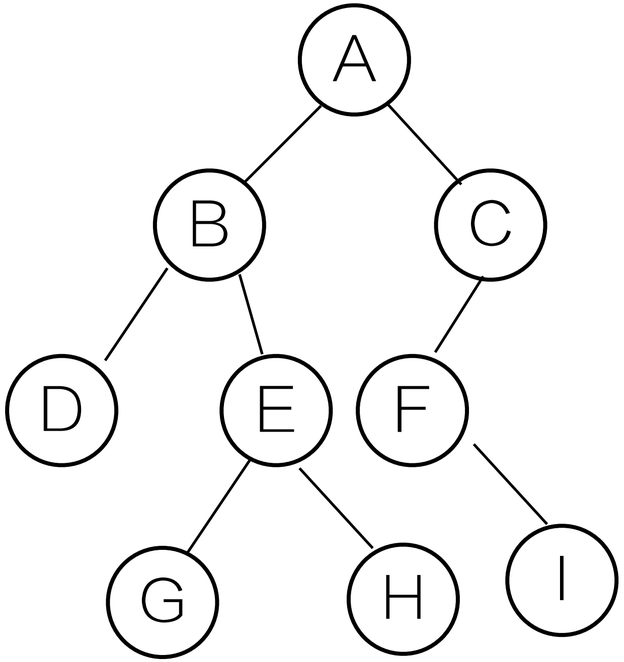


前序：A B D E G H C F I

中序：

后序：

下列二叉树的遍历结果是?

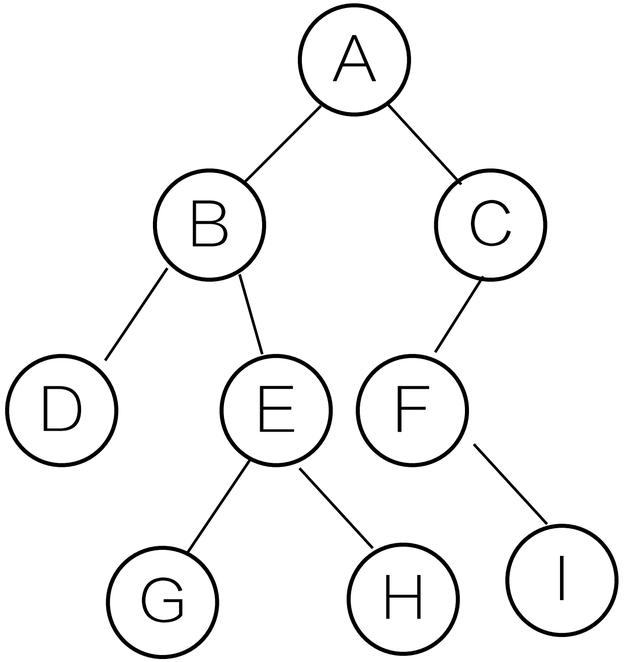


前序：A B D E G H C F I

中序：D B G E H A F I C

后序：

下列二叉树的遍历结果是?



前序：A B D E G H C F I

中序：D B G E H A F I C

后序：D G H E B I F C A

二叉树的遍历 – 非递归前序遍历

既然递归可以用栈来完成，那么我们自己也可以显式地使用“栈”来模拟这个过程，手动完成遍历操作，这就是所谓的非递归遍历。

换句话说：非递归的遍历，就是“自己手动维护一个栈”，来代替系统递归调用时自动压栈的行为。

前序遍历：案例：树 / test2.c

前序遍历：案例：树 / test3.c

前序遍历：案例：树 / test4.c

根据遍历结果推导二叉树

已知一棵二叉树的前序遍历序列为 ABCDEF，中序遍历序列为 CBAEDF

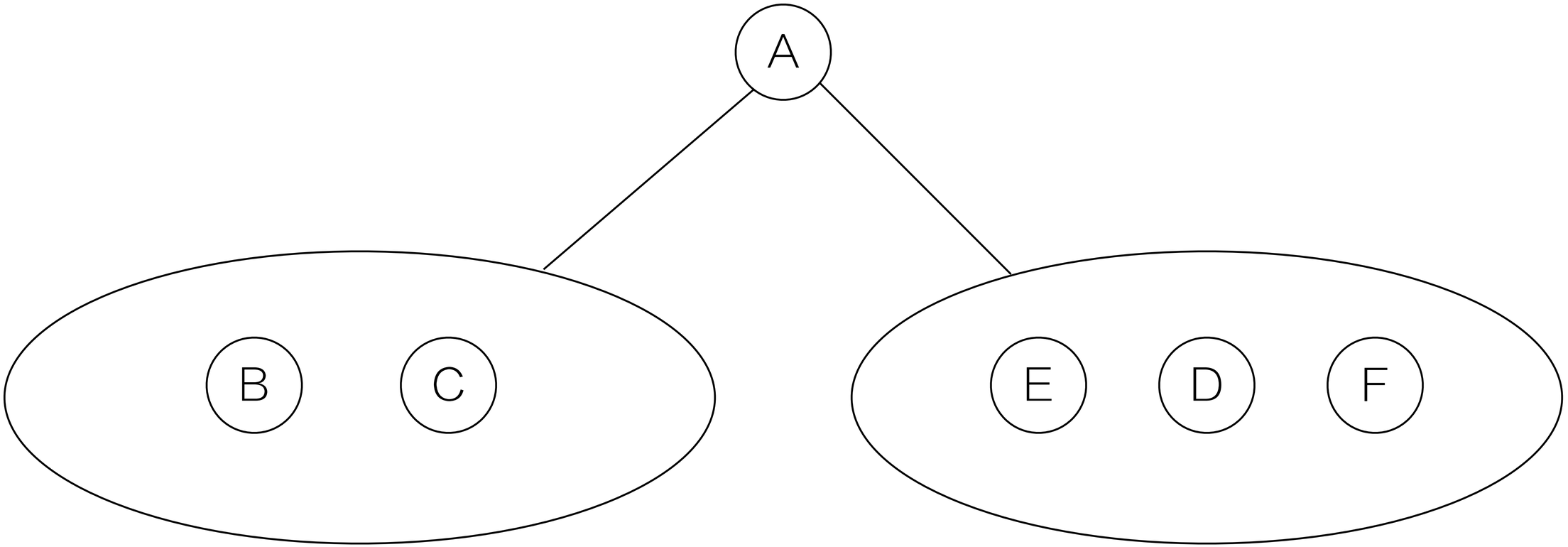
请问这棵二叉树的后序遍历结果是多少?

A

根据遍历结果推导二叉树

已知一棵二叉树的前序遍历序列为 ABCDEF，中序遍历序列为 CBAEDF

请问这棵二叉树的后序遍历结果是多少?

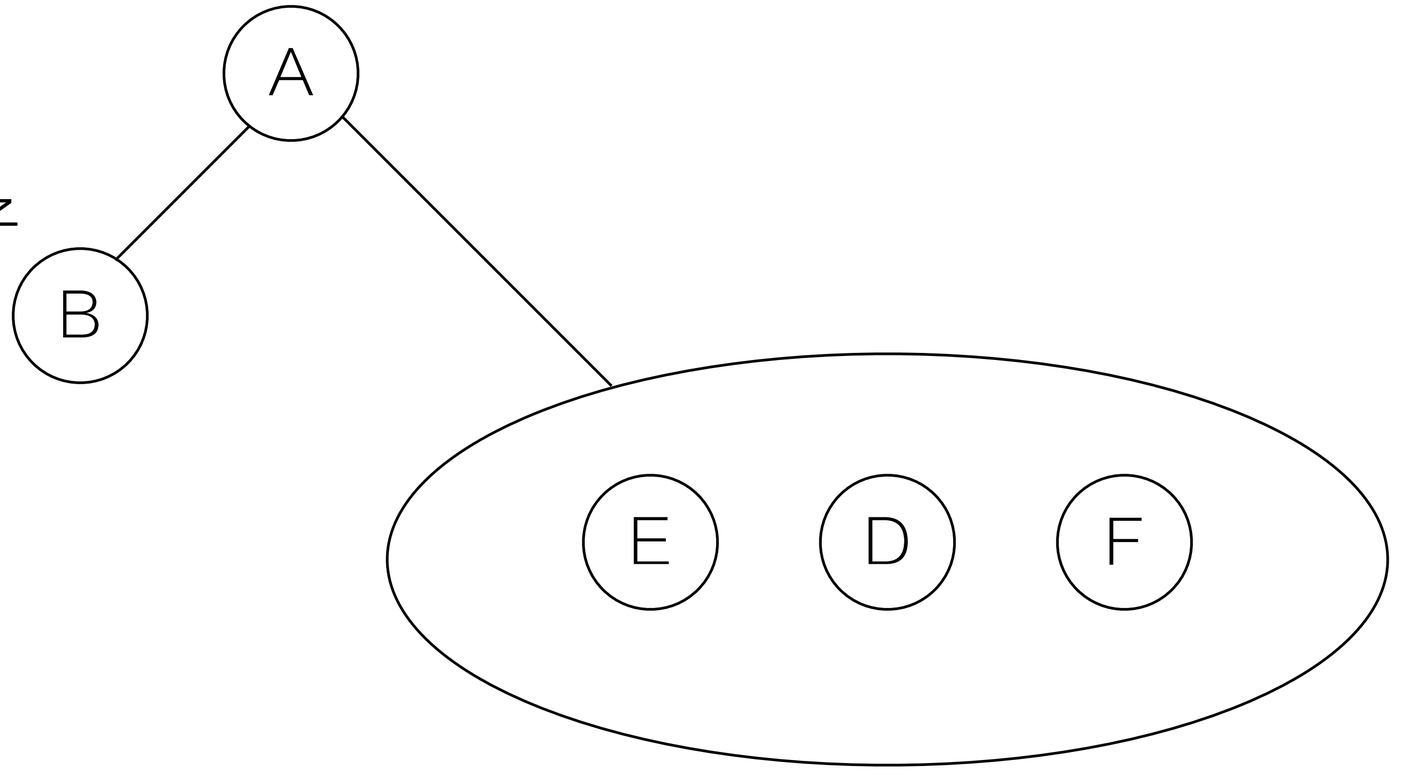


根据遍历结果推导二叉树

已知一棵二叉树的前序遍历序列为 ABCDEF，中序遍历序列为 CBAEDF

请问这棵二叉树的后序遍历结果是多少?

根据前序序列，判断B是A的左孩子

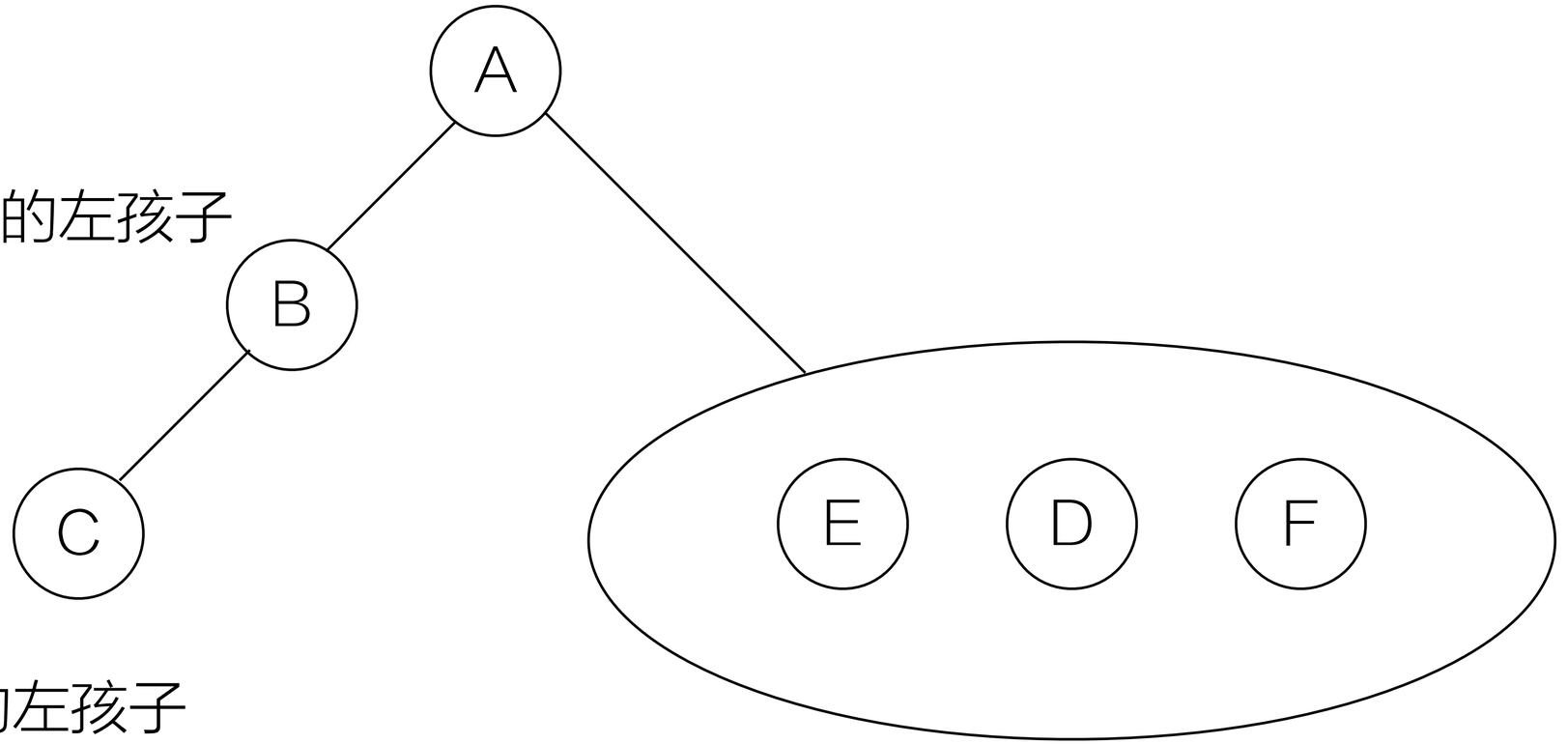


根据遍历结果推导二叉树

已知一棵二叉树的前序遍历序列为 ABCDEF，中序遍历序列为 CBAEDF

请问这棵二叉树的后序遍历结果是多少?

根据前序序列，判断B是A的左孩子

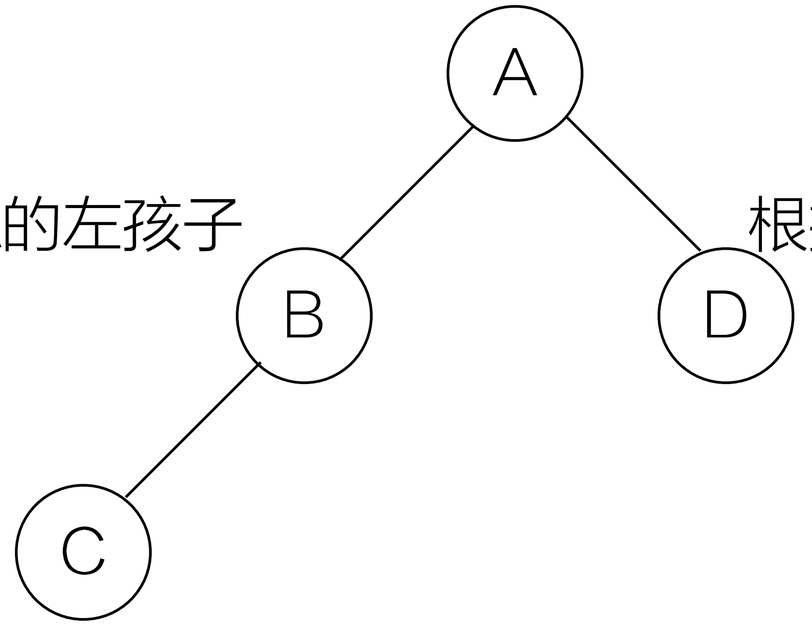


根据中序序列，判断C是B的左孩子

根据遍历结果推导二叉树

已知一棵二叉树的前序遍历序列为 ABCDEF，中序遍历序列为 CBAEDF

请问这棵二叉树的后序遍历结果是多少?



根据前序序列，判断B是A的左孩子

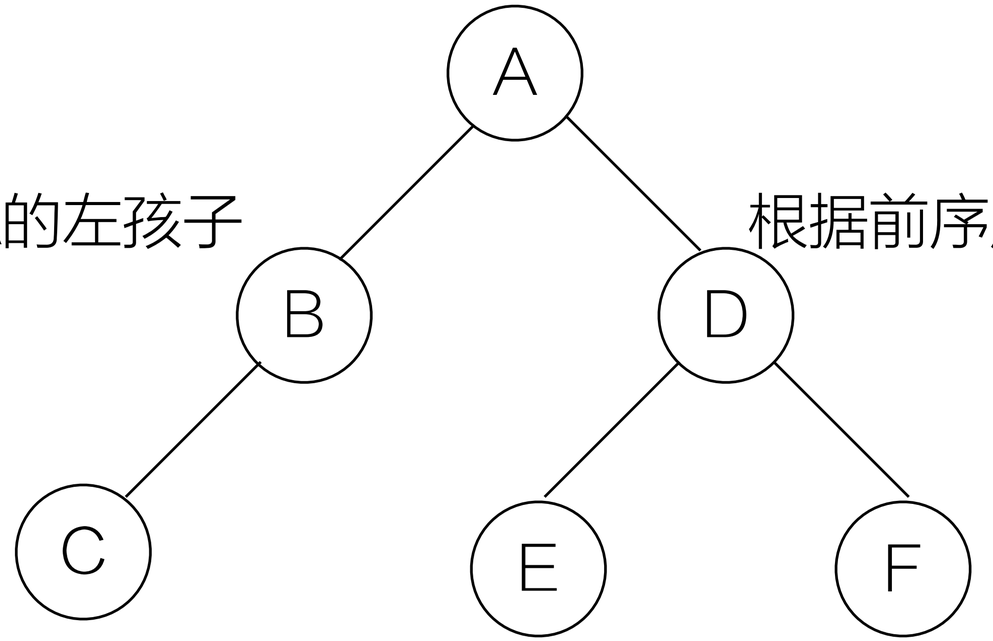
根据前序序列，判断D是A的右孩子

根据中序序列，判断C是B的左孩子

根据遍历结果推导二叉树

已知一棵二叉树的前序遍历序列为 ABCDEF，中序遍历序列为 CBAEDF

请问这棵二叉树的后序遍历结果是多少?



根据前序序列，判断B是A的左孩子

根据前序序列，判断D是A的右孩子

根据中序序列，判断C是B的左孩子

仅根据前序序列，F有可能是E的孩子

根据中序序列，F是D的右孩子

二叉树遍历性质

已知前序遍历和中序遍历，可以唯一确定一棵二叉树。

已知中序遍历和后序遍历，可以唯一确定一棵二叉树。

二叉树遍历性质

已知前序遍历和中序遍历，可以唯一确定一棵二叉树。

已知中序遍历和后序遍历，可以唯一确定一棵二叉树。

已知前序遍历和后序遍历，是不能确定一棵二叉树的。

二叉树遍历性质

已知前序遍历和中序遍历，可以唯一确定一棵二叉树。

已知中序遍历和后序遍历，可以唯一确定一棵二叉树。

已知前序遍历和后序遍历，是不能确定一棵二叉树的。

例：前序序列是ABC，后序序列是CBA

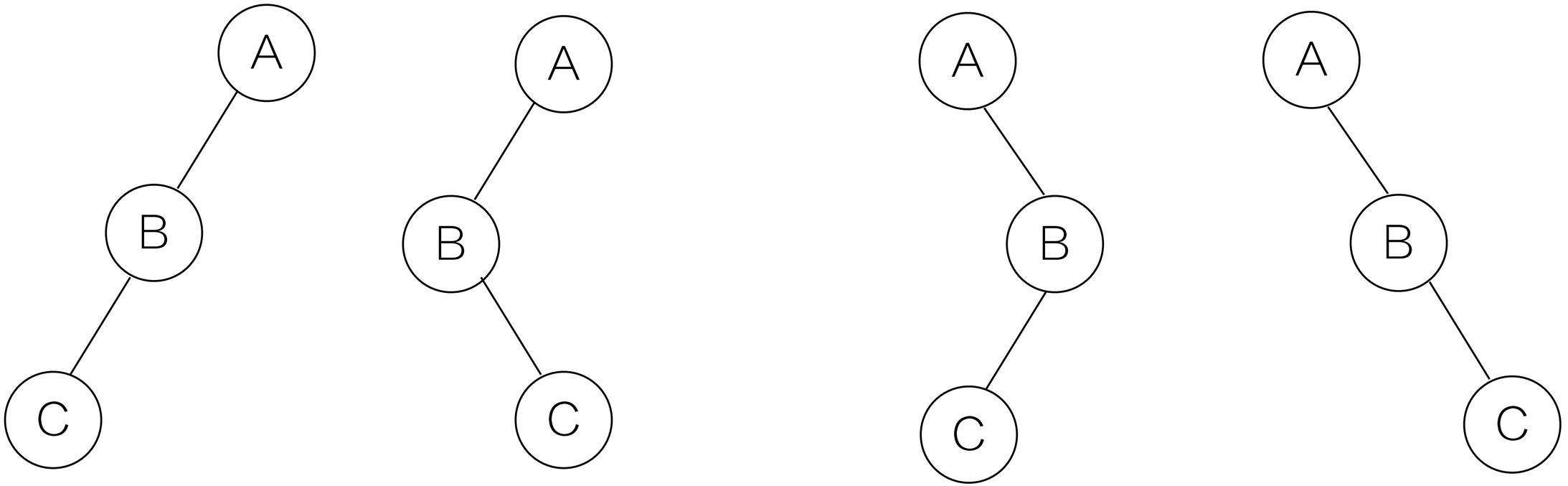
二叉树遍历性质

已知前序遍历和中序遍历，可以唯一确定一棵二叉树。

已知中序遍历和后序遍历，可以唯一确定一棵二叉树。

已知前序遍历和后序遍历，是不能确定一棵二叉树的。

例：前序序列是ABC，后序序列是CBA

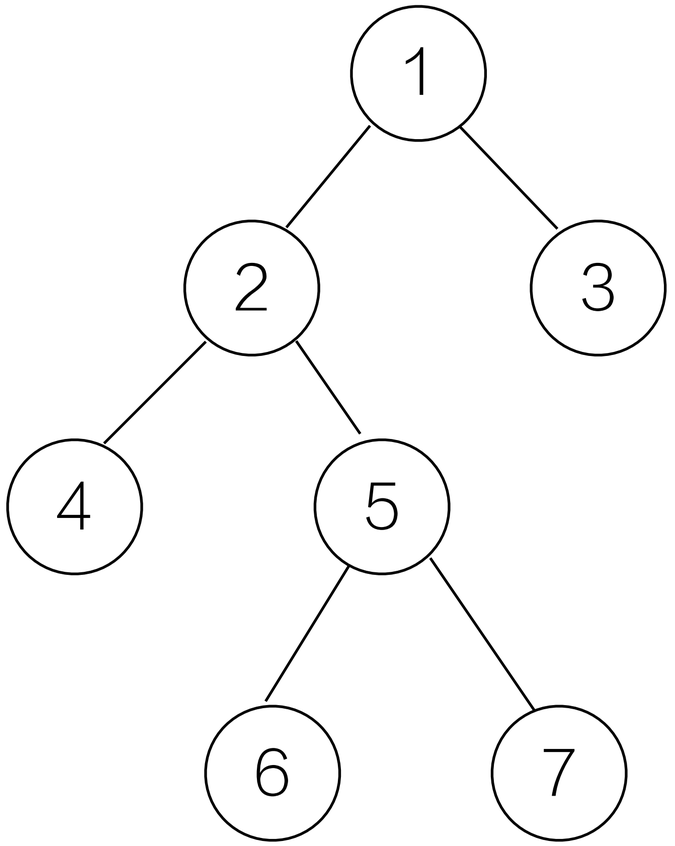


二叉树 – 真题解析

[2009] 3. 给定二叉树如右图所示。设N代表二叉树的根，L代表根结点的左子树，R代表根结点的右子树。若遍历后的结点序列是3，1，7，5，6，2，4，则其遍历方式是_____。

A. LRN　　B. NRL

C. RLN　　D. RNL



二叉树 – 真题解析

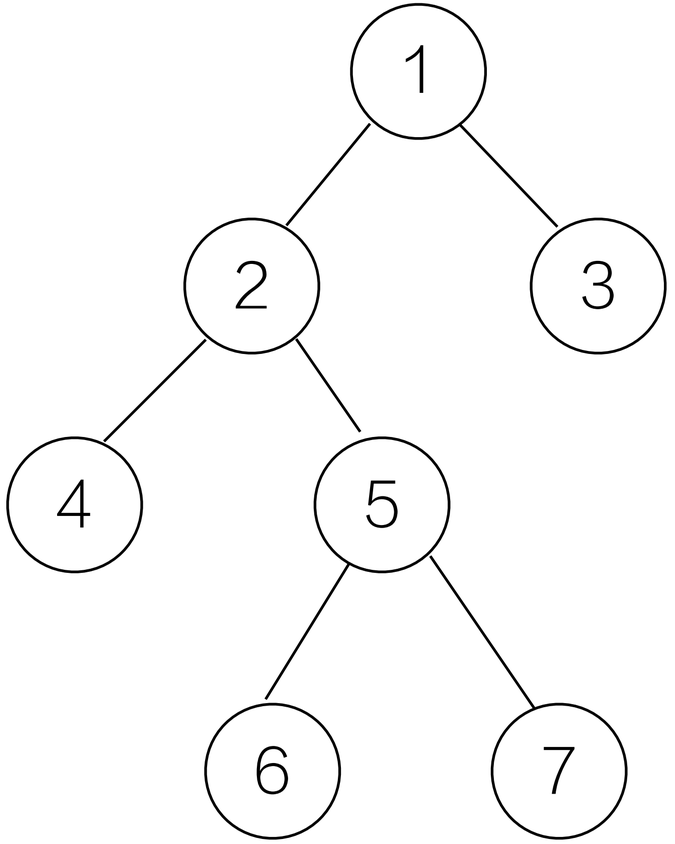
[2009] 3. 给定夺叉树如右图所示。设N代表二叉树的根，L代表根结点的左子树，R代表根结点的右子树。若遍历后的结点序列是3，1，7，5，6，2，4，则其遍历方式是_____。

A. LRN

B. NRL

C. RLN

D. RNL



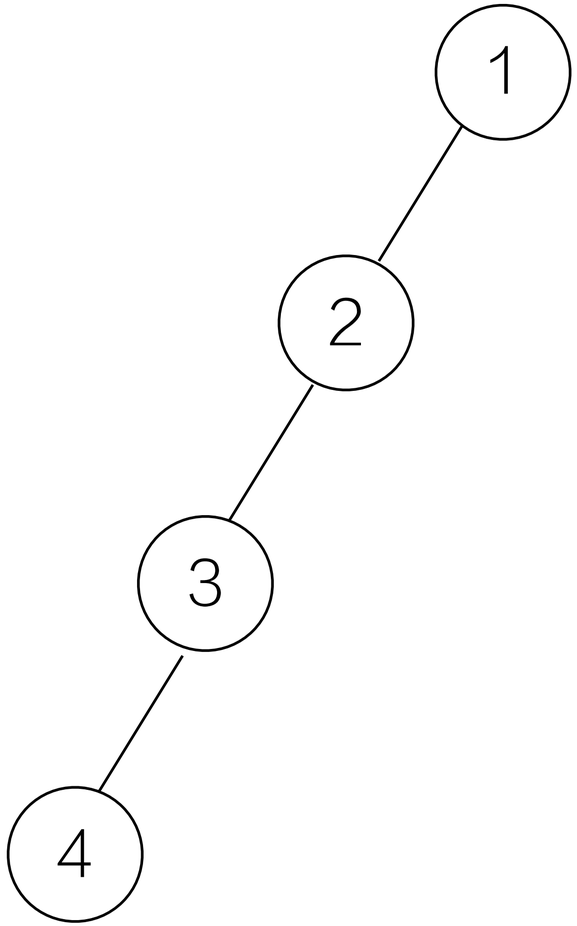
二叉树 – 真题解析

[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

A. 1，2，3，4　　B. 2，3，4，1　　C. 3，2，4，1　　D. 4，3，2，1

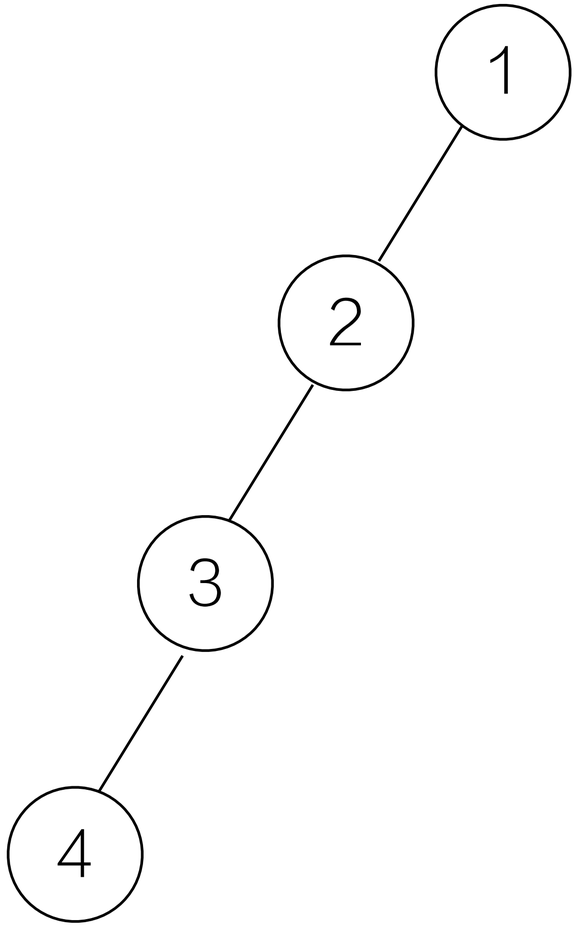
[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

A. 1，2，3，4　　B. 2，3，4，1　　C. 3，2，4，1　　D. 4，3，2，1



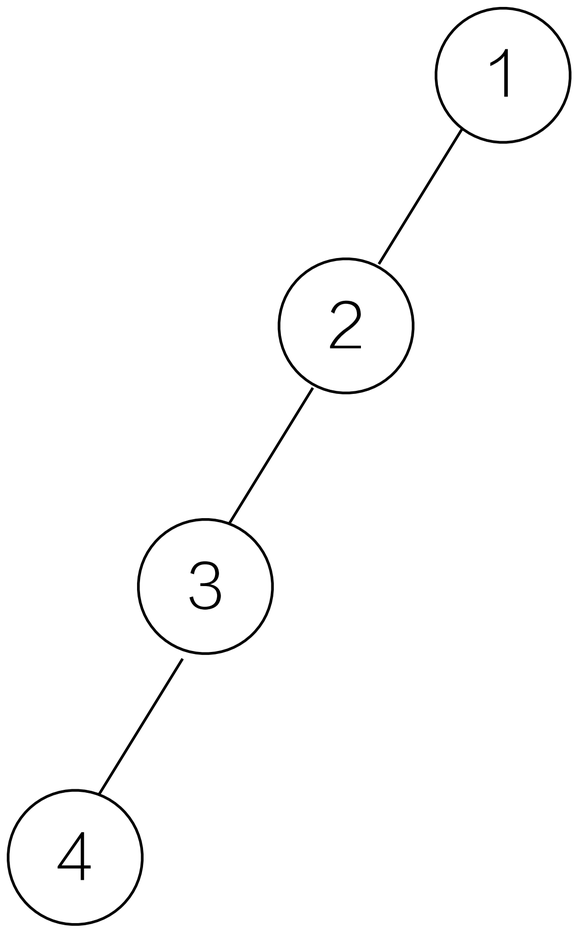
[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

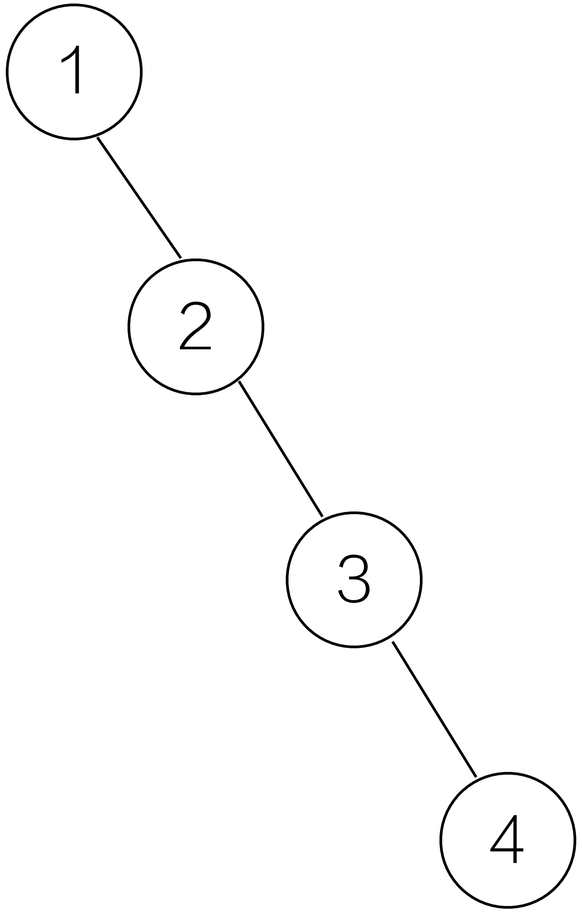
A. 1，2，3，4　　B. 2，3，4，1　　C. 3，2，4，1　　D. 4，3，2，1



[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

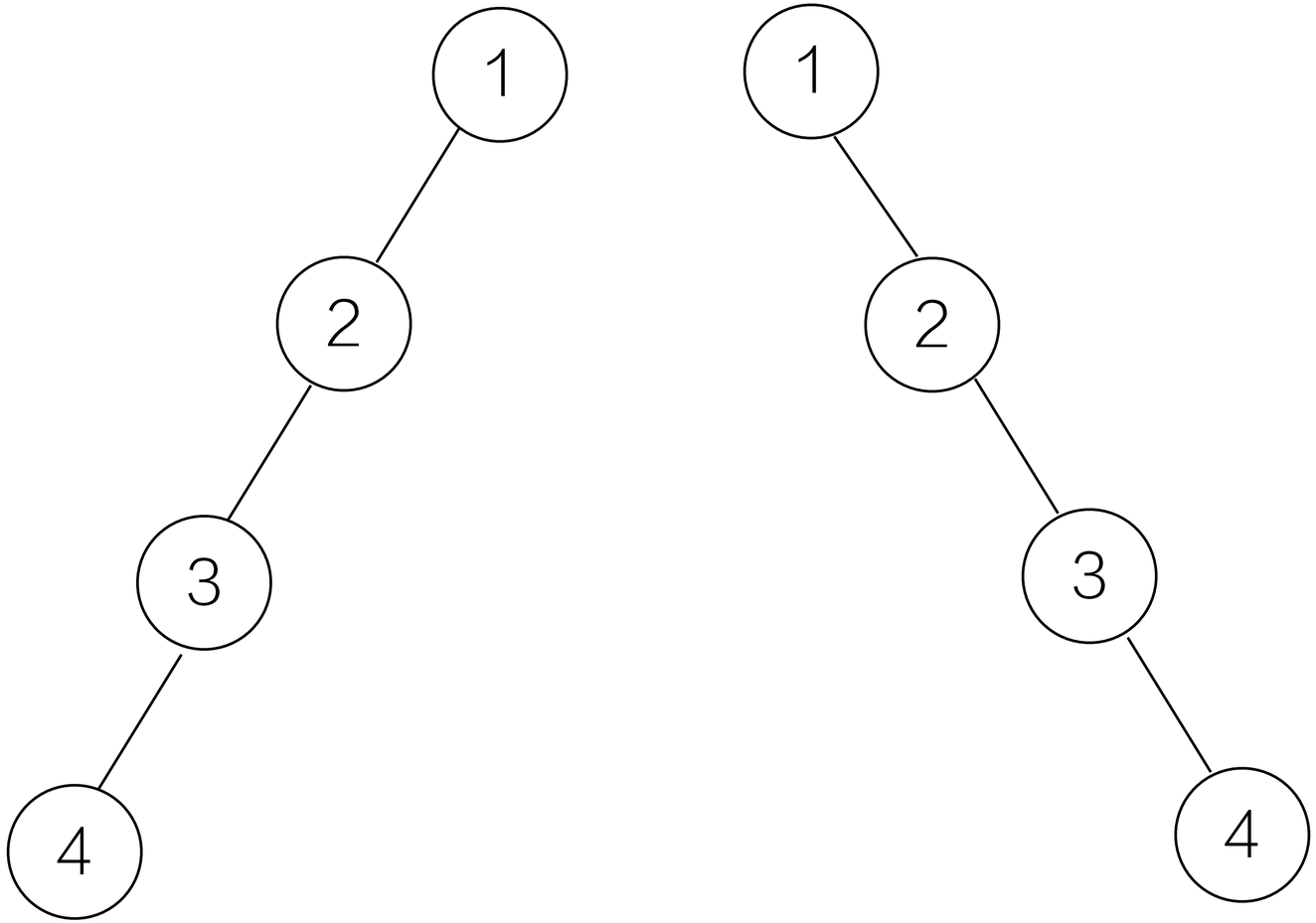
A. 1，2，3，4　　B. 2，3，4，1　　C. 3，2，4，1　　D. 4，3，2，1





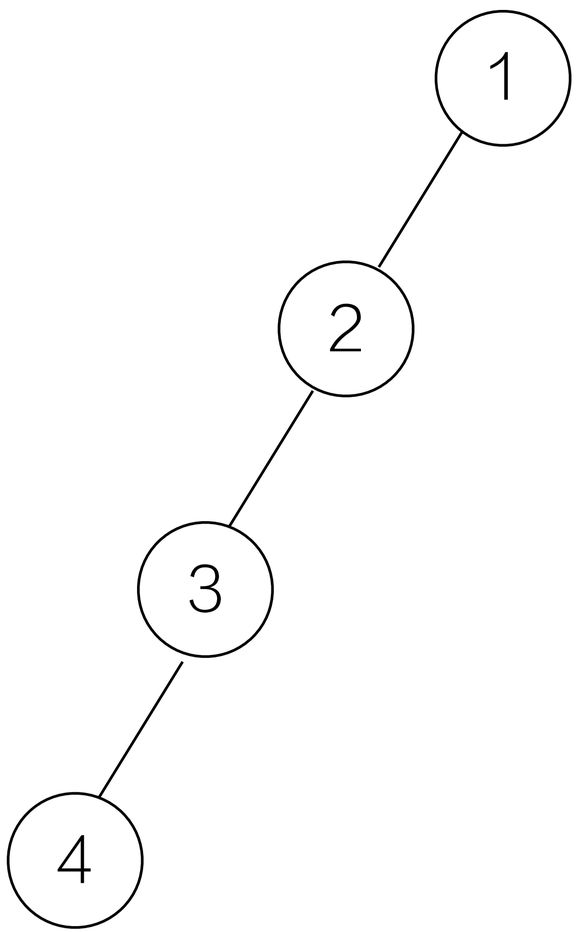
[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

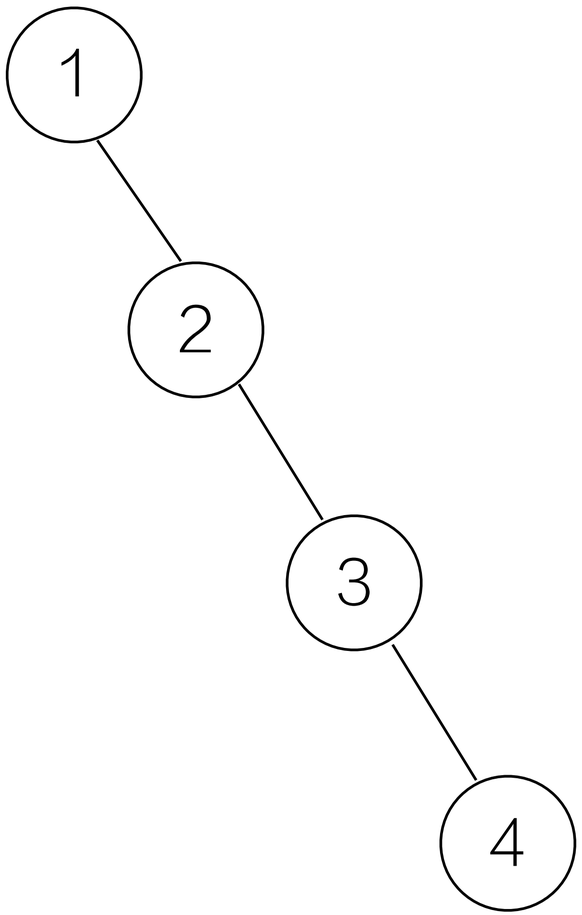
A. 1，2，3，4 ✓　　B. 2，3，4，1　　C. 3，2，4，1　　D. 4，3，2，1 ✓

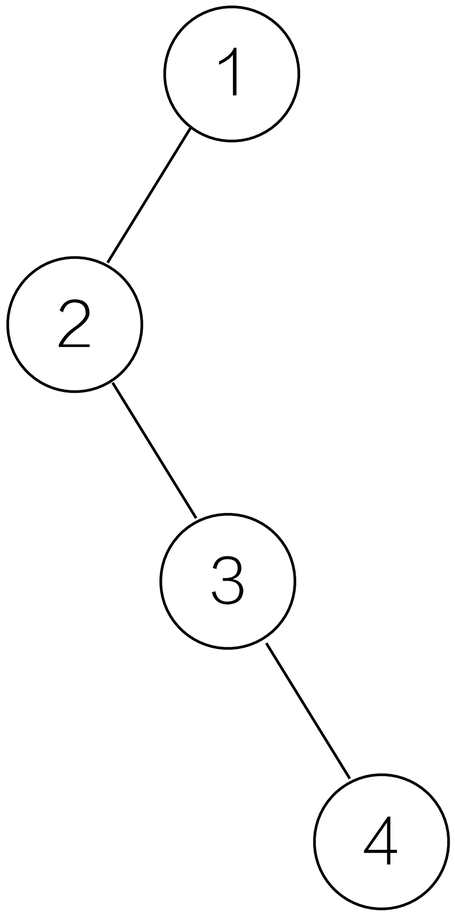


[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

A. 1，2，3，4 ✓　B. 2，3，4，1　C. 3，2，4，1　D. 4，3，2，1 ✓

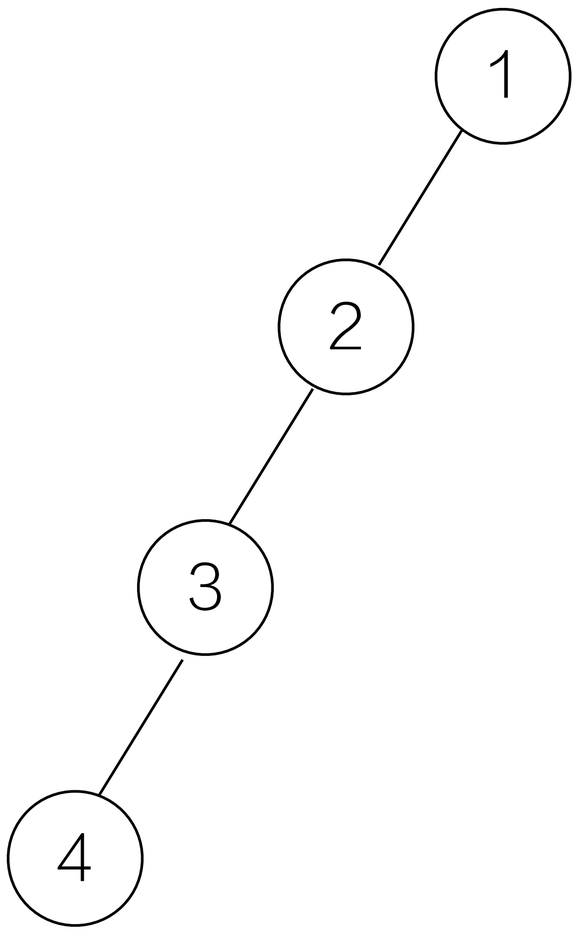


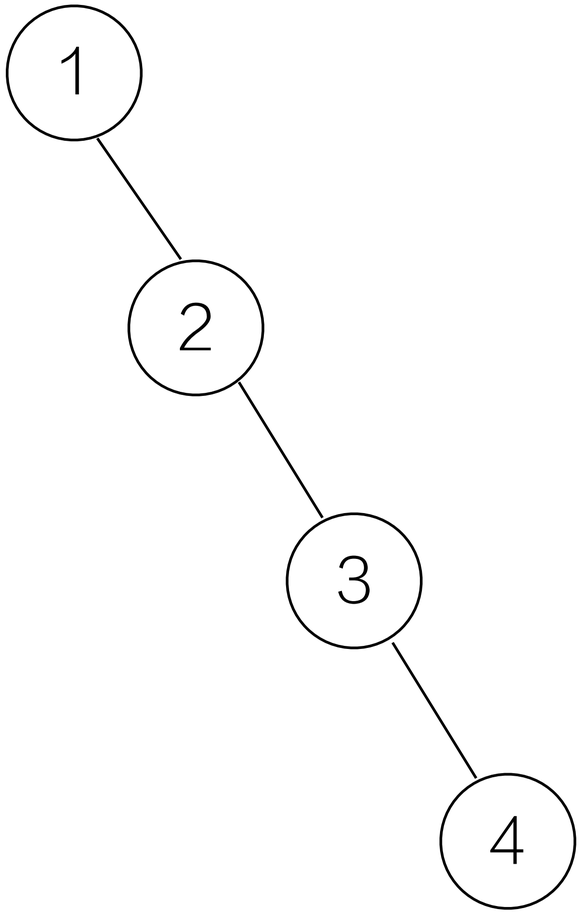


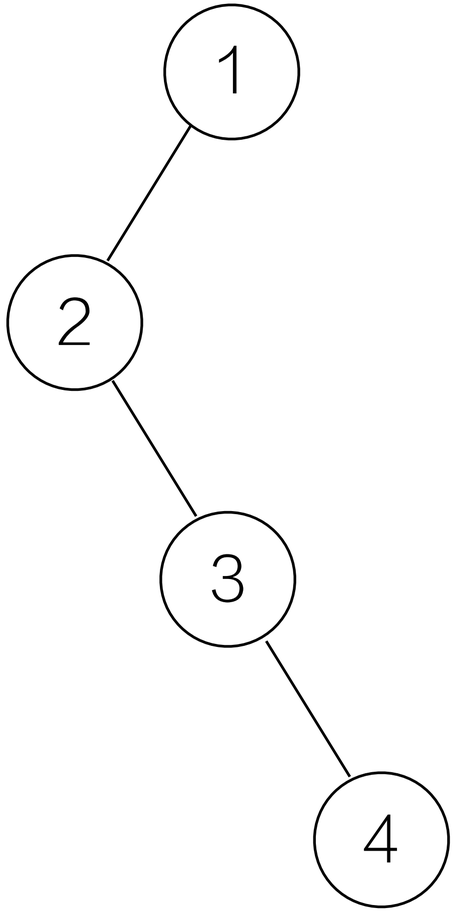


[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

A. 1，2，3，4 ✓　B. 2，3，4，1 ✓　C. 3，2，4，1　D. 4，3，2，1 ✓







二叉树 – 真题解析

[2011] 5.若一棵二叉树的前序遍历序列和后序遍历序列分别为1, 2, 3, 4和4, 3, 2, 1，则该二叉树的中序遍历序列不会是_____。

A. 1，2，3，4　　B. 2，3，4，1　　C. 3，2，4，1　　D. 4，3，2，1

二叉树 – 真题解析

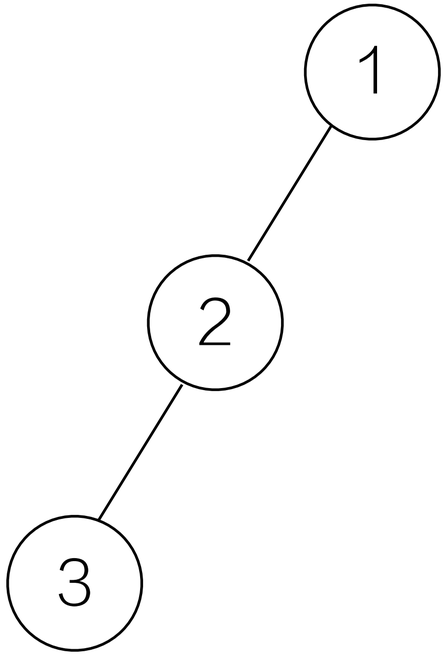
[2017] 4. 要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的是_____。

A. 只有左子树　　B. 只有右子树　　C. 结点的度均为1　　D. 结点的度均为2

二叉树 – 真题解析

[2017] 4. 要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的是_____。

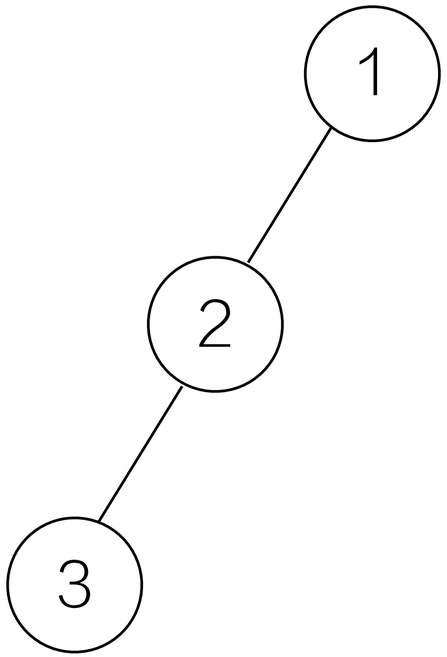
A. 只有左子树　　B. 只有右子树　　C. 结点的度均为1　　D. 结点的度均为2

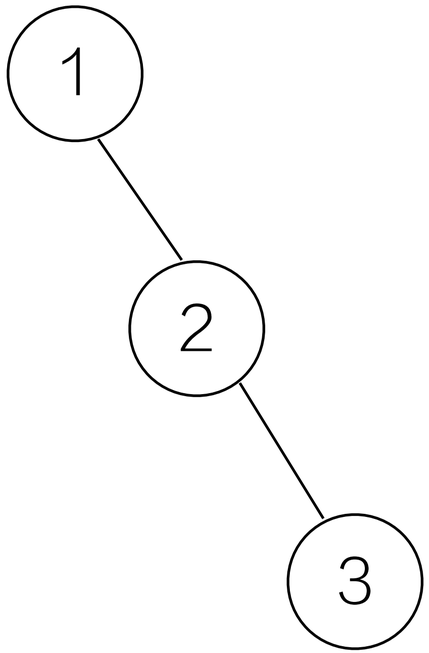


二叉树 – 真题解析

[2017] 4. 要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的是_____。

A. 只有左子树　　B. 只有右子树　　C. 结点的度均为1　　D. 结点的度均为2





二叉树 – 真题解析

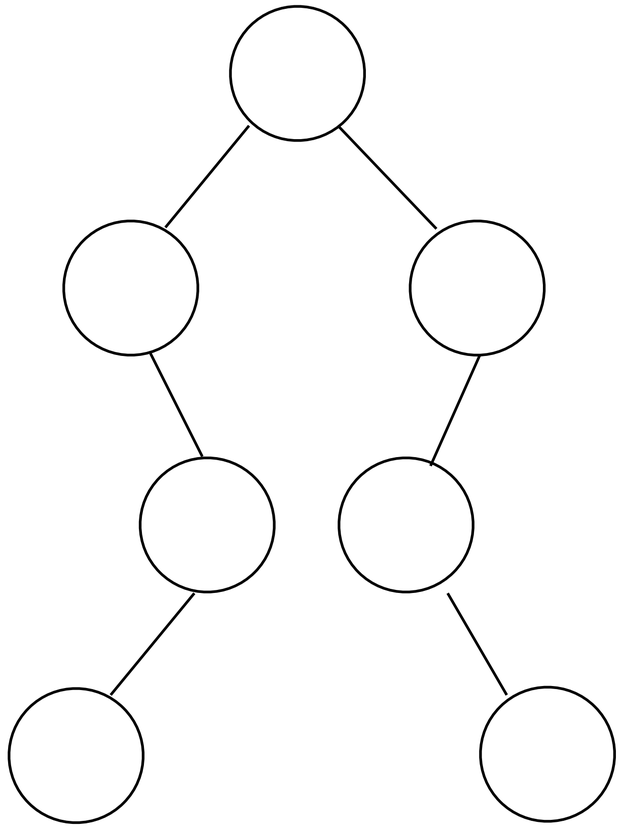
[2017] 4. 要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须满足的是_____。

A. 只有左子树　　B. 只有右子树　　C. 结点的度均为1　　D. 结点的度均为2

二叉树 – 真题解析

[2017] 5. 已知一棵二叉树的树形如右图所示，其后序序列为e, a, c, b, d, g, f，树中与结点a同层的结点是_____。

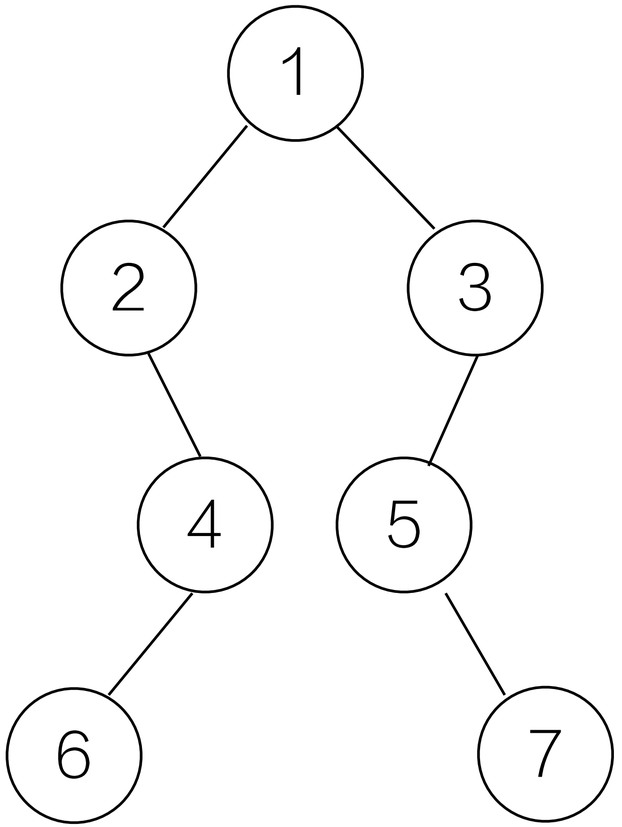
A. c　　B. d　　C. f　　D. g



二叉树 – 真题解析

[2017] 5. 已知一棵二叉树的树形如右图所示，其后序序列为e, a, c, b, d, g, f，树中与结点a同层的结点是_____。

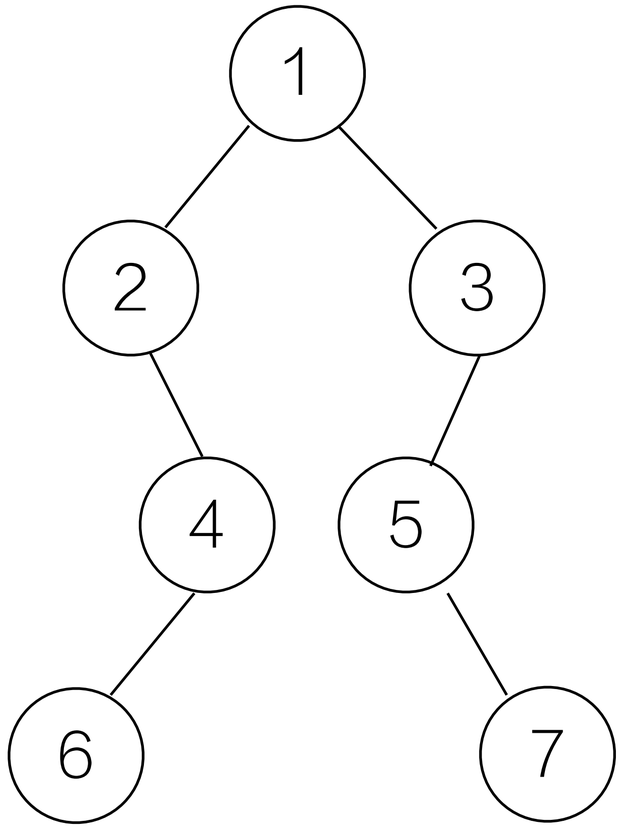
A. c　　B. d　　C. f　　D. g



二叉树 – 真题解析

[2017] 5. 已知一棵二叉树的树形如右图所示，其后序序列为e, a, c, b, d, g, f，树中与结点a同层的结点是_____。

A. c　　B. d　　C. f　　D. g

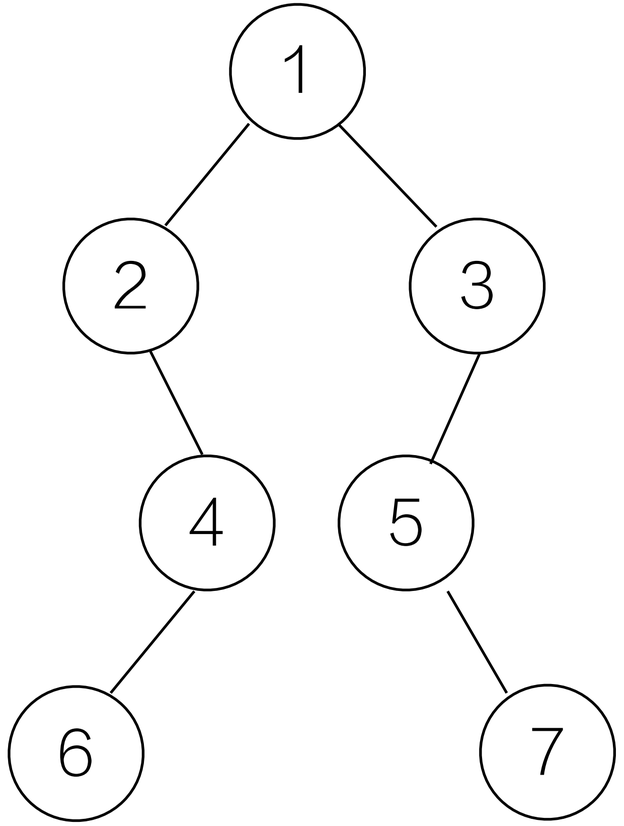


6　4　2　7　5　3　1

二叉树 – 真题解析

[2017] 5. 已知一棵二叉树的树形如右图所示，其后序序列为e, a, c, b, d, g, f，树中与结点a同层的结点是_____。

A. c　　B. d　　C. f　　D. g



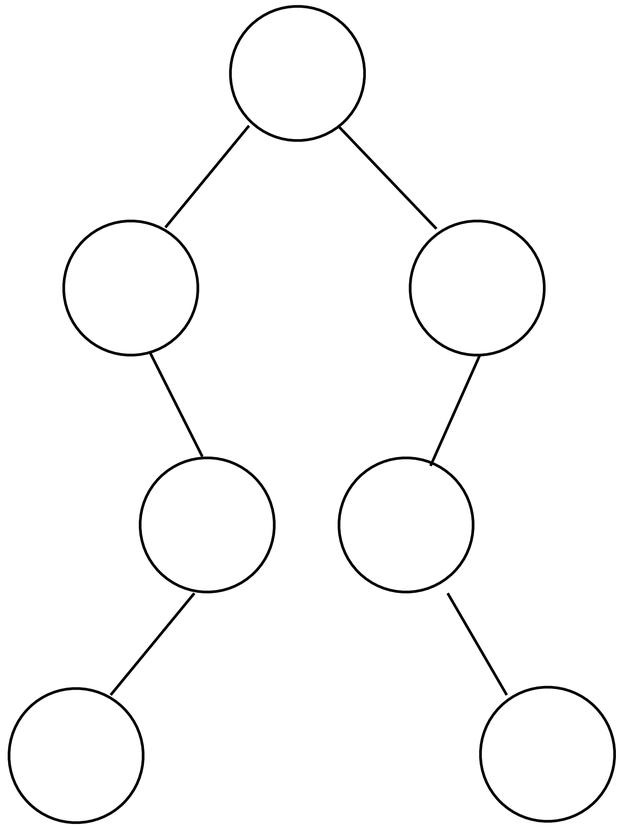
6 4 2 7 5 3 1

e a c b d g f

二叉树 – 真题解析

[2017] 5. 已知一棵二叉树的树形如右图所示，其后序序列为e, a, c, b, d, g, f，树中与结点a同层的结点是_____。

A. c　　B. d　　C. f　　D. g



二叉树 – 真题解析

[2020] 5. 对于任意一棵高度为5且有10个结点的二叉树，若采用顺序存储结构保存，每个结点占1个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元至少是_____。

A. 31　　B. 16　　C. 15　　D. 10

A	B			C					

二叉树 – 真题解析

[2020] 5. 对于任意一棵高度为5且有10个结点的二叉树，若采用顺序存储结构保存，每个结点占1个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元至少是_____。

A. 31　　B. 16　　C. 15　　D. 10

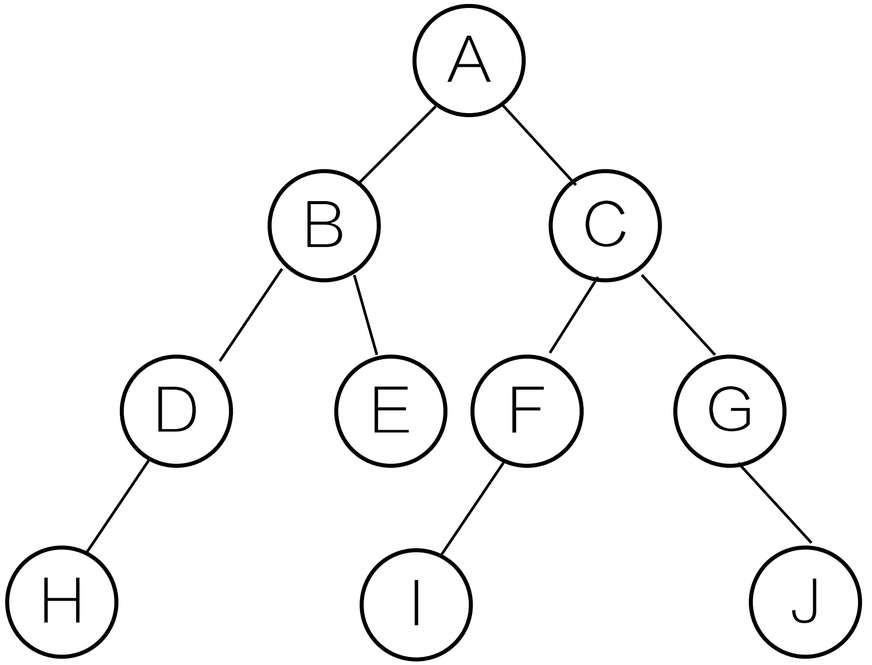
线索二叉树

通过遍历可以得到二叉树的线性排列

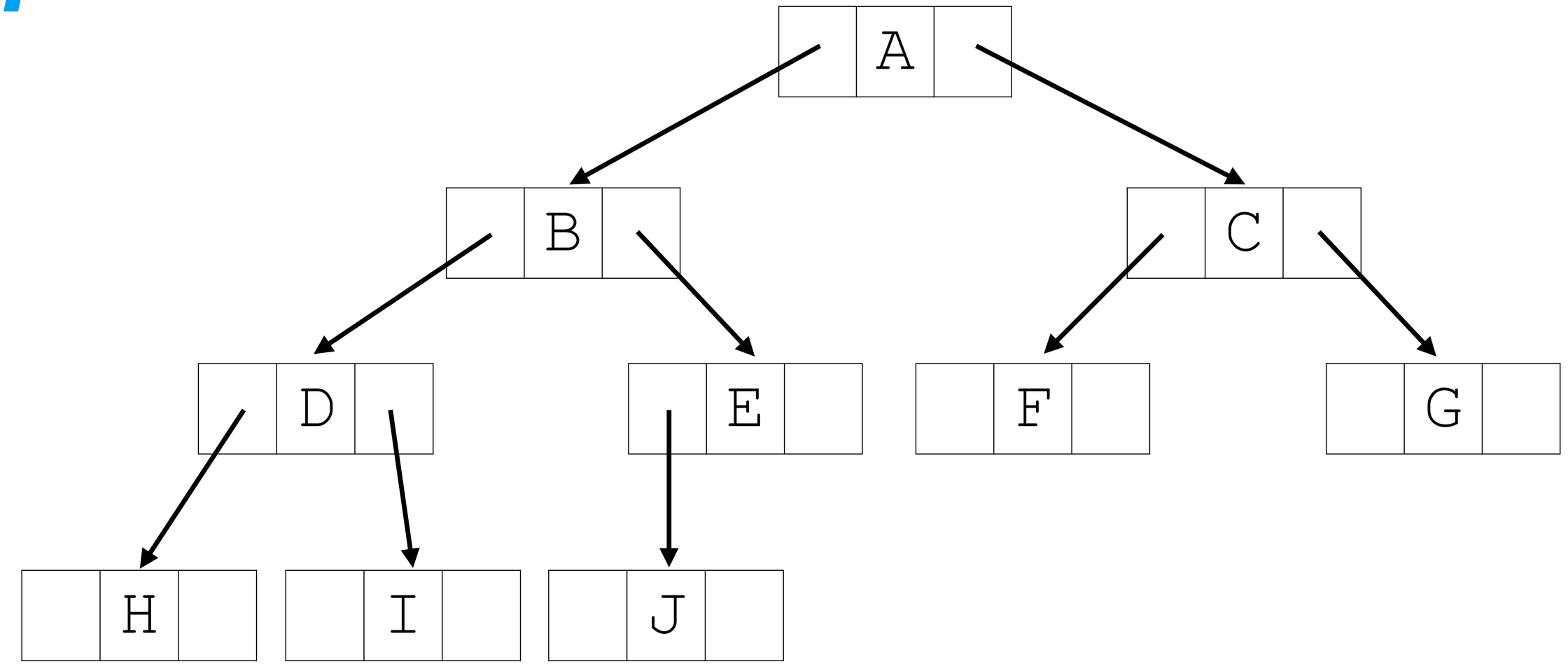
但这样线性序列只有在遍历时才能得到

将二叉树线索化得到线索二叉树可以解决这个问题

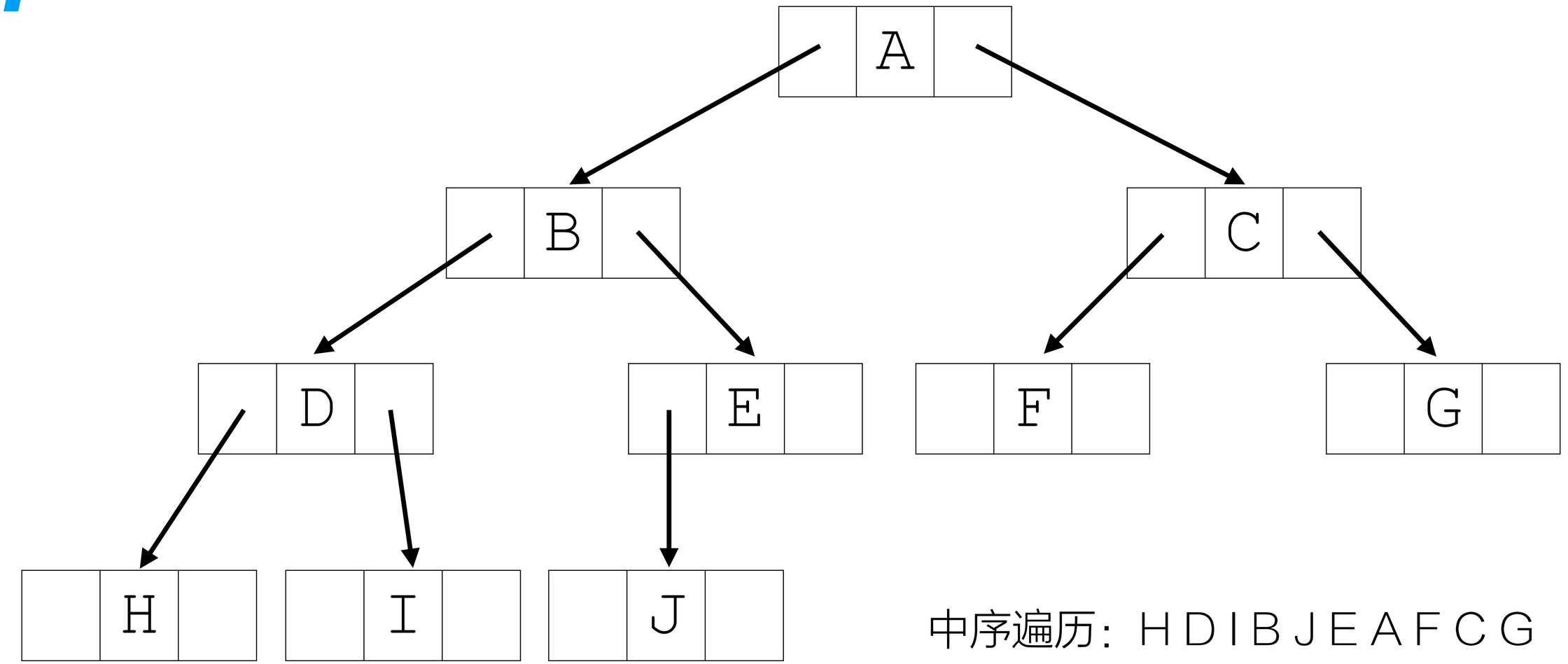
线索化：利用叶节点的空余空间记录前驱、后继



线索二叉树

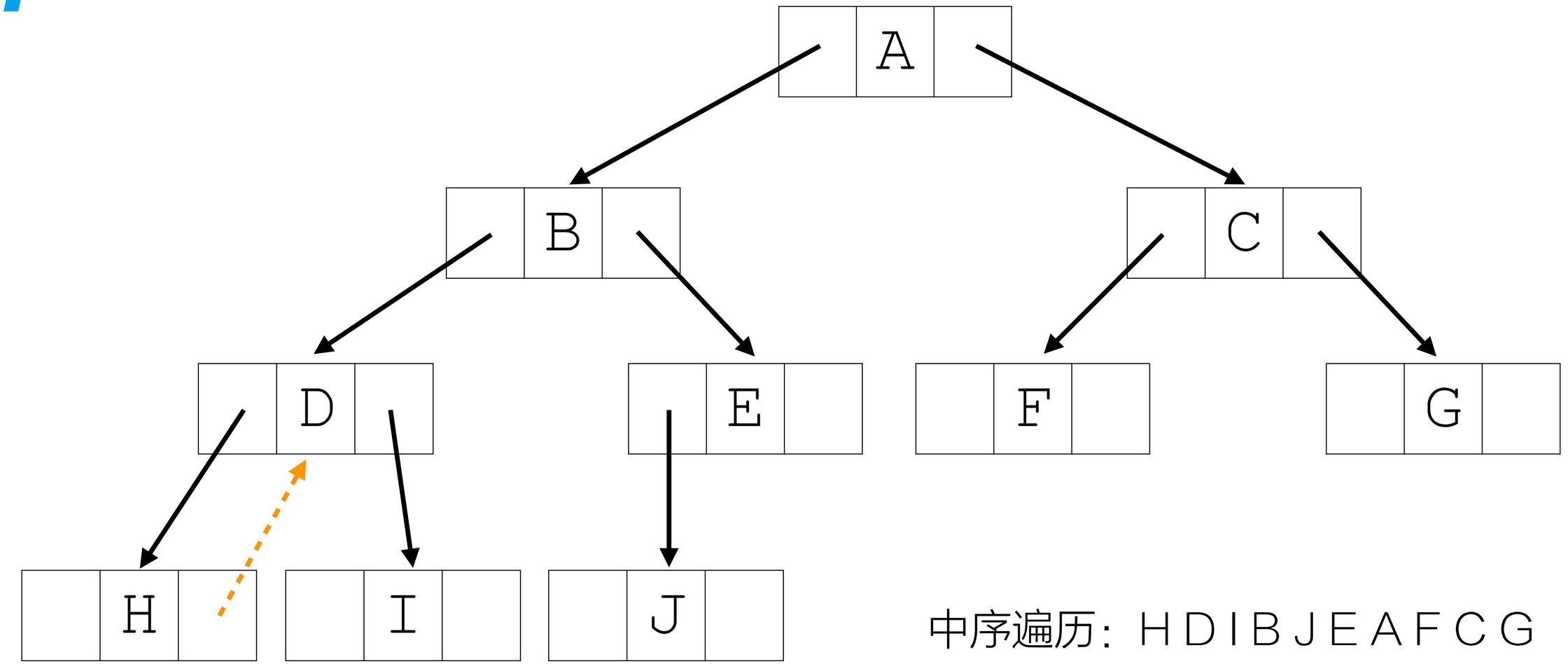


线索二叉树



中序遍历：H D I B J E A F C G

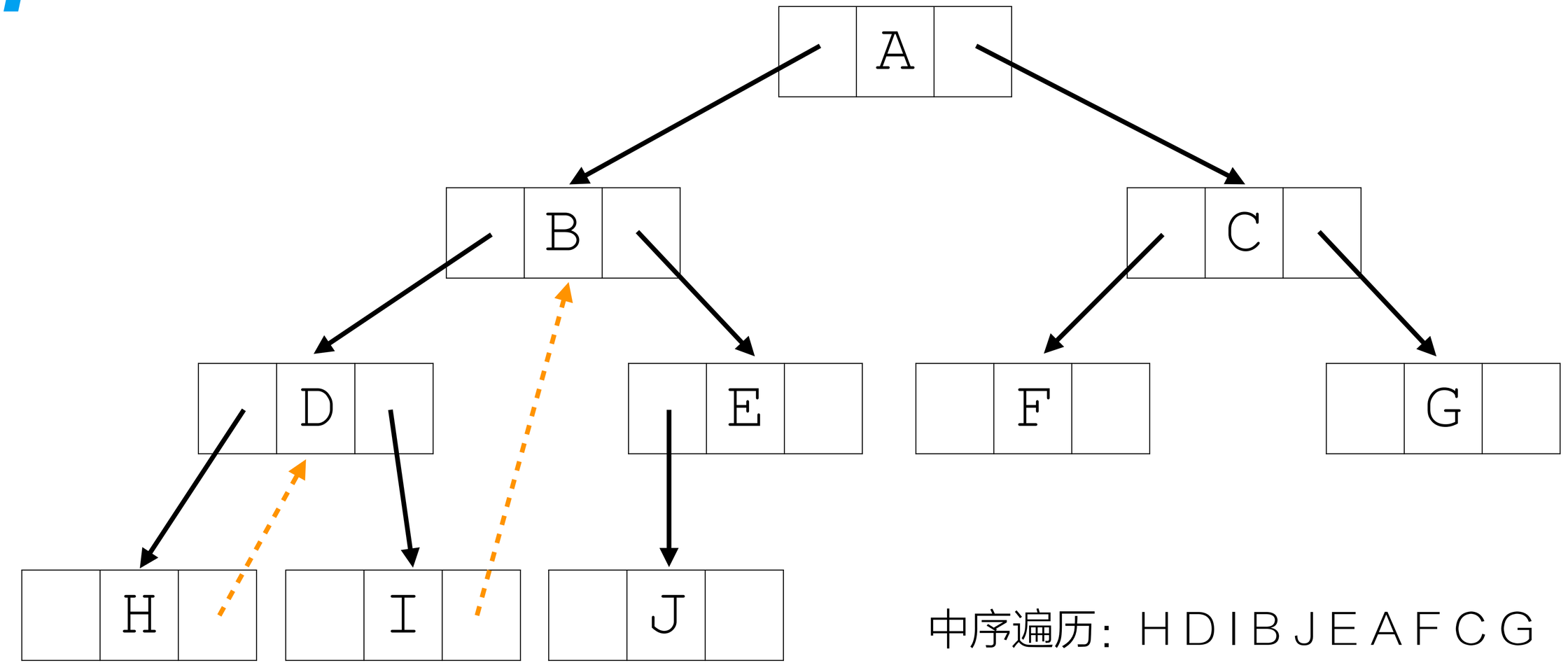
线索二叉树



中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

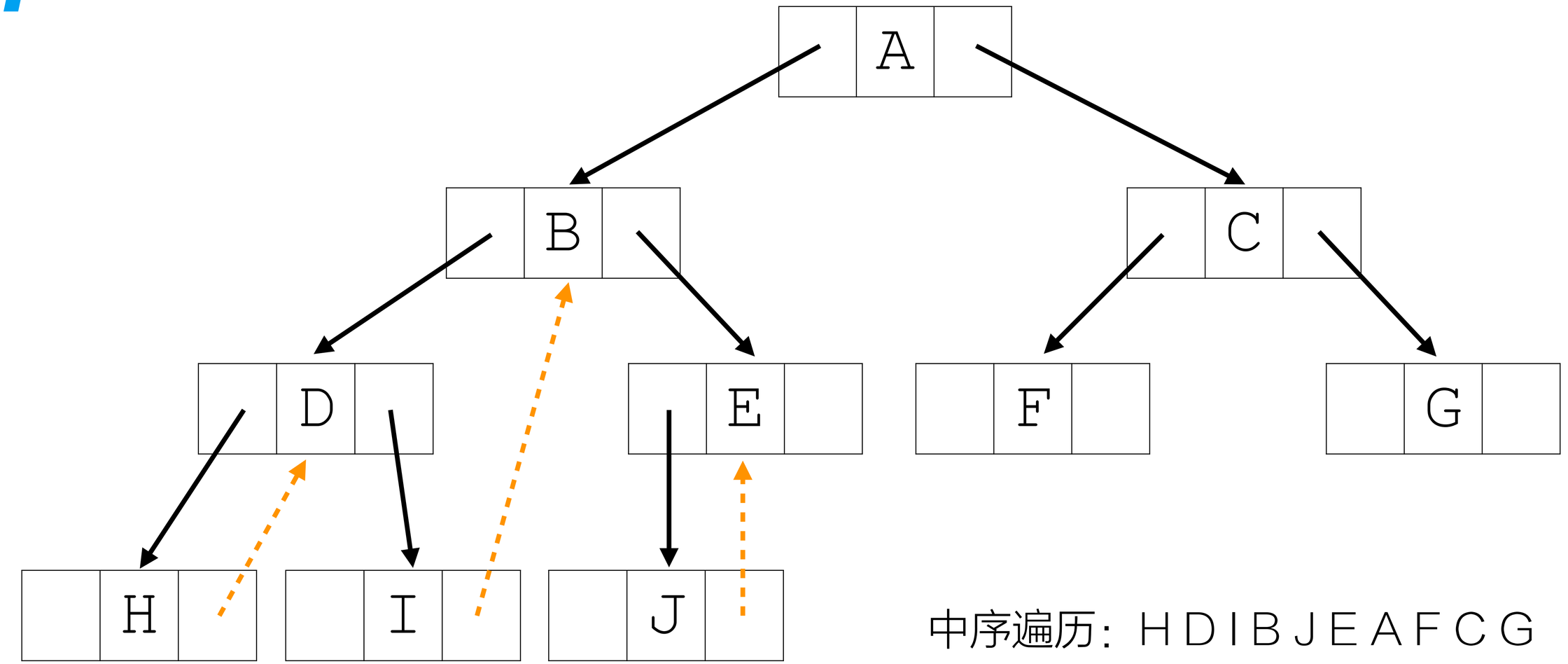
线索二叉树



中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

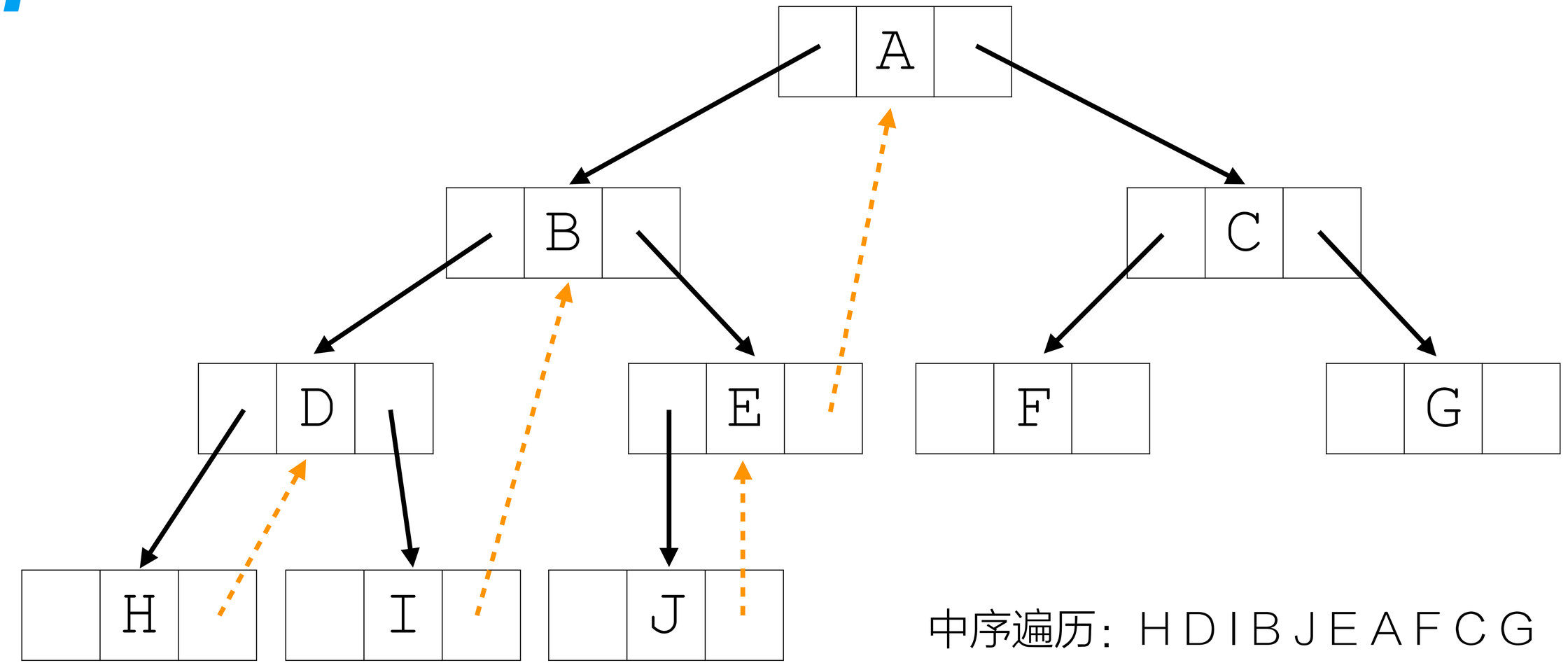
线索二叉树



中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

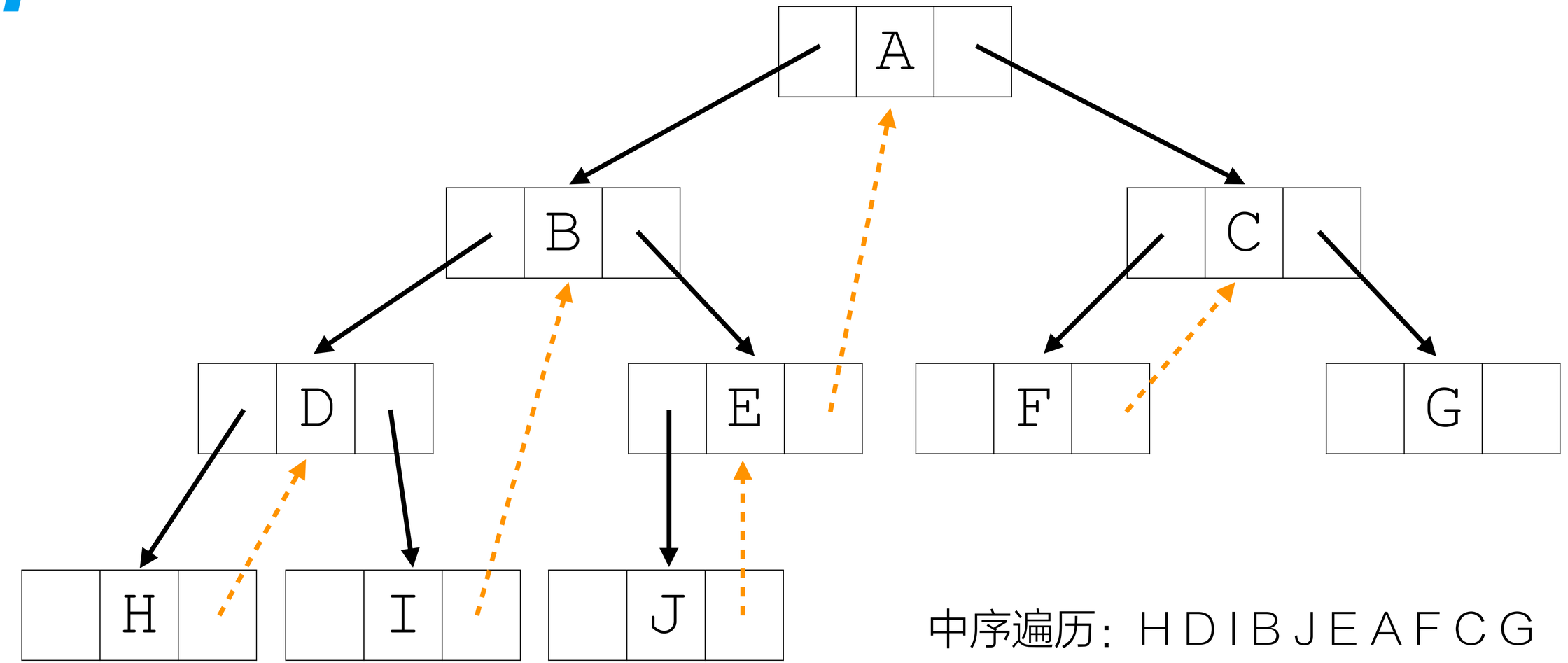
线索二叉树



中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

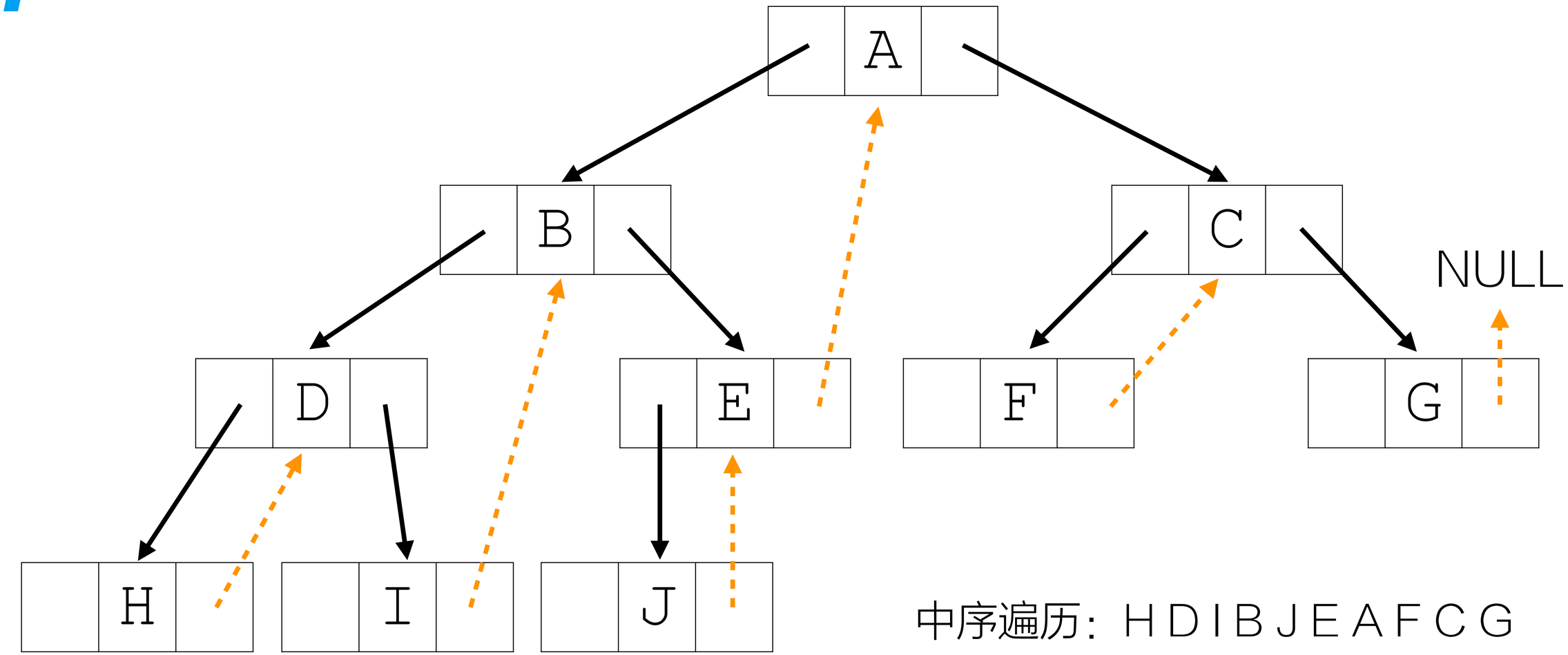
线索二叉树



中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

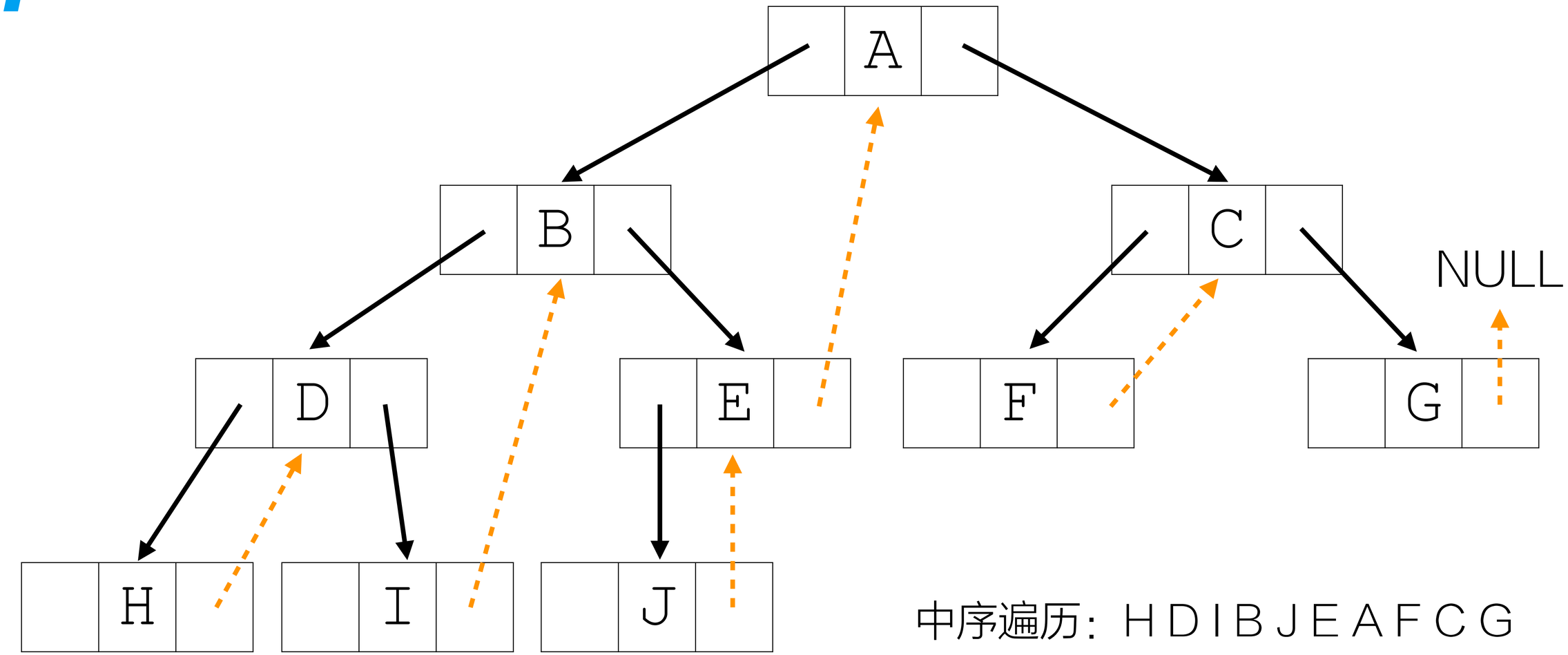
线索二叉树



中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

线索二叉树

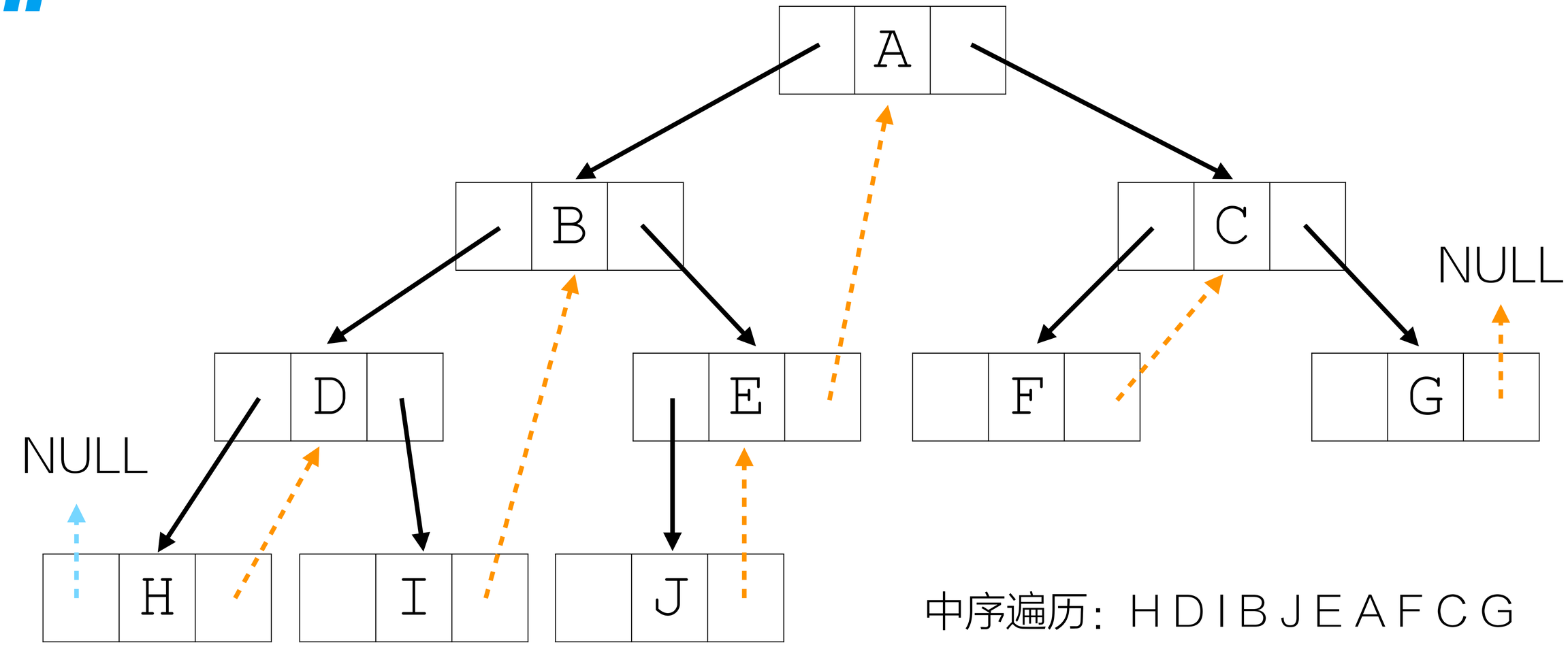


中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

左孩子为空值的节点，用来记录前驱

线索二叉树

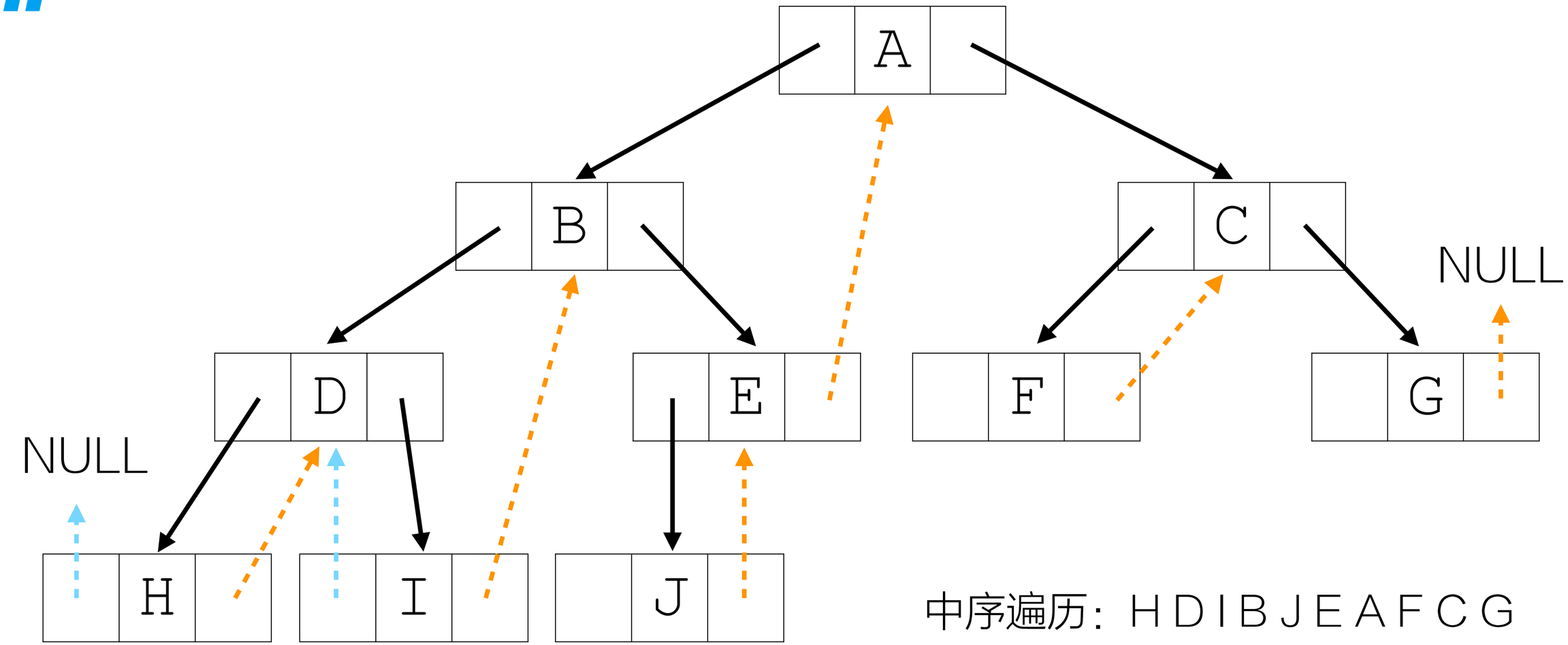


中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

左孩子为空值的节点，用来记录前驱

线索二叉树

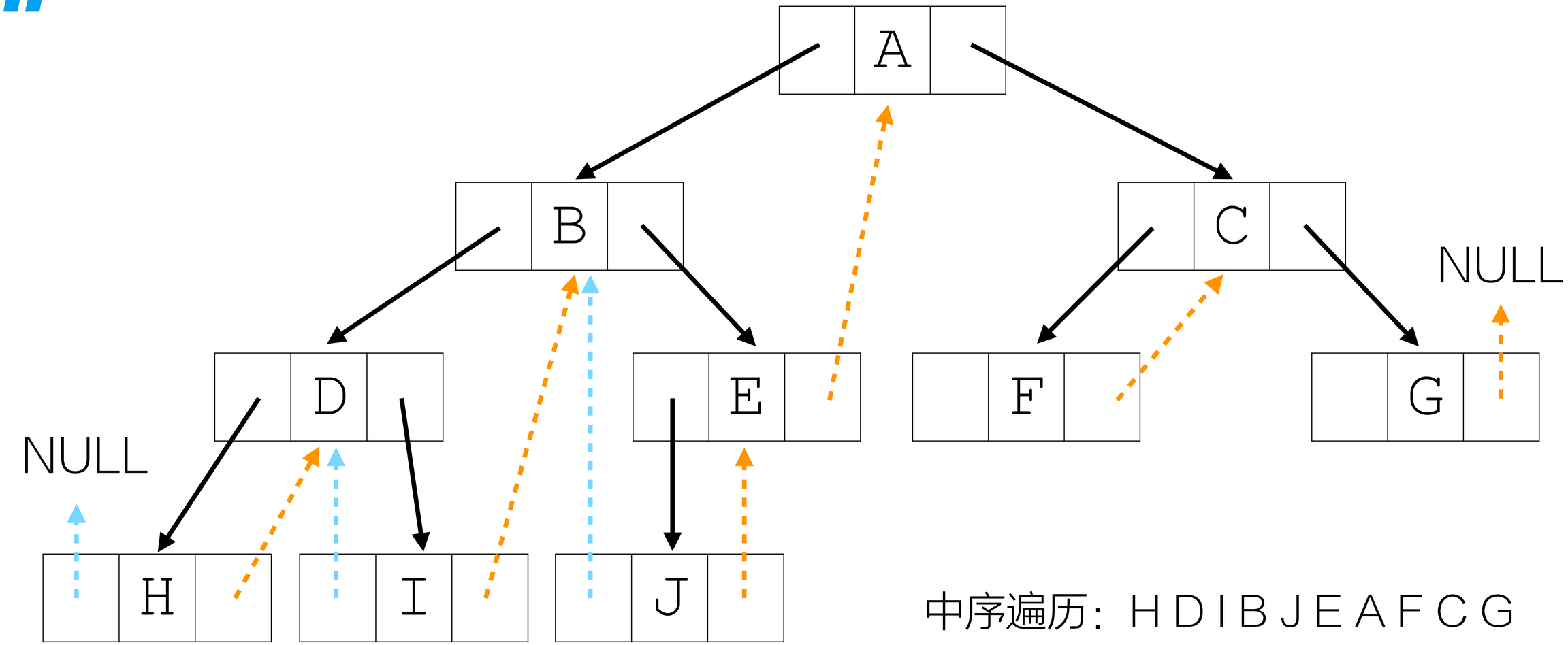


中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

左孩子为空值的节点，用来记录前驱

线索二叉树

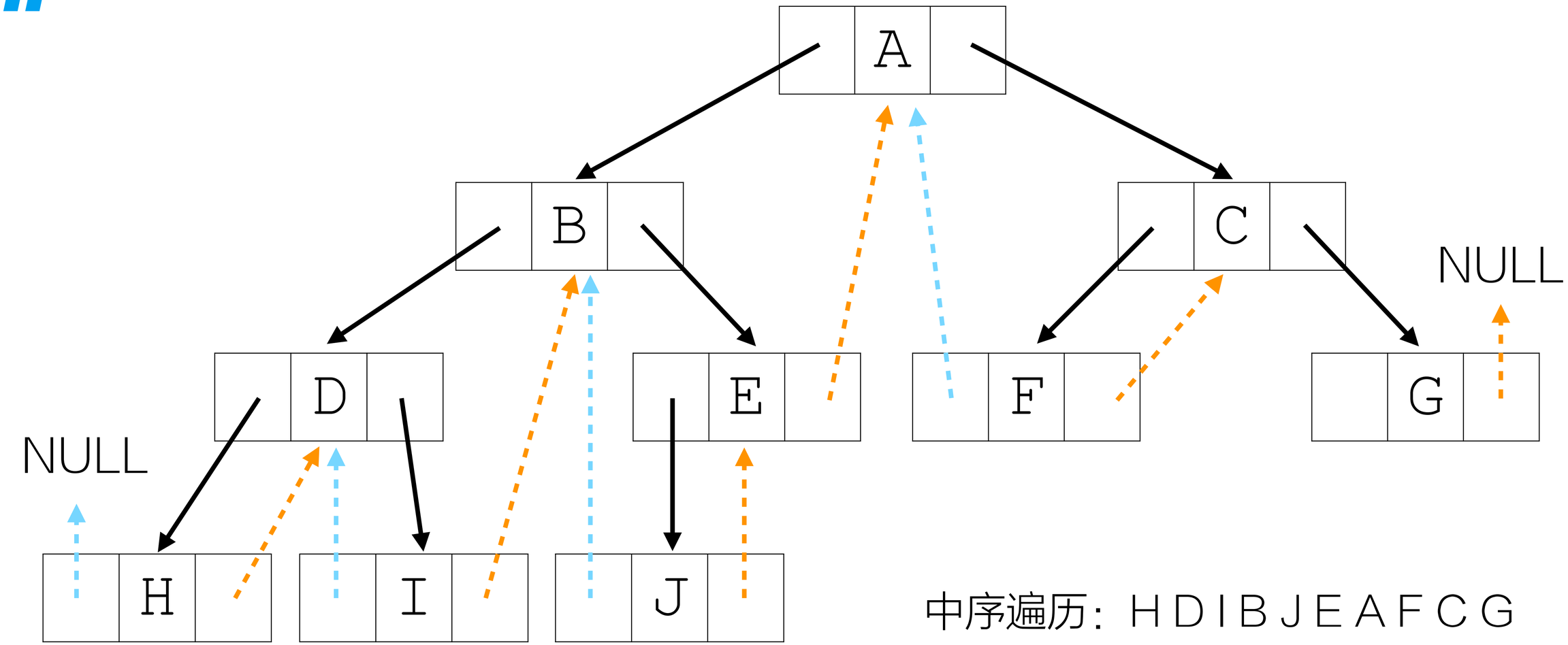


中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

左孩子为空值的节点，用来记录前驱

线索二叉树



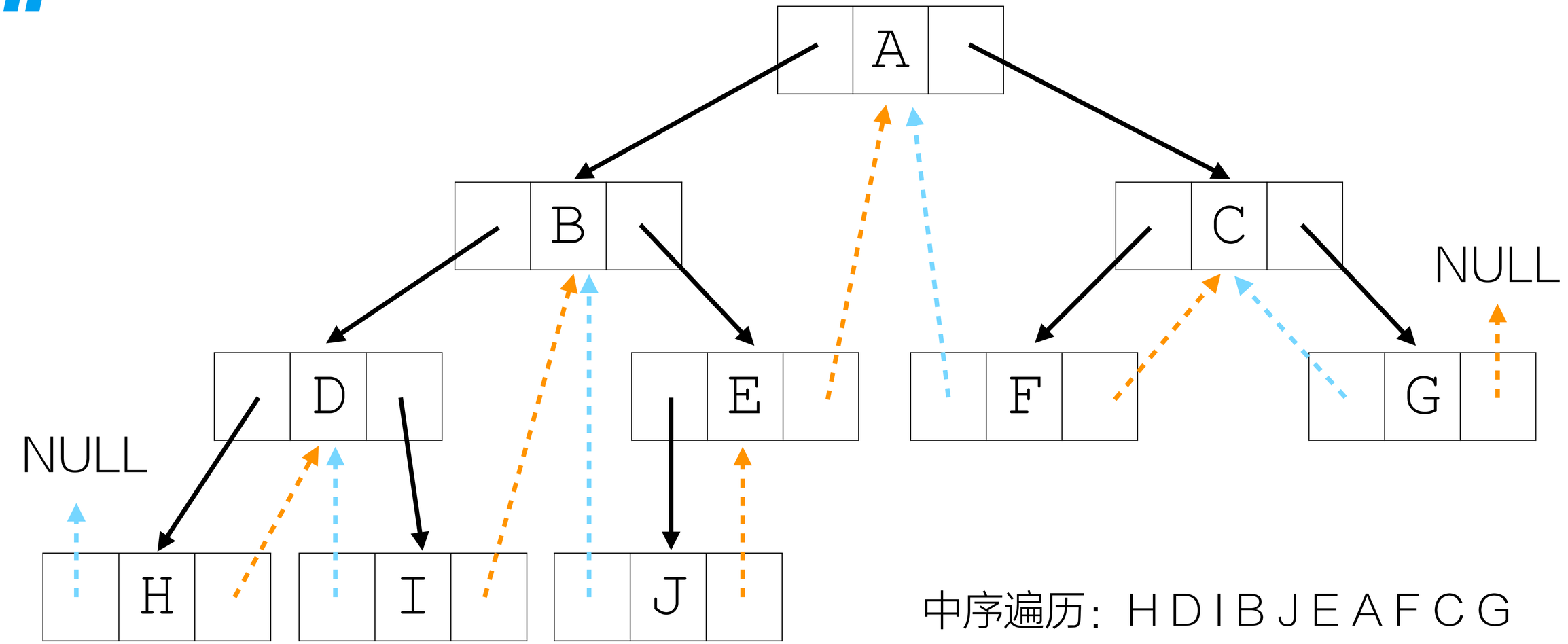
中序遍历：H D I B J E A F C G

右孩子为空值的节点，用来记录后继

左孩子为空值的节点，用来记录前驱

线索二叉树

案例：树 / test5.c



中序遍历：H D I B J E A F C G

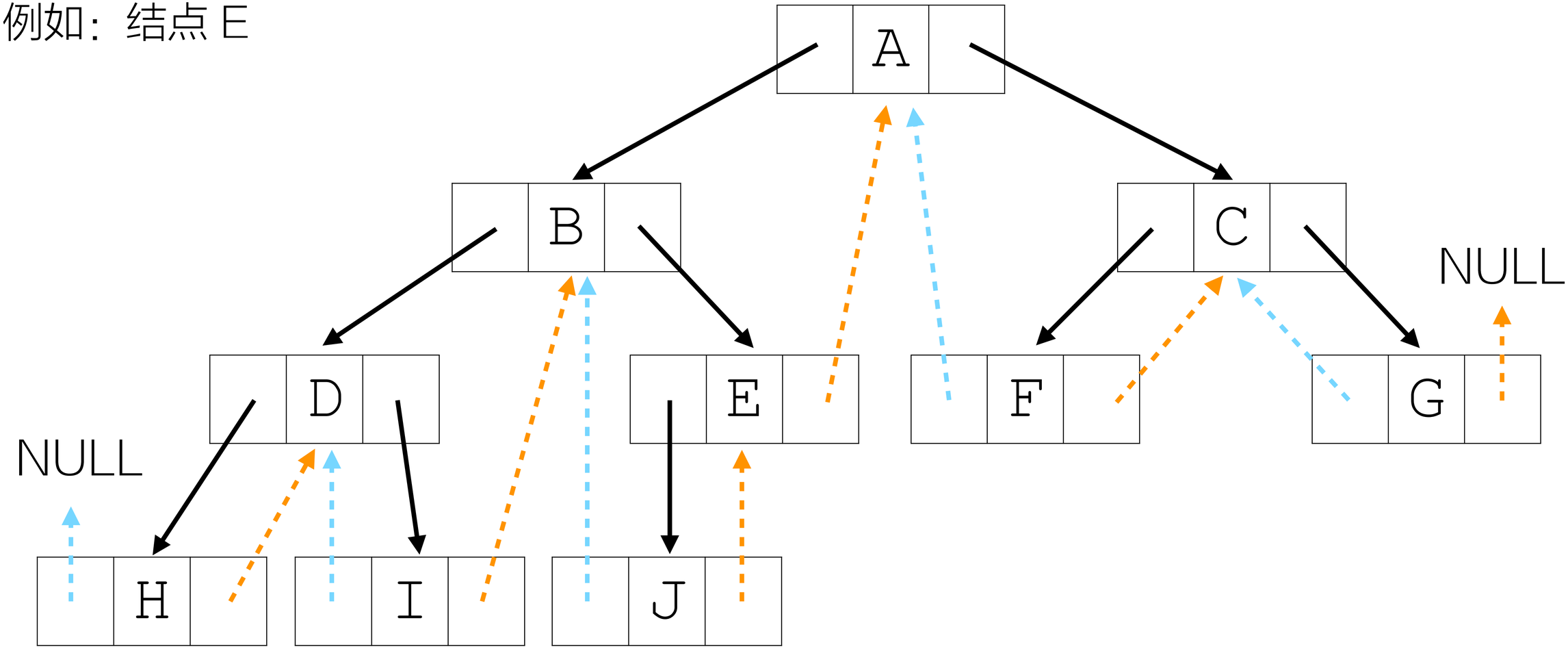
右孩子为空值的节点，用来记录后继

左孩子为空值的节点，用来记录前驱

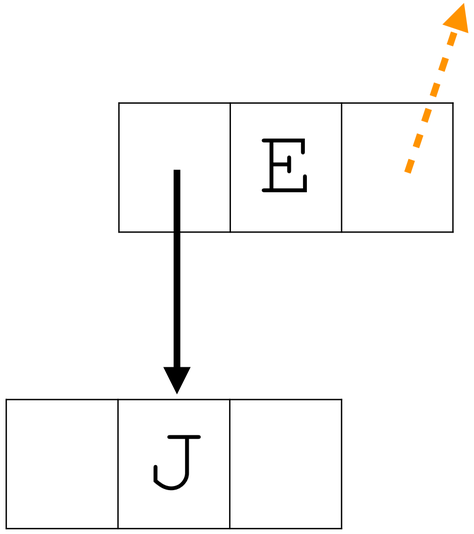
线索二叉树

从代码实现的角度，如何判断节点中存放的是孩子还是线索?

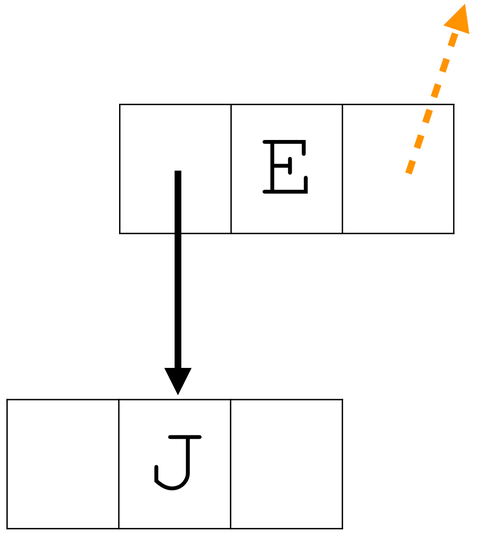
例如：结点 E

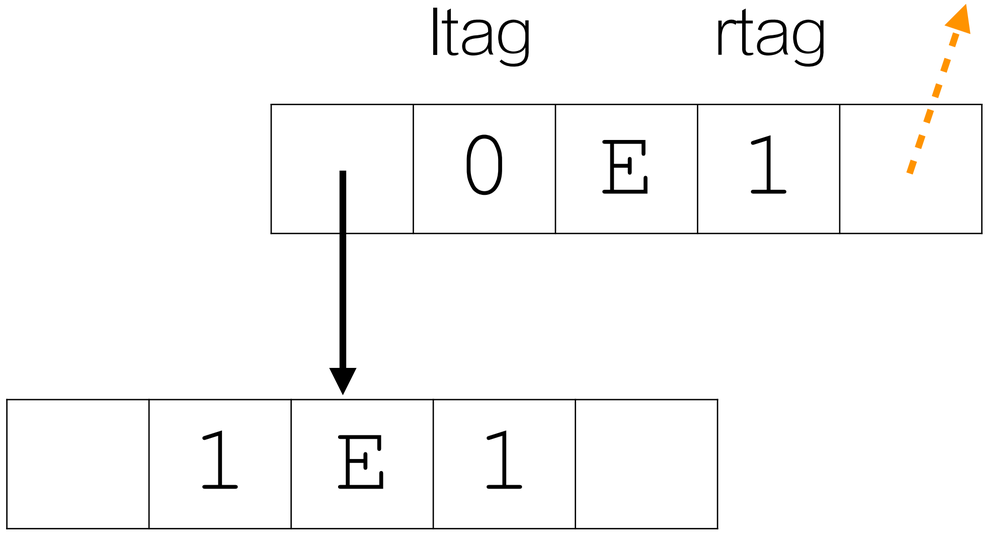


线索二叉树



线索二叉树





ltag、rtag：0 表示孩子节点，1 表示线索

线索二叉树 – 存储结构

```
typedef char ElemType;

typedef struct ThreadNode
{
        ElemType data;
        struct ThreadNode *lchild;
        struct ThreadNode *rchild;
        int ltag;
        int rtag;
}ThreadNode;

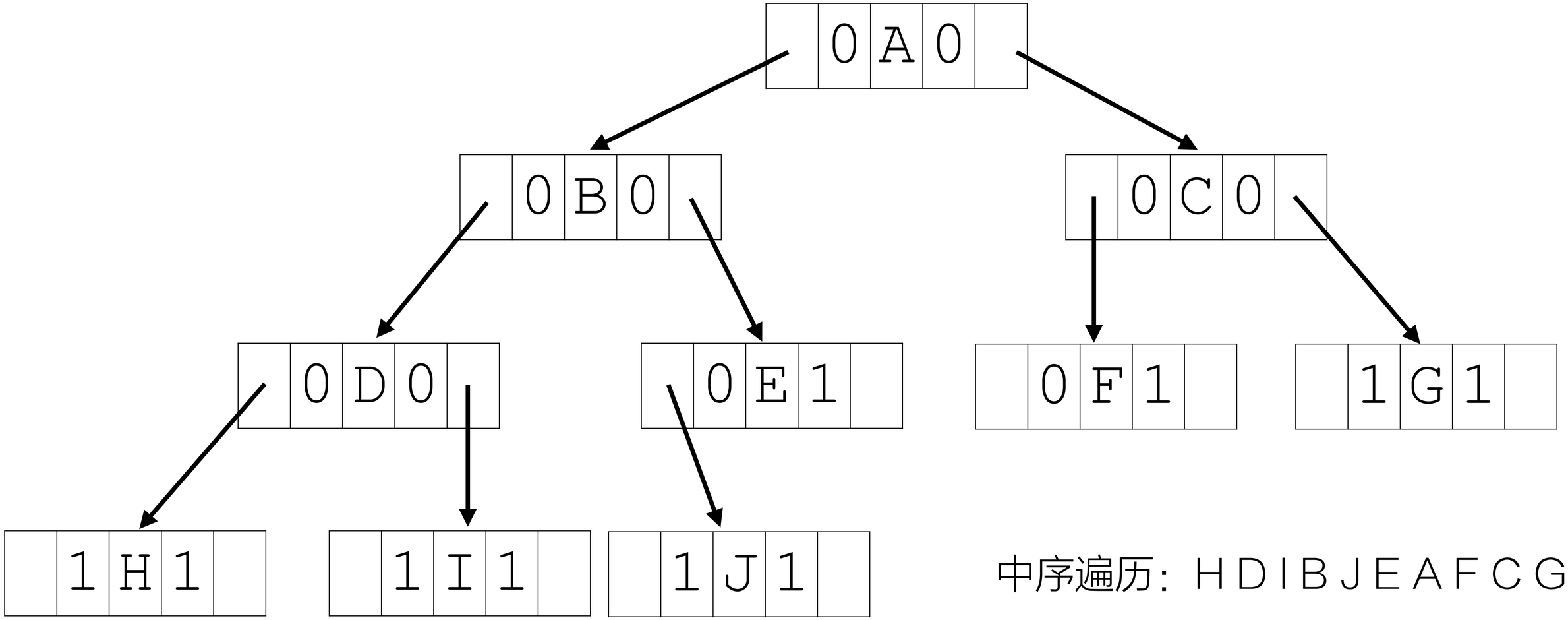
typedef ThreadNode* ThreadTree;
```

线程　　现成

ltag为 0 时，指向该结点左孩子，
为 1 时，指向该结点的前驱线索

rtag为 0 时，指向该结点右孩子，
为 1 时，指向该结点的后继线索

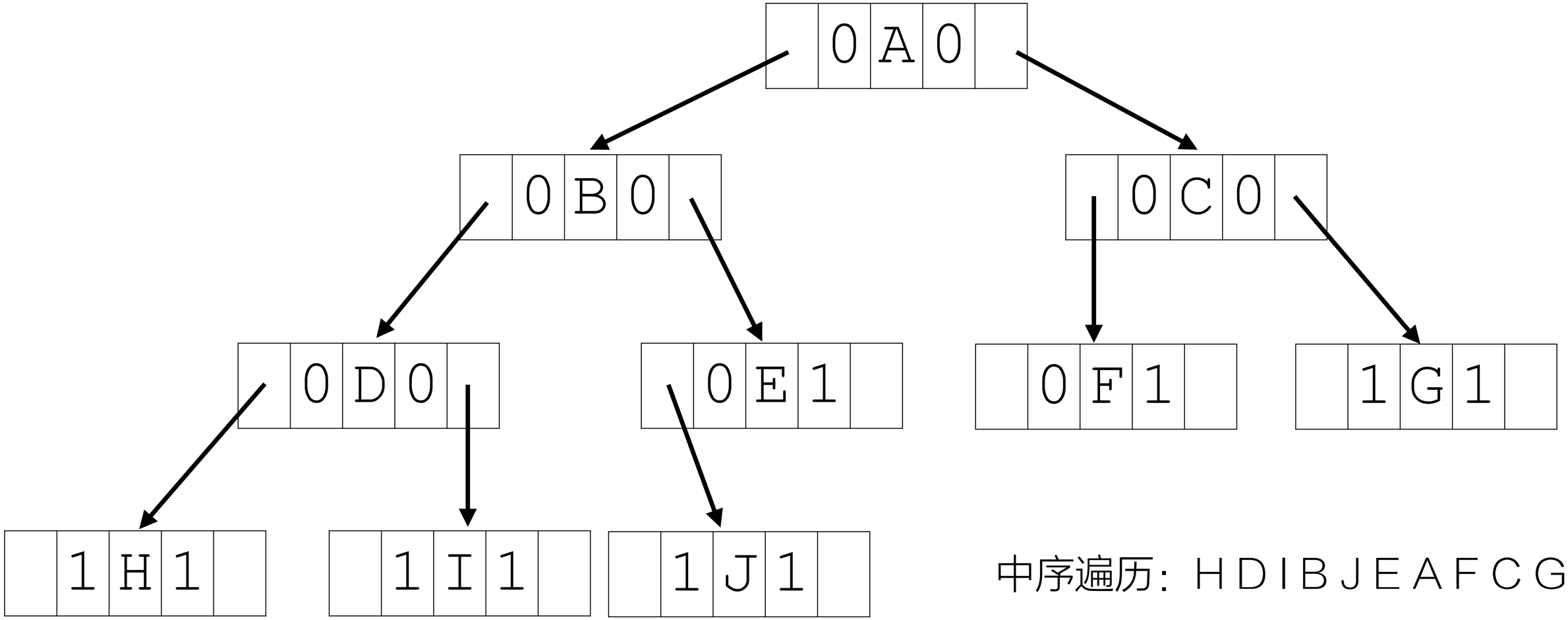
线索二叉树 – 中序遍历线索化



中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

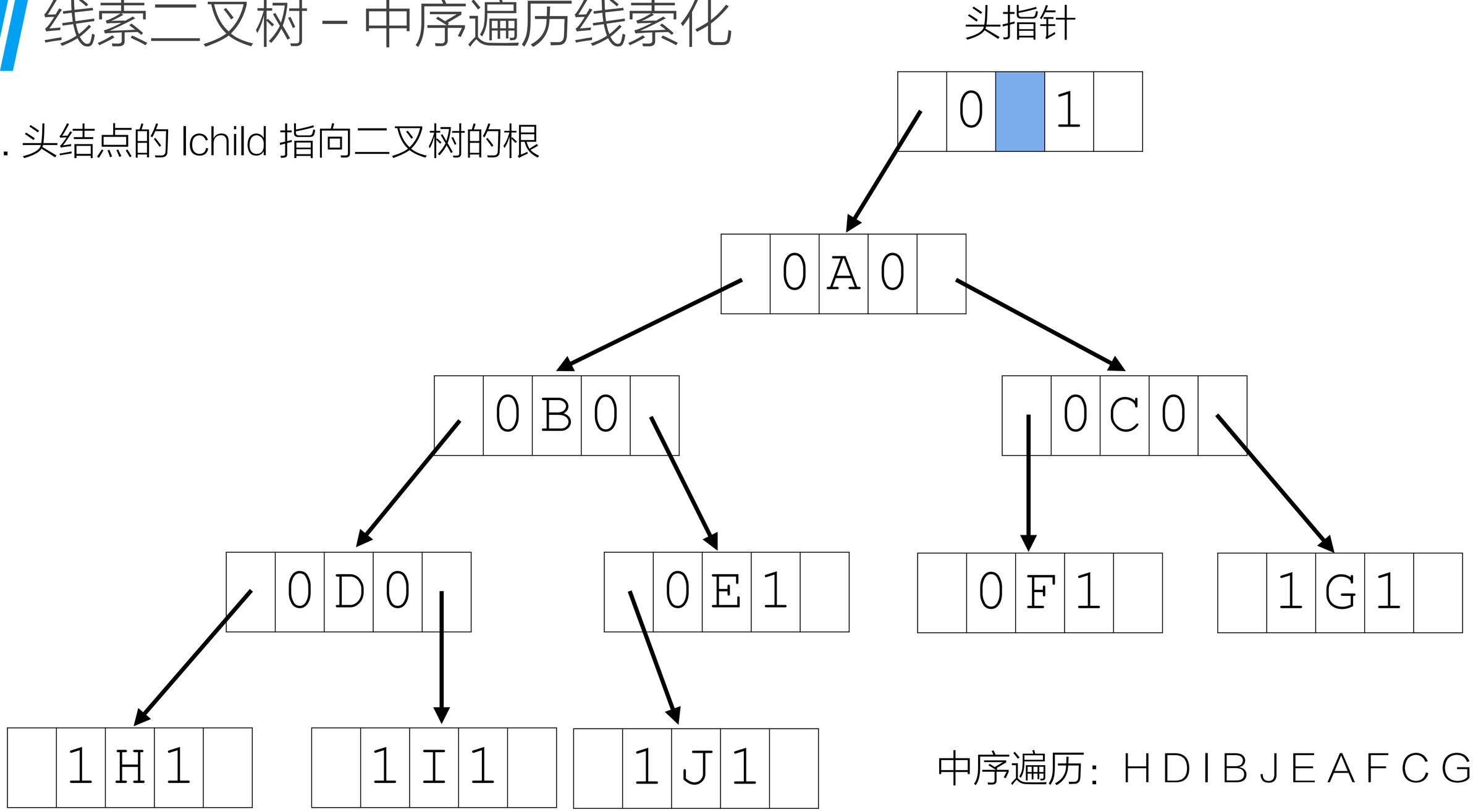
1. 头结点的 lchild 指向二叉树的根



中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

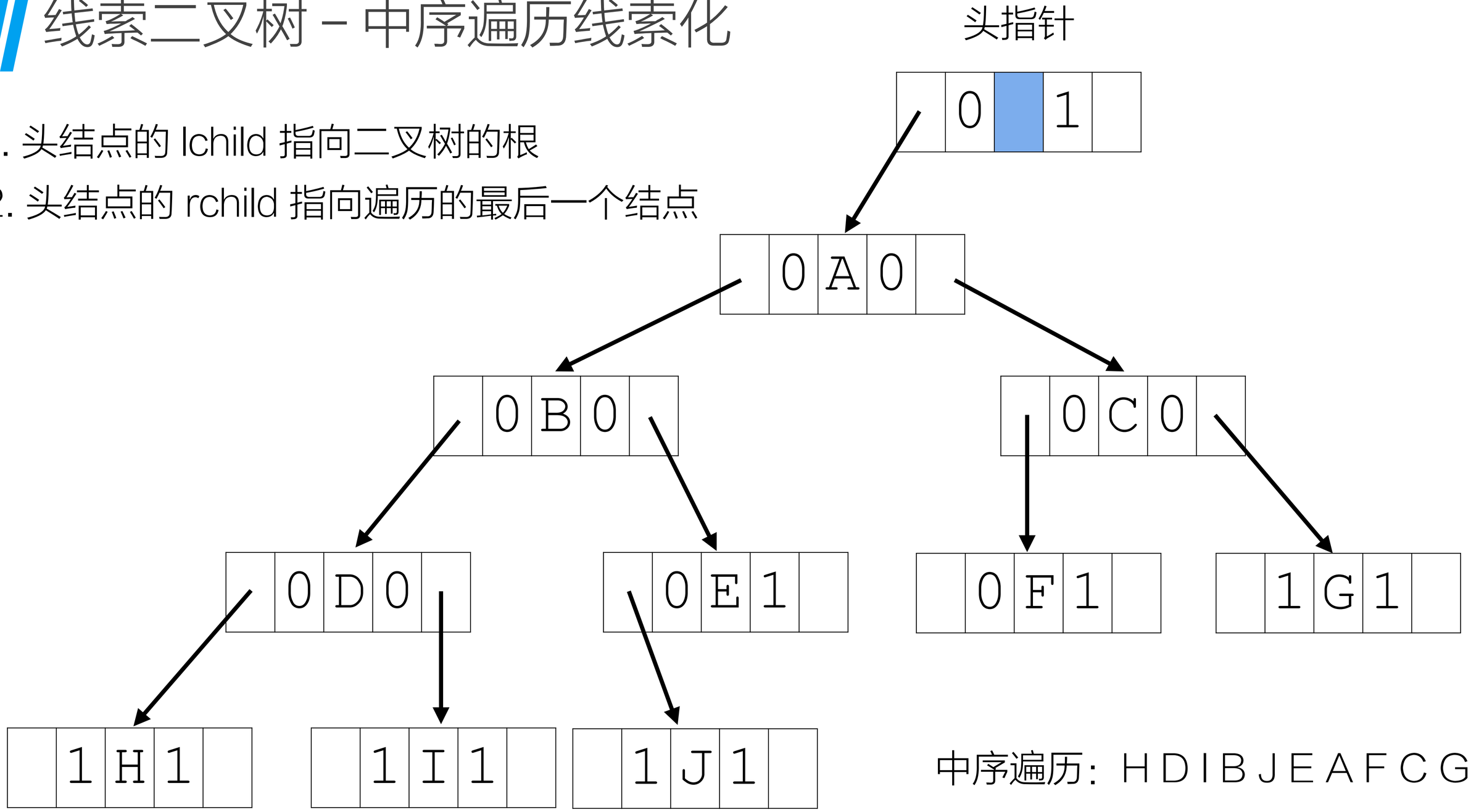
1. 头结点的 lchild 指向二叉树的根



中序遍历：H D I B J E A F C G

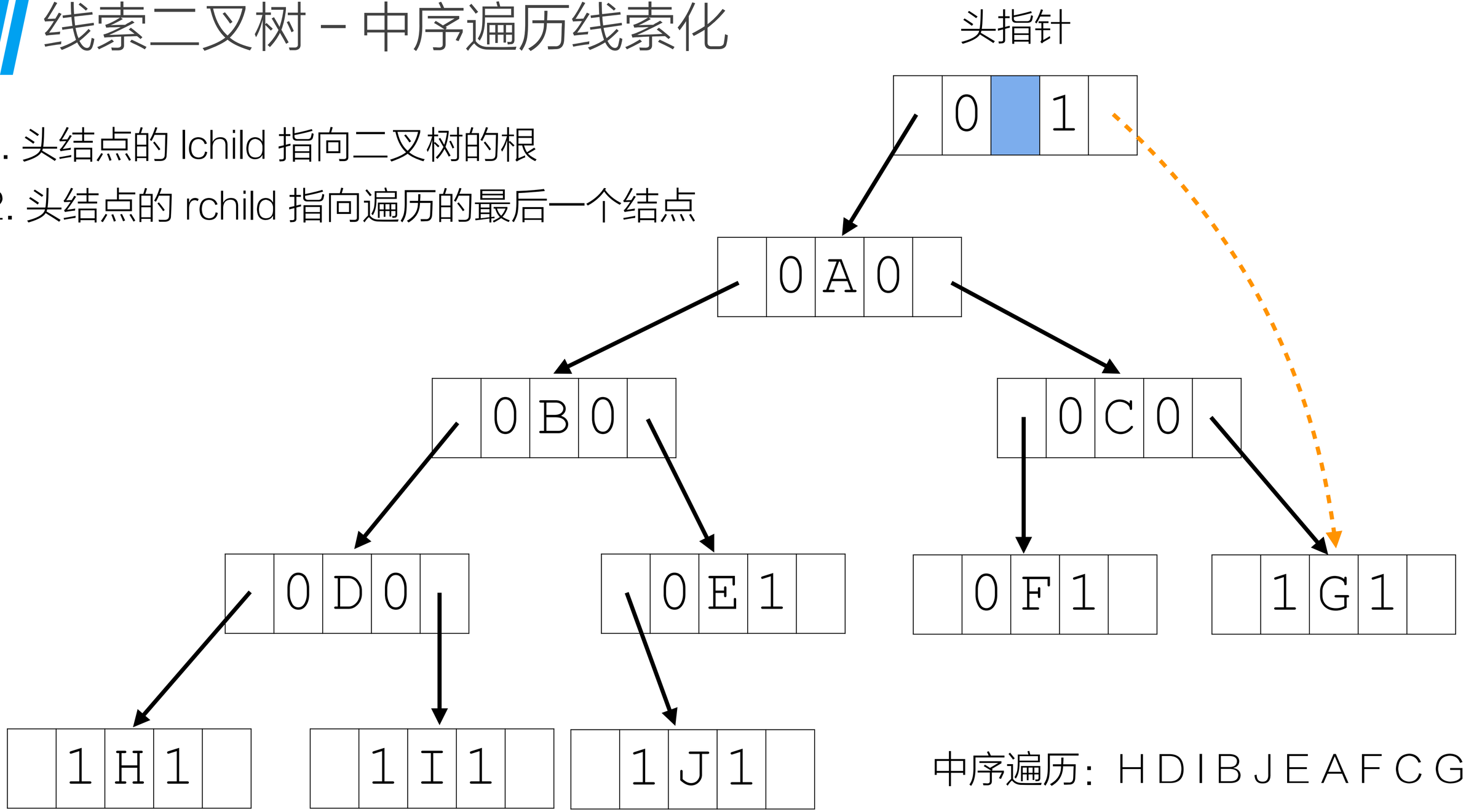
线索二叉树 – 中序遍历线索化

1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点



线索二叉树 – 中序遍历线索化

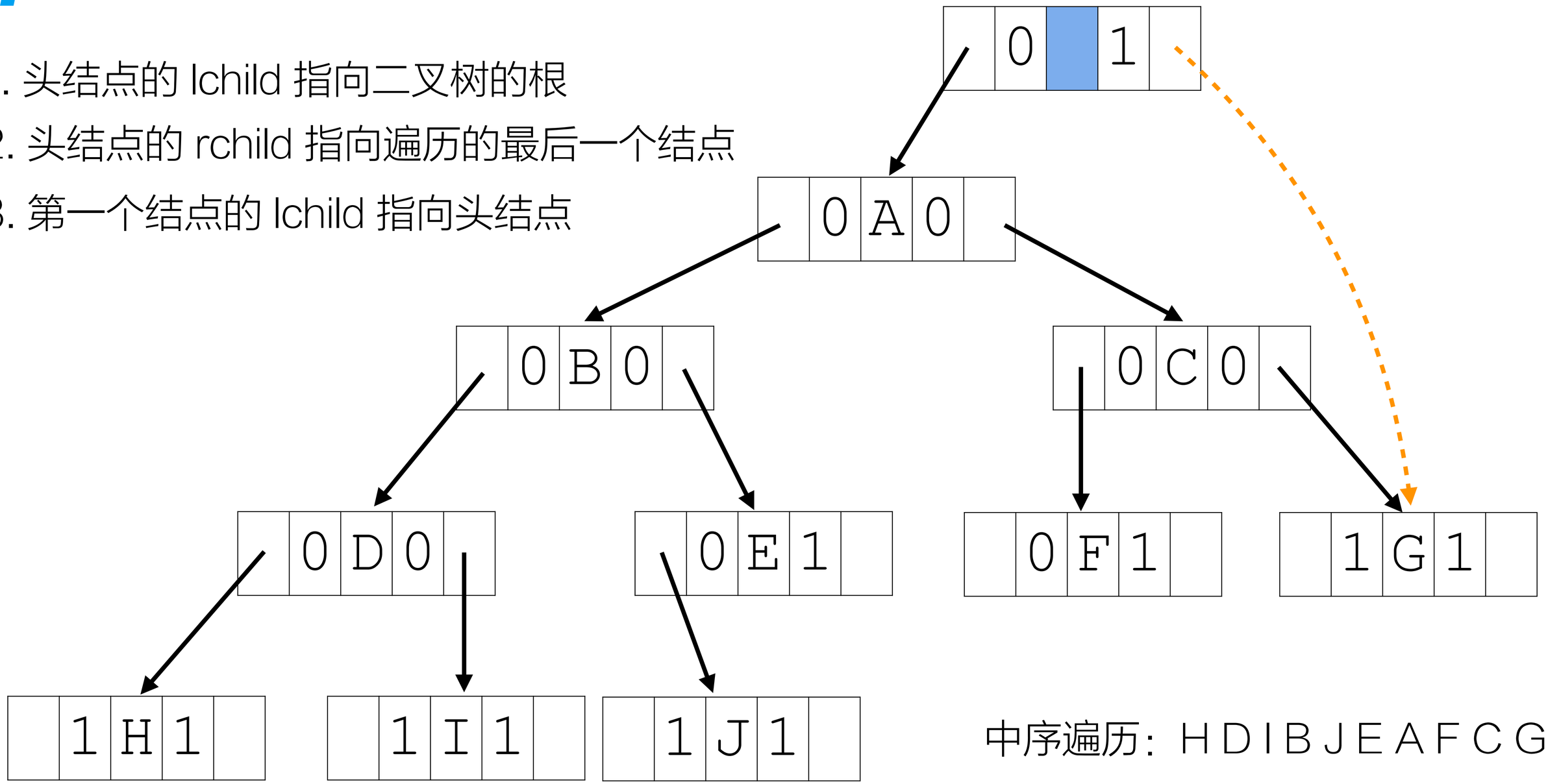
1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点



中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

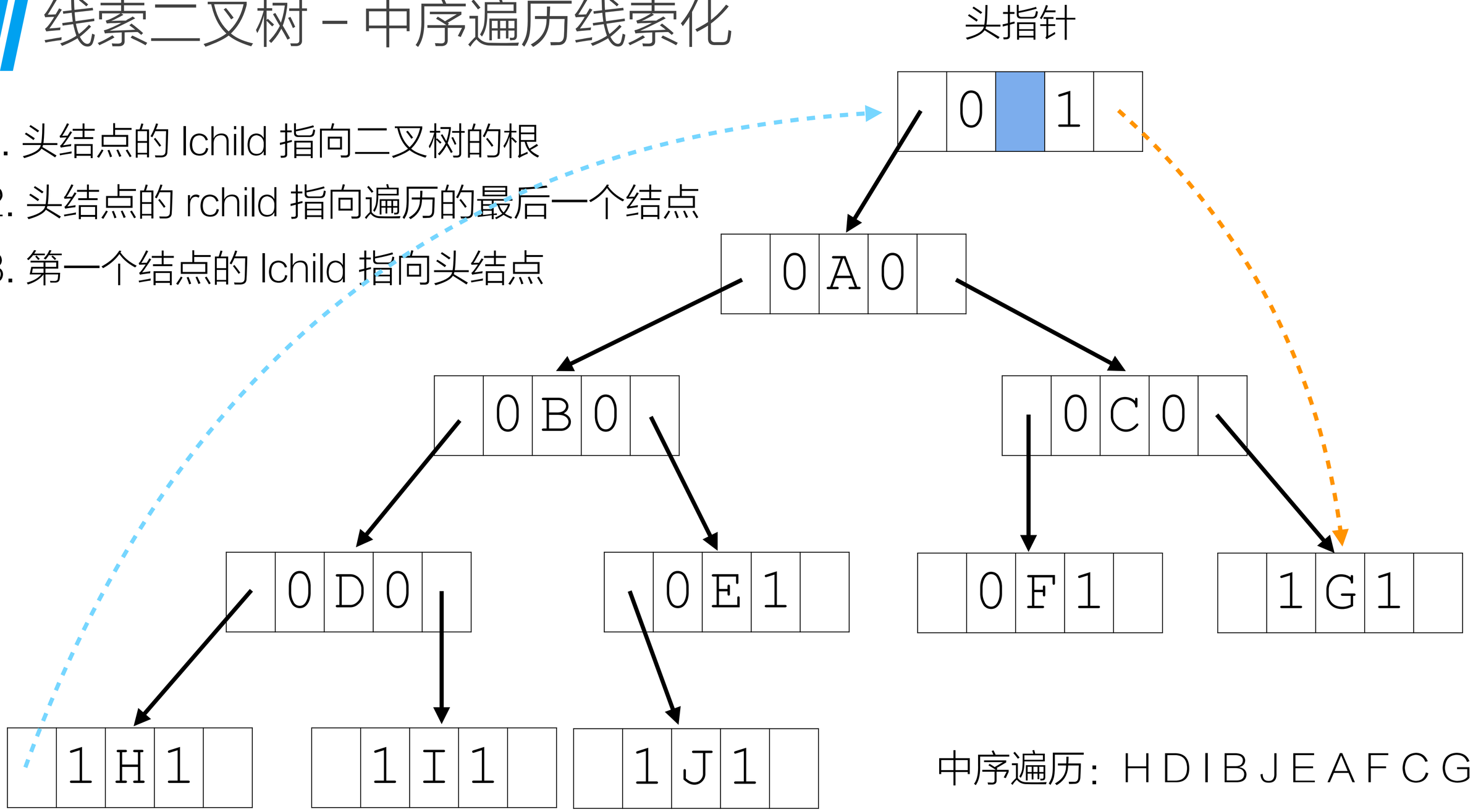
1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点
3. 第一个结点的 lchild 指向头结点



中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

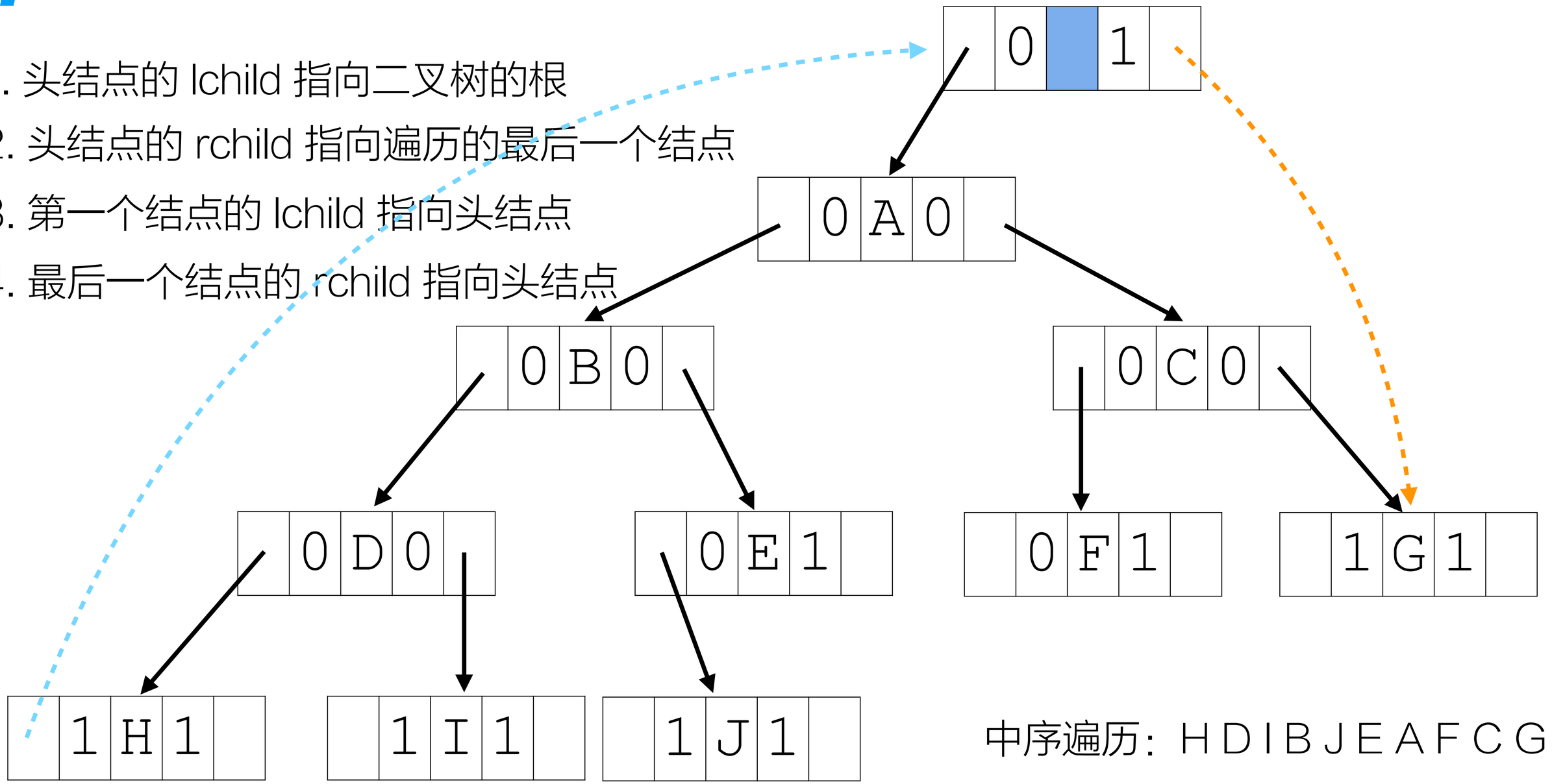
1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点
3. 第一个结点的 lchild 指向头结点



中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点
3. 第一个结点的 lchild 指向头结点
4. 最后一个结点的 rchild 指向头结点

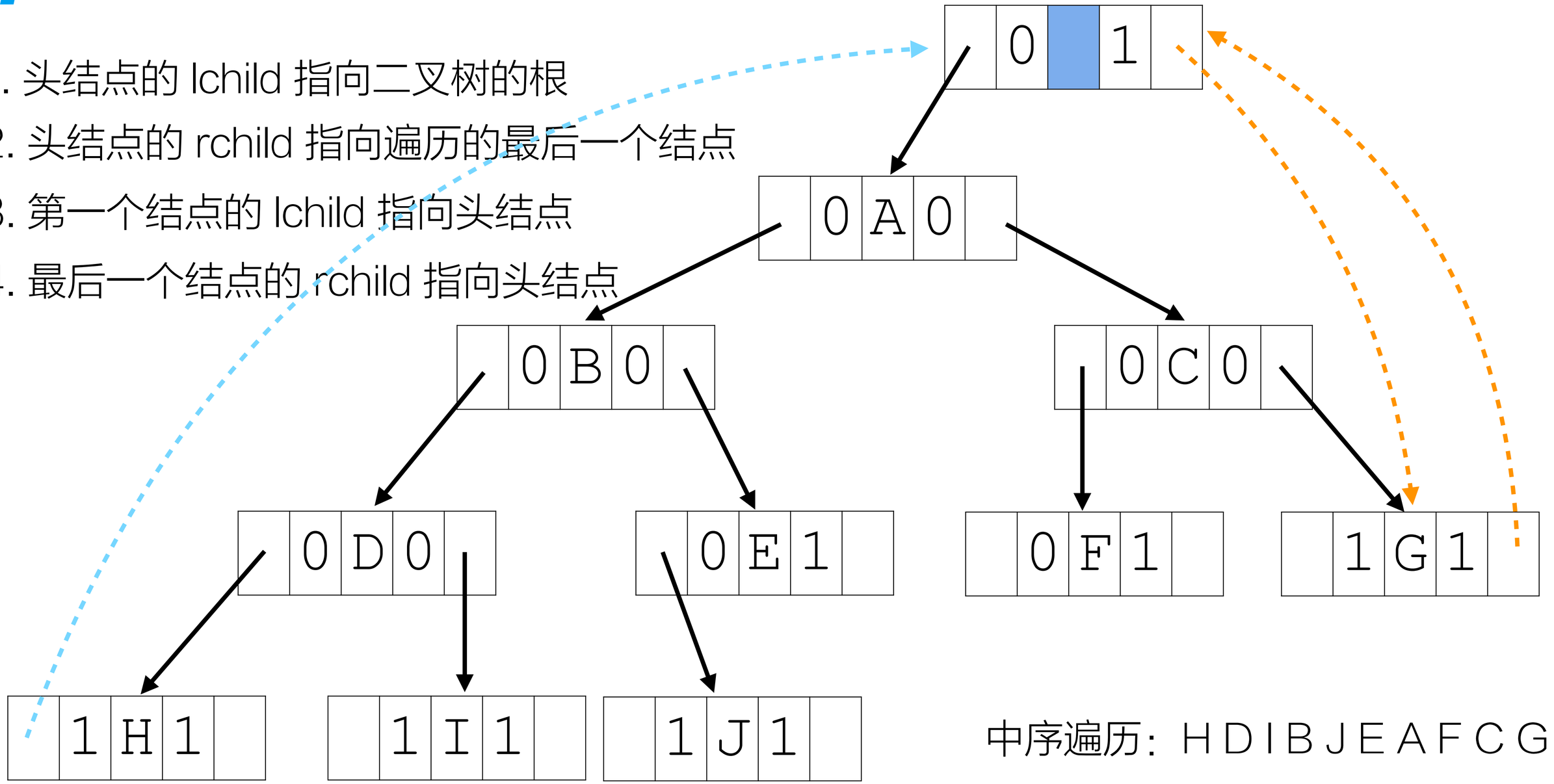


中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

头指针　　案例：树 / test5.c

1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点
3. 第一个结点的 lchild 指向头结点
4. 最后一个结点的 rchild 指向头结点

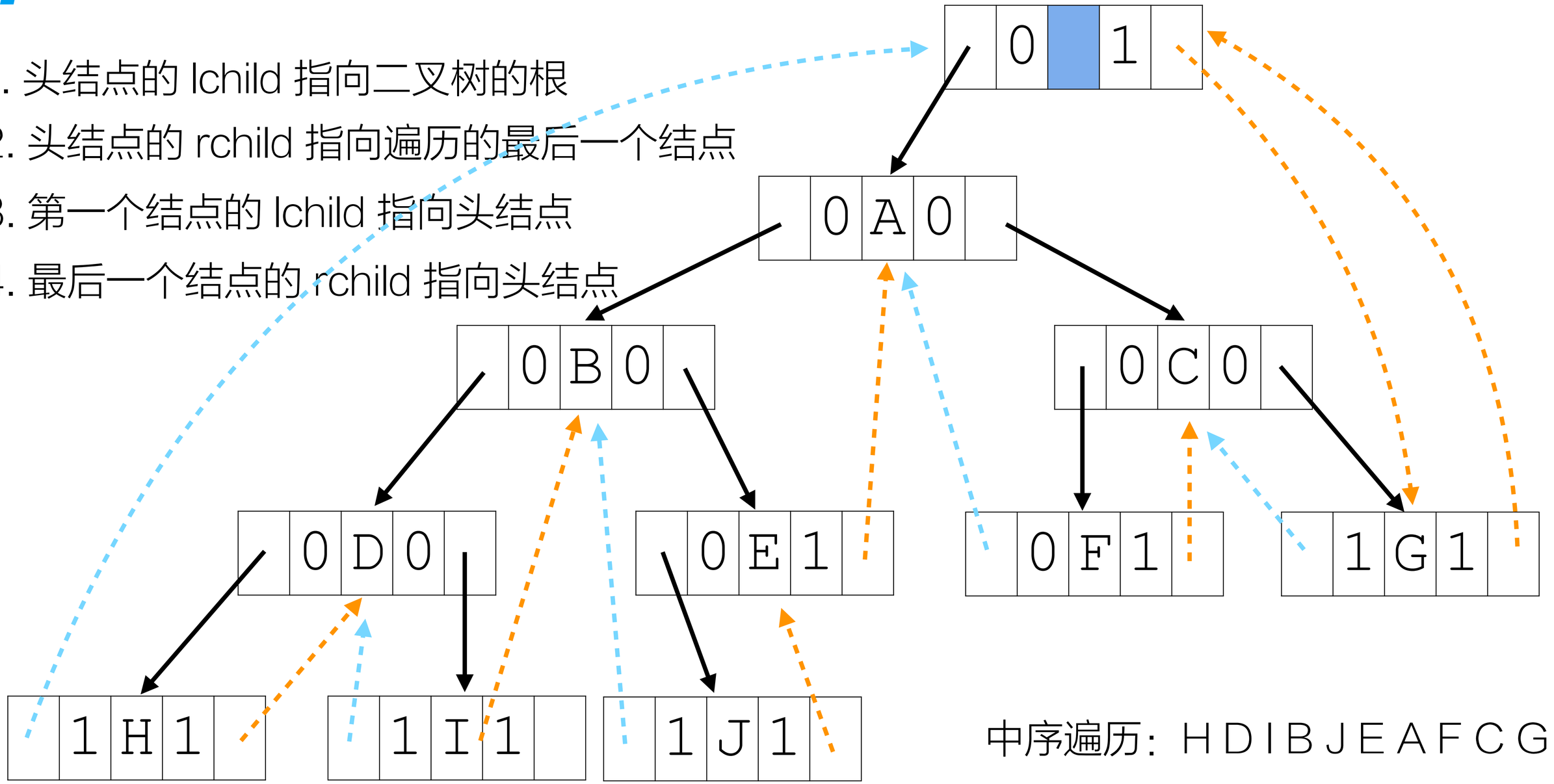


中序遍历：H D I B J E A F C G

线索二叉树 – 中序遍历线索化

头指针　　案例：树 / test5.c

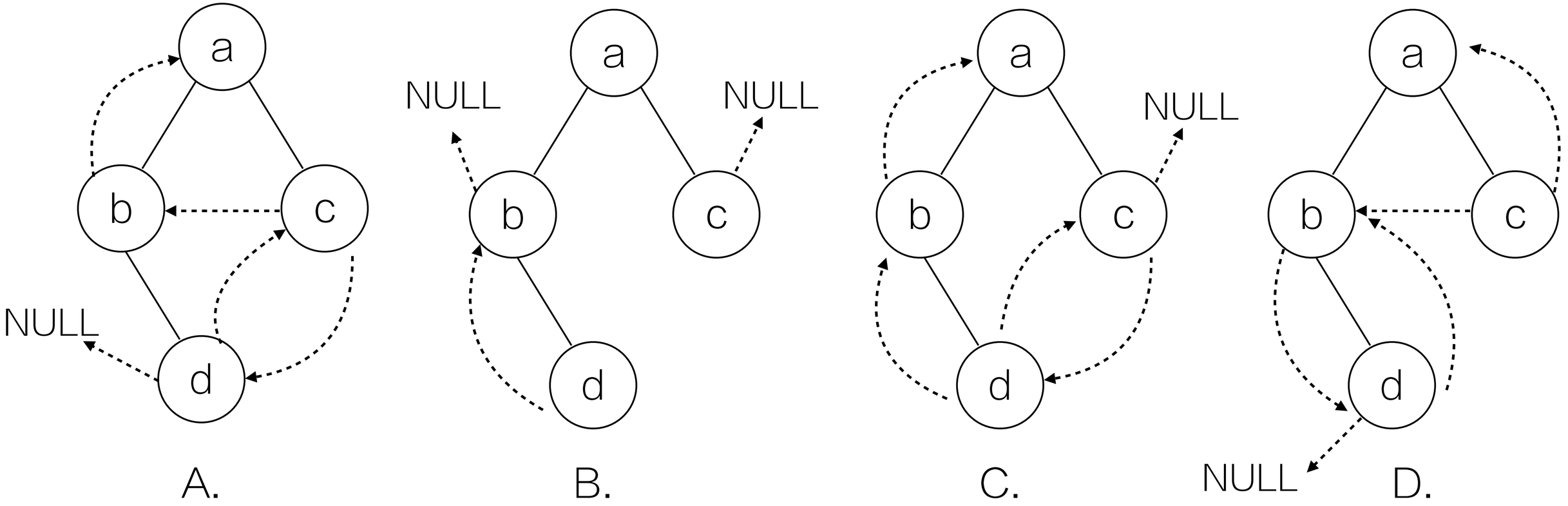
1. 头结点的 lchild 指向二叉树的根
2. 头结点的 rchild 指向遍历的最后一个结点
3. 第一个结点的 lchild 指向头结点
4. 最后一个结点的 rchild 指向头结点



中序遍历：H D I B J E A F C G

二叉树 – 真题解析

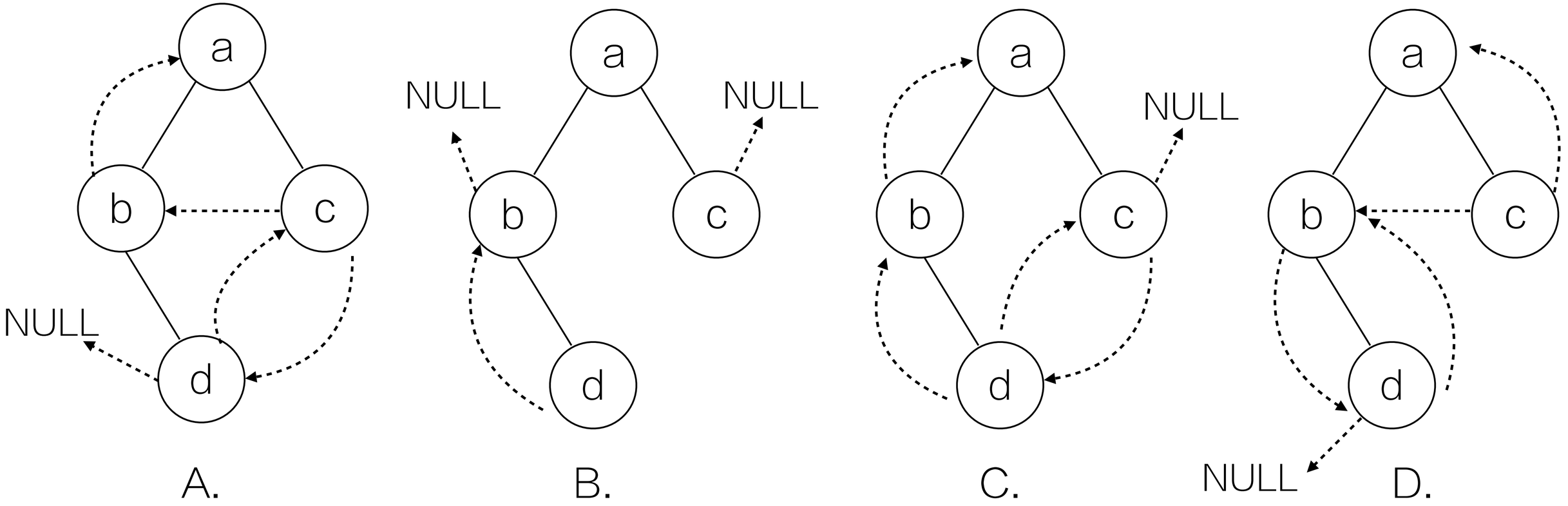
[2010] 3. 下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是_____。



二叉树 – 真题解析

d b c a

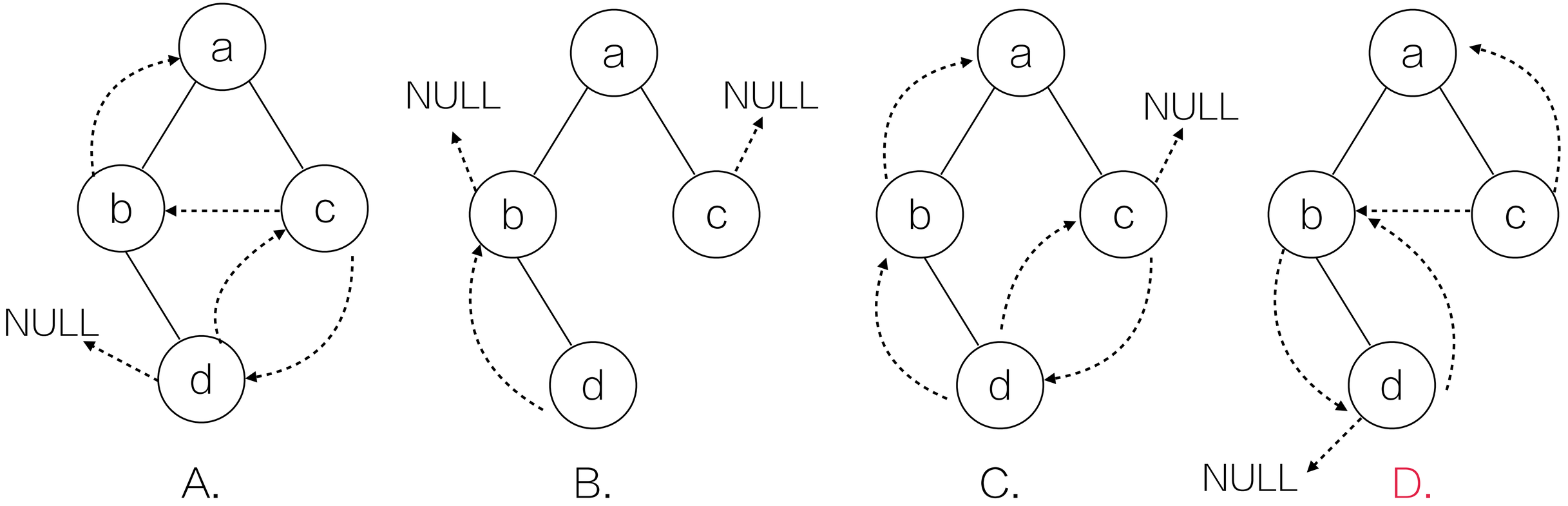
[2010] 3. 下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是_____。



二叉树 – 真题解析

d b c a

[2010] 3. 下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是_____。



二叉树 – 真题解析

[2013] 5. 若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是_____。

A. X 的父结点

B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟结点 Y

D. 以 Y 为根的子树的最右下结点

二叉树 – 真题解析

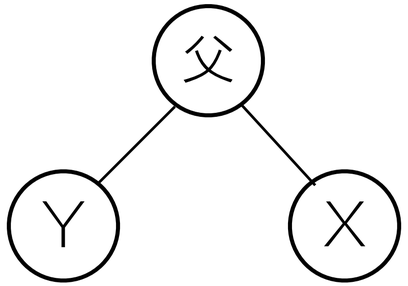
[2013] 5. 若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是_____。

A. X 的父结点

B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟结点 Y

D. 以 Y 为根的子树的最右下结点



二叉树 – 真题解析

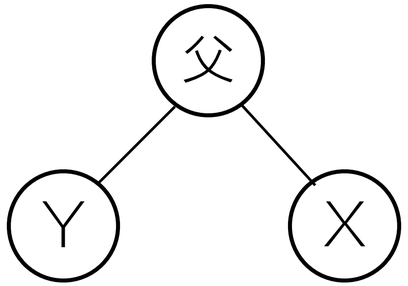
[2013] 5. 若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是_____。

A. X 的父结点

B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟结点 Y

D. 以 Y 为根的子树的最右下结点

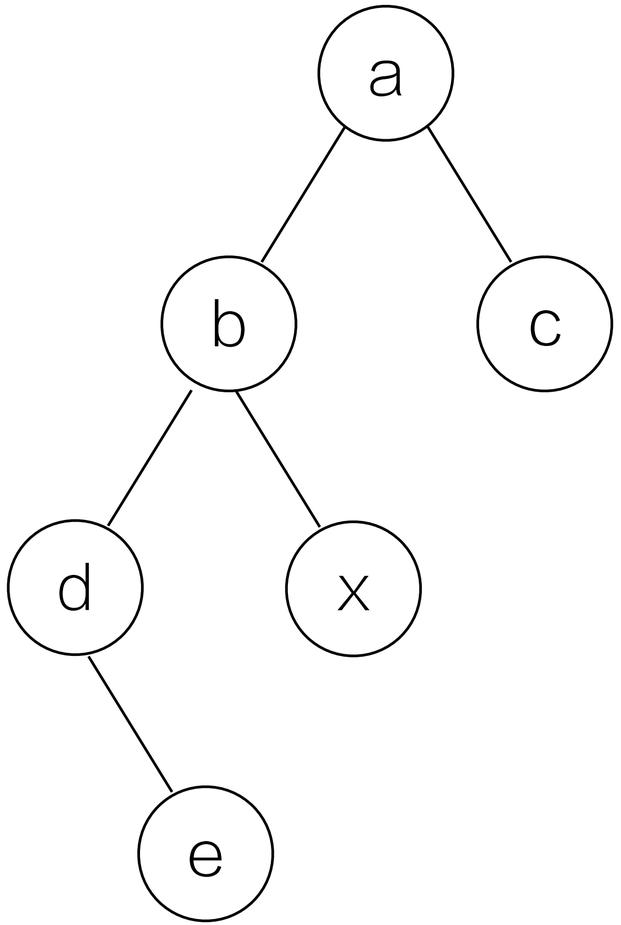


二叉树 – 真题解析

[2013] 5. 若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是_____。

A. e、c　　B. e、a

C. d、c　　D. b、a

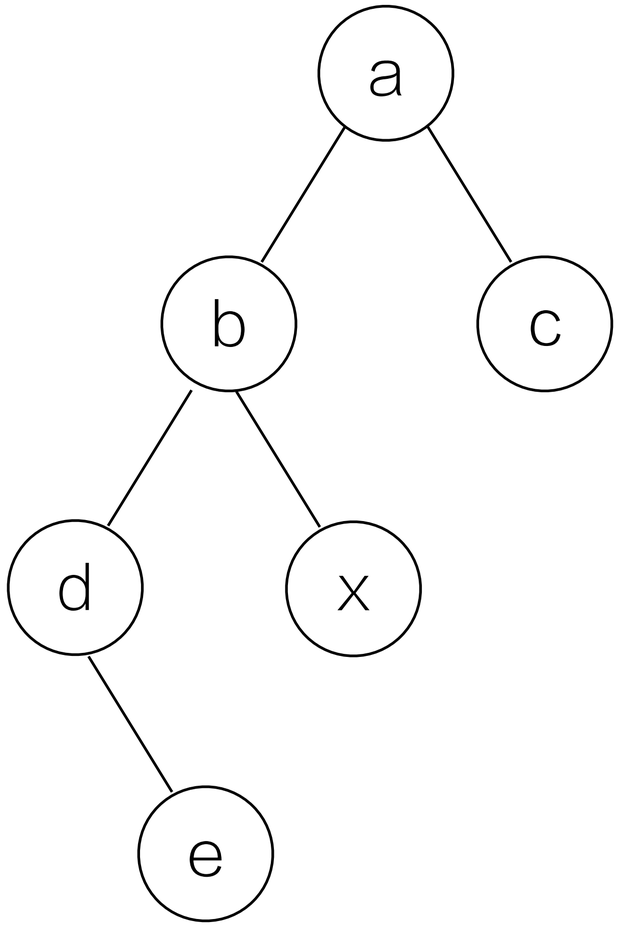


二叉树 – 真题解析

[2013] 5. 若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是_____。

A. e、c　　B. e、a

C. d、c　　D. b、a



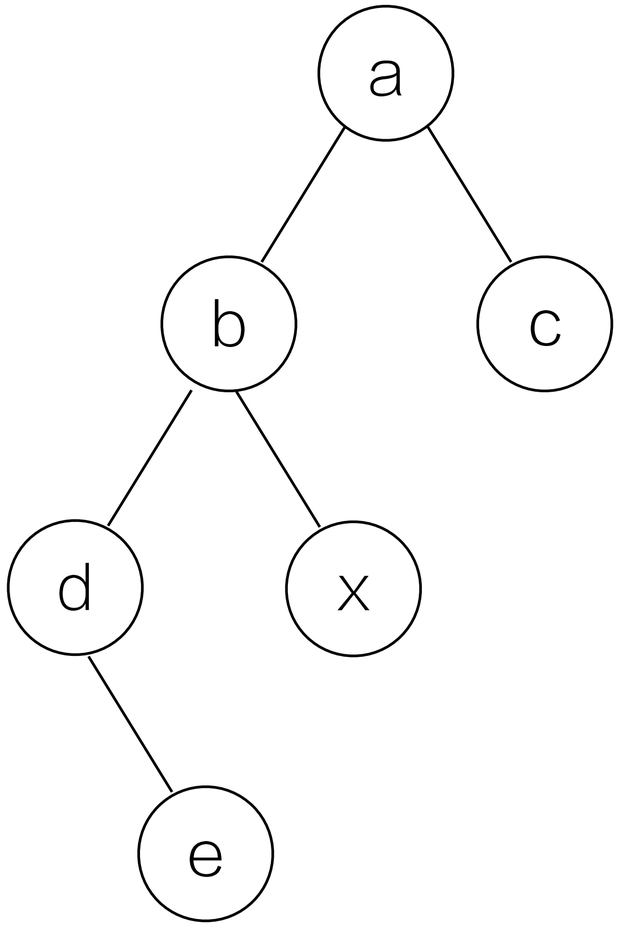
d e b x a c

二叉树 – 真题解析

[2013] 5. 若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是_____。

A. e、c　　B. e、a

C. d、c　　D. b、a



d e b x a c

哈夫曼树 - 引入

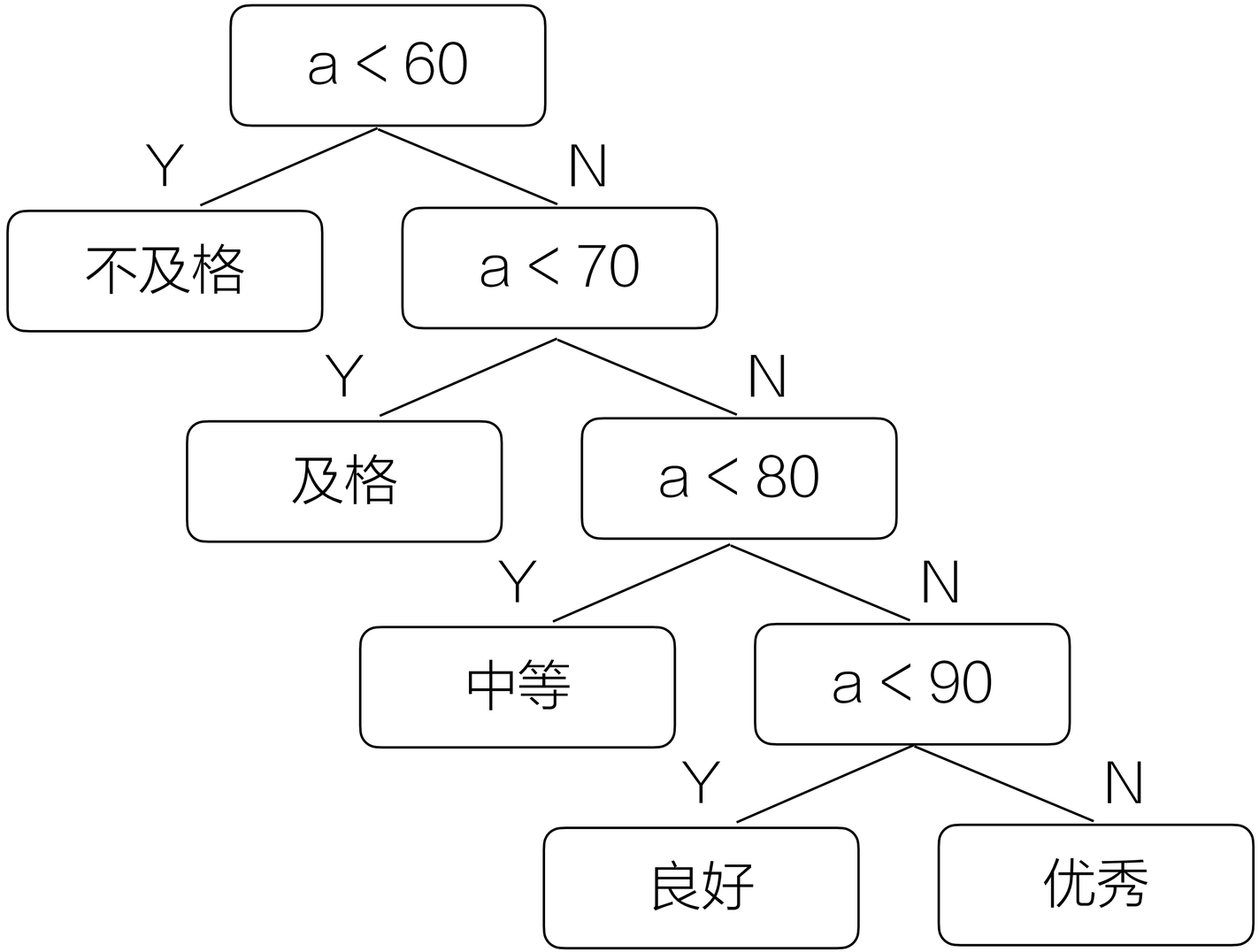
分数	0 - 59	60 - 69	70 - 79	80 - 89	90 - 100
类别	不及格	及格	中等	良好	优秀
所占比例	5%	15%	40%	30%	10%

哈夫曼树 – 引入

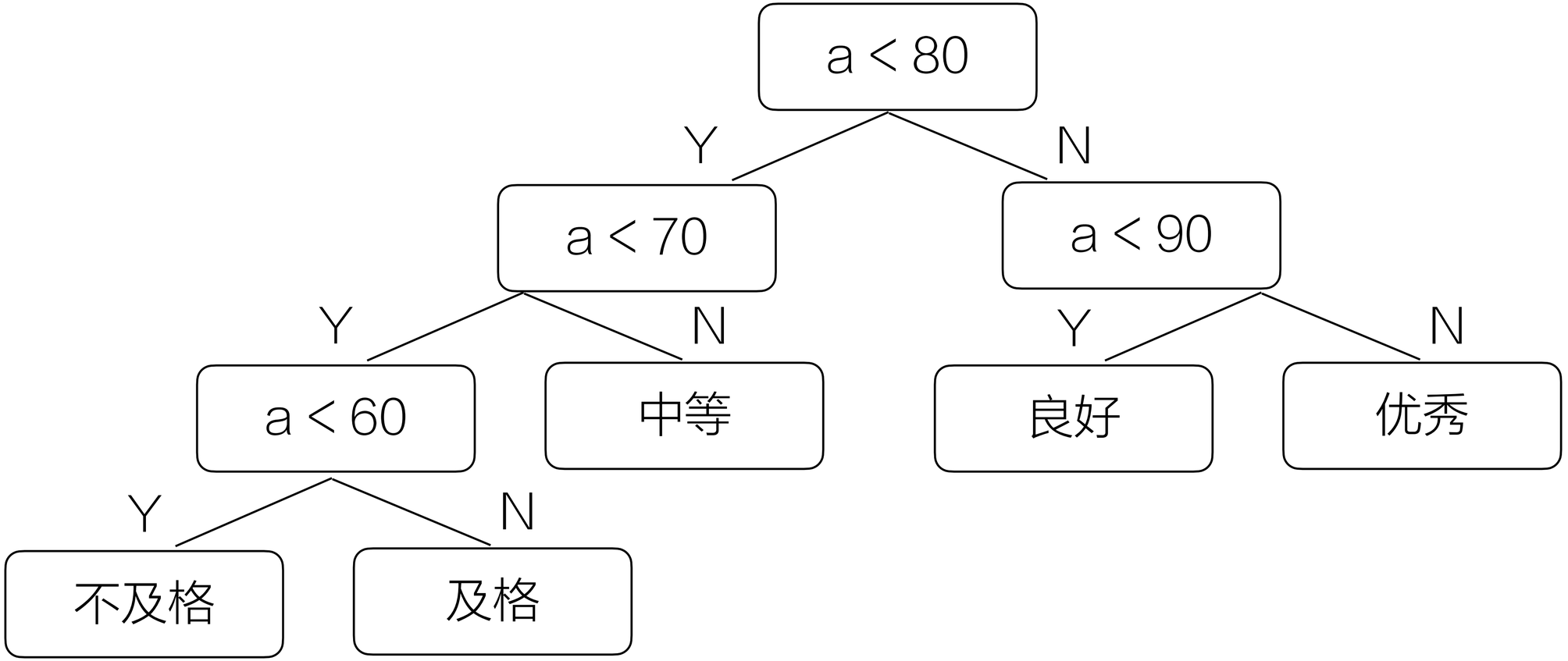
```
if(a < 60)
    b = “不及格”;
else if(a < 70)
    b = “及格”;
else if(a < 80)
    b = “中等”;
else if(a < 90)
    b = “良好”;
else
    b = “优秀”;
```

分数	0 – 59	60 – 69	70 – 79	80 – 89	90 – 100
类别	不及格	及格	中等	良好	优秀
所占比例	5%	15%	40%	30%	10%

哈夫曼树 – 引入



哈夫曼树 – 引入

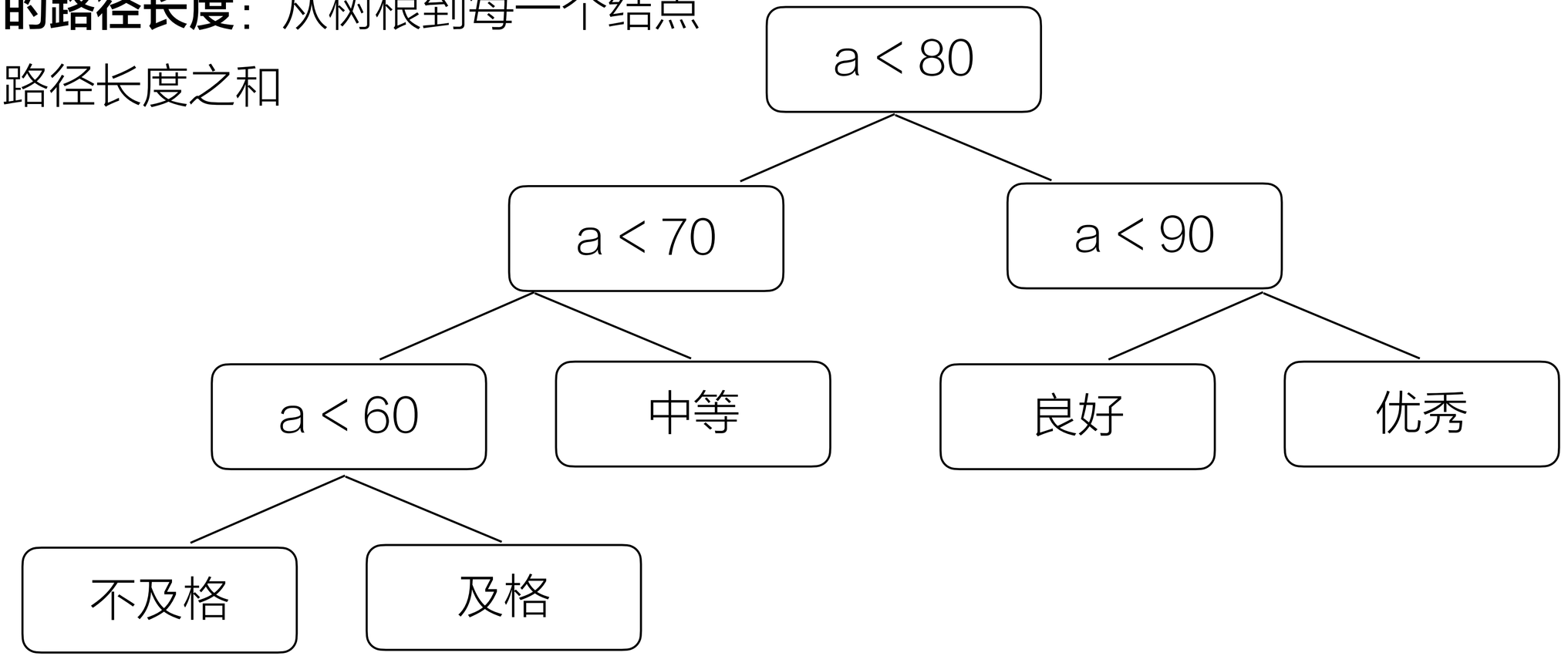


哈夫曼树 – 基本概念

路径：从树中一个结点到另一个结点之间的分支构成这两个结点之间的路径

路径长度：路径上的分支数目

树的路径长度：从树根到每一个结点的路径长度之和

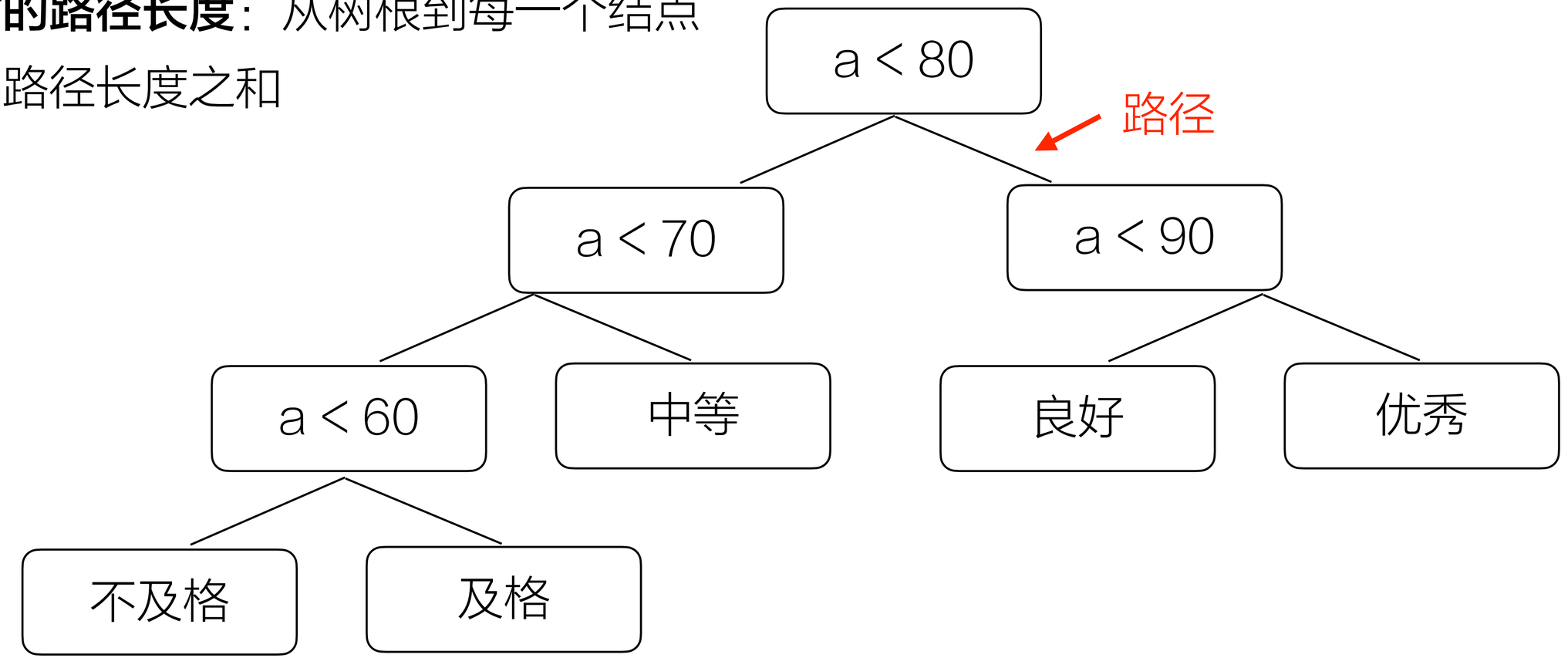


哈夫曼树 – 基本概念

路径：从树中一个结点到另一个结点之间的分支构成这两个结点之间的路径

路径长度：路径上的分支数目

树的路径长度：从树根到每一个结点的路径长度之和

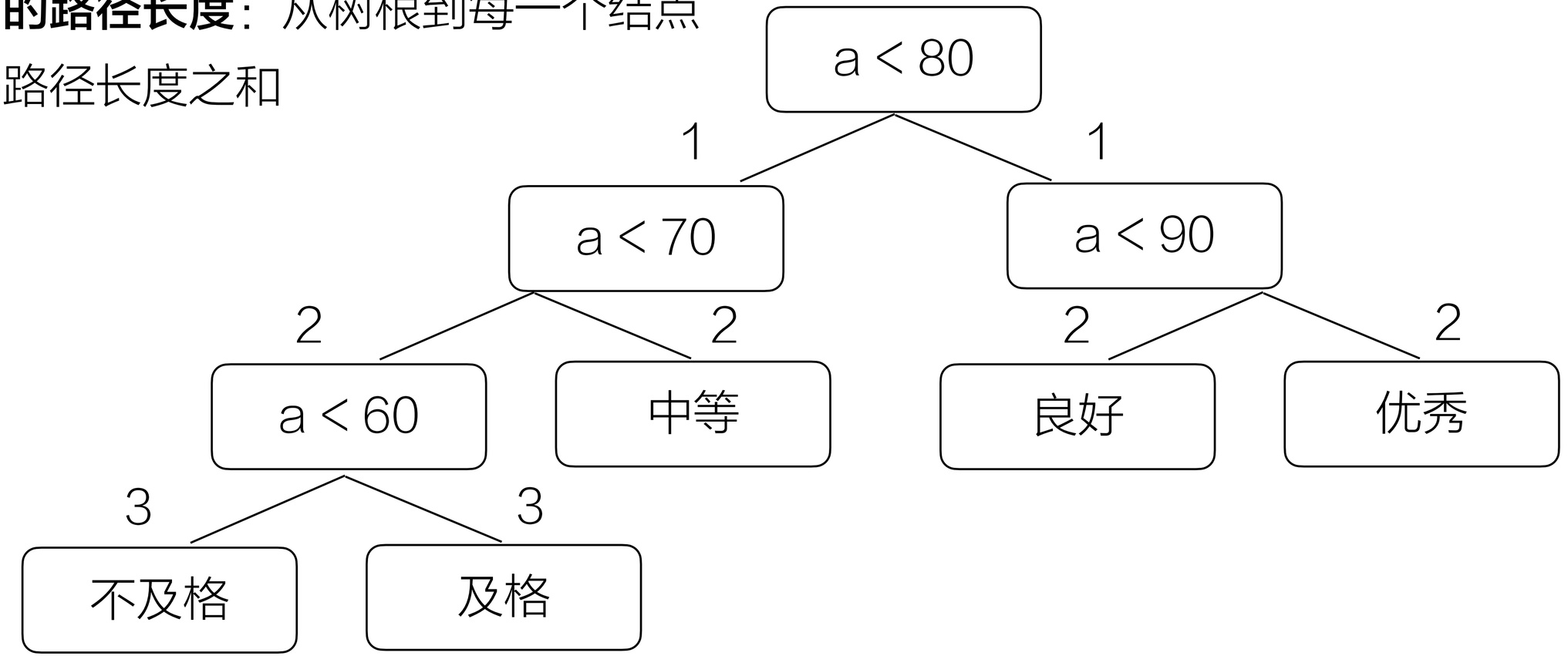


哈夫曼树 – 基本概念

路径：从树中一个结点到另一个结点之间的分支构成这两个结点之间的路径

路径长度：路径上的分支数目

树的路径长度：从树根到每一个结点的路径长度之和

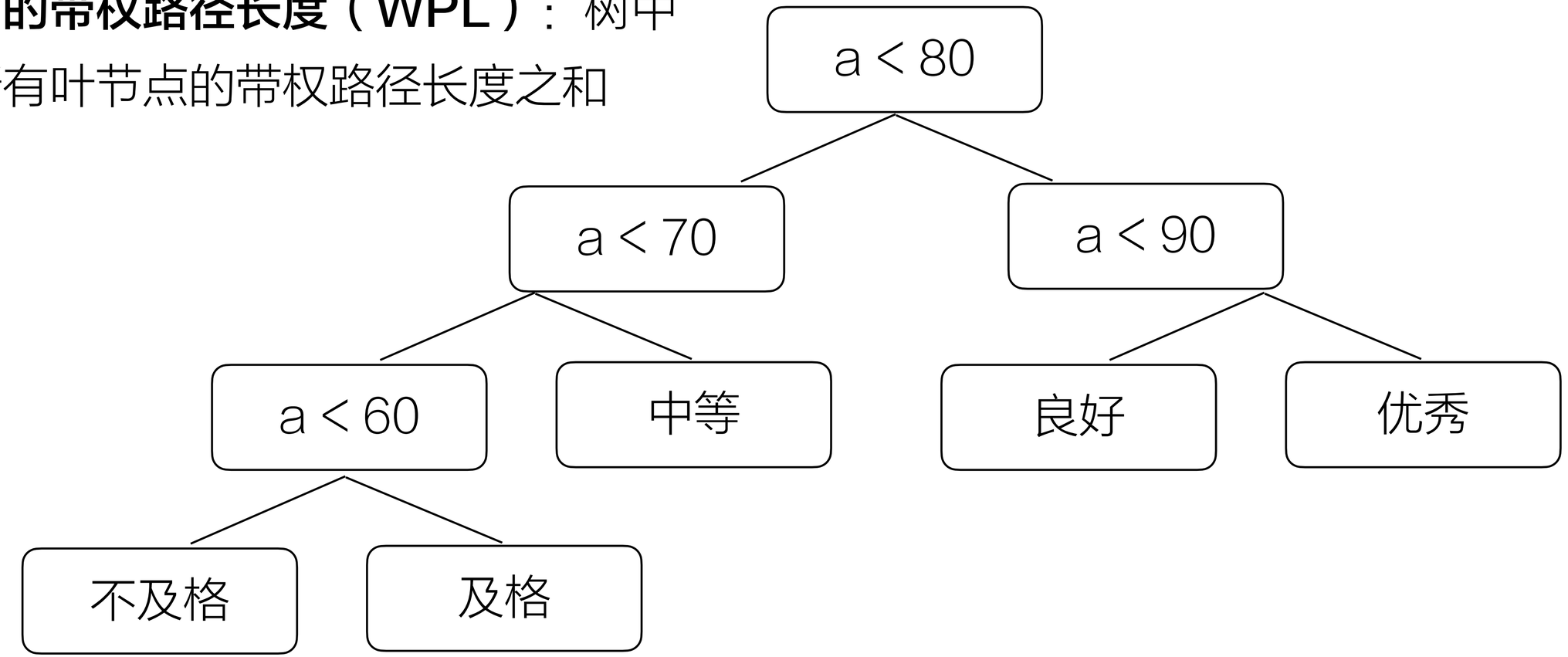


哈夫曼树 – 基本概念

结点的权：在实际应用中，给树中的结点赋予代表某种含义的数值

结点的带权路径长度：从该节点到树根之间的路径长度与该结点权的乘积

树的带权路径长度（WPL）：树中所有叶节点的带权路径长度之和

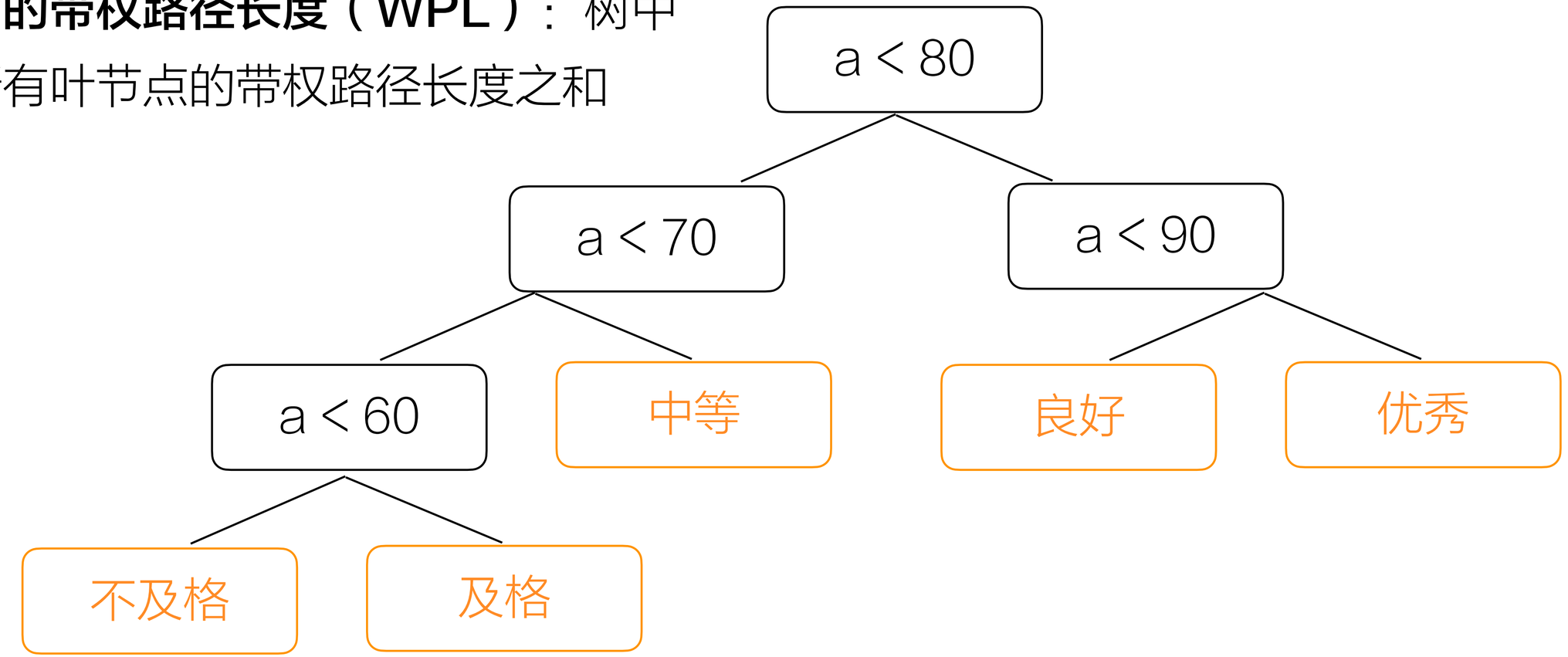


哈夫曼树 – 基本概念

结点的权：在实际应用中，给树中的结点赋予代表某种含义的数值

结点的带权路径长度：从该节点到树根之间的路径长度与该结点权的乘积

树的带权路径长度（WPL）：树中所有叶节点的带权路径长度之和

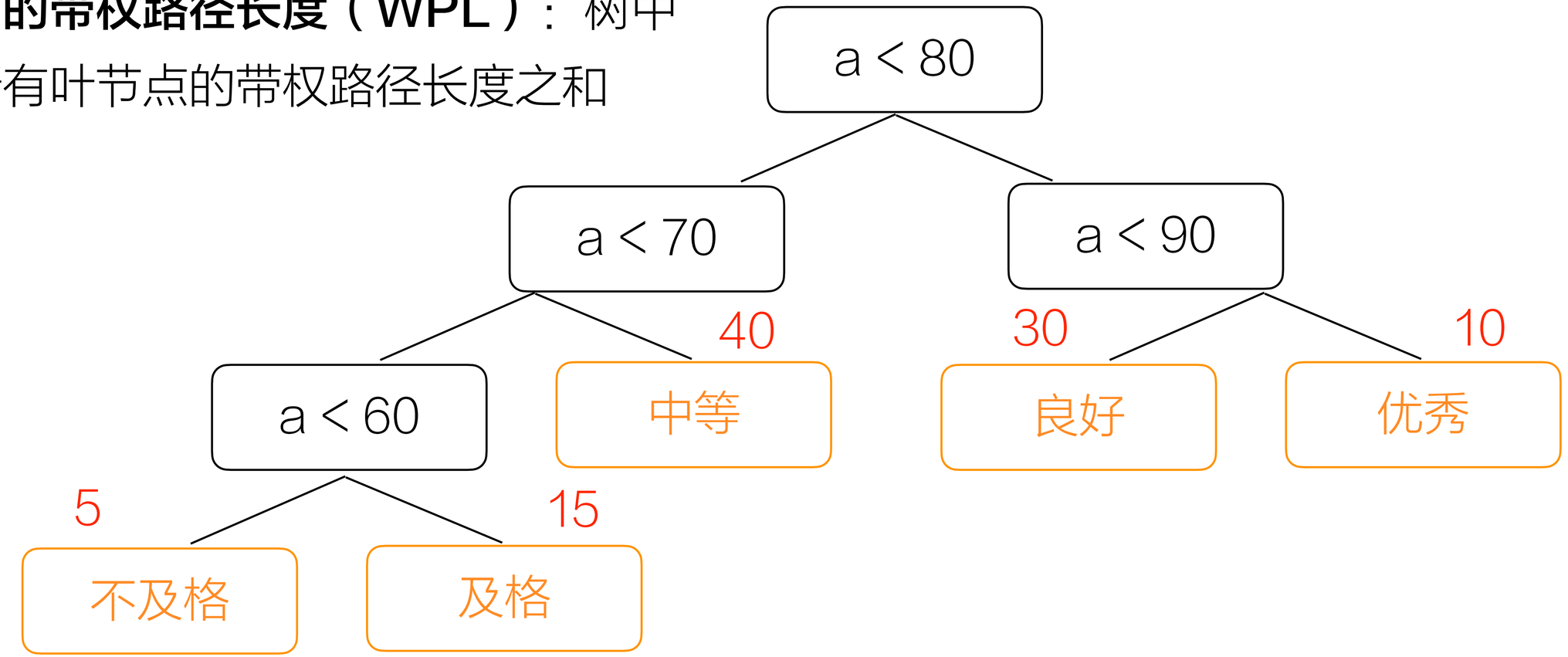


哈夫曼树 – 基本概念

结点的权：在实际应用中，给树中的结点赋予代表某种含义的数值

结点的带权路径长度：从该节点到树根之间的路径长度与该结点权的乘积

树的带权路径长度（WPL）：树中所有叶节点的带权路径长度之和

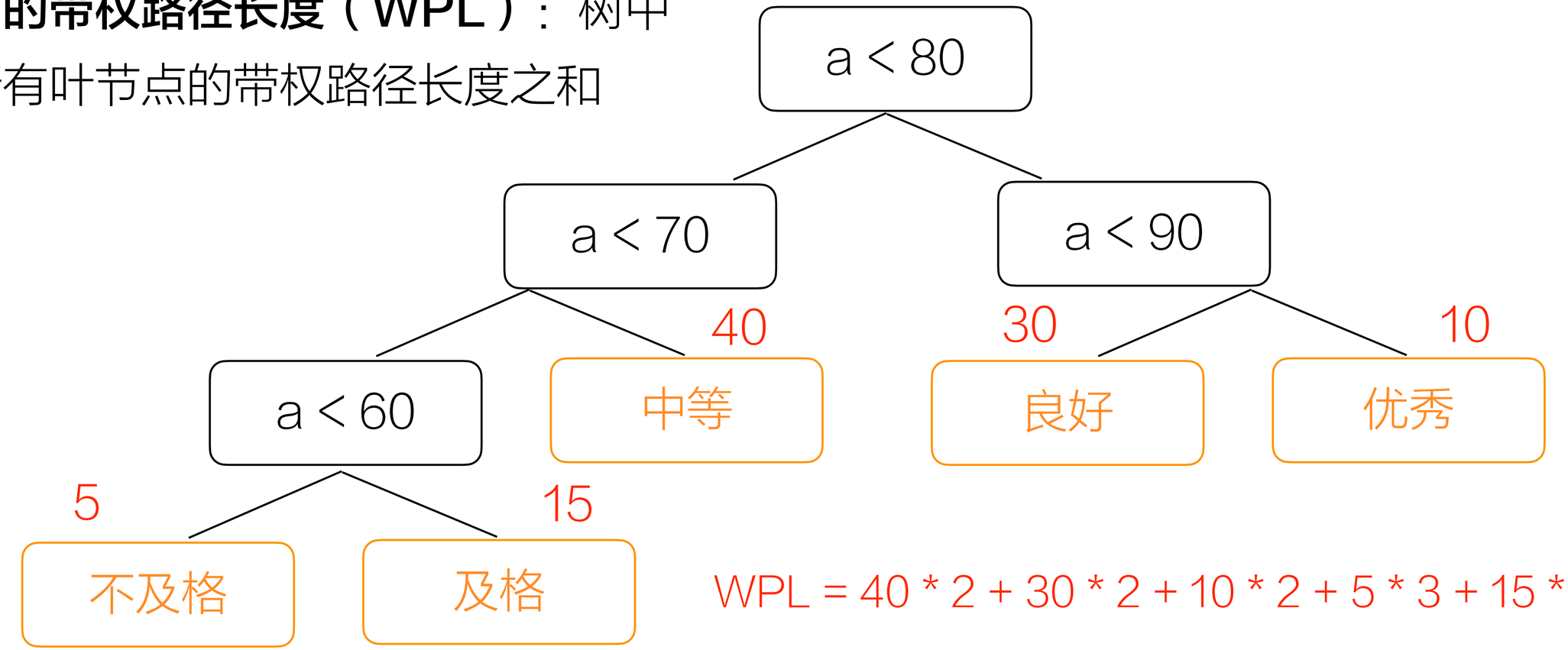


哈夫曼树 – 基本概念

结点的权：在实际应用中，给树中的结点赋予代表某种含义的数值

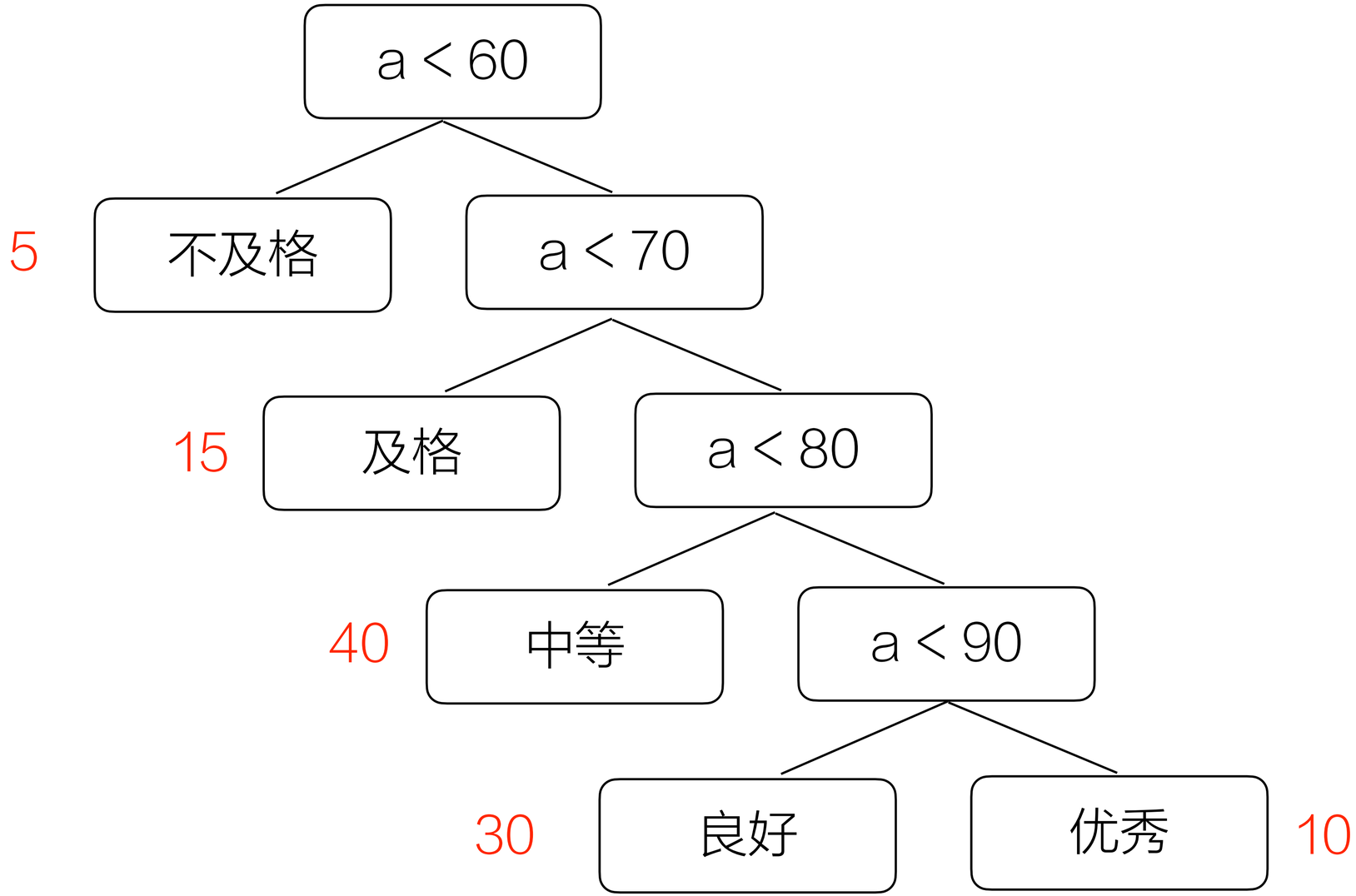
结点的带权路径长度：从该节点到树根之间的路径长度与该结点权的乘积

树的带权路径长度（WPL）：树中所有叶节点的带权路径长度之和

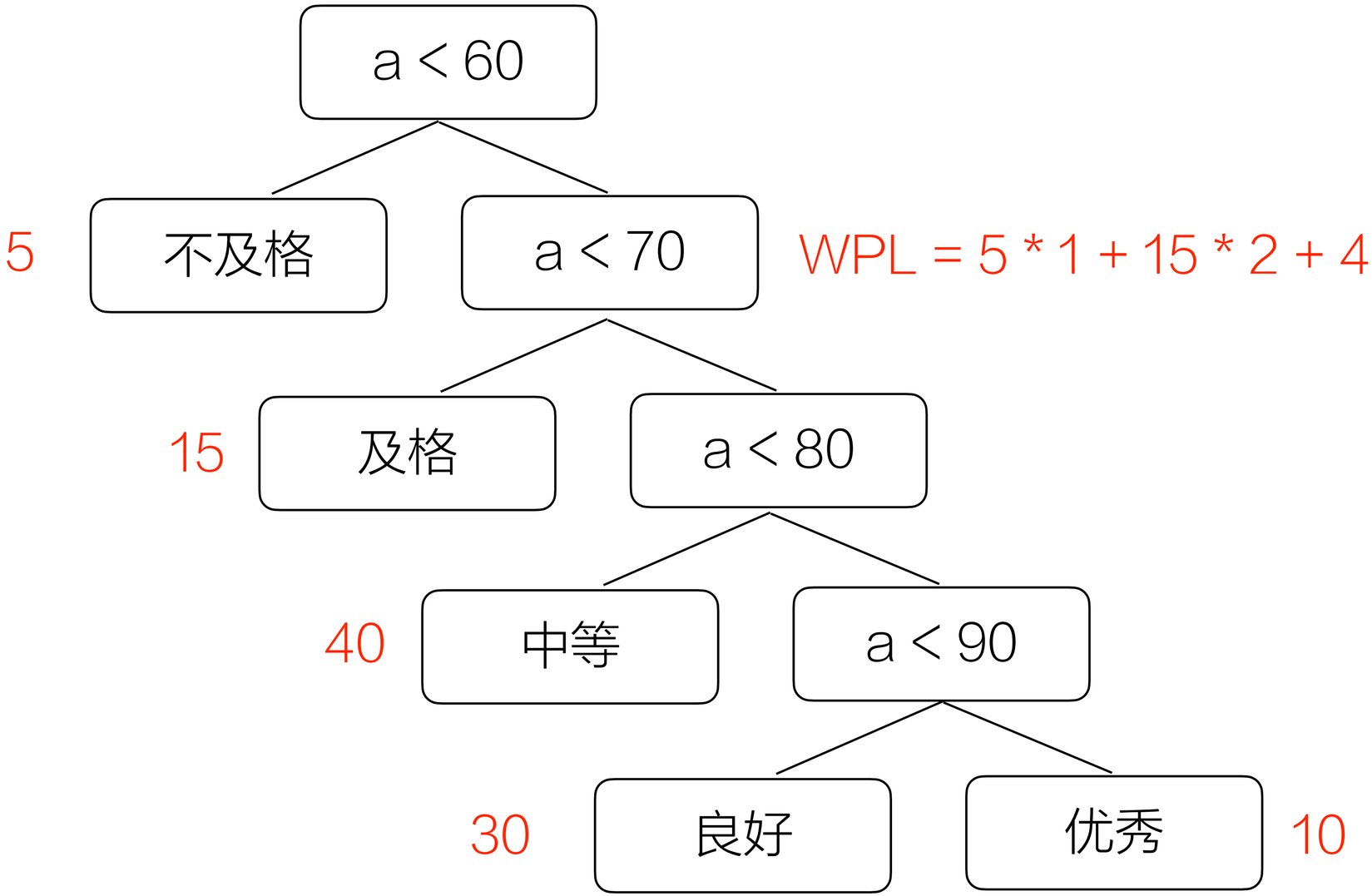


WPL = 40 * 2 + 30 * 2 + 10 * 2 + 5 * 3 + 15 * 3 = 220

哈夫曼树 – 计算 WPL



哈夫曼树 – 计算 WPL



WPL = 5 * 1 + 15 * 2 + 40 * 3 + 30 * 4 + 10 * 4 = 315

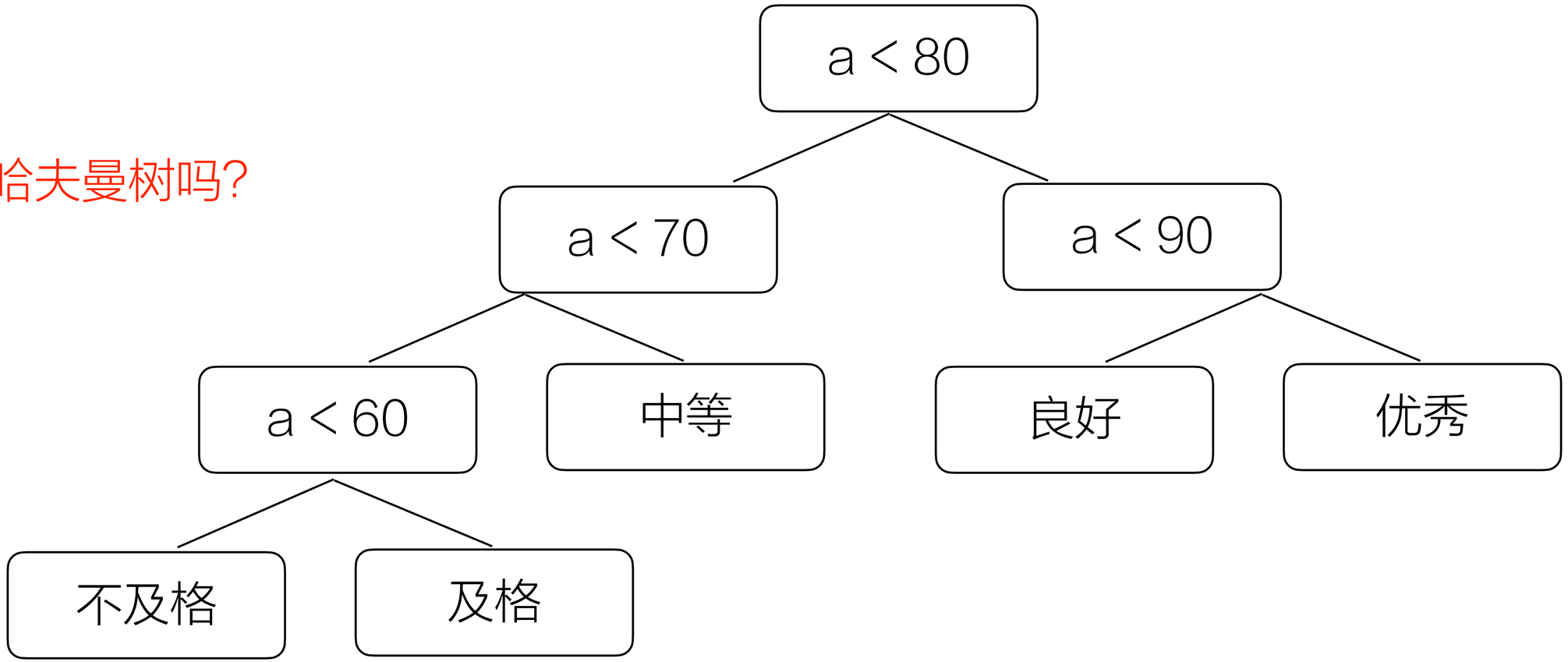
哈夫曼树 – 定义

带权路径长度 WPL 最小的二叉树称为哈夫曼树。

哈夫曼树 – 定义

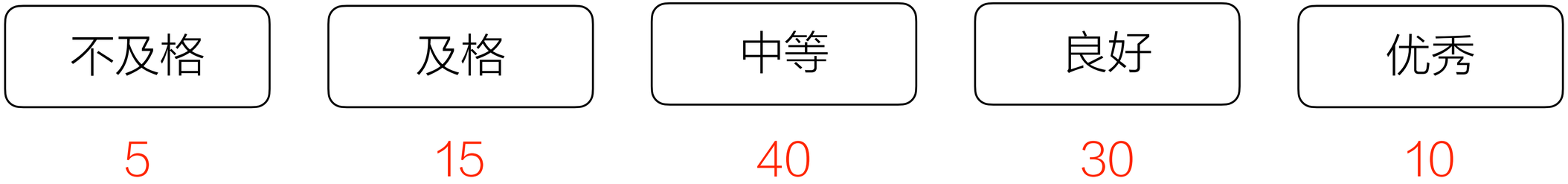
带权路径长度 WPL 最小的二叉树称为哈夫曼树。

这是哈夫曼树吗?



构造哈夫曼树

1. 先把有权值的叶子结点按照从小到大的顺序排列成一个有序序列



构造哈夫曼树

1. 先把有权值的叶子结点按照从小到大的顺序排列成一个有序序列



构造哈夫曼树

2. 取两个最小权值的结点作为一个新结点 N_1 的子结点



构造哈夫曼树

2. 取两个最小权值的结点作为一个新结点 N_1 的子结点



构造哈夫曼树

2. 取两个最小权值的结点作为一个新结点 N_1 的子结点



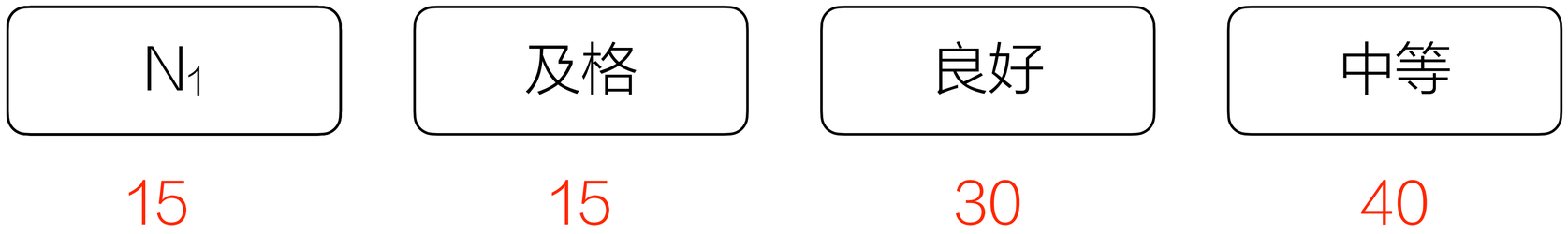
构造哈夫曼树

3. 将结点 N_1 替换刚刚取出的两个结点，并加入到有序序列中



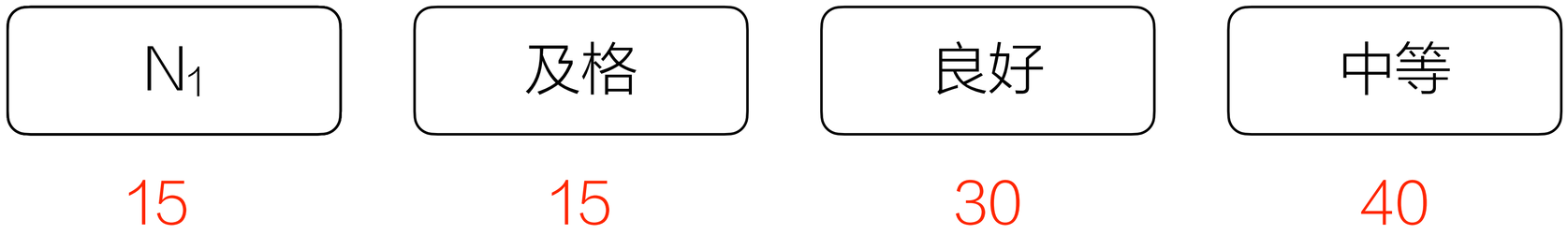
构造哈夫曼树

3. 将结点 N_1 替换刚刚取出的两个结点，并加入到有序序列中



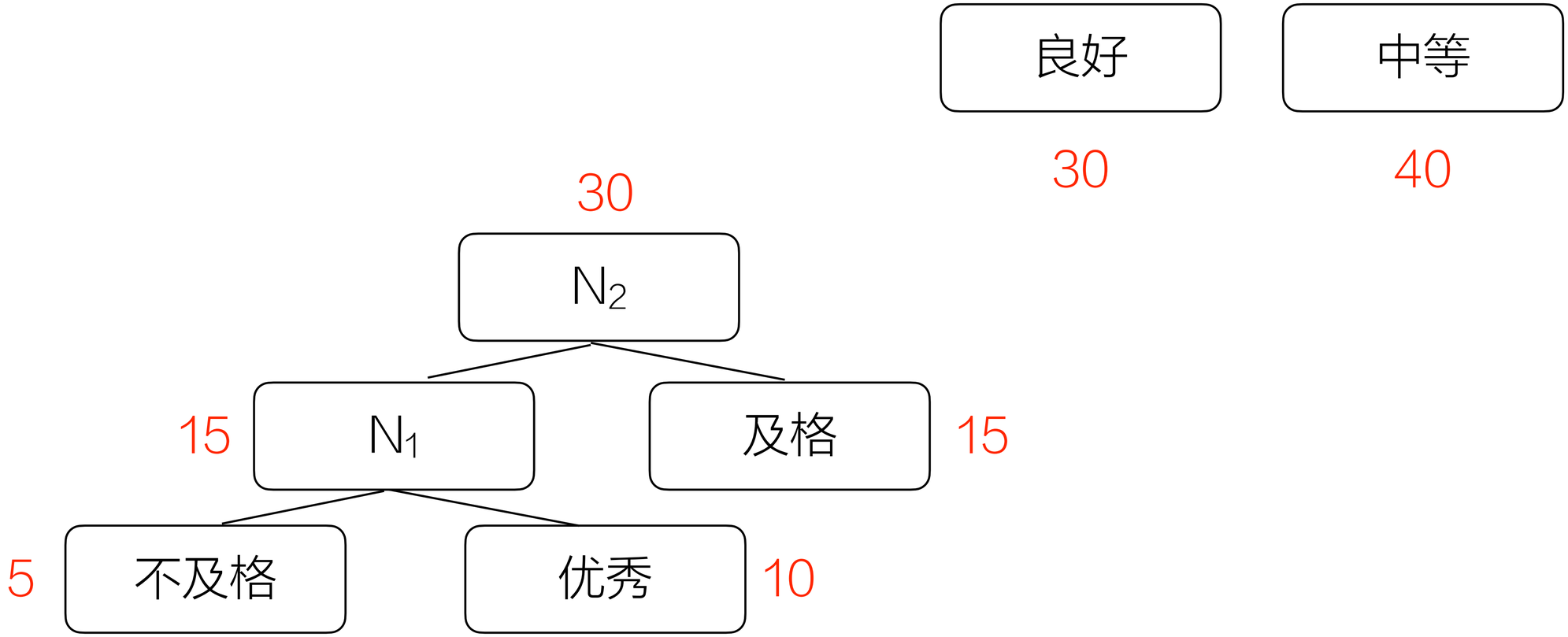
构造哈夫曼树

4. 重复步骤 2 ， 将 N_1 和 “及格”作为一个新结点 N_2 的子结点



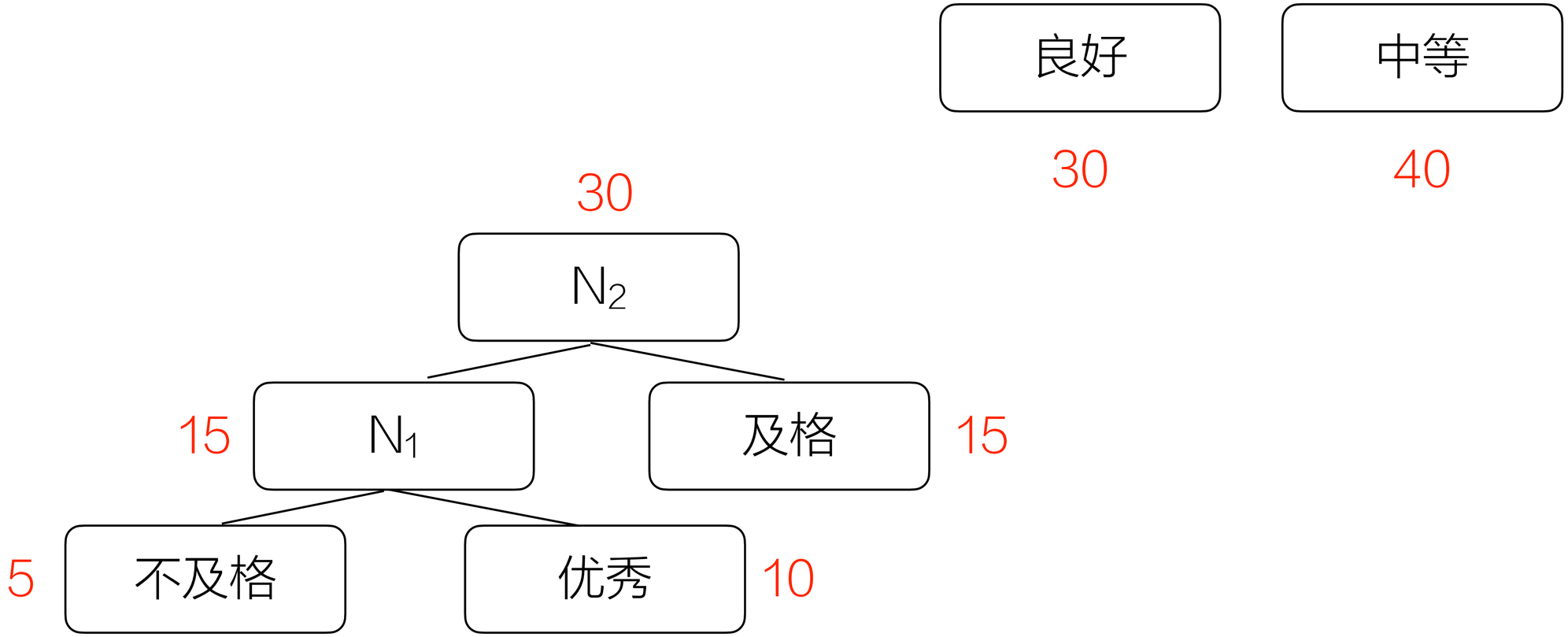
构造哈夫曼树

4. 重复步骤 2 ， 将 N_1 和 “及格”作为一个新结点 N_2 的子结点



构造哈夫曼树

5. 将结点 N_2 替换刚刚取出的两个结点，并加入到有序序列中



构造哈夫曼树

5. 将结点 N_2 替换刚刚取出的两个结点，并加入到有序序列中



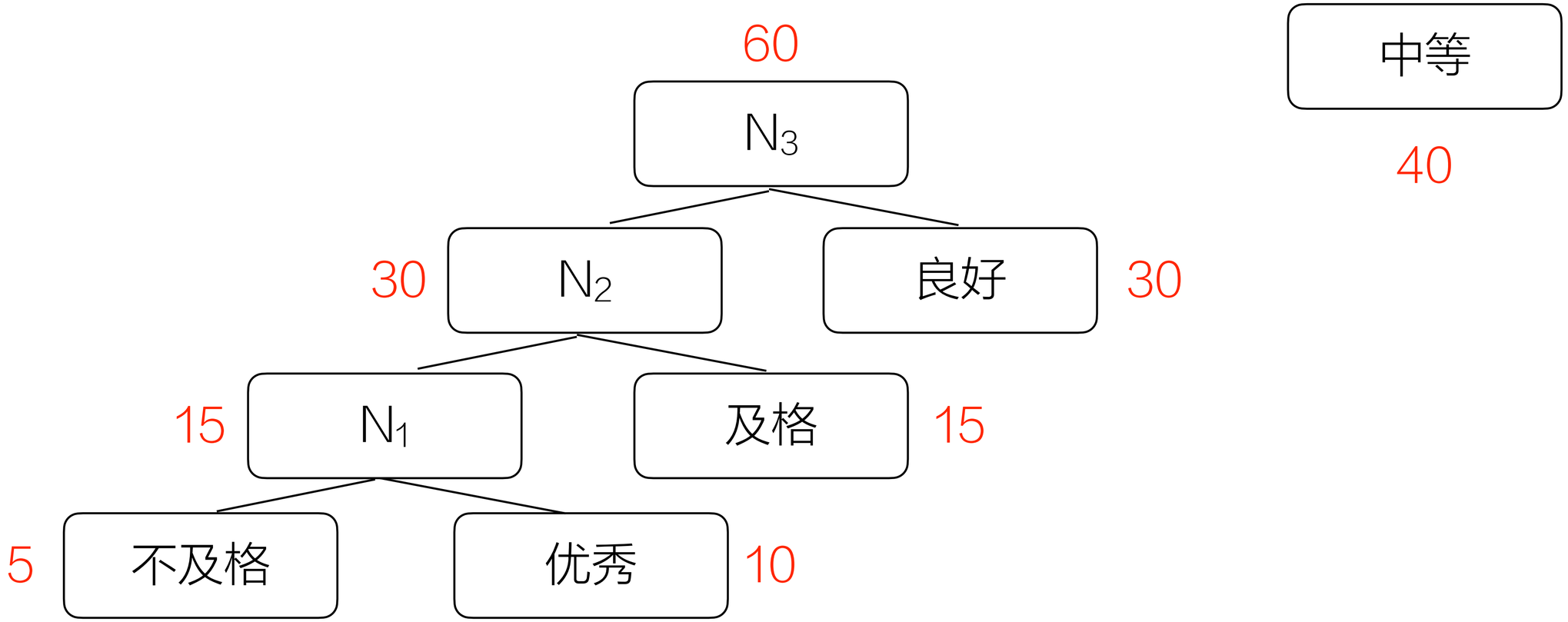
构造哈夫曼树

6. 重复步骤 2 ， 将 N_2 和 “良好”作为一个新结点 N_3 的子结点



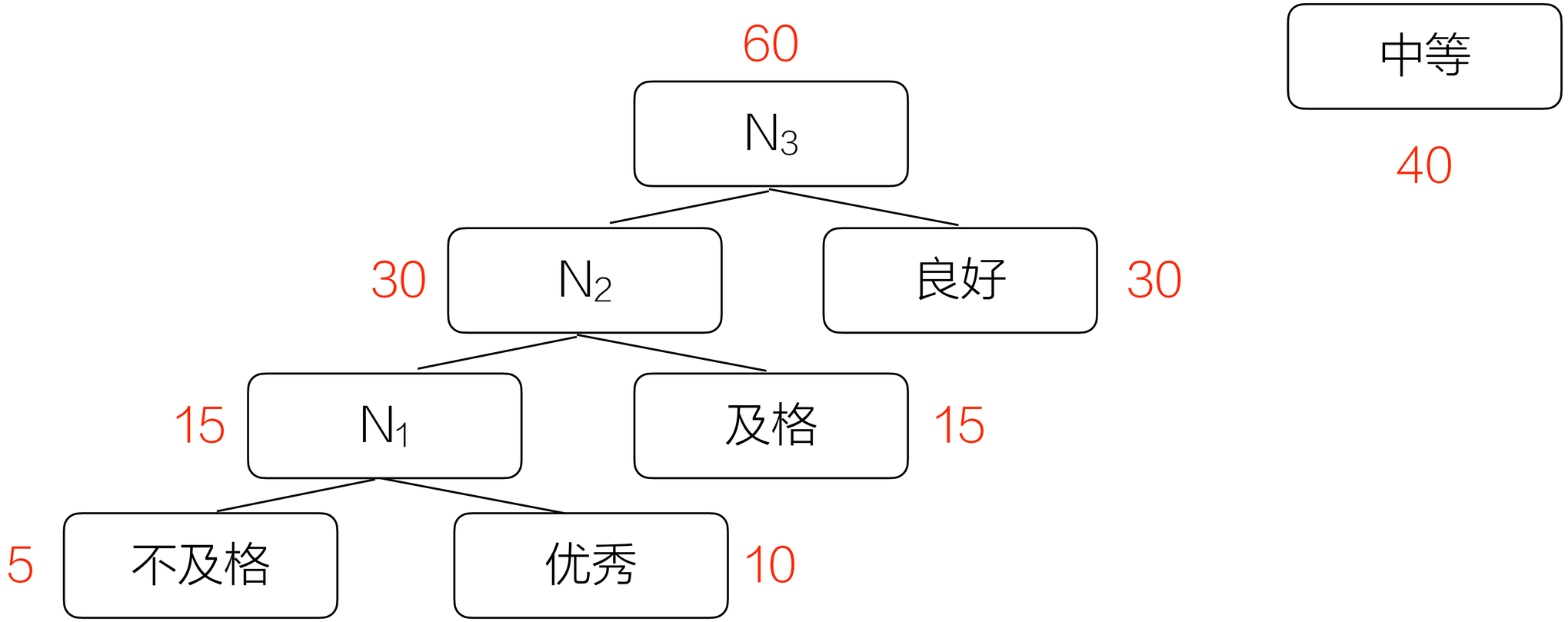
构造哈夫曼树

6. 重复步骤 2 ， 将 N_2 和 “良好”作为一个新结点 N_3 的子结点



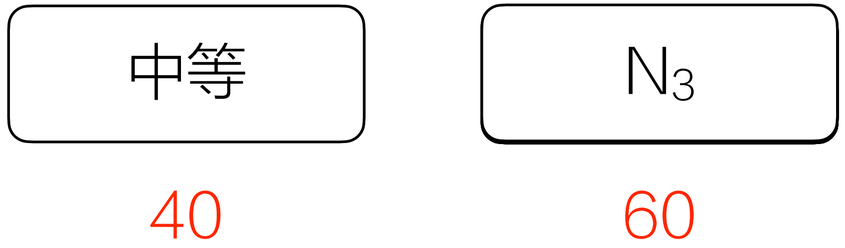
构造哈夫曼树

7. 将结点 N_3 替换刚刚取出的两个结点，并加入到有序序列中



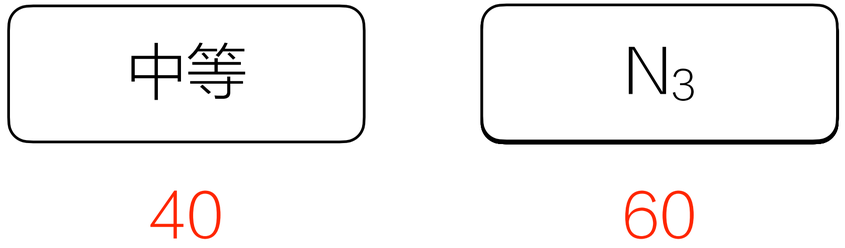
构造哈夫曼树

7. 将结点 N_3 替换刚刚取出的两个结点，并加入到有序序列中



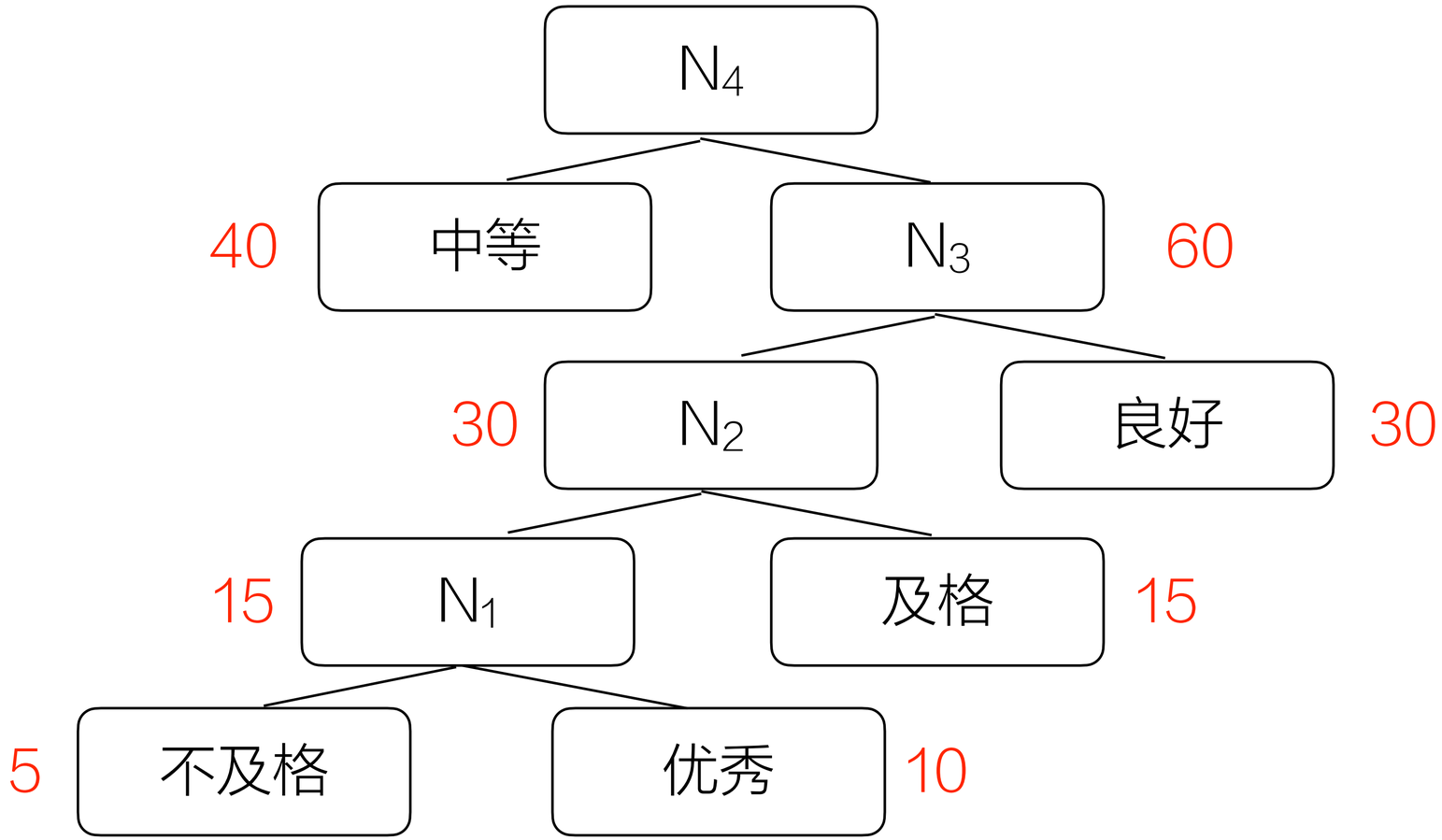
构造哈夫曼树

8. 重复步骤 2，将 N_3 和 “中等”作为一个新结点 N_4 的子结点



构造哈夫曼树

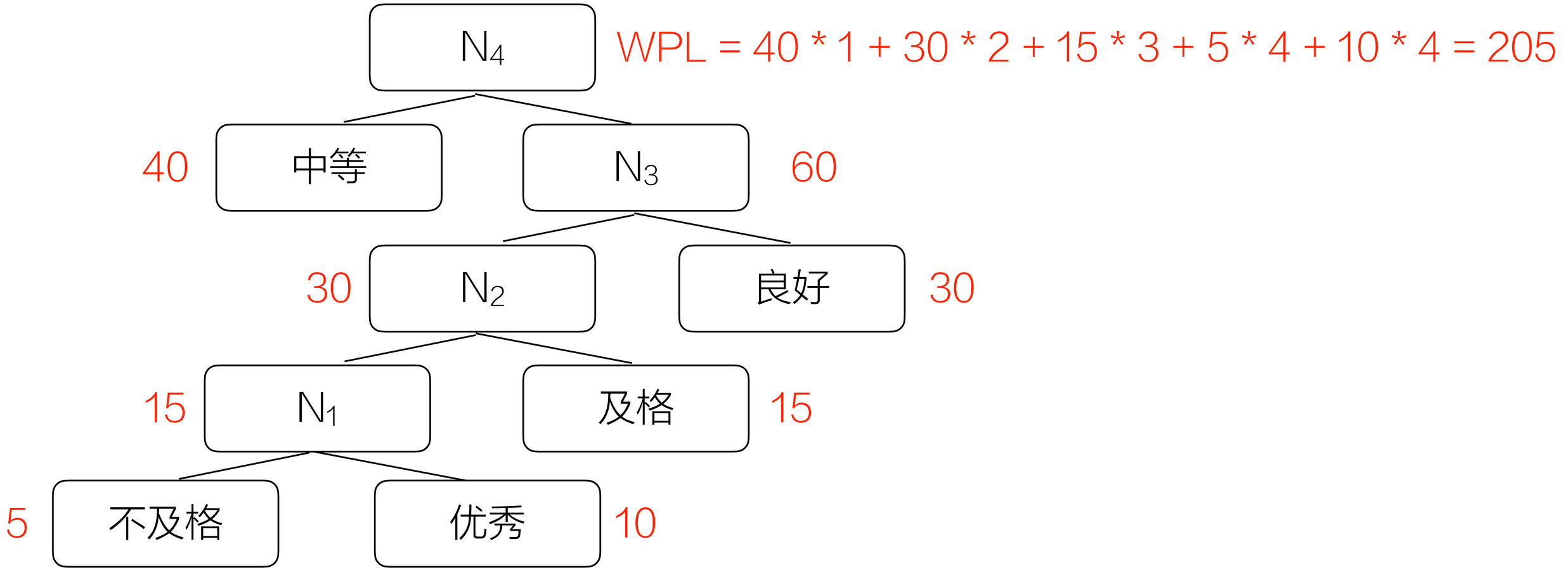
8. 重复步骤 2，将 N_3 和 “中等”作为一个新结点 N_4 的子结点



构造哈夫曼树

案例：树 / test6.c

8. 重复步骤 2 ，将 N_3 和 “中等”作为一个新结点 N_4 的子结点



哈夫曼编码

为了解决当年远距离通信的数据传输最优化问题

例：有一段字符内容“BADCADFEED”通过网络传输。

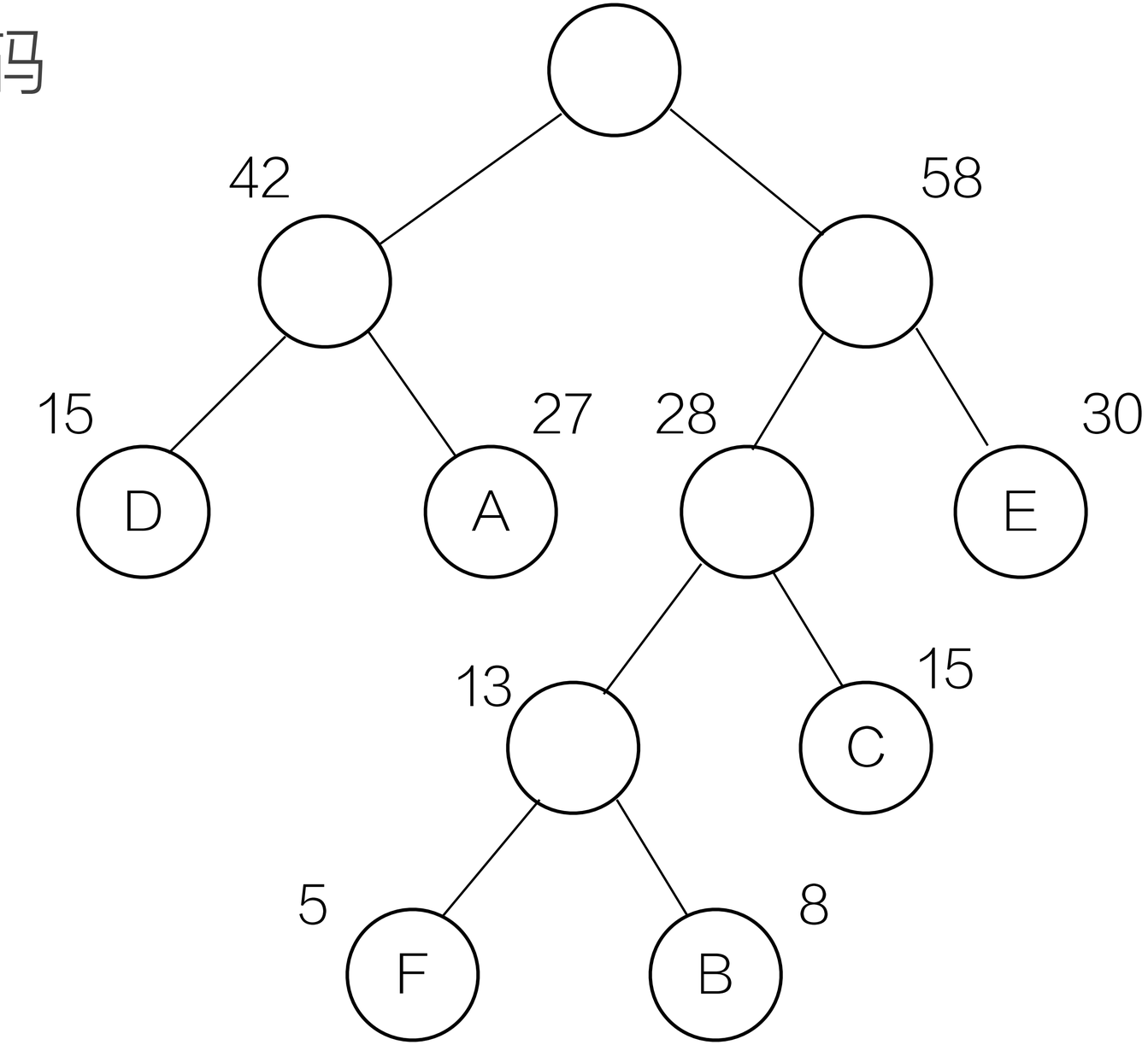
字母	A	B	C	D	E	F
二进制字符	000	001	010	011	100	101

哈夫曼编码

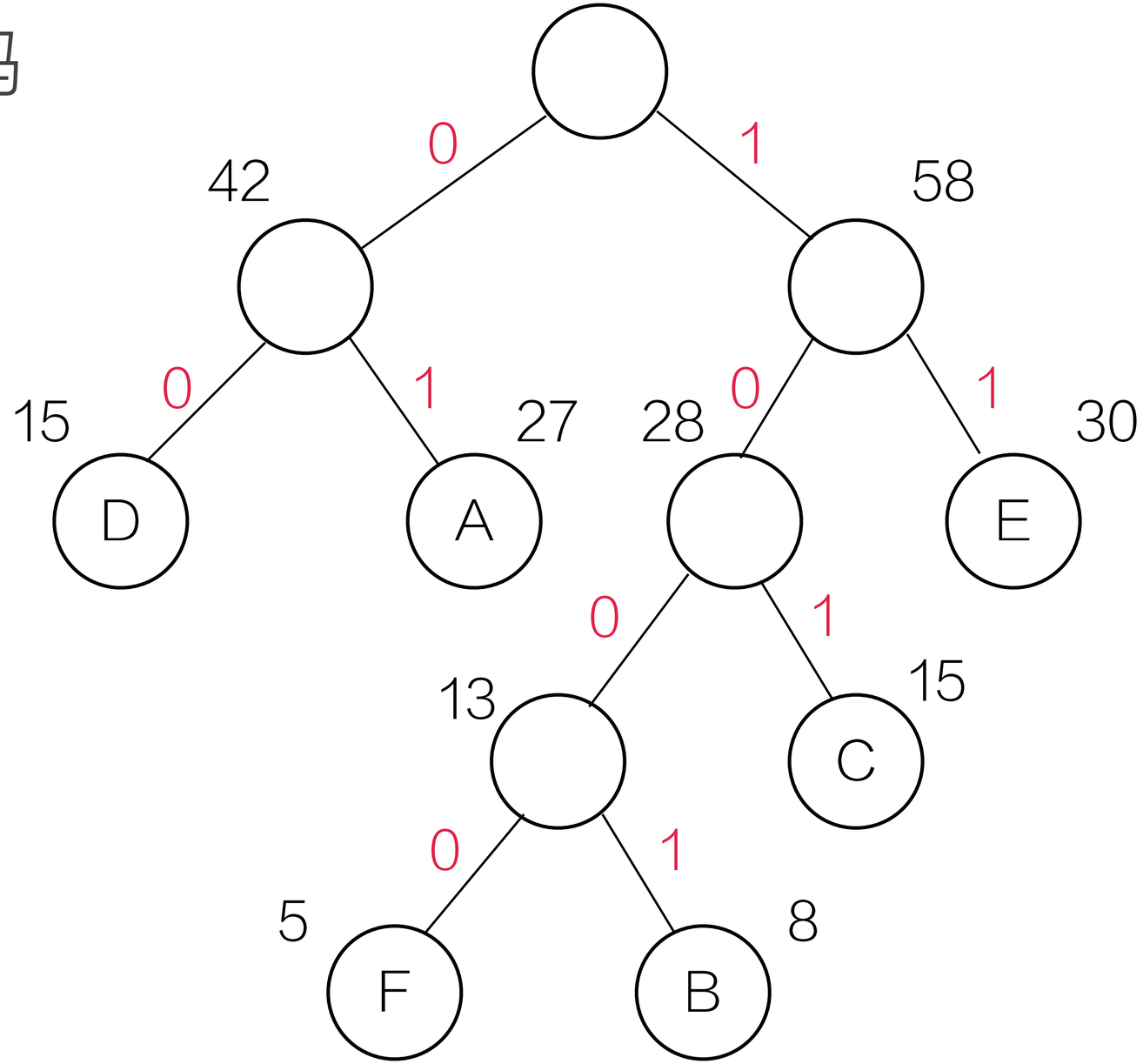
假设六个字母的使用频率为

字母	A	B	C	D	E	F
频率	27	8	15	15	30	5

哈夫曼编码



哈夫曼编码



哈夫曼编码

字母	A	B	C	D	E	F
二进制字符	000	001	010	011	100	101

字母	A	B	C	D	E	F
哈夫曼编码	01	1001	101	00	11	1000

哈夫曼编码

字母	A	B	C	D	E	F
二进制字符	000	001	010	011	100	101

字母	A	B	C	D	E	F
哈夫曼编码	01	1001	101	00	11	1000

对 BADCADFEED 进行编码

原编码二进制串：

哈夫曼编码进制串：

哈夫曼编码

字母	A	B	C	D	E	F
二进制字符	000	001	010	011	100	101

字母	A	B	C	D	E	F
哈夫曼编码	01	1001	101	00	11	1000

对 BADCADFEED 进行编码

原编码二进制串：001 000 011 010 000 011 101 100 100 011

哈夫曼编码进制串：

哈夫曼编码

字母	A	B	C	D	E	F
二进制字符	000	001	010	011	100	101

字母	A	B	C	D	E	F
哈夫曼编码	01	1001	101	00	11	1000

对 BADCADFEED 进行编码

原编码二进制串：001 000 011 010 000 011 101 100 100 011

哈夫曼编码进制串：1001 01 00 101 01 00 1000 11 11 00

哈夫曼树 – 真题解析

[2010] 6. 对 n (n≥2) 个权值均不相同的字符构造成哈夫曼树。下列关于该哈夫曼树的叙述中，错误的是_____。

A. 该树一定是一棵完全二叉树

B. 树中一定没有度为 1 的结点

C. 树中两个权值最小的结点一定是兄弟结点

D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值

哈夫曼树 – 真题解析

[2010] 6. 对 n (n≥2) 个权值均不相同的字符构造成哈夫曼树。下列关于快哈夫曼树的叙述中，错误的是_____。

A. 该树一定是一棵完全二叉树

B. 树中一定没有度为 1 的结点

C. 树中两个权值最小的结点一定是兄弟结点

D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值

哈夫曼树 – 真题解析

[2014] 6. 5个字符有如下4种编码方案，不是前缀编码的是_____。

A. 01, 0000, 0001, 001, 1

B. 011, 000, 001, 010, 1

C. 000, 001, 010, 011, 100

D. 0, 100, 110, 1110, 1100

哈夫曼树 – 真题解析

[2014] 6. 5个字符有如下4种编码方案，不是前缀编码的是_____。

A. 01, 0000, 0001, 001, 1

B. 011, 000, 001, 010, 1

C. 000, 001, 010, 011, 100

D. 0, 100, 110, 1110, 1100

哈夫曼树 – 真题解析

[2017] 6. 已知字符集 { a, b, c, d, e, f, g, h }，若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001，则编码序列 0100011001001011110101 的译码结果是_____。

A. a c g a b f h　　B. a d b a g b b　　C. a f b e a g d　　D. a f e e f g d

哈夫曼树 – 真题解析

[2017] 6. 已知字符集 { a, b, c, d, e, f, g, h }，若各字符的哈夫曼编码依次是 0100, 10, 0000, 0101, 001, 011, 11, 0001，则编码序列 0100011001001011110101 的译码结果是_____。

A. a c g a b f h　　B. a d b a g b b　　C. a f b e a g d　　D. a f e e f g d

哈夫曼树 – 真题解析

[2018] 5. 已知字符集 { a, b, c, d, e, f, } ， 若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

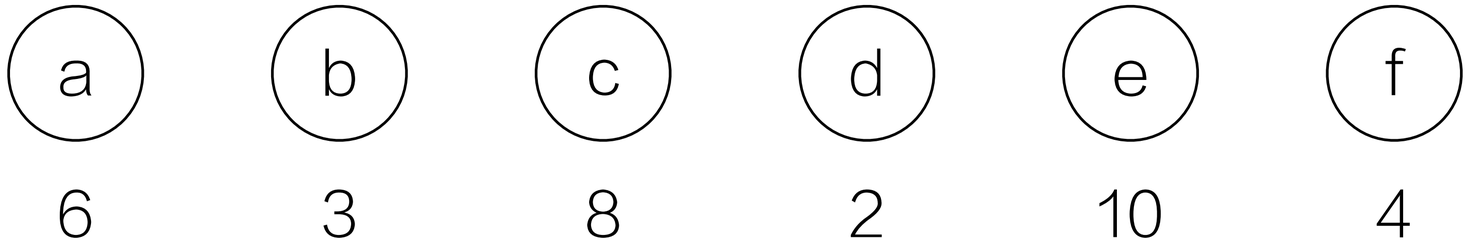
A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 000, 0010, 01

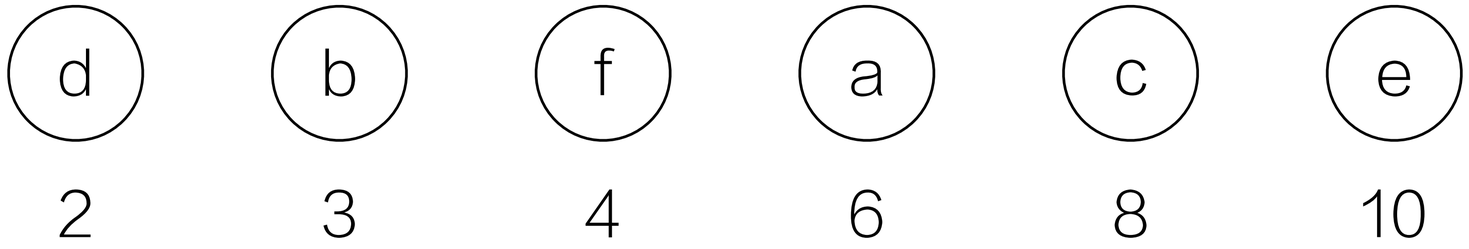
C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01, 000

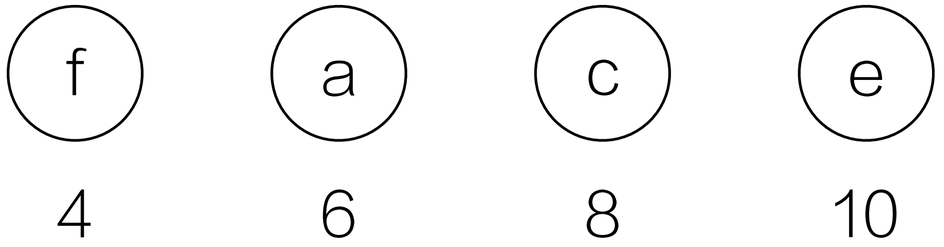
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

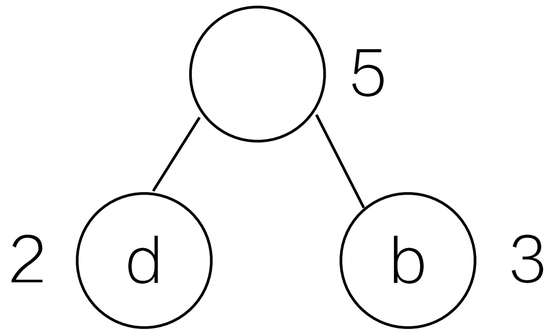


[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

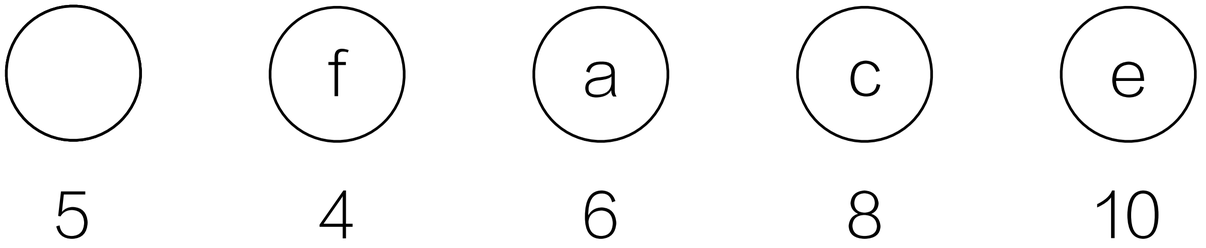


[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

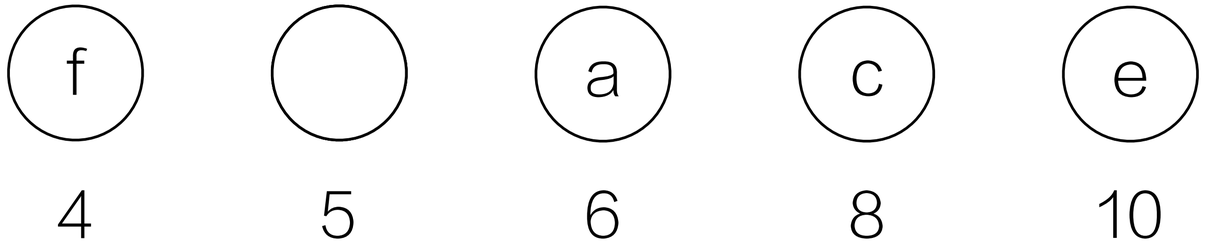




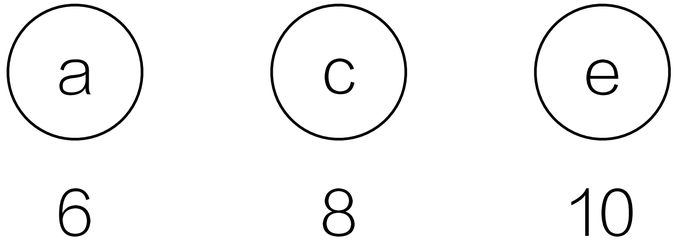
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

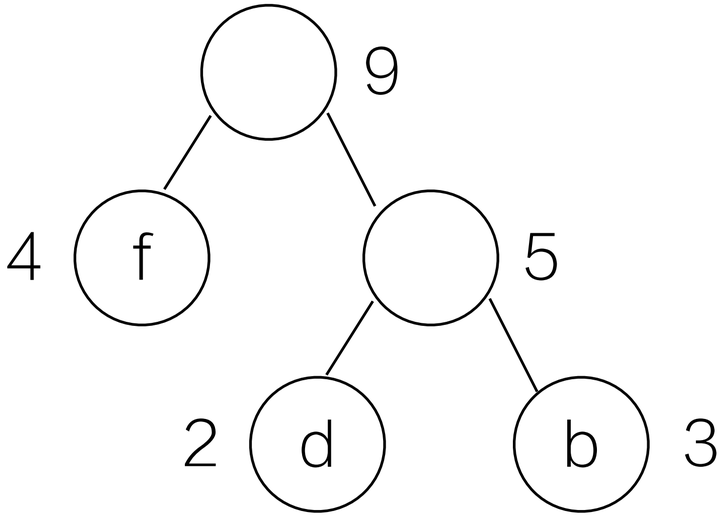


[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

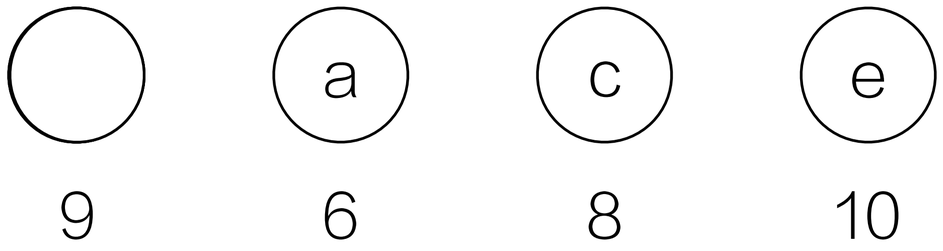


[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

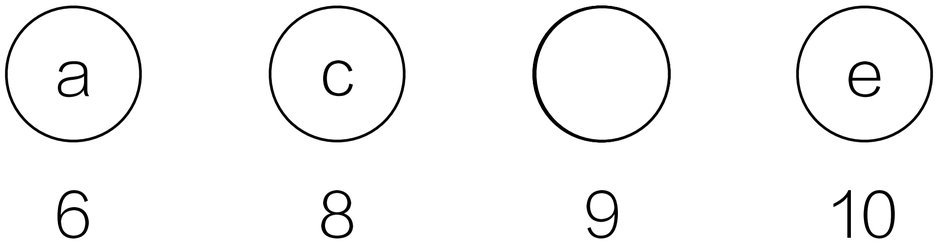




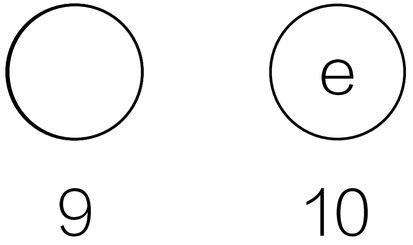
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

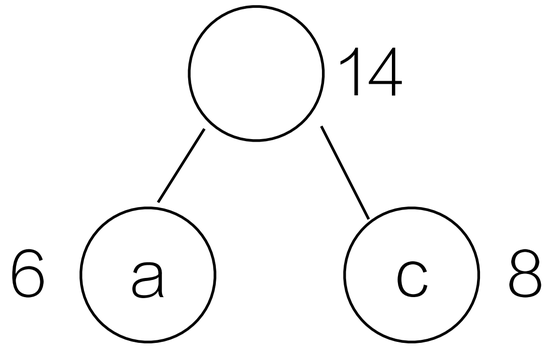


[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

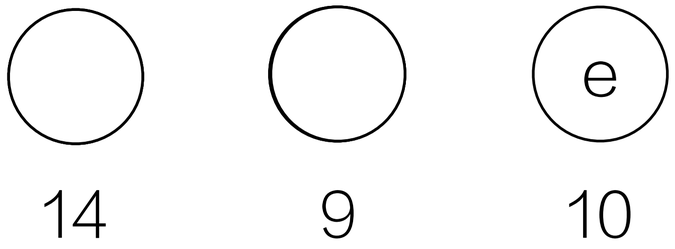


[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

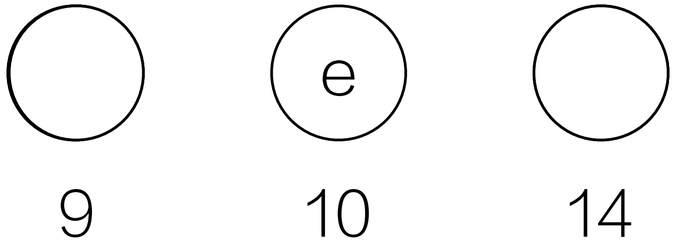




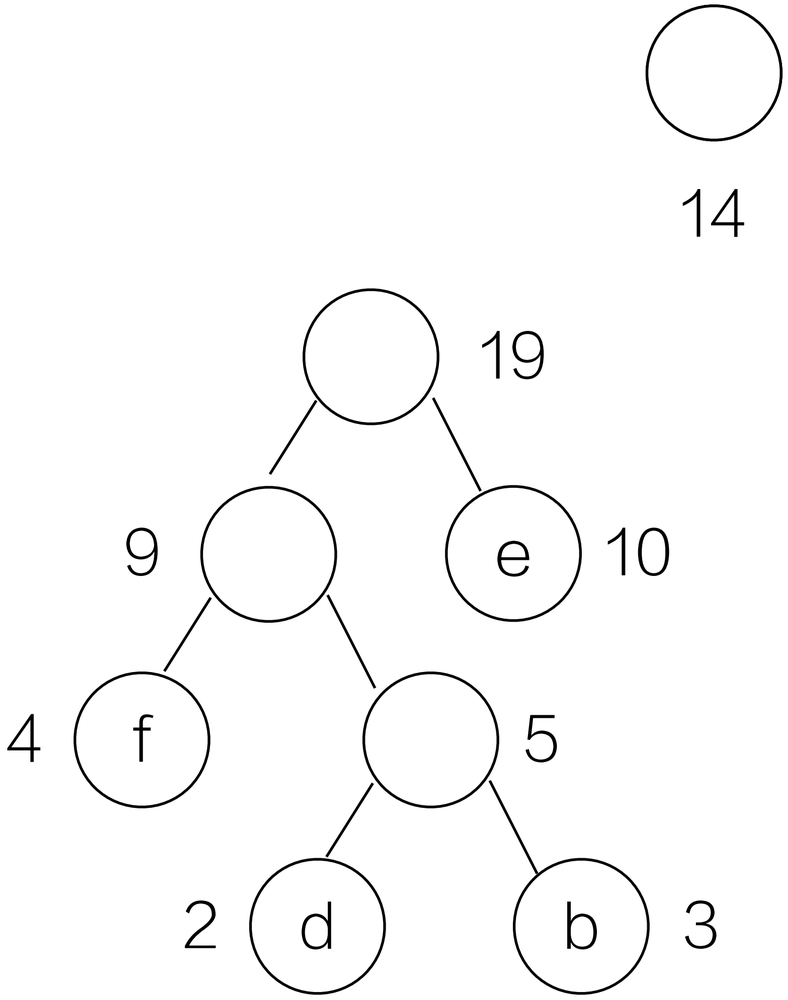
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。



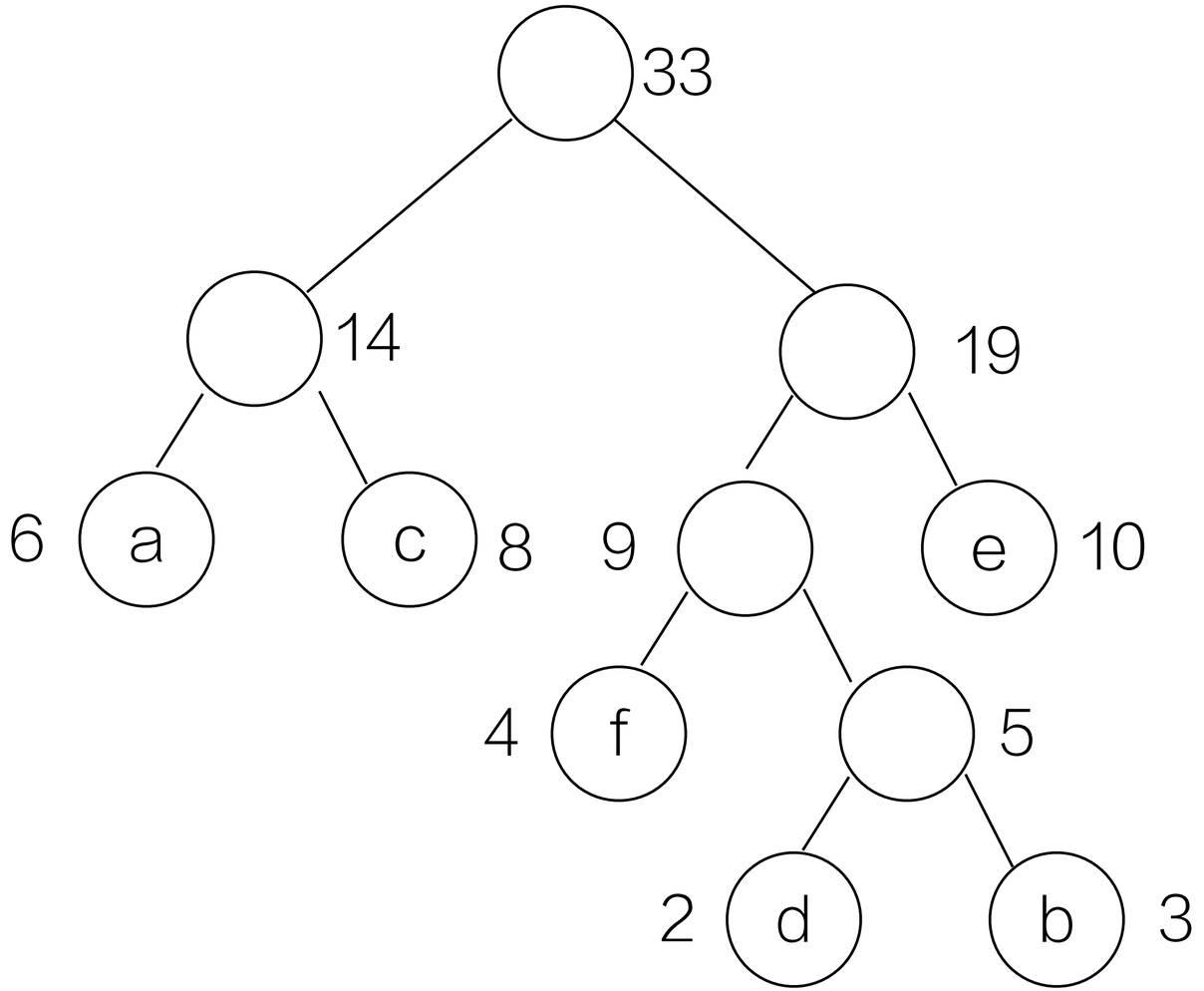
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。



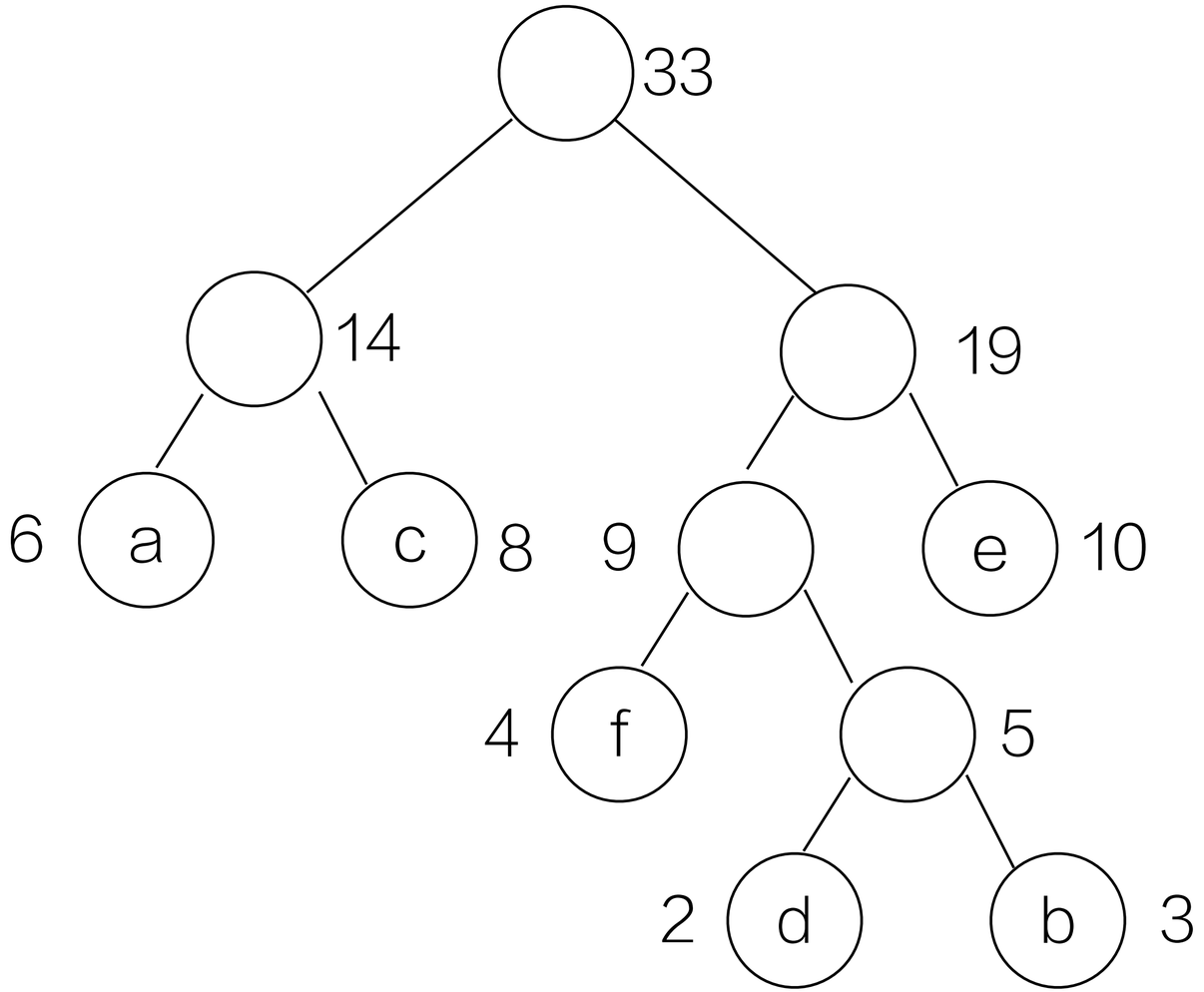
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。



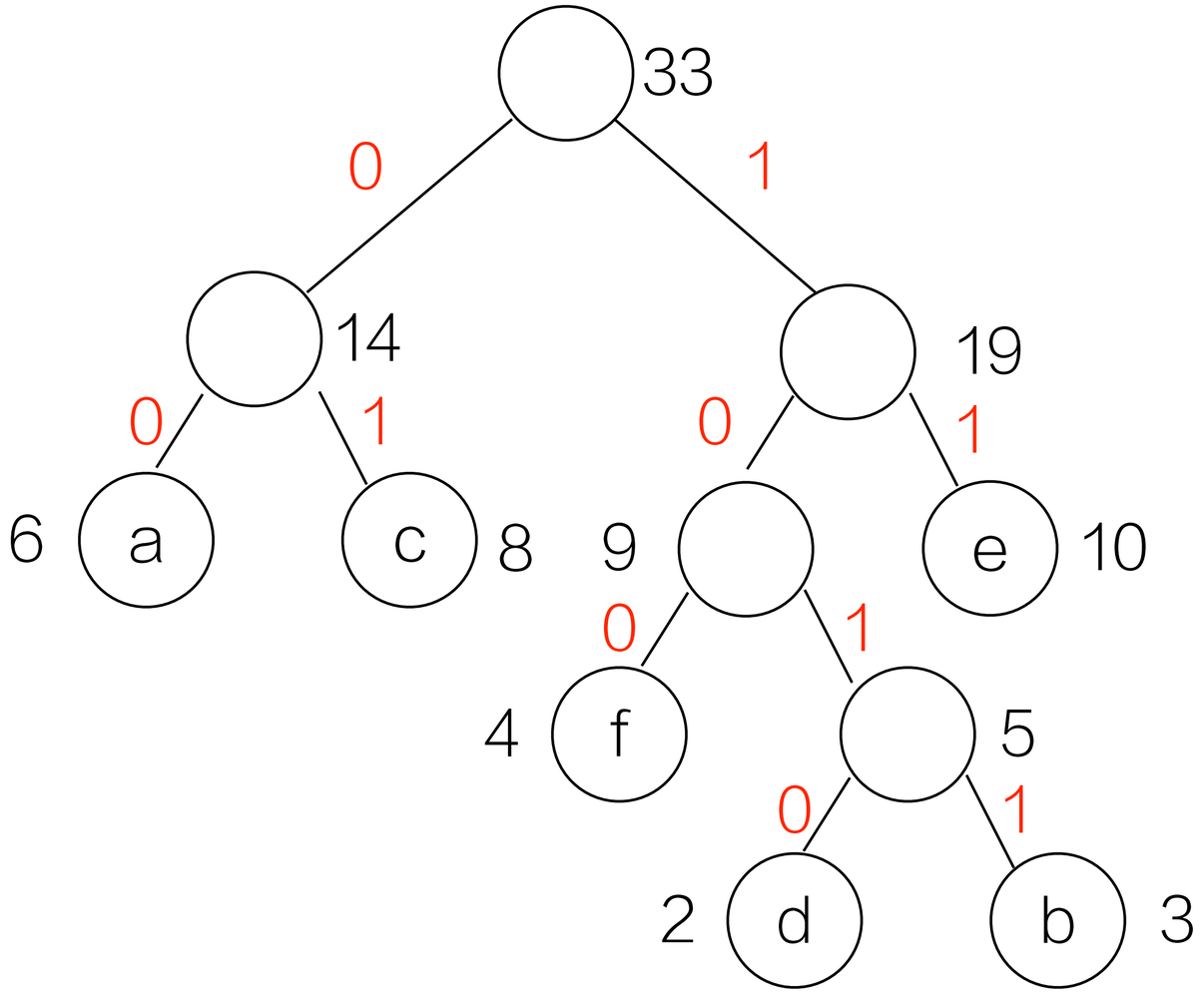
[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。



[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。



[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。



[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

字母	a	b	c	d	e	f
二进制字符	00	1011	01	1010	11	100

[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01, 000

字母	a	b	c	d	e	f
二进制字符	00	1011	01	1010	11	100

[2018] 5. 已知字符集 { a, b, c, d, e, f, } ，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是_____。

A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01, 000

字母	a	b	c	d	e	f
二进制字符	00	1011	01	1010	11	100

哈夫曼树 – 真题解析

[2019] 3. 对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼数共有115个结点，则 n 的值是_____。

A. 56　　B. 57　　C. 58　　D. 60

哈夫曼树 – 真题解析

[2019] 3. 对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼数共有115个结点，则 n 的值是_____。

A. 56　　B. 57　　C. 58　　D. 60

哈夫曼树有度为 1 的结点吗?

哈夫曼树 – 真题解析

[2019] 3. 对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼数共有115个结点，则 n 的值是_____。

A. 56　　B. 57　　C. 58　　D. 60

哈夫曼树有度为 1 的结点吗?

进行哈夫曼编码的符号是什么样的结点，度为几?

哈夫曼树 – 真题解析

[2019] 3. 对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼数共有115个结点，则 n 的值是_____。

A. 56　　B. 57　　C. 58　　D. 60

哈夫曼树有度为 1 的结点吗?

进行哈夫曼编码的符号是什么样的结点，度为几?

还记得二叉树的性质三吗?

哈夫曼树 – 真题解析

[2019] 3. 对 n 个互不相同的符号进行哈夫曼编码。若生成的哈夫曼数共有115个结点，则 n 的值是_____。

A. 56　　B. 57　　C. 58　　D. 60

哈夫曼树 – 真题解析

[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

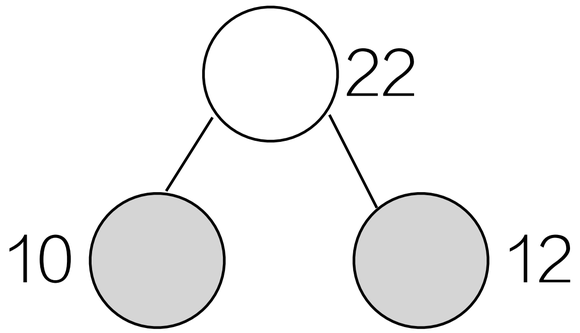
A. 89　　B. 200　　C. 208　　D. 289

[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

A. 89　　B. 200　　C. 208　　D. 289

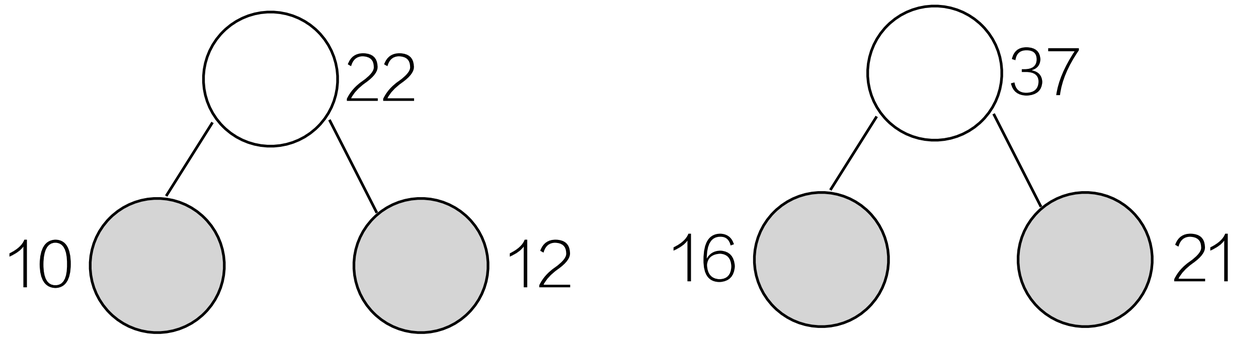
[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

A. 89　　B. 200　　C. 208　　D. 289



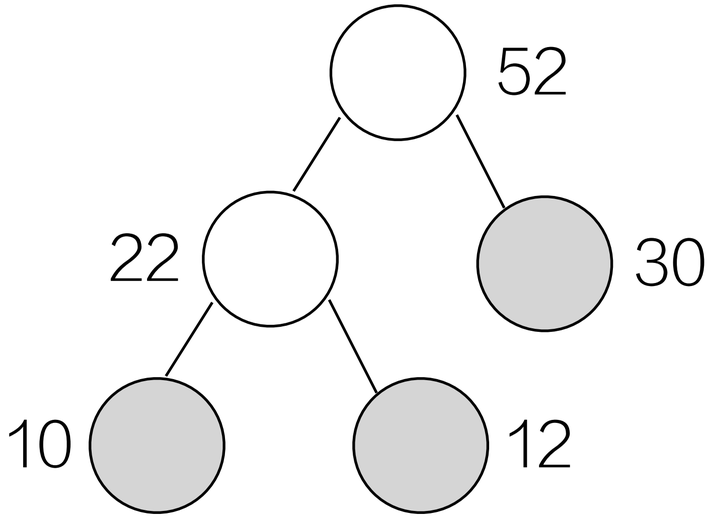
[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

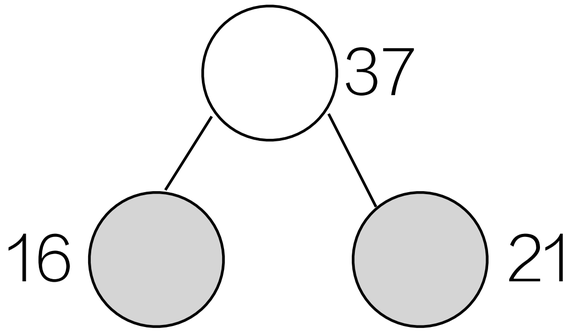
A. 89　　B. 200　　C. 208　　D. 289



[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

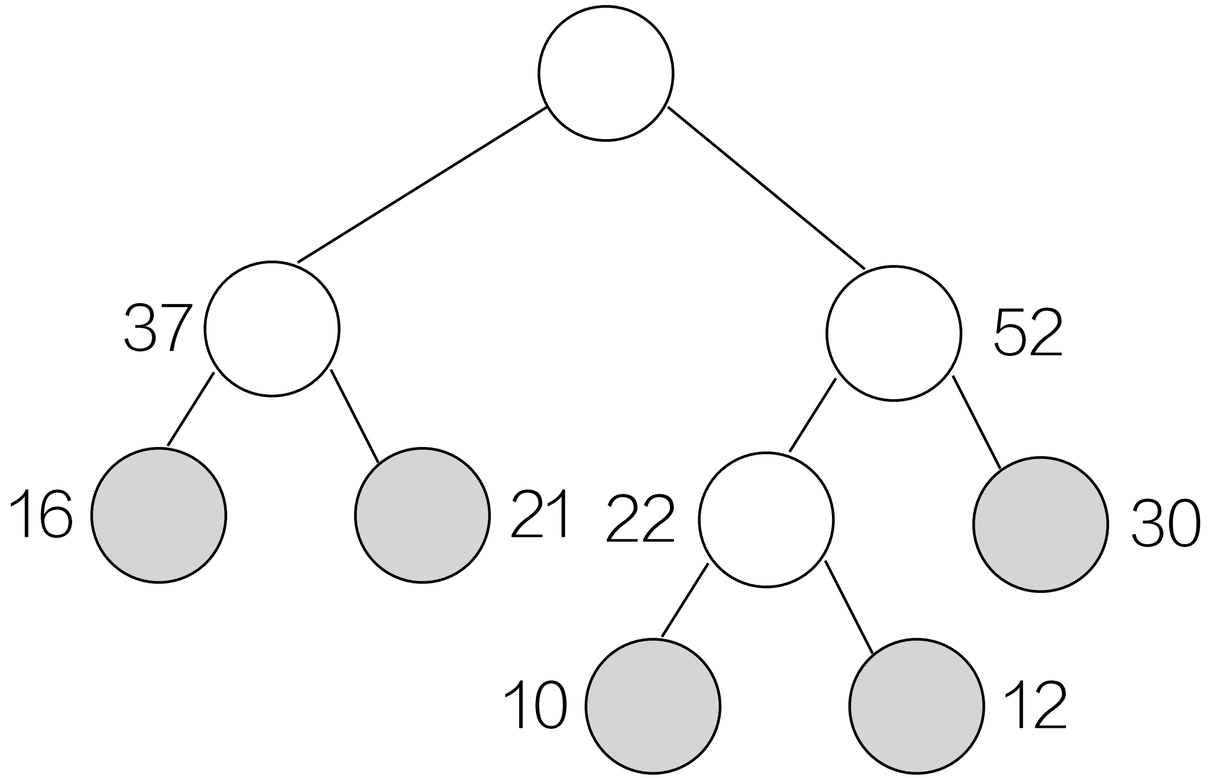
A. 89　　B. 200　　C. 208　　D. 289





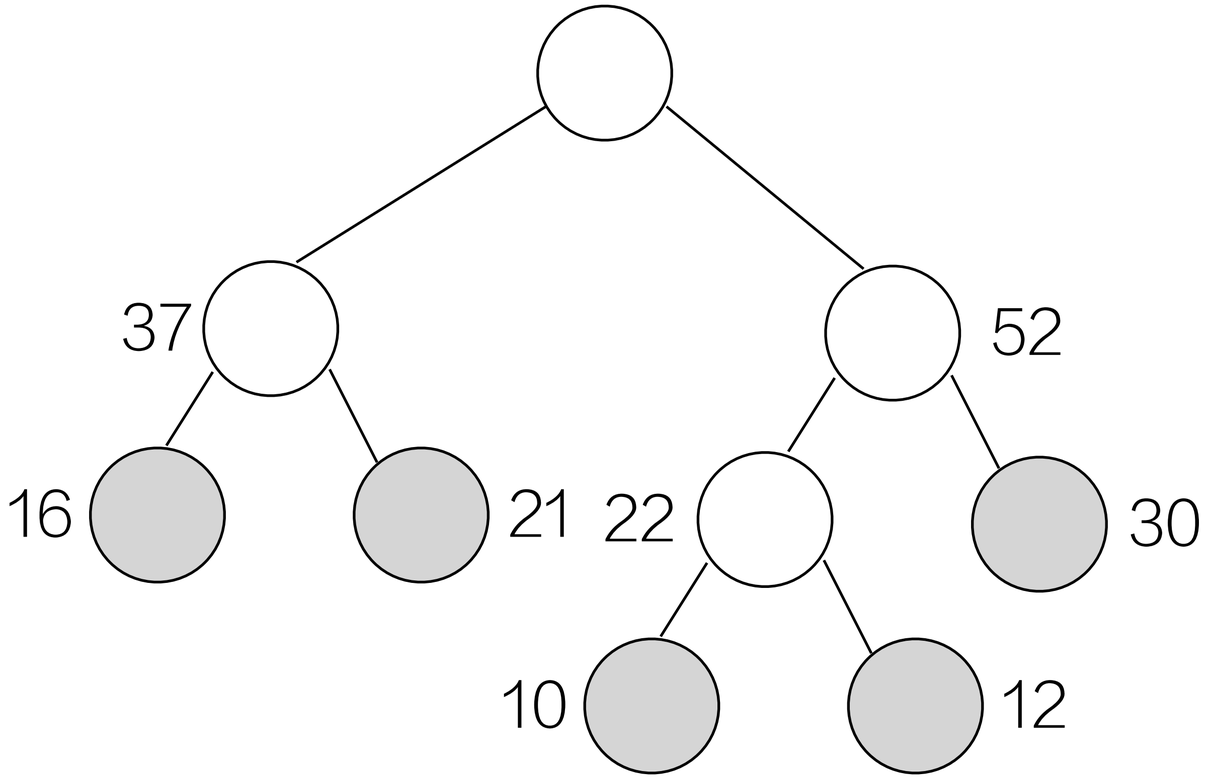
[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

A. 89　　B. 200　　C. 208　　D. 289



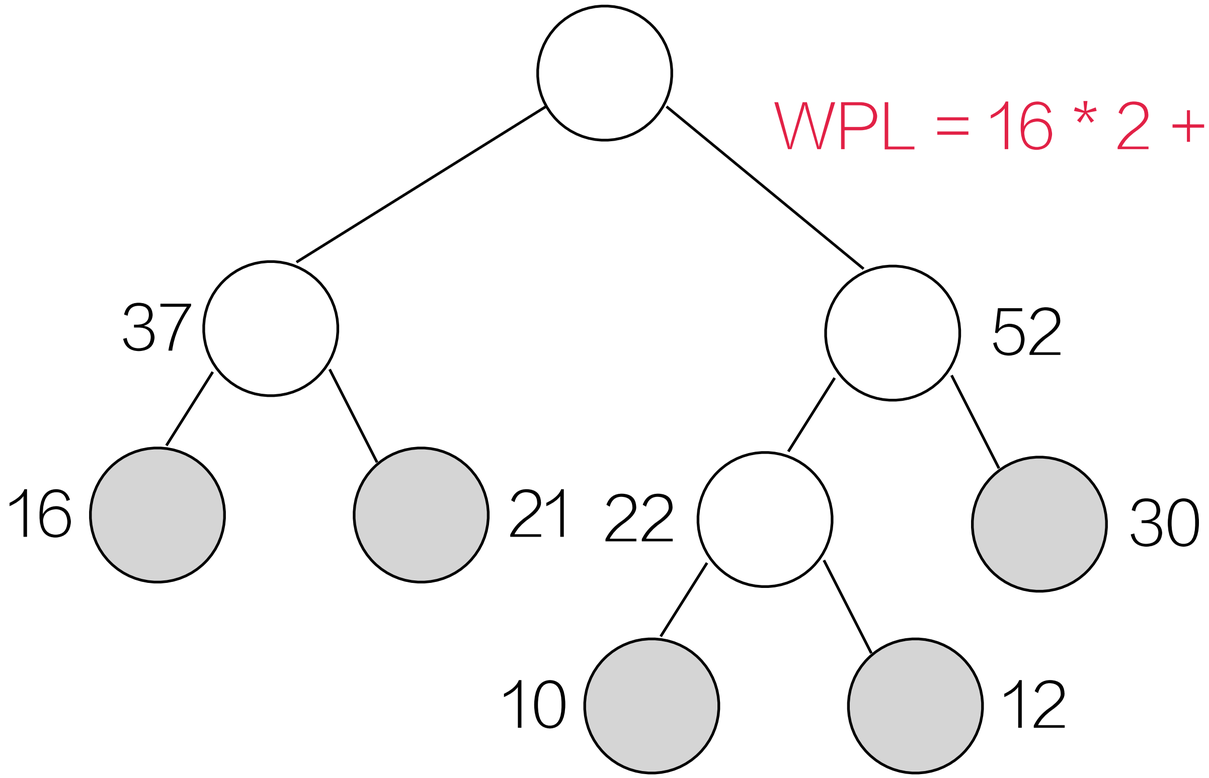
[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

A. 89　　B. 200　　C. 208　　D. 289



[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

A. 89　　B. 200　　C. 208　　D. 289



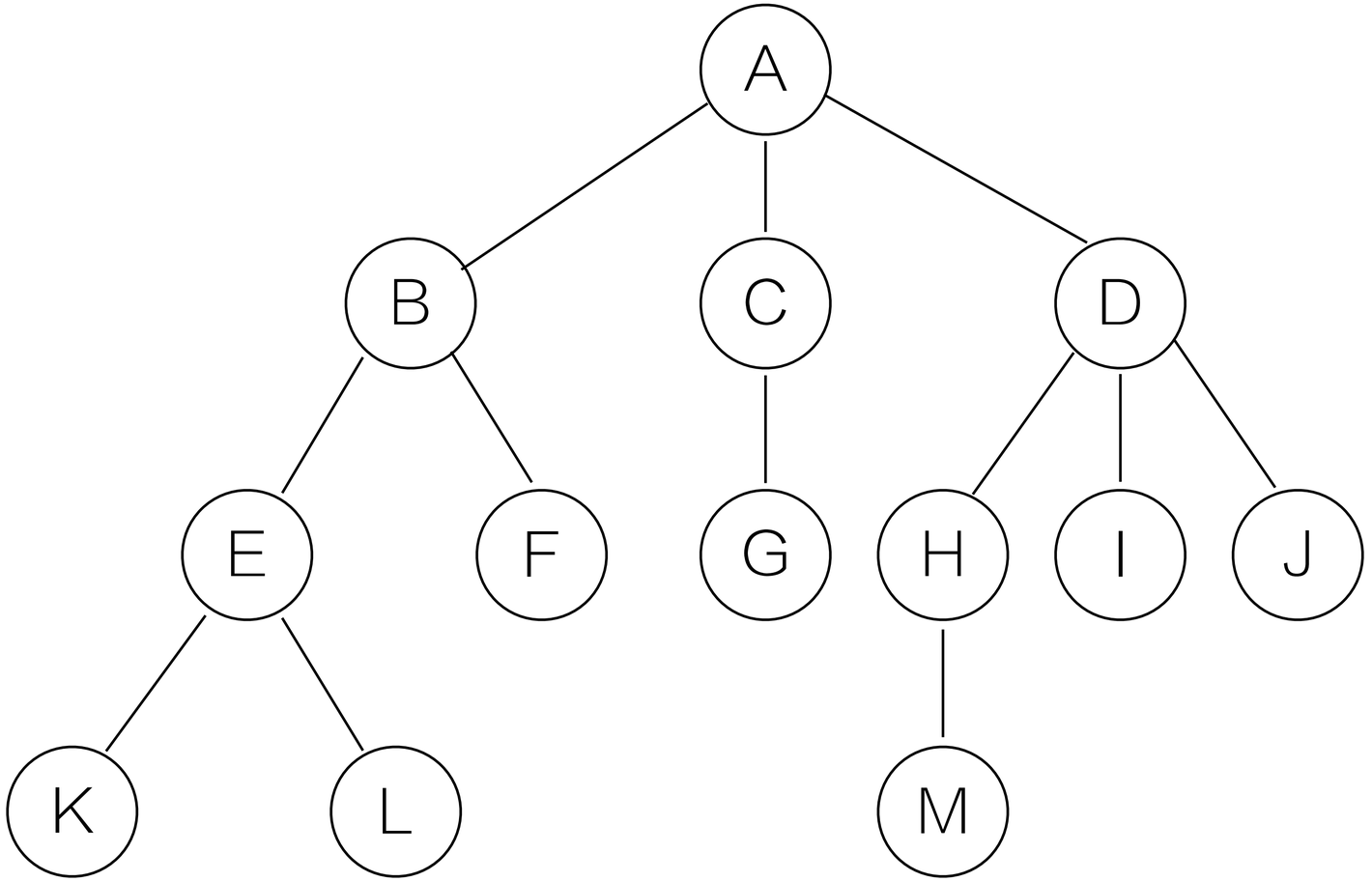
WPL = 16 * 2 + 21 * 2 + 30 * 2 + 10 * 3 + 12 * 3 =200

哈夫曼树 – 真题解析

[2021] 5. 若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（ WPL ）是_____。

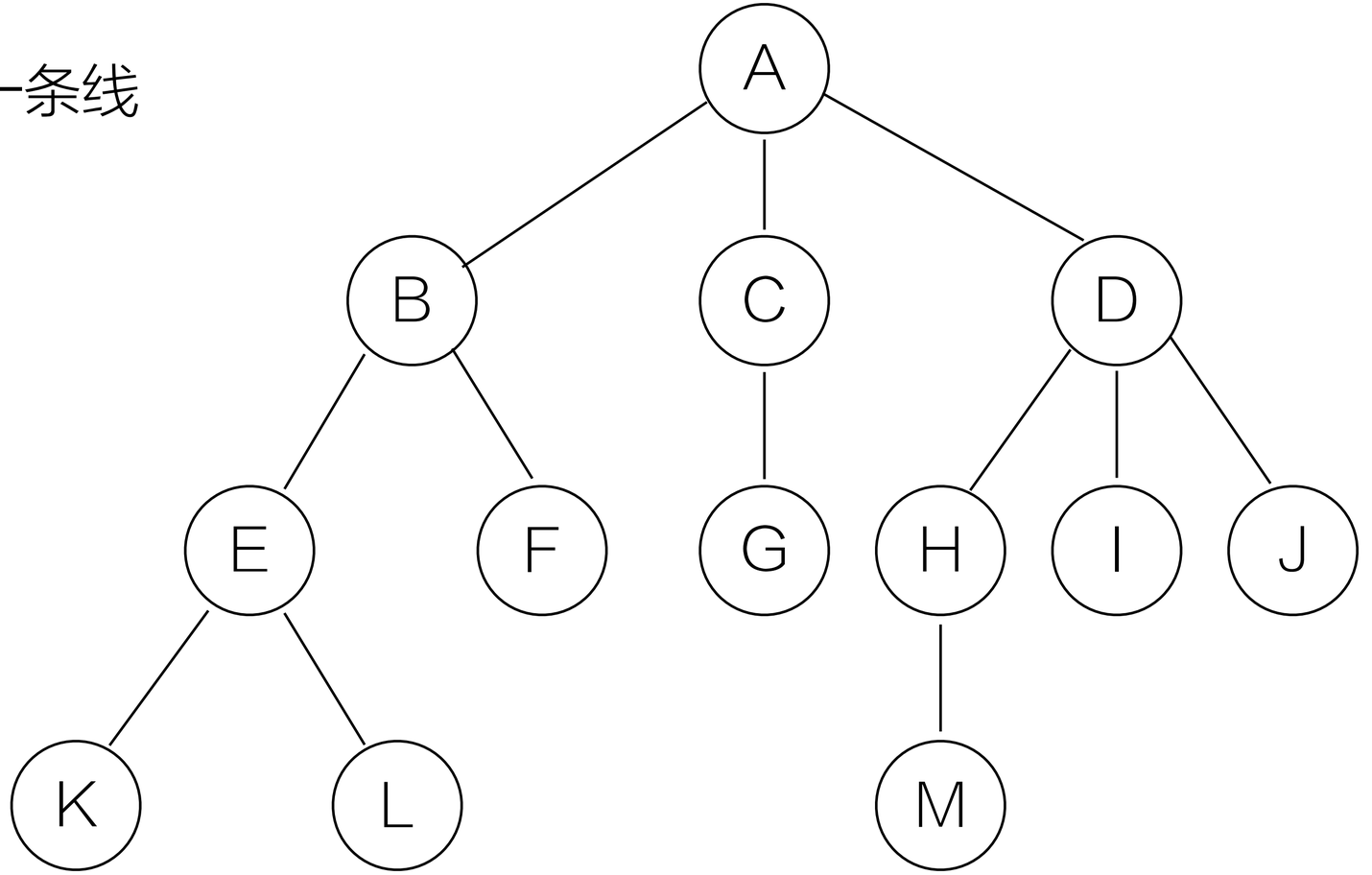
A. 89　　B. 200　　C. 208　　D. 289

树与二叉树的转换



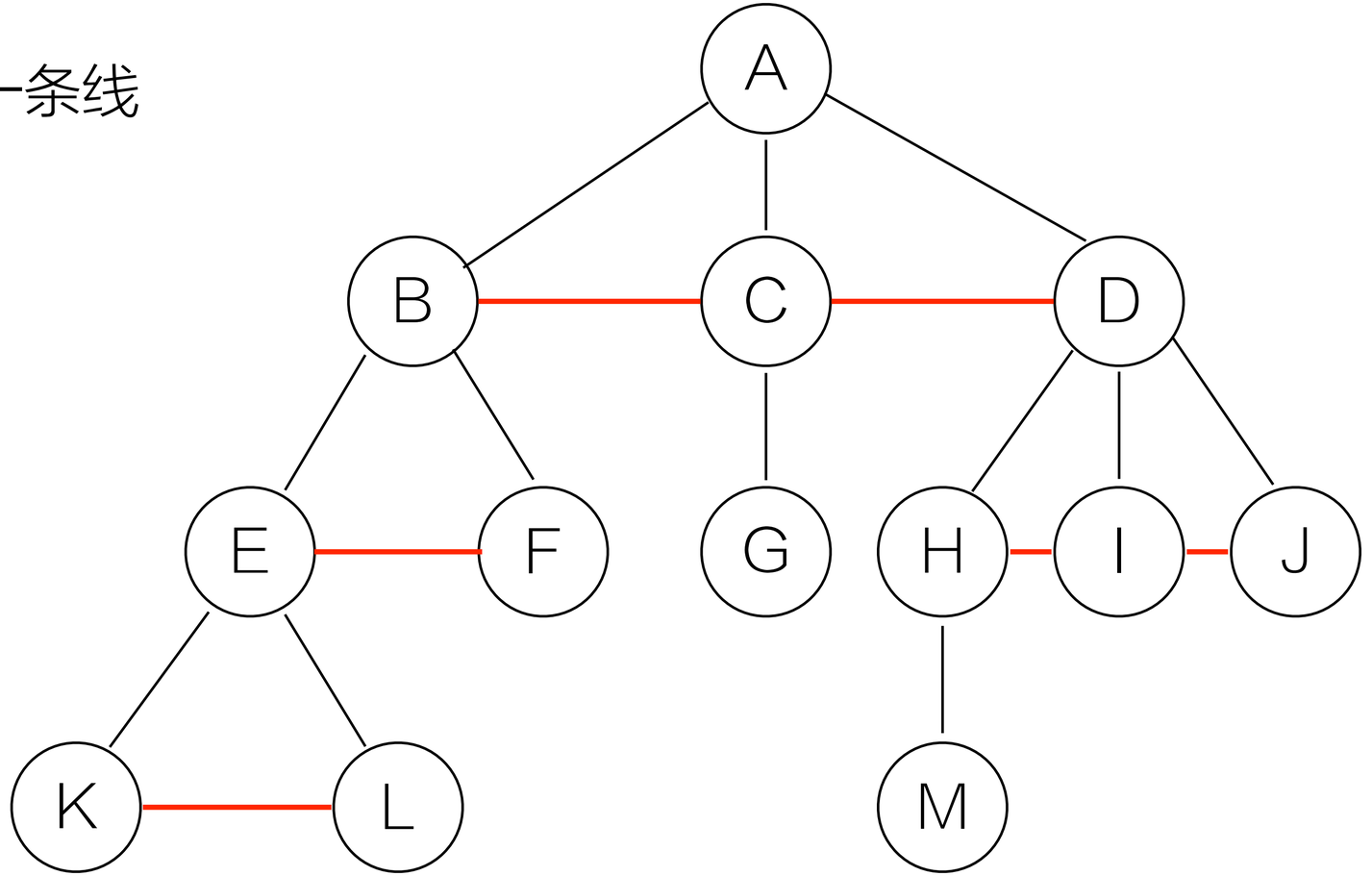
树与二叉树的转换

1. 加线，在所有兄弟结点之间加一条线



树与二叉树的转换

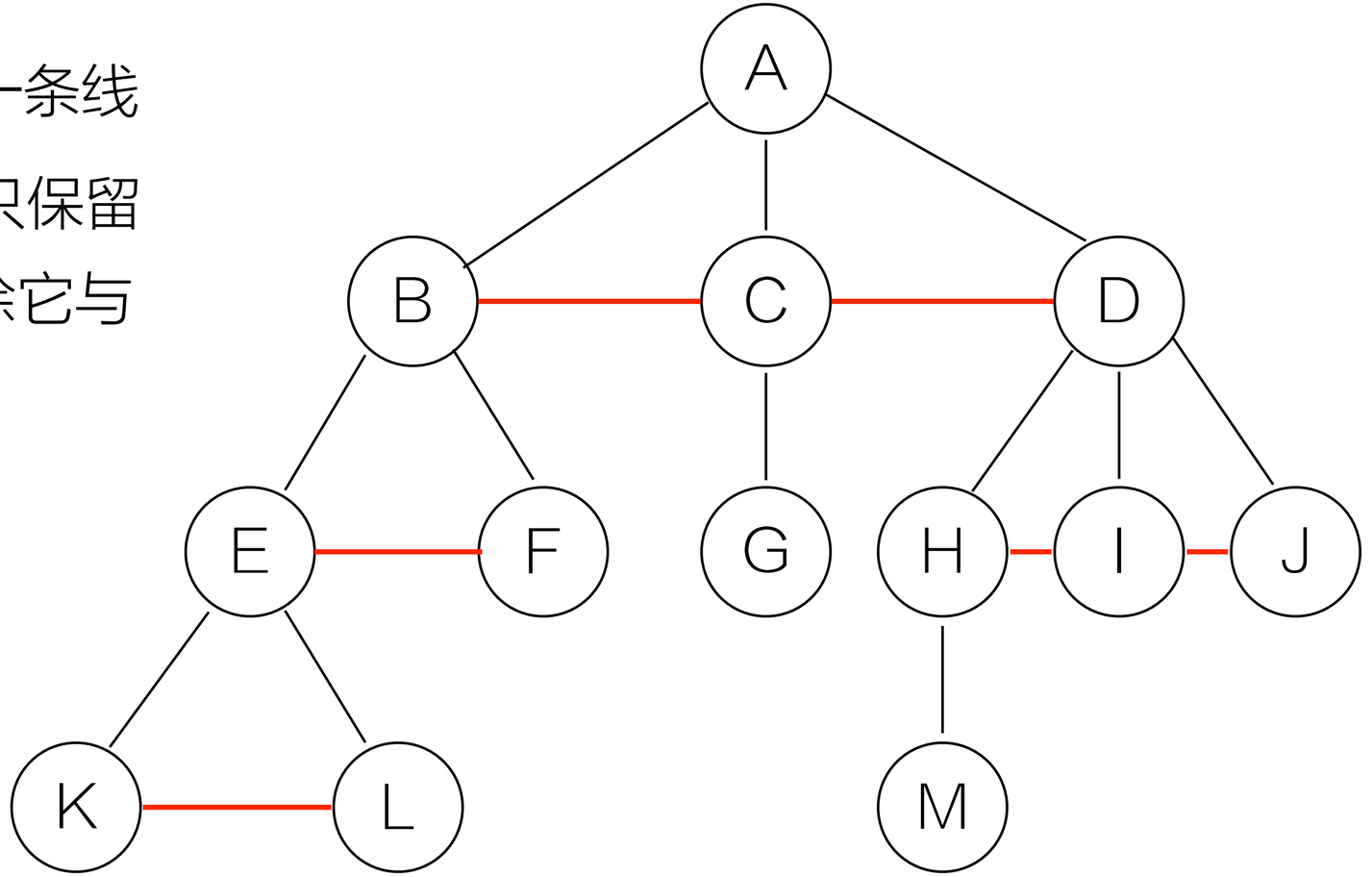
1. 加线，在所有兄弟结点之间加一条线



树与二叉树的转换

1. 加线，在所有兄弟结点之间加一条线

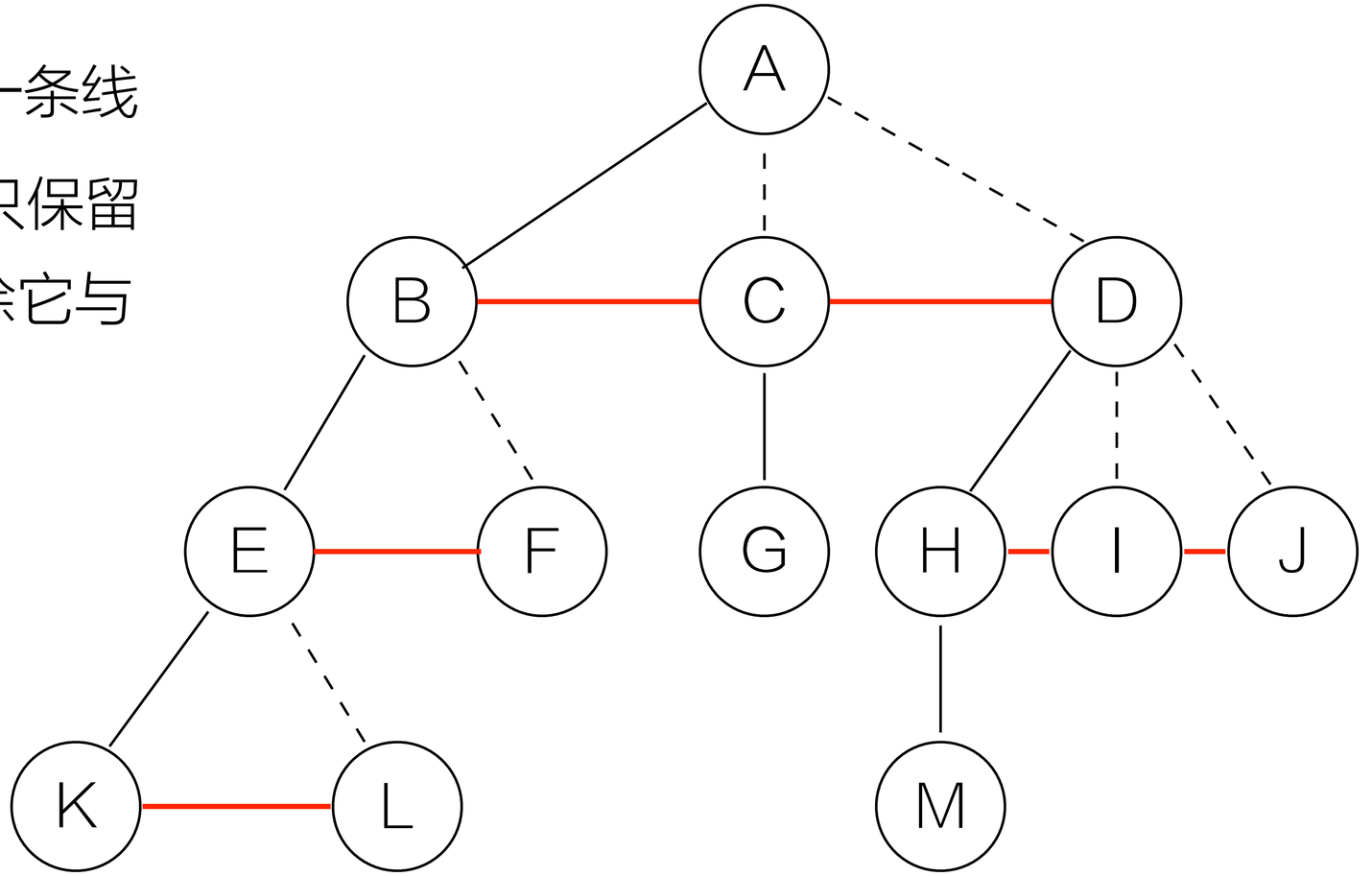
2. 去线，对树中的每一个结点，只保留它与第一个孩子结点的连线，删除它与其它孩子结点之间的连线。



树与二叉树的转换

1. 加线，在所有兄弟结点之间加一条线

2. 去线，对树中的每一个结点，只保留它与第一个孩子结点的连线，删除它与其它孩子结点之间的连线。

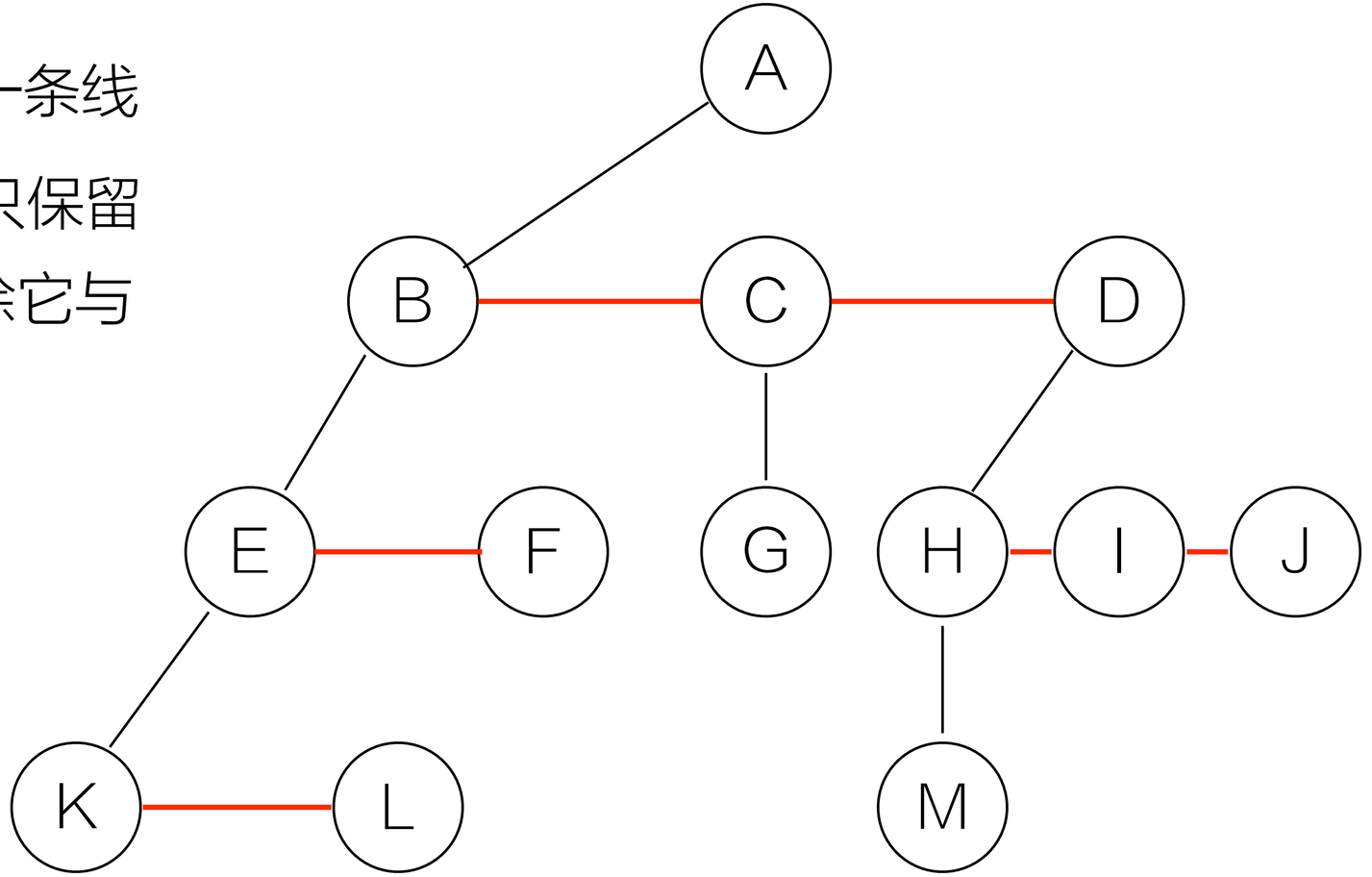


树与二叉树的转换

1. 加线，在所有兄弟结点之间加一条线

2. 去线，对树中的每一个结点，只保留它与第一个孩子结点的连线，删除它与其它孩子结点之间的连线。

3. 层次调整，以树的根结点为轴心，将整棵数顺时针旋转一定角度，使之层次分明。注意第一个孩子是二叉树结点的左孩子。兄弟转过来的孩子是结点的右孩子。

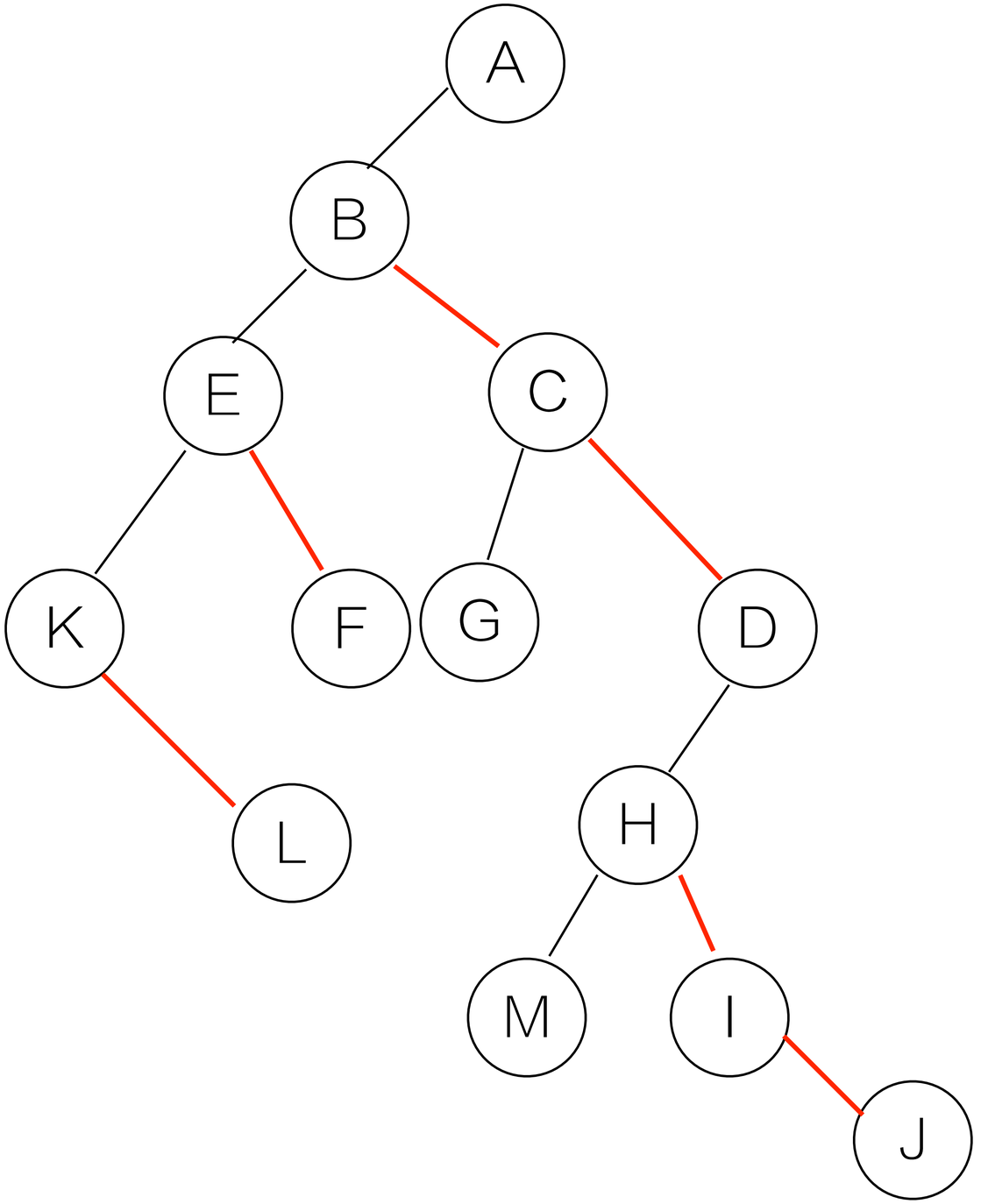


树与二叉树的转换

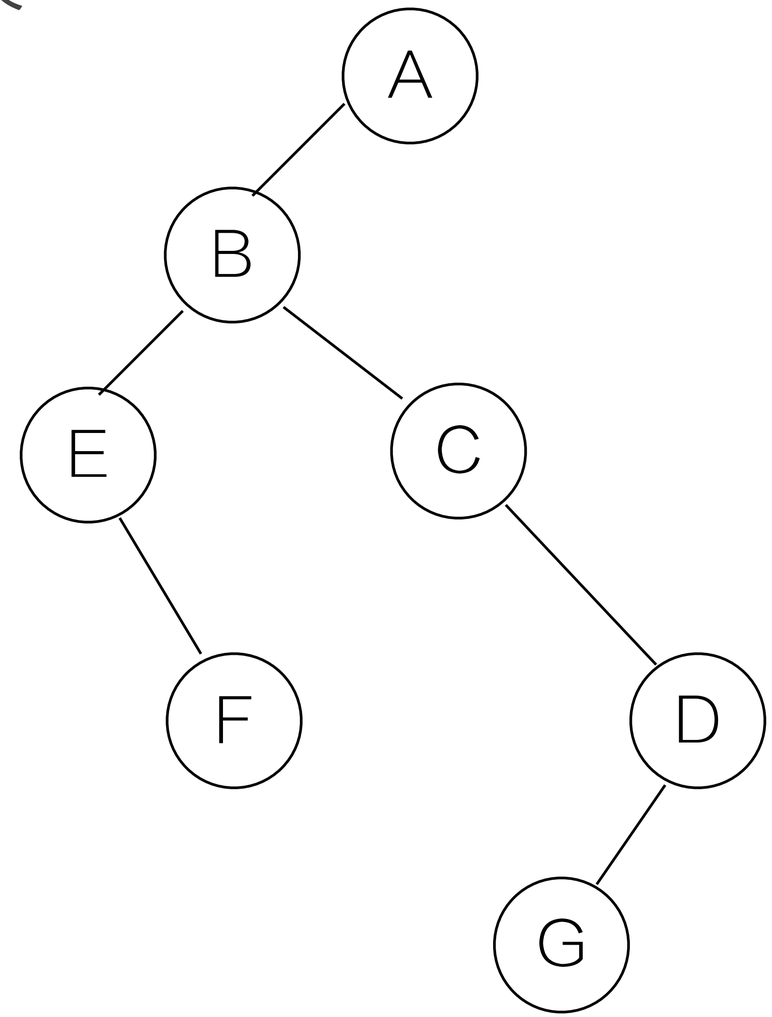
1. 加线，在所有兄弟结点之间加一条线

2. 去线，对树中的每一个结点，只保留它与第一个孩子结点的连线，删除它与其它孩子结点之间的连线。

3. 层次调整，以树的根结点为轴心，将整棵数顺时针旋转一定角度，使之层次分明。注意第一个孩子是二叉树结点的左孩子。兄弟转过来的孩子是结点的右孩子。

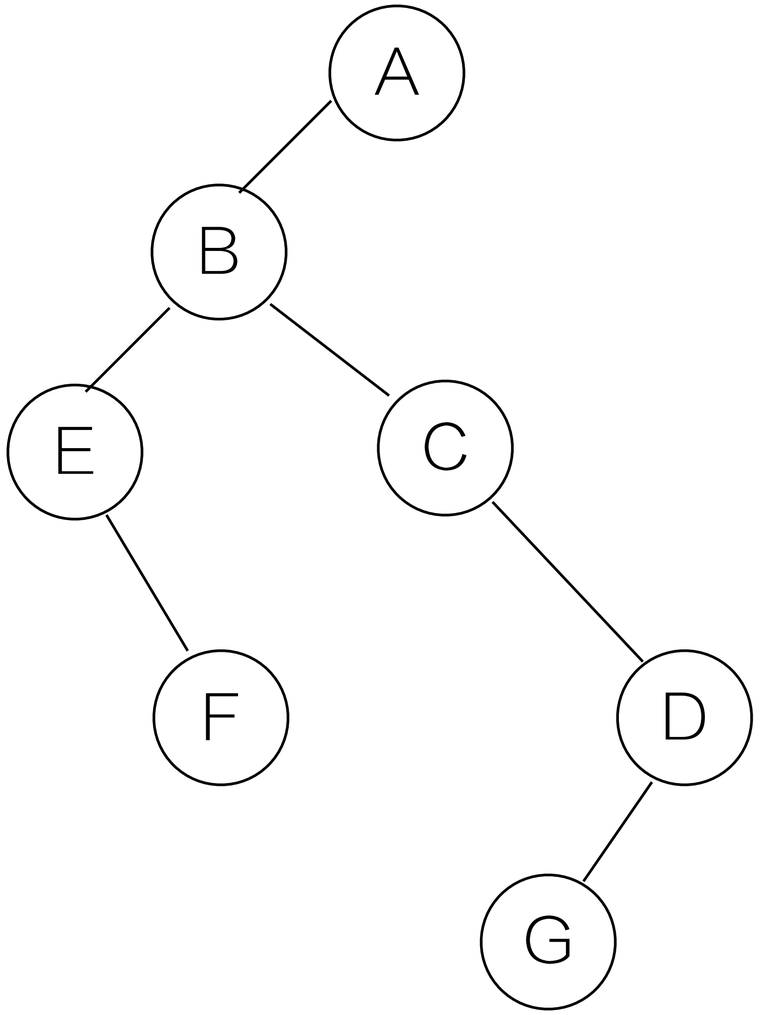


树与二叉树的转换



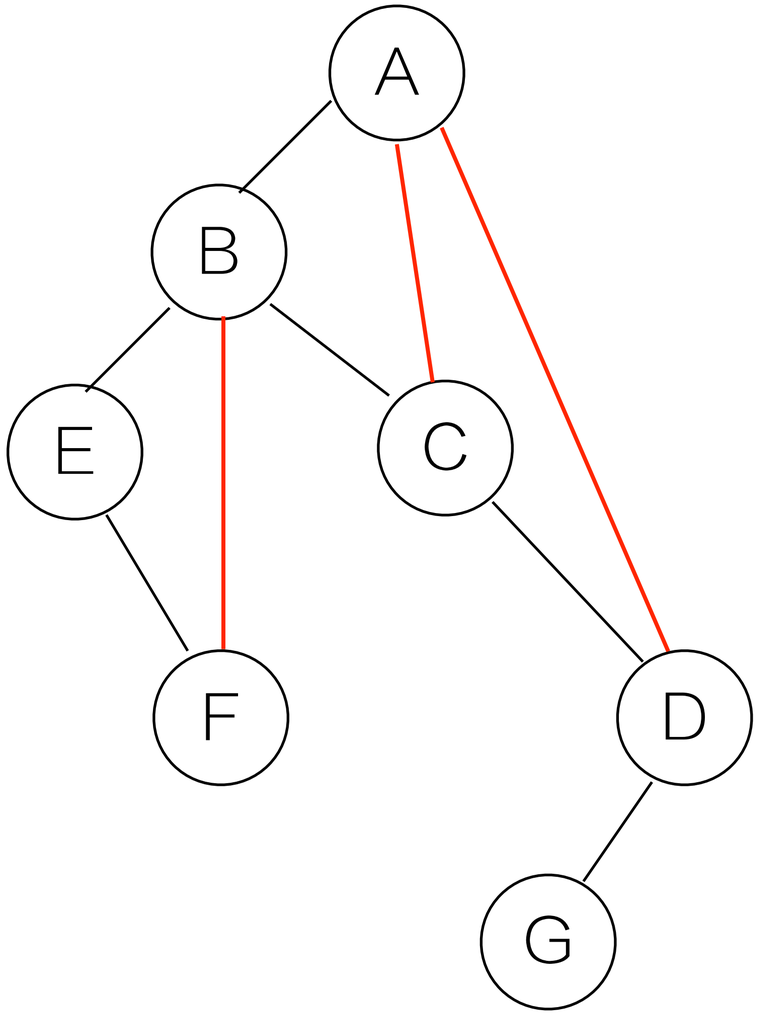
树与二叉树的转换

1. 加线，若某个结点的左孩子存在，则将这个左孩子的所有右孩子结点都作为此结点的孩子，将该结点与这些右孩子结点用线连起来。



树与二叉树的转换

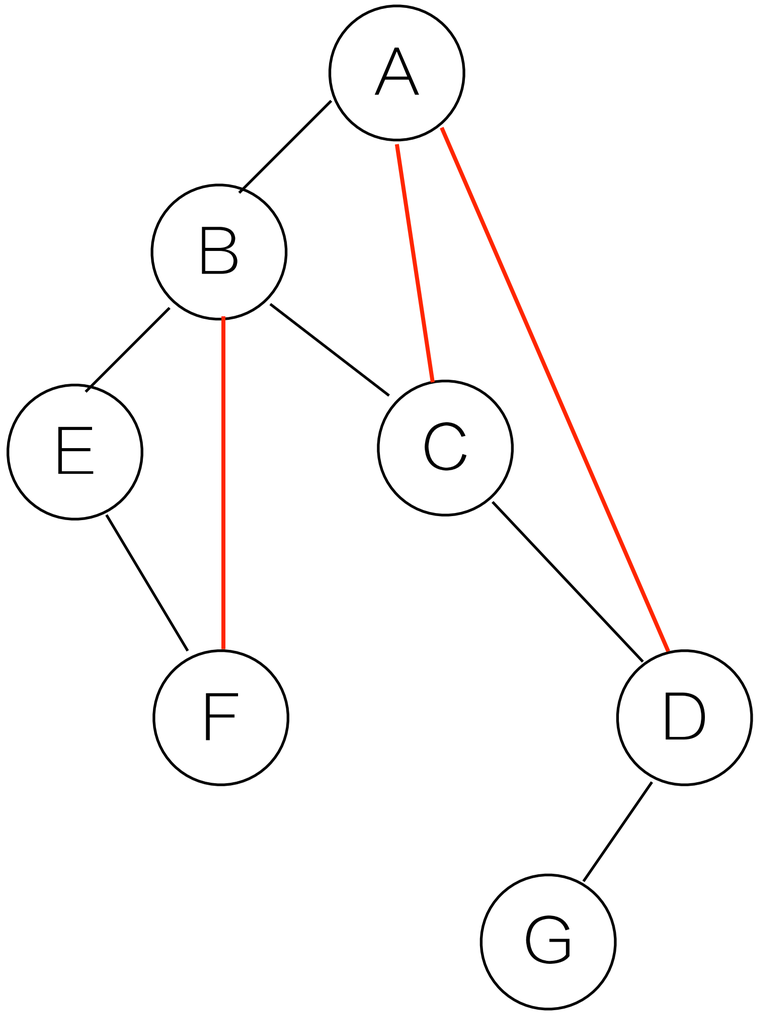
1. 加线，若某个结点的左孩子存在，则将这个左孩子的所有右孩子结点都作为此结点的孩子，将该结点与这些右孩子结点用线连起来。



树与二叉树的转换

1. 加线，若某个结点的左孩子存在，则将这个左孩子的所有右孩子结点都作为此结点的孩子，将该结点与这些右孩子结点用线连起来。

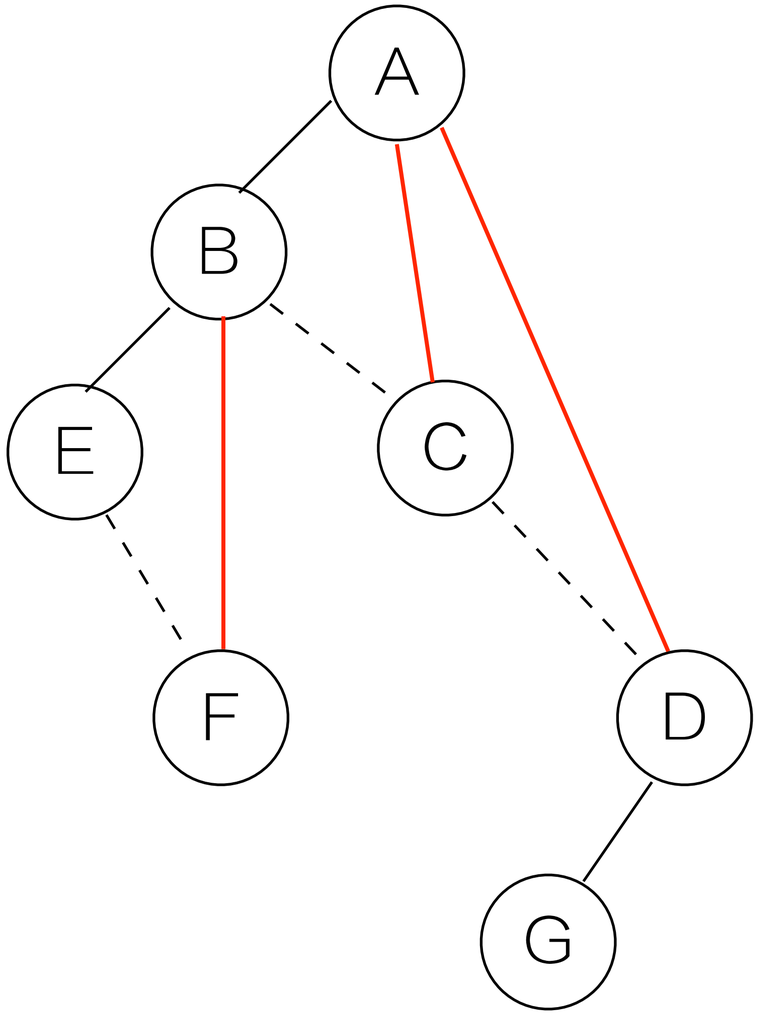
2. 去线，删除二叉树中所有结点与其右孩子结点的连线



树与二叉树的转换

1. 加线，若某个结点的左孩子存在，则将这个左孩子的所有右孩子结点都作为此结点的孩子，将该结点与这些右孩子结点用线连起来。

2. 去线，删除二叉树中所有结点与其右孩子结点的连线

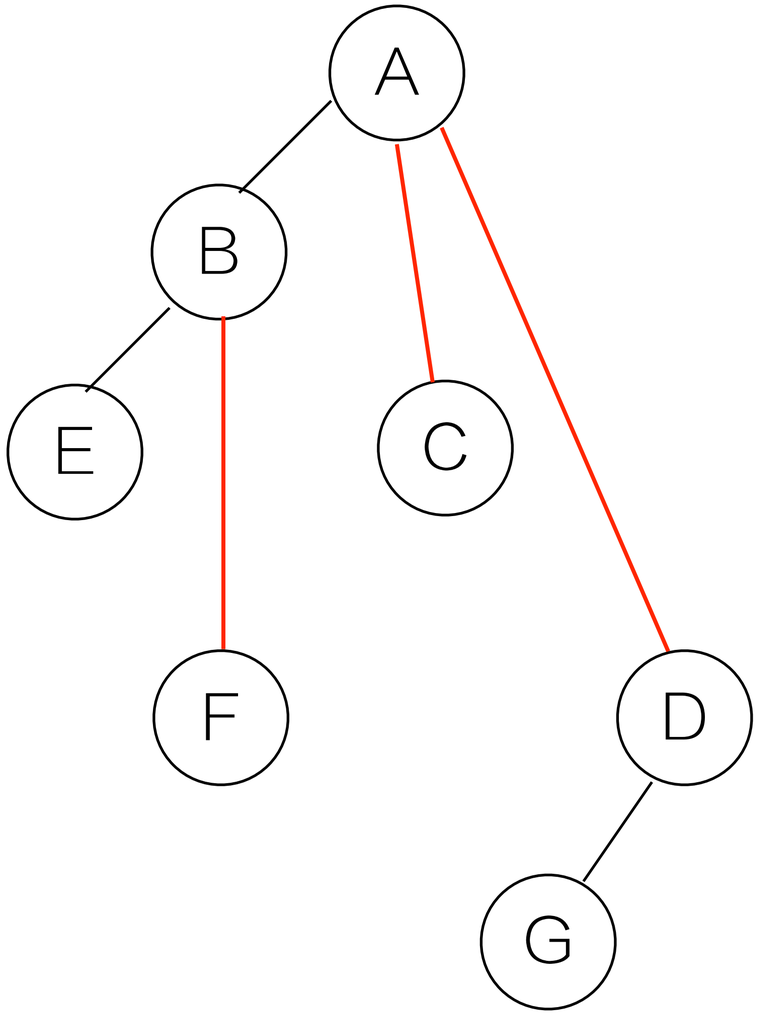


树与二叉树的转换

1. 加线，若某个结点的左孩子存在，则将这个左孩子的所有右孩子结点都作为此结点的孩子，将该结点与这些右孩子结点用线连起来。

2. 去线，删除二叉树中所有结点与其右孩子结点的连线

3. 调整，转一下

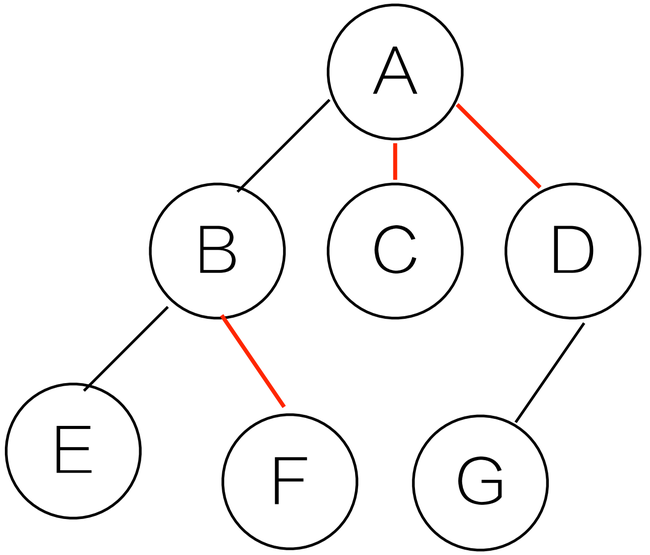


树与二叉树的转换

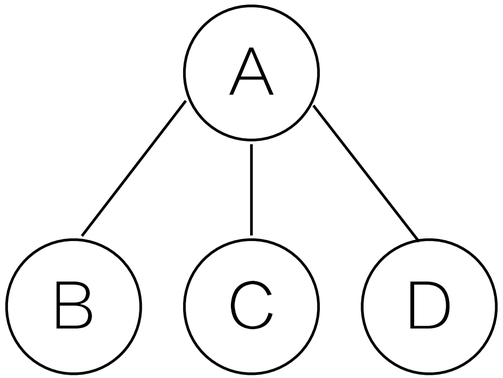
1. 加线，若某个结点的左孩子存在，则将这个左孩子的所有右孩子结点都作为此结点的孩子，将该结点与这些右孩子结点用线连起来。

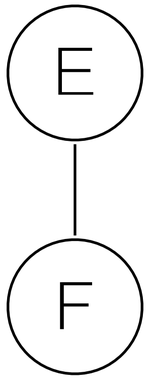
2. 去线，删除二叉树中所有结点与其右孩子结点的连线

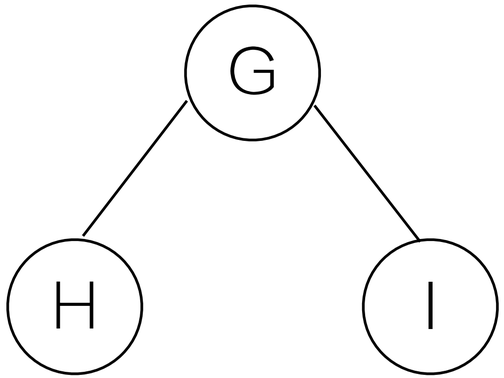
3. 调整，转一下



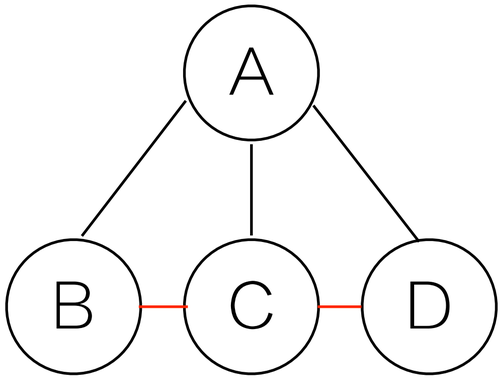
森林与二叉树的转换

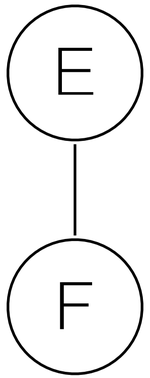


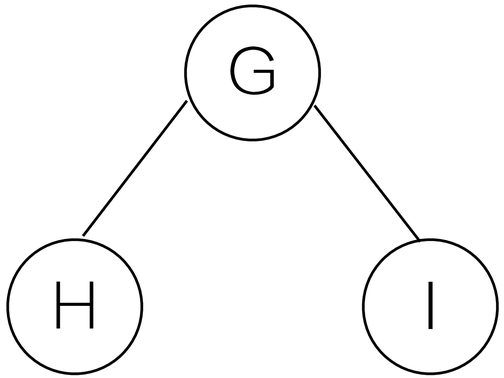




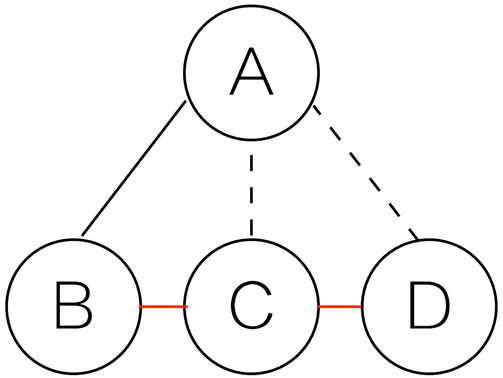
森林与二叉树的转换

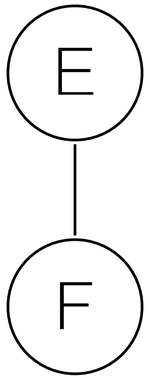


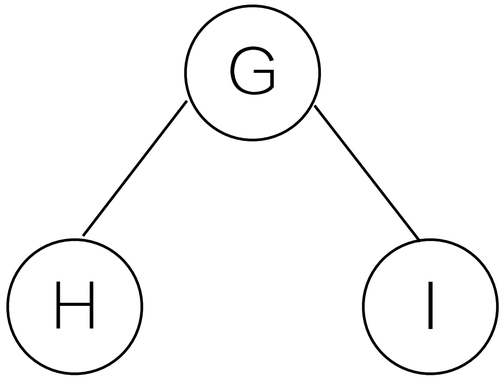




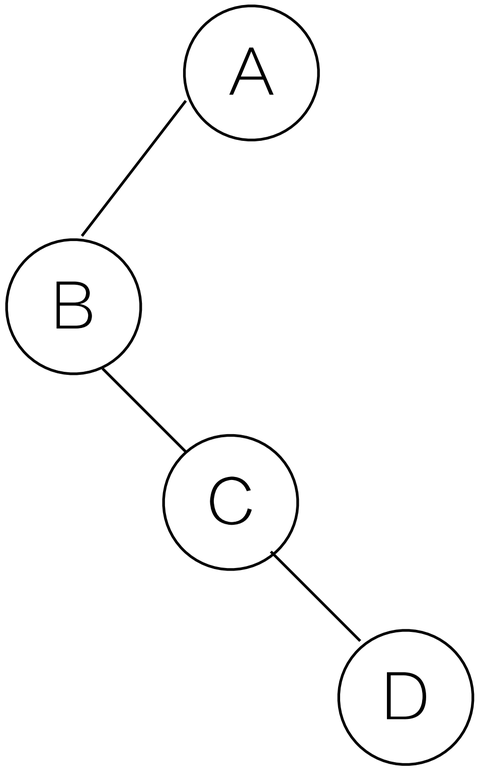
森林与二叉树的转换

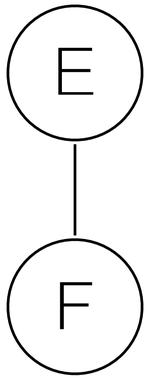


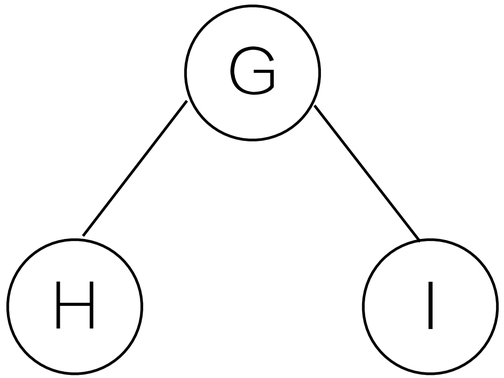




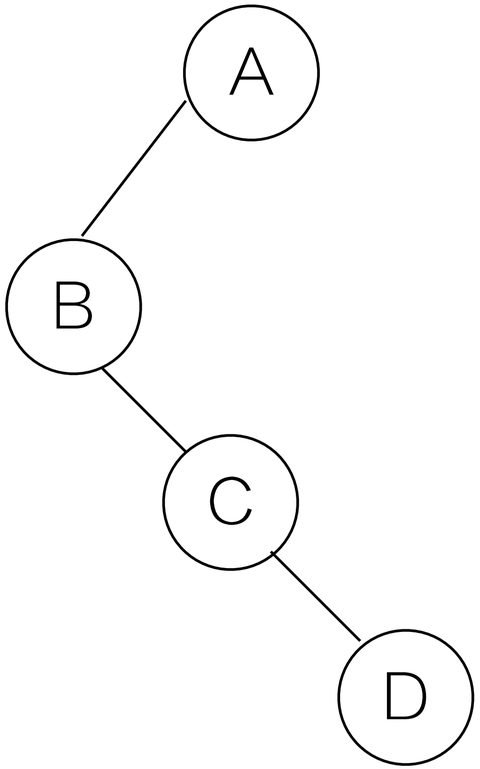
森林与二叉树的转换

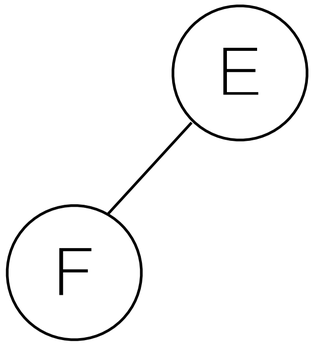


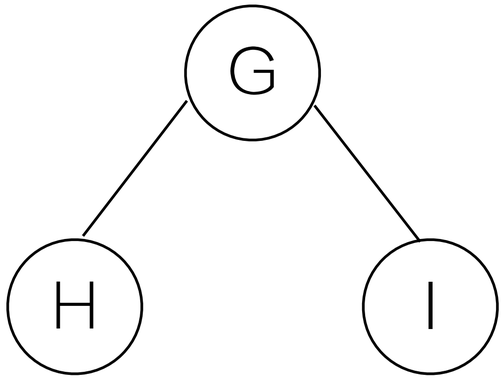




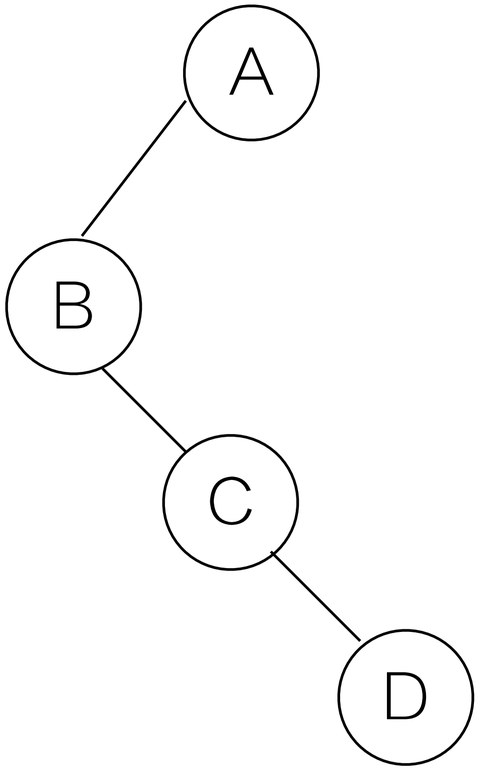
森林与二叉树的转换

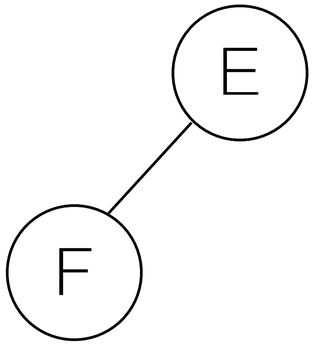


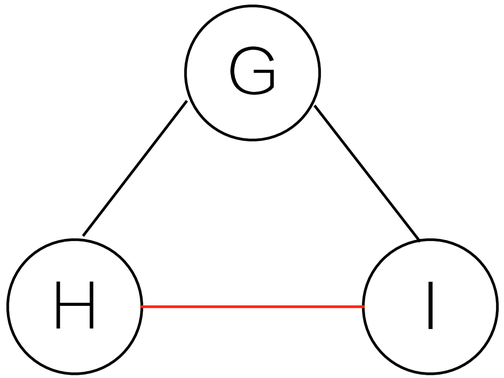




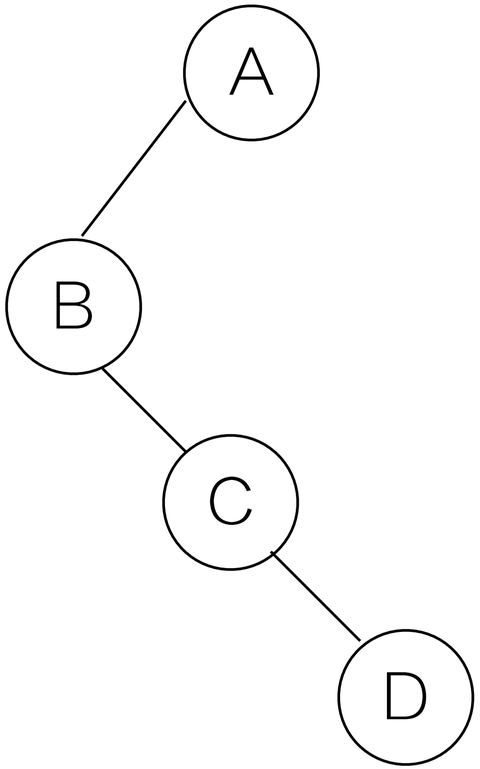
森林与二叉树的转换

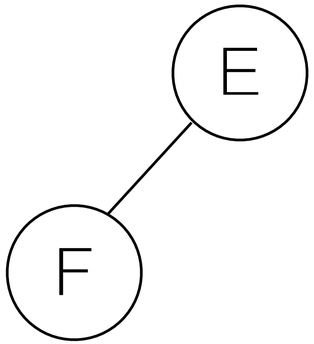


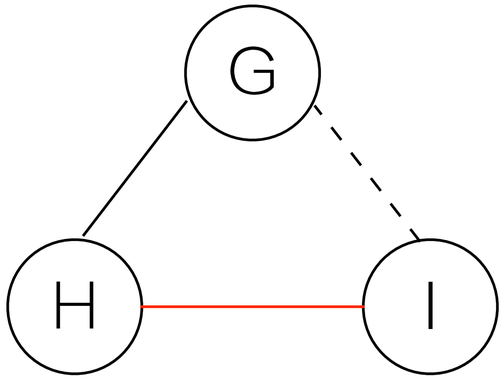




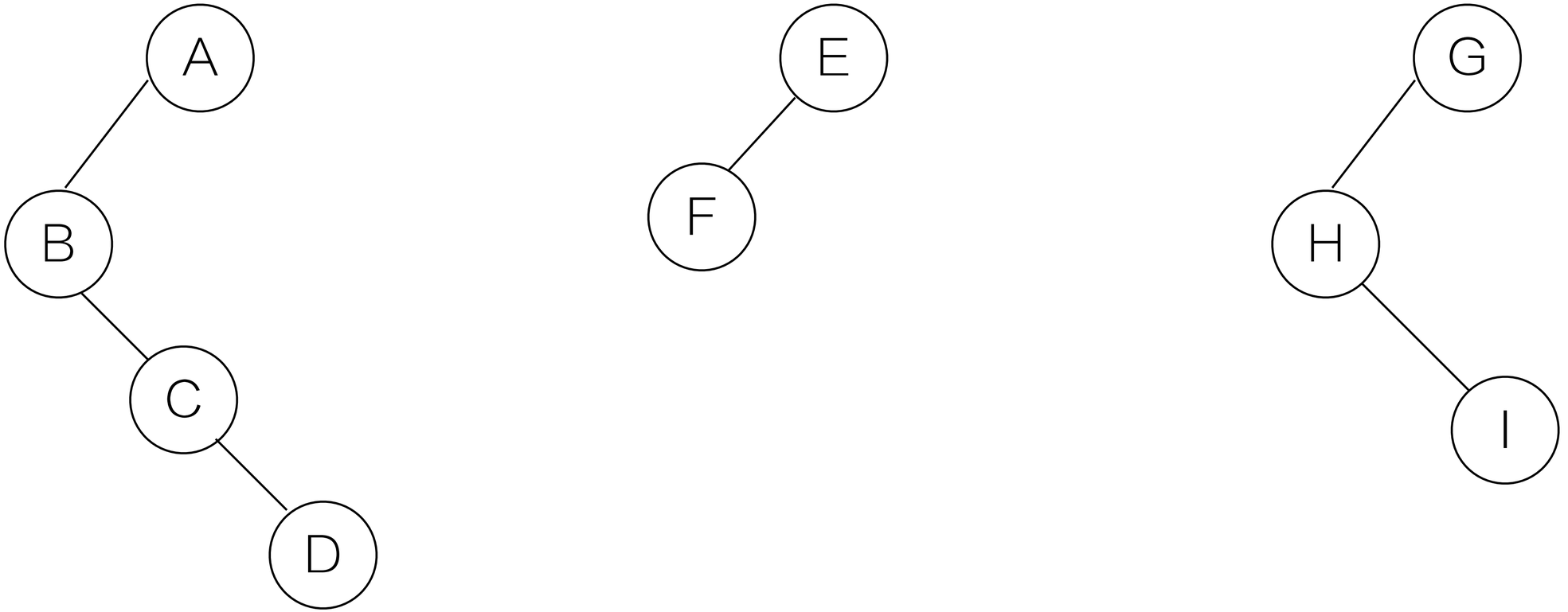
森林与二叉树的转换





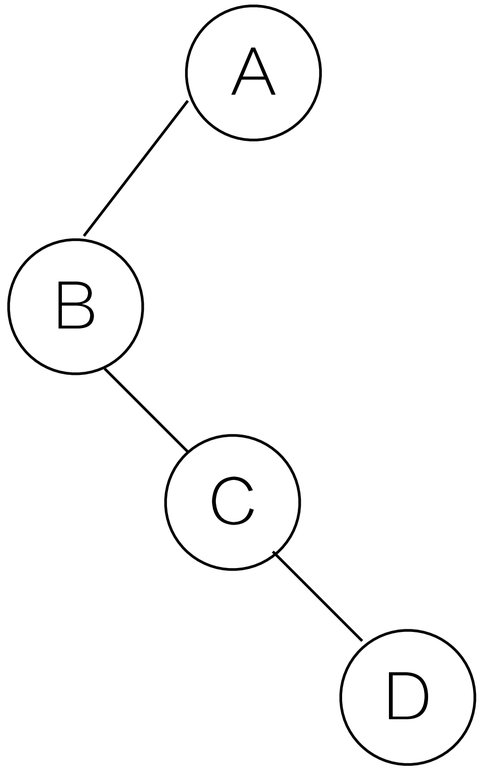


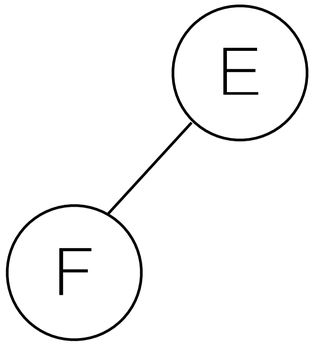
森林与二叉树的转换

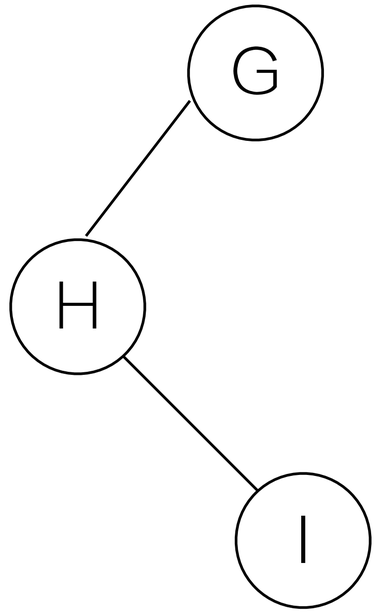


森林与二叉树的转换

第一棵二叉树不动，从第二棵树开始，依次把后一棵二叉树的根结点作为前一棵二叉树的根结点的右孩子，然后用线连起来。

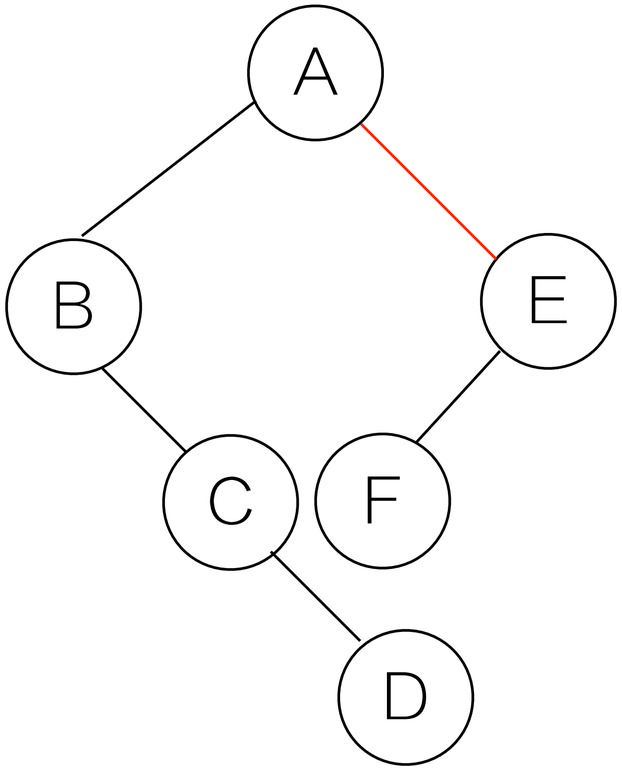


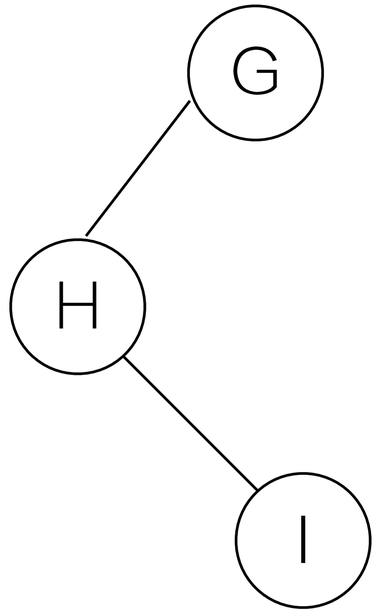




森林与二叉树的转换

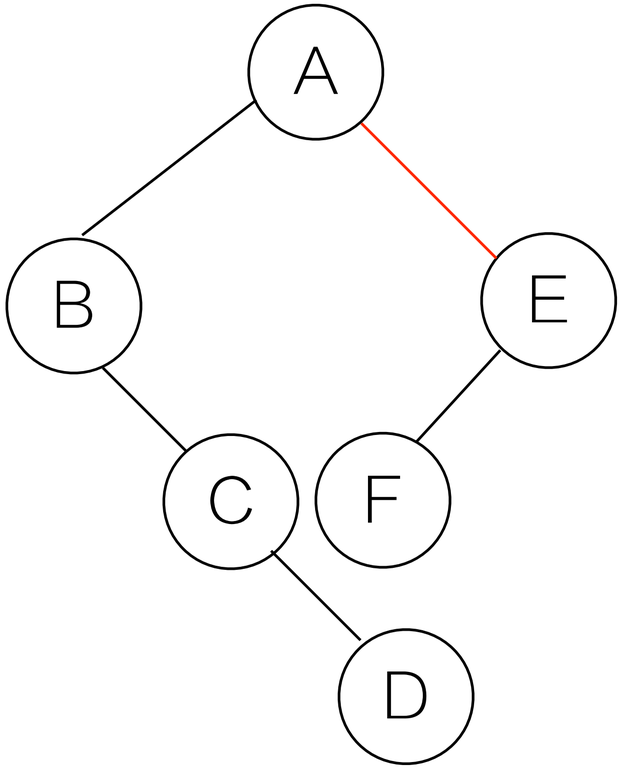
第一棵二叉树不动，从第二棵树开始，依次把后一棵二叉树的根结点作为前一棵二叉树的根结点的右孩子，然后用线连起来。

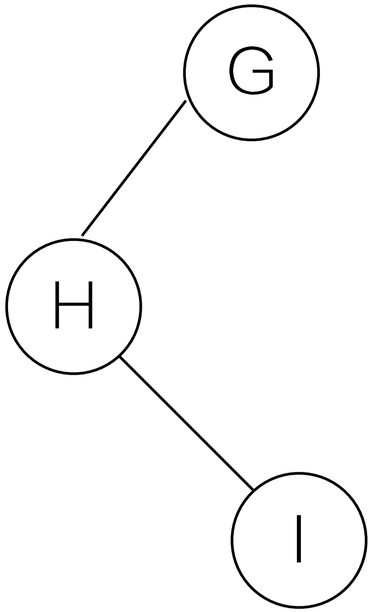




森林与二叉树的转换

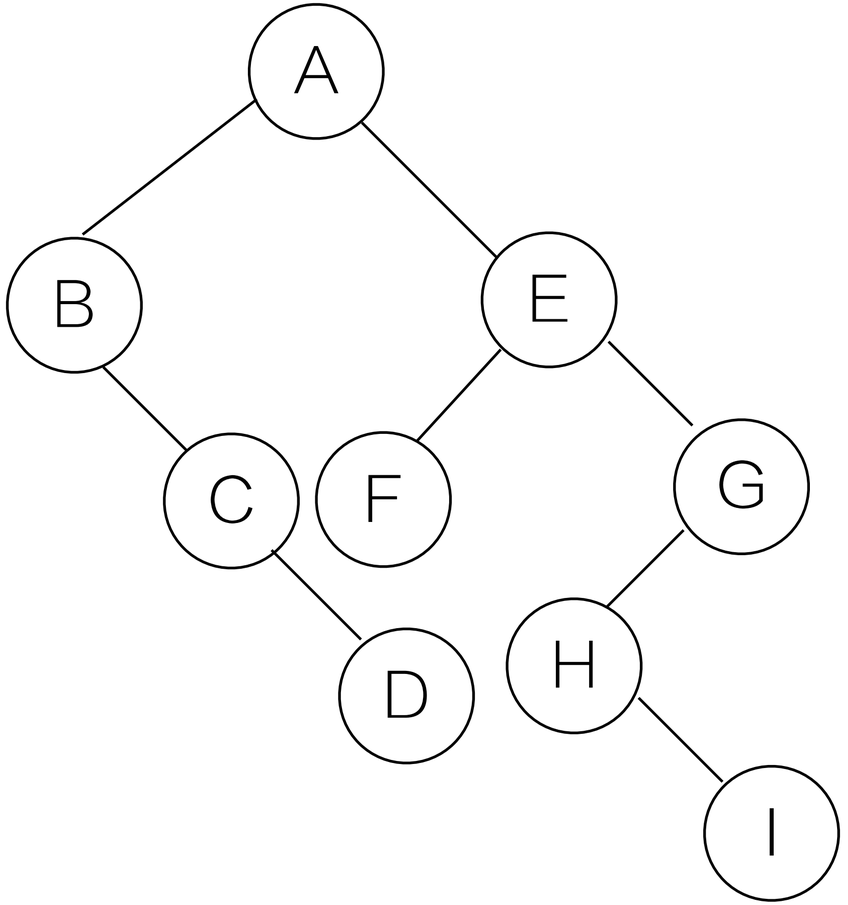
第一棵二叉树不动，从第二棵树开始，依次把后一棵二叉树的根结点作为前一棵二叉树的根结点的右孩子，然后用线连起来。



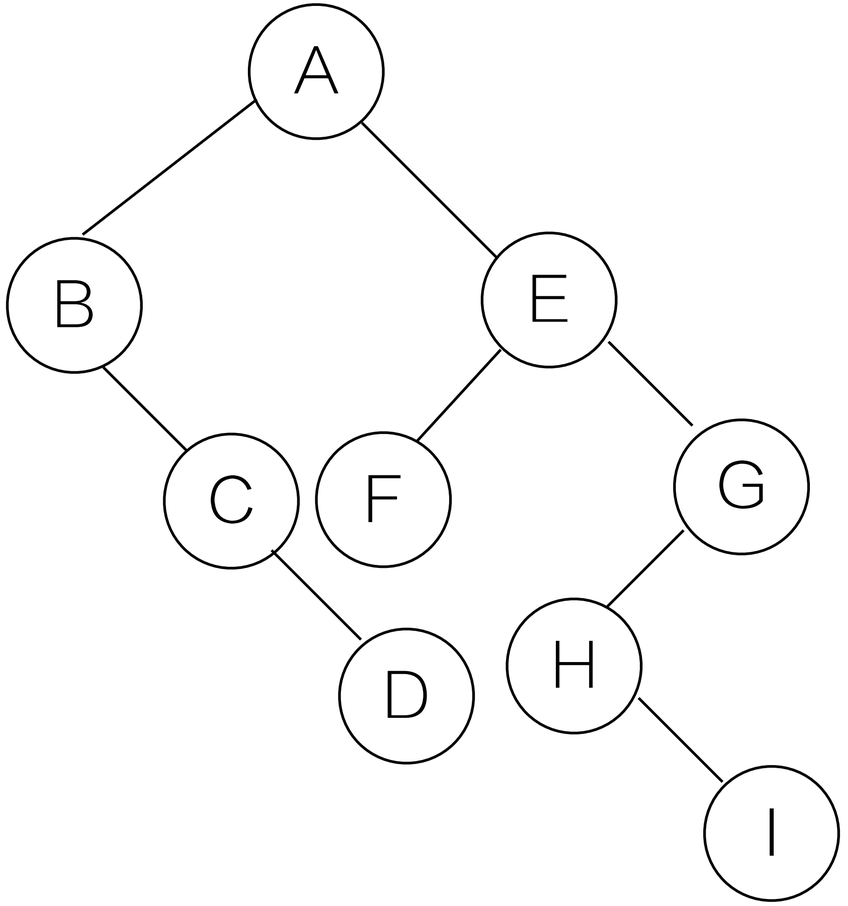


森林与二叉树的转换

第一棵二叉树不动，从第二棵树开始，依次把后一棵二叉树的根结点作为前一棵二叉树的根结点的右孩子，然后用线连起来。

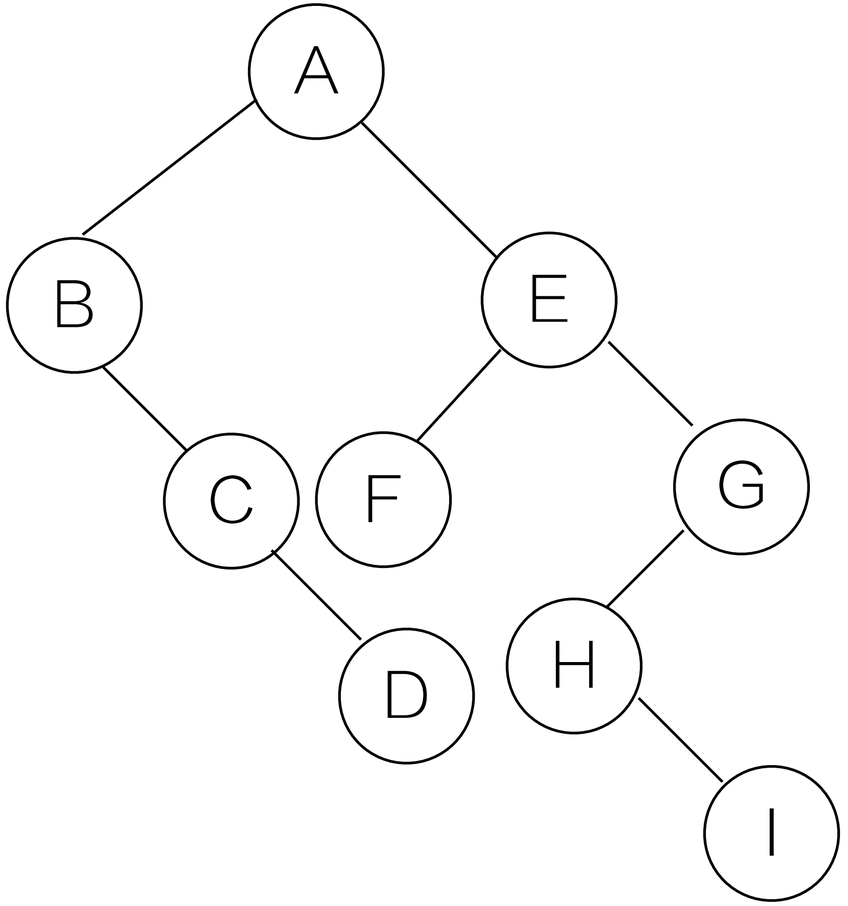


森林与二叉树的转换



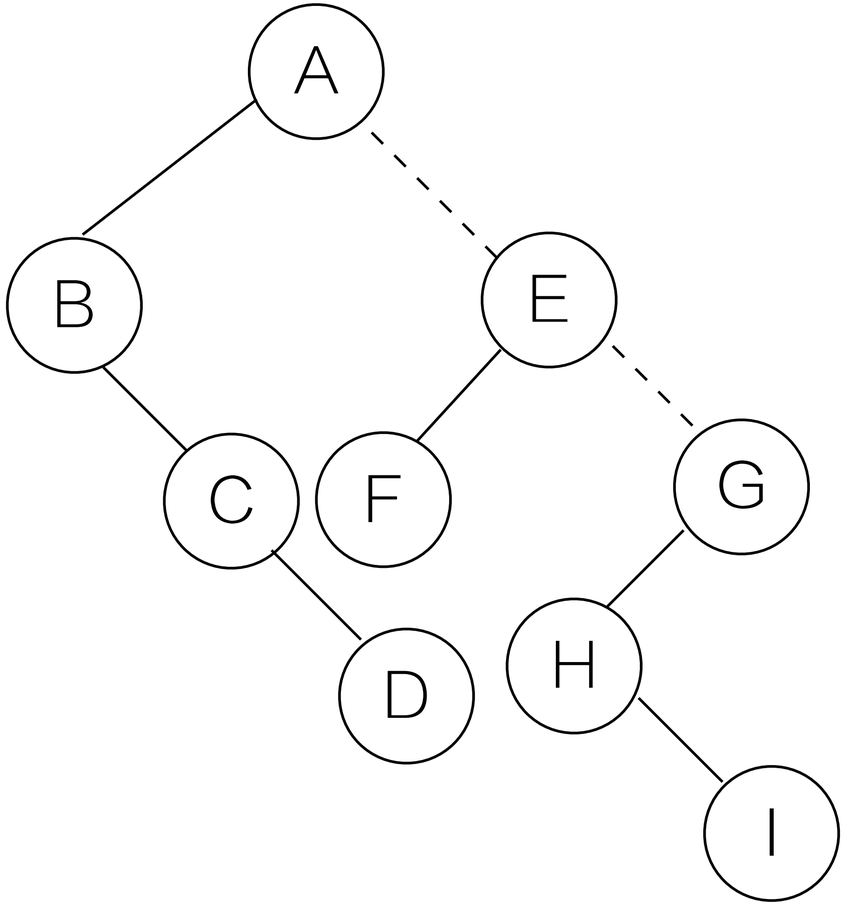
森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。



森林与二叉树的转换

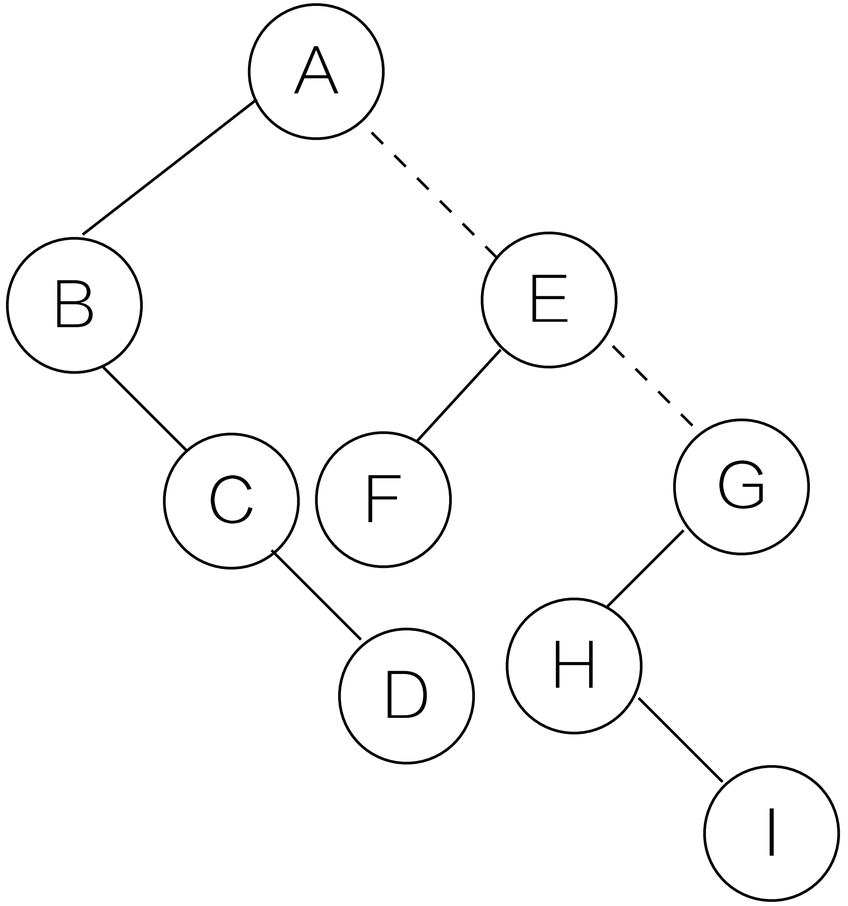
从根结点开始，若右孩子存在，则把右孩子结点的连线删除。



森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

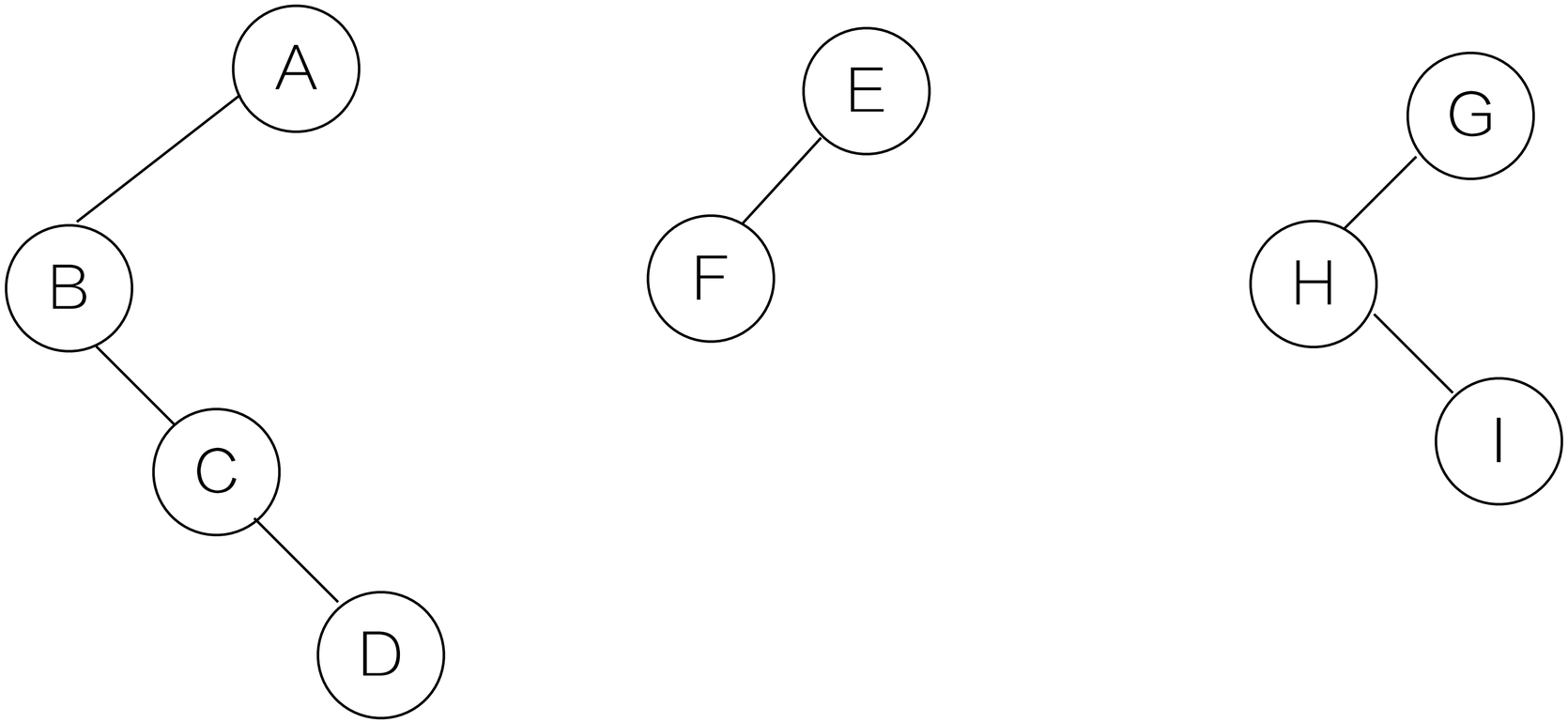
拆



森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

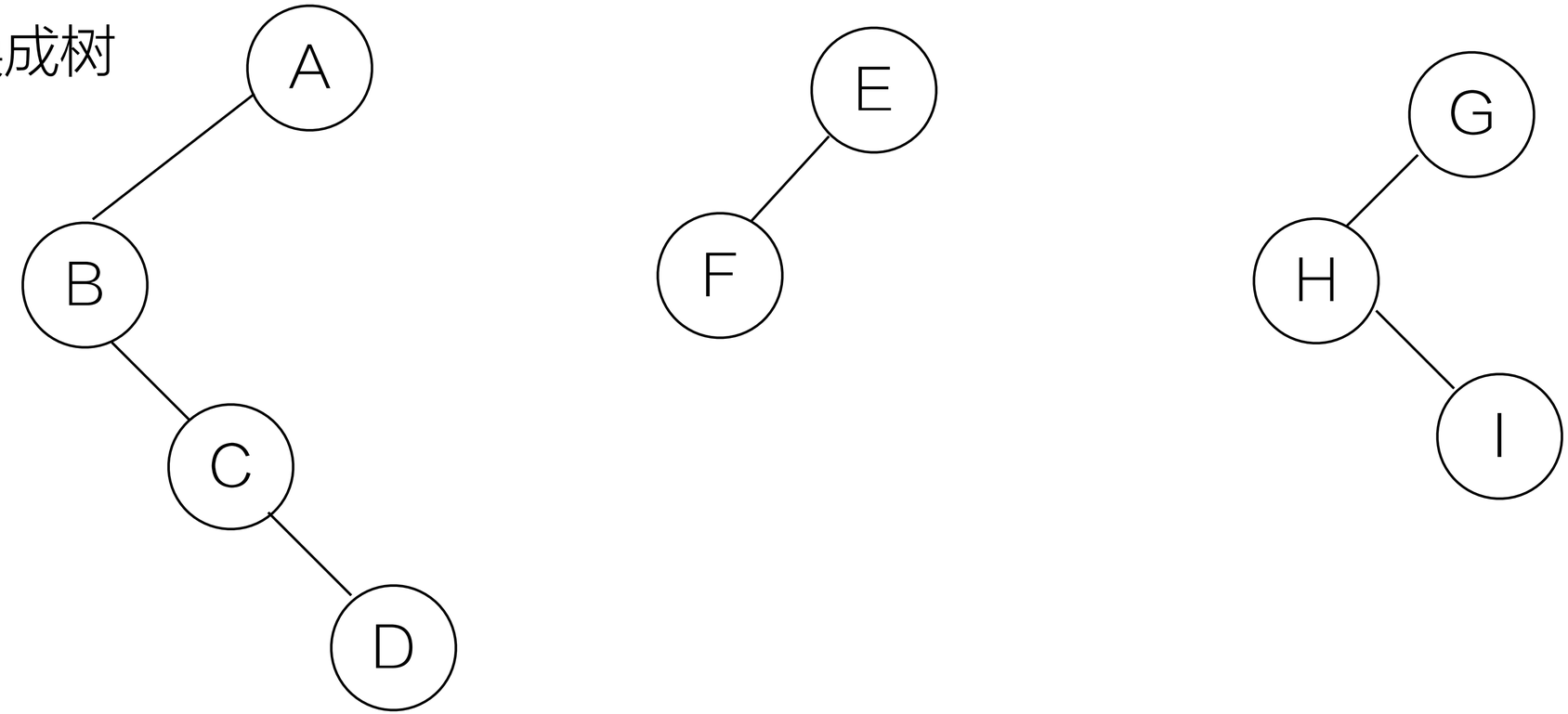


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

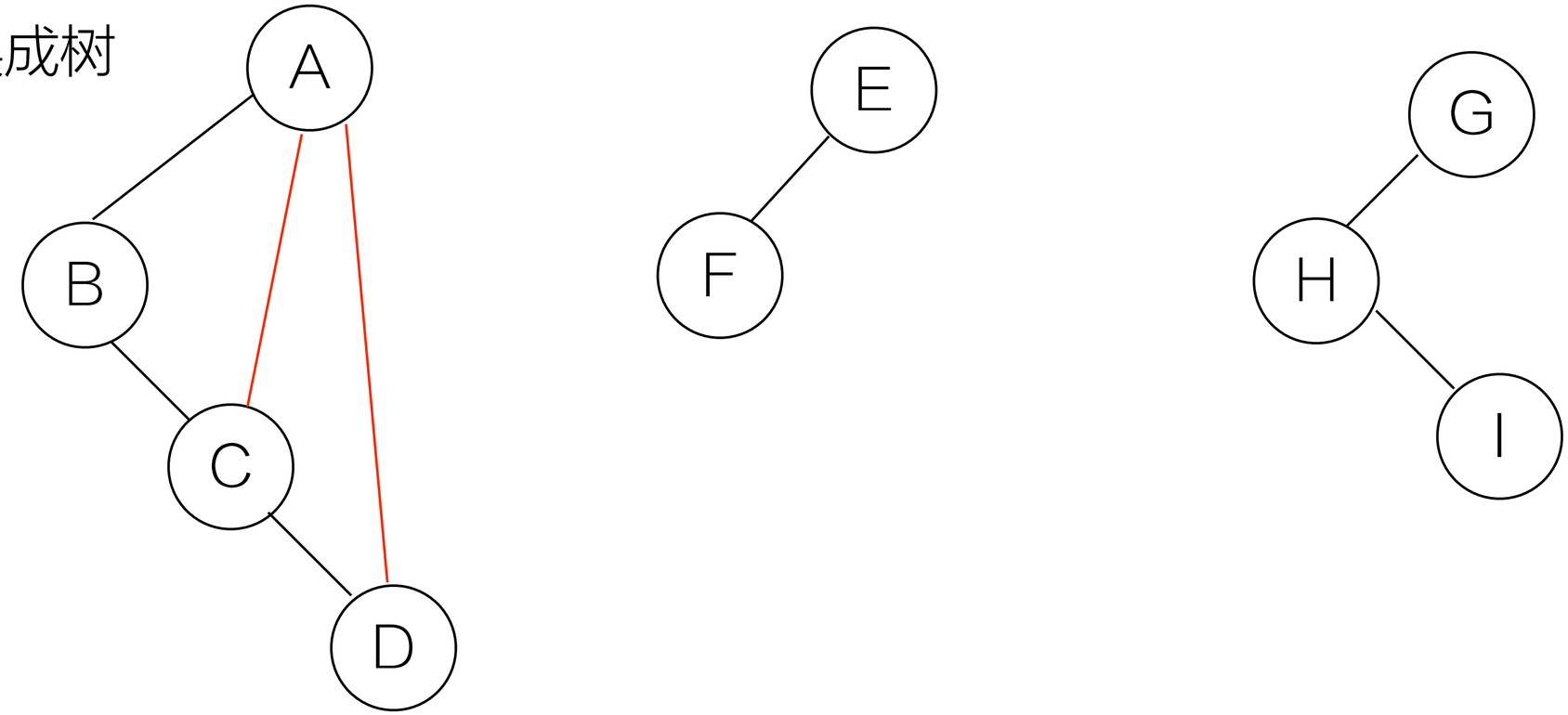


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

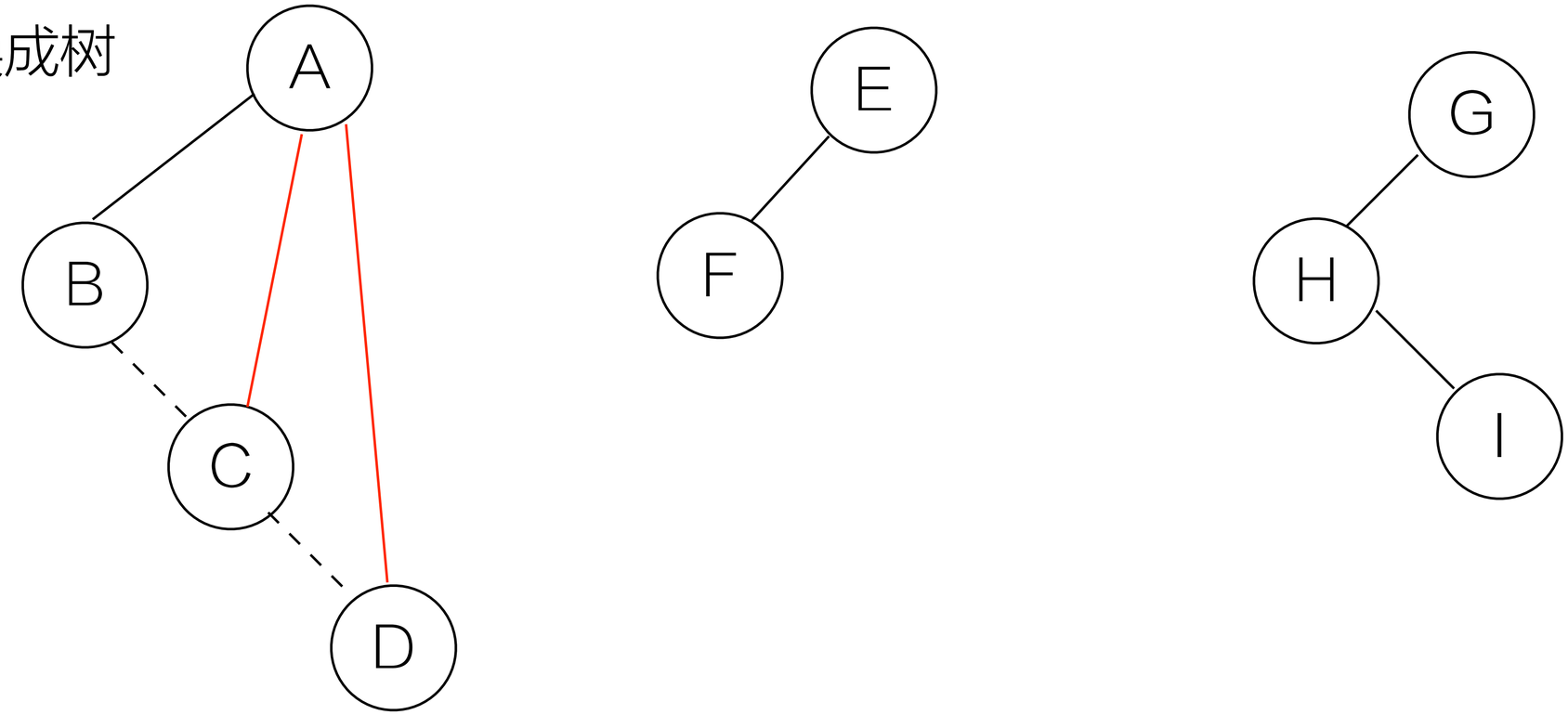


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

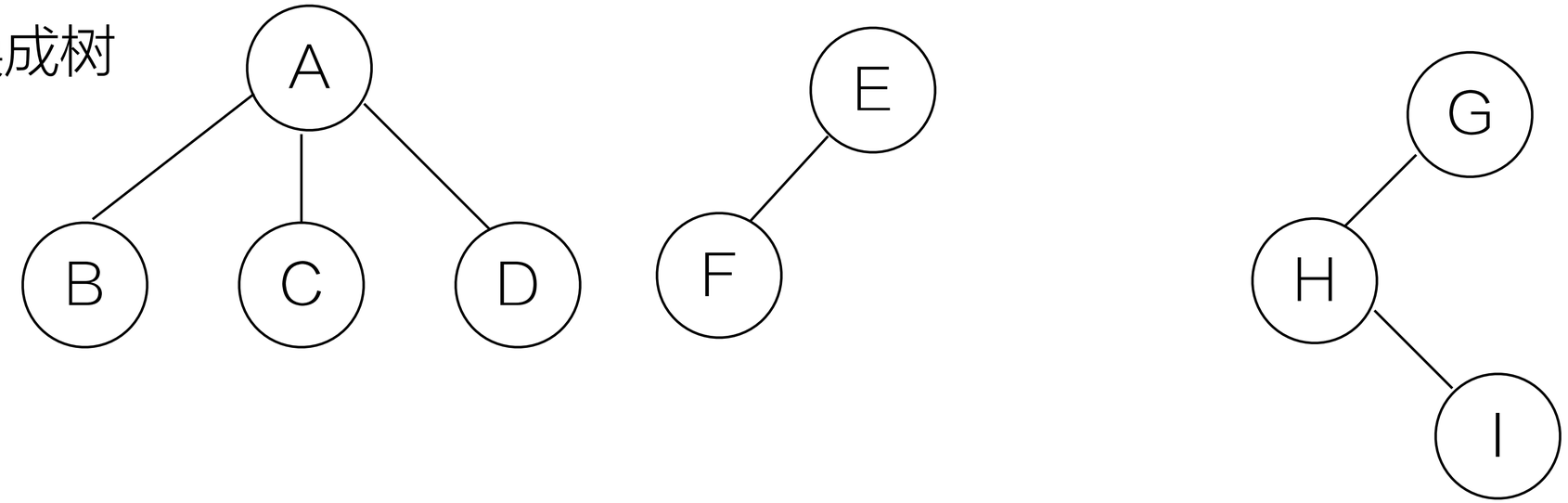


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

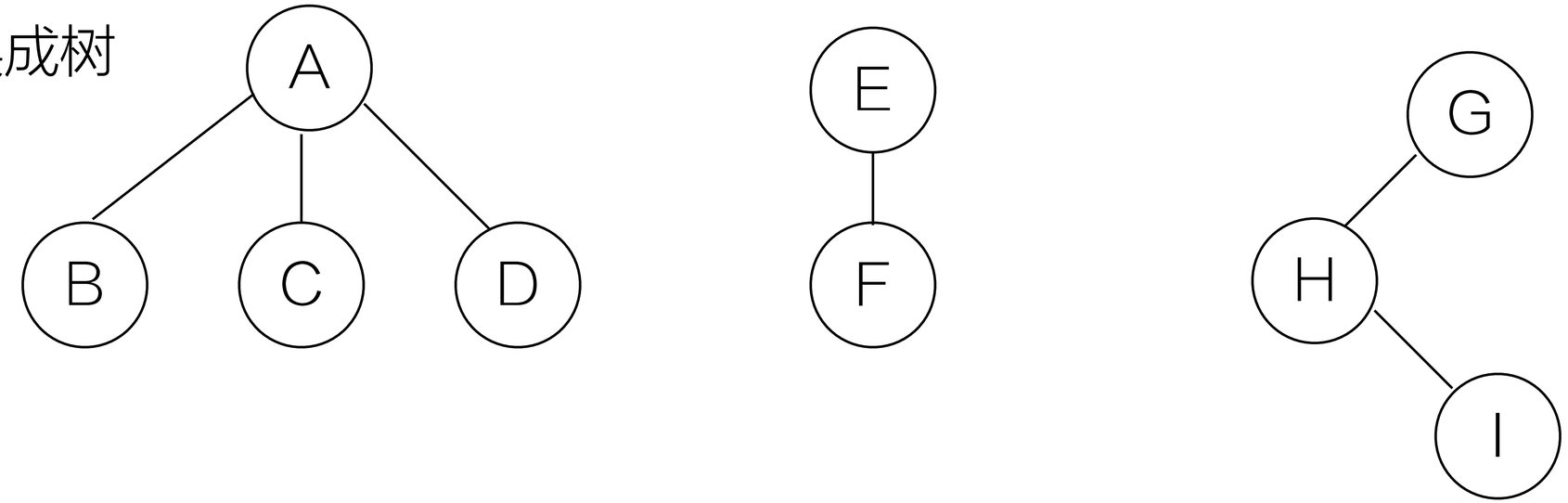


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

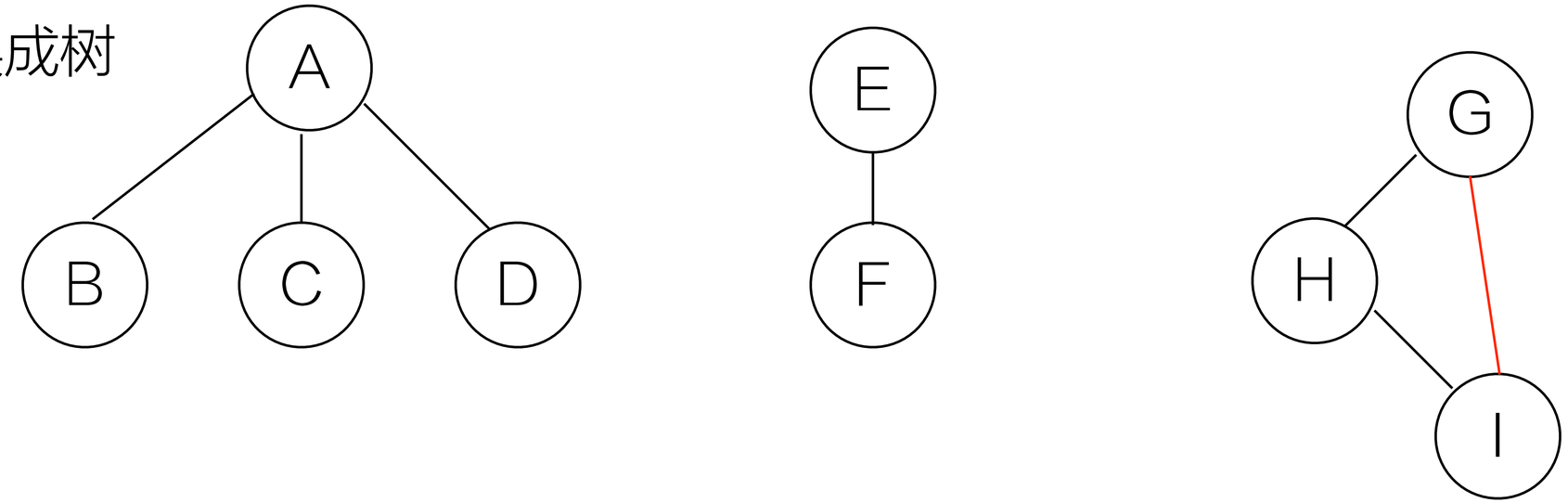


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

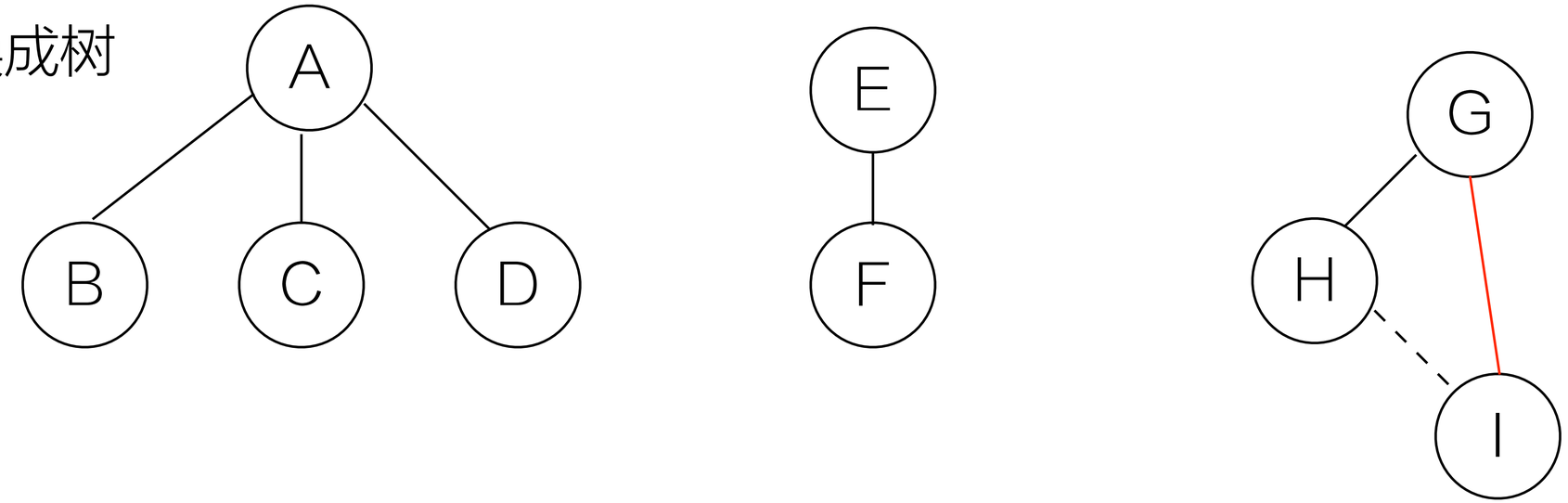


森林与二叉树的转换

从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

拆

二叉树转换成树

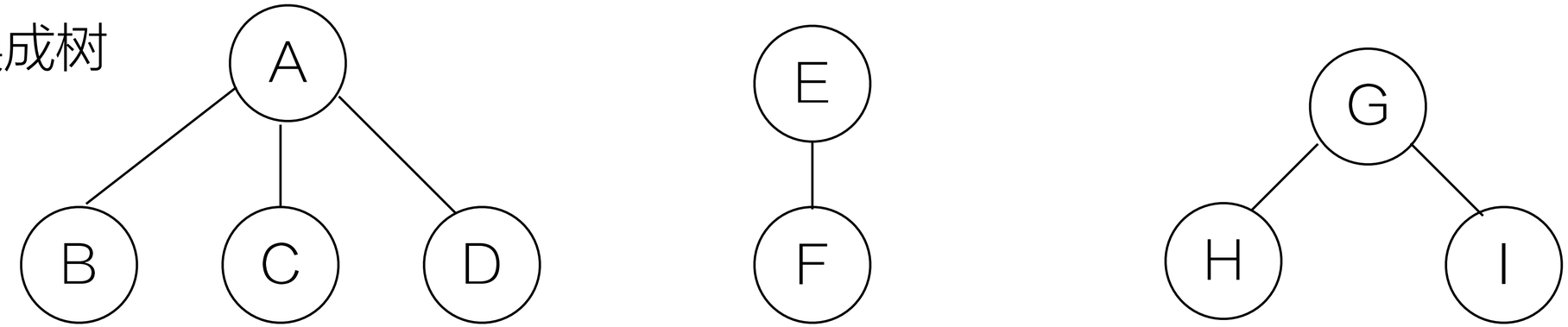


森林与二叉树的转换

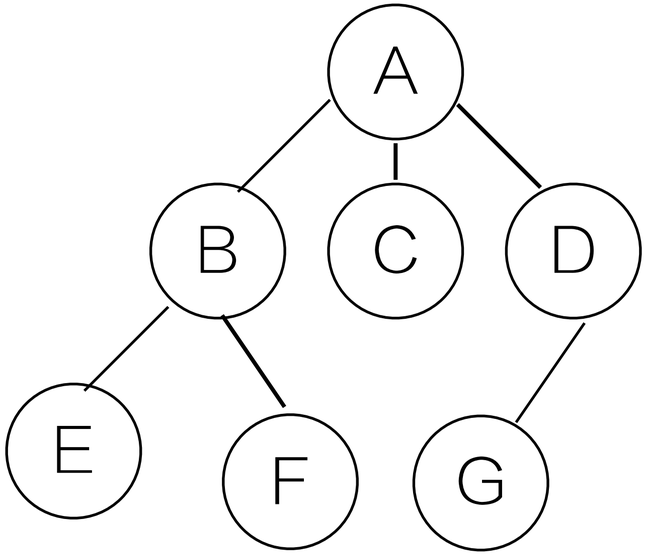
从根结点开始，若右孩子存在，则把右孩子结点的连线删除。

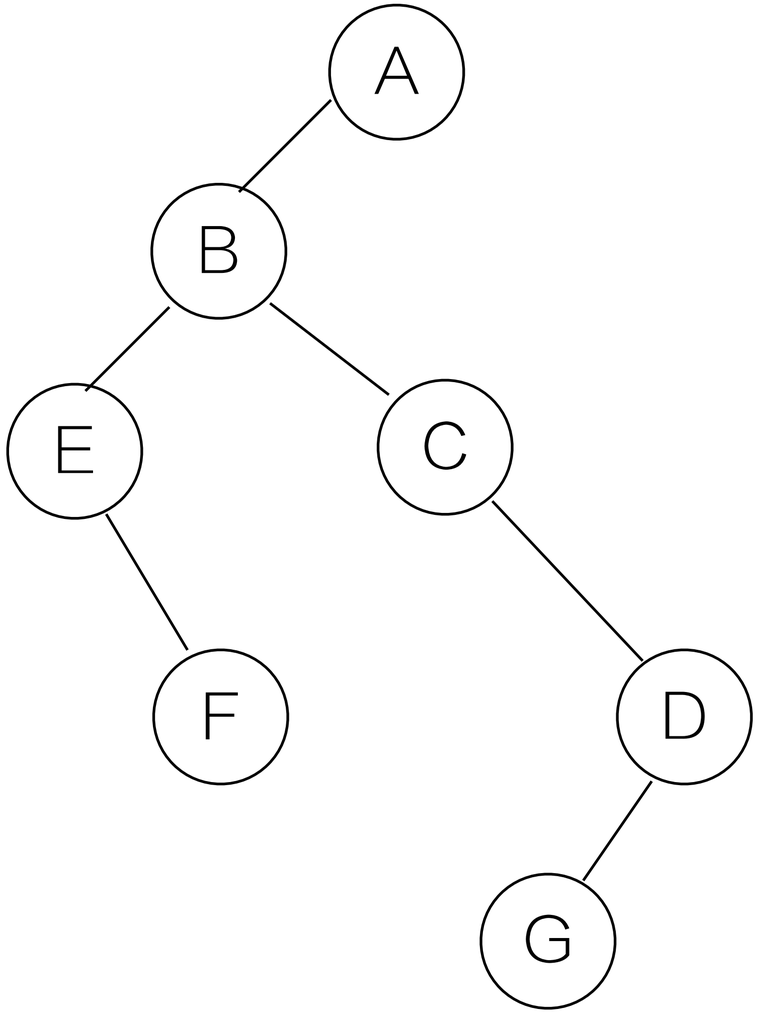
拆

二叉树转换成树



树的遍历





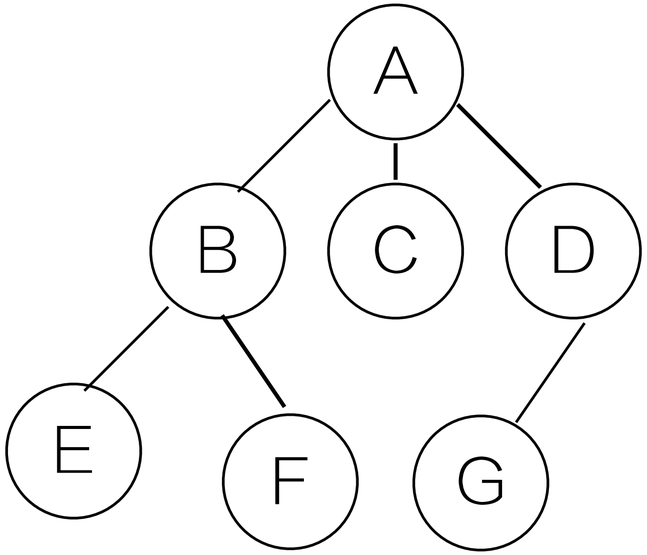
先根遍历：

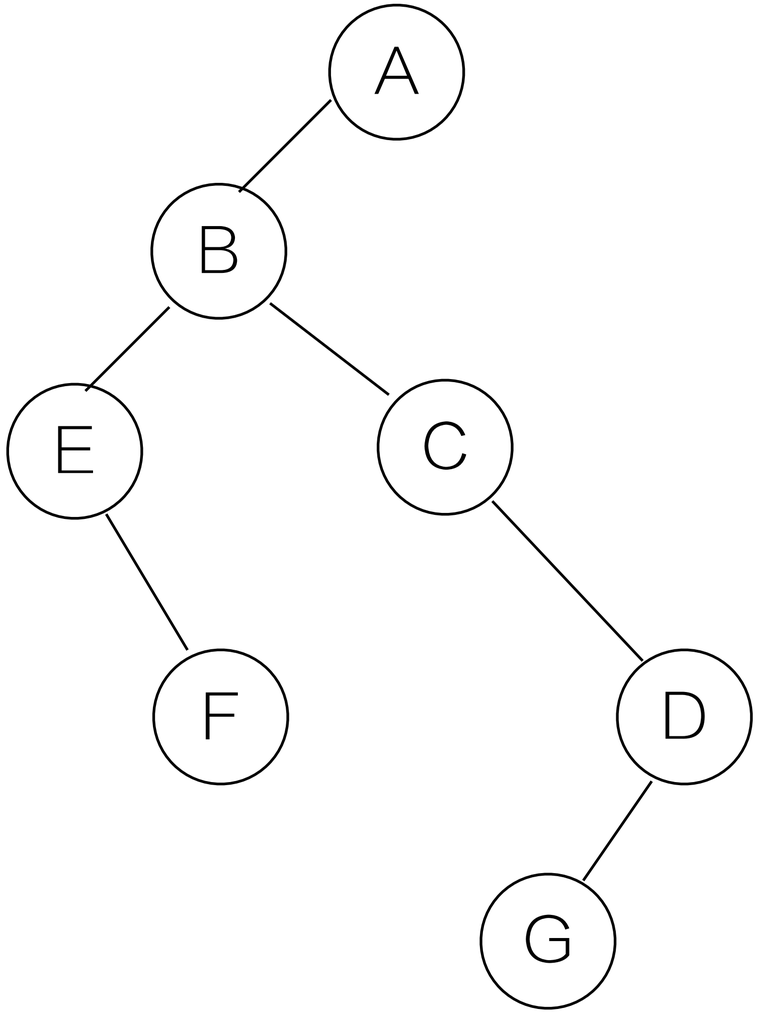
后根遍历：

前序遍历：

中序遍历：

树的遍历





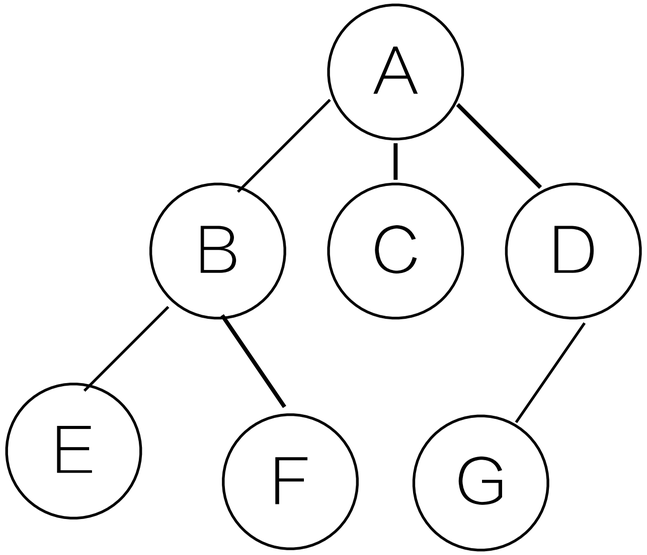
先根遍历：A B E F C D G

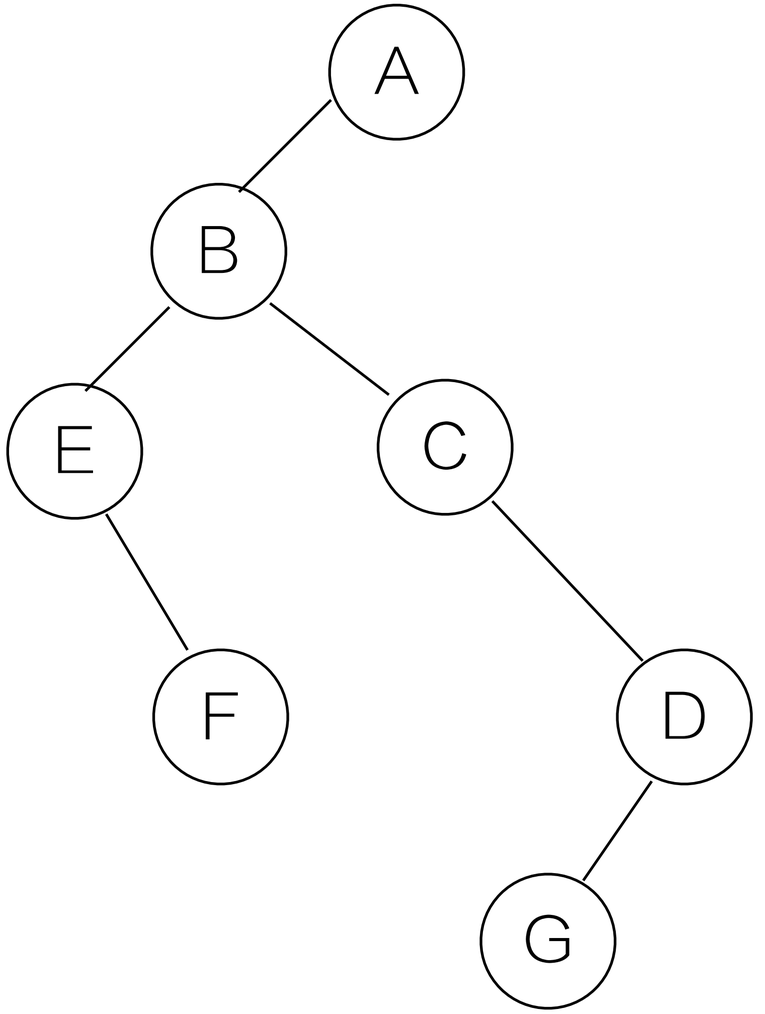
后根遍历：

前序遍历：

中序遍历：

树的遍历





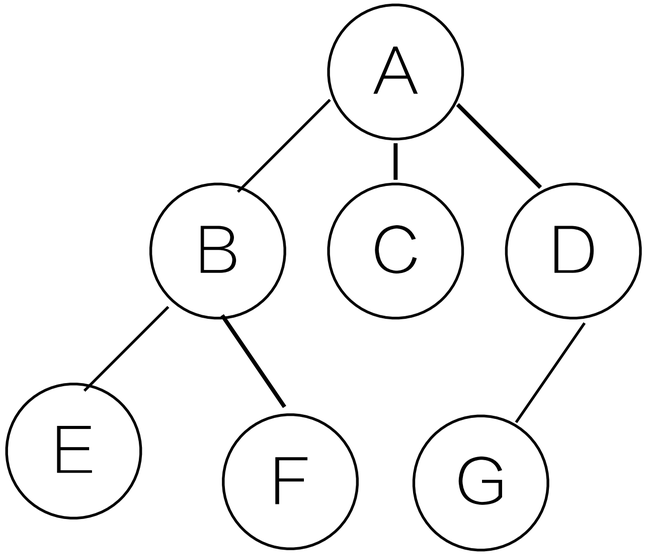
先根遍历：A B E F C D G

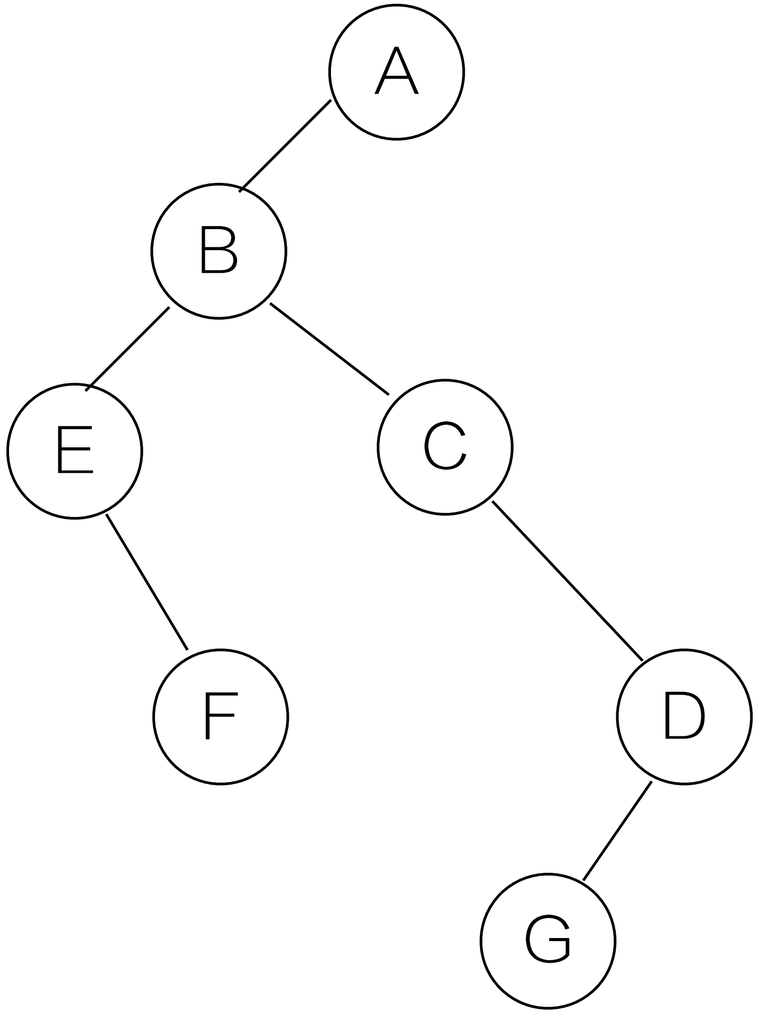
后根遍历：E F B C G D A

前序遍历：

中序遍历：

树的遍历





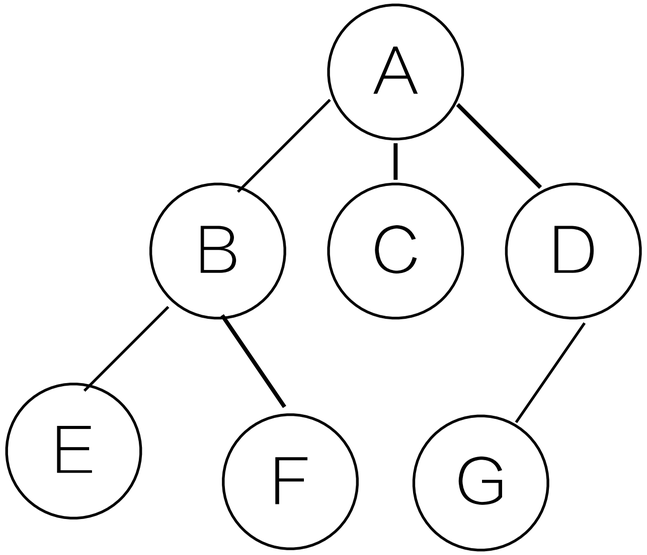
先根遍历：A B E F C D G

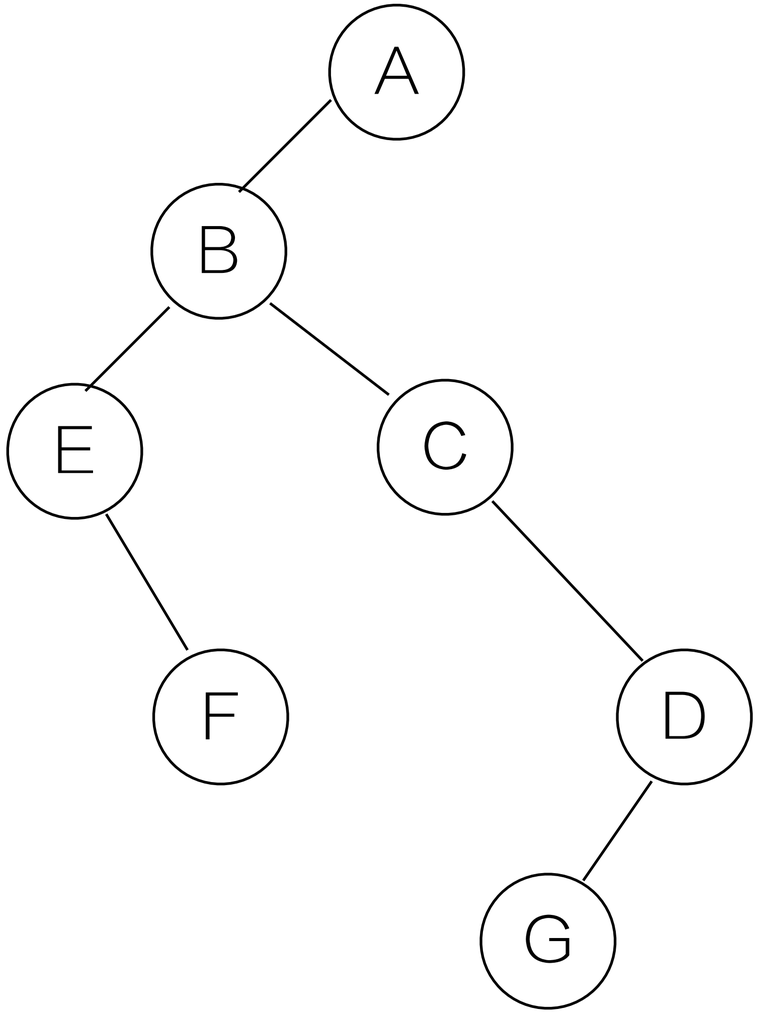
后根遍历：E F B C G D A

前序遍历：A B E F C D G

中序遍历：

树的遍历





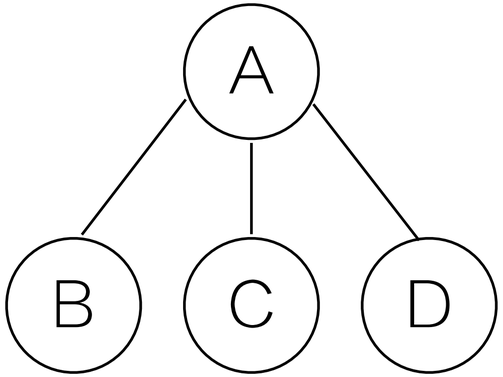
先根遍历：A B E F C D G

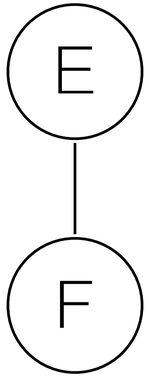
后根遍历：E F B C G D A

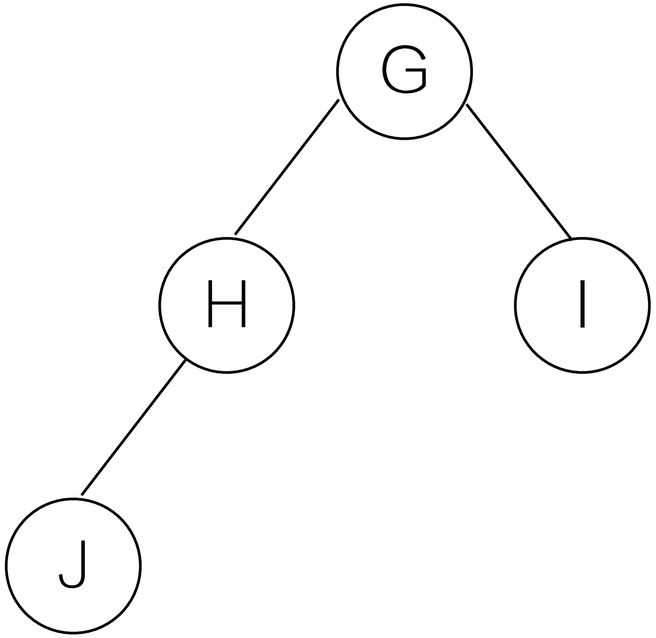
前序遍历：A B E F C D G

中序遍历：E F B C G D A

森林的遍历



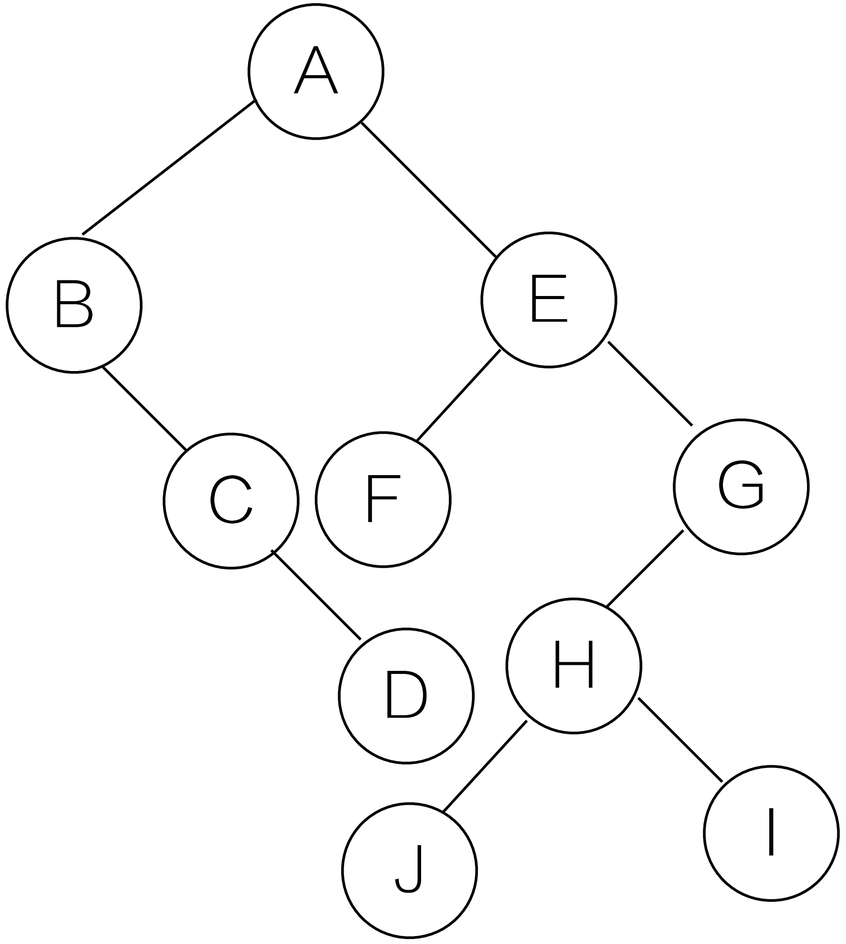




前序遍历：A B C D E F G H J I

中序遍历：B C D A F E J H I G

森林与二叉树的遍历



前序遍历：A B C D E F G H J I

中序遍历：B C D A F E J H I G

树、森林与二叉树的遍历关系

树	森林	二叉树
先根遍历	前序遍历	前序遍历
后根遍历	中序遍历	中序遍历

树、森林、二叉树 - 真题解析

[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

Ⅰ. 父子关系　　Ⅱ. 兄弟关系　　Ⅲ. u 的父结点与 v 的父结点是兄弟关系

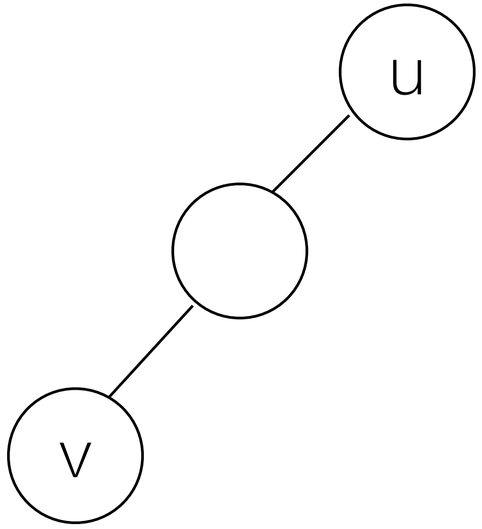
A. 只有 Ⅱ　　B. Ⅰ 和 Ⅱ　　C. Ⅰ 和 Ⅲ　　D. Ⅰ、Ⅱ 和 Ⅲ

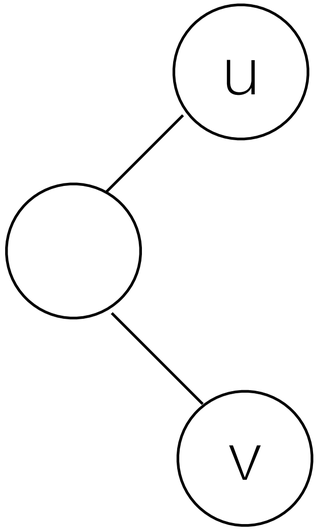
树、森林、二叉树 – 真题解析

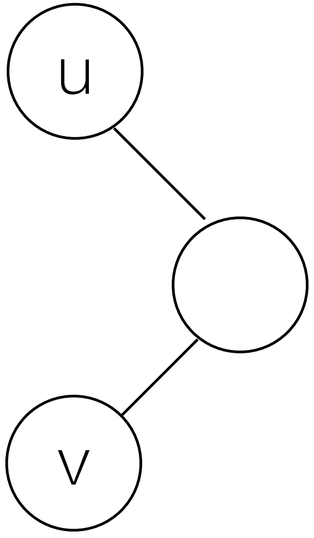
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

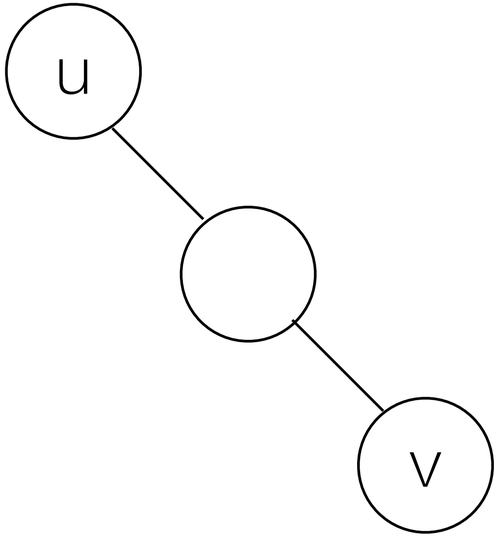
I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　　B. I 和 II　　C. I 和 III　　D. I、II 和 III







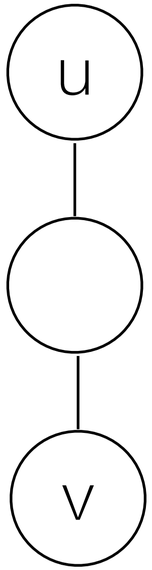


树、森林、二叉树 - 真题解析

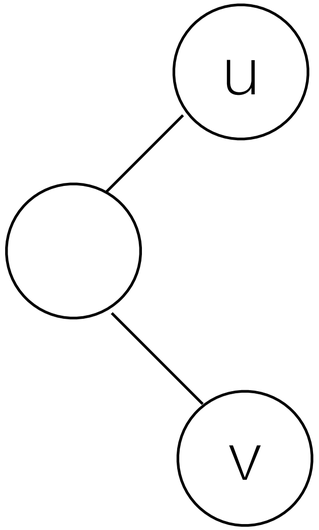
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

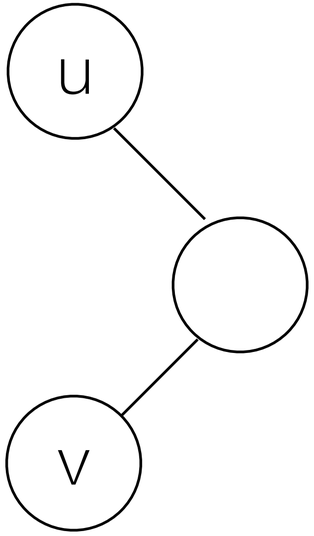
A. 只有 II



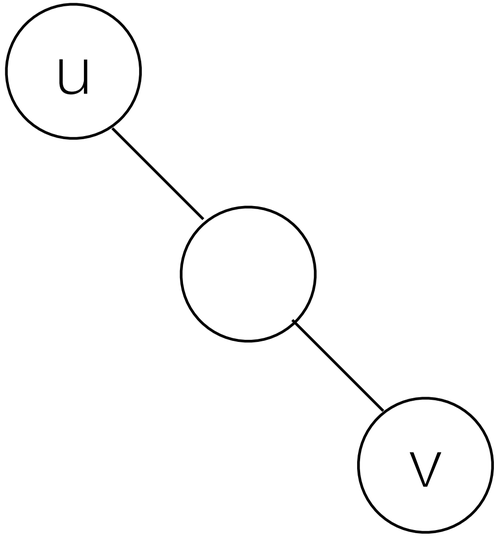
B. I 和 II



C. I 和 III



D. I、II 和 III

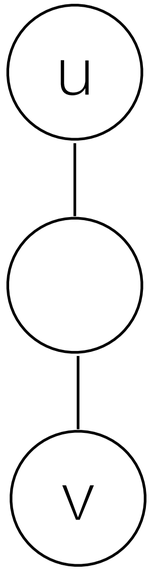


树、森林、二叉树 – 真题解析

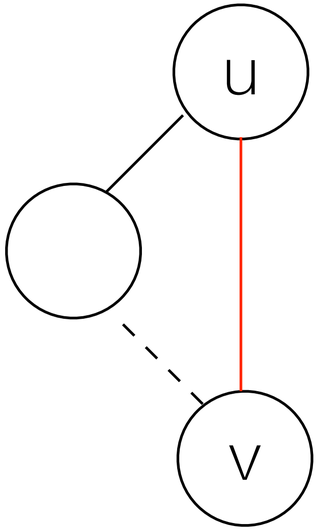
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

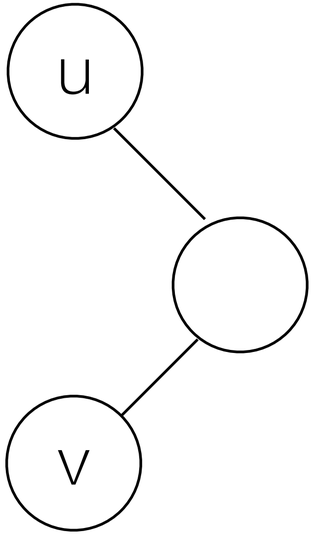
A. 只有 II



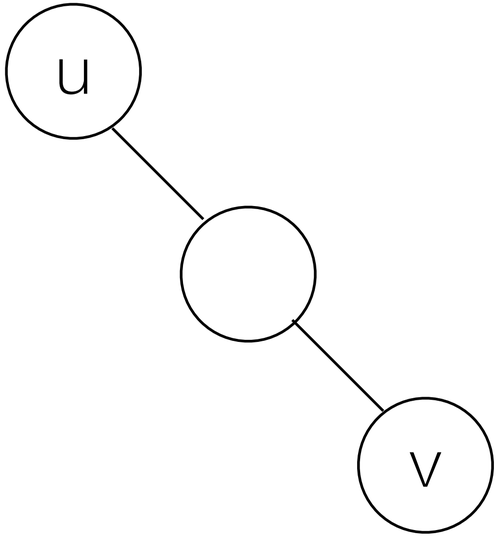
B. I 和 II



C. I 和 III



D. I、II 和 III

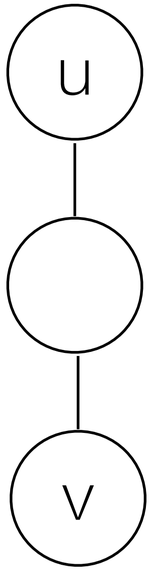


树、森林、二叉树 - 真题解析

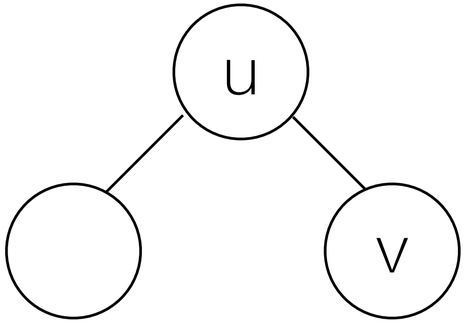
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

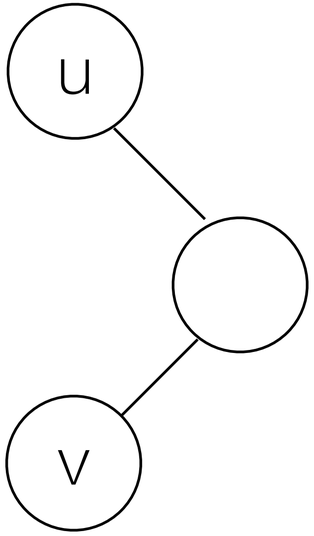
A. 只有 II



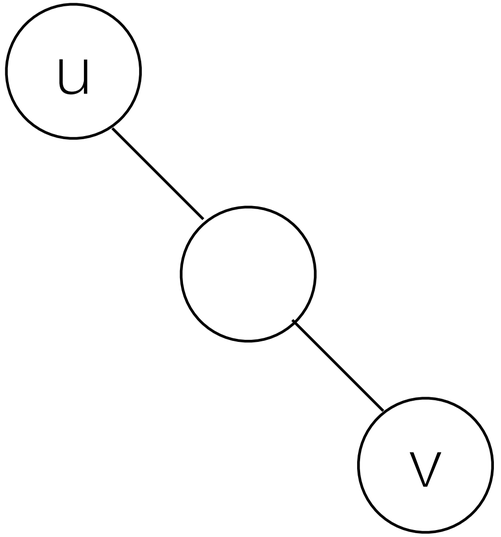
B. I 和 II



C. I 和 III



D. I、II 和 III

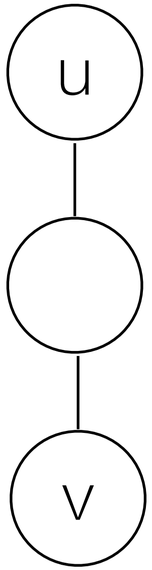


树、森林、二叉树 – 真题解析

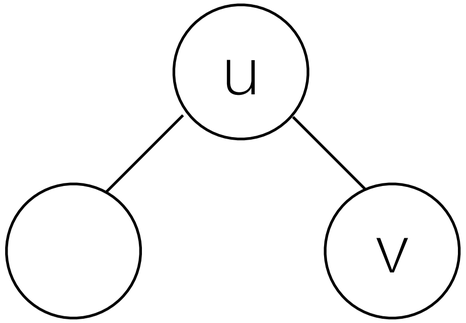
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

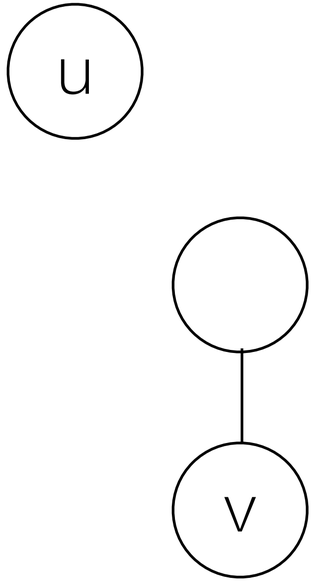
A. 只有 II



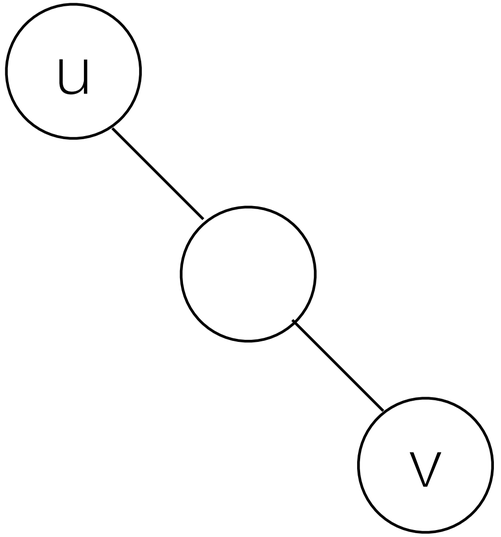
B. I 和 II



C. I 和 III



D. I、II 和 III

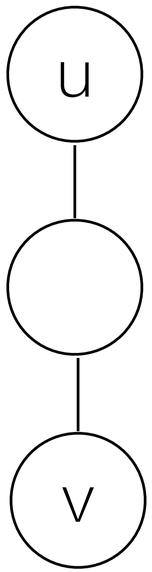


树、森林、二叉树 – 真题解析

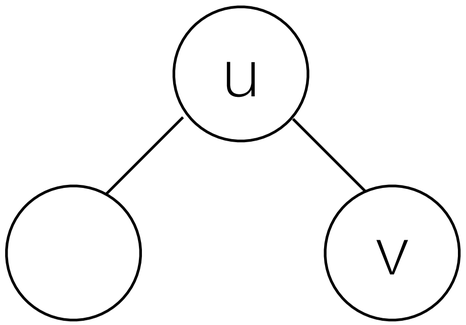
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

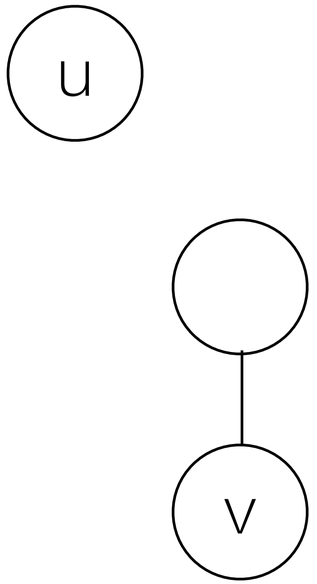
A. 只有 II



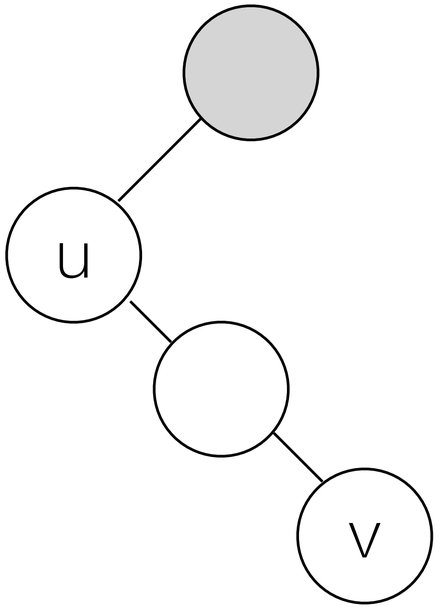
B. I 和 II



C. I 和 III



D. I、II 和 III

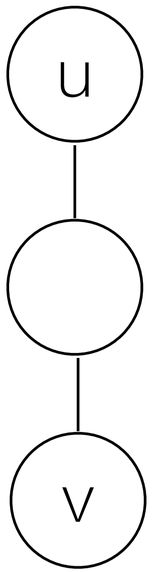


树、森林、二叉树 – 真题解析

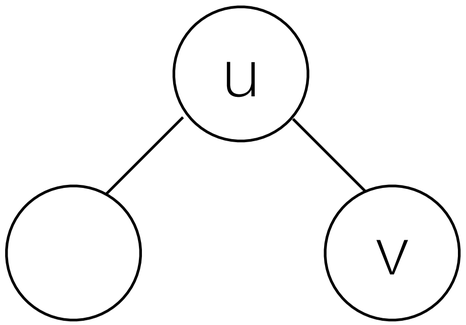
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

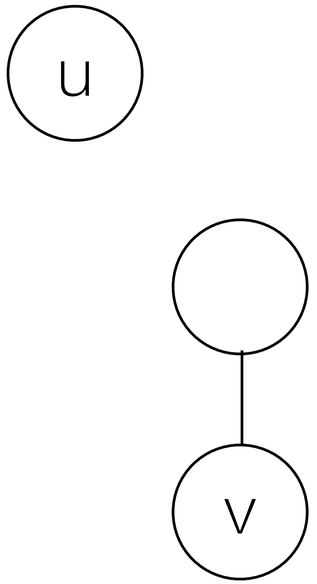
A. 只有 II



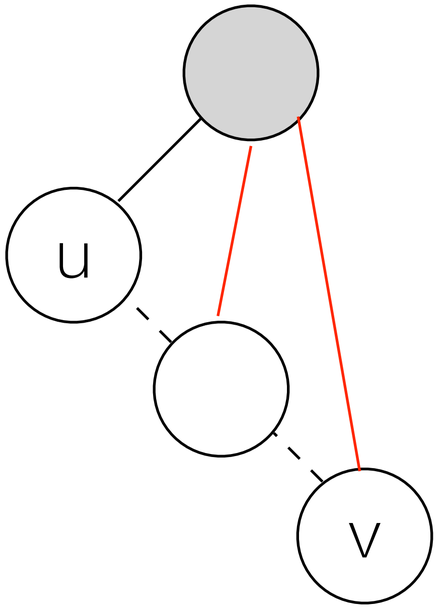
B. I 和 II



C. I 和 III



D. I、II 和 III

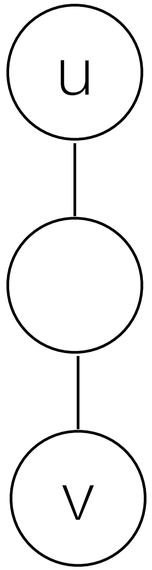


树、森林、二叉树 – 真题解析

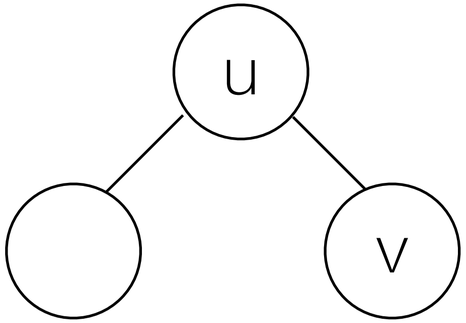
[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

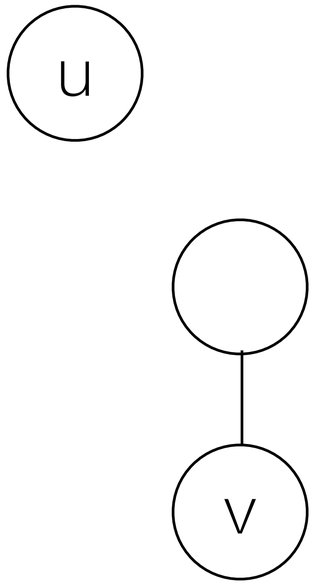
A. 只有 II



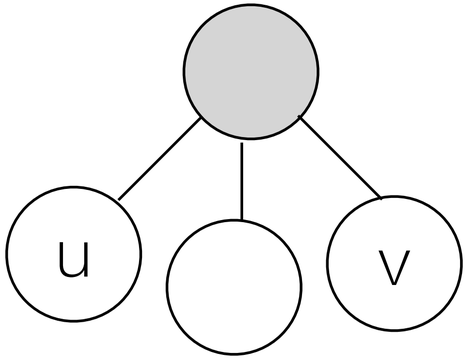
B. I 和 II



C. I 和 III



D. I、II 和 III



树、森林、二叉树 - 真题解析

[2009] 6. 将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 父结点的父结点，则在原来的森林中，u 和 v可能具有的关系是_____。

I. 父子关系　　II. 兄弟关系　　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　　B. I 和 II　　C. I 和 III　　D. I、II 和 III

树、森林、二叉树 – 真题解析

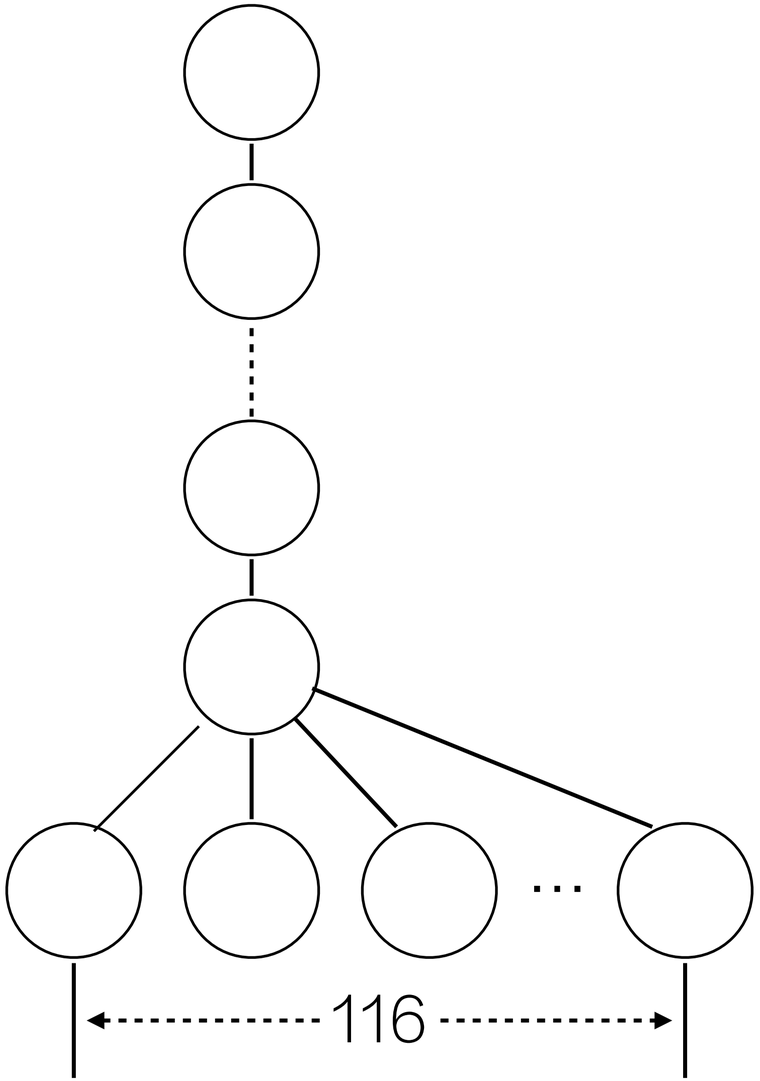
[2011] 6. 已知一棵有 2011 个结点的树，其叶结点个数为 116 ， 该树对应的二叉树中无右孩子的结点个数是_____。

A. 115　　B. 116　　C.1895　　D. 1896

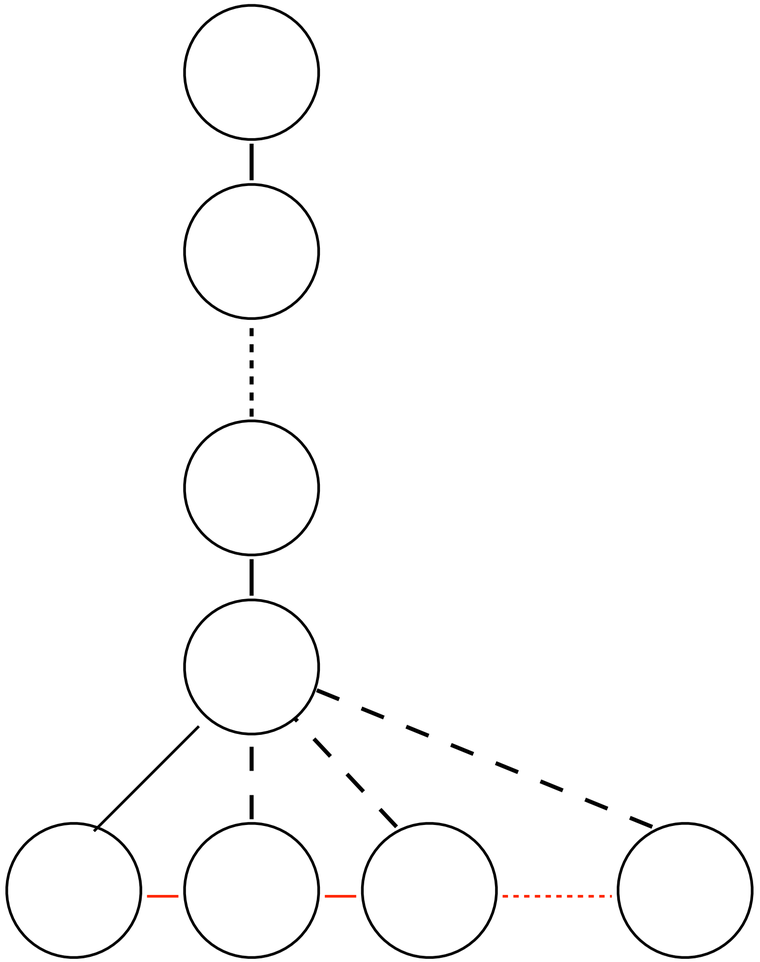
[2011] 6. 已知一棵有 2011 个结点的树，其叶结点个数为 116 ， 该树对应的二叉树中无右孩子的结点个数是_____。

A. 115　　B. 116　　C.1895　　D. 1896

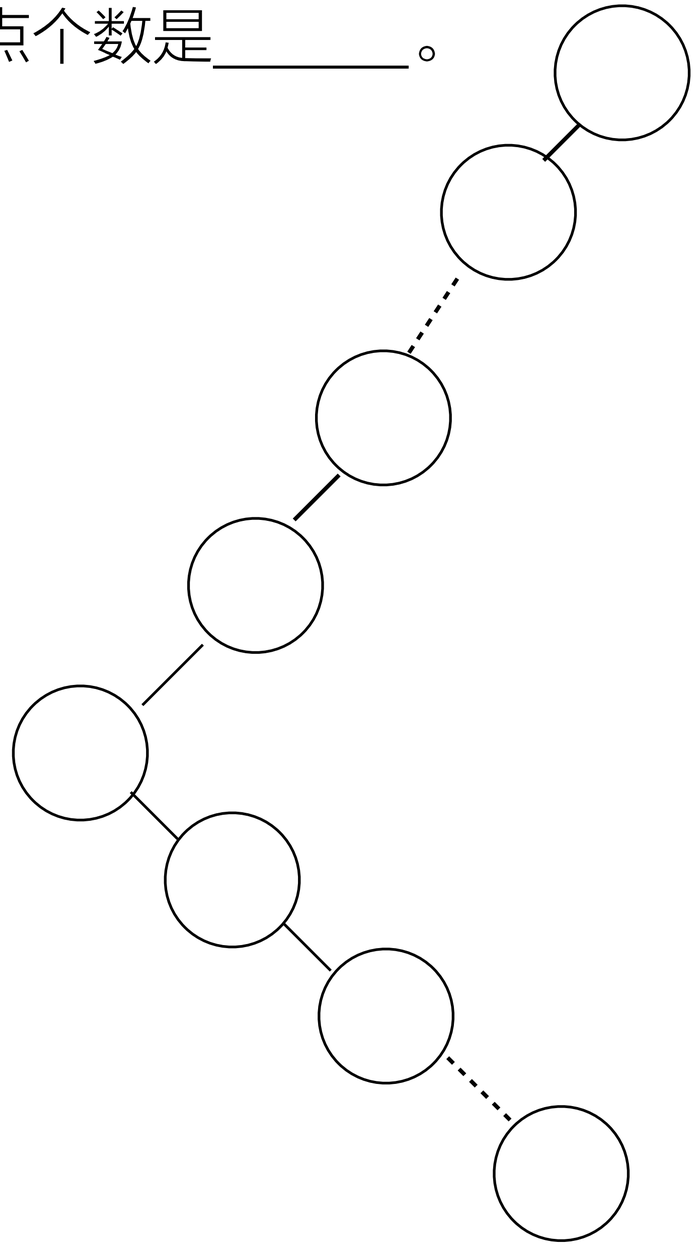
[2011] 6. 已知一棵有 2011 个结点的树，其叶结点个数为 116 ， 该树对应的二叉树中无右孩子的结点个数是_____。



[2011] 6. 已知一棵有 2011 个结点的树，其叶结点个数为 116 ， 该树对应的二叉树中无右孩子的结点个数是_____。



[2011] 6. 已知一棵有 2011 个结点的树，其叶结点个数为 116 ， 该树对应的二叉树中无右孩子的结点个数是_____。



树、森林、二叉树 – 真题解析

[2011] 6. 已知一棵有 2011 个结点的树，其叶结点个数为 116 ， 该树对应的二叉树中无右孩子的结点个数是_____。

A. 115　　B. 116　　C.1895　　D. 1896

树、森林、二叉树 – 真题解析

[2014] 5. 将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于_____。

A. T 中叶结点的个数

B. T 中度为 1 的结点个数

C. T 中左孩子指针为空的结点个数

D. T 中右孩子指针为空的结点个数

[2014] 5. 将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于_____。

A. T 中叶结点的个数

B. T 中度为 1 的结点个数

C. T 中左孩子指针为空的结点个数

D. T 中右孩子指针为空的结点个数

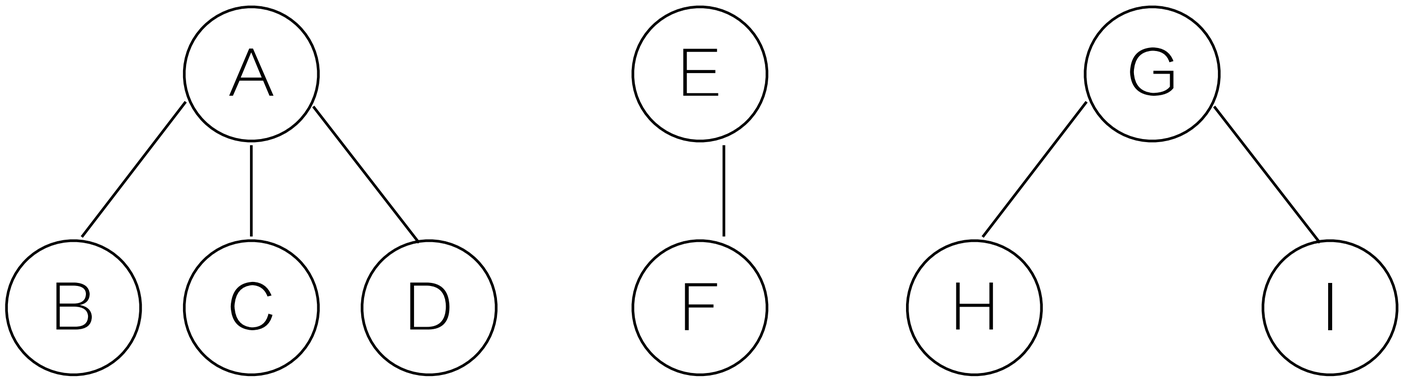
[2014] 5. 将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于_____。

A. T 中叶结点的个数

B. T 中度为 1 的结点个数

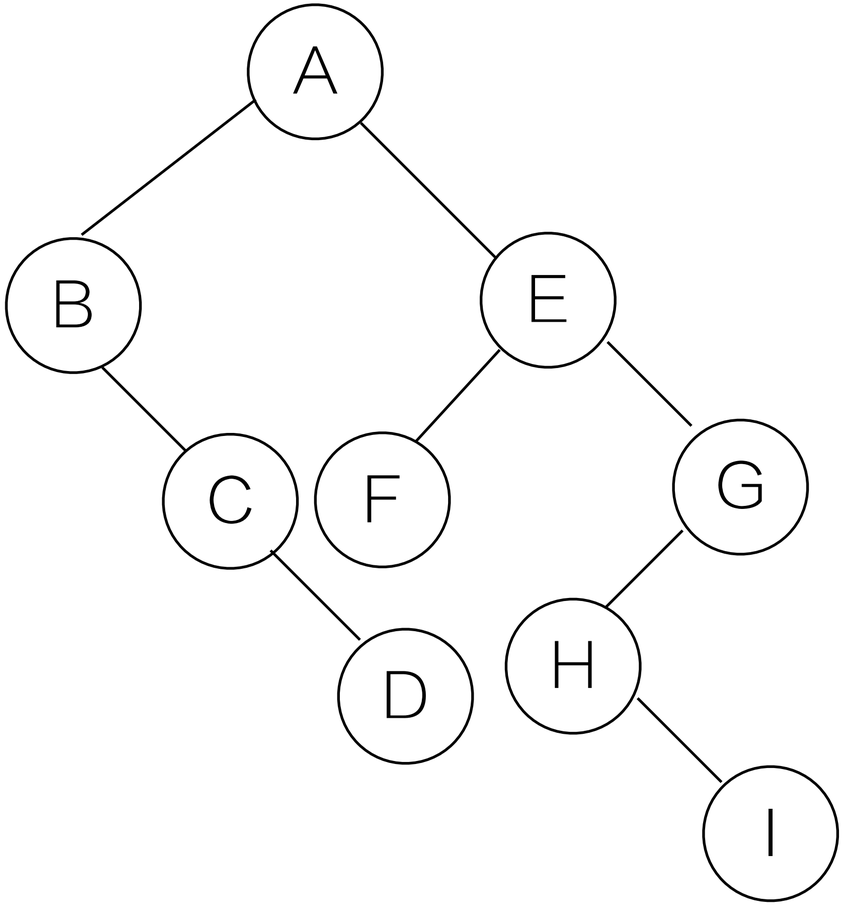
C. T 中左孩子指针为空的结点个数

D. T 中右孩子指针为空的结点个数



F

T



树、森林、二叉树 – 真题解析

[2014] 5. 将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于_____。

A. T 中叶结点的个数

B. T 中度为 1 的结点个数

C. T 中左孩子指针为空的结点个数

D. T 中右孩子指针为空的结点个数

树、森林、二叉树 – 真题解析

[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

树、森林、二叉树 – 真题解析

[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

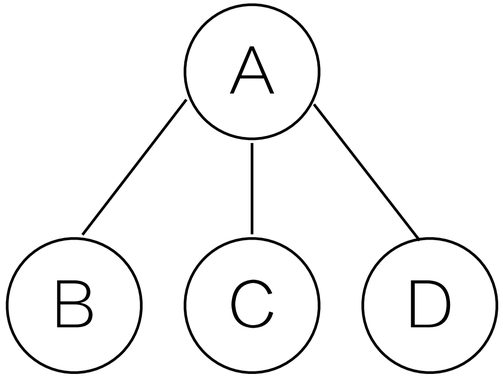
随便画森林观察森林中每棵树边数与结点数的关系

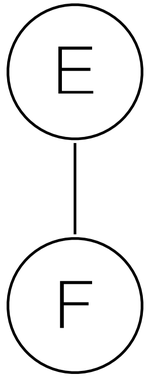
树、森林、二叉树 – 真题解析

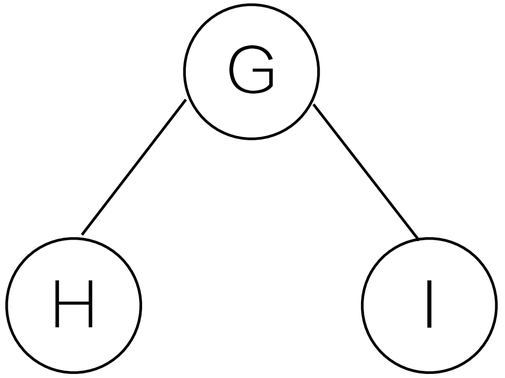
[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

随便画森林观察森林中每棵树边数与结点数的关系





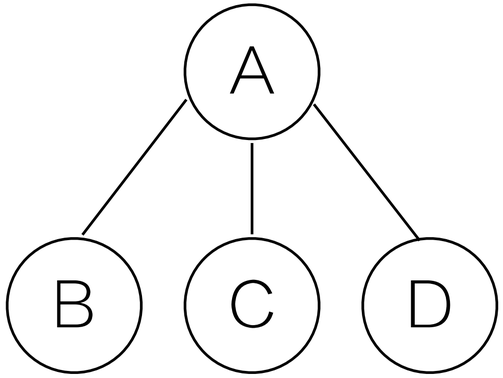


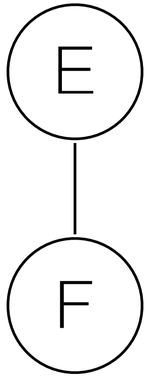
树、森林、二叉树 – 真题解析

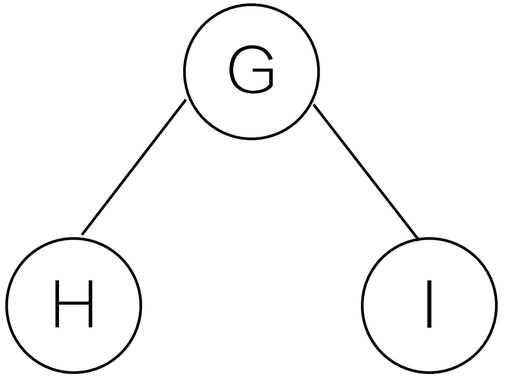
[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

随便画森林观察森林中每棵树边数与结点数的关系







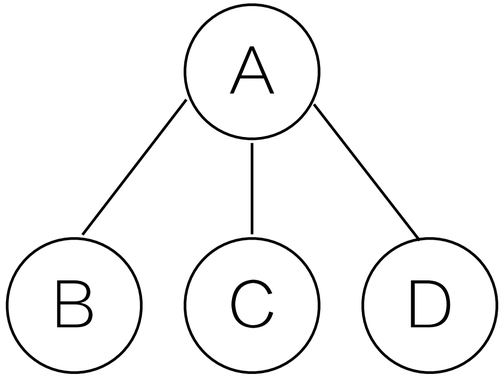
边：3　　结点：4

树、森林、二叉树 – 真题解析

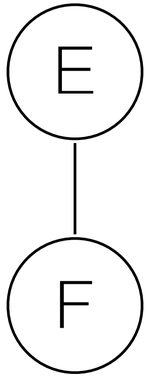
[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

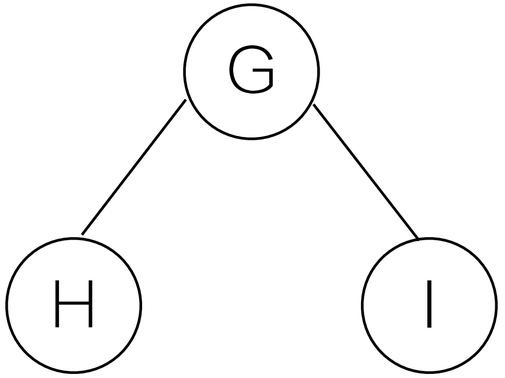
随便画森林观察森林中每棵树边数与结点数的关系



边：3　　结点：4



边：1　　结点：2

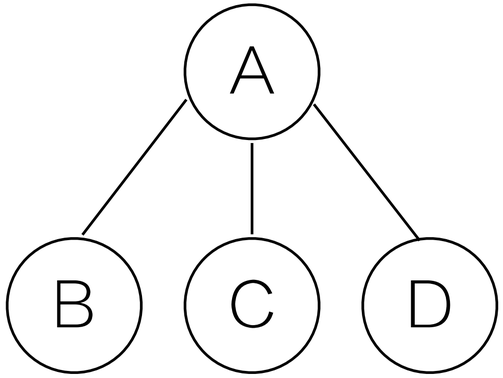


树、森林、二叉树 – 真题解析

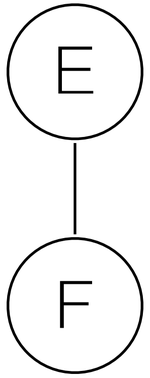
[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

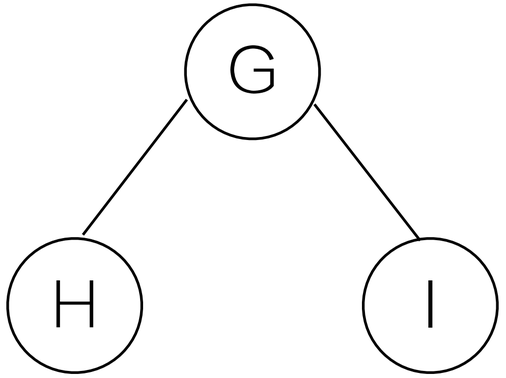
随便画森林观察森林中每棵树边数与结点数的关系



边：3　　结点：4



边：1　　结点：2



边：2　　结点：3

树、森林、二叉树 – 真题解析

[2016] 5. 若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是_____。

A. 8　　B. 9　　C. 10　　D. 11

树、森林、二叉树 – 真题解析

[2019] 2. 若将一棵树 T 转化为对应的二叉树 BT ，则下列对 BT 的遍历中，其遍历序列与 T 的后根遍历序列相同的是_____。

A. 先序遍历　　B. 中序遍历　　C. 后序遍历　　D. 按层遍历

树、森林、二叉树 – 真题解析

[2019] 2. 若将一棵树 T 转化为对应的二叉树 BT ，则下列对 BT 的遍历中，其遍历序列与 T 的后根遍历序列相同的是_____。

A. 先序遍历　　B. 中序遍历　　C. 后序遍历　　D. 按层遍历

树、森林、二叉树 – 真题解析

[2021] 4. 某森林 F 对应的二叉树为 T ， 若 T 的先序遍历序列是 a, b, d, c, e, g, f， 中序遍历序列是 b, d, a, e, g, c, f， 则 F 中树的棵数是_____。

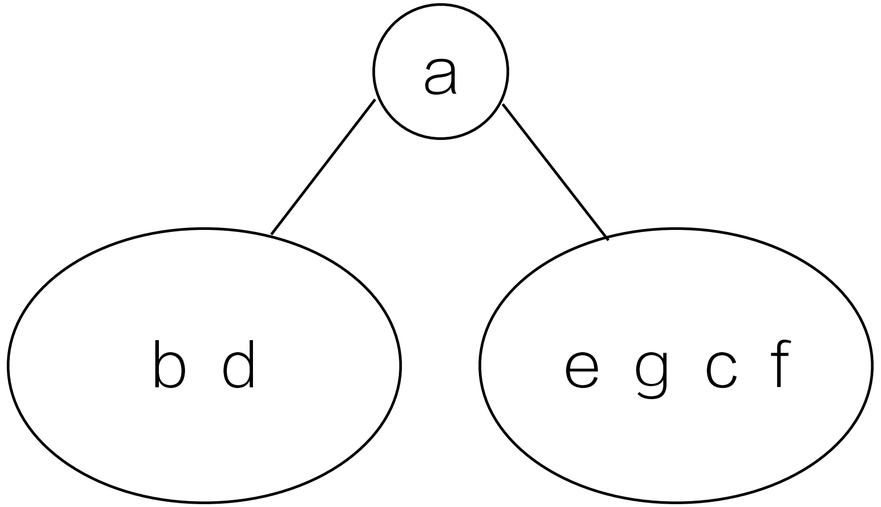
A. 1　　B. 2　　C. 3　　D. 4

[2021] 4. 某森林 F 对应的二叉树为 T ，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是_____。

A. 1　　B. 2　　C. 3　　D. 4

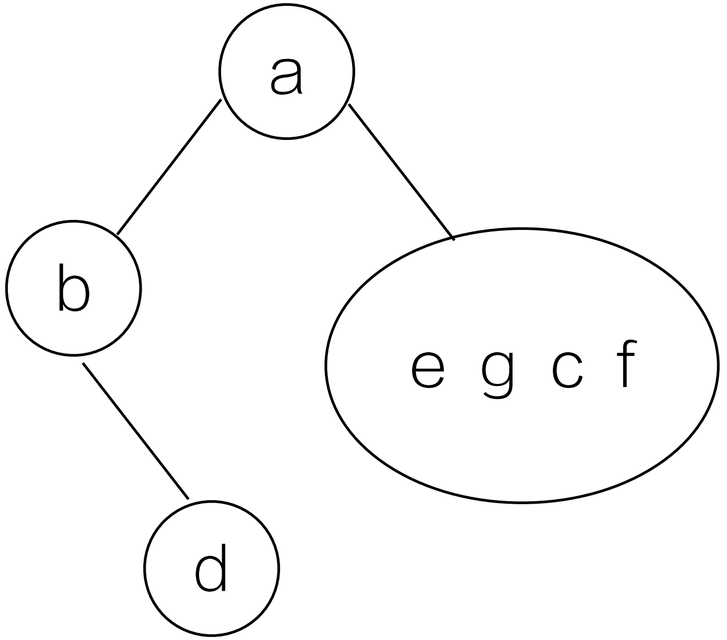
[2021] 4. 某森林 F 对应的二叉树为 T ，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是_____。

A. 1　　B. 2　　C. 3　　D. 4



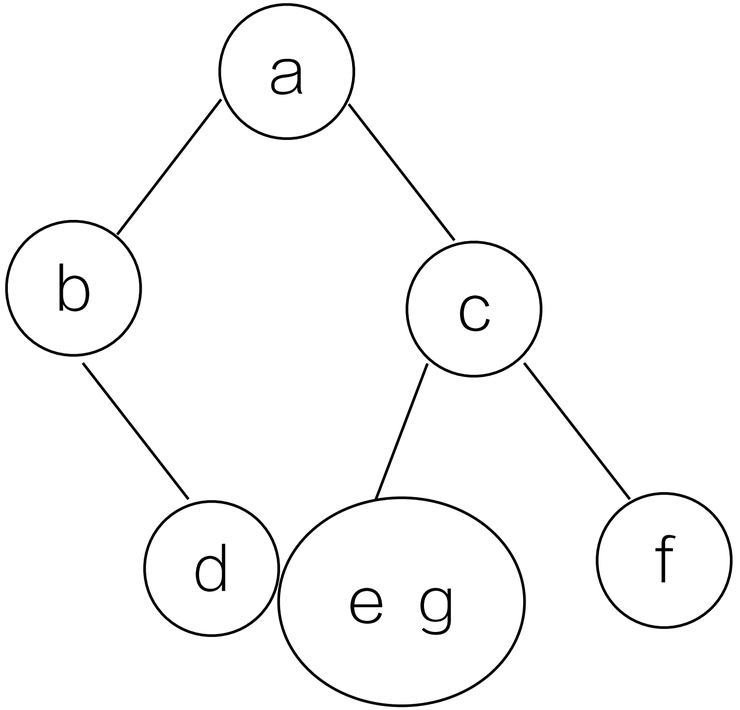
[2021] 4. 某森林 F 对应的二叉树为 T ，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是_____。

A. 1　　B. 2　　C. 3　　D. 4



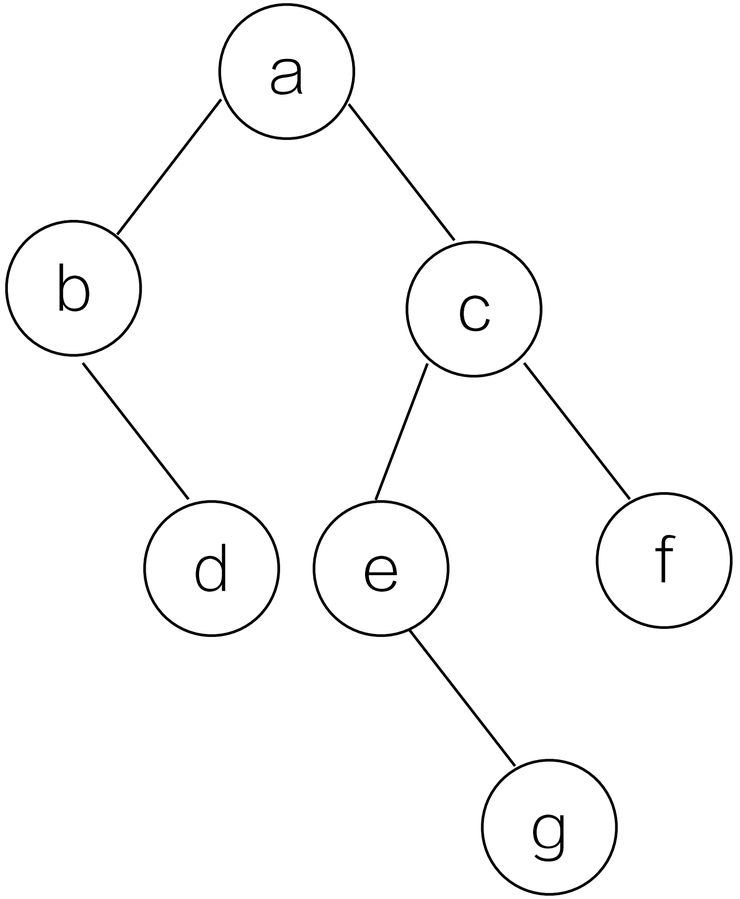
[2021] 4. 某森林 F 对应的二叉树为 T ，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是_____。

A. 1　　B. 2　　C. 3　　D. 4



[2021] 4. 某森林 F 对应的二叉树为 T ，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是_____。

A. 1　　B. 2　　C. 3　　D. 4



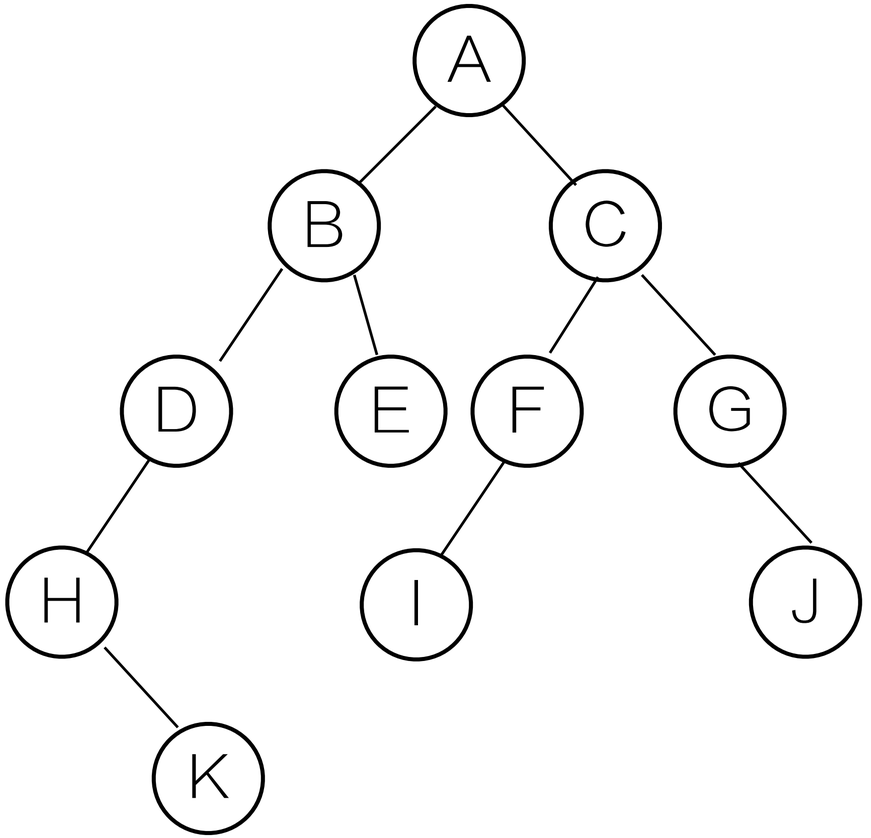
树、森林、二叉树 – 真题解析

[2021] 4. 某森林 F 对应的二叉树为 T ， 若 T 的先序遍历序列是 a, b, d, c, e, g, f， 中序遍历序列是 b, d, a, e, g, c, f， 则 F 中树的棵数是_____。

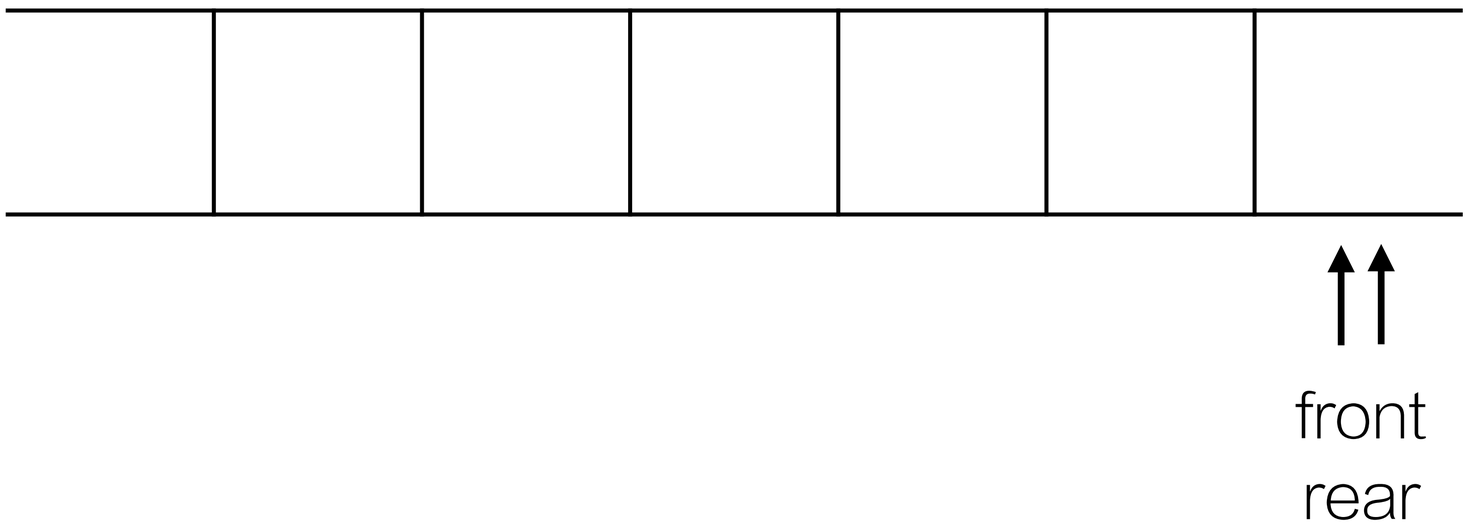
A. 1　　B. 2　　C. 3　　D. 4

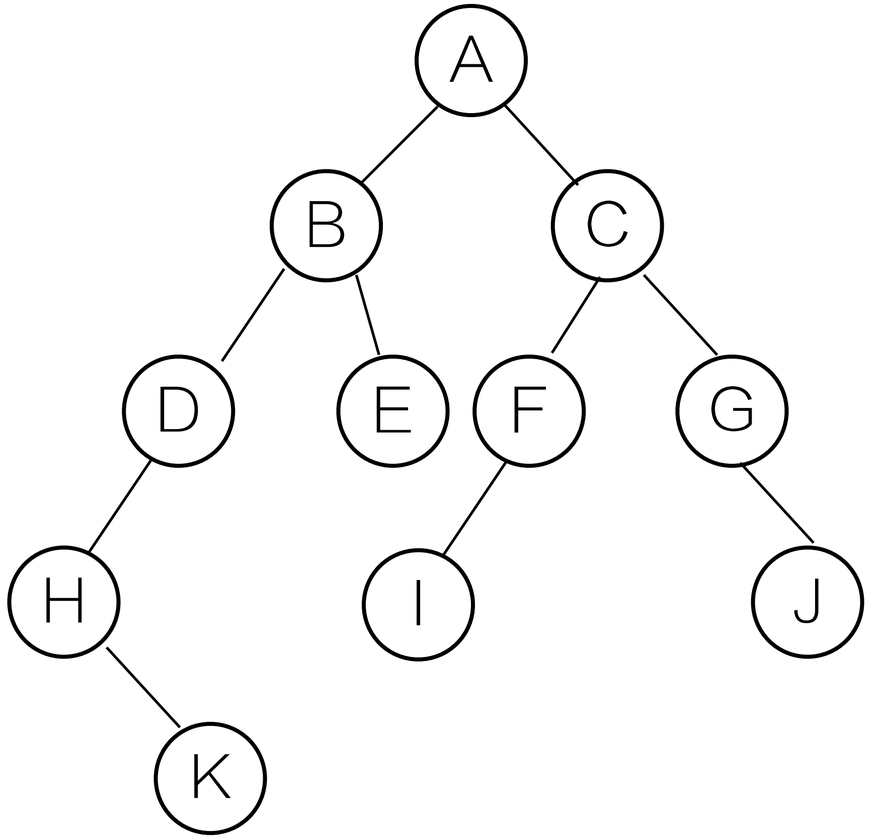
层序遍历

如何判断这棵二叉树的深度?

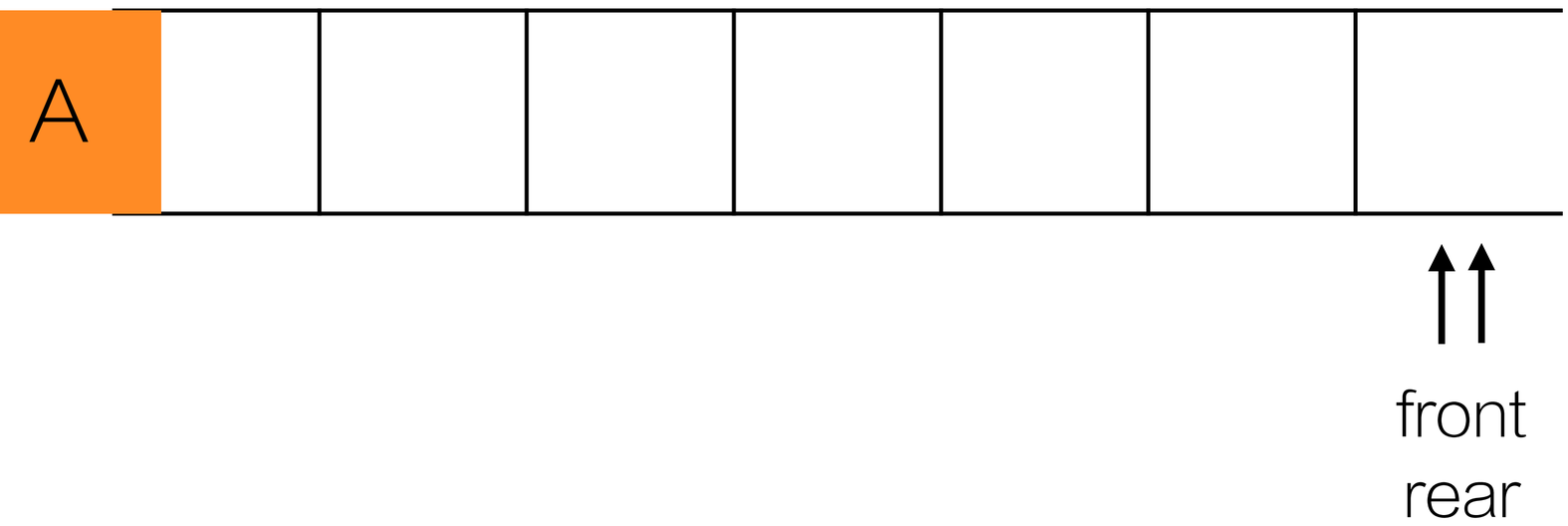


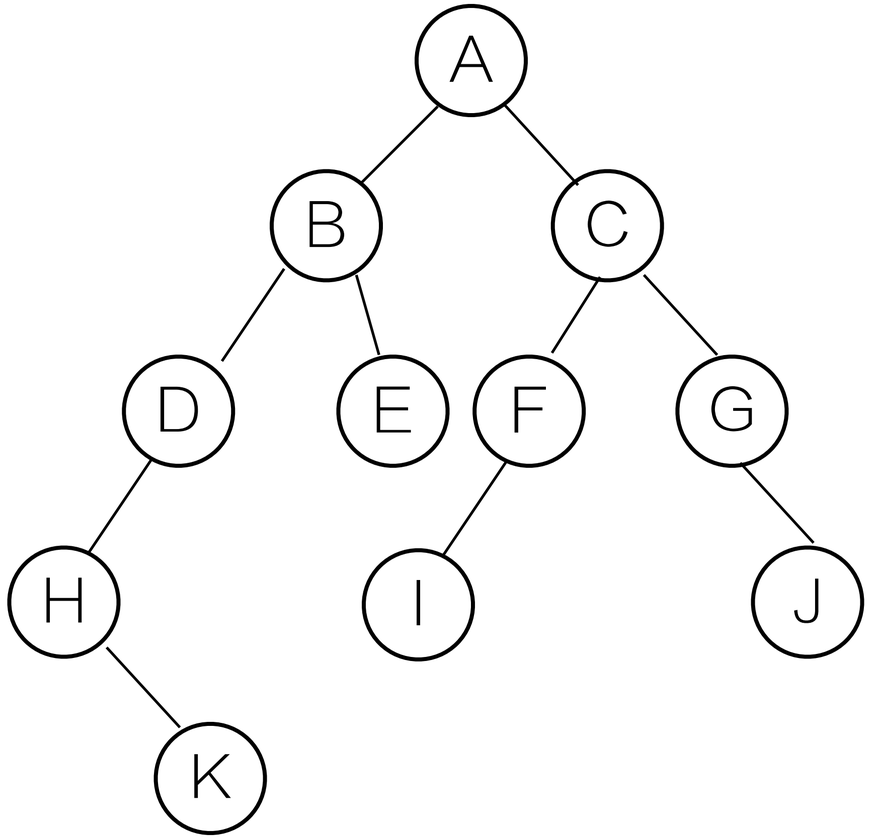
层序遍历



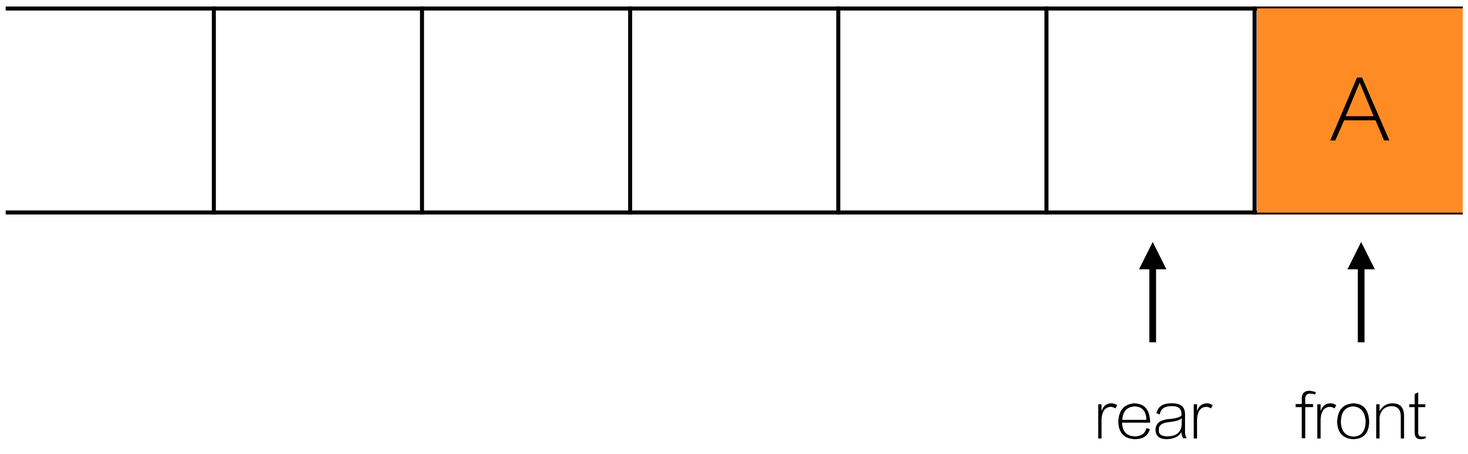


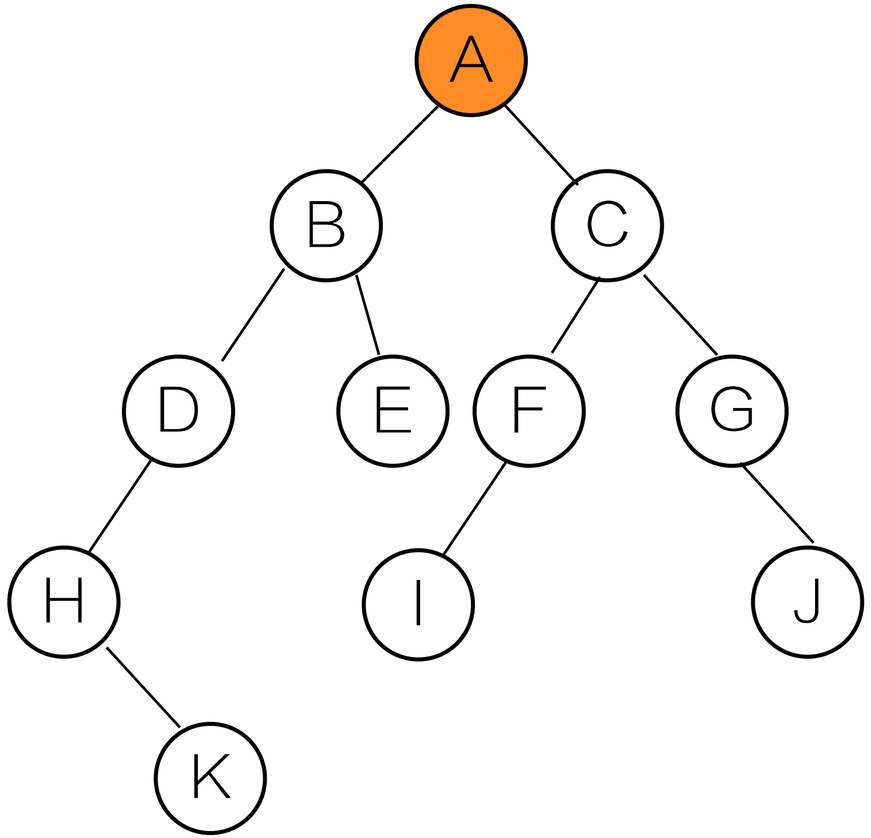
层序遍历



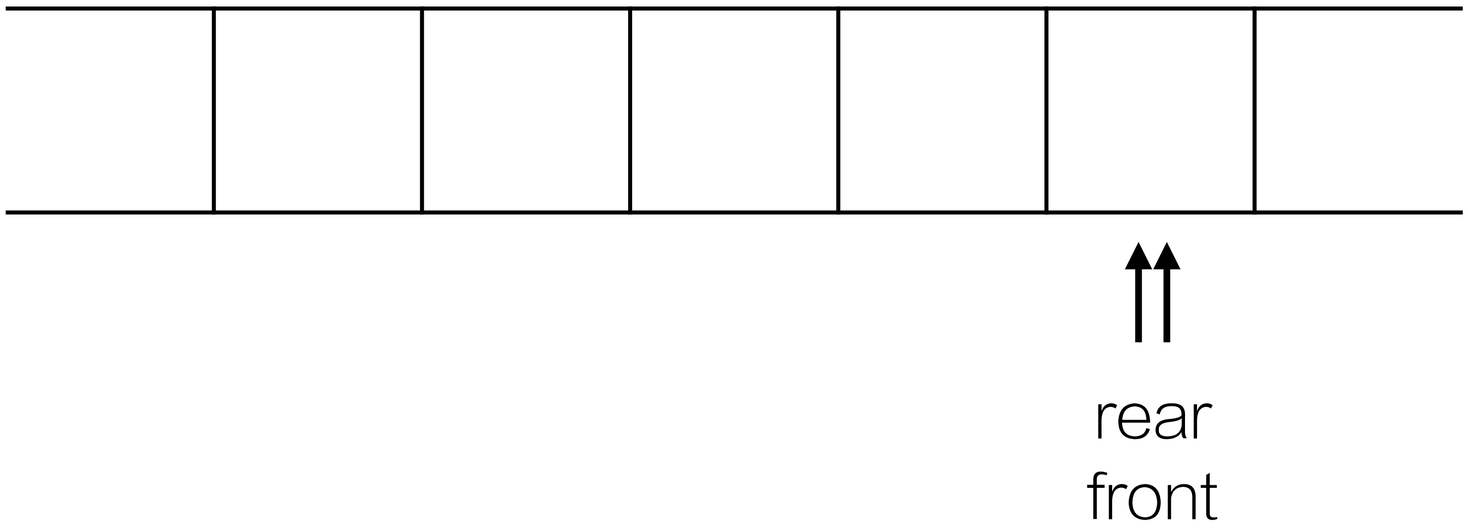


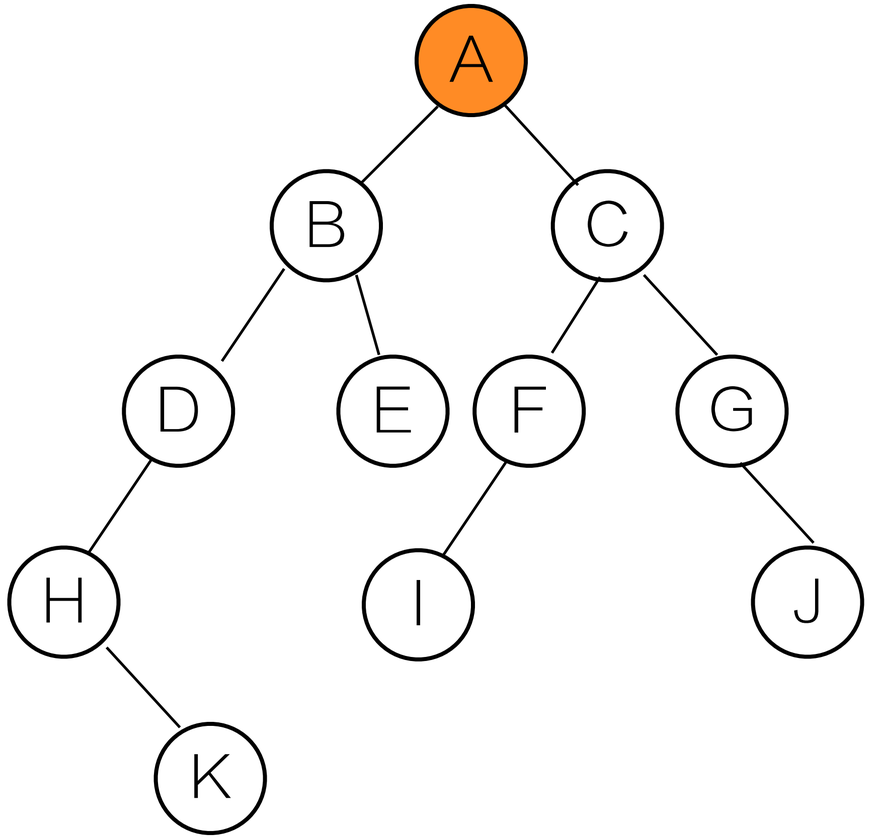
层序遍历



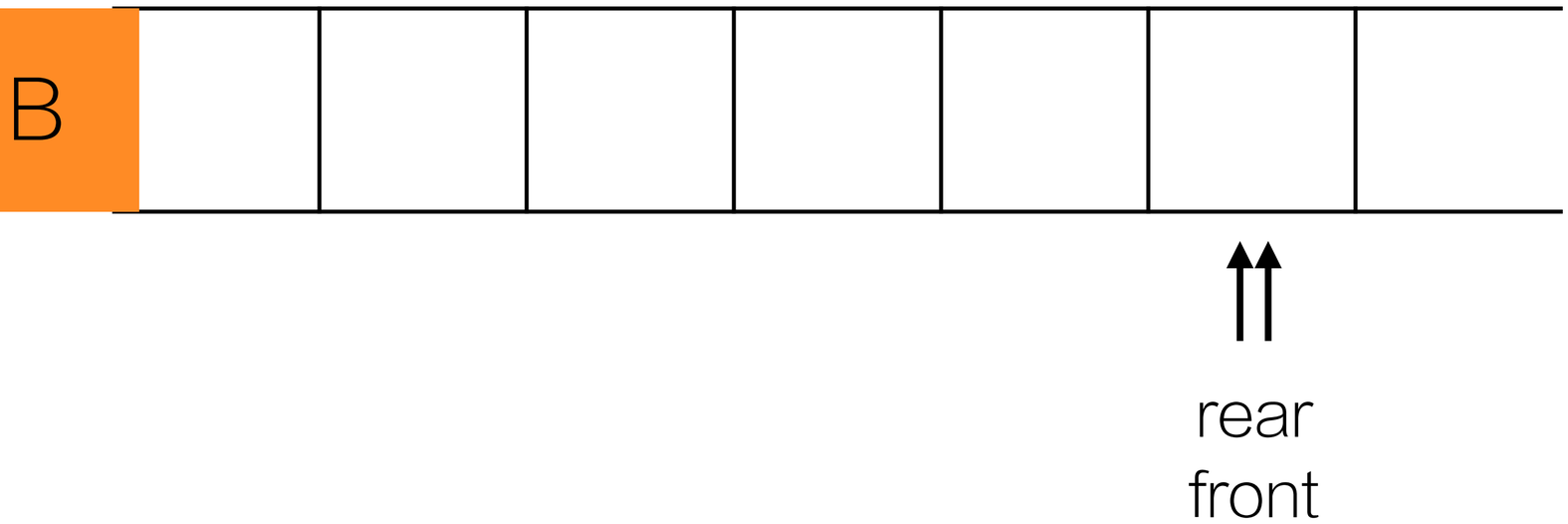


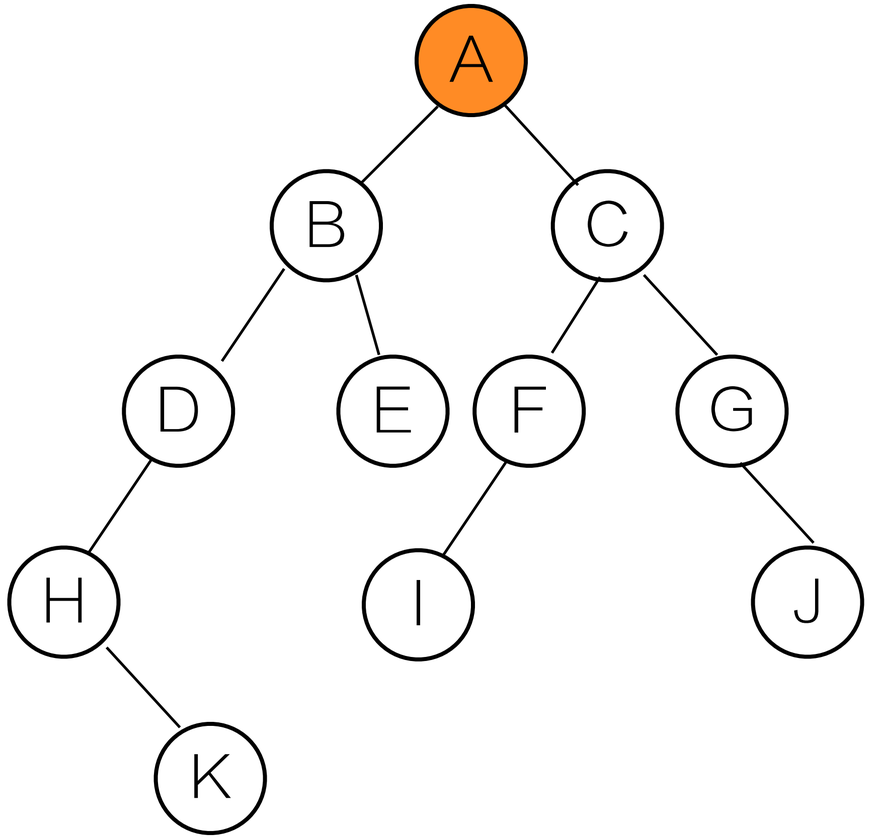
层序遍历



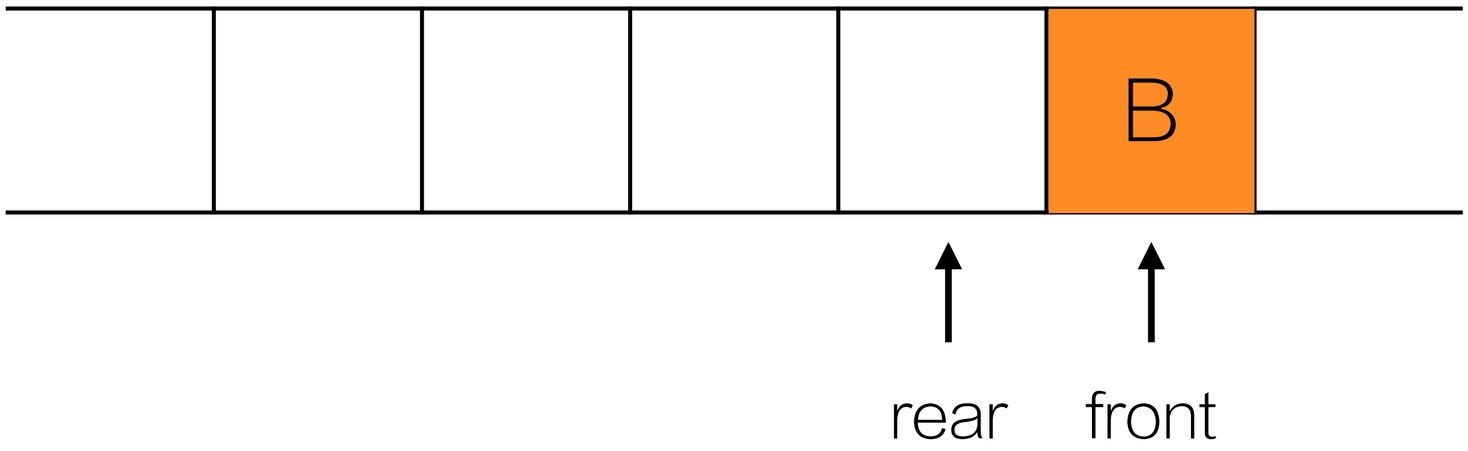


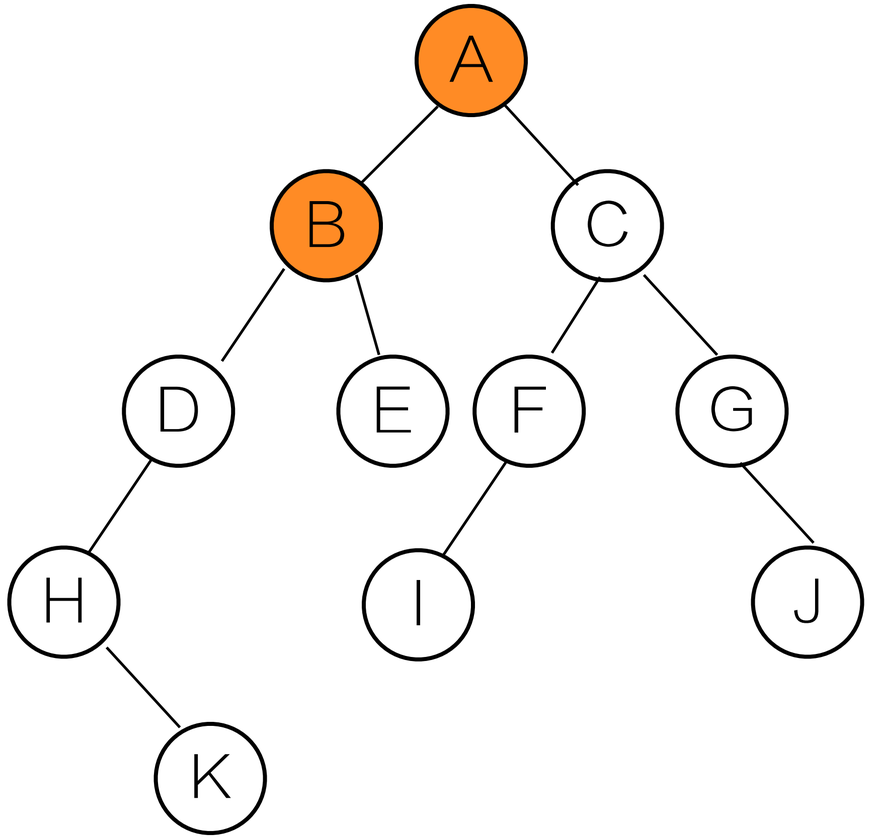
层序遍历



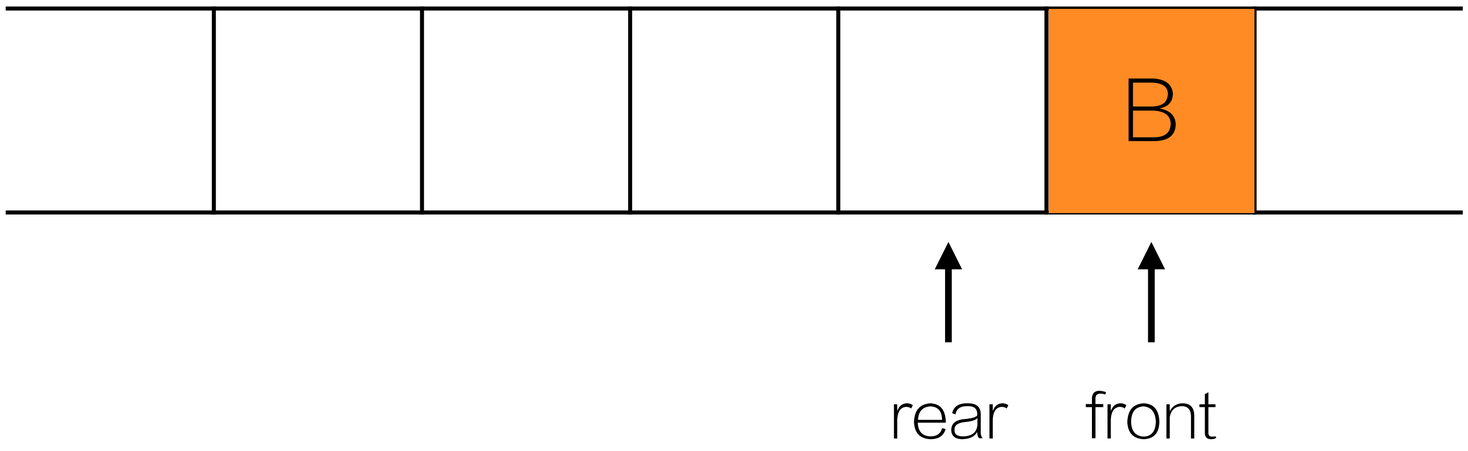


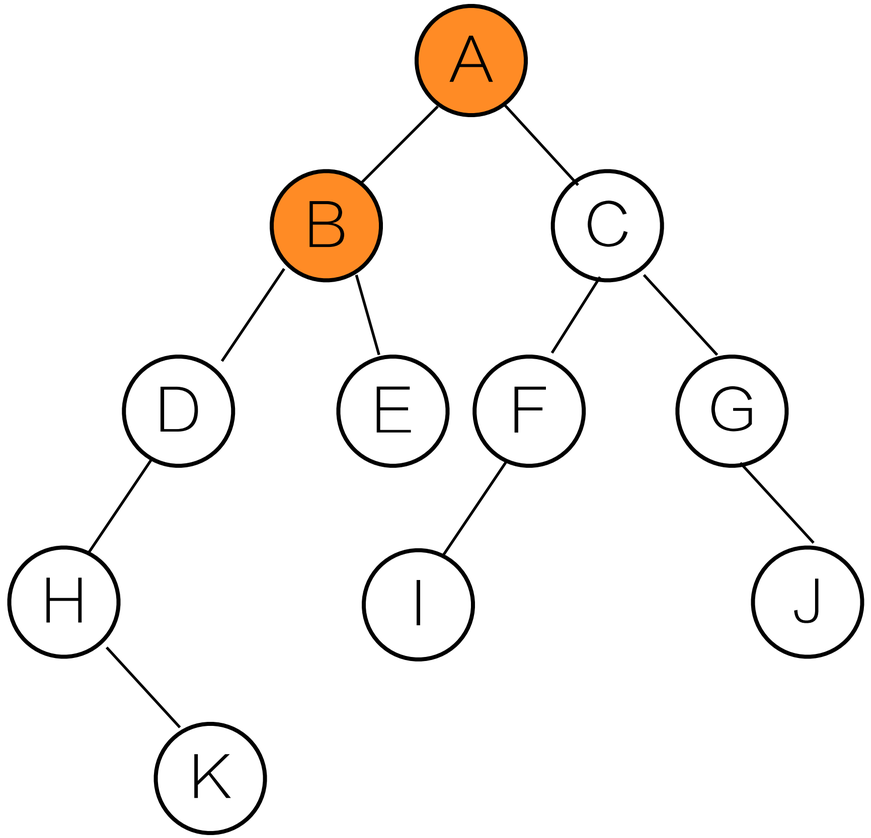
层序遍历



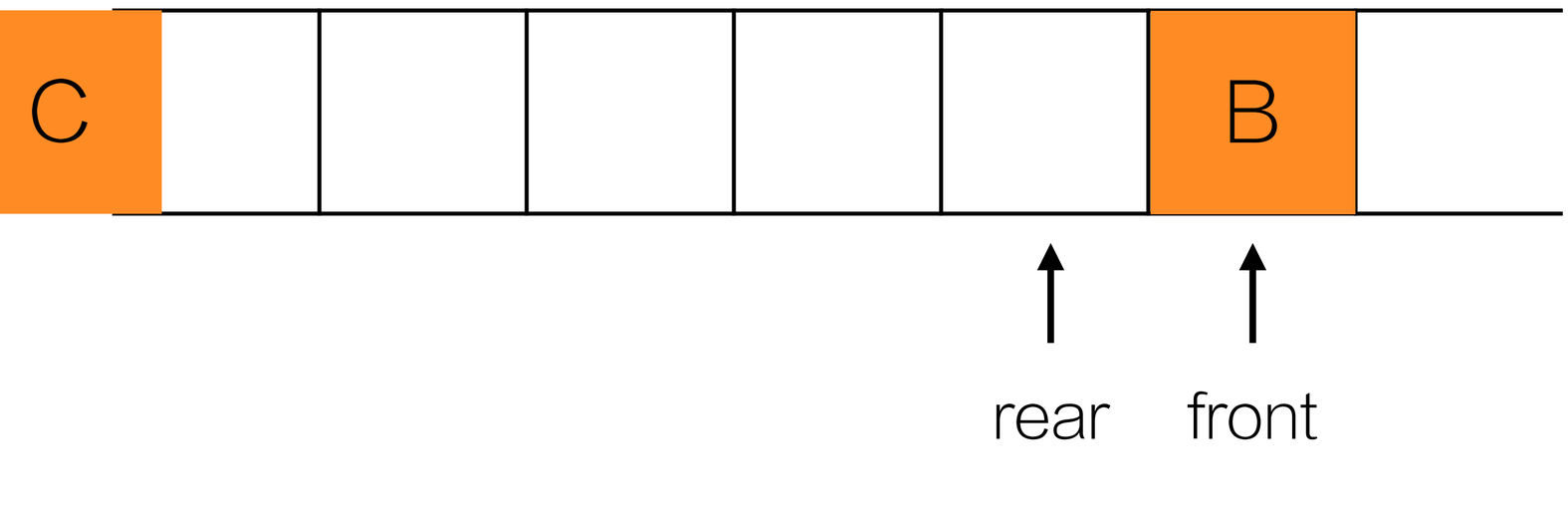


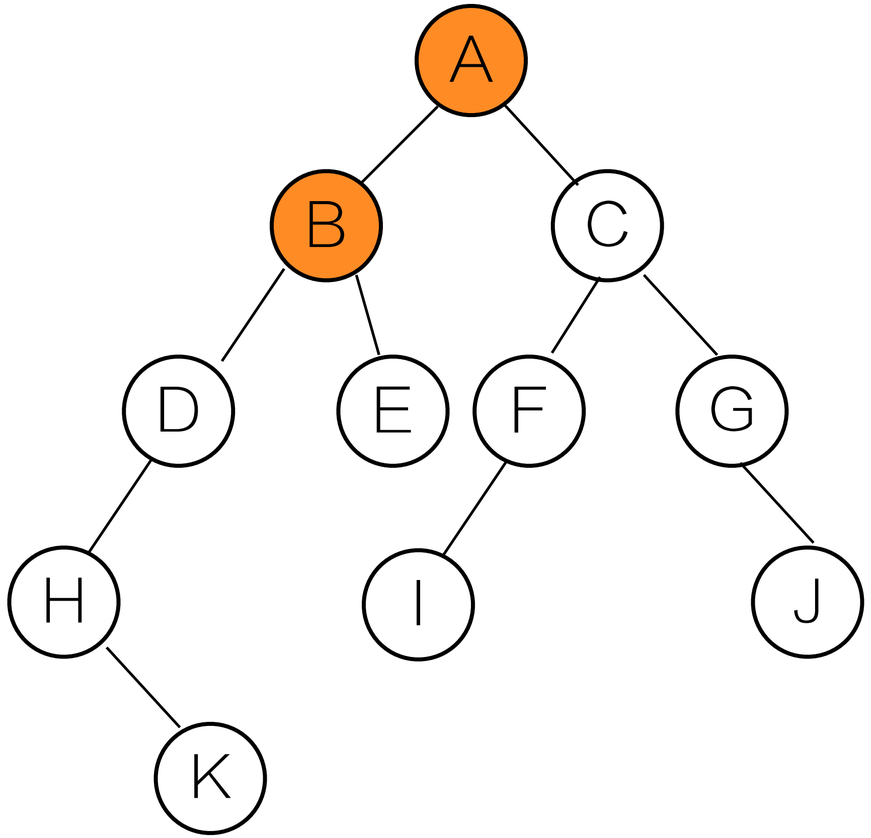
层序遍历



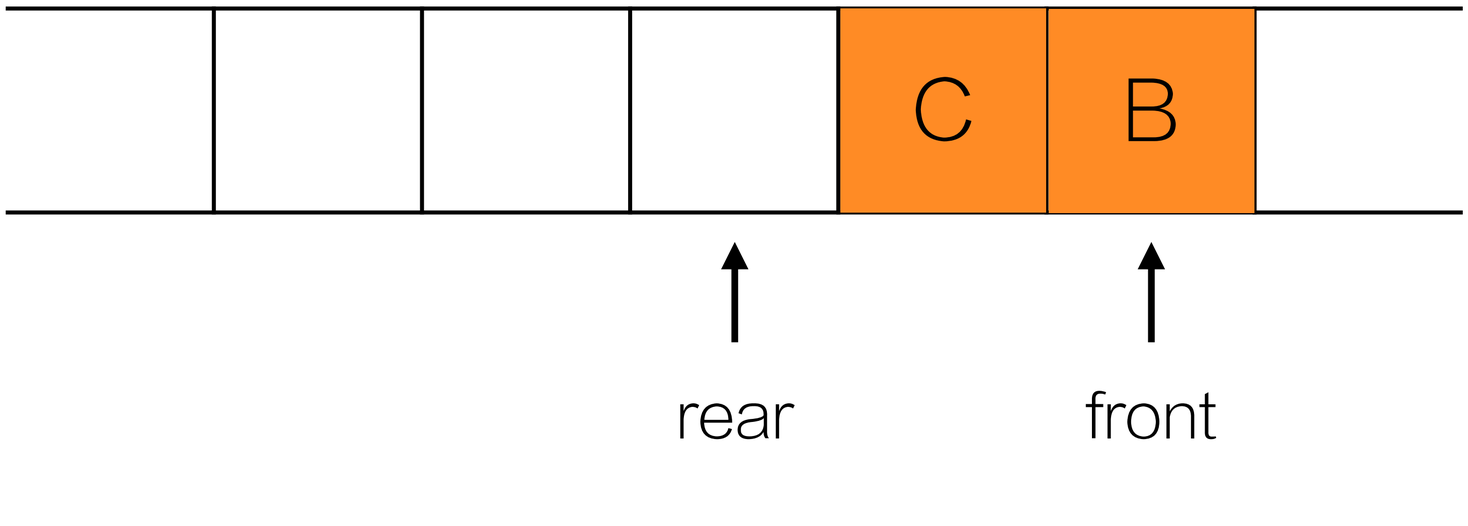


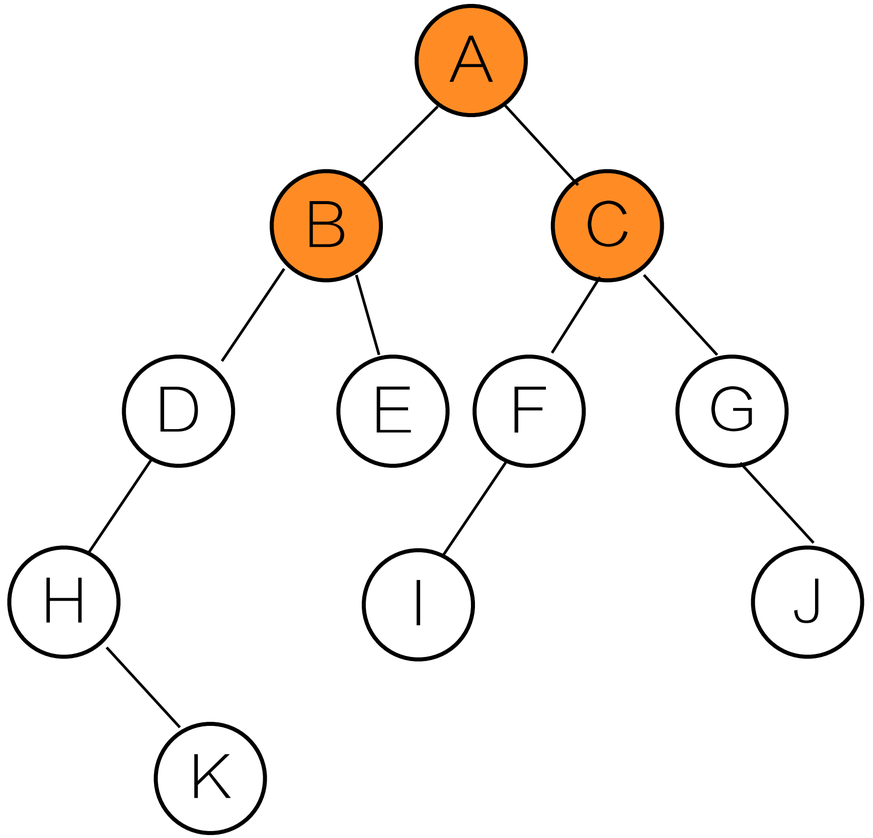
层序遍历



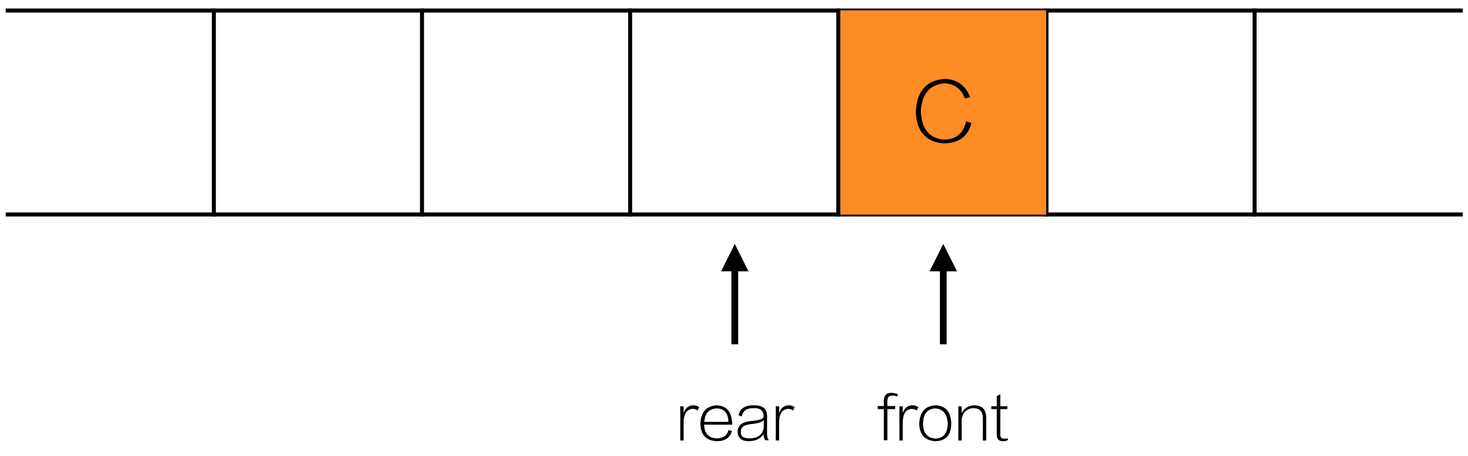


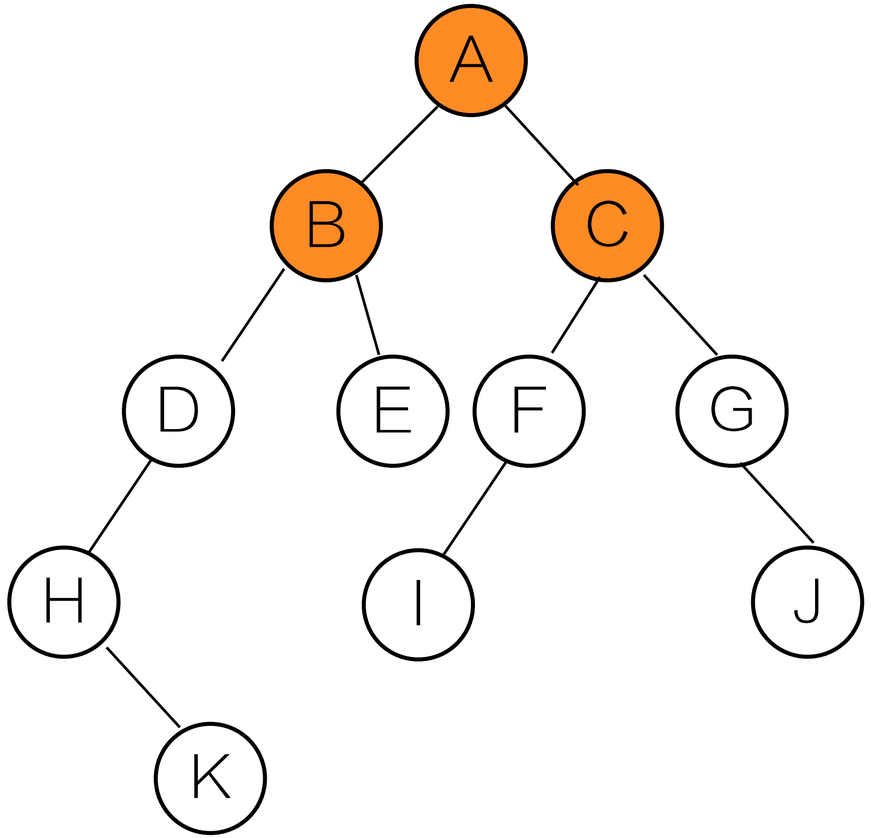
层序遍历



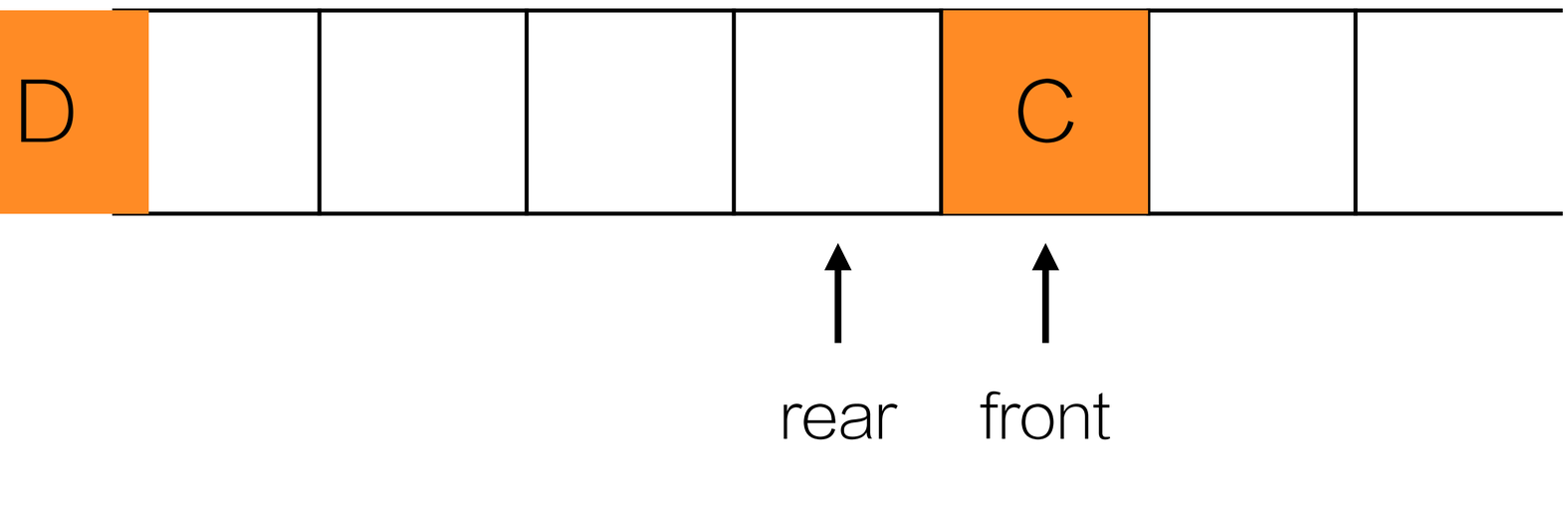


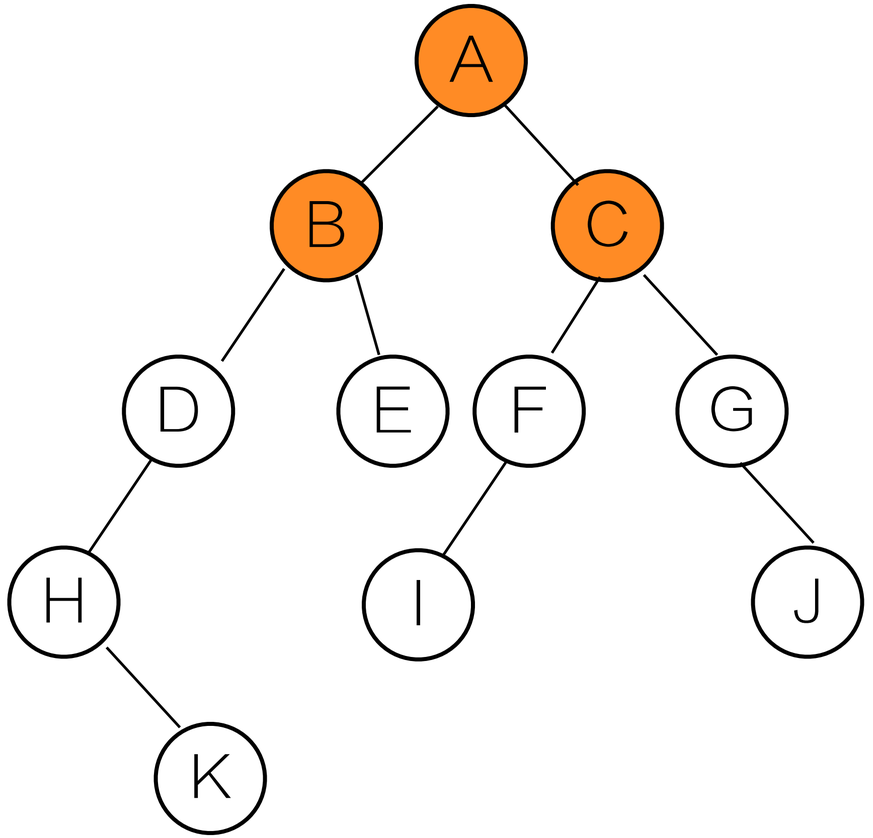
层序遍历



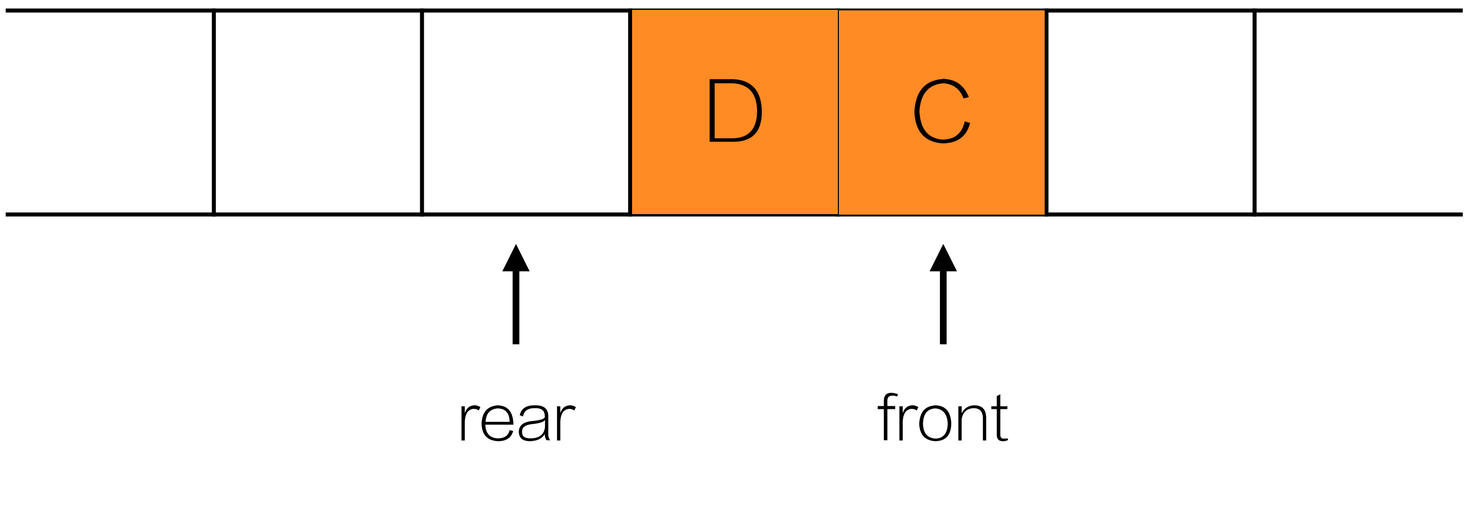


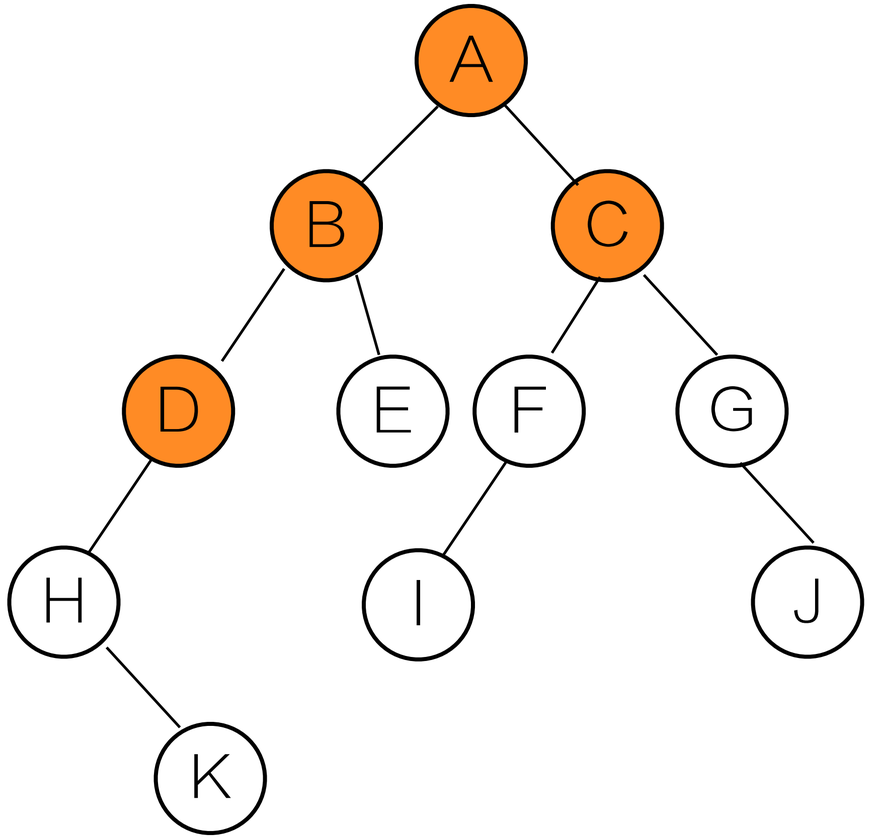
层序遍历



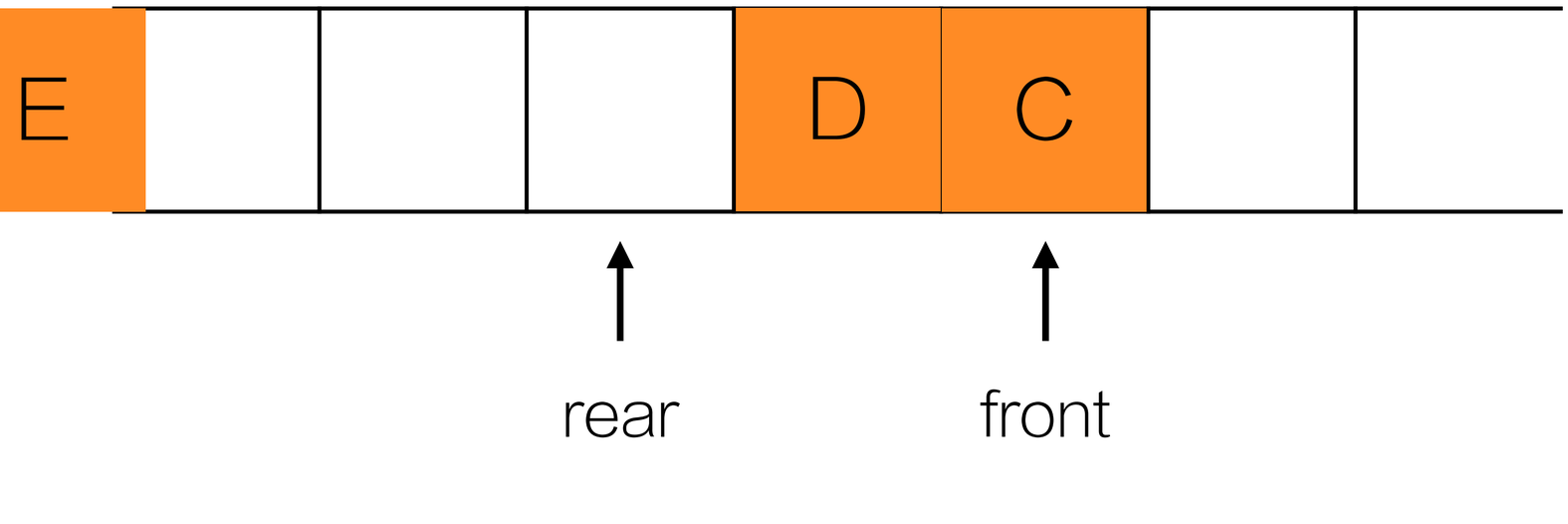


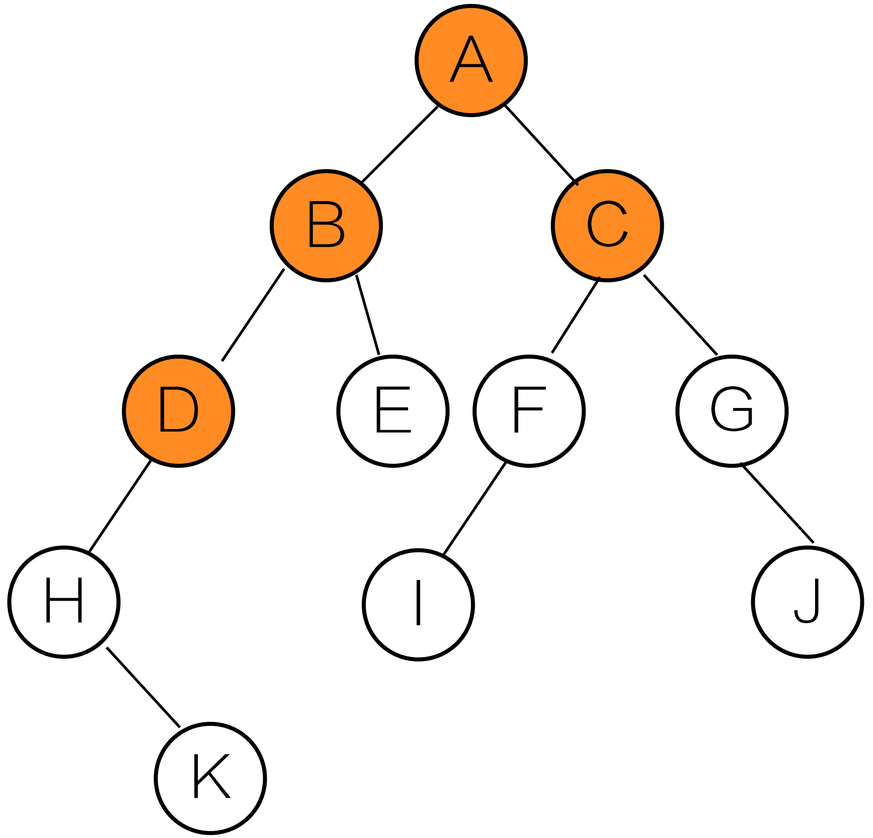
层序遍历



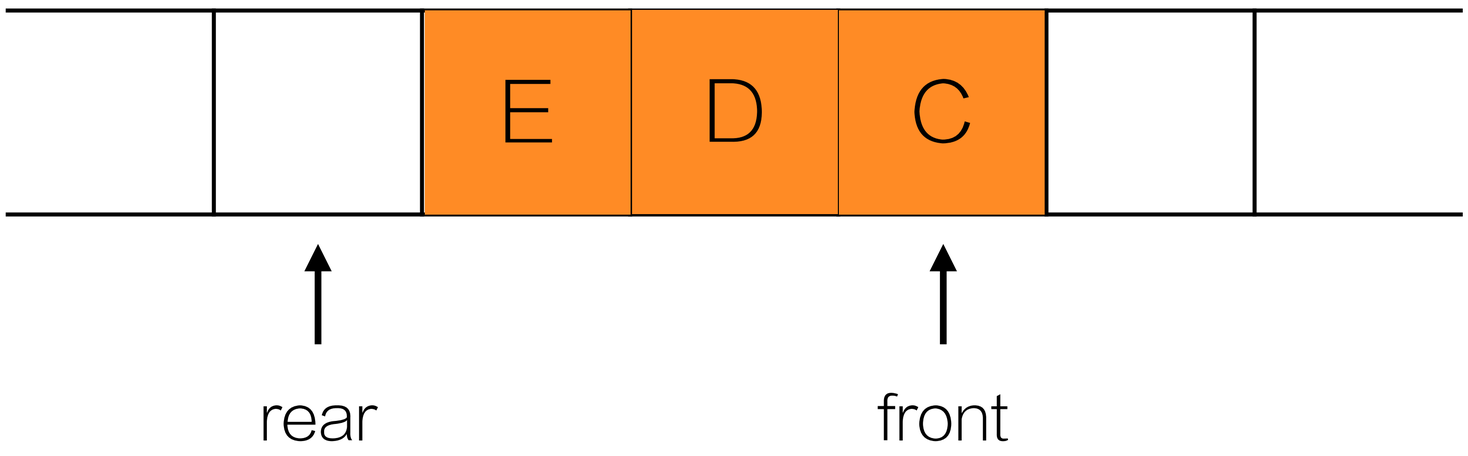


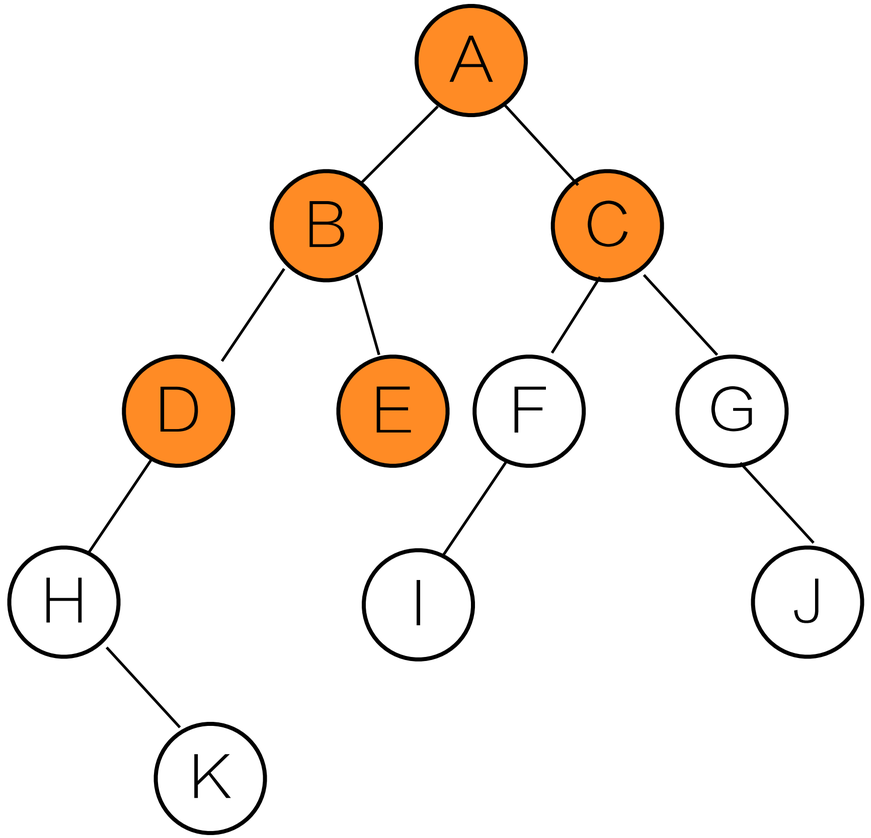
层序遍历



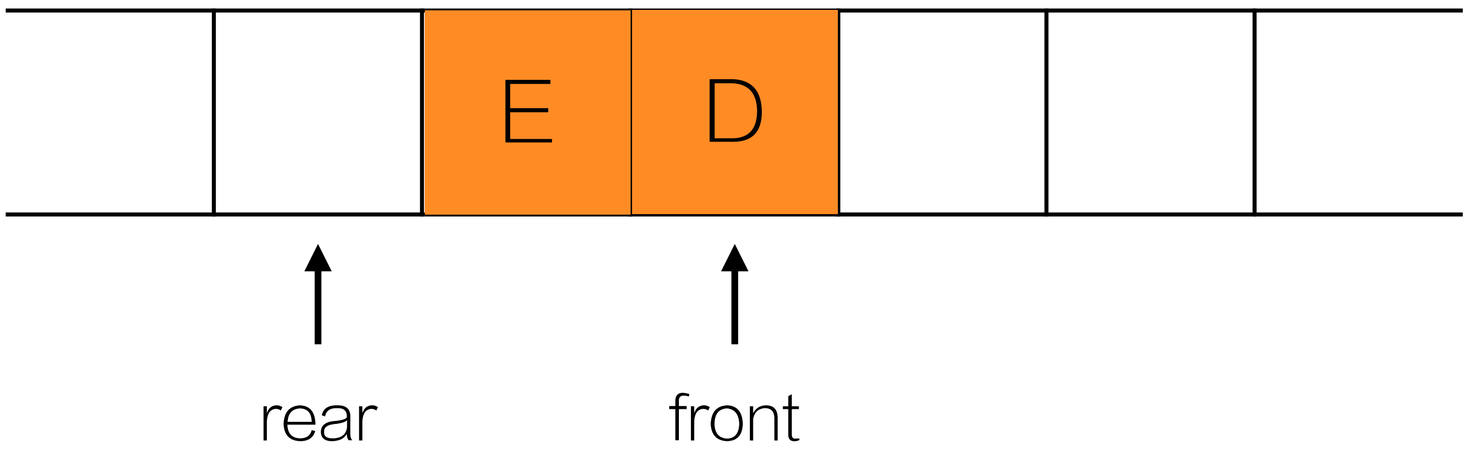


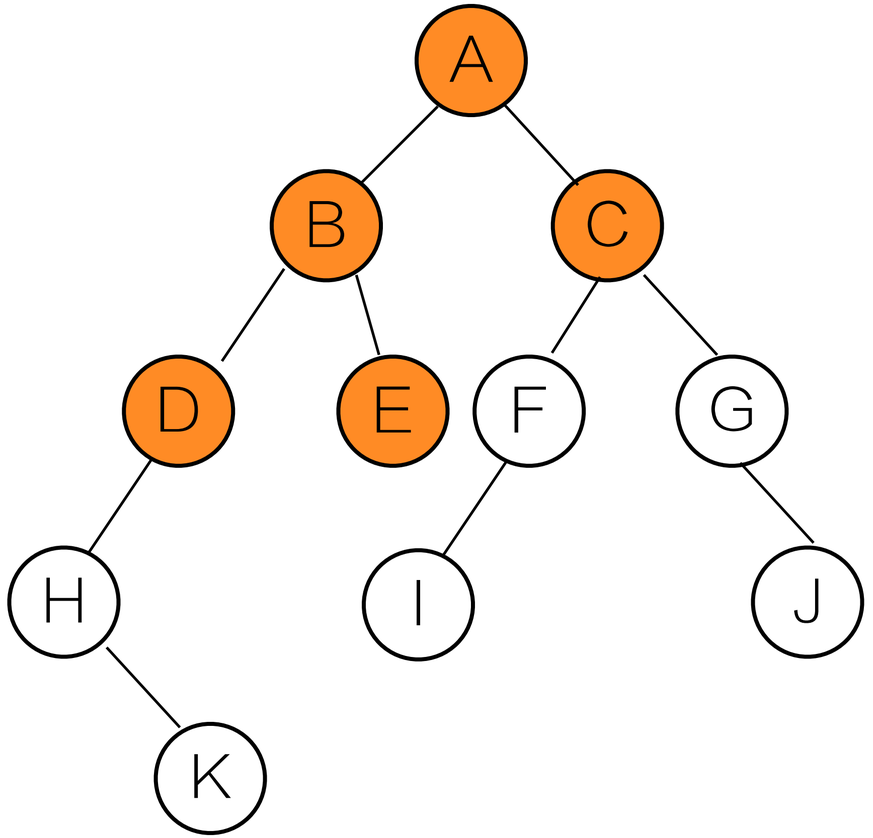
层序遍历



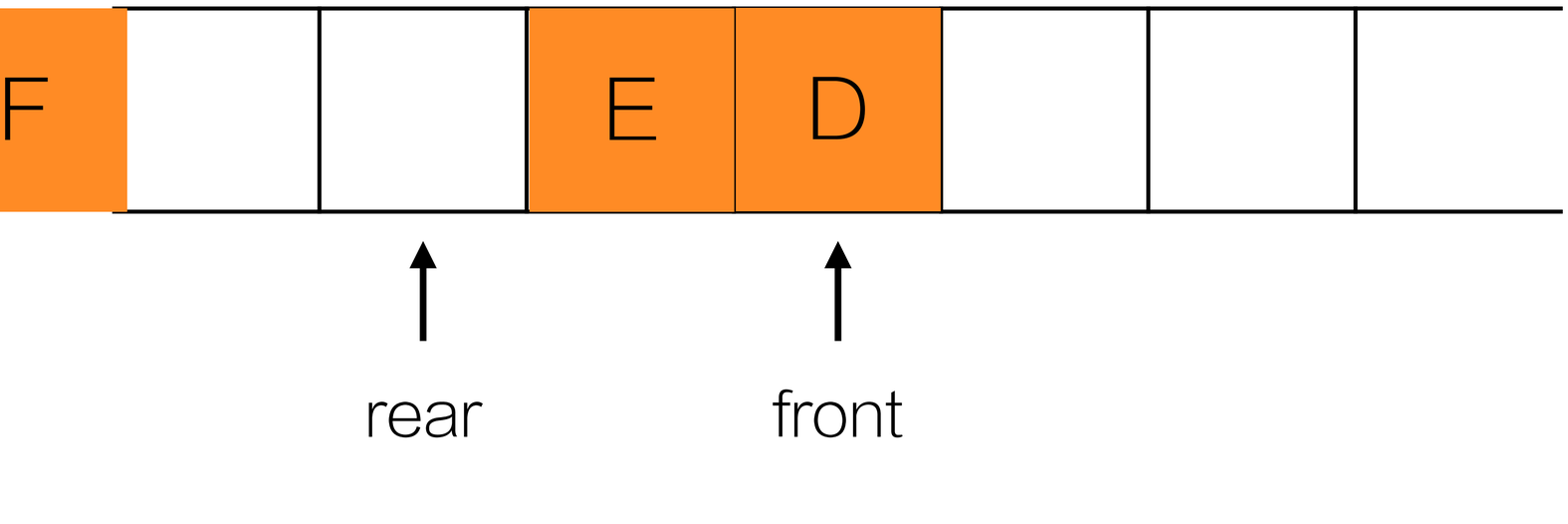


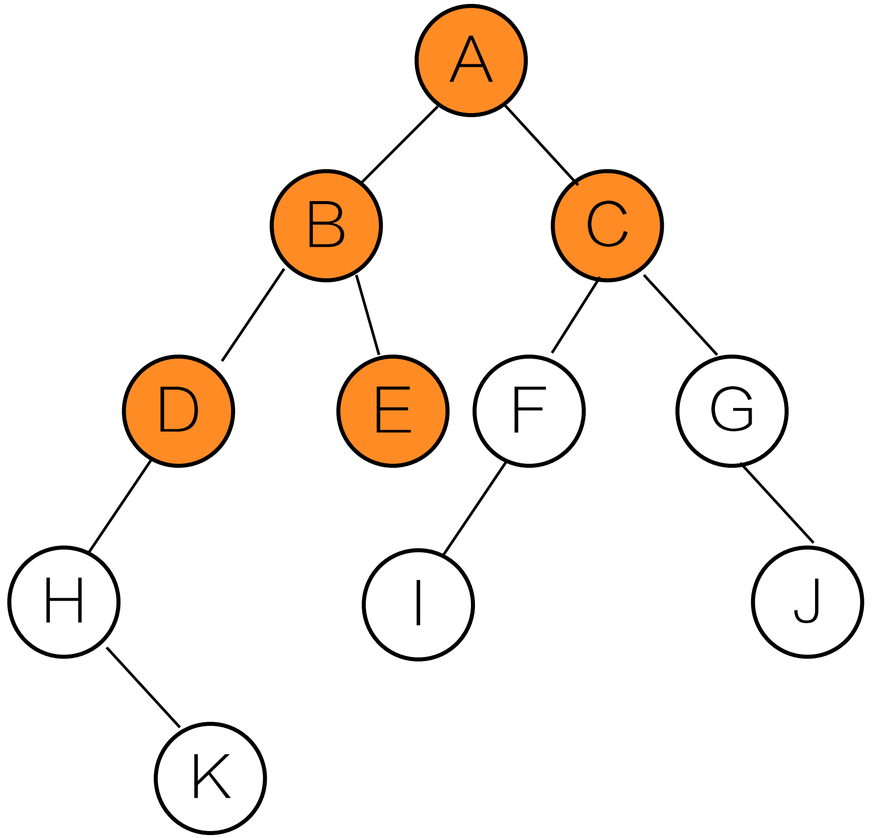
层序遍历



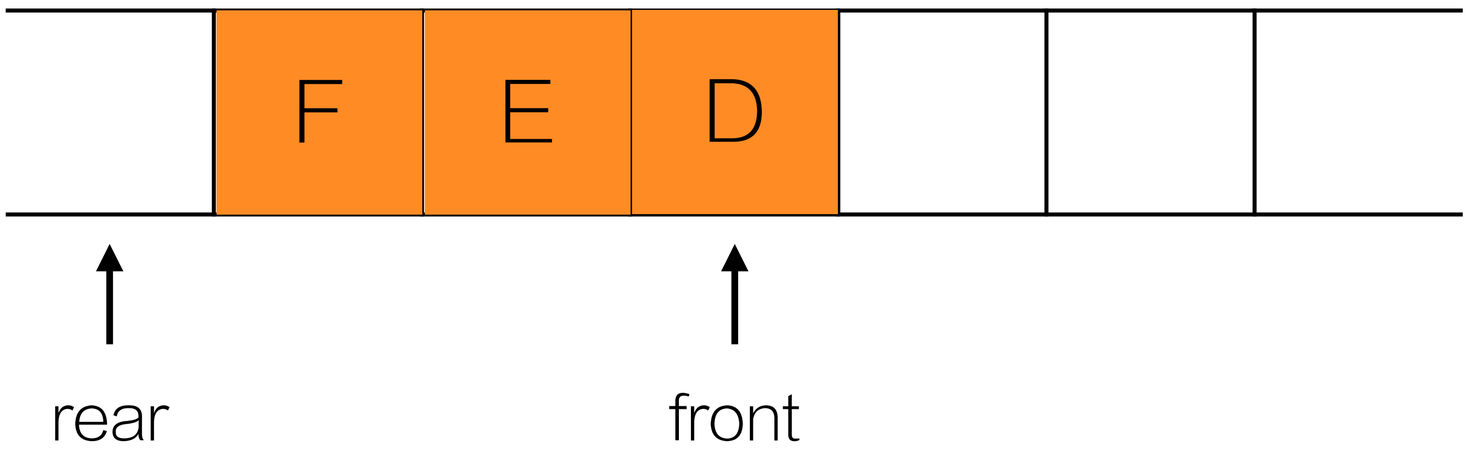


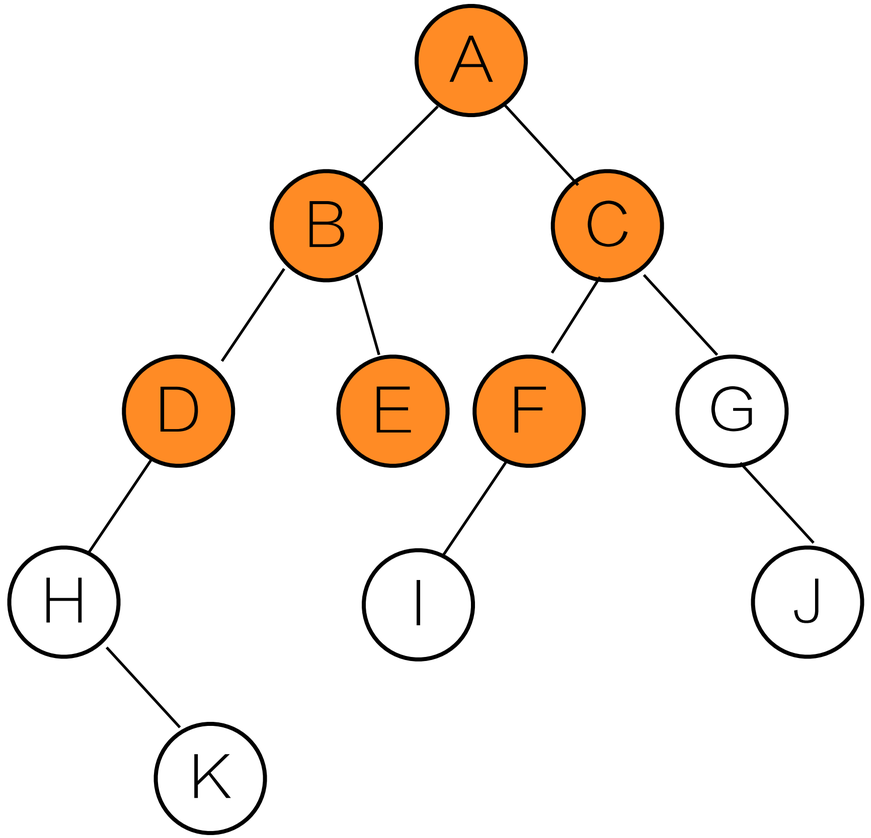
层序遍历



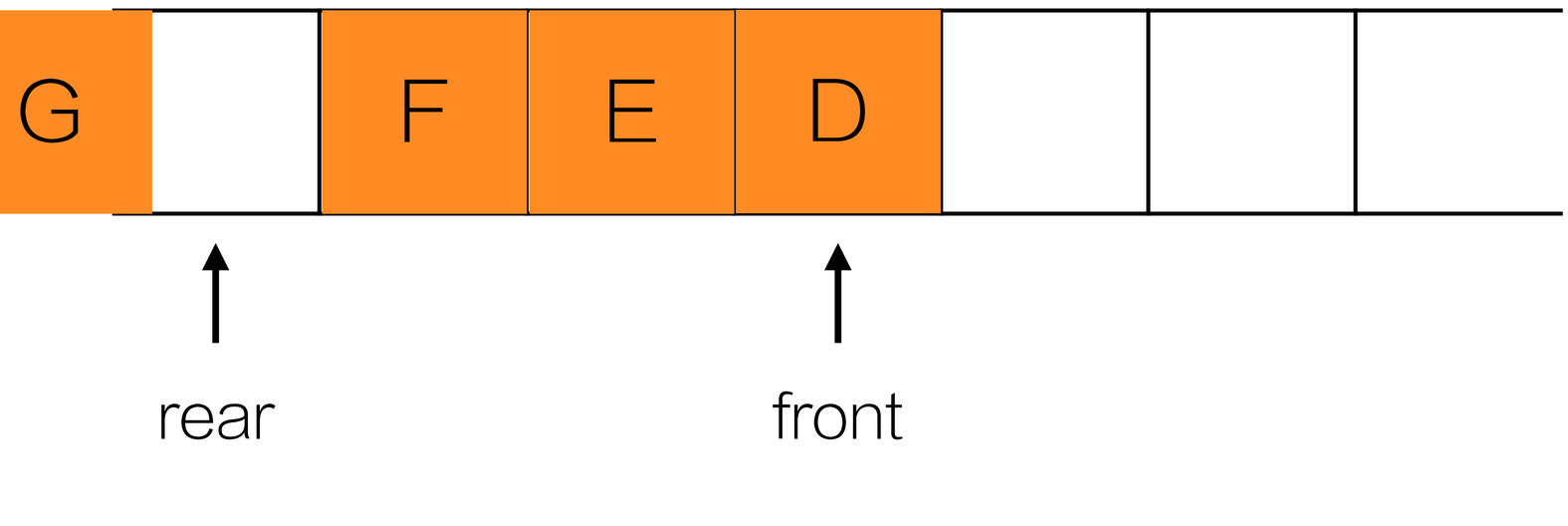


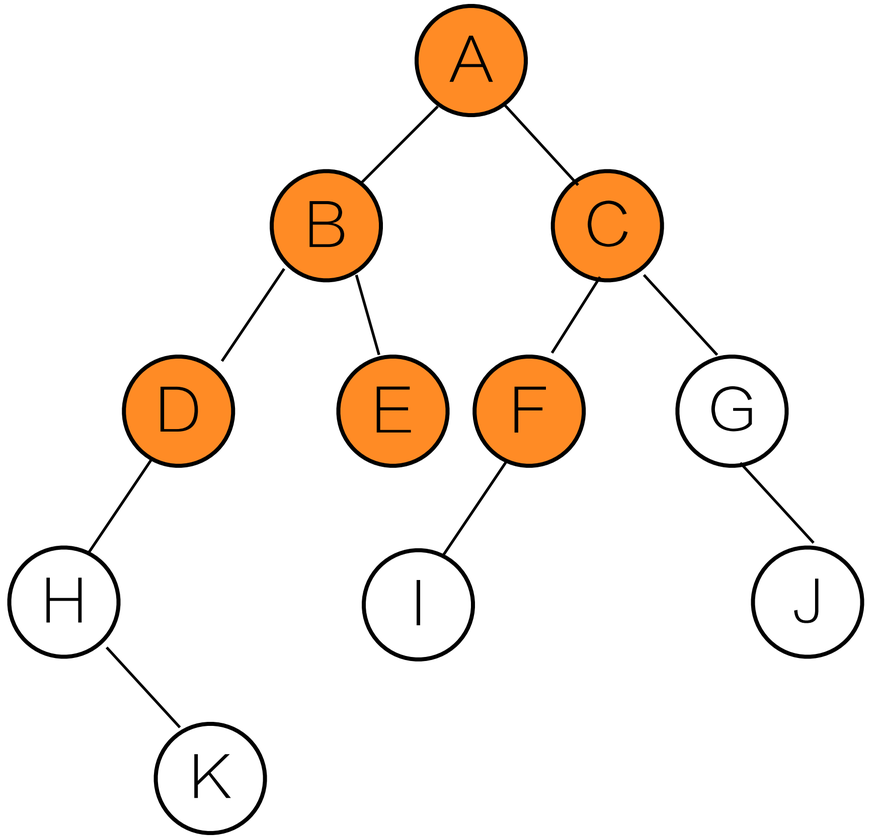
层序遍历



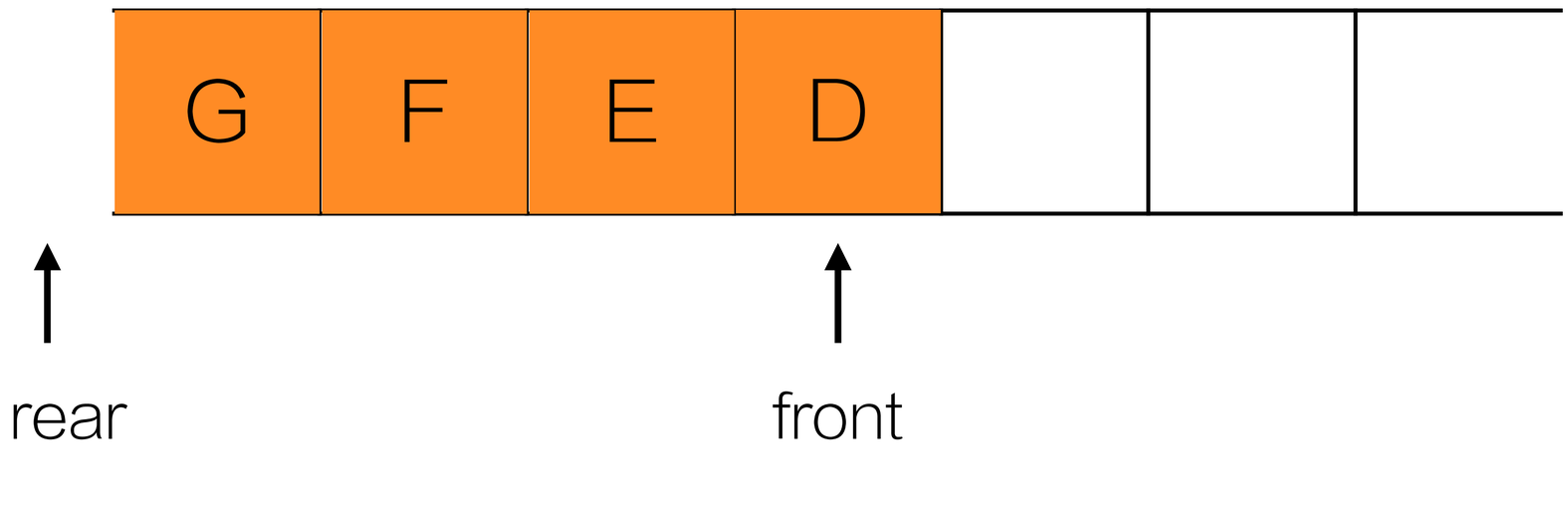


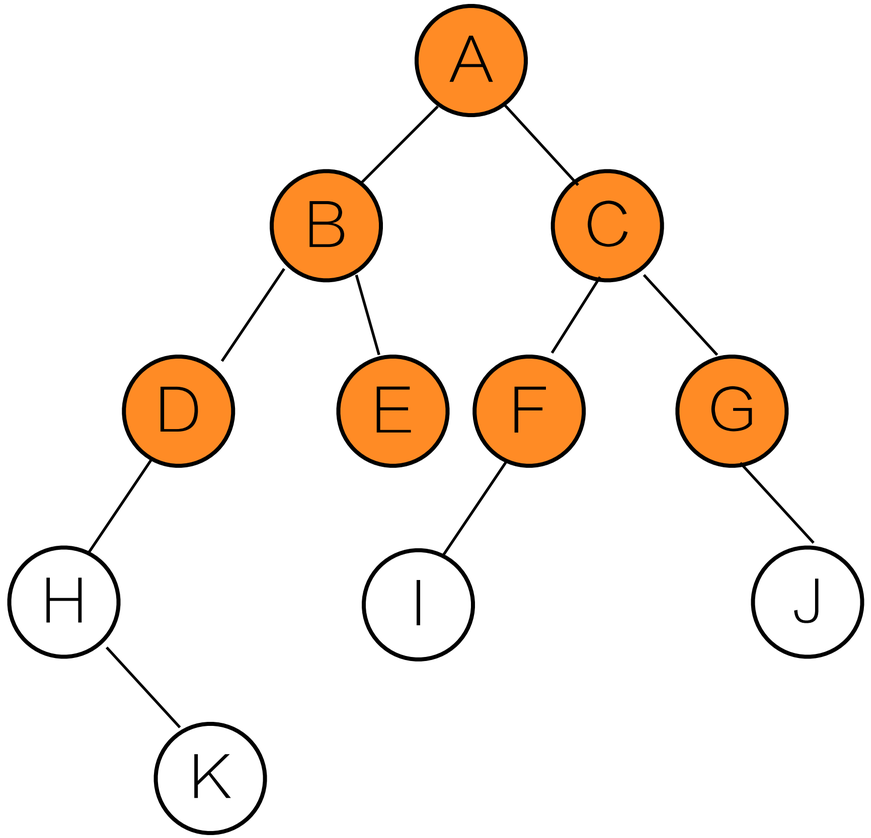
层序遍历





层序遍历

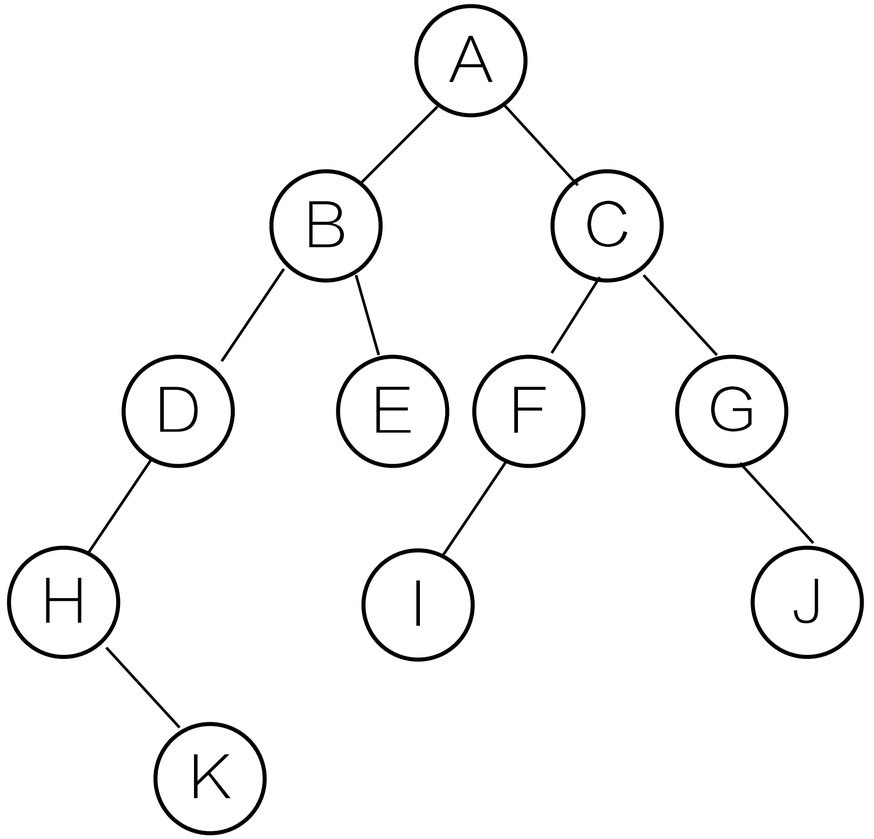




```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

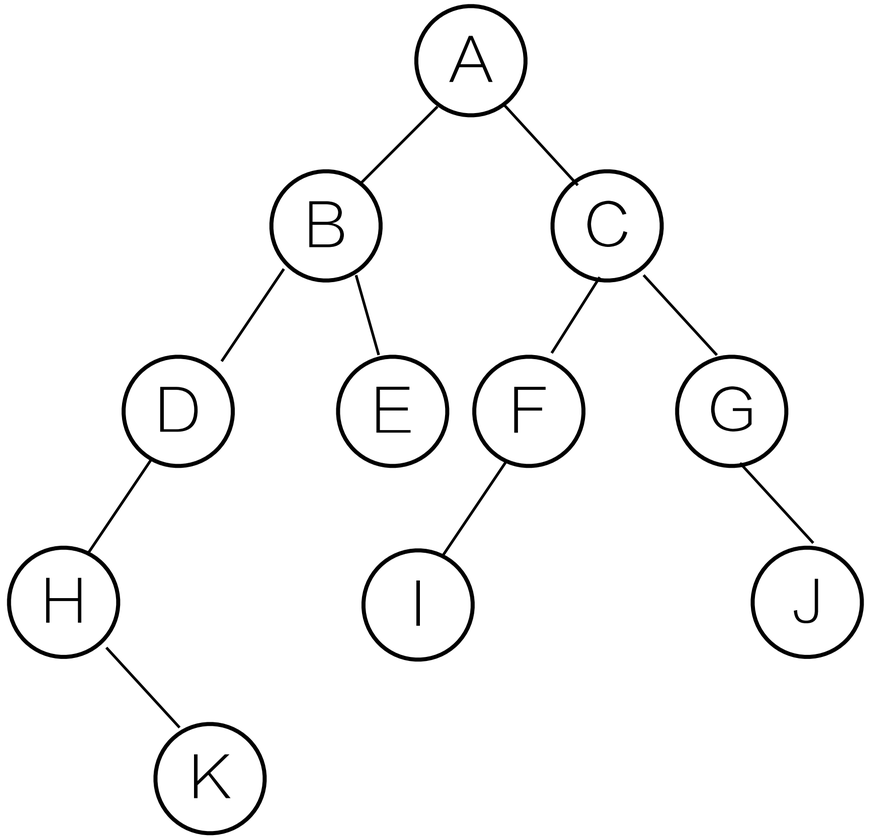
    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
            }
            if (curr->rchild != NULL)
            {
                equeue(q, curr->rchild);
            }
            count--;
        }
        depth++;
    }
    return depth;
}
```



```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

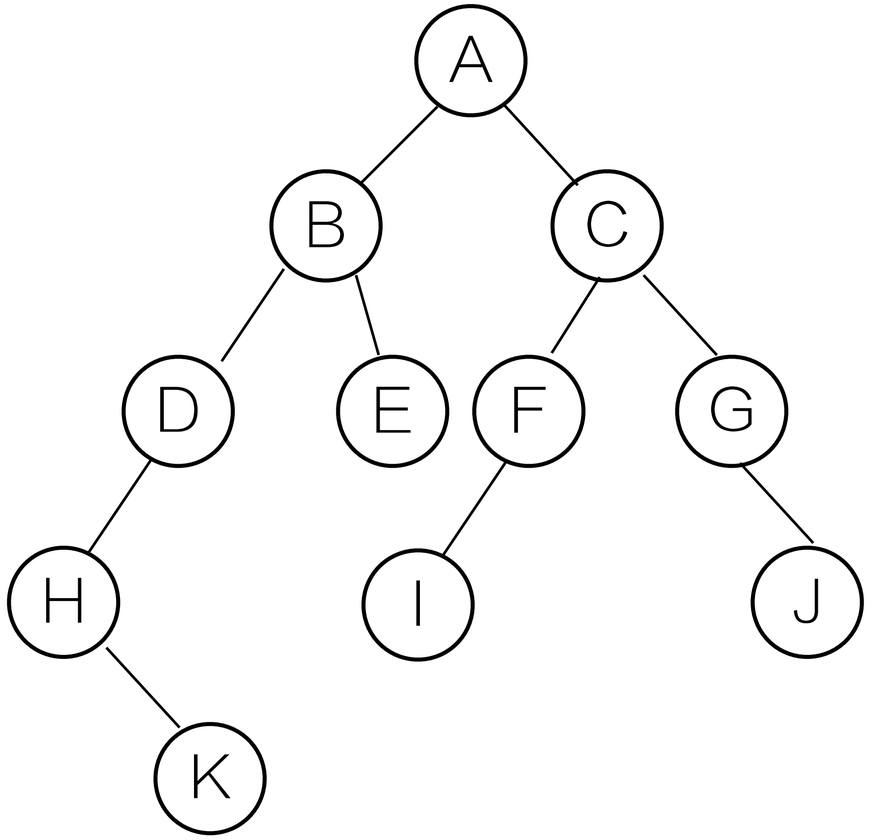
    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```



```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

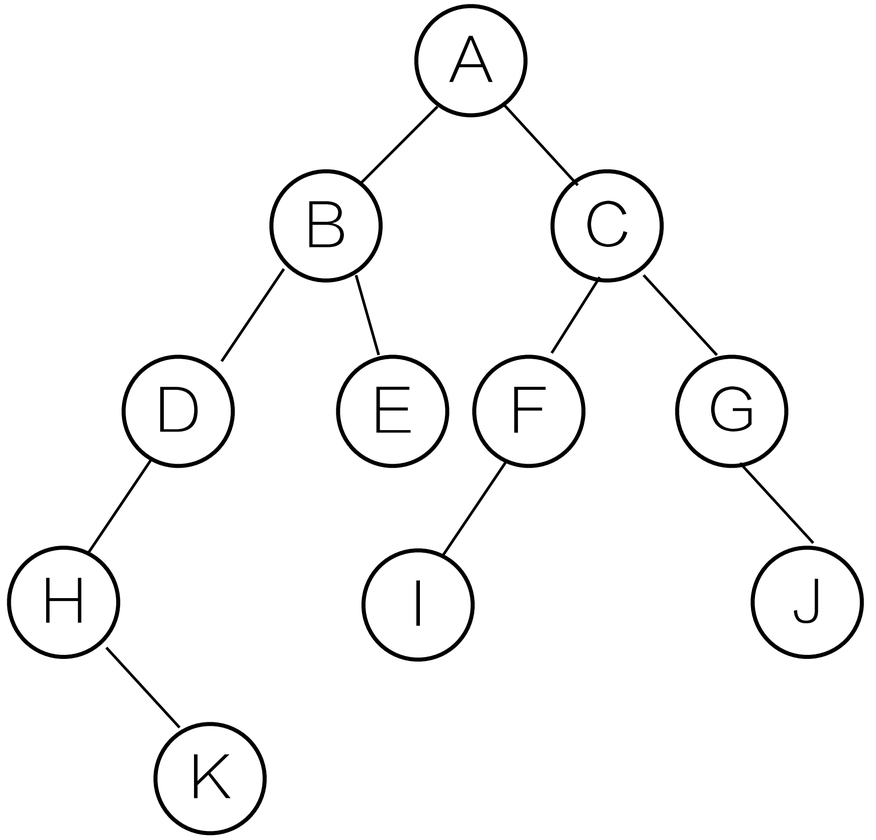
    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```



```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```



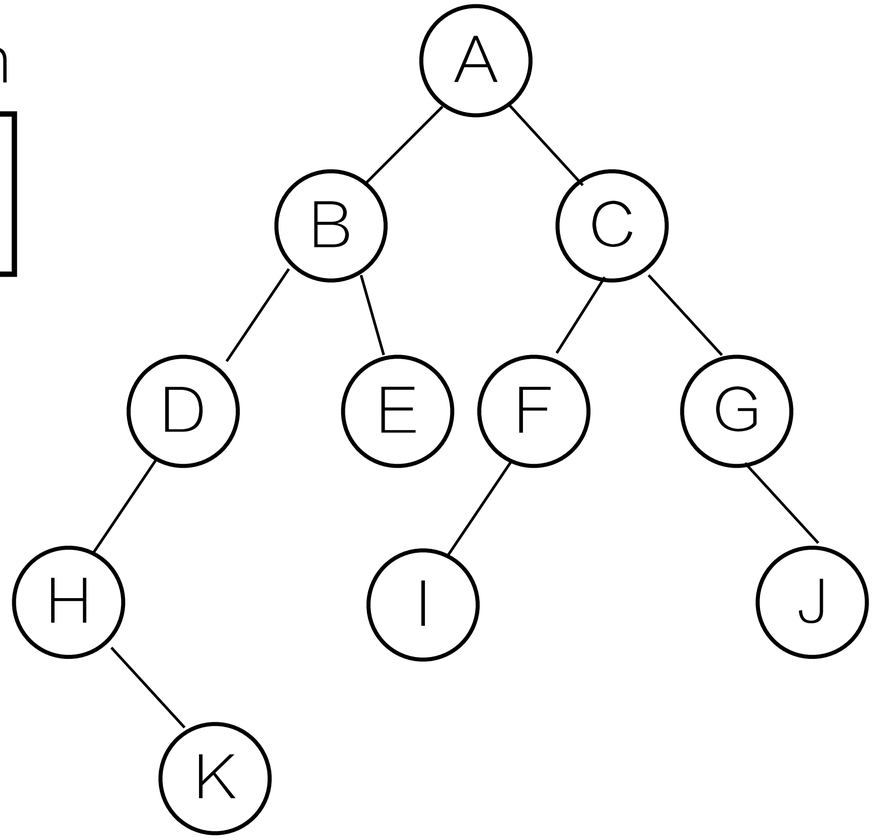
```
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```

depth

0



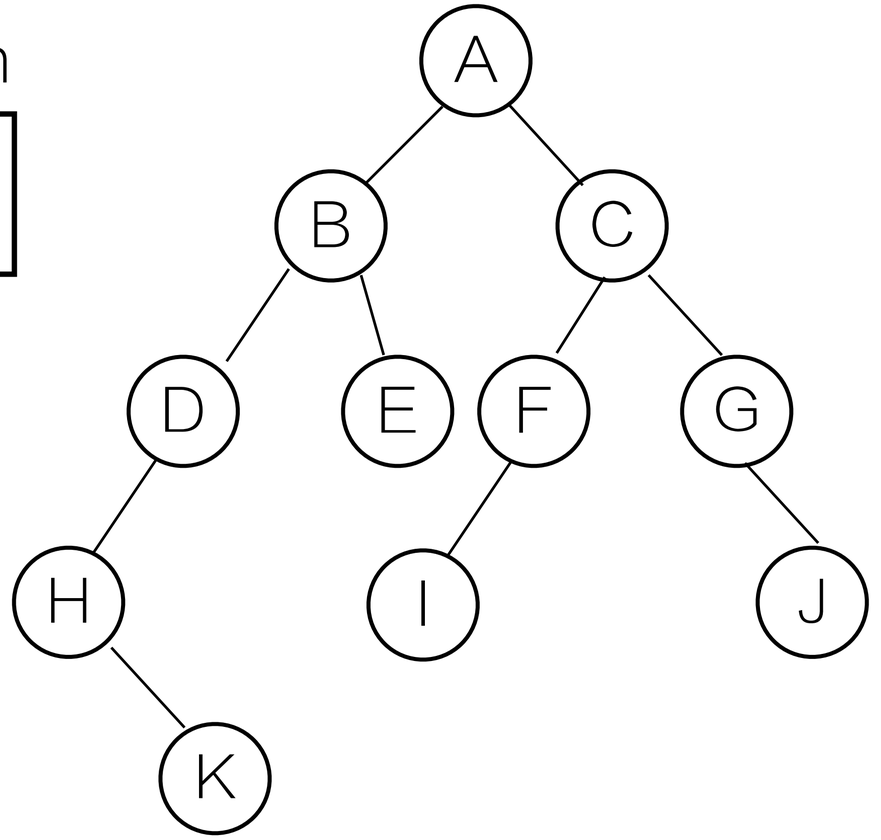
```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```

depth

0



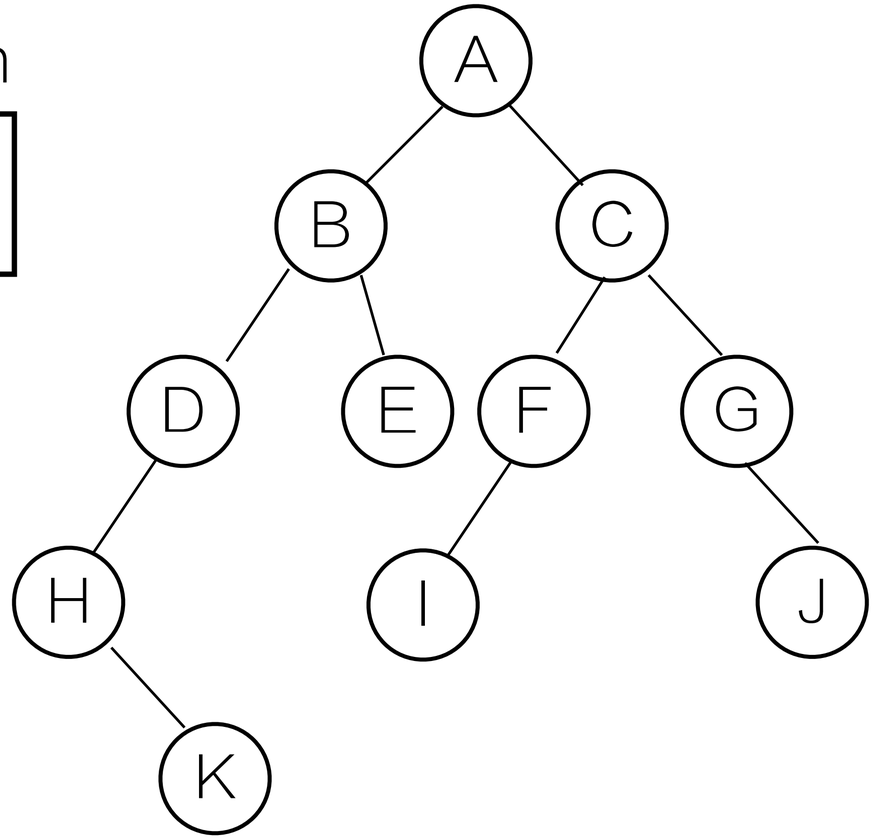
```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

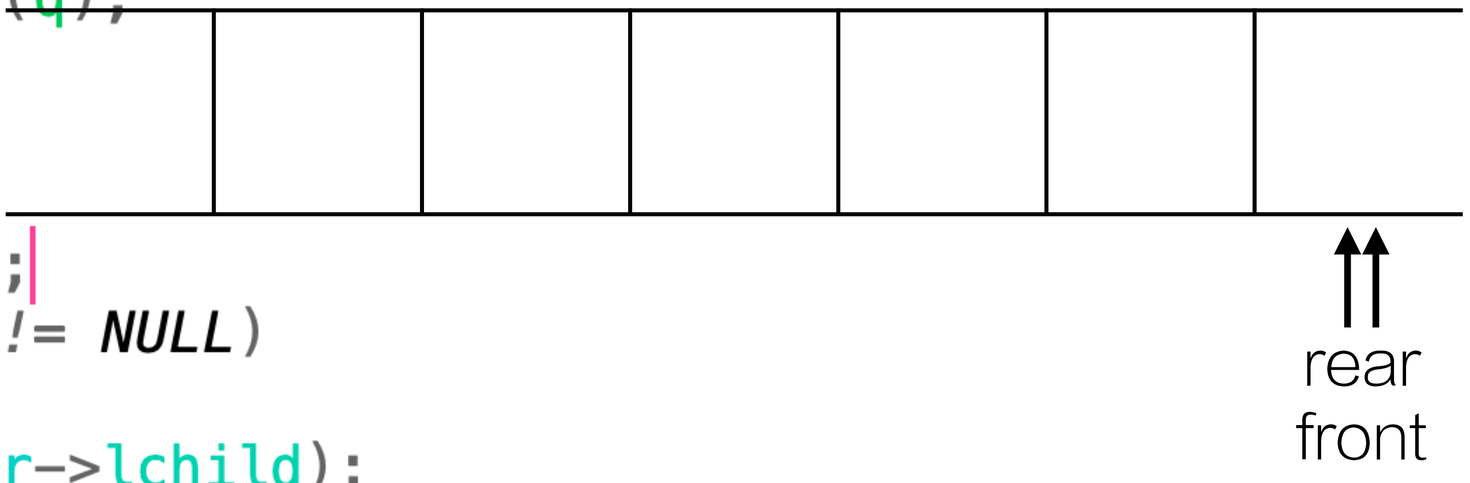
    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```

depth

0





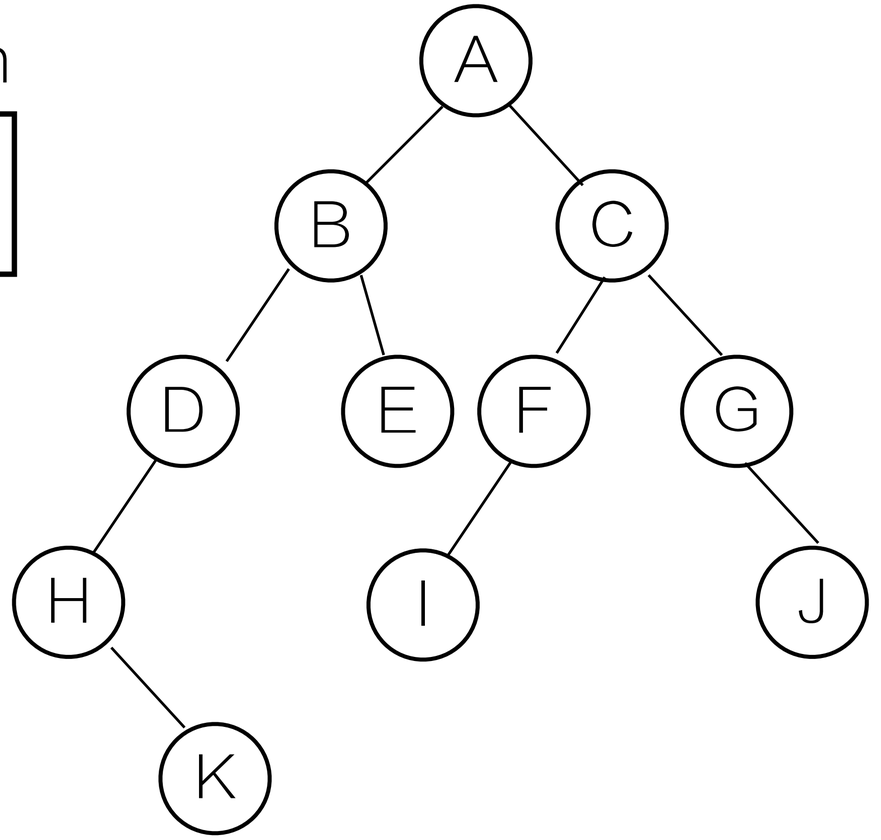
```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

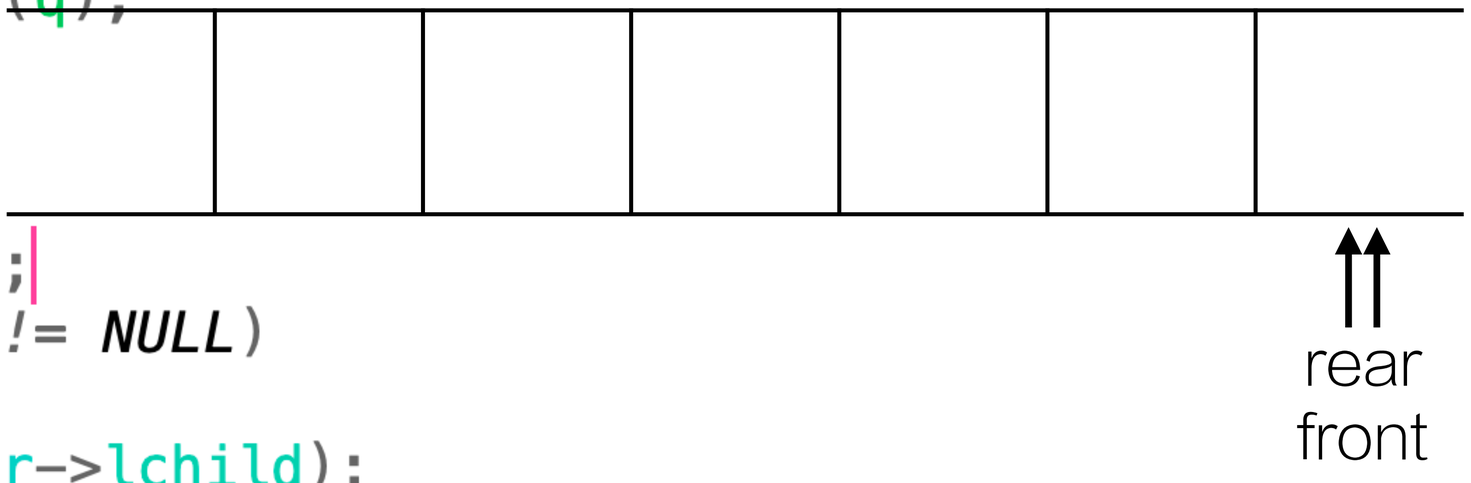
    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```

depth

0





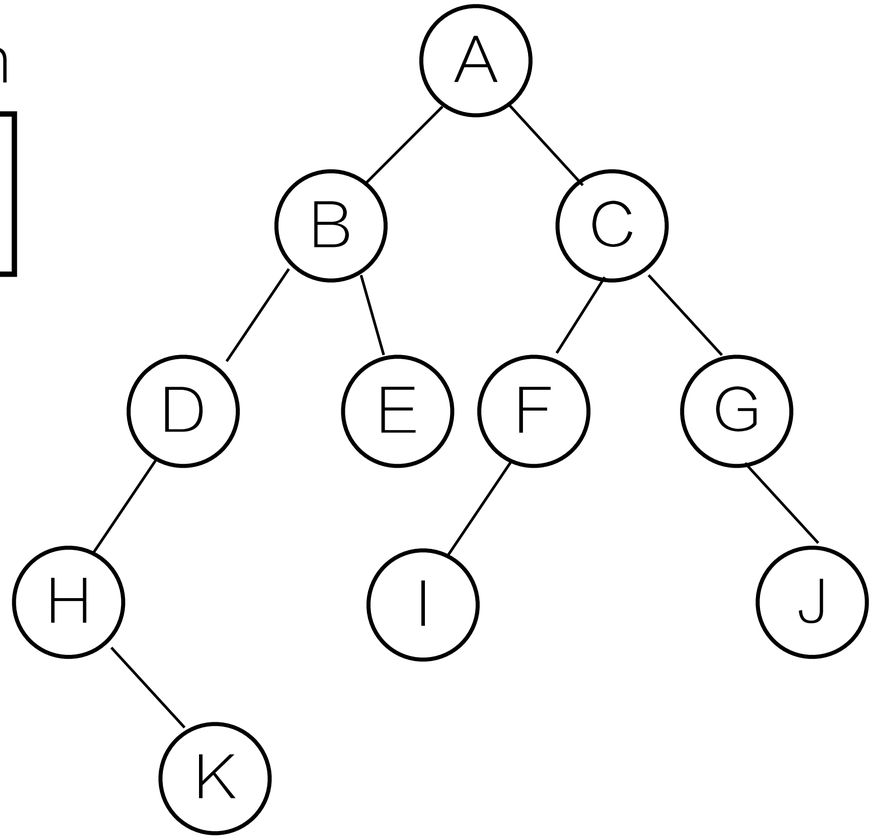
```
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

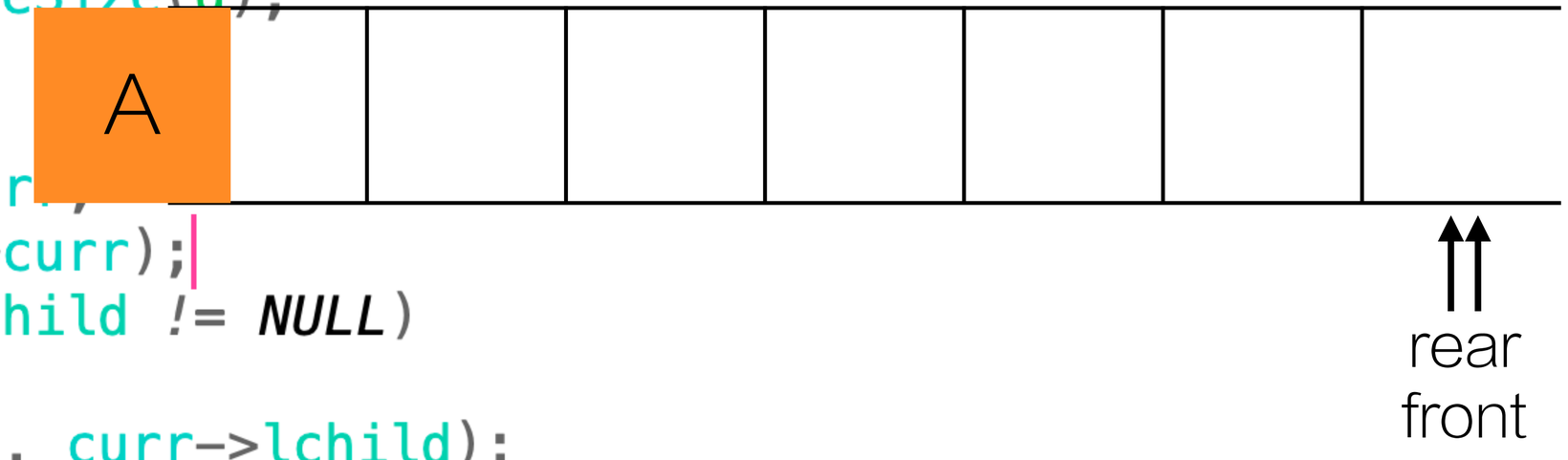
    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```

depth

0





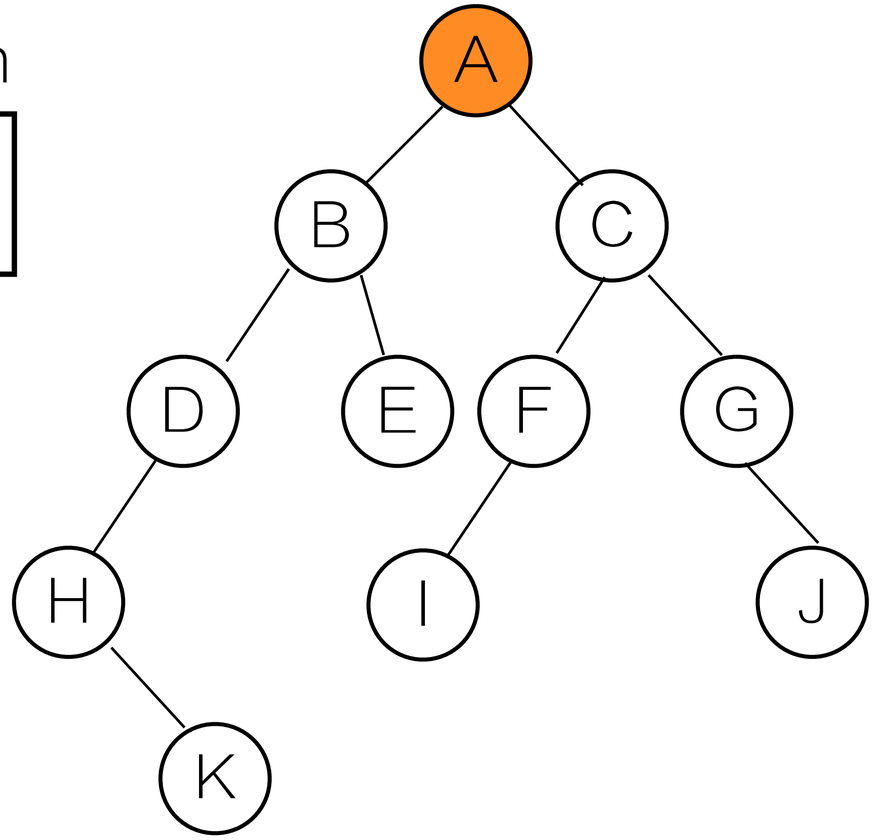
```c
int maxDepth(TreeNode* root)
{
    if (root == NULL)
    {
        return 0;
    }

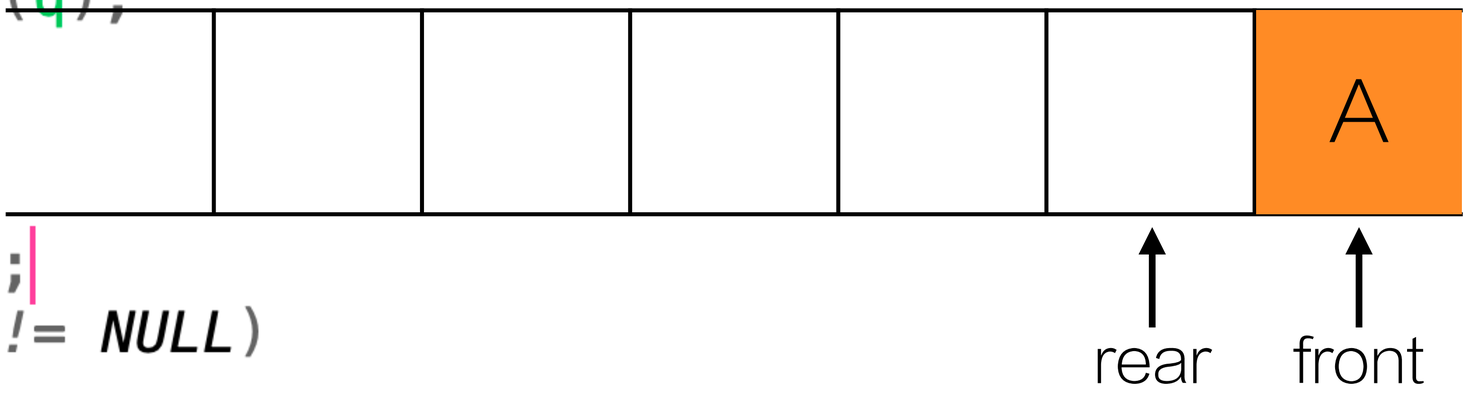
    int depth = 0;
    Queue *q = initQueue();
    equeue(q, root);

    while(!isEmpty(q))
    {
        int count = queueSize(q);
        while(count > 0)
        {
            TreeNode* curr;
            dequeue(q, &curr);
            if (curr->lchild != NULL)
            {
                equeue(q, curr->lchild);
```

depth

0

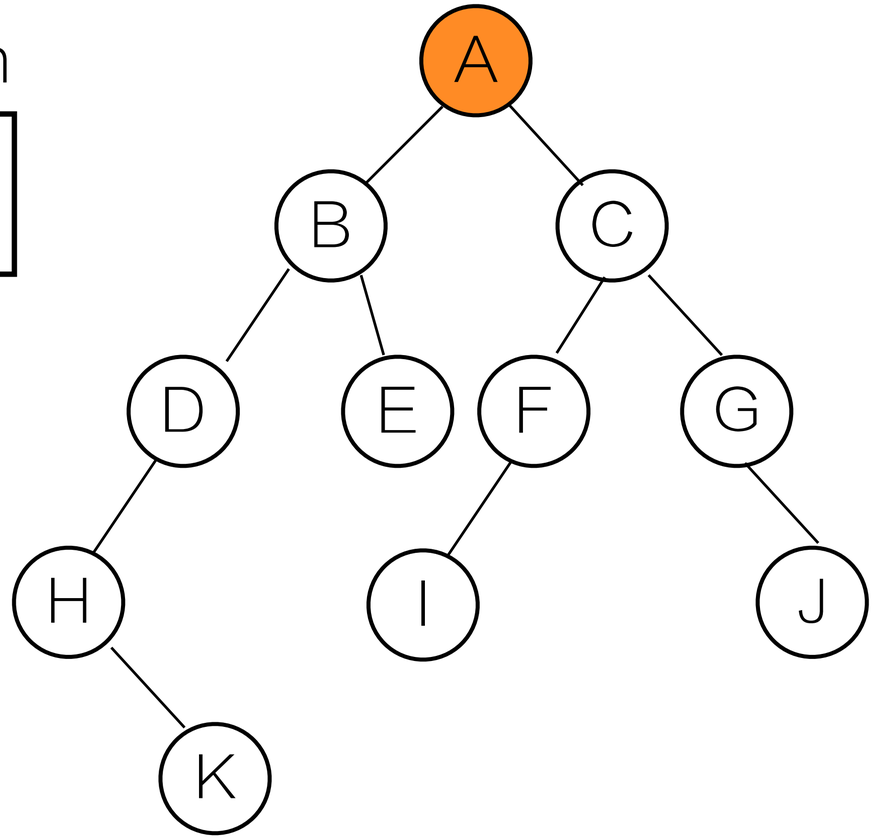


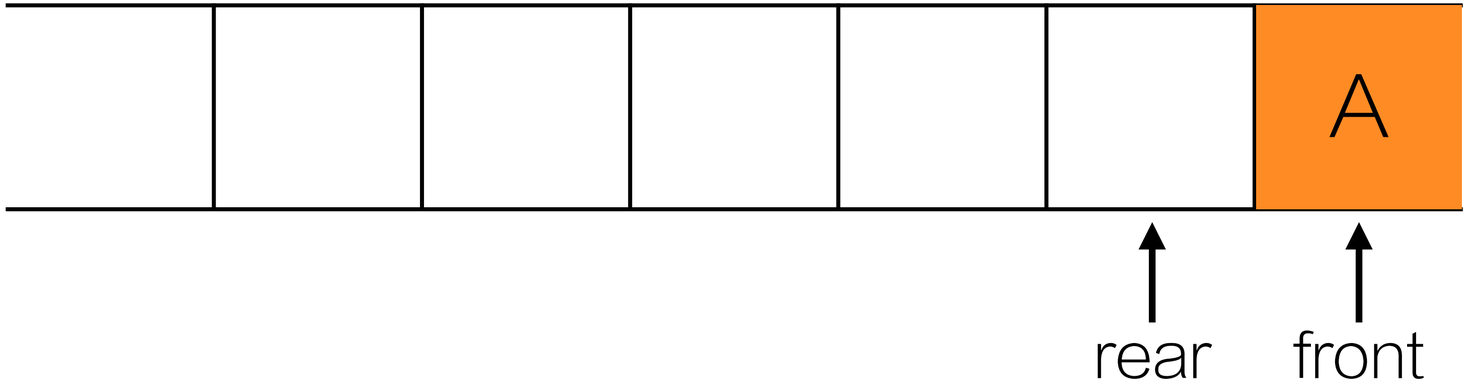


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

depth

0

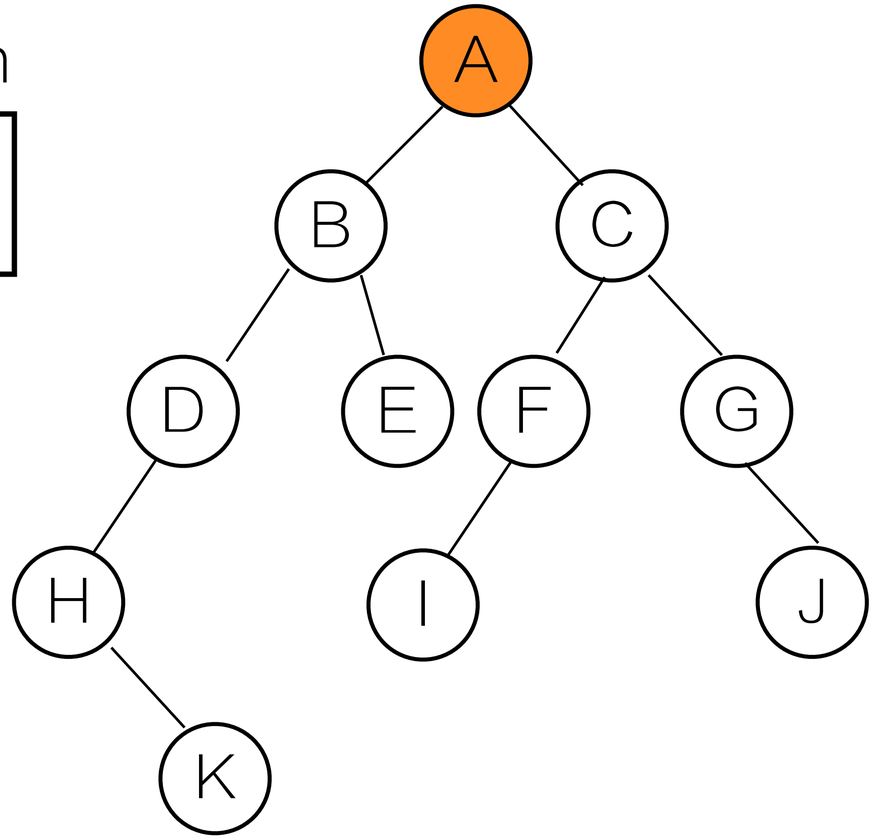


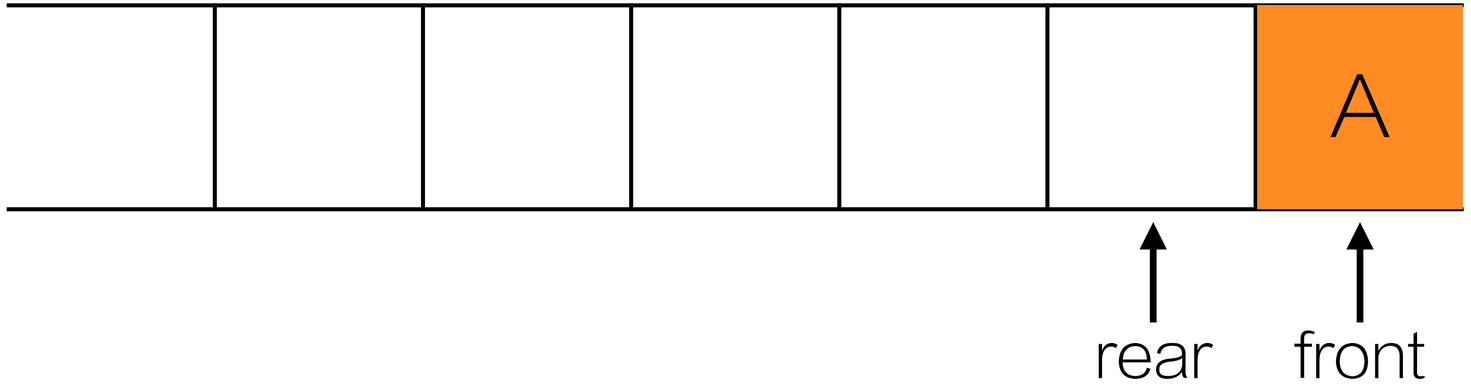


```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

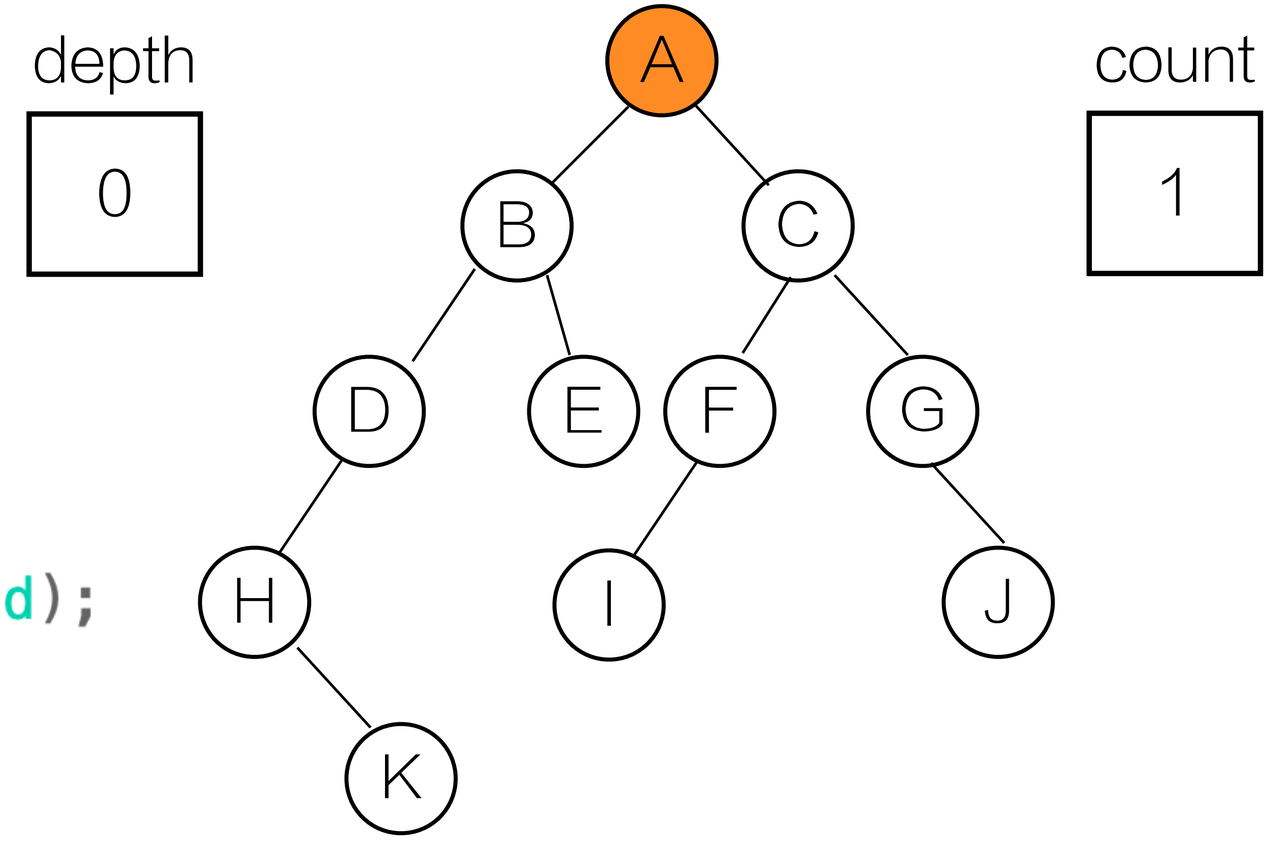
depth

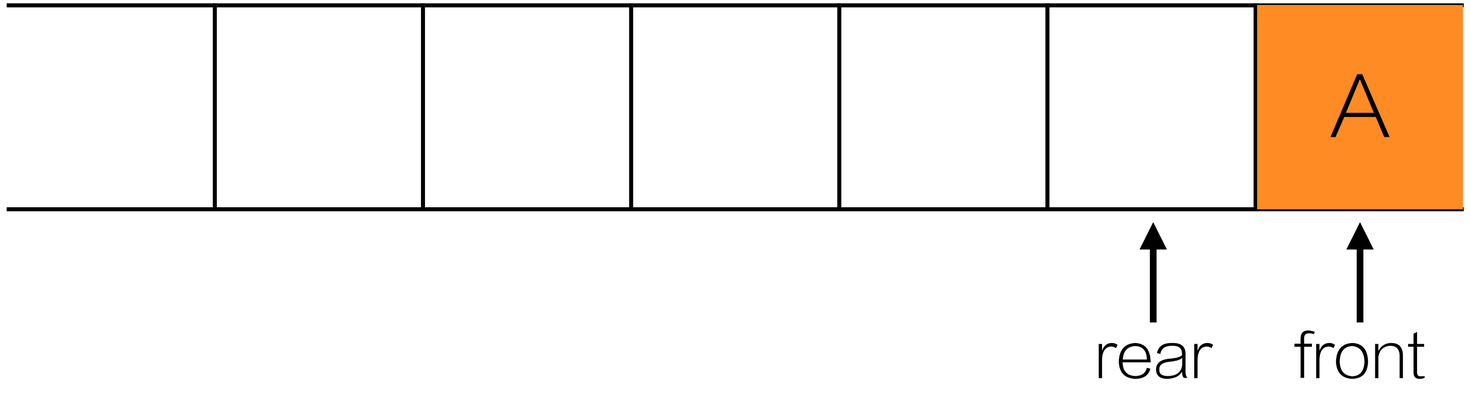
0



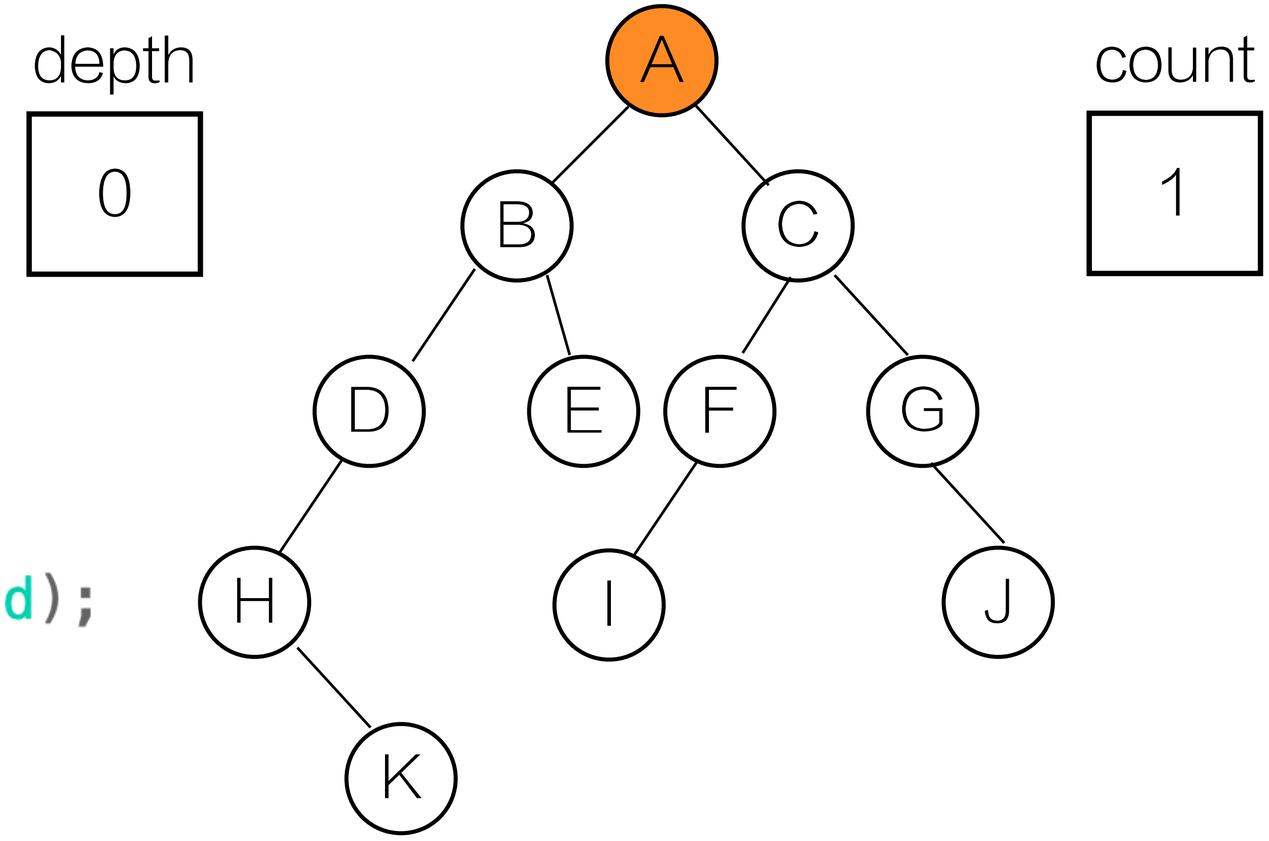


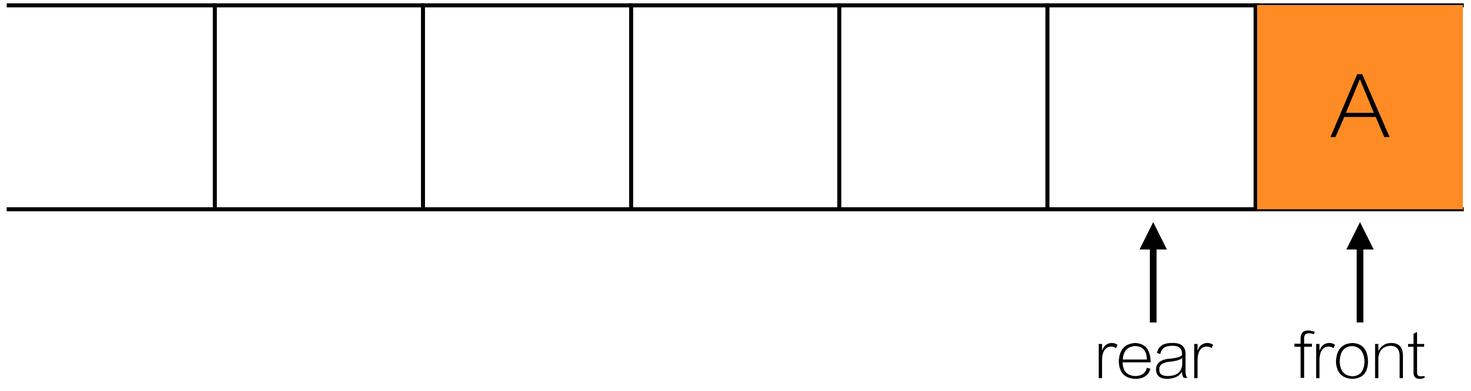
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



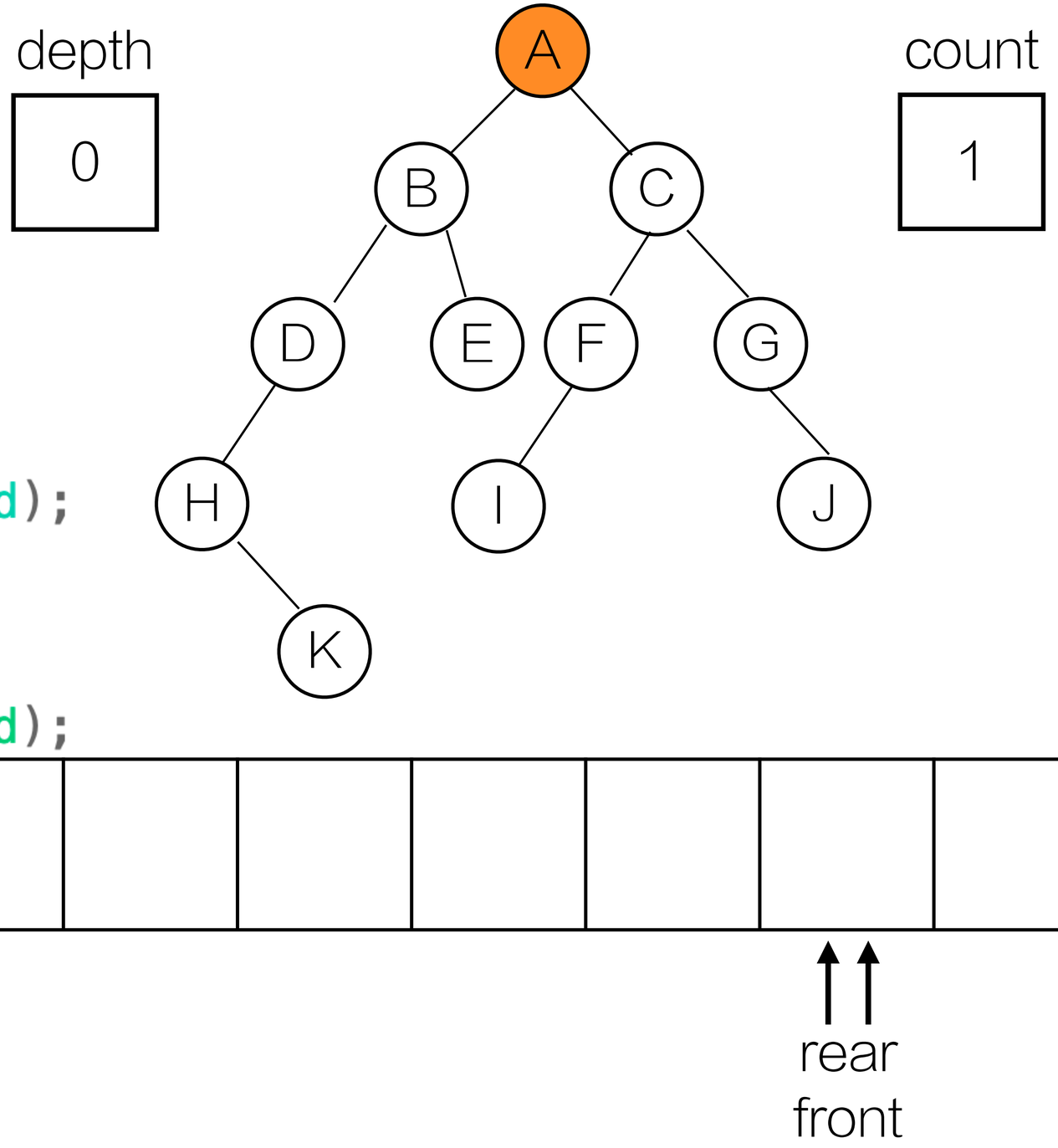


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```





```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



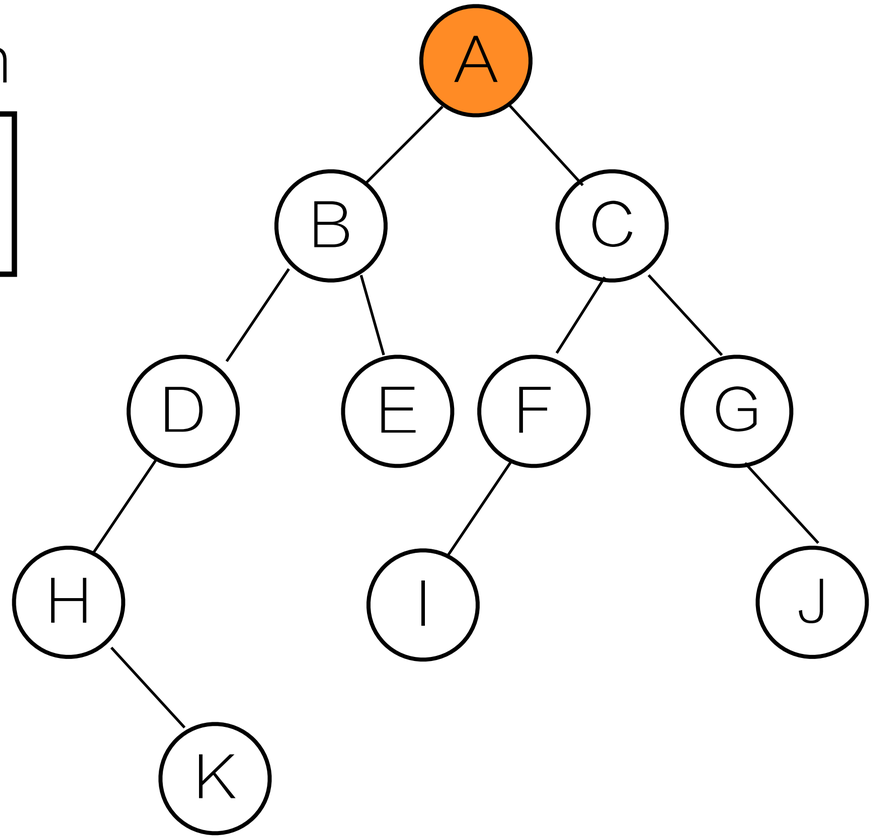
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

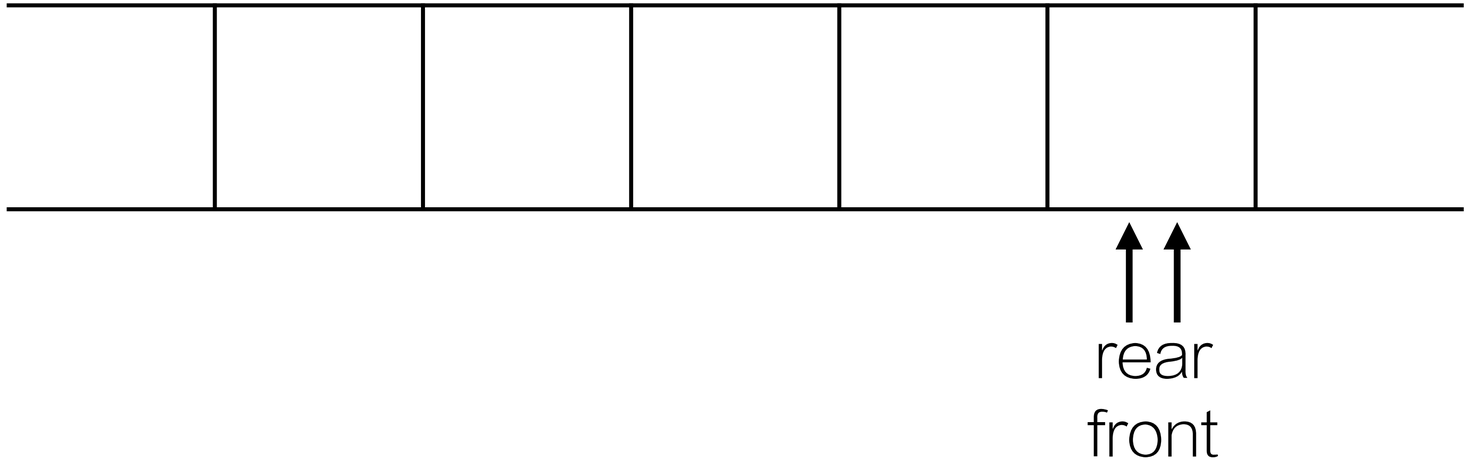
depth

0

count

1





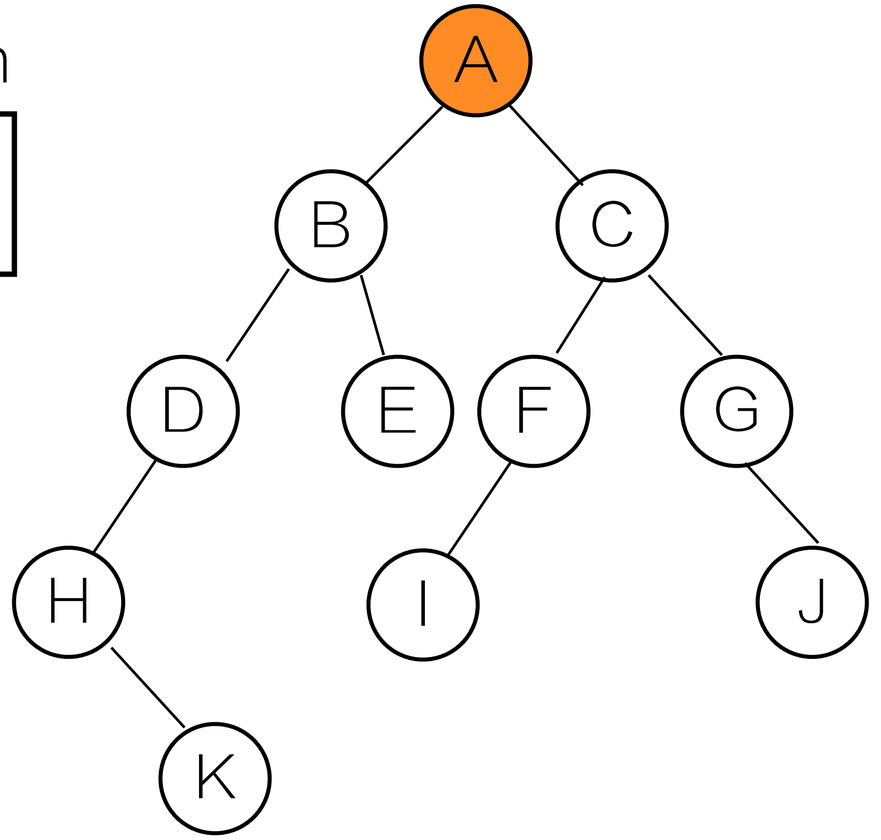
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

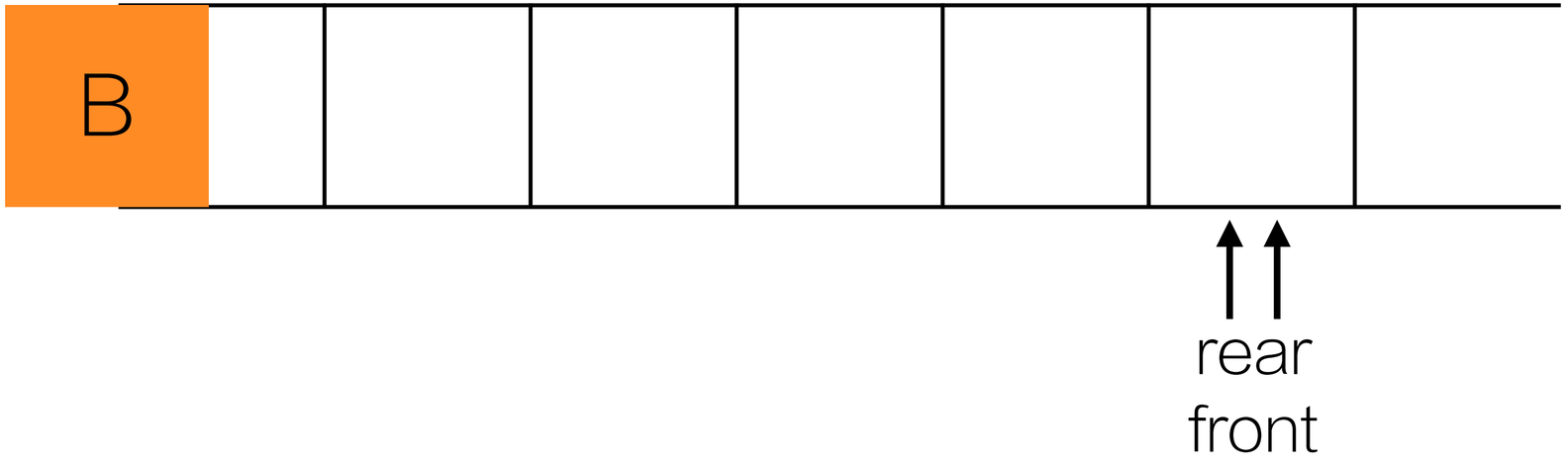
depth

0

count

1





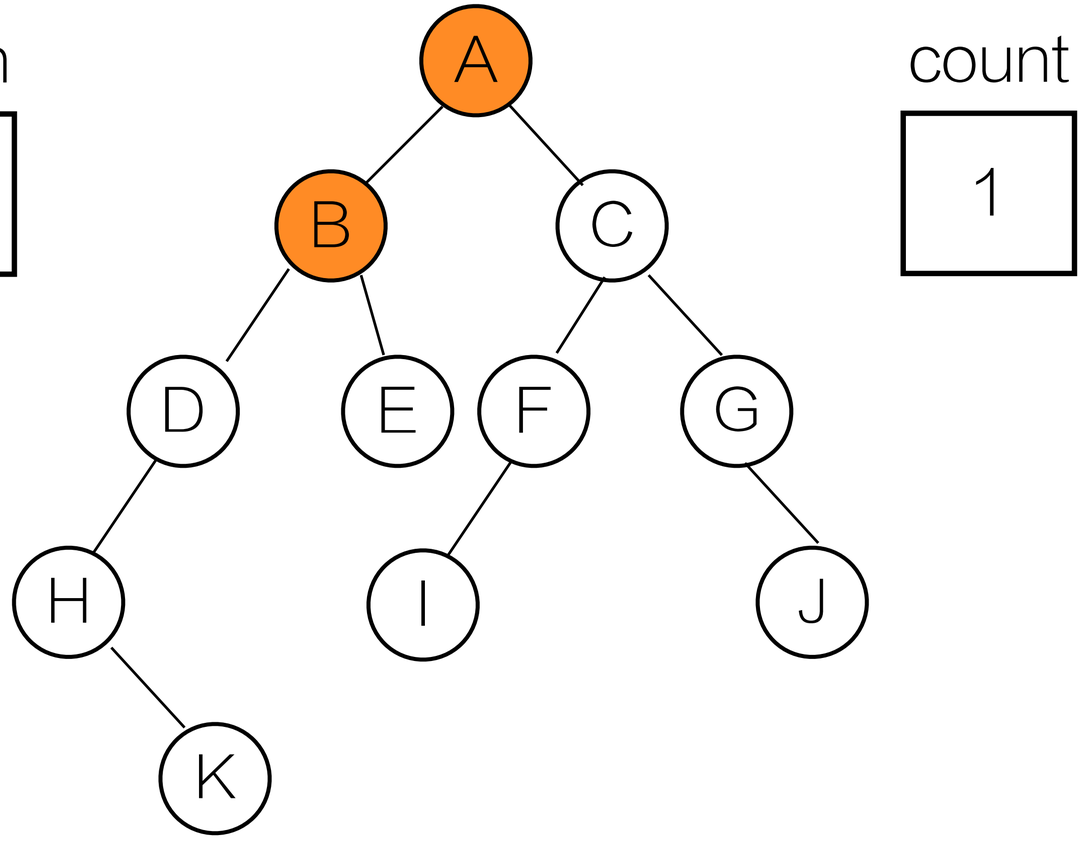
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

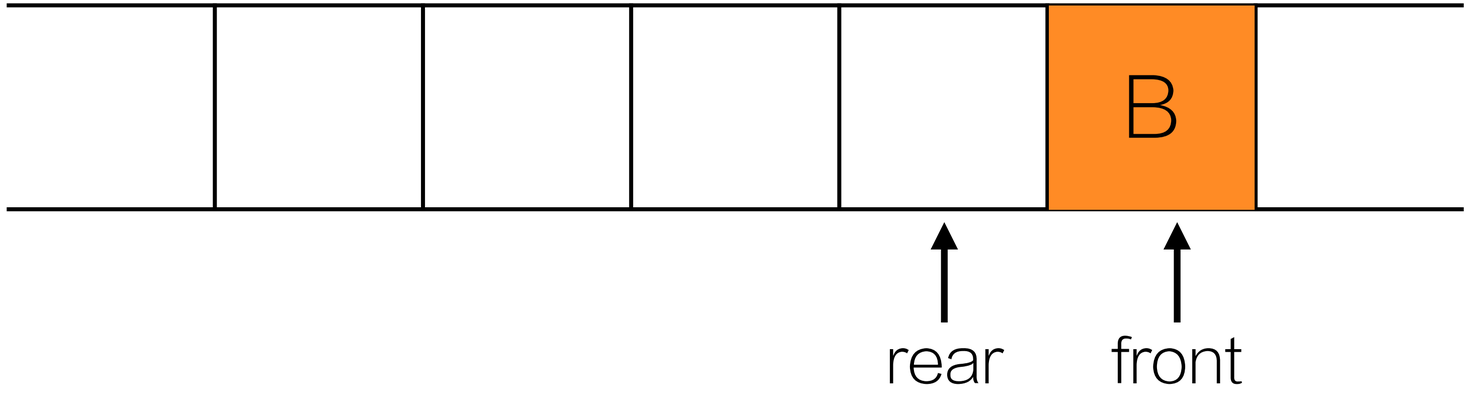
depth

0

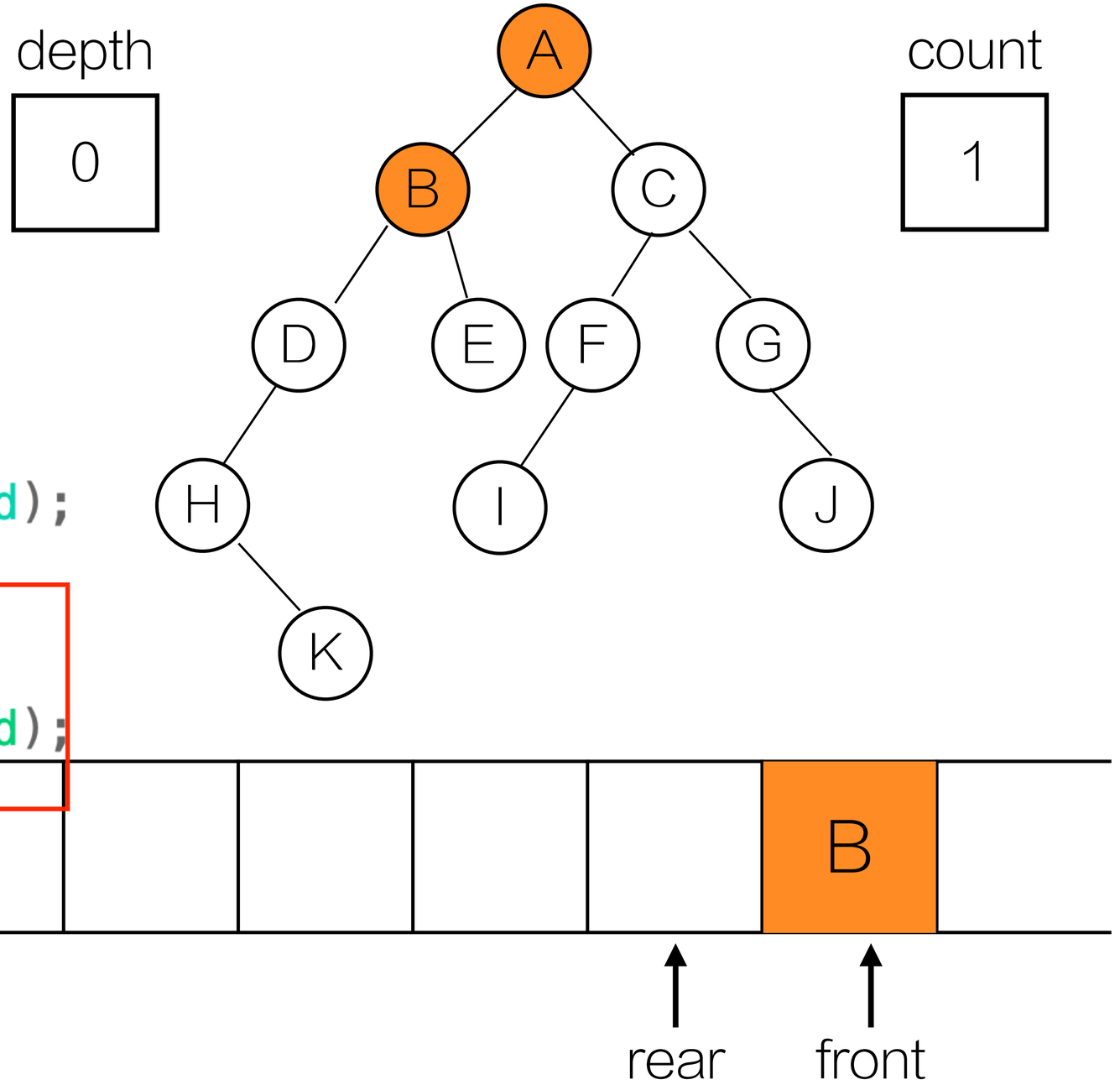
count

1





```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



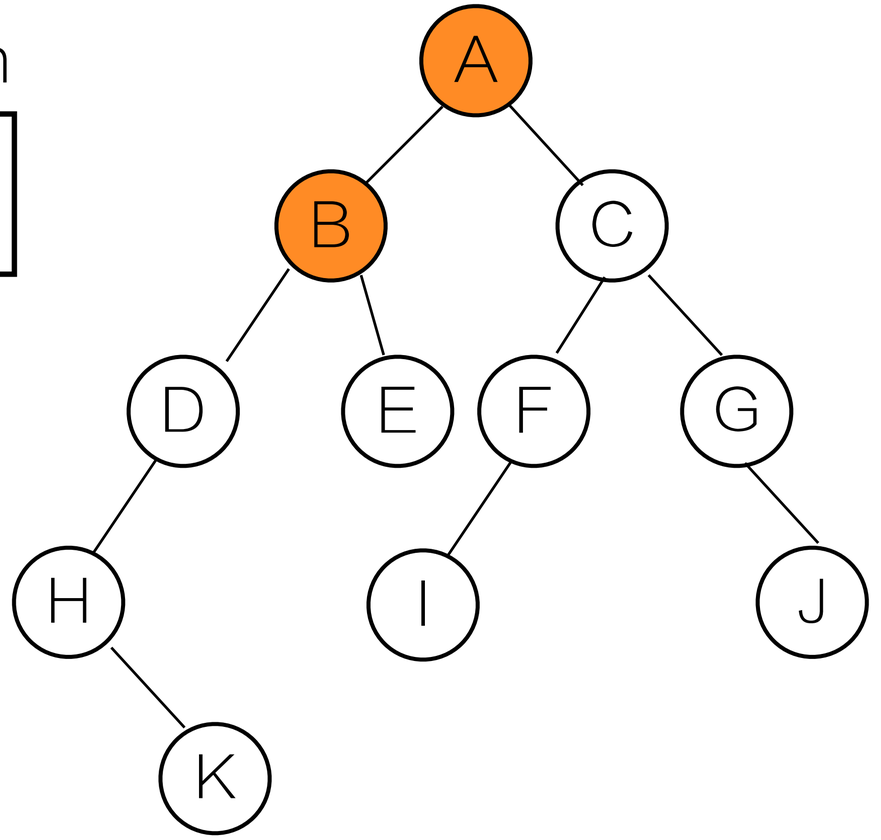
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

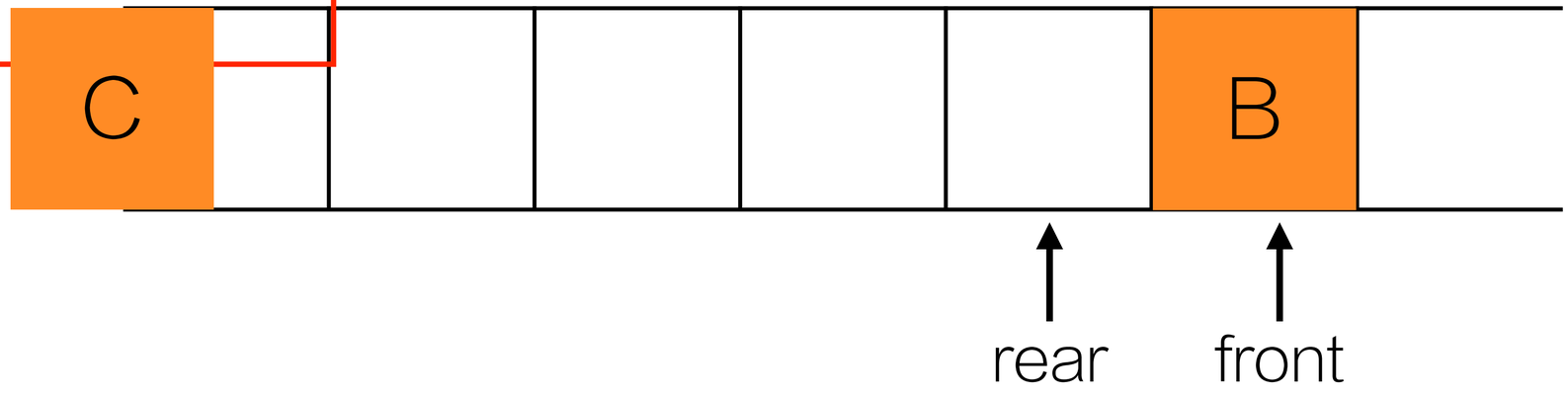
depth

0

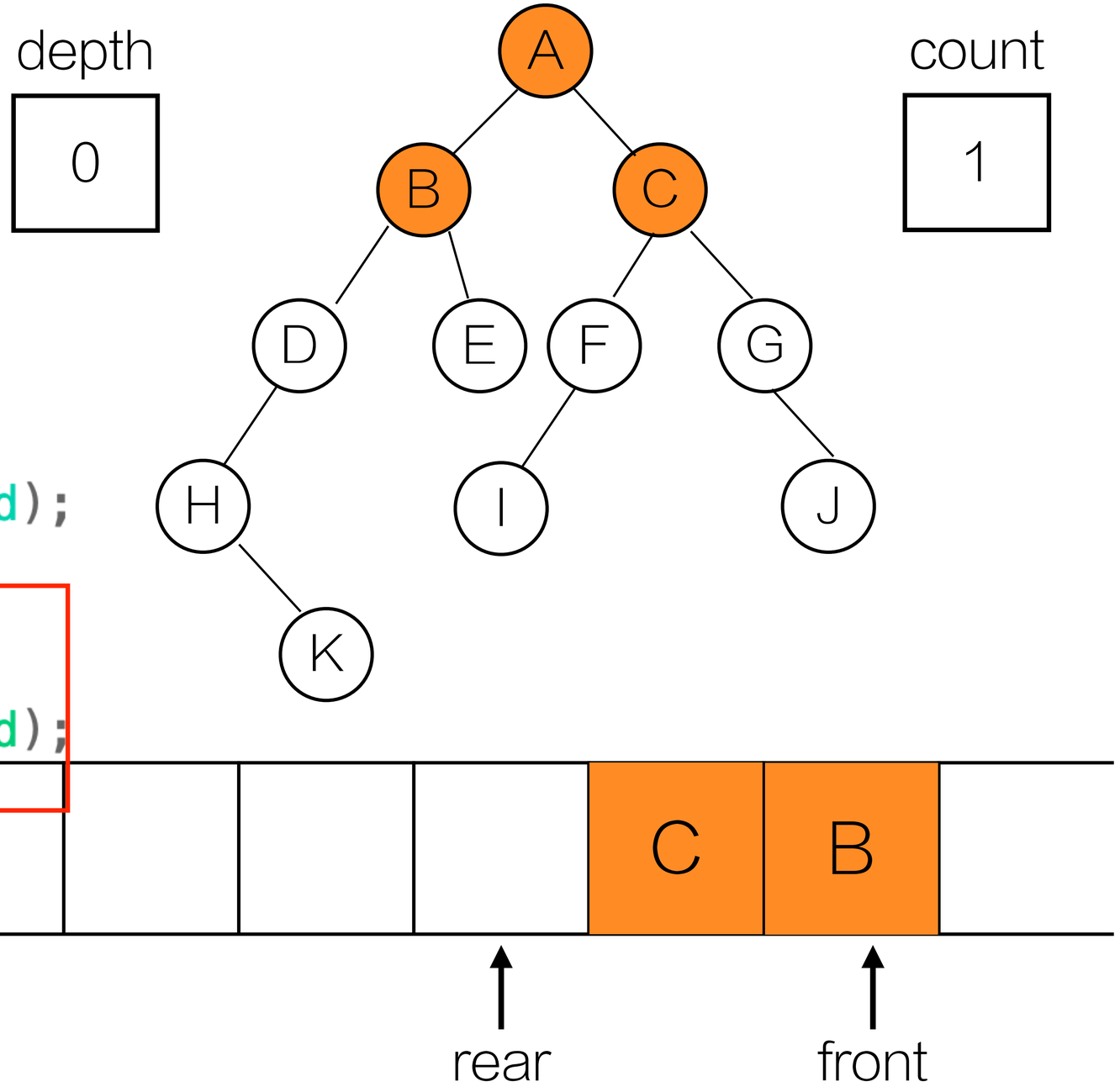
count

1

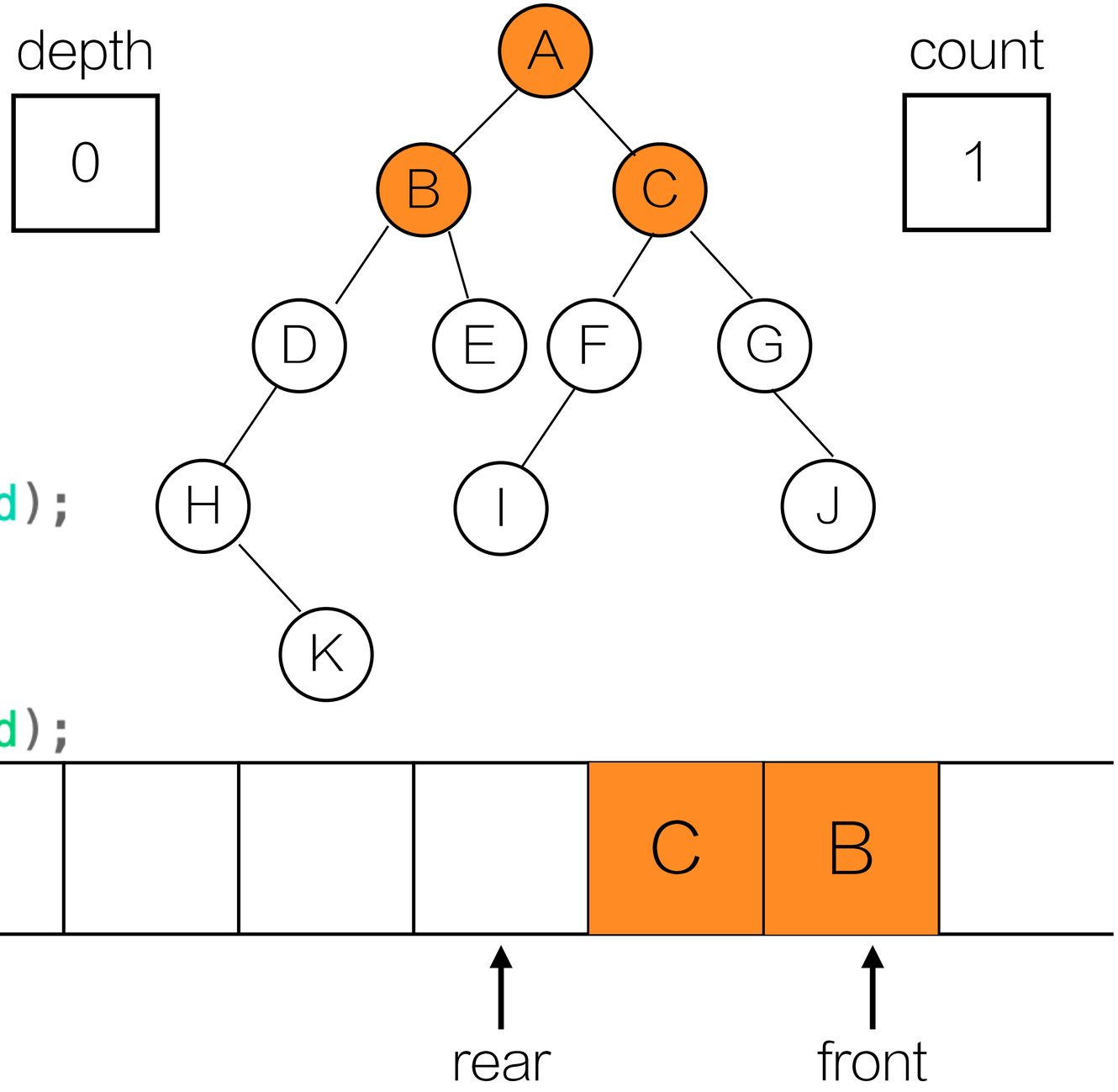




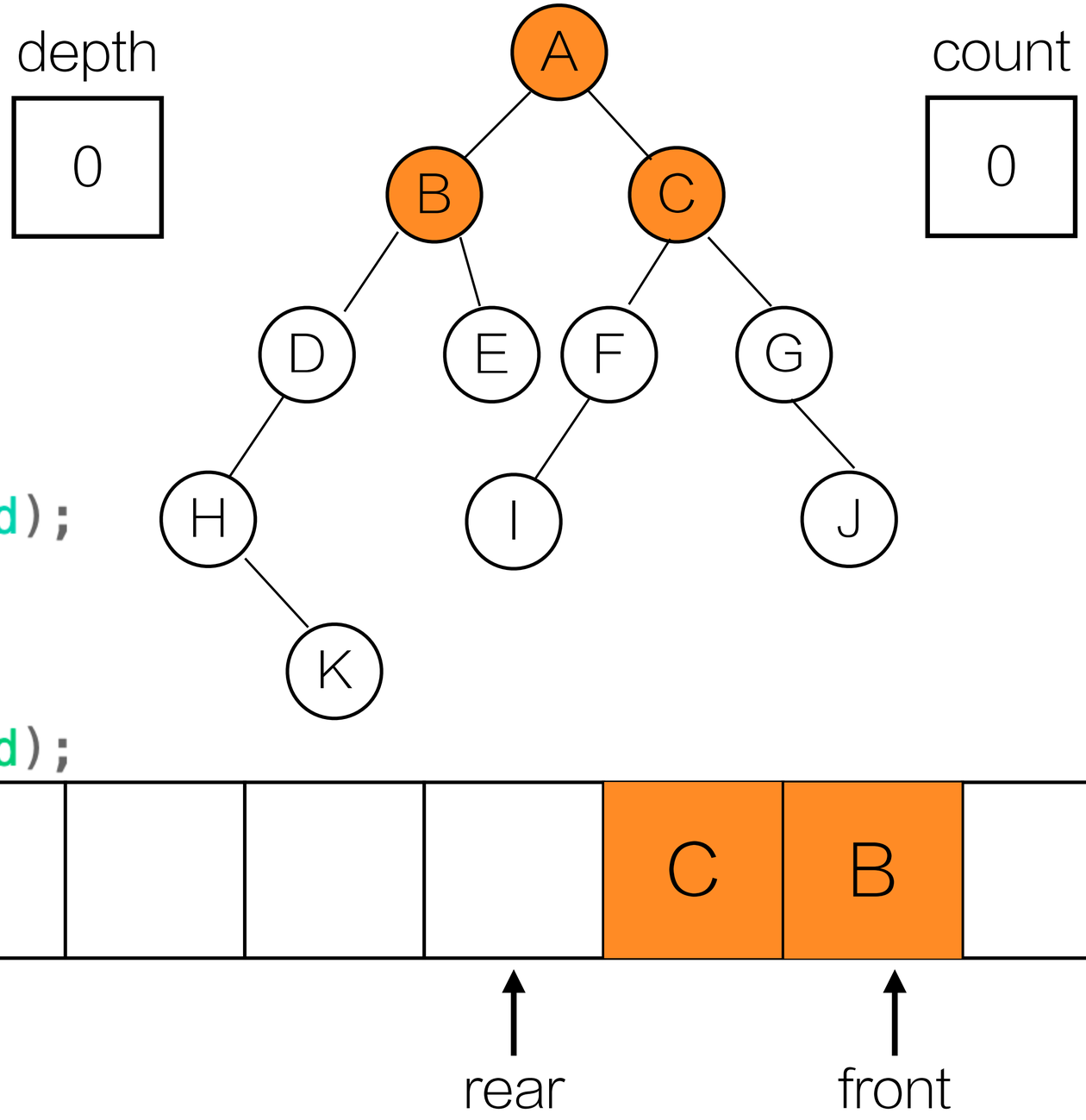
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



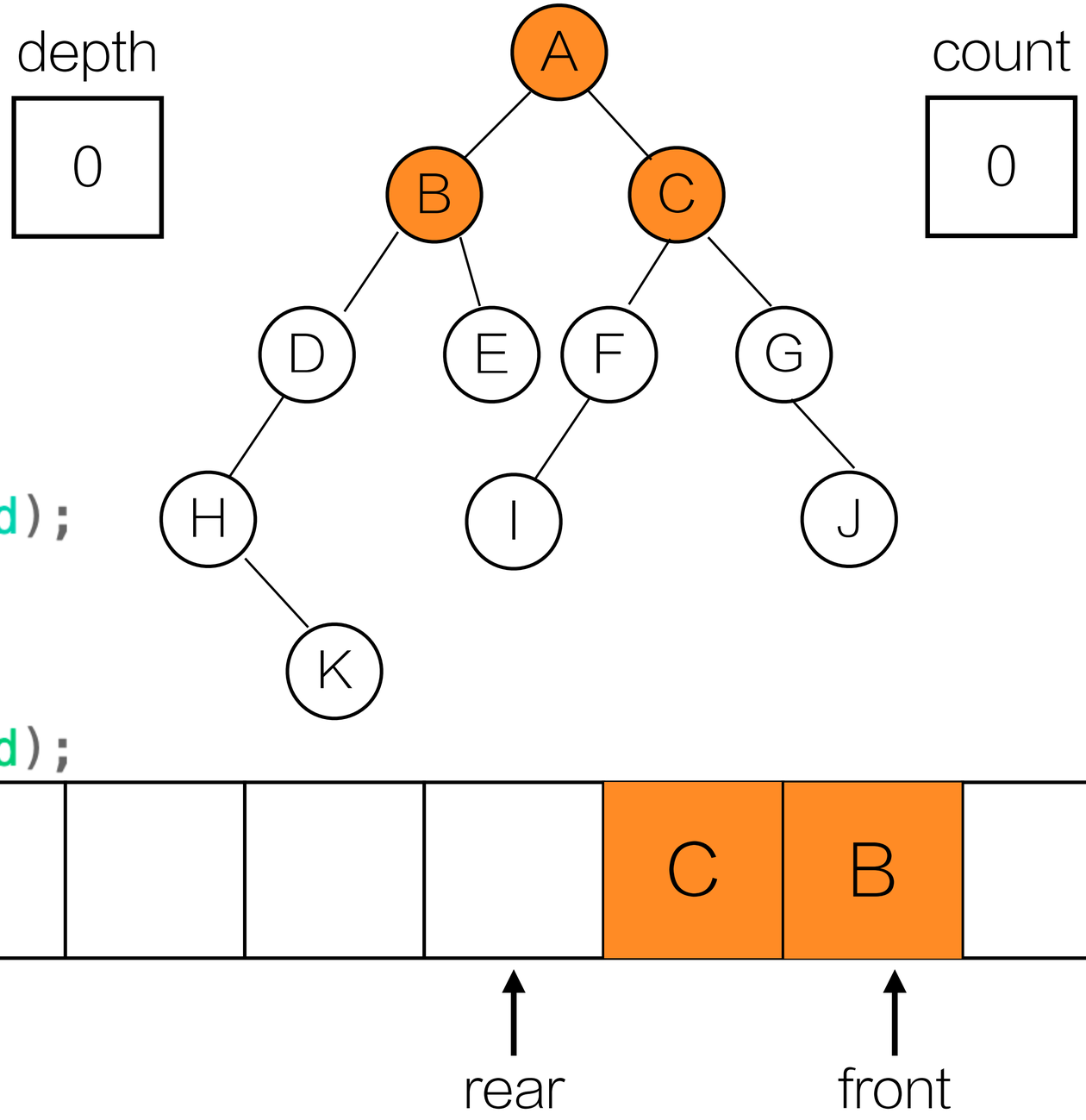
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



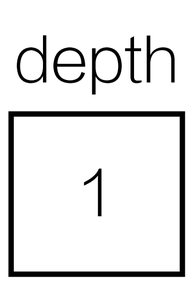
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

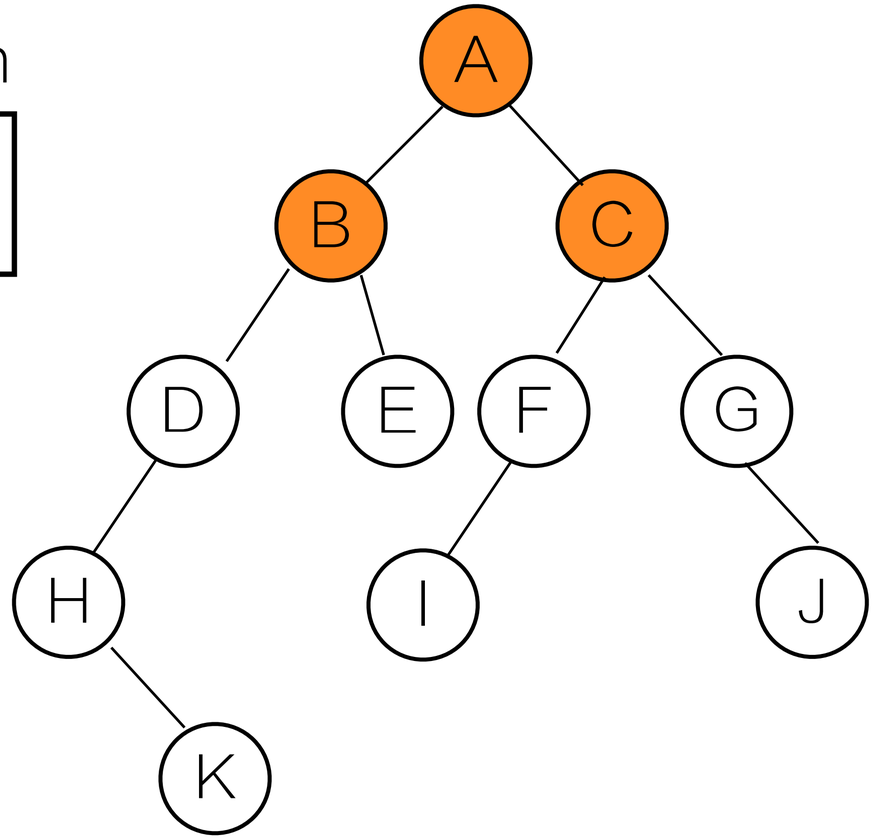


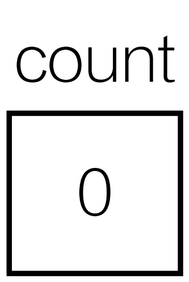
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

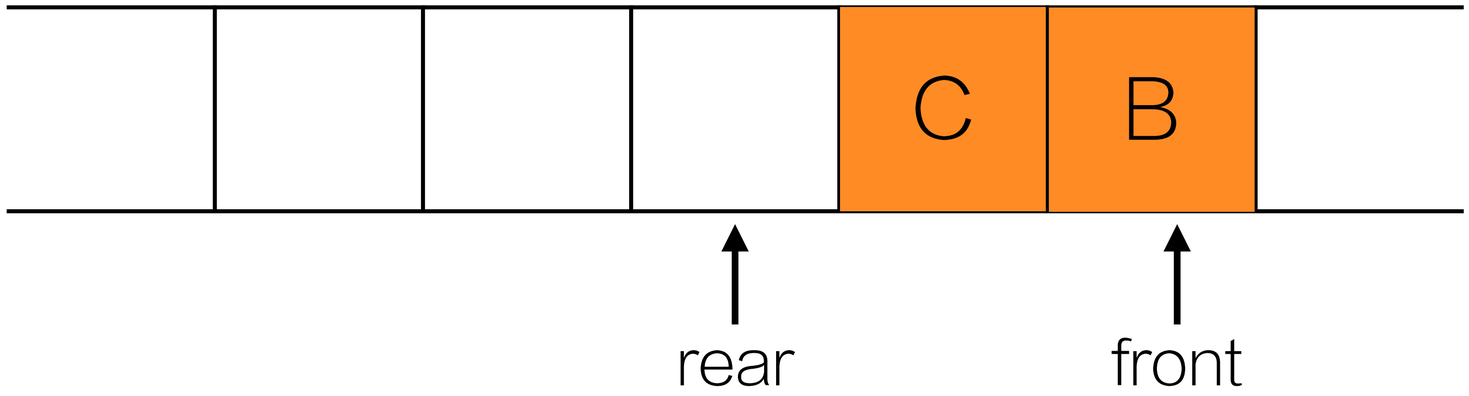


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

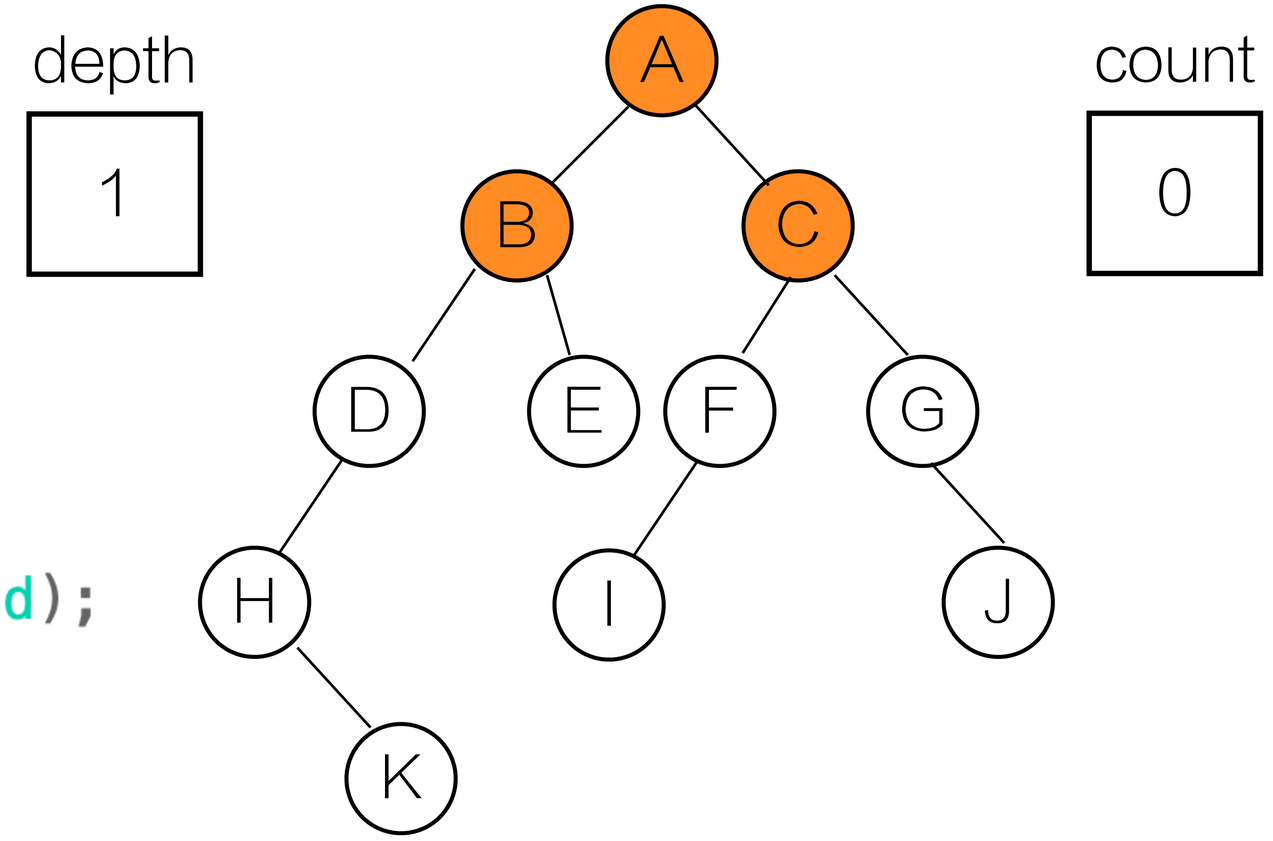


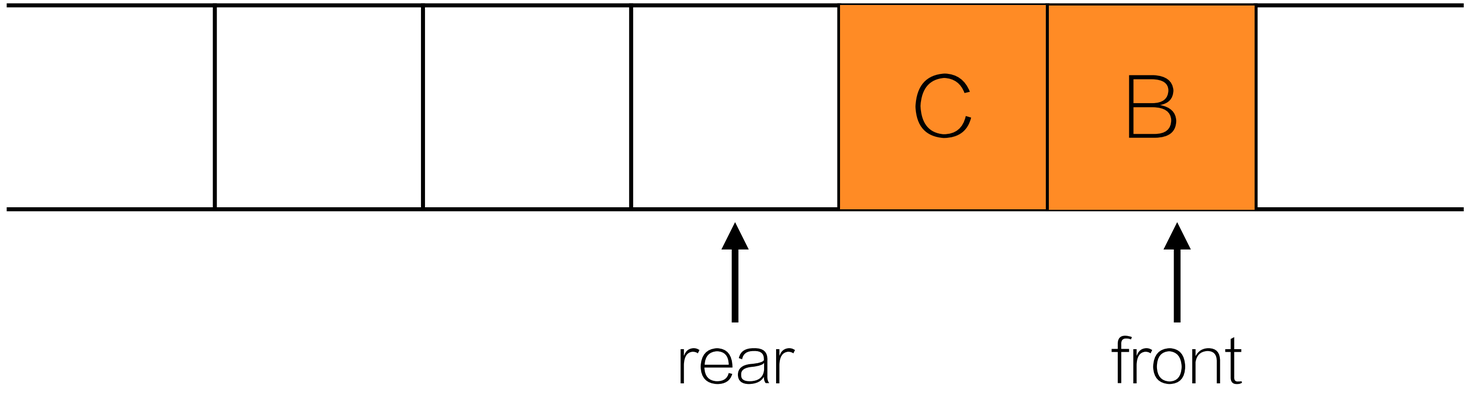




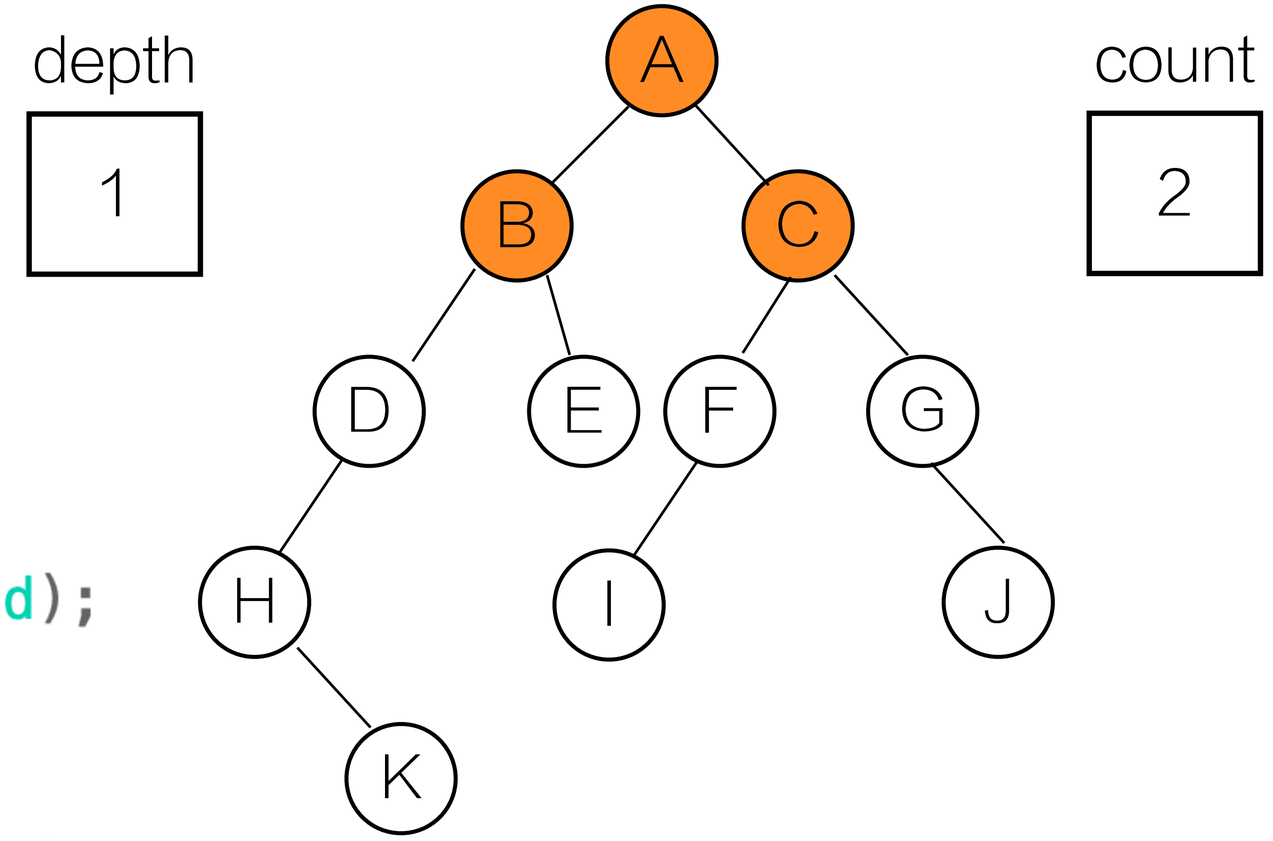


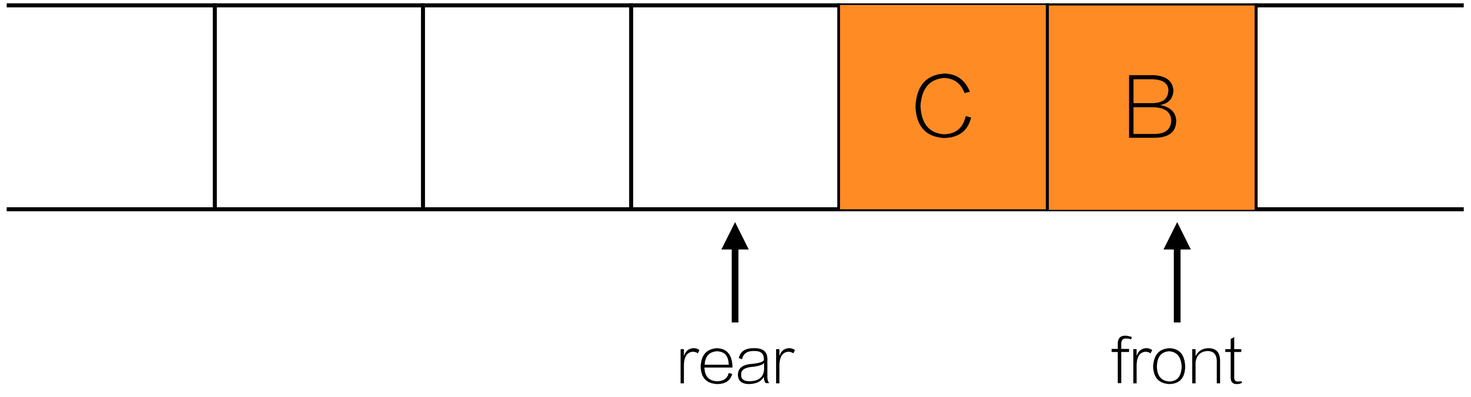
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



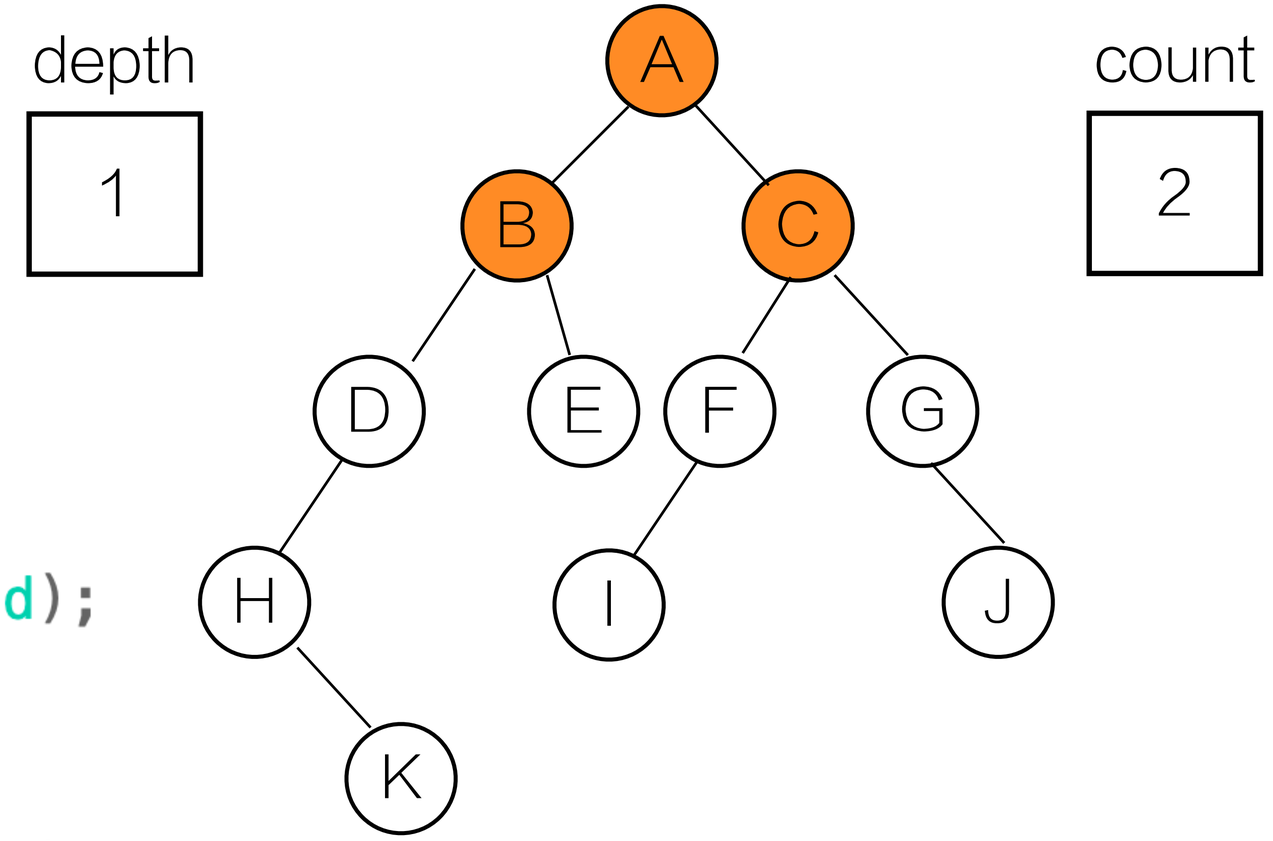


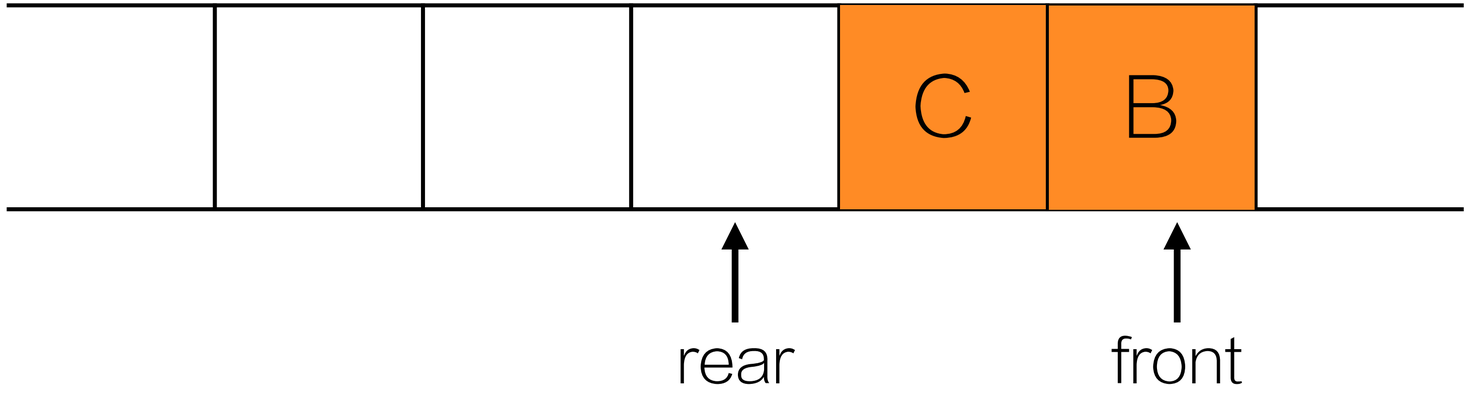
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



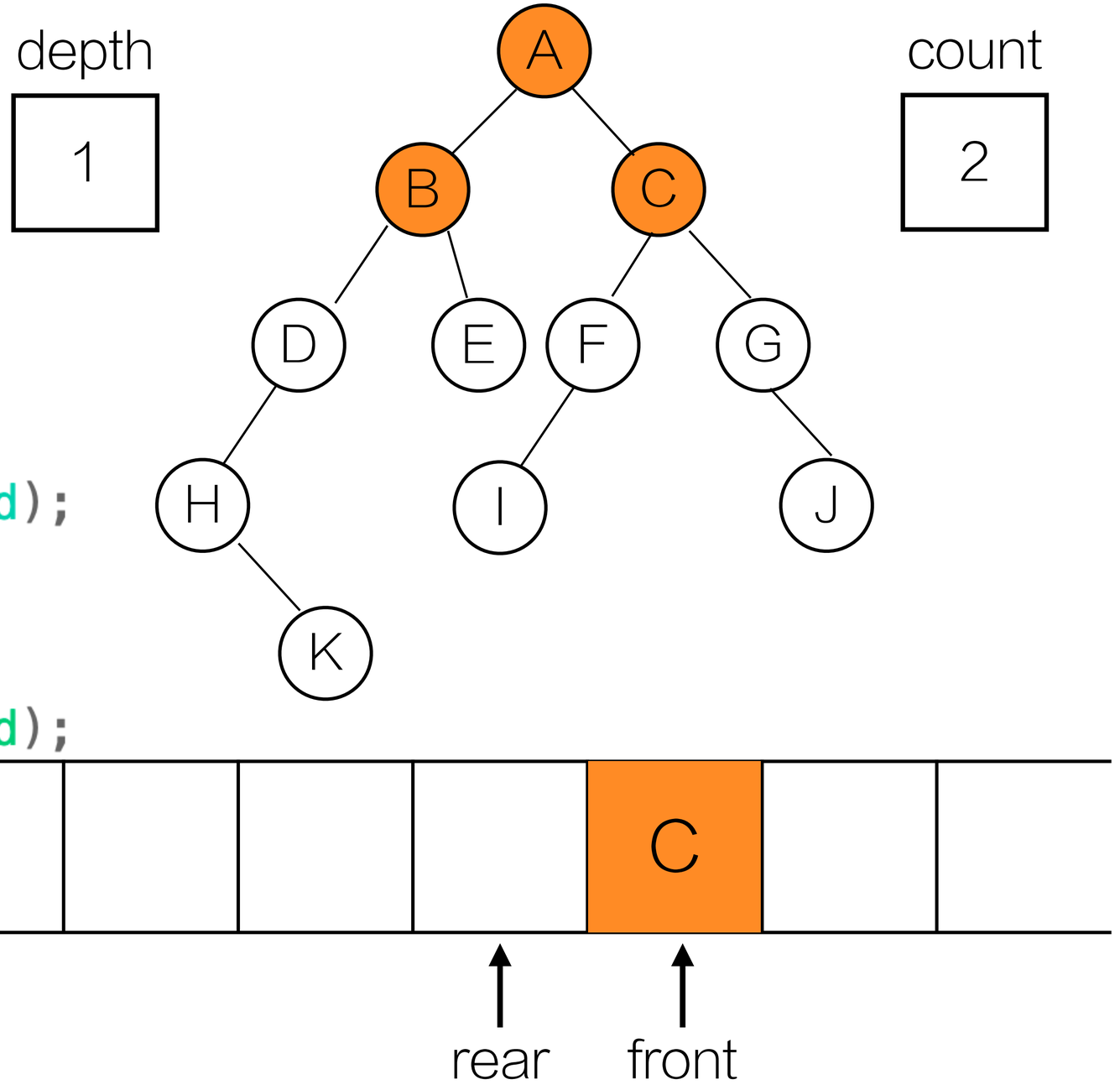


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

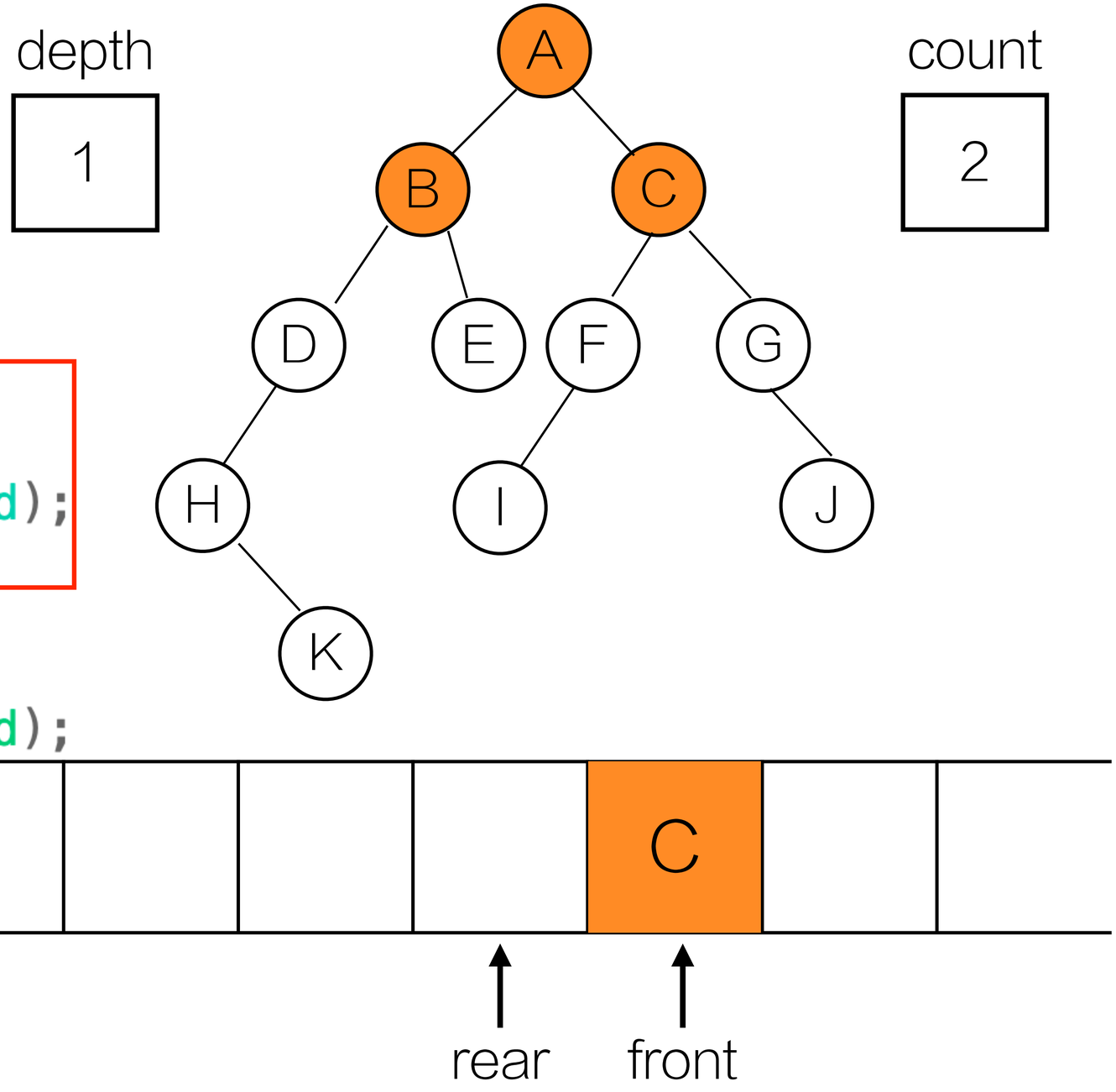




```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



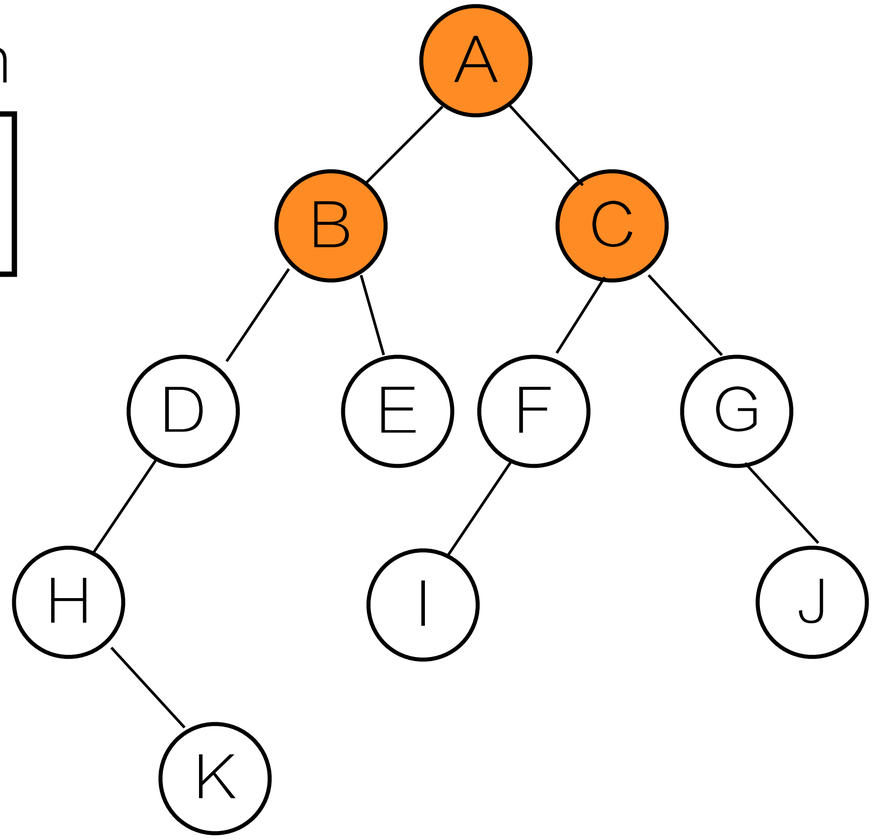
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

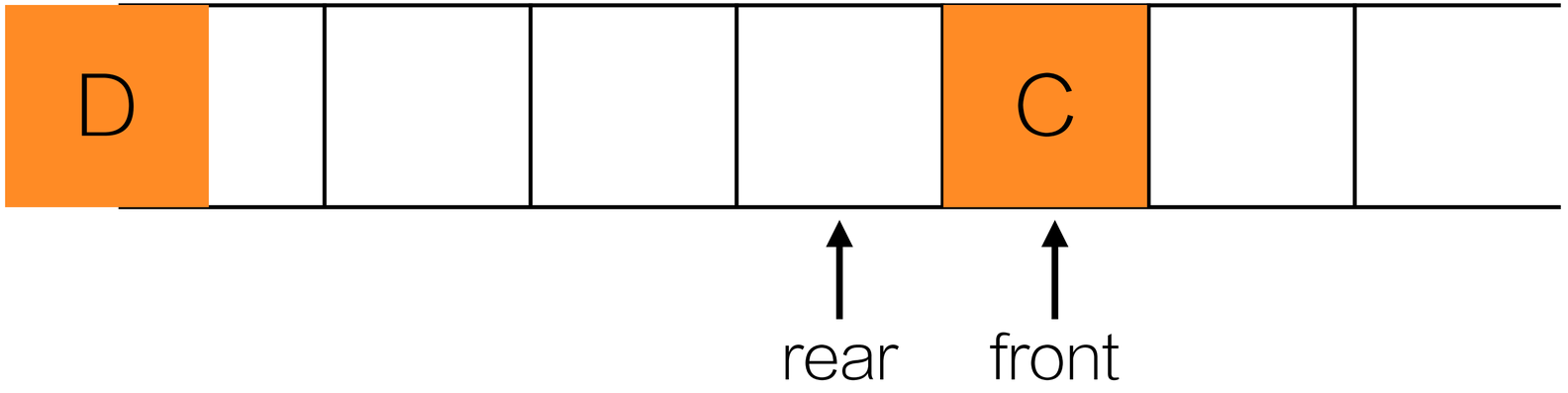
depth

1

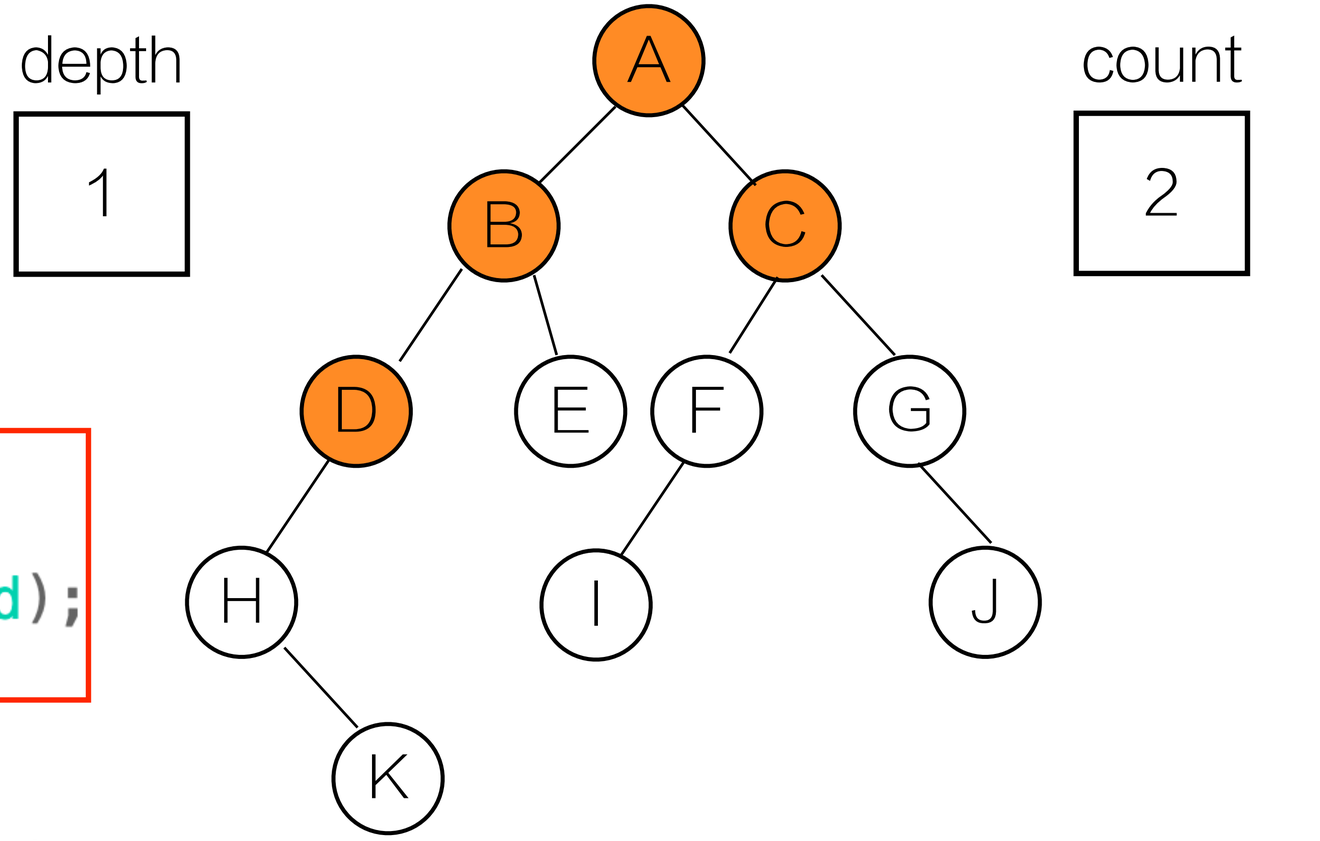
count

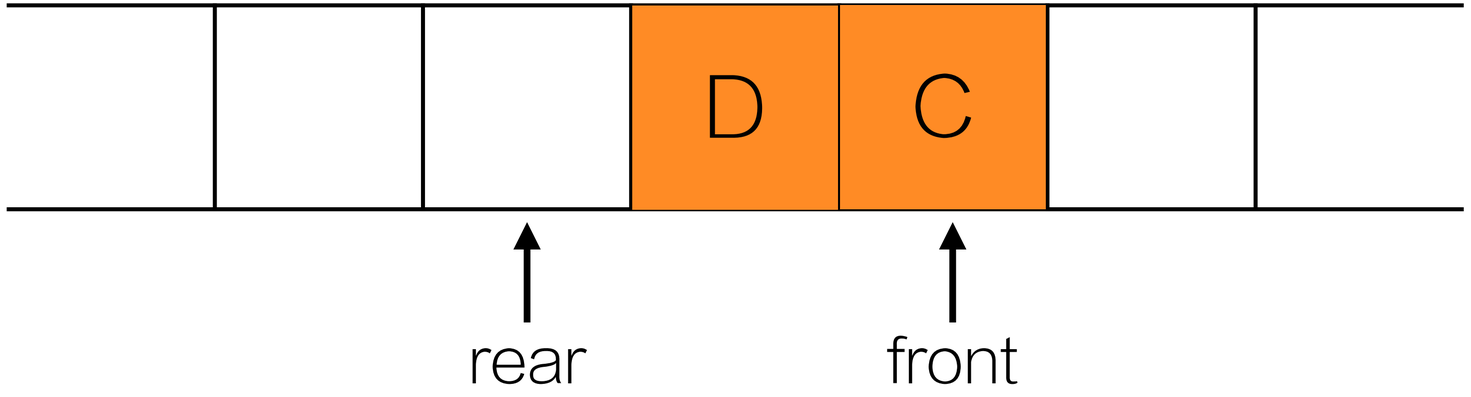
2



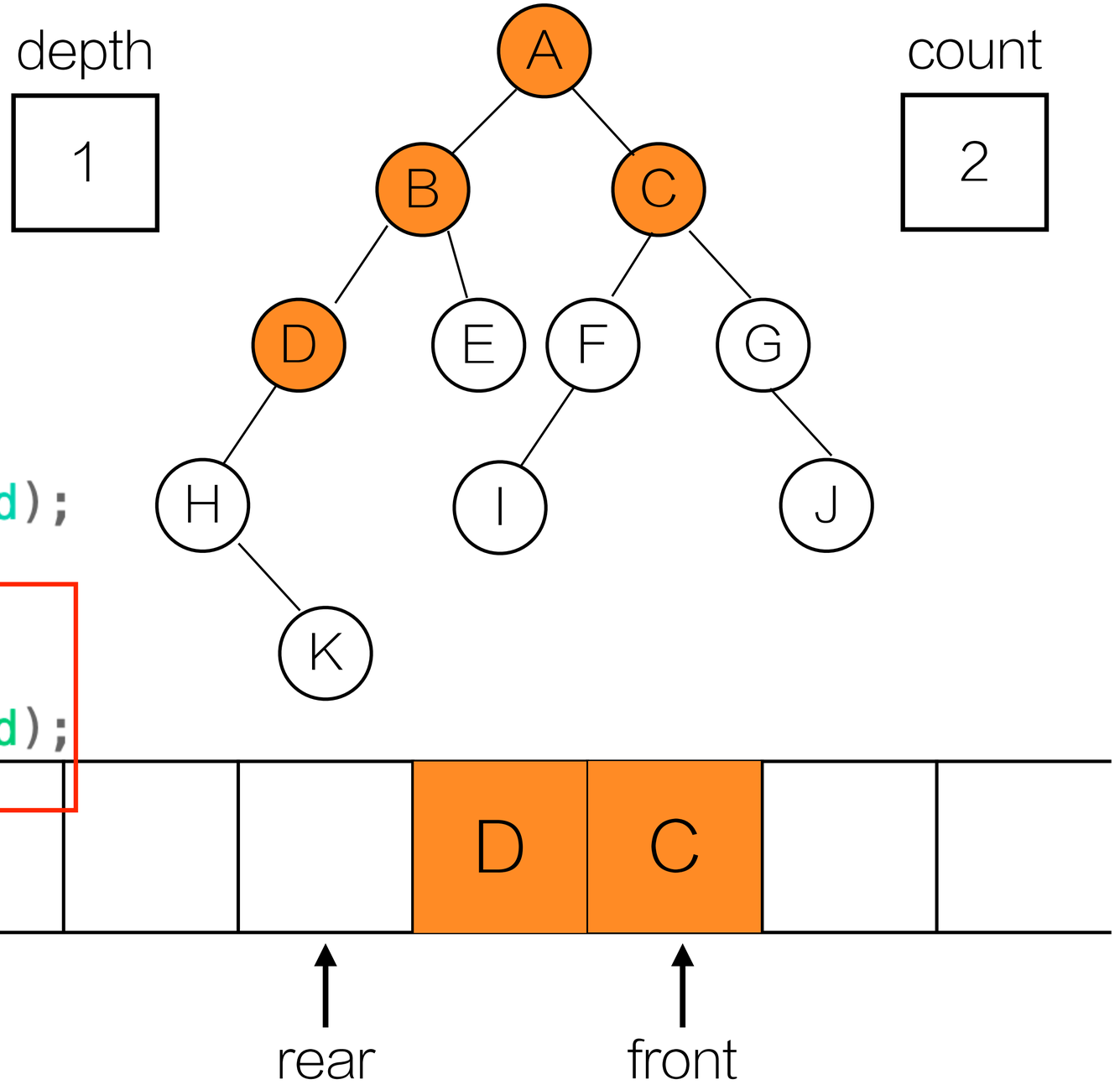


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

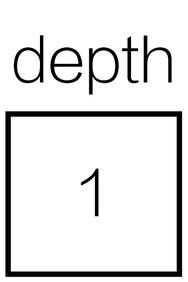


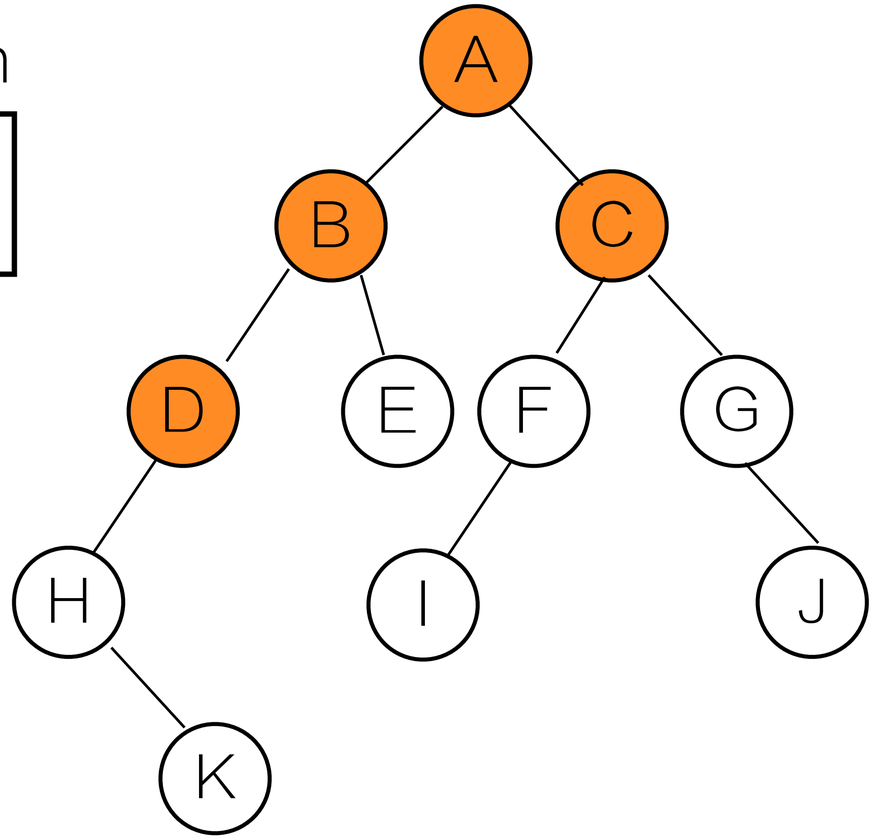


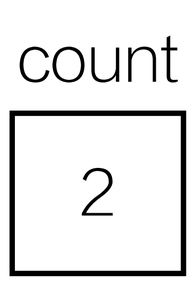
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

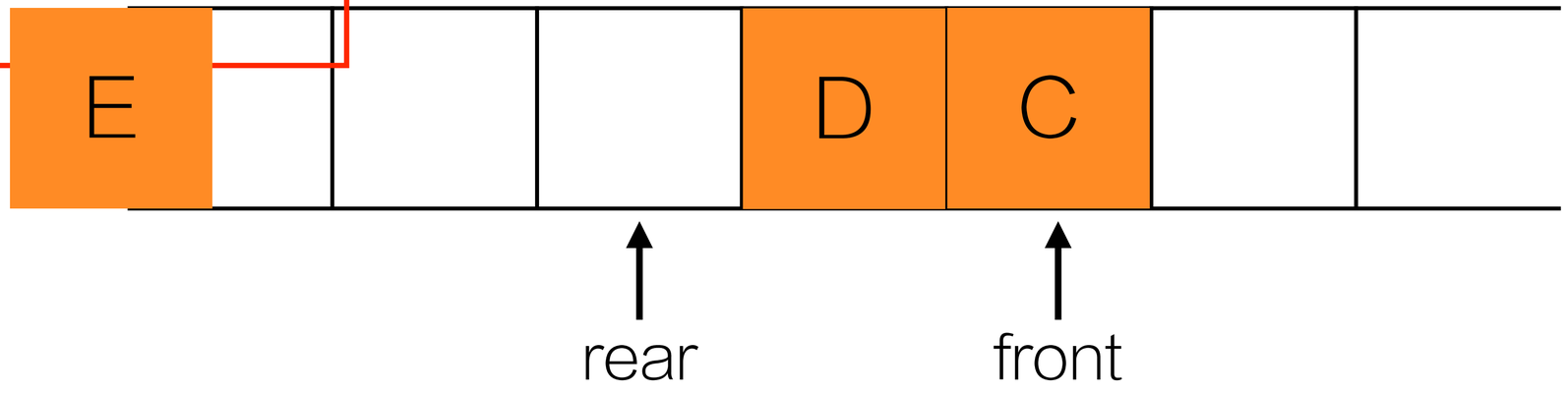


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

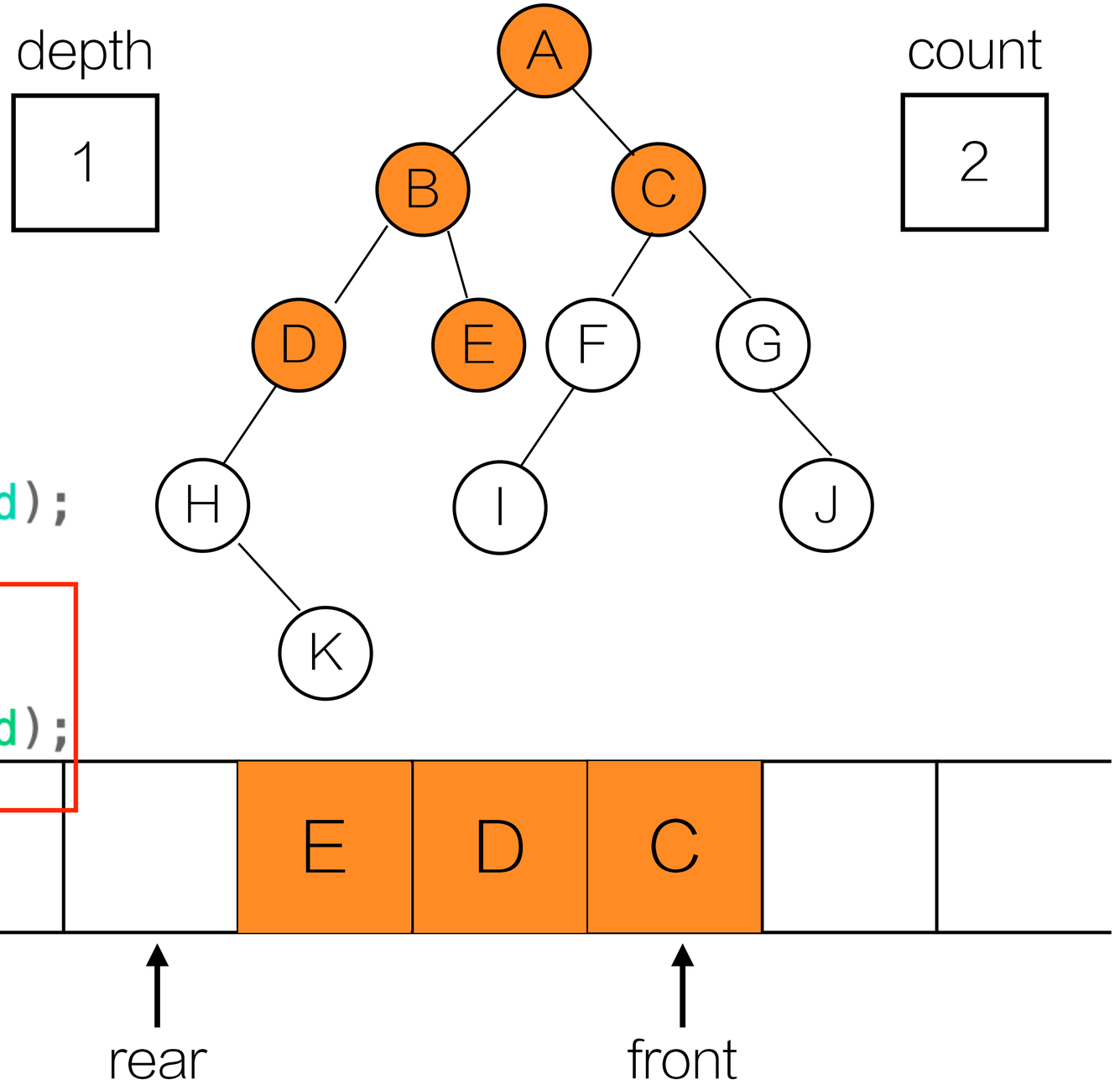








```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



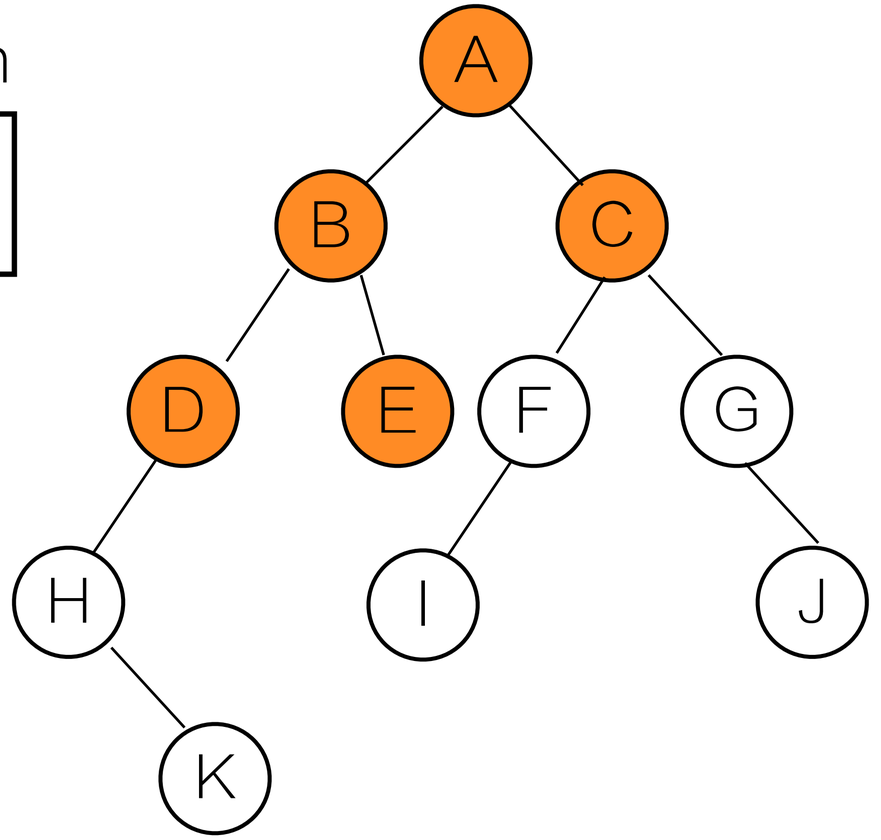
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

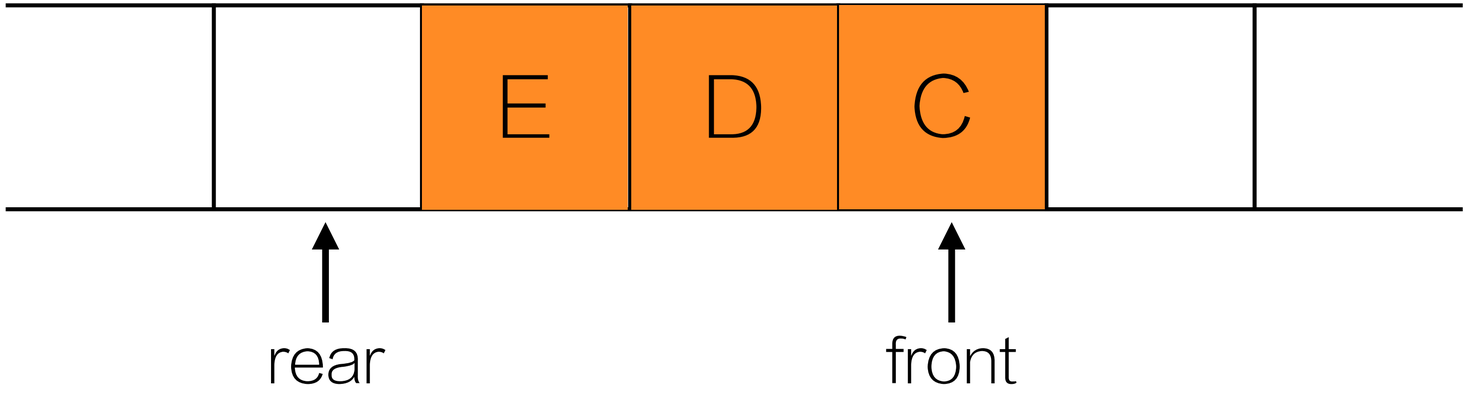
depth

1

count

2





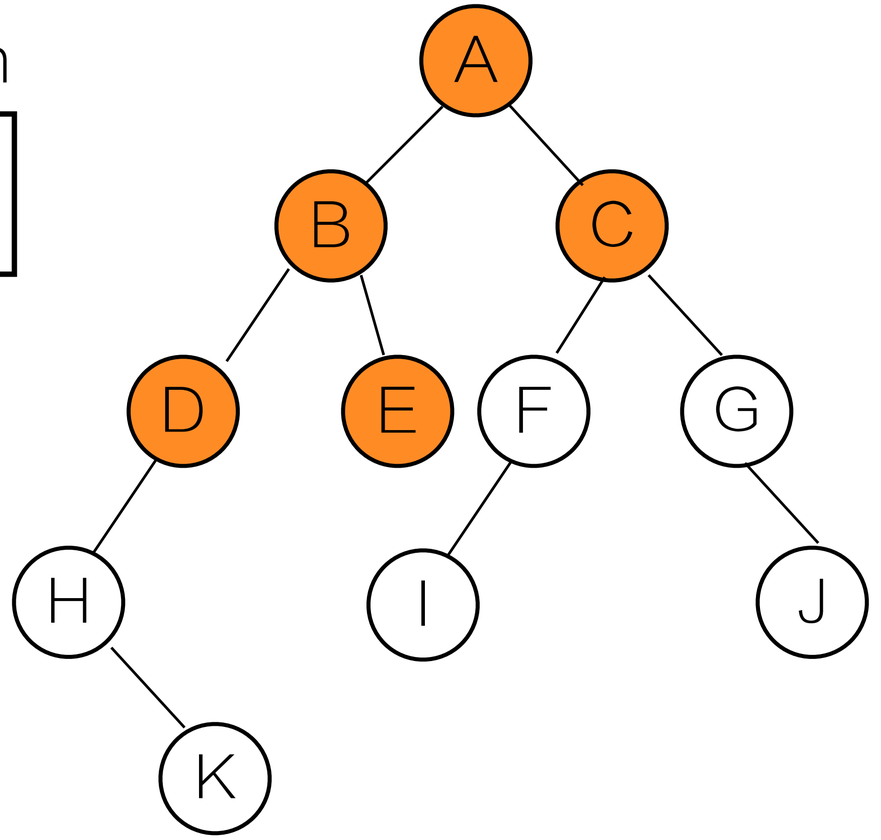
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

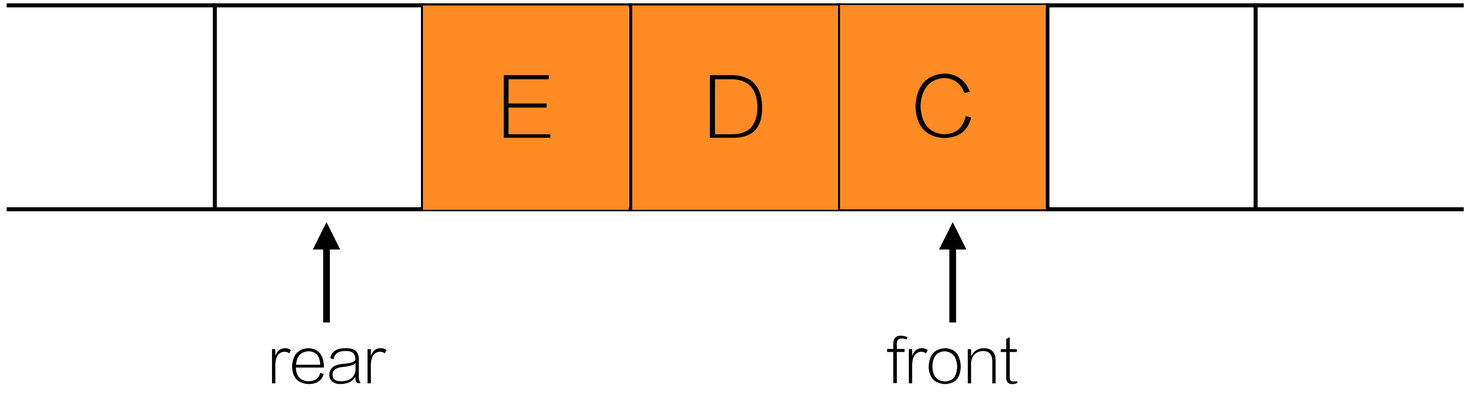
depth

1

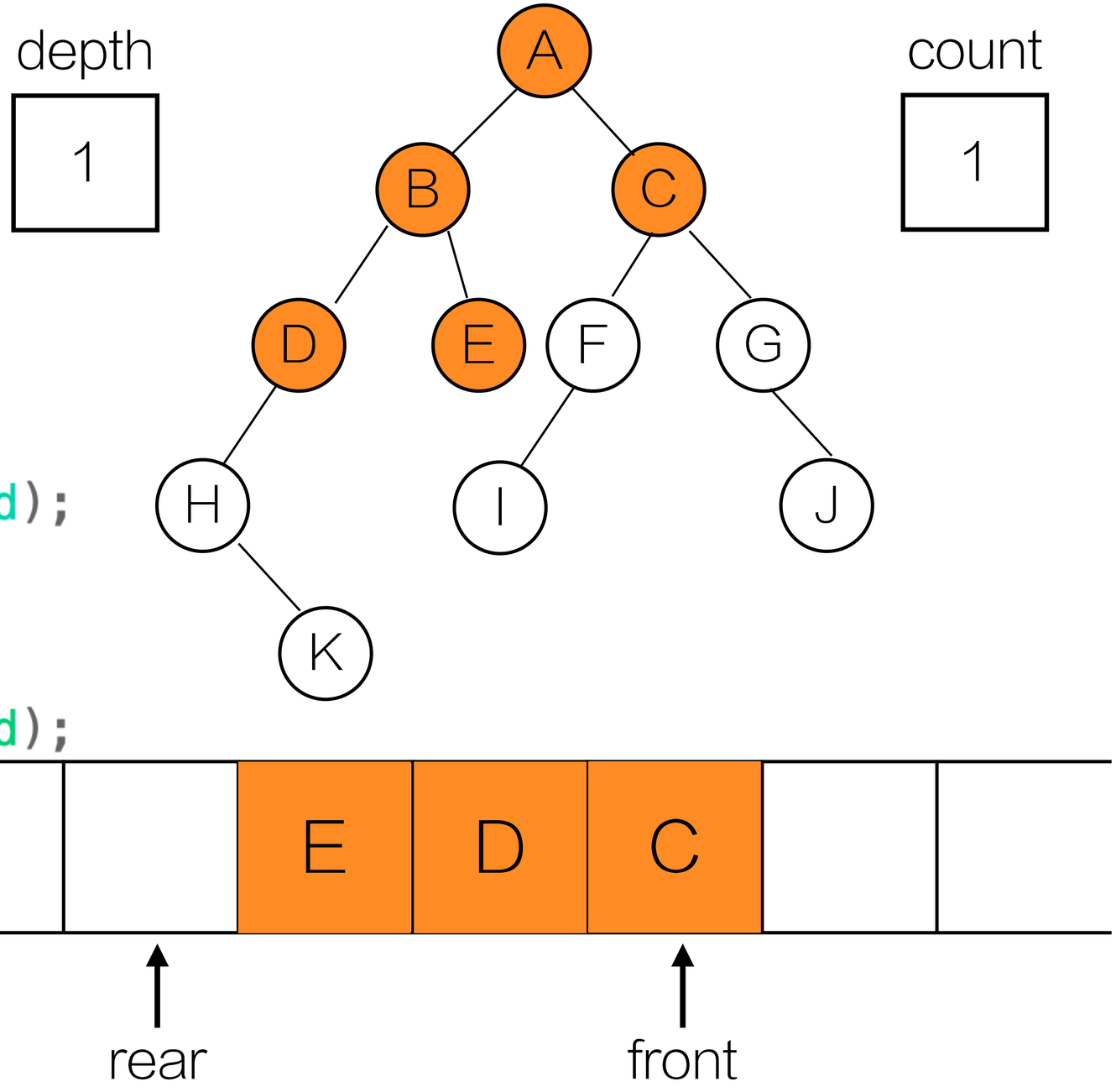
count

1

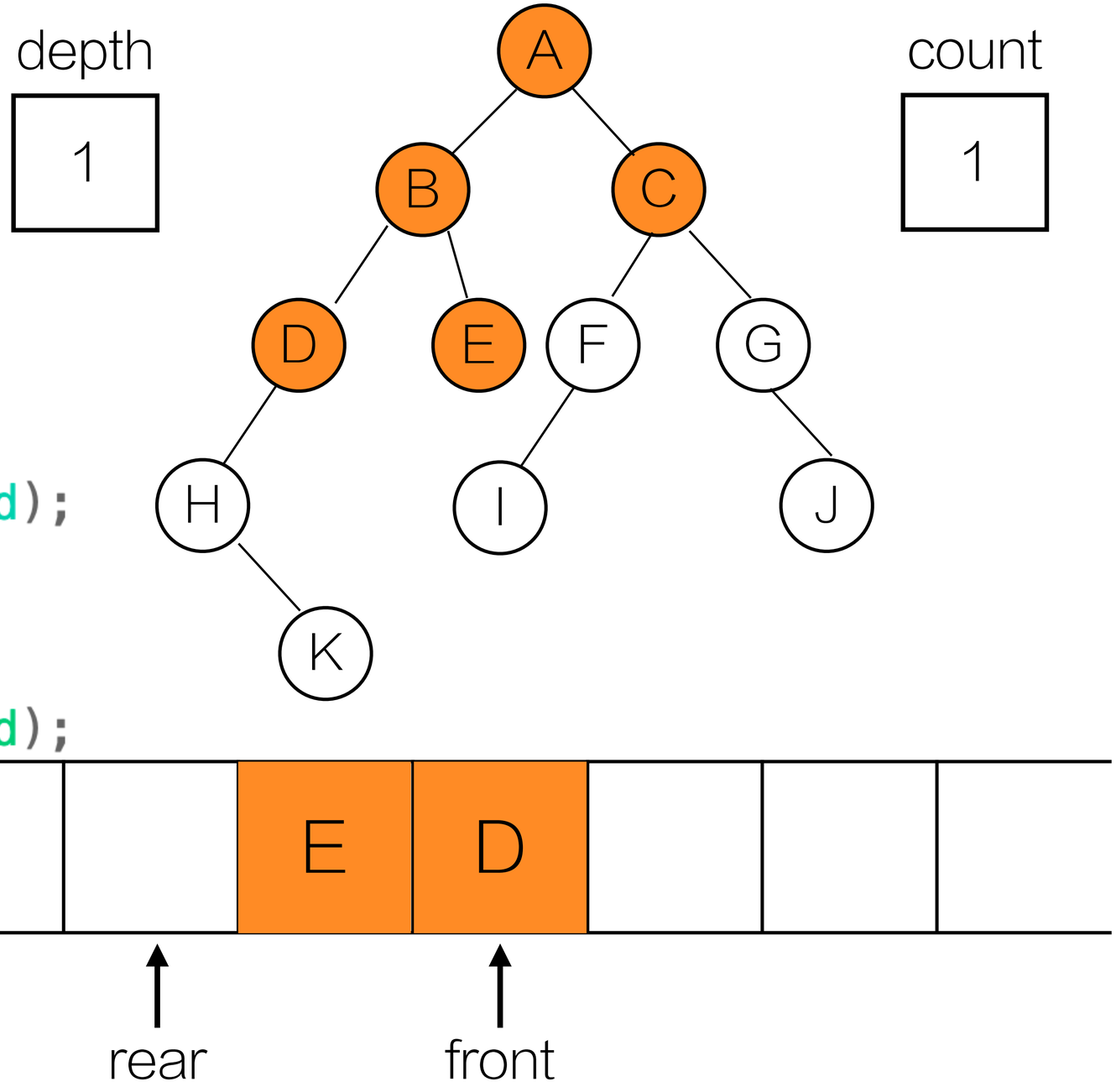




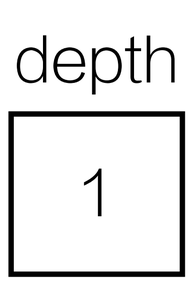
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

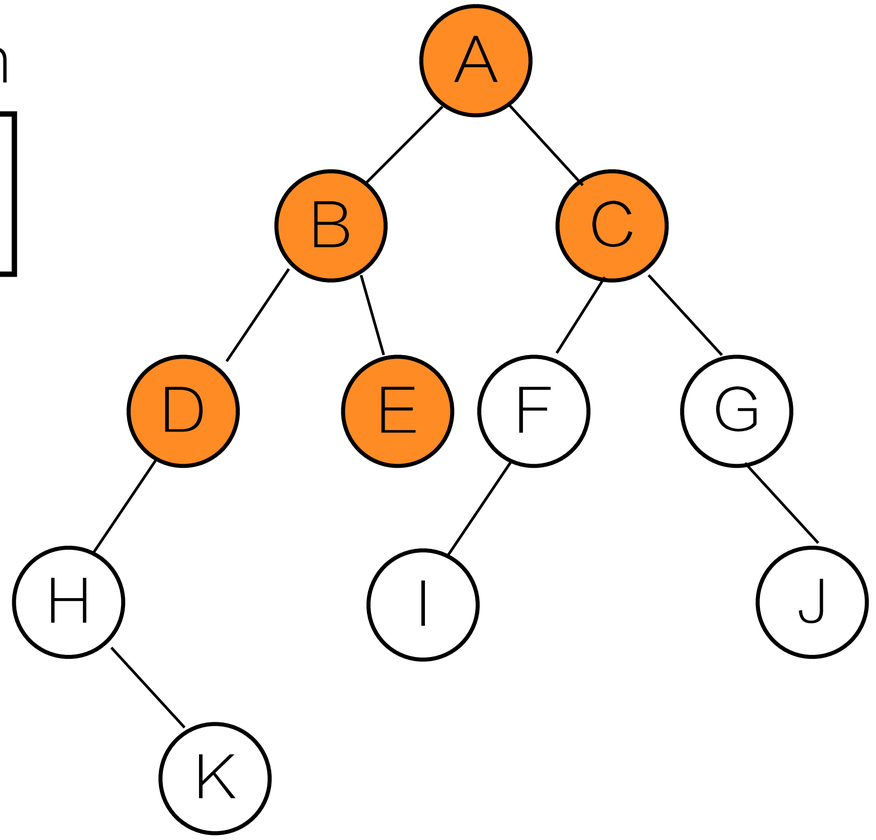


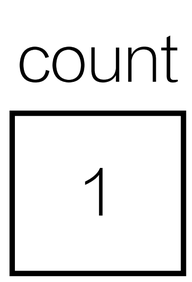
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

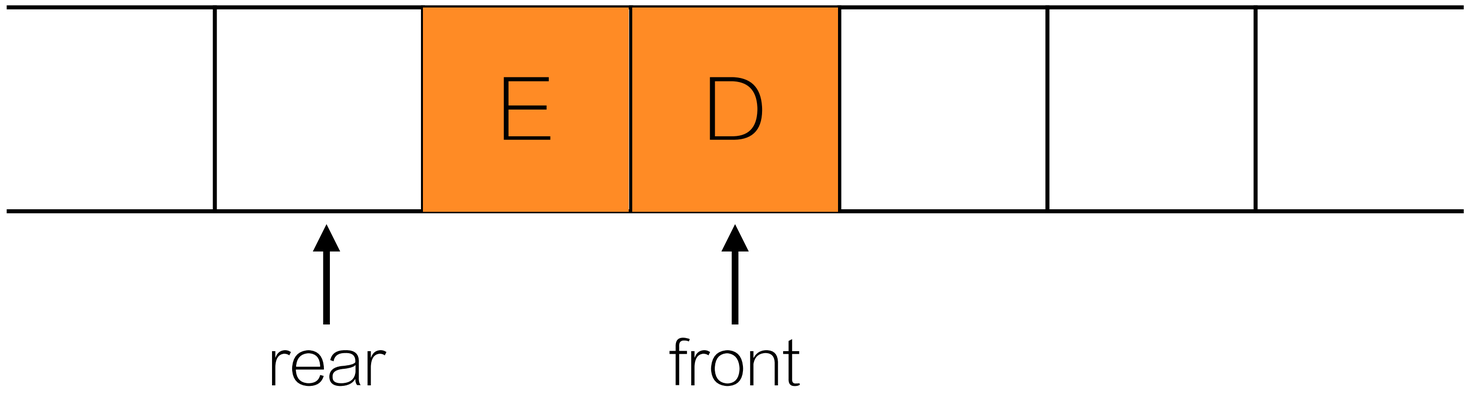


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

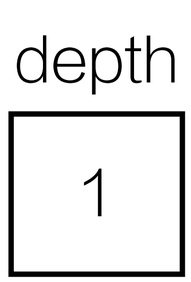


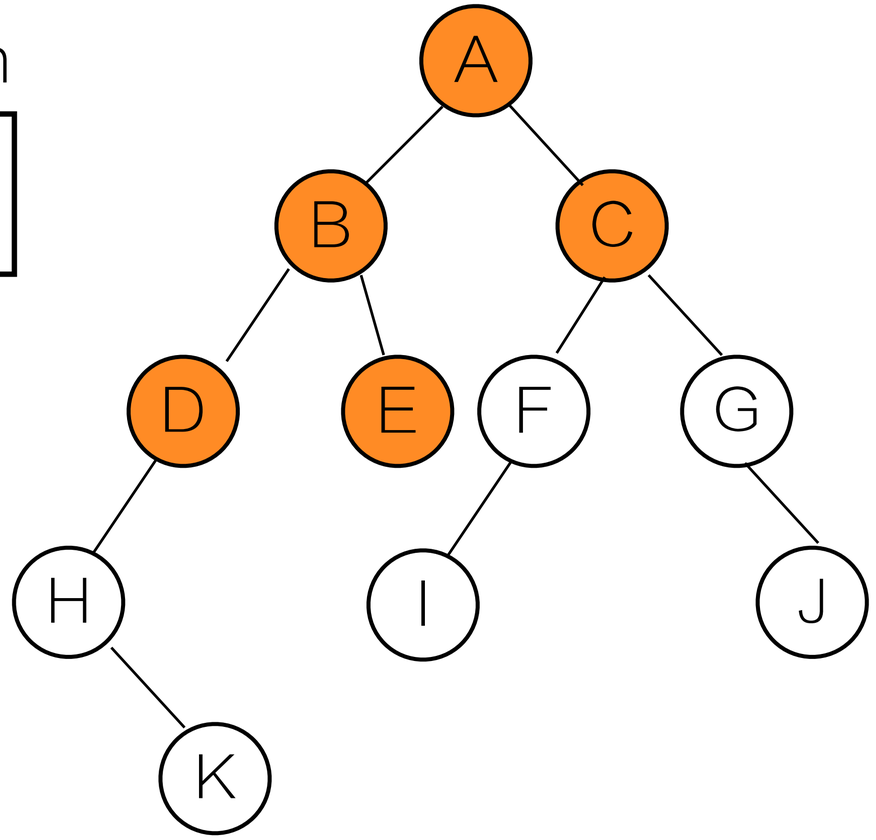


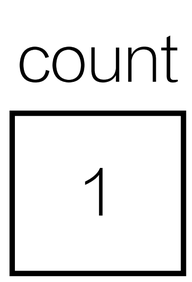


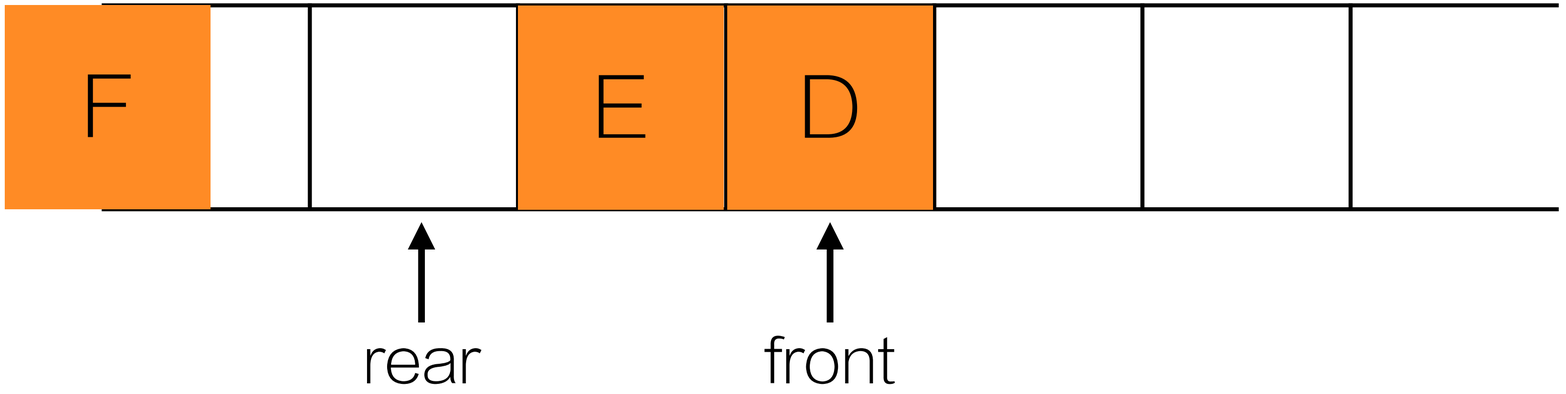


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

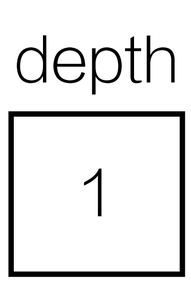


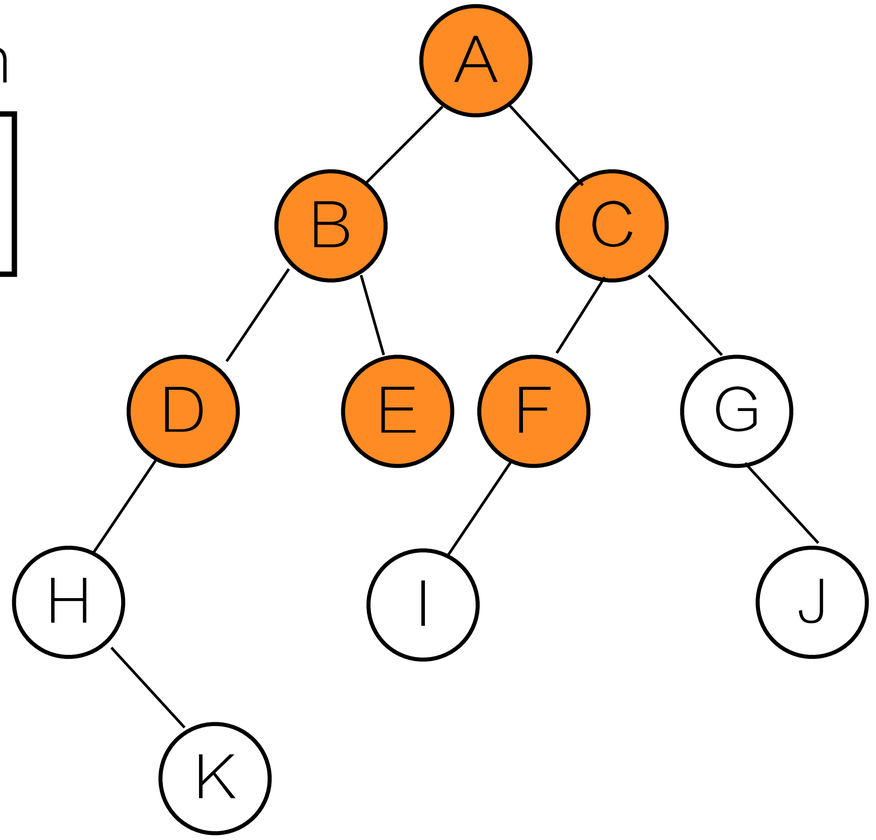


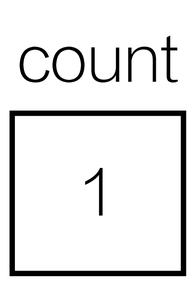


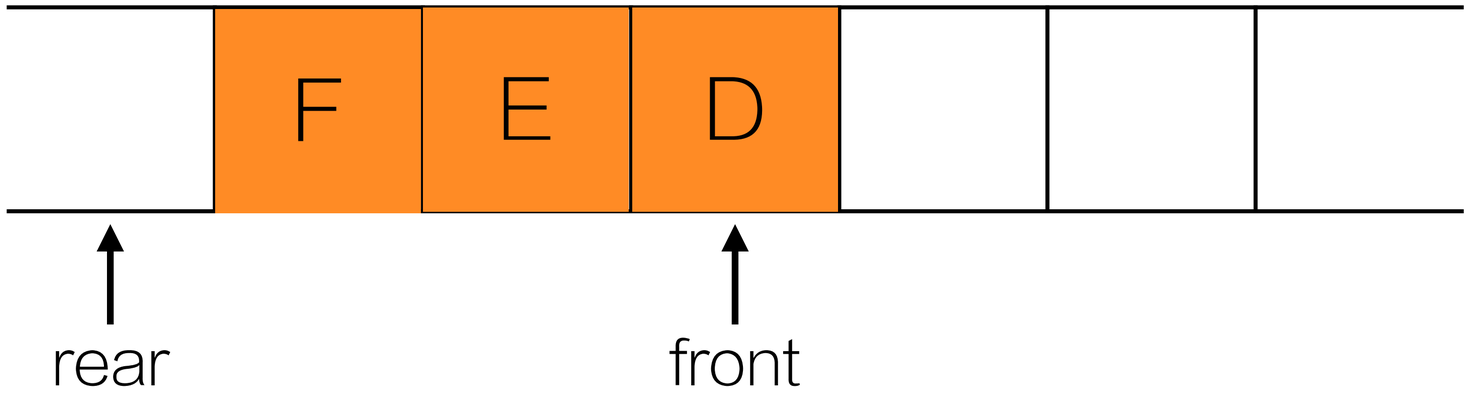


```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

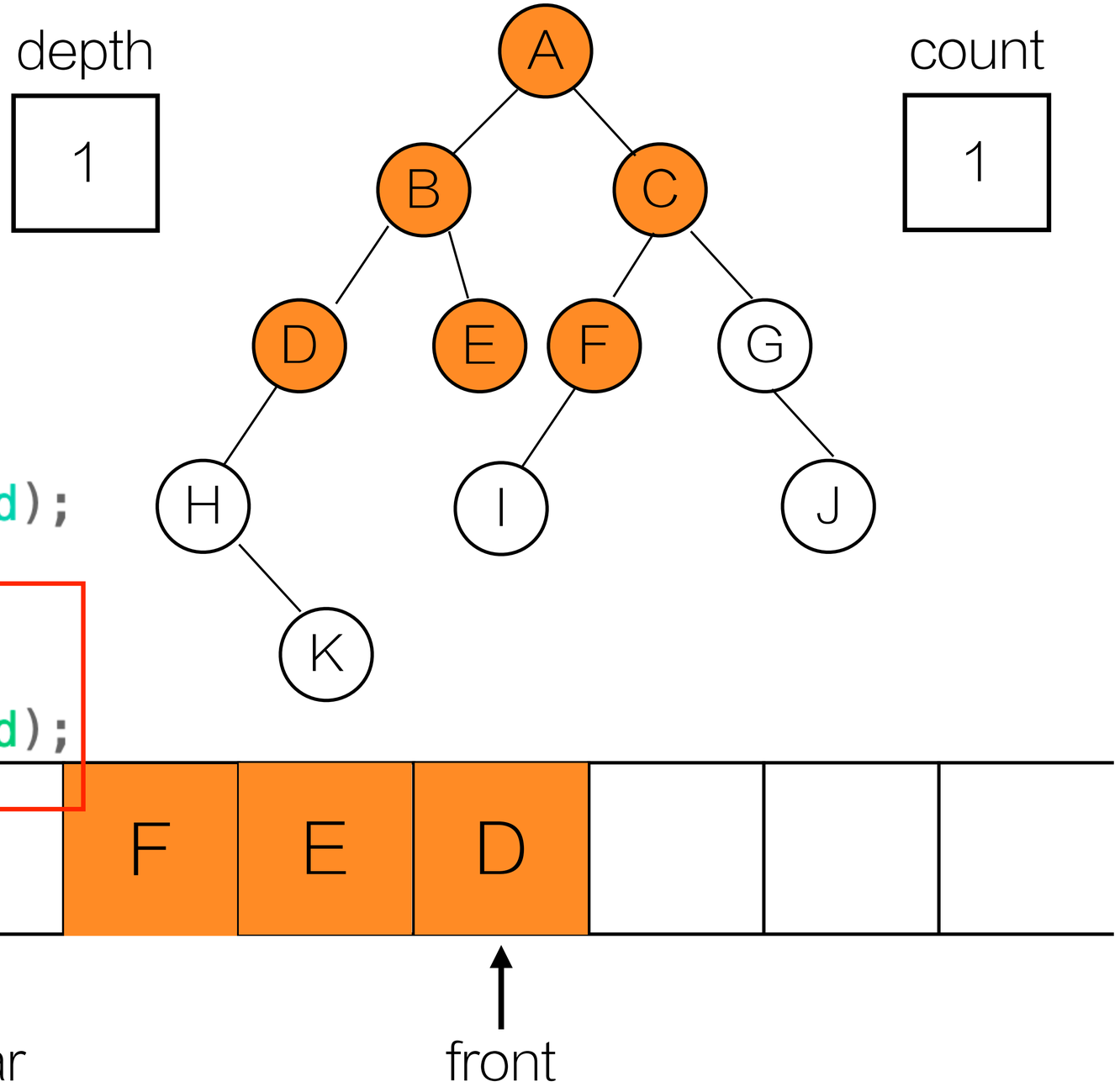








```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```



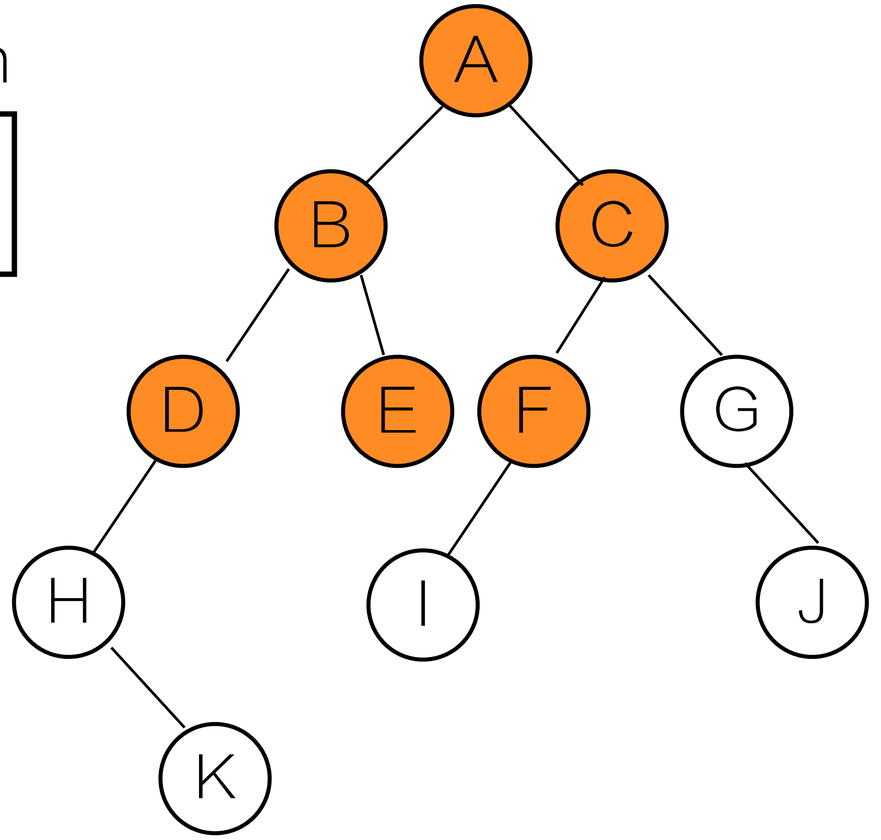
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

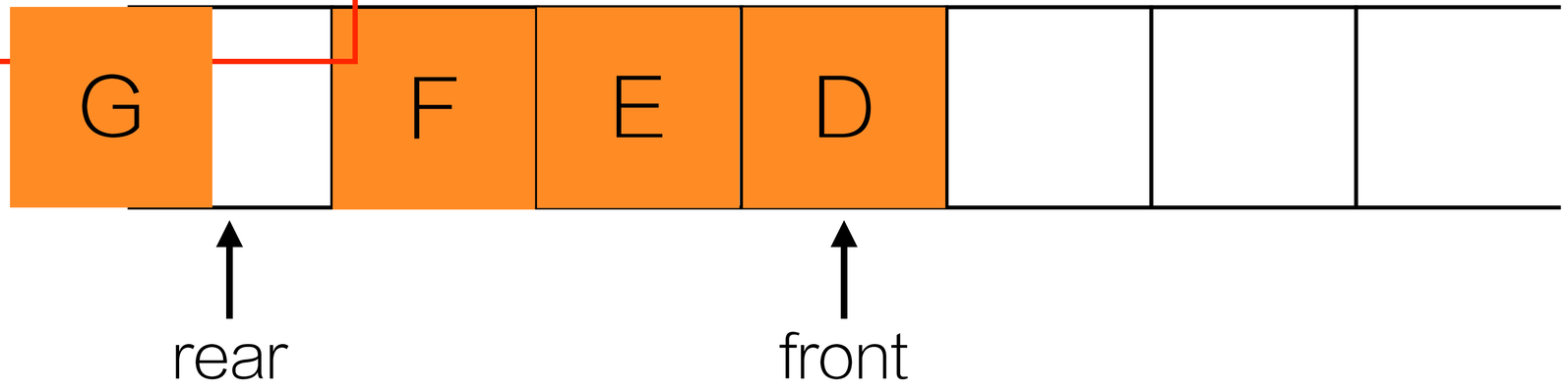
depth

1

count

1





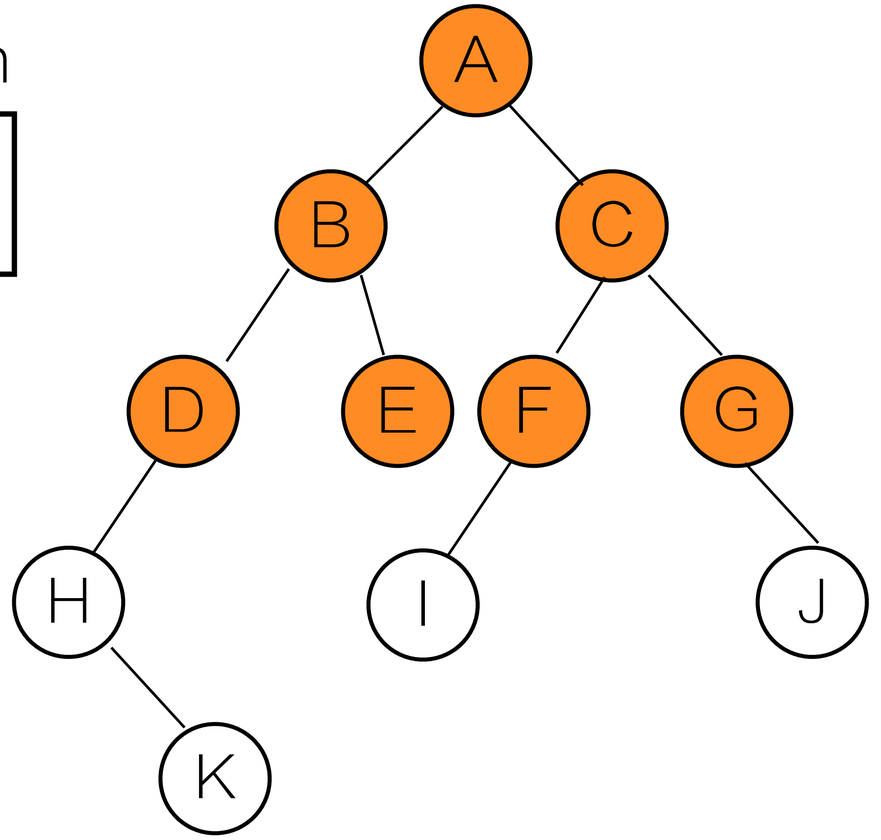
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

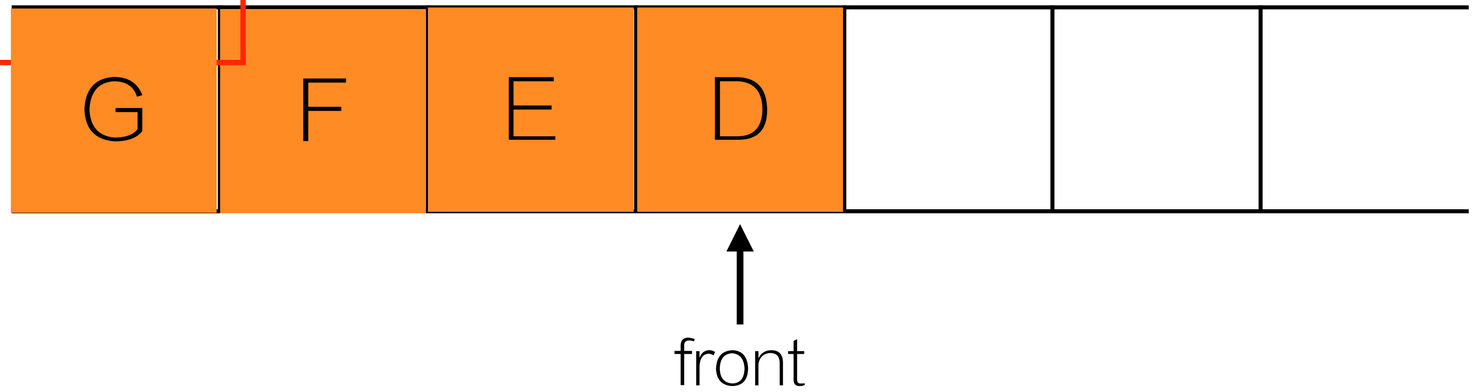
depth

1

count

1





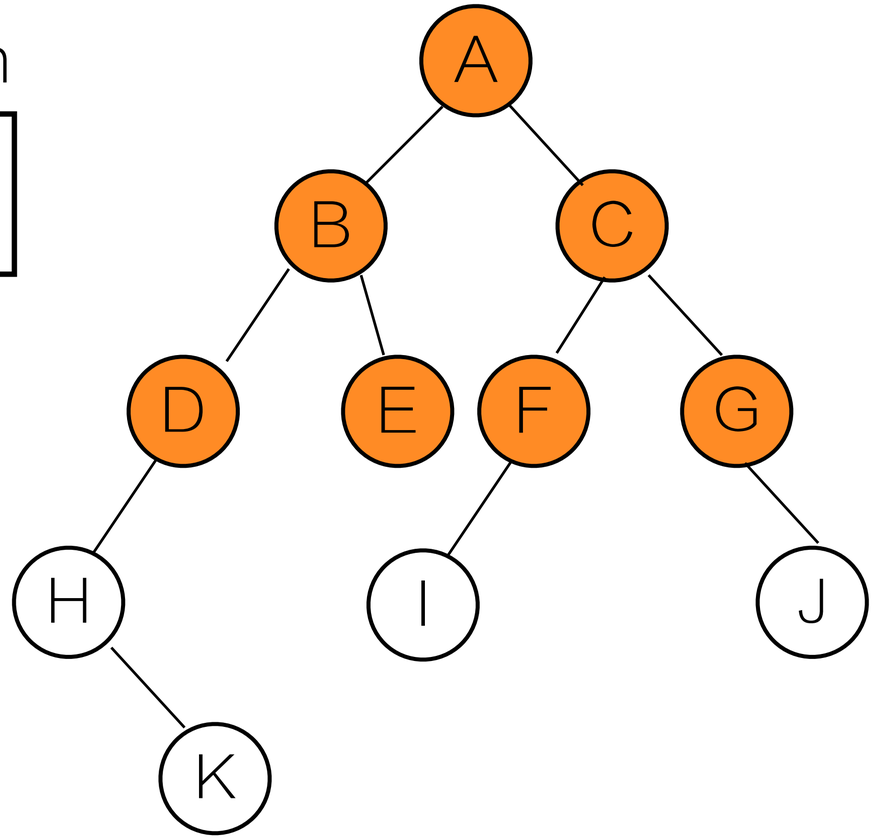
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

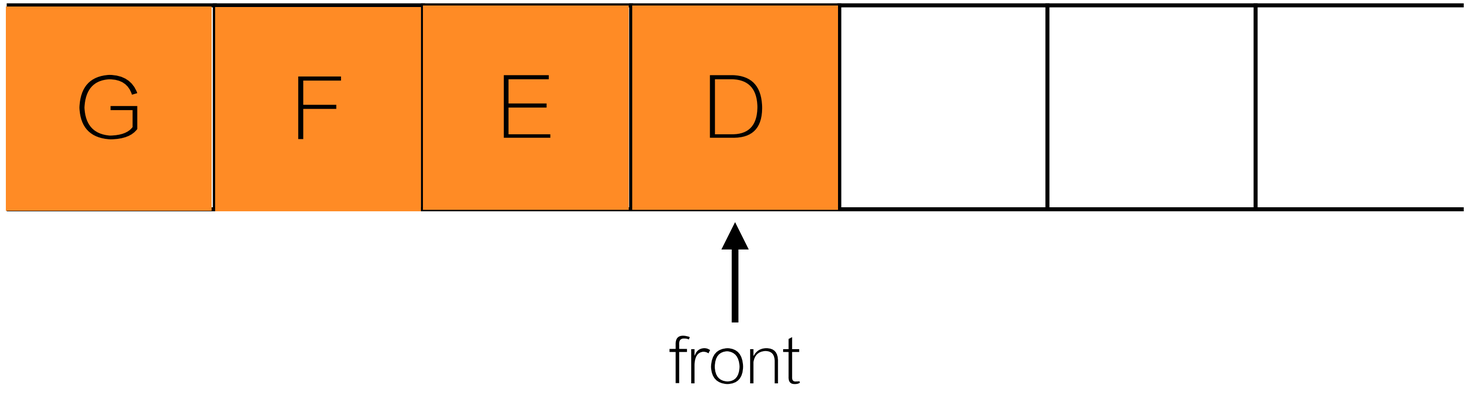
depth

1

count

1





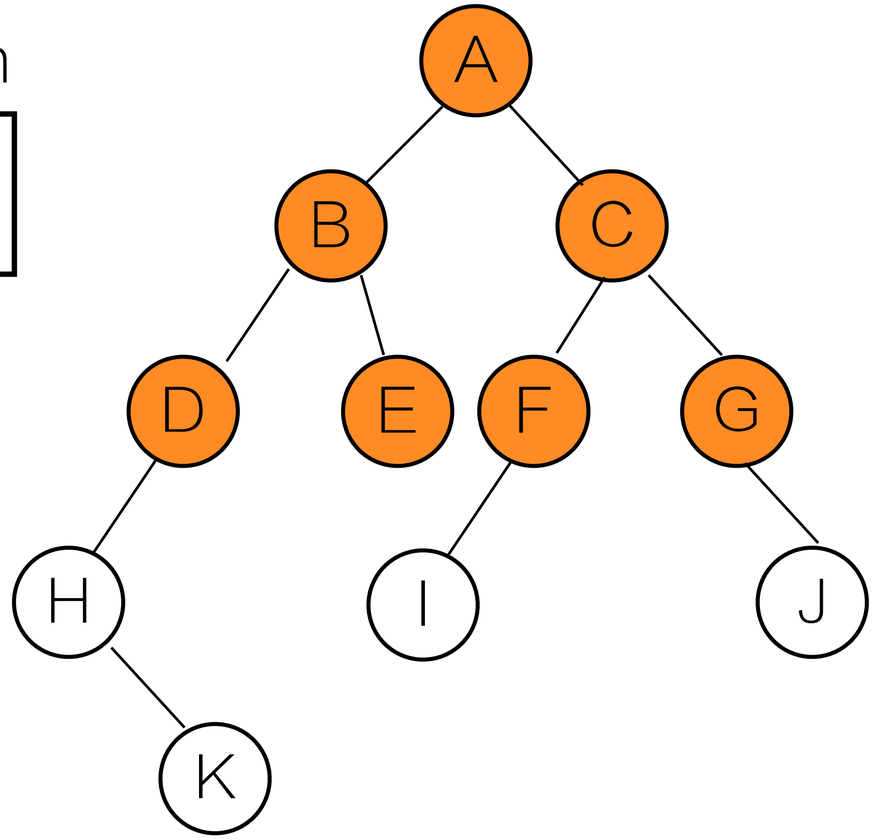
```
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

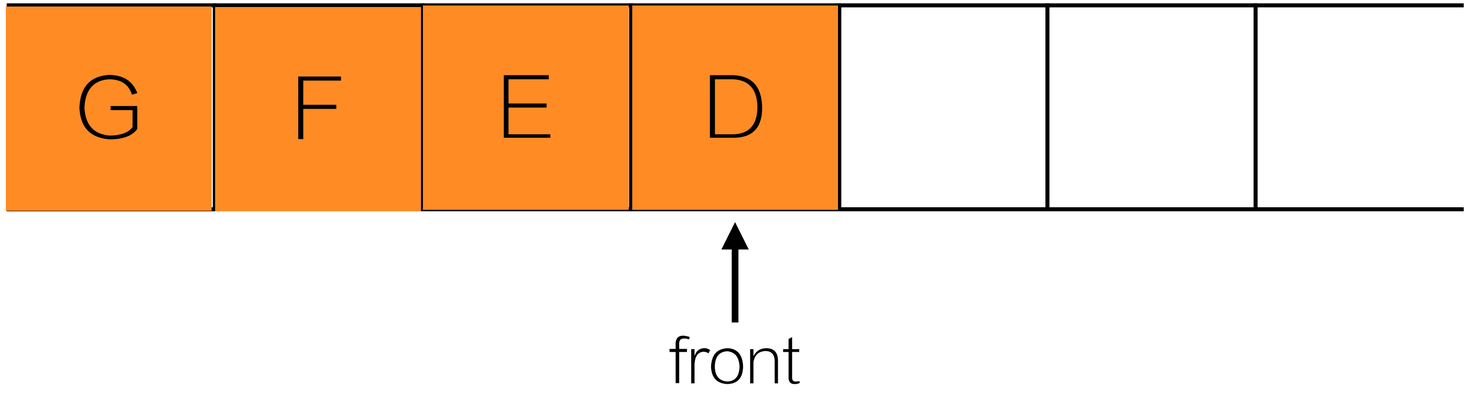
depth

1

count

0





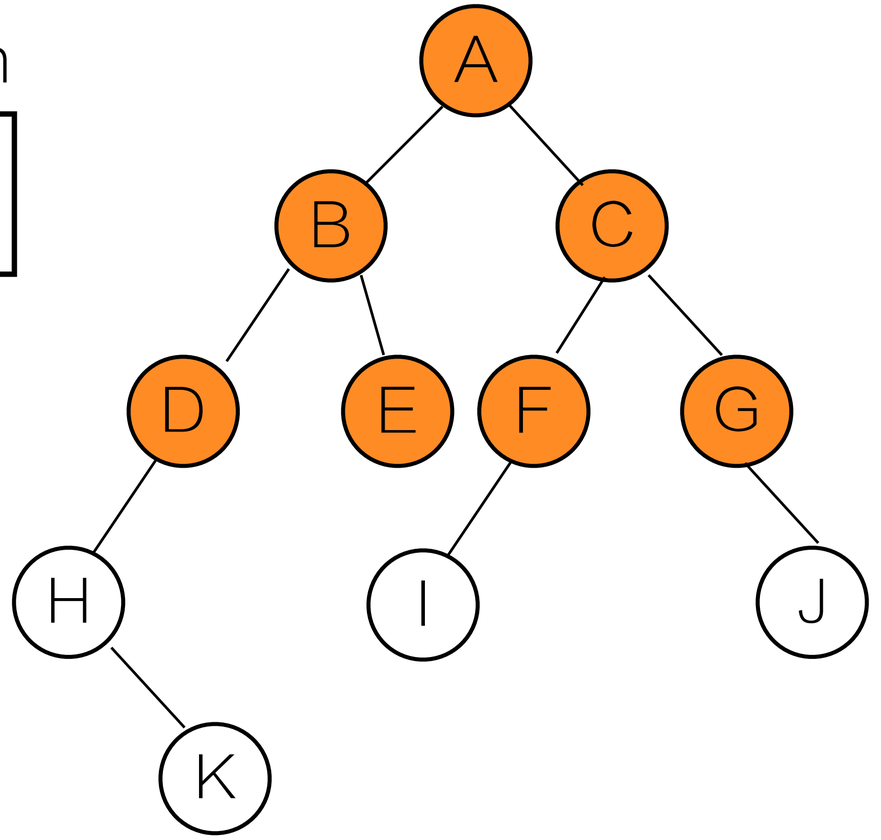
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

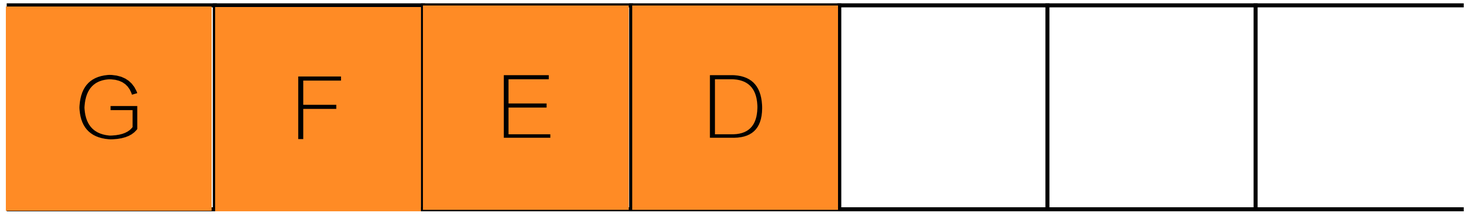
depth

1

count

0





rear

front

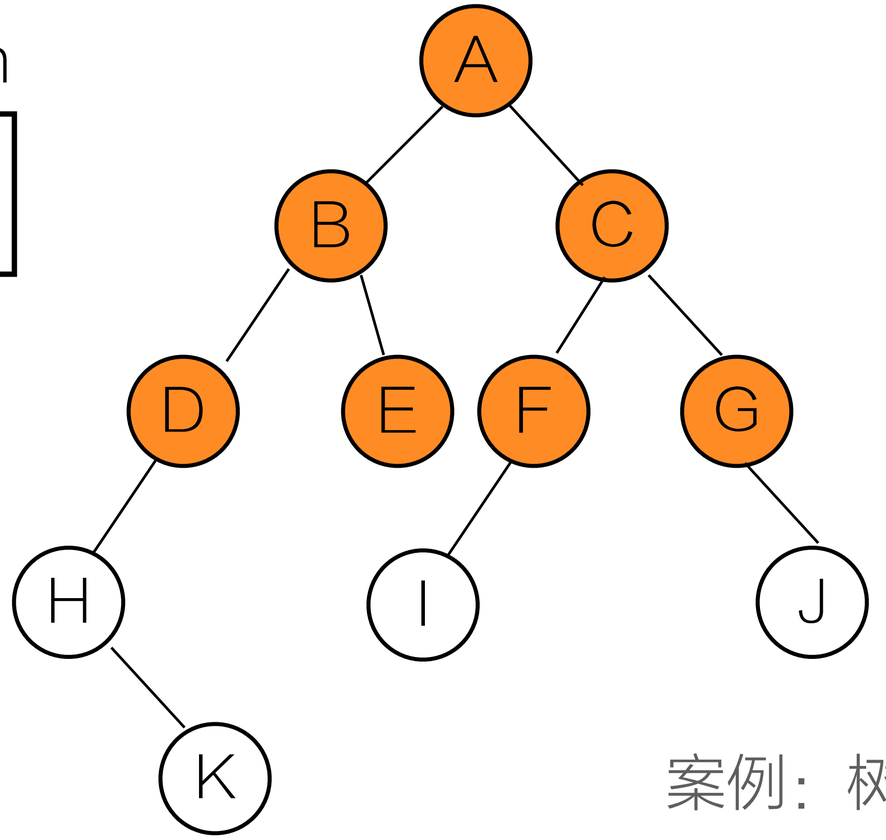
```c
while(!isEmpty(q))
{
    int count = queueSize(q);
    while(count > 0)
    {
        TreeNode* curr;
        dequeue(q, &curr);
        if (curr->lchild != NULL)
        {
            equeue(q, curr->lchild);
        }
        if (curr->rchild != NULL)
        {
            equeue(q, curr->rchild);
        }
        count--;
    }
    depth++;
}
return depth;
```

depth

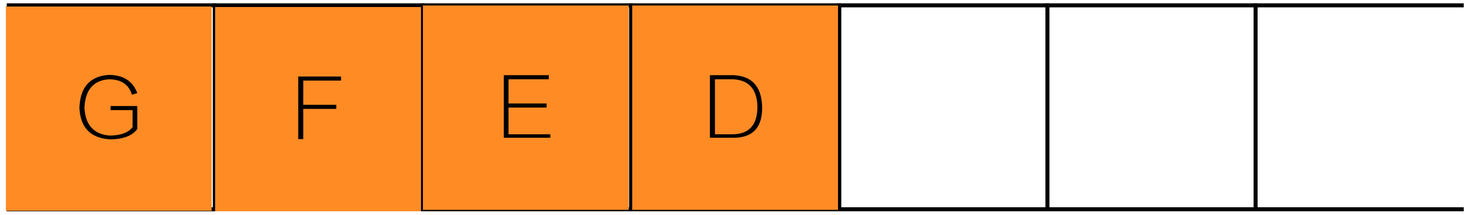
2

count

0



案例：树 / test7.c



rear

front

二叉树 – 真题解析

[2014] 41.（13分）二叉树的带权路径长度（WPL）是二叉树中所有叶结点的带权路径长度之和。给定一棵二叉树 T，采用二叉链表存储，结点结构如下：



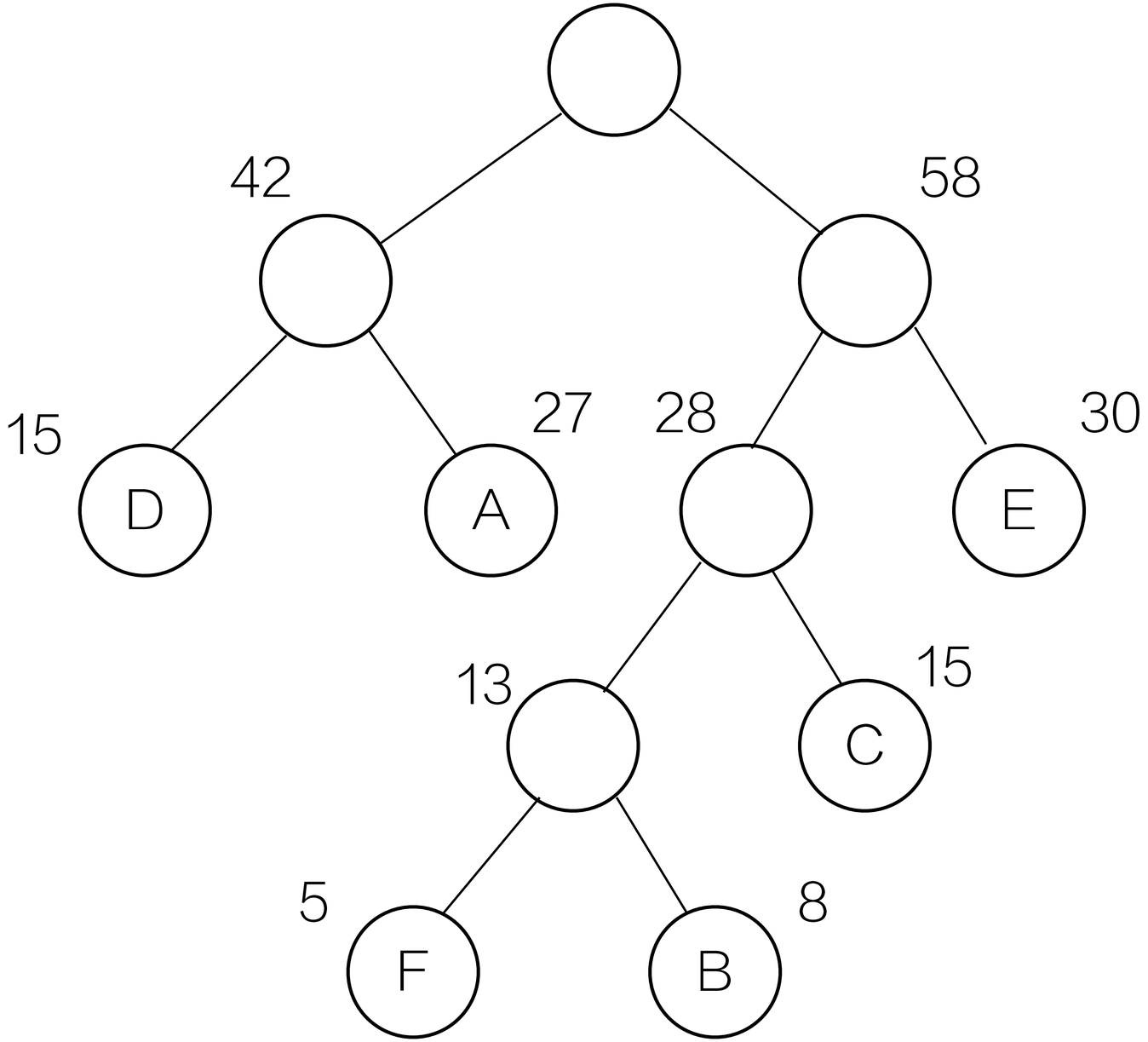
其中叶结点的 weight 域保存该结点的非负权值。设 root 为指向 T 的根结点的指针，请设计求 T 的 WPL 的算法，要求：

（1）给出算法的基本设计思想。

（2）用 C 或 C++ 语言，给出二叉树结点的数据类型定义。

（3）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

二叉树 – 真题解析

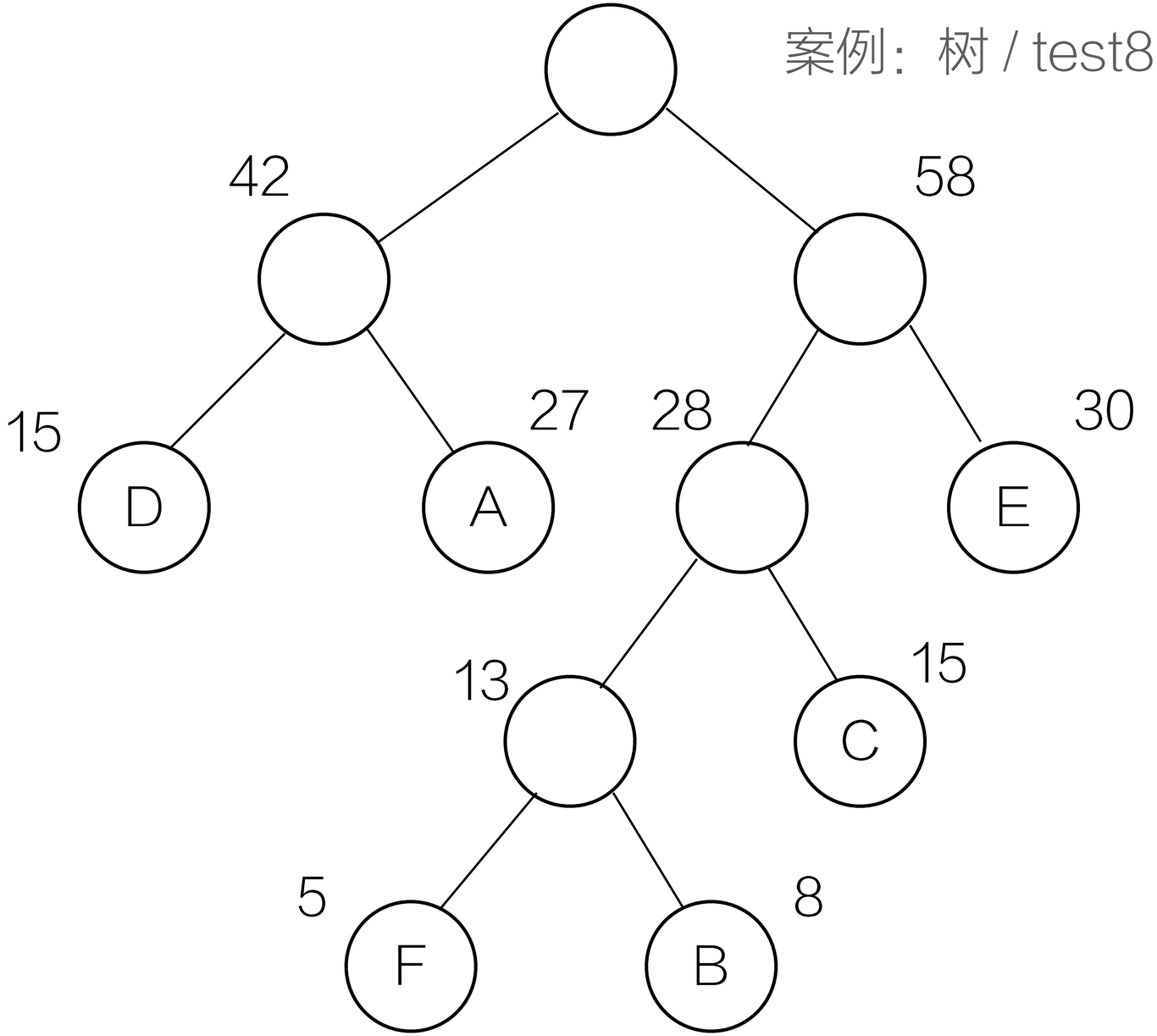


二叉树 – 真题解析

案例：树 / test8.c

1.判断是否是叶子结点?

2.确定叶子结点所在层数?



二叉树 – 真题解析

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

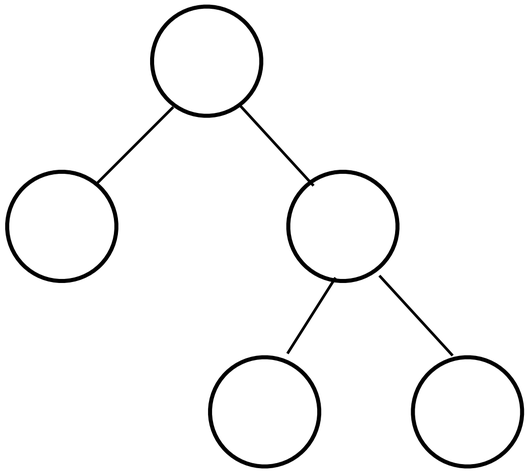
（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个?

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

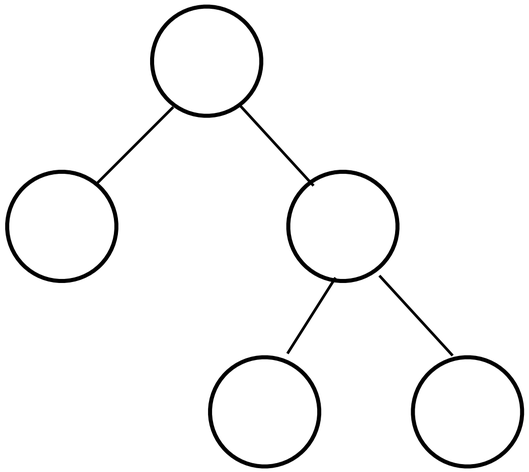
[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

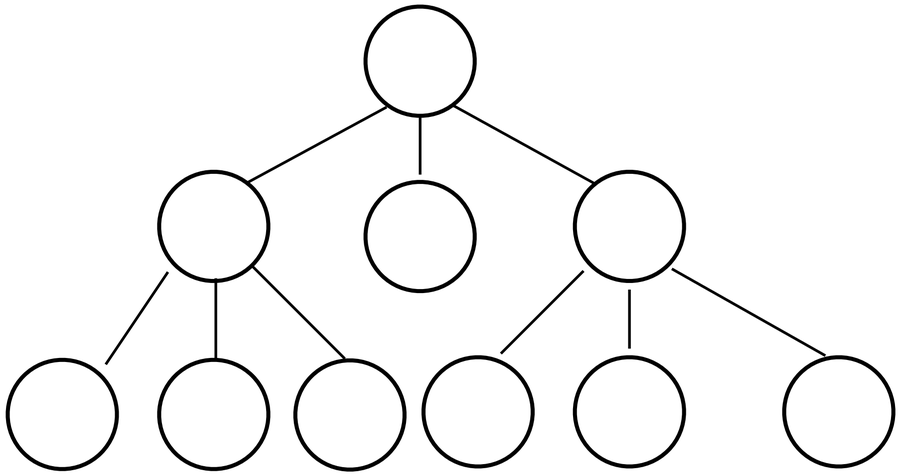
（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

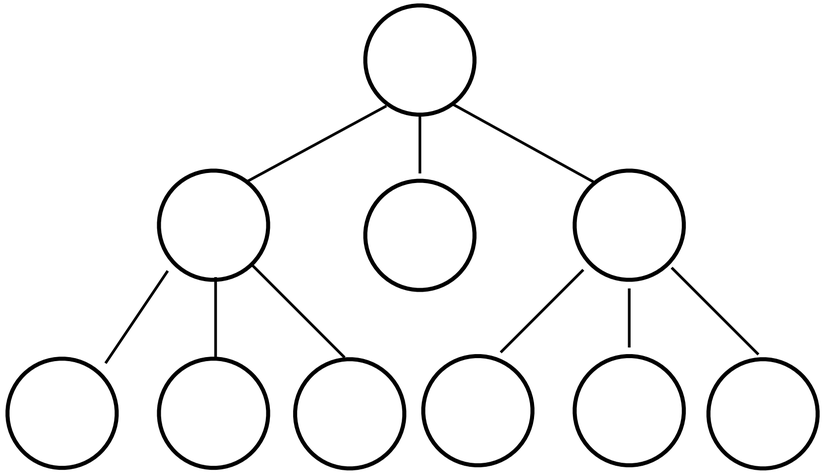
（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?





[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

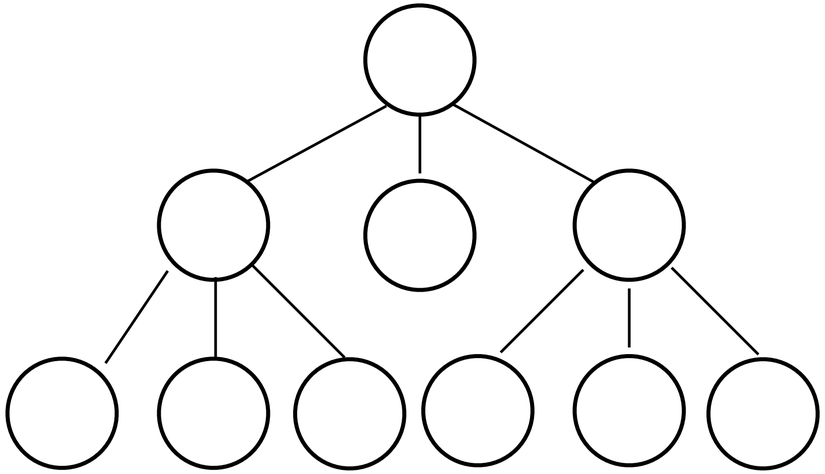


[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

$n = n_0 + n_3$



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

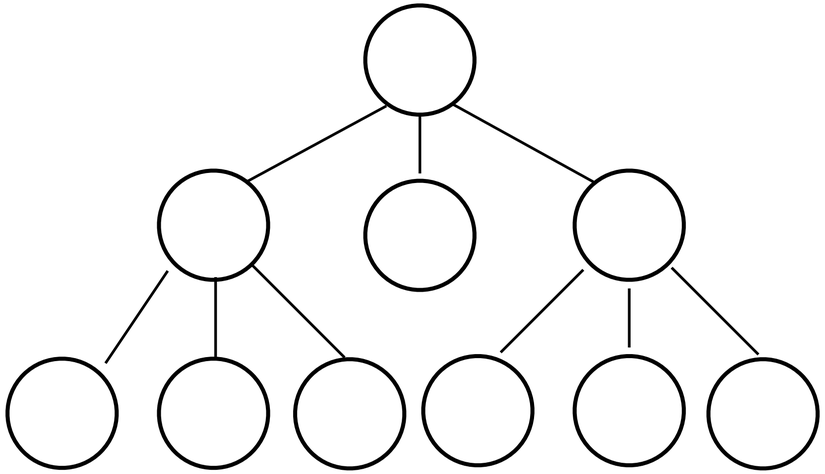
（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

$n = n_0 + n_3$

正则k叉树

$n = n_0 + n_k$



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

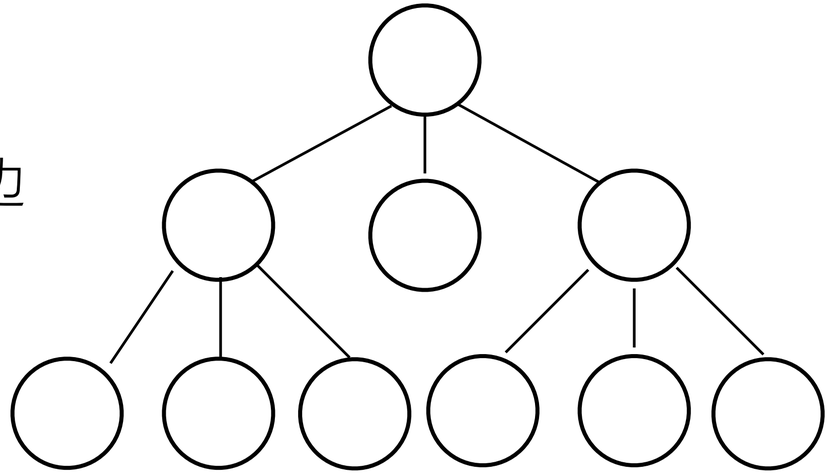
$n = n_0 + n_3$

正则三叉树每个非叶结点：3条边

正则k叉树

$n = n_0 + n_k$

正则k叉树每个非叶结点：k条边



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

$n = n_0 + n_3$

正则三叉树每个非叶结点：3条边

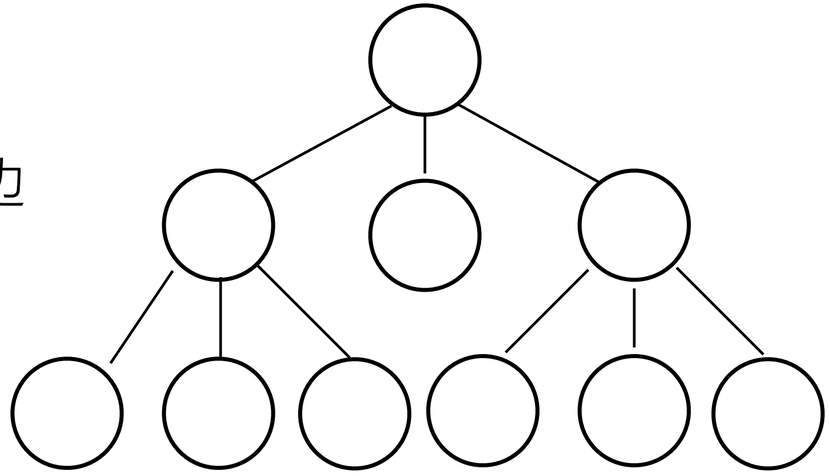
m个非叶结点：3m条边

正则k叉树

$n = n_0 + n_k$

正则k叉树每个非叶结点：k条边

m个非叶结点：km条边



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

$n = n_0 + n_3$

正则三叉树每个非叶结点：3条边

m个非叶结点：3m条边

$n = 3m + 1$

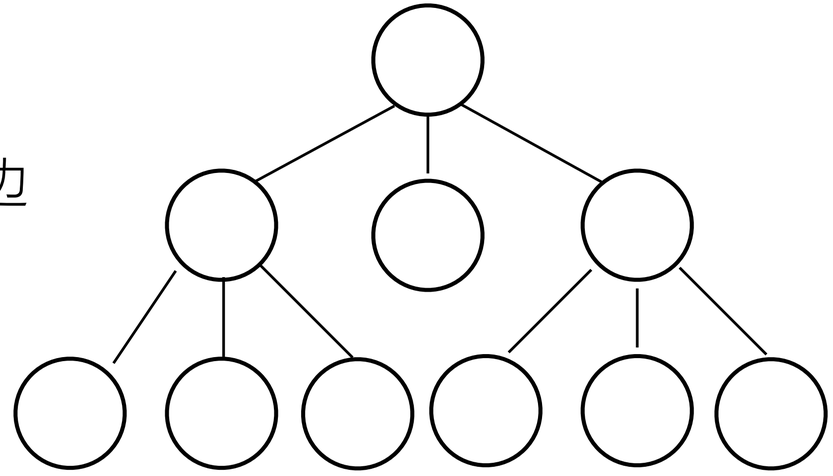
正则k叉树

$n = n_0 + n_k$

正则k叉树每个非叶结点：k条边

m个非叶结点：km条边

$n = km + 1$



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

$n = n_0 + n_3$

正则三叉树每个非叶结点：3条边

m个非叶结点：3m条边

$n = 3m + 1$

正则k叉树

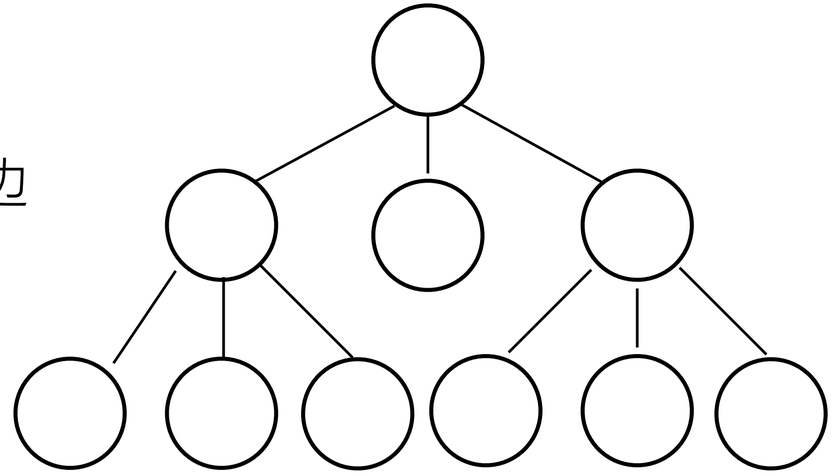
$n = n_0 + n_k$

正则k叉树每个非叶结点：k条边

m个非叶结点：km条边

$n = km + 1$

$km + 1 = n_0 + m$



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（1）若 T 有 m 个非叶结点，则 T 中的叶结点有多少个?

正则三叉树

$n = n_0 + n_3$

正则三叉树每个非叶结点：3条边

m个非叶结点：3m条边

$n = 3m + 1$

正则k叉树

$n = n_0 + n_k$

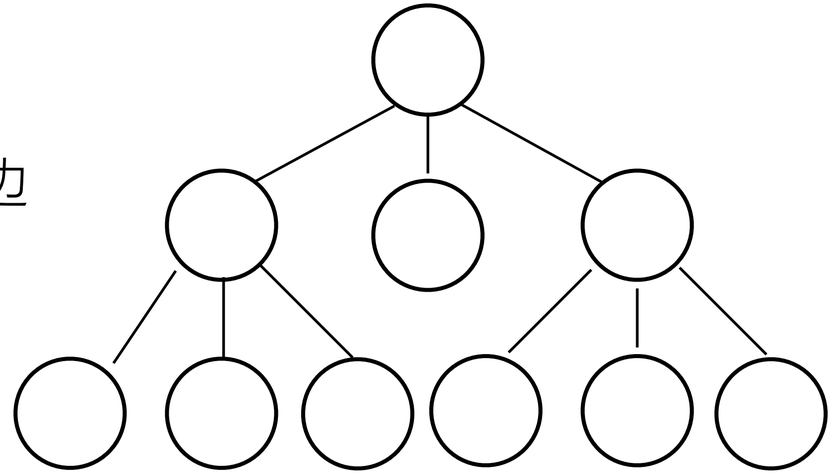
正则k叉树每个非叶结点：k条边

m个非叶结点：km条边

$n = km + 1$

$km + 1 = n_0 + m$

$n_0 = km - m + 1$

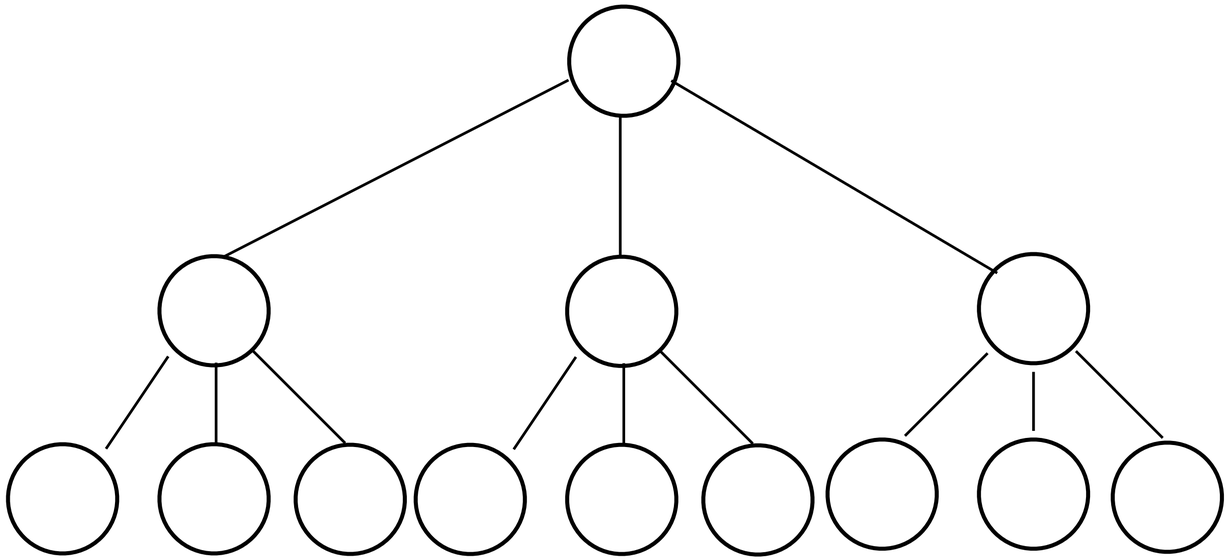


[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？

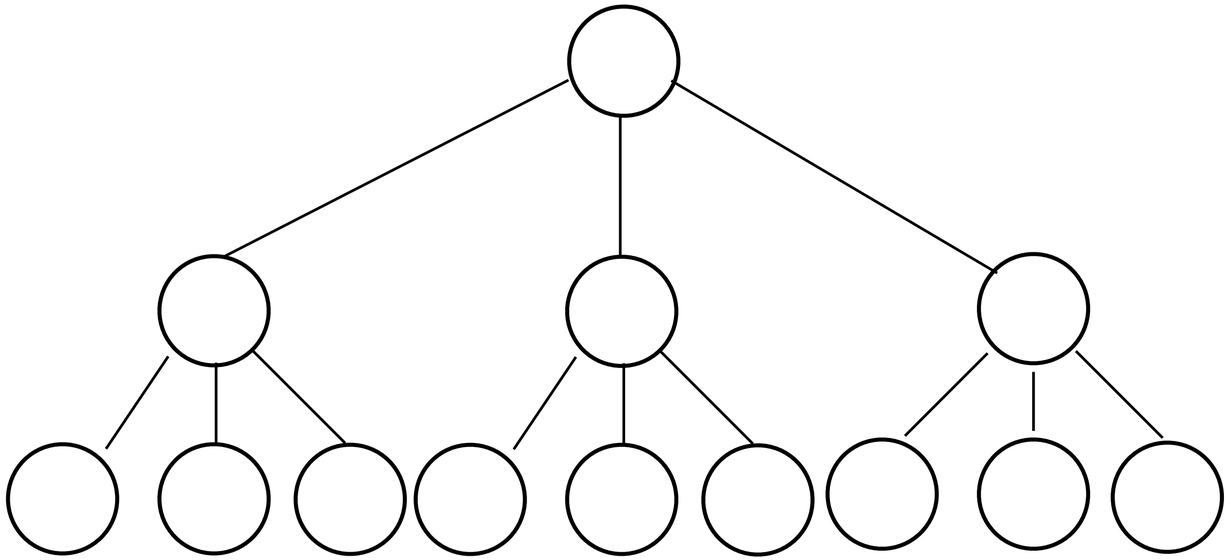
[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

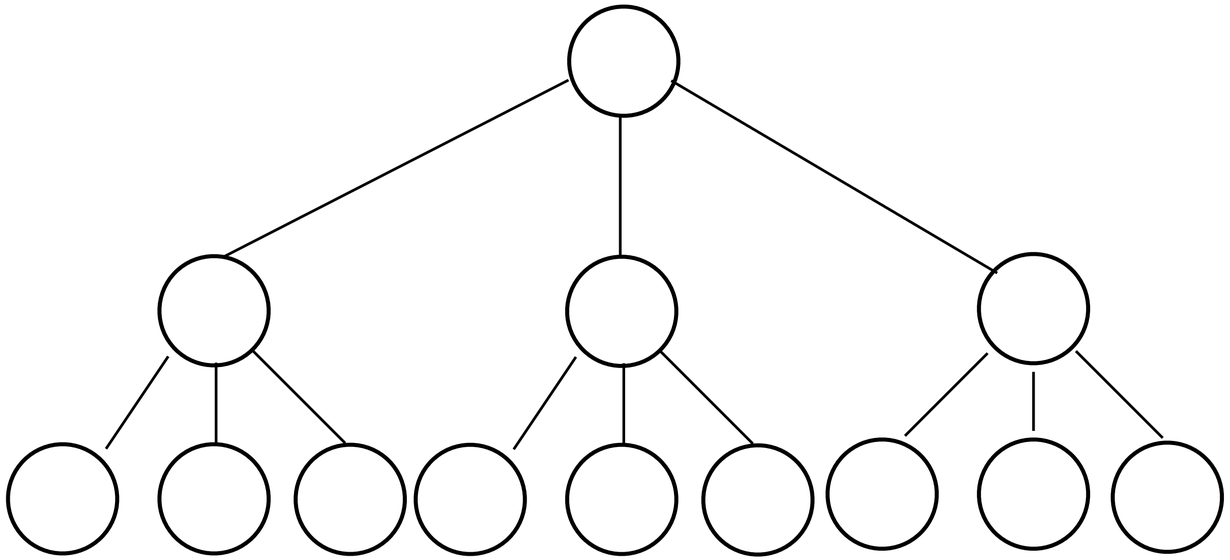
（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



第一层：1个节点

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？

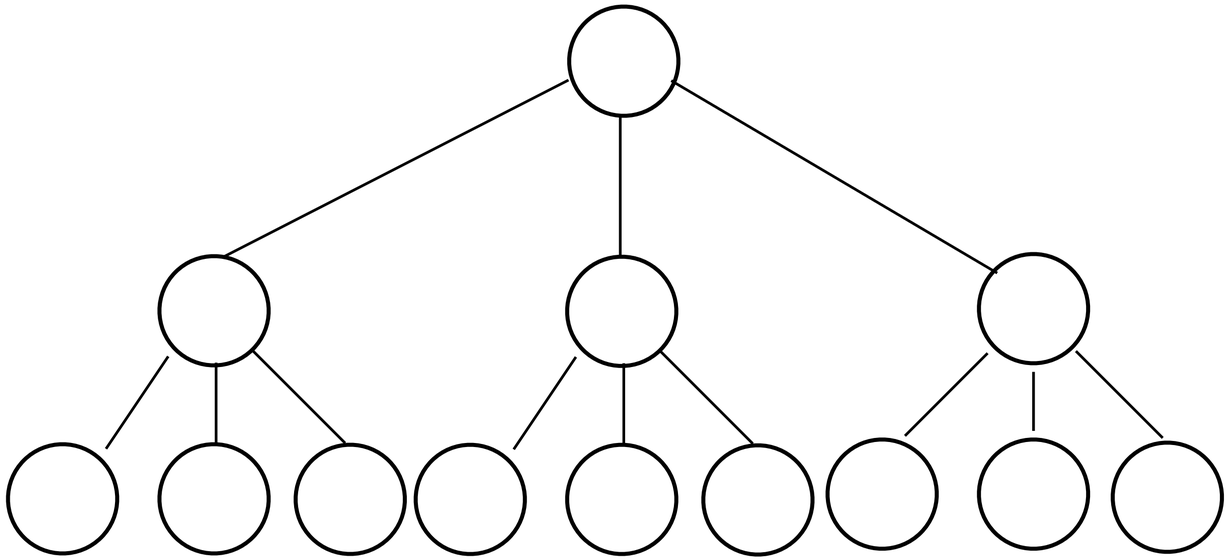


第一层：1个节点

第二层：3个节点

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



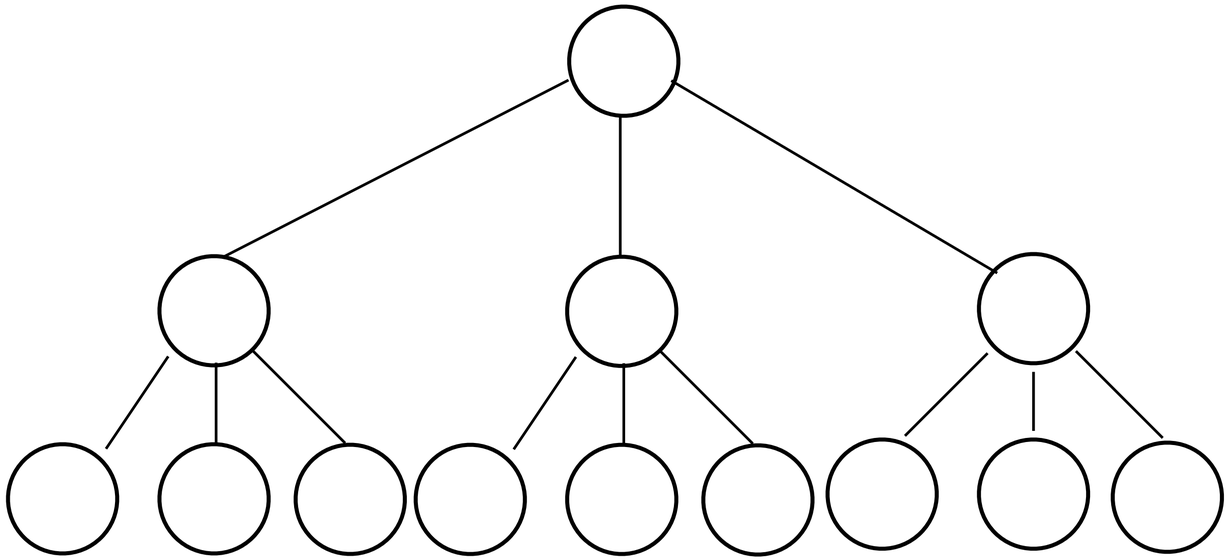
第一层：1个节点

第二层：3个节点

第三层：9个节点

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



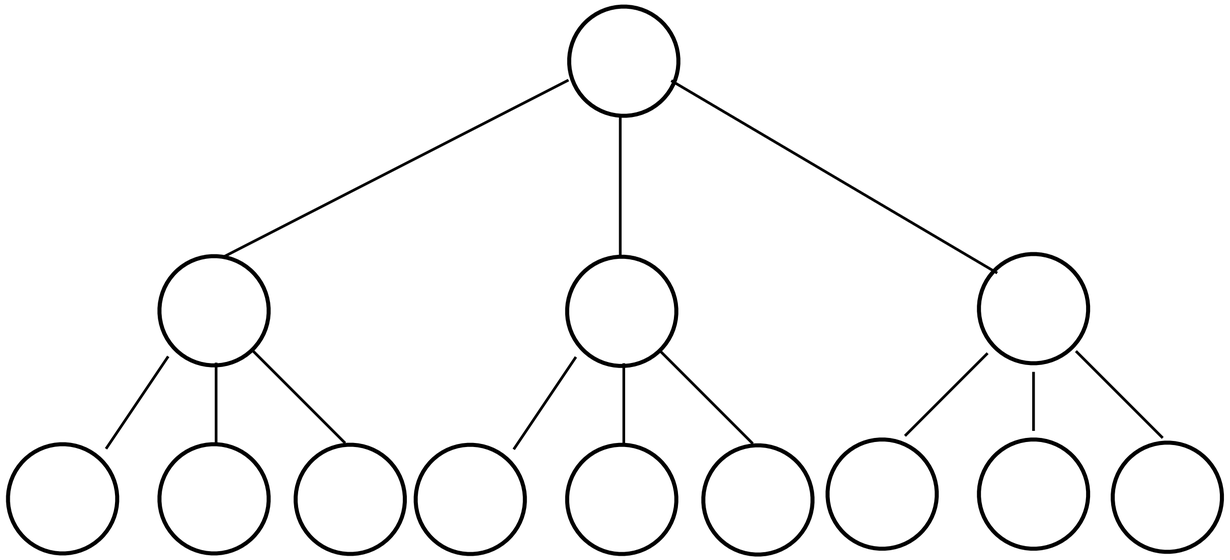
第一层：1个节点　3^0

第二层：3个节点　3^1

第三层：9个节点　3^2

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



第一层：1个节点　3^0

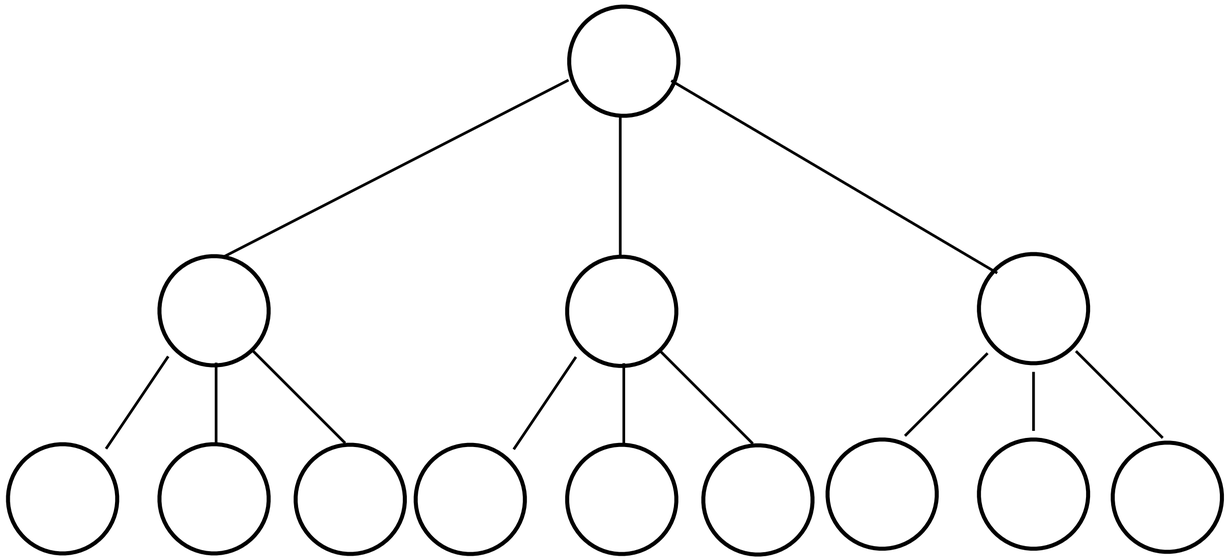
第二层：3个节点　3^1

第三层：9个节点　3^2

第h层：3^{h-1}个节点

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



第一层：1个节点　3^0

第二层：3个节点　3^1

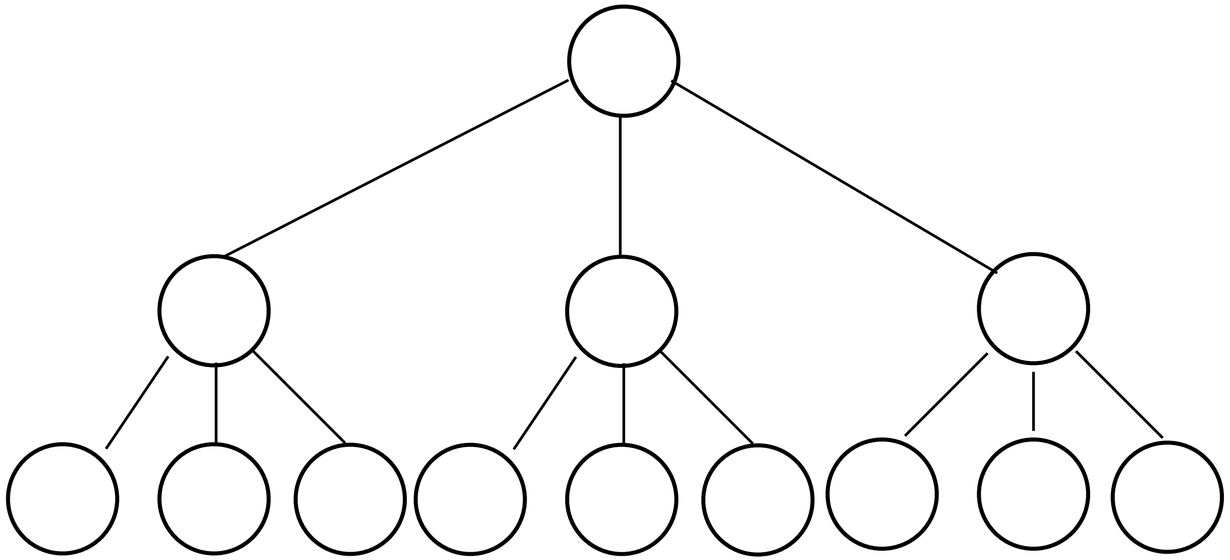
第三层：9个节点　3^2

k叉树第h层：k^{h-1}个节点

第h层：3^{h-1}个节点

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

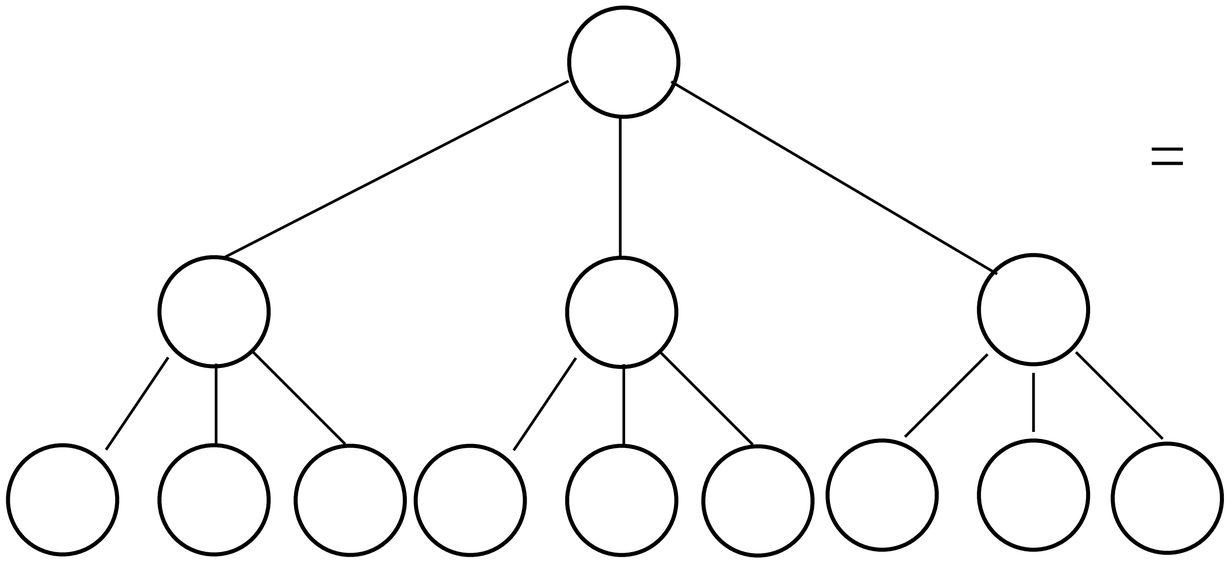
（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



$k^0 + k^1 + k^2 + \cdots + k^{h-1}$

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？

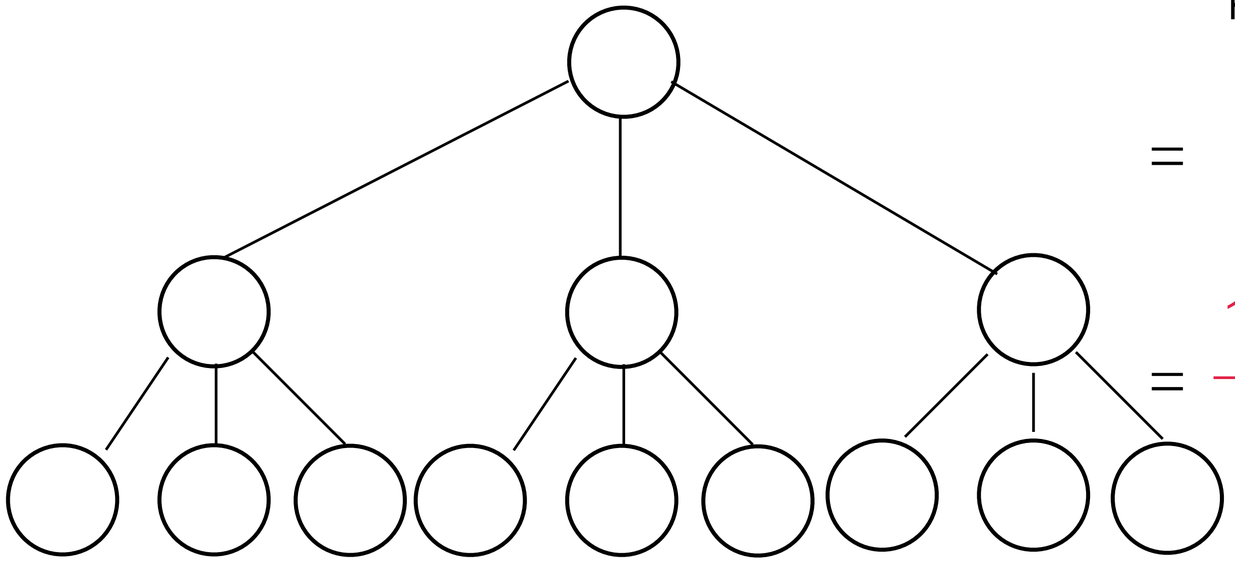


$$k^0 + k^1 + k^2 + \cdots + k^{h-1}$$

$$= k^0 \times \frac{1 - k^h}{1 - k}$$

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



$$k^0 + k^1 + k^2 + \cdots + k^{h-1}$$

$$= k^0 \times \frac{1 - k^h}{1 - k}$$

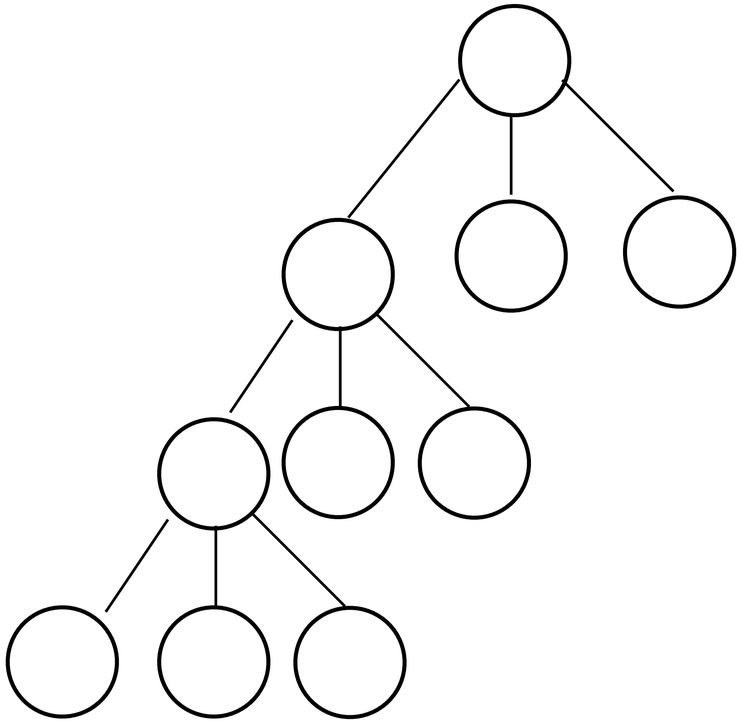
$$= \frac{1 - k^h}{1 - k}$$

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？

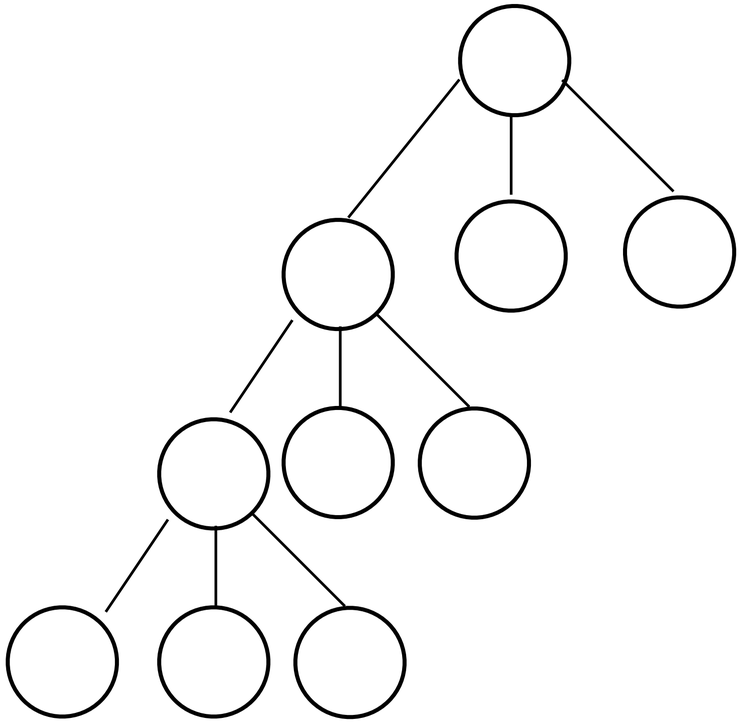
[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



1

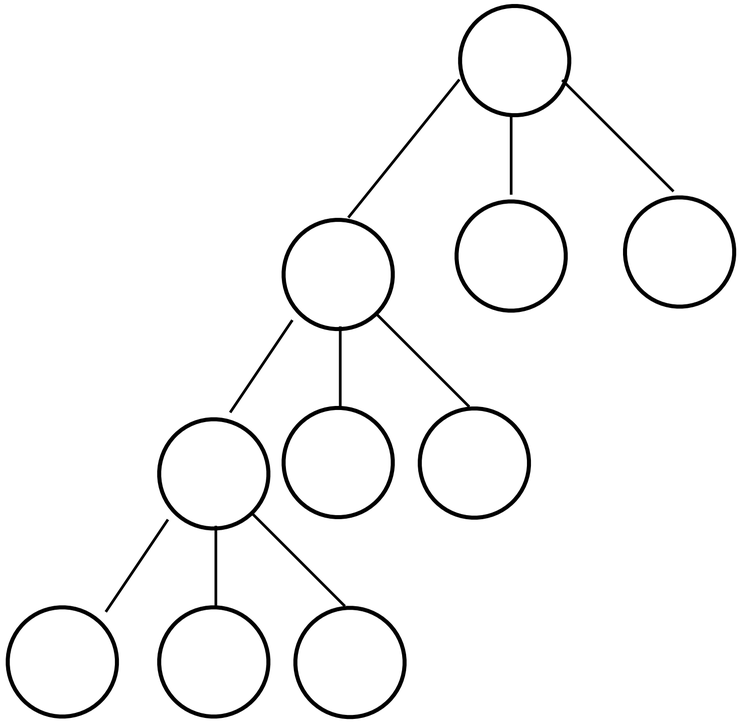
3

3

3

[2016] 42. 如果一棵非空 k（$k \geqslant 2$）叉树 T 中每个非叶结点都有 k 个孩子，则称 T 为正则 k 叉树。请回答下列问题，并给出推导过程。

（2）若 T 的高度为 h（单结点的树 h=1），则 T 的结点数最多为多少个？最少为多少个？



1

3

3

3

$1 + k(h-1)$

$= 1 + kh - k$